POCKET RIVALS

Understand every folder. Find the code to change.

Complete project and modification handbook

Architecture / every game / trajectories / AI / variables / file and function atlas

CURRENT COLLECTION	PRESERVED SOURCE	REFERENCE DEPTH
15 rivalry entries + 3 Arcade entries	4 future modules + legacy golf duel	Every maintained source file, named block, test and resource

Prepared for the project owner and friends

16 October 2026 | Source-grounded local edition

Includes searchable bookmarks, code locations, change guides and an editable companion source reference.

Contents

1. The application in one explanation 7

2. Folder map and ownership 9

3. Core, screens and shared utilities 11

4. Motion and trajectory: how the numbers become movement 14

5. Active rivalry games 17

5.1 Pong - folder games/pong . . . 17
5.2 Air Hockey - folder games/air-hockey . . . 17
5.3 Tic-Tac-Toe - folder games/tic-tac-toe . . . 18
5.4 Connect Four - folder games/connect-four . . . 18
5.5 Dots and Boxes - folder games/dots-and-boxes . . . 19
5.6 Hammer Hit - folder games/hammer-hit . . . 19
5.7 Light Fingers - folder games/light-fingers . . . 19
5.8 Fruit Duel - folder games/fruit-duel . . . 20
5.9 Math Quiz - folder games/math-quiz . . . 20
5.10 Lumberjack - folder games/lumberjack . . . 21
5.11 Water Guess - folder games/water-guess . . . 21
5.12 Perfect Timer - folder games/perfect-timer . . . 22
5.13 Gravity Runner - folder games/gravity-runner . . . 22
5.14 Happy Birds - folder games/happy-birds . . . 23
5.15 Spike Attacks - folder games/spike-attacks . . . 23

6. Arcade games and Ultimate Golf 24

6.1 Classic Snake - folder arcade/snake . . . 24
6.2 Star Patrol - folder arcade/space-shooter . . . 24
6.3 Ultimate Golf - active folder games/mini-golf/ultimate . . . 25

7. Preserved future games 28

7.1 Flappy Jump - future-games/flappy-jump . . . 28
7.2 Car Duel - future-games/car-duel . . . 28
7.3 Archery Duel - future-games/archery-duel . . . 28
7.4 Chicken Jump - future-games/chicken-jump . . . 29

8. Retained legacy golf and the example module 30

9. Build, offline support, Android and assets 31

10. Testing, troubleshooting and safe modification 33

11. How to use the complete file atlas 36

12. Visual maps of execution and motion 37

Free-flight examples and what to change . . . 37

Coordinate conversion for a golf drag . . . 38

13. Complete source file and code-block atlas 39

src . . . 39

src/core . . . 39

src/scenes . . . 57

src/shared/ai . . . 62

src/shared/components . . . 64

src/shared/contracts . . . 67

src/shared/effects . . . 68

src/shared/interfaces . . . 69

src/shared/physics . . . 69

src/shared/rules . . . 71

src/shared/utilities . . . 72

src/styles . . . 73

src/tournament . . . 84

src/games/air-hockey . . . 85

src/games/connect-four . . . 94

src/games/dots-and-boxes . . . 100

src/games/fruit-duel . . . 108

src/games/gravity-runner . . . 115

src/games/hammer-hit . . . 128

src/games/happy-birds . . . 138

src/games/light-fingers . . . 148

src/games/lumberjack . . . 153

src/games/math-quiz . . . 159

src/games/math-quiz/questions . . . 164

src/games/math-quiz . . . 169

src/games/mini-golf . . . 171

src/games/mini-golf/ultimate . . . 179

src/games/mini-golf/ultimate/ai . . . 196

src/games/mini-golf/ultimate/components . . . 203

src/games/mini-golf/ultimate/courses . . . 209

src/games/mini-golf/ultimate/modes . . . 216

src/games/perfect-timer . . . 220

src/games/pong . . . 230

src/games/spike-attacks . . . 235

src/games/tic-tac-toe/classic . . . 249

src/games/tic-tac-toe . . . 253
src/games/tic-tac-toe/shared . . . 254
src/games/tic-tac-toe/ultimate . . . 254
src/games/water-guess . . . 262
src/arcade/snake . . . 272
src/arcade/space-shooter . . . 276
src/future-games/archery-duel . . . 286
src/future-games/car-duel . . . 290
src/future-games/chicken-jump . . . 297
src/future-games/flappy-jump . . . 301
examples/tap-duel . . . 305
scripts . . . 306
android/app/src/androidTest/java/com/getcapacitor/myapp . . . 311
android/app/src/main/java/com/pocketrivals/games . . . 312
android/app/src/test/java/com/getcapacitor/myapp . . . 312
. . . 312

14. Tests and fixtures: every file and declared check 314

tests/browser . . . 314
tests/fixtures . . . 319
tests/future . . . 319
tests/unit . . . 320

15. Every folder, resource, data file and document 330

. . . 330
android . . . 331
android/app . . . 333
android/app/src/main . . . 334
android/app/src/main/assets . . . 335
android/app/src/main/res/drawable . . . 335
android/app/src/main/res/drawable-land-hdpi . . . 335
android/app/src/main/res/drawable-land-mdpi . . . 335
android/app/src/main/res/drawable-land-xhdpi . . . 335
android/app/src/main/res/drawable-land-xxhdpi . . . 335
android/app/src/main/res/drawable-land-xxxhdpi . . . 336
android/app/src/main/res/drawable-port-hdpi . . . 336
android/app/src/main/res/drawable-port-mdpi . . . 336
android/app/src/main/res/drawable-port-xhdpi . . . 336
android/app/src/main/res/drawable-port-xxhdpi . . . 336
android/app/src/main/res/drawable-port-xxxhdpi . . . 336
android/app/src/main/res/drawable-v24 . . . 336
android/app/src/main/res/layout . . . 336

android/app/src/main/res/mipmap-anydpi-v24 . . . 336
android/app/src/main/res/mipmap-anydpi-v26 . . . 336
android/app/src/main/res/mipmap-hdpi . . . 337
android/app/src/main/res/mipmap-mdpi . . . 337
android/app/src/main/res/mipmap-xhdpi . . . 337
android/app/src/main/res/mipmap-xxhdpi . . . 337
android/app/src/main/res/mipmap-xxxhdpi . . . 337
android/app/src/main/res/values . . . 338
android/app/src/main/res/xml . . . 338
android/capacitor-cordova-android-plugins . . . 338
android/capacitor-cordova-android-plugins/src/main . . . 339
android/capacitor-cordova-android-plugins/src/main/java . . . 339
android/capacitor-cordova-android-plugins/src/main/res . . . 339
android/gradle/wrapper . . . 339
docs . . . 340
docs/licenses . . . 347
docs/specifications . . . 347
examples/tap-duel/assets . . . 347
public . . . 347
src/arcade/snake . . . 347
src/arcade/snake/assets . . . 347
src/arcade/space-shooter . . . 348
src/arcade/space-shooter/assets . . . 348
src/future-games . . . 348
src/future-games/archery-duel . . . 348
src/future-games/archery-duel/assets . . . 348
src/future-games/car-duel . . . 348
src/future-games/car-duel/assets . . . 348
src/future-games/chicken-jump . . . 348
src/future-games/chicken-jump/assets . . . 348
src/future-games/flappy-jump . . . 349
src/future-games/flappy-jump/assets . . . 349
src/games/air-hockey . . . 349
src/games/air-hockey/assets . . . 349
src/games/connect-four . . . 349
src/games/connect-four/assets . . . 349
src/games/dots-and-boxes . . . 349
src/games/dots-and-boxes/assets . . . 349
src/games/fruit-duel . . . 350
src/games/fruit-duel/assets . . . 350
src/games/gravity-runner . . . 350
src/games/gravity-runner/assets . . . 350

src/games/hammer-hit . . . 350
src/games/hammer-hit/assets . . . 350
src/games/happy-birds . . . 350
src/games/happy-birds/assets . . . 350
src/games/light-fingers . . . 351
src/games/light-fingers/assets . . . 351
src/games/lumberjack . . . 351
src/games/lumberjack/assets . . . 351
src/games/math-quiz . . . 351
src/games/math-quiz/assets . . . 351
src/games/math-quiz/questions . . . 351
src/games/mini-golf . . . 352
src/games/mini-golf/assets . . . 352
src/games/mini-golf/ultimate . . . 352
src/games/perfect-timer . . . 352
src/games/perfect-timer/assets . . . 352
src/games/pong . . . 353
src/games/pong/assets . . . 353
src/games/spike-attacks . . . 353
src/games/spike-attacks/assets . . . 353
src/games/tic-tac-toe . . . 353
src/games/tic-tac-toe/assets . . . 353
src/games/water-guess . . . 353
src/games/water-guess/assets . . . 353
Generated and local directories . . . 354

16. Snapshot, coverage and validation ledger 355

Rebuilding the documentation . . . 355

Prepared from the local source on 16 October 2026 (Asia/Dhaka).

This handbook explains the folder structure, each maintained code file, the blocks inside those files, all game families, motion and trajectory calculations, AI, persistence, Android packaging, and the practical places to edit behavior. It is intended for the project owner and friends who want to understand or modify the application.

Read this first

All paths in this document are relative to the pocket-rivals project folder. The absolute project location is /Users/nafis/Documents/ChatGPT/Exam Grading/pocket-rivals. The surrounding Exam Grading/tmp directory contains unrelated grading scans and is outside this game handbook. The surrounding .git directory is repository metadata.

Read Chapters 1-5 to understand the application. Then open the relevant game chapter. Use the file atlas to find an exact function, line range, state field, decision condition, dependency, and return expression. Use the maintenance chapters when testing or preparing Android assets. The accompanying Markdown, code index and source reference preserve searchable, editable details.

Line numbers refer to this snapshot and will move after edits. Search for the named symbol when a number becomes stale. The code index includes SHA-256 fingerprints so a future reader can check whether a file still matches this edition. This is an explanation of the implementation, not a claim that every device, seed or strategy has been tested.

What completeness means here

Every maintained source/configuration file and test is inventoried. Every TypeScript function, class, interface/type, method and getter found by the source parser is mapped. Anonymous callbacks are explained within their owning block; worker handlers and other top-level execution have separate entries. Class fields and module constants identify stored state and initial values. CSS has a selector/block index. Static assets, data, documents and native resources have a separate file inventory. A companion source reference reproduces the complete maintained textual sources, so small local literals and nested expressions remain available even when the PDF summarizes a large block.

Installed dependency internals, Git objects, generated bundles and caches are explained by their purpose and regeneration path. They are not hand-authored project logic. The PDF does not reproduce third-party dependency source or binary contents. No application behavior was changed to create this handbook.

1. The application in one explanation

Pocket Rivals is a static offline-capable game collection. TypeScript is checked and bundled by Vite. Phaser draws gameplay, while HTML/CSS draws menus and some game controls. Capacitor packages the same web build into an Android application. There is no backend or remote AI service in this project.

There are 18 registered cards: 15 rivalry entries and 3 Arcade entries, including Ultimate Golf. Tic-Tac-Toe contains Classic and Ultimate implementations behind one card. Four additional modules are preserved in future-games and do not currently register. The original five-course golf duel also remains in source, but the active mini-golf factory launches Ultimate Golf.

The launch path

1. index.html provides #app, #menu, #game-stage and #overlays.

2. src/main.ts loads local fonts and CSS, waits for fonts, then constructs GameManager.

3. GameRegistry eagerly discovers small config.ts files from games/* and arcade/*.

4. MainMenu renders their metadata. GameSetup collects mode, difficulty and variant.

5. GameManager.play starts the reusable GameHost scene with a fresh run number.

6. GameLoader imports only the selected index.ts factory. That factory loads its implementation.

7. GameHost constructs GameContext and calls module.initialize, then module.update on the existing Phaser loop.

8. A module reports ctx.finish for a duel or ctx.finishArcade for an Arcade score. Core records the correct kind of result.

9. Pause clears controls and suspends module work. Exit destroys module-owned resources and returns to the menu.

The run number and host generation prevent delayed imports or asynchronous AI replies from affecting a later match. Do not remove those guards when simplifying navigation.

Three different kinds of state

Kind	Examples	Correct edit location
Rules and simulation	Score, legal moves, ball velocity, current phase	Game-local Rules, Physics or world class
Presentation	Colors, animation pose, camera interpolation, text spacing	Art, View, Renderer, UI and CSS
Application coordination	Current screen, tournament attempt, saved records	core and scenes

Changing a drawing coordinate does not necessarily change collision. Changing AI preferences does not change a legal speed limit. Changing a menu description does not change the win condition. Follow the responsibility boundary to reach the actual behavior.

A short explanation for friends

"The app has one host engine. Each game is a small independent module. Its config describes the menu card, its factory loads the game, its rules decide what is legal, its physics moves objects, its AI chooses ordinary inputs, and its renderer draws the resulting state. Shared core handles navigation, pause, sound and saved results."

2. Folder map and ownership

Folder/file	What it contains	When you edit it
src/main.ts	Application entry, local fonts, development snapshot, web service-worker registration	Startup behavior or global stylesheet/font imports
src/core/	Registry, loader, navigation, input, audio, storage, scores, tournaments, diagnostics	Behavior shared across the whole app
src/scenes/	Reusable Phaser host plus DOM menu/setup/results/settings controllers	Application screens and host HUD
src/shared/contracts/	Public module, result and AI types	A capability shared by independent modules
src/shared/interfaces/	Compatibility re-export of the module contract	Usually no separate logic to change
src/shared/physics/	Bounded fixed-step accumulator and smooth legal motion controller	Shared timing/movement helpers
src/shared/rules/	First-input arbitration and Pong/Hockey match clock	Shared timing/result policy
src/shared/ai/	Seeded personality/replay utilities	Common deterministic identity, not game tactics
src/shared/components/	Modal, hot-seat rotation, drawing and icons	Reusable UI mechanics
src/shared/effects/	Bounded impact pulse pool	Shared visual feedback
src/styles/	Main app CSS, responsive menus and transitions	Global visual design
src/tournament/	Tournament setup/progress/results UI	Tournament selection or progress presentation
src/games/	Rivalry modules plus the Ultimate Golf entry	A specific game's rules, art, physics or AI
src/arcade/	Snake and Star Patrol solo implementations	Solo score games
src/future-games/	Four unregistered preserved modules	Future development or reactivation
examples/tap-duel/	Small complete module template, not shipped	Learn how to add a game
public/	Static icon copied directly into the build	Public web icon
scripts/	Build/offline plugin, discovery and bundle checks, performance/math audit utilities	Development infrastructure
tests/unit/	Rules, physics, AI and persistence tests, including deferred games	Behavioral checks
tests/browser/	Active Playwright UI, touch, keyboard, worker and offline regressions	Browser behavior
tests/future/	Deferred browser specs excluded by active testDir	Reactivate with their game

Folder/file	What it contains	When you edit it
tests/fixtures/	Legal golf shot routes, optional route search and expensive AI audit	Reproduce course completion or investigate AI
docs/	Architecture, implementation history, QA, reports, screenshots and license copies	Existing explanations and evidence
docs/handbook/	This guide, machine index and reproducible document generator	Update this handbook after code changes
android/	Native Capacitor wrapper, Gradle configuration, resources and packaged web assets	Native identity, platform packaging and release build
dist/	Generated Vite output, manifest, chunks, fonts, service worker	Rebuild rather than hand-edit
node_modules/	Installed third-party packages	Recreate with npm ci; edit package.json for dependencies
test-results/	Generated Playwright state/artifacts	Inspect failures; regenerate through tests
tmp/	Local intermediate files, including this document's render checks	Temporary work; not runtime
output/pdf/	Final generated handbook	Share the PDF from here

Configuration at the root

package.json defines dependencies and runnable commands. package-lock.json records exact resolved versions for npm ci; the caret versions in package.json alone are not an exact installation record. tsconfig.json enables strict TypeScript, browser DOM types and Vite-compatible module resolution. vite.config.ts sets relative URLs, ES2022 output, a build manifest, the offline plugin, and a separate Phaser chunk. vitest.config.ts configures unit testing. playwright.config.ts points to tests/browser, uses Pixel 7 emulation and installed Chrome, runs one worker, and starts dev and preview servers. vercel.json describes static hosting behavior; it does not itself deploy anything. capacitor.config.ts defines app ID/name, dist as webDir, Android background and splash behavior. index.html contains the application roots. .prettierrc.json controls formatting, while .gitignore excludes generated/local material. AGENTS.md files describe maintenance constraints, not runtime code.

The repeated structure inside a module

config.ts owns metadata: stable ID, displayed name, instructions, ordering, colors, thumbnail, modes, variants and tournament eligibility. index.ts owns the lazy factory. assets/preview.svg is a menu image, not the live renderer. AGENTS.md states module-specific constraints. A Game controller owns lifecycle, inputs, display objects, updates and result reporting. Rules owns legal actions, score progression and terminal state. Physics/world owns movement and collision. AI owns observations and action selection. Art/View/Renderer owns drawing or interpolation. Some compact modules combine several responsibilities in one controller; use their block map in the atlas.

3. Core, screens and shared utilities

Navigation and lifecycle

core/GameManager.ts is the application coordinator. Its constructor creates storage/audio/score/tournament services, the one Phaser.Game, screen controllers and event cleanup hooks. nativeLifecycle installs Capacitor foreground/back-button behavior. selectGame opens setup. play selects metadata, validates AI support, clears an old quick-match record when appropriate, creates a fresh run, exposes the opaque loading cover, resizes the canvas and starts GameHost. resizeGame uses 420x760 for ordinary modules and the available stage rectangle for responsive modules.

onReady reveals gameplay only for the current run. checkpoint routes saved state to the current tournament attempt or the quick-match key. savedMatch validates basic stored options and rejects Arcade resume. finish accepts a current duel once; finishArcade records a separate high score. pause opens the shared modal after pausing host input/module work. restart replaces the current attempt. background pauses and suspends audio. goMenu increments the run, stops the host, sleeps the Phaser loop and rebuilds the DOM menu. dispose removes application listeners and destroys shared resources.

scenes/GameHost.ts is the bridge between the selected module and core. init records options and a new generation. create sets the camera, draws the ordinary HUD unless presentation is custom, creates InputManager and lazily initializes the module. The supplied context contains scene, input, audio, options, restoredState, saveState, HUD functions and terminal result functions. update derives foreground dt from performance.now. pauseGame pauses controls first; resumeGame resets the time reference. cleanup disposes the module and its input manager. HUD score animations belong here; game score arithmetic does not.

Metadata, screens and result routes

core/GameRegistry.ts contains the two discovery globs, stable-ID validation, order sorting, lookup and tournament eligibility filtering. core/GameLoader.ts maps an ID to the matching lazy factory and optional tutorial. Future-game paths are intentionally absent from both globs.

scenes/MainMenu.ts renders the duel/Arcade sections from metadata and binds cards/settings/tournament actions. scenes/GameSetup.ts renders local/AI selection, difficulty, variants and tutorials. scenes/Results.ts formats the duel result markup. scenes/Settings.ts controls sound, volume, explicit offline download and score-reset confirmation. tournament/TournamentScene.ts builds eligible pools and displays progress/champion/rematch controls. shared/components/Modal.ts preserves a live modal panel during content updates, handles back/close and keyboard focus trapping.

Input ownership

core/InputManager.ts uses scene-local subscriptions. dragHalves keeps a pointer assigned to one player until release/cancel; crossing the middle must not steal ownership. tap maps pointer-up actions. drag exposes a pointer-owned start/move/end/cancel stream and optional coordinate bounds. press filters keyboard repeats for one-shot actions. keys owns held-key sets. clear resets pointers, held controls and custom clear handlers. pause disables actions and clears state; resume enables clean controls; dispose unsubscribes everything it owns.

For a new control, prefer ctx.input APIs. If a module needs a separate listener, retain and remove it explicitly. A constructor should not install active listeners; initialize is the correct place. Do not add a second requestAnimationFrame or Phaser.Game.

Saves and scoring

Storage key	Owner and content	Resume behavior
pocket-rivals.v1	StorageManager: sound, legacy music field, volume, duel totals	Persists preferences and overall duel results
pocket-rivals.match.v1	GameManager: current quick-match specification/checkpoint	Only modules that validate/restore their state resume
pocket-rivals.tournament.v1	TournamentManager: schedule, results, attempt token, checkpoint	Resumes valid tournament progress
pocket-rivals.arcade.v1	ArcadeScoreManager: per-game high scores	A new Arcade run starts after closure
pocket-rivals.ultimate-golf.v1	GolfProgress: four casual best scores	Current course restarts after closure
pocket-rivals.golf-versus.v1	CompetitiveStats: golf competition counters	Separate from casual course records
pocket-rivals.math-history.v1	QuestionHistory: up to 150 recently used IDs	Avoids recently seen questions across sessions

StorageManager validates primitive settings, clamps volume to 0..1, and keeps record writes in an in-memory map if localStorage is unavailable. ScoreManager begins a fresh round and records a duel winner once. ArcadeScoreManager validates finite scores and retains a maximum by game ID.

TournamentManager stores phases between, playing and complete. shuffledSchedule makes shuffled bags from the eligible pool and supports 1..25 rounds. startRound returns the current spec, checkpoint and token. A token includes tournament ID, results index and attempt. restartRound increments the attempt and discards its checkpoint. finish verifies phase/token, stores one result, adds one point for a win or half per player for a draw, and moves to the next phase. restore rebuilds points from results rather than trusting stored totals. GameManager separately rejects an unavailable or Arcade game at launch. To change tournament points, edit addPoints, then update restore/result expectations consistently.

Timing, AI and feedback helpers

FixedStep.advance clamps one render delta to 0..0.1 seconds, performs at most twelve 1/120-second ticks, and exposes a remainder/fraction for rendering. reset clears pending catch-up on lifecycle boundaries. Do not use this capped time for a strict real-time match deadline. GameHost's foreground elapsed time and MatchClock serve that purpose.

MatchClock starts with 120 seconds and a five-point target. A lead at regulation expiry wins; 0-0 draws; a nonzero tie starts 60 seconds of sudden death. The first subsequent lead wins; an unresolved overtime draws. Pong/Hockey retain this clock through goals. Changing only PongWorld.goal's target leaves MatchClock inconsistent; edit both.

MotionController smooths a requested target using 1-exp(-18*dt), limits desired speed by distance and braking distance, limits acceleration, and applies a subpixel deadband. It returns an ordinary target position; each game still enforces its own legal field and speed.

FirstResponse stores at most one entry per player, waits at least 9ms before resolution, and treats entries within 8ms as tied. Those are model-time arbitration tolerances, not a guarantee about physical touchscreen latency.

MatchRandom owns seeded pseudo-random decisions and a 24-profile identity: role, tempo and side. freshSeed can read development aiSeed, otherwise uses crypto when available. sample creates bounded jitter; rank breaks comparable choices deterministically; turnSeed gives repeatable turn seeds. replayMode enables deterministic candidate caps for development audits. Game-specific tactics stay in each AI file.

WorkerTask permits one outstanding computation, terminates it when replaced/canceled, reports measured time, and applies a 2-second watchdog. Controllers add generation/turn guards before accepting a result. Cancel workers on pause and disposal.

AudioManager synthesizes short Web Audio effects from the effects map. Each entry controls frequencies, waveform, duration and gain. play applies a per-kind rate limit, a maximum of ten active oscillators, shared mute/volume and a short envelope. unlock is called after interaction; optional audio failures do not block play. There is no soundtrack despite the compatibility music field.

TurnView rotates module-owned content for local Blue turns and supplies an exact inverse point transform. Its 280ms transition blocks actions until ready. Host navigation stays fixed. Impacts uses a 14-pulse pool. drawing.ts provides outlined primitives/text; icons.ts supplies small SVG UI icons. ThemeManager is the central palette and default 420x760/FIELD geometry. Diagnostics provides development counters; main.ts exposes the read-only __POCKET__ snapshot only in development.

4. Motion and trajectory: how the numbers become movement

Units and coordinate spaces

Most games render in a 420x760 logical canvas with x increasing right and y increasing down. FIELD is left 24, right 396, top 112, bottom 650 and middle 381. A velocity is logical units per second; acceleration is units per second squared; dt is seconds. Increasing positive vertical velocity moves downward.

Some games use a smaller internal arena and map it to the canvas: Happy Birds 320x460, Spike Attacks 320x320, Gravity Runner cross-track y 0..170. Ultimate Golf keeps original 1920x1080 source coordinates. Never copy a display pixel value into its physics without applying the inverse transform.

Integration and collision

A common fixed-step update is velocity += acceleration*dt, then position += velocity*dt. This is semi-implicit Euler. It approximates a continuous path with small legal steps. An analytic trajectory such as y(t)=y0+v0*t+0.5*g*t*t is useful for an explanation or initial AI estimate, but it does not include the engine's collisions, moving hazards, discrete releases or state changes.

A circle collision uses a unit contact normal n and relative velocity u=v-objectVelocity. The dot product u.n tells whether the bodies approach. Reflect only an approaching velocity, then project the overlapping body outside the obstacle with a small clearance. This avoids repeated reversals and sticking. Swept tests examine the segment between previous/current positions so fast motion cannot jump over a thin target.

Presentation interpolation computes a display position between captured physics states using FixedStep.fraction. It smooths frames without changing the authoritative state. Changing View smoothing changes appearance; changing Physics changes outcomes and AI forecasts.

Trajectory owner map

Movement or trajectory	Authoritative file/block	Related prediction/presentation
Pong flight, paddle bounce, serve	games/pong/PongPhysics.ts: PongWorld.step/goal	PongAI.reflectedX/update; PongGame trail
Hockey puck, mallets, posts, rounded rails	games/air-hockey/AirHockeyPhysics.ts and rink.ts	AirHockeyAI, MotionController, PossessionClock
Active golf ball, surfaces, attraction and hazards	games/mini-golf/ultimate/GolfPhysics.ts: tick	GolfDynamics, GolfCollision, ShotPlanner, GolfRenderer
Active golf pullback/camera	ultimate/GolfInput.ts: shotVector/project/unproject	UltimateGolfGame gesture ownership
Legacy golf ball	games/mini-golf/GolfPhysics.ts: golfStep	Legacy GolfAI/ai.worker; inactive factory path
Gravity flip and trap contact	gravity-runner/RunnerPhysics.ts: advanceRunner	ObstacleGeometry, GravityAI forecasts, GravityView
Bird flight and triangle spikes	happy-birds/ArenaPhysics.ts: advanceBird	Rules.speed, BirdsAI forecasts, BirdView
Bird recovery curve	happy-birds/Recovery.ts: advanceRecovery	recoveryPose/recoveryColor, BirdArt
Star collection sweep	happy-birds/Stars.ts: pickupDistance	Rules.step chooses nearest eligible bird
Spike piston motion/player routing	spike-attacks/Rules.ts: tick and ArenaPhysics.ts	SpikeAI, SpikeView and SpikeArt
Hammer puck and bell oscillation	hammer-hit/StrikePhysics.ts	HammerGame presentation; Rules owns score
Fruit sword swing and return	fruit-duel/SwordMotion.ts: swordPose	FruitGame uses Rules' swing start/recovery duration
Snake grid motion	arcade/snake/SnakeRules.ts: tick	SnakeGame interpolation
Shooter projectiles/formation	arcade/space-shooter/ShooterWorld.ts: step	SpaceShooterGame geometric rendering
Basketball jump/release/hoop crossing	future-games/flappy-jump/Rules.ts: step	projectedLanding and FlappyAI
Car body and rotation	future-games/car-duel/Physics.ts	CarAI, shared point/head/colliders geometry
Chicken/block landing	future-games/chicken-jump/Rules.ts: step	ChickenAI alignment offset
Archery moving crosshair	future-games/archery-duel/Rules.ts: step	ArcheryAI locks actual crosshair

Board tokens, gem reaches, tube fill and stopwatch graphics have animation paths, but their scores are determined by discrete rules or timing rather than a projectile simulation. Their controller/render blocks are mapped in the atlas.

Example: changing a jump or flap

For Happy Birds, GRAVITY is 650 and FLAP is -195. In free flight, the analytic rise after a flap is approximately FLAP^2/(2*GRAVITY)=29.25 units and time to apex is 0.30s. Actual gameplay uses 120Hz integration and wall/ceiling/floor contacts. A more negative FLAP increases the rise; increasing GRAVITY makes descent faster and reduces rise at the same flap velocity. Update the common constants so AI forecasts and human motion continue to share physics. Recheck arena reachability, star access and collision tests.

For Hammer Hit, StrikePhysics launches from y=602 with acceleration 2200. Initial upward velocity is -sqrt(2*2200*height). A score of 100 uses height 365 and reaches the bell; other scores use height=(score/100)*348 and peak below bell contact y=254. This is a visual consequence of a legal score. Editing it must not award more points.

For Gravity Runner, a flip reverses the gravity sign; it does not replace vy with a new jump impulse. The runner therefore takes time to brake and reverse. Gravity is 900 and legal flips are separated by 0.16s. That momentum explains why a late flip can fail even when a visual opening looks reachable.

Presentation paths and animation variables

These paths are visual feedback unless their row explicitly shares a legal cooldown. They are useful when a friend asks to make an action feel faster without changing its score.

Effect	Exact expression / owner	What to change
Fruit split halves	FruitGame.update: x=210+side*t*125; y=381-45*t+150*t*t; angle=side*t*1.5, after the0.15s impact beat	125 changes sideways spread;45 initial rise;150 downward curvature;1.5 rotation
Fruit droplets	x=210+cos(a)*t*130; y=381+sin(a)*t*90+100*t*t, eight droplets fading over0.6s	130/90 spread axes;100 falling curvature; keep the count/lifetime bounded
Sword return	SwordMotion.swordPose: rest->wind over0.065s, impact at0.15s, follow-through until0.26s, return by the shared recovery duration	Pose arrays change x/y/angle; durations must match Rules input unlock
Gem grab	LightGame.update: hand travel=sin(clampedAge*pi)*140; winning gem uses smoothstep from age0.32 over0.5s toward x+/-45,y+/-200	Change amplitude or eased destination without changing signalAt/arbitration
Lumber chop	LumberGame.update: scroll*=exp(-26*dt); swing=max(0,swing-7*dt), scroll capped28	Decay26 changes settling;7 changes swing duration; side switches remain immediate
Gravity facing/landing	GravityView.sample: angle approaches target by1-exp(-22*dt); landing*=exp(-16*dt)	Changes visual rotation/impact, not gravity sign or vy
Bird facing/body/trail	BirdView.sample: facing blend26, body angle blend14 from clamped vy/400; bump decay18; six trail positions sampled every0.045s	Tune responsiveness/trail appearance while preserving collision radius
Spike facing	SpikeView.sample: shortest angular delta blended by1-exp(-18*dt); position and shaft length use fixed-step alpha	Changes drawing smoothness; legal movement and piston contact stay in Rules/Physics
Classic symbol entrance	ClassicGame.update: t=min(1,age/0.2); scale=1+2.7*(t-1)^3+1.7*(t-1)^2	0.2 is duration; coefficients produce overshoot; board legality is immediate
Connect Four token drop	ConnectFourGame.update: t=min(1,age/0.3); y=182+(targetY-182)*(1-(1-t)^2)	0.3 changes fall time; keep the input lock synchronized
Water fill/reveal	WaterGuessGame.update: fillProgress+=1.3*dt to1; reveal advances after3.2s	1.3 changes fill speed;3.2 changes feedback wait; percentages remain integers
Timer reveal/clock tick	PerfectTimerGame.update: reveal waits3.2s; tick=floor(foregroundMs/500)	3.2 changes result pacing;500 tick cadence; human/AI stop time stays TimerClock

5. Active rivalry games

5.1 Pong - folder games/pong

Pong keeps the stable registration ID ping-pong. PongGame.ts exports the PingPong controller. It initializes graphics, inputs, AI and optional restored rally; update obtains legal targets, runs fixed physics, advances the separate MatchClock, draws paddles/ball/trail and checkpoints about once per second. paddleTarget interprets keyboard or pointer ownership. goal triggers feedback but does not reset the match deadline. pause saves and clears timing; dispose releases graphics/effects. PongState.ts serializes and validates position, velocity, scores and clock rather than trusting arbitrary local data.

PongPhysics.ts is the motion authority. The ball starts at (210,381), radius 9, velocity (110,340). Paddles start at y 608 and 154, half-width 44, and move at at most 1800 units/s. step advances the ball, reflects side walls, detects a swept crossing of each paddle plane, and awards a point after the ball passes the top/bottom. On a paddle contact, hit=(ball.x-paddle.x)/44 is clamped to -1..1. New speed=min(820,oldSpeed*1.055+8). Horizontal velocity includes sin(hit*1.03)*speed plus paddle.vx*0.13 and is capped at 0.88*speed; vertical velocity preserves total speed. goal uses 1.1s serve cooldown and alternates horizontal serve direction.

PongAI.ts predicts the intercept with reflectedX, a triangular-wave fold that accounts for repeated side-wall bounces. Medium uses delay 0.24s, error 56 and speed 420; Hard 0.075/14/820; Impossible 0.018/0/1750 before personality adjustments. A once-per-flight offensive shot offset seeks an angle away from the observed rival, but is abandoned when interception needs the full reach. MotionController smooths targets and the world still enforces legal speed/position.

Desired change	Exact owner	Consequence/check
Ball faster after each hit	PongWorld.step speed expression	Recheck swept hits and maximum 820 cap
Wider paddle	Half-width 44 in physics, bounds, drawing and AI offsets	Keep hit normalization and visual width consistent
Win at a different score	PongWorld.goal and shared MatchClock.score	Both world and deadline result rules must agree
Longer match	shared/rules/MatchClock.ts	Also affects Hockey
Easier AI	PongAI.update settings	Keep world limits; do not teleport paddles

5.2 Air Hockey - folder games/air-hockey

AirHockeyGame.ts coordinates world, two targets, keyboard/touch input, MatchClock, PossessionClock, save checkpoints and rendering. AirHockeyPhysics.ts exports HockeyWorld. The puck radius is 11; mallets radius 29. Targets are clamped to each half and capped at 1500 units/s. Each 120Hz step contains three collision substeps with mallets interpolated across them.

step applies exp(-0.1*substep) damping, moves the puck, resolves mallet/rail/post contacts in bounded passes, repairs pinches and caps speed at 1100. malletContact uses relative velocity and coefficient 1.9 on the approaching normal. posts uses 1.96. These correspond to near-elastic responses while preserving tangent motion. rink.ts owns CORNER=44, GOAL_HALF=63, rounded corners, goal-mouth clearance and reachablePuck. Moving a drawn goal alone will not move the scoring region.

solvePinch searches 64 nearby legal contact positions. checkUnplayable waits 0.75s for unreachable/invalid geometry before local repair. A reachable stationary puck is legal and must not be reset for being slow. awardPoint resets the puck/mallets and 1.1s cooldown. HockeyState.ts validates the combined physics and clock checkpoint.

PossessionClock counts continuous residence on a half with center +/-3 hysteresis and a 15s violation. Contacts do not reset it. The controller awards the rival a point for a violation. AirHockeyAI.ts predicts incoming reflected flight or routes behind the puck to strike downfield. Medium/Hard/Impossible delay-error-speed values are 0.19-24-530, 0.065-7-1050 and 0.02-0-1450, with personality variation and legal MotionController output.

To change table geometry, edit ThemeManager/rink plus drawing, restore validation and AI prediction bounds together. To alter friction, edit the exponential damping in HockeyWorld.step. To tune goals, edit inMouth and post geometry together. To alter the possession rule, edit PossessionClock rather than mallet contact. Relevant checks are engine, phase2, phase3 and air-hockey-keyboard.

5.3 Tic-Tac-Toe - folder games/tic-tac-toe

index.ts lazily selects classic/ClassicGame.ts or ultimate/UltimateGame.ts from one config entry. shared/lines.ts defines eight 3x3 winning triples. ClassicRules.ts owns nine cells, current player, winner, winning line and legal history. play rejects illegal/finished moves, updates the line/draw and changes turn. restore replays a history through play. ClassicGame.ts maps input, animates symbols and turn backgrounds, schedules ordinary AI decisions and reports one result.

ClassicAI.ts implements alpha-beta minimax through classicMove. ORDER prefers center, corners, then edges as a search ordering, with seeded comparable-choice variety. ClassicAI.choose wraps mode-dependent choice and personality. Classic Impossible is the one mode whose unbeatable legal play is exhaustively covered by tests. Adjust difficulty behavior here; altering the shared WIN_LINES changes the game rules.

UltimateRules.ts owns 81 cells, nine board results and forced board routing. A move is encoded board*9+cell. Playing a cell directs the next player to that numbered local board; a closed destination permits any open board. Local wins contribute to the macro board. clone supports independent search; key represents search state; save/restore use legal histories.

UltimateSearch.ts evaluates local and macro threats and performs bounded iterative deepening with ordered moves, alpha-beta and a transposition table. UltimateAI.ts delegates to a cancellable worker. UltimateGame.ts owns pending/generation guards, normal/focused coordinate mapping, highlighting, animation, save points and lifecycle cancellation. UltimateTutorial.ts is the interactive setup explanation. ai.worker.ts restores a search board and returns a chosen legal action.

To change focus layout, edit UltimateGame.drawBoard and pointer conversion while preserving board*9+cell. To tune strength, edit UltimateSearch budgets/evaluation and UltimateAI setup rather than the legal rules. To change routing, start in UltimateBoard.play/legal and update tutorial/tests. Ultimate Impossible is bounded practical search; do not describe it as guaranteed optimal.

5.4 Connect Four - folder games/connect-four

ConnectFourRules.ts creates the 69 winning four-cell windows for a 7-column, 6-row board. drop finds a free cell under gravity, records history, detects a win/draw and advances turn. legal returns nonfull columns. undo reverses a search move. restore replays legal drops; it does not accept arbitrary stored cells.

ConnectFourGame.ts maps taps to columns, blocks input during falling-token animation or an AI turn, checkpoints complete moves and checks generation/disposal before accepting a worker reply. The delay is presentation/AI scheduling; the rules determine the true board. ConnectFourAI.ts evaluates center control and four-cell patterns, then uses bounded iterative-deepening alpha-beta with move ordering and transposition entries. Search temporarily mutates and undoes only its private board. The worker owns its restored copy.

To change AI valuation, edit evaluate. To change search strength, edit difficulty budgets/depth/node settings inside searchConnectFour. To change grid dimensions, update all indexing/window generation, rules, hit mapping, drawing and tests; replacing only a display width is insufficient. To change token animation speed, use ConnectFourGame's animation-age calculations without changing board gravity. Relevant coverage includes phase3, engine and AI refinement.

5.5 Dots and Boxes - folder games/dots-and-boxes

The current config exposes 4x4, 5x5 and 6x6 box variants. Rules.ts is parameterized: size n has n*n boxes, (n+1)*n horizontal edges and 2*n*(n+1) total edges. The supported variants therefore use 40/60/84 edges and 16/25/36 boxes. Some older project notes describe only 4x4; the source is authoritative.

createBoxes builds each box's four edge IDs. createAdjacent maps an edge to adjoining boxes. DotsBoard.play validates the edge, assigns its owner, claims every newly completed box, adds score and keeps the turn if at least one box was captured. A single edge can complete two boxes. When all boxes are owned, the higher score wins. clone gives an independent board; save/restore preserve size and a replayed edge history. edgeEnds converts an edge ID into visible endpoints. pickEdge selects a nearby middle portion with a size-scaled threshold.

AI.ts estimates sacrifice damage/chains, orders capture/safe/strategic moves, handles the extra-turn rule and runs bounded alpha-beta/endgame search. ai.worker.ts returns a chosen edge. DotsGame.ts owns selected-edge keyboard navigation, taps, animation, worker cancellation and background/turn feedback.

To add a board size, update config variants, board-size validation/cache tables, the controller's variant parsing and geometry assumptions, search budget, save/test expectations and preview. To make taps more forgiving, edit pickEdge's threshold, keeping endpoint ambiguity controlled. To change the extra-turn rule, change play and AI search turn handling together. The current extra turn is deliberate and central to chain strategy.

5.6 Hammer Hit - folder games/hammer-hit

Rules.ts owns ATTEMPTS=3, a separate gauge clock/cooldown per player, needle and strikeScore. needle=sin(time*(2.6+attempt*0.35)-pi/2). strikeScore=round(100*(1-min(1,abs(position)))). hit records an integer score, resets that gauge and imposes 1.1s reveal cooldown. totals decide the winner; averages divide totals by three for display only. Never choose a winner from rounded averages or best strike.

HammerGame.ts presents alternating full-field turns, resets the active gauge after handoff, delays AI preparation, owns TurnView for local Blue, and blocks new input until strike feedback finishes. It subdivides the bounded host update for 600Hz gauge/AI timing. Search its active-player, preparation and update blocks when changing handoff pacing. Pause freezes gauge, handoff and presentation clocks.

AI.ts samples a target score band once per strike and hits the actual needle through the ordinary legal action. Impossible chooses 97..100 with probability 70%, 90..96 with 25%, and 75..89 with 5%; 97 belongs to the top band. It never assigns a score directly. Hard/Medium have their own band tables.

StrikePhysics.ts owns the presentation-only ballistic puck. step integrates y/velocity, handles separating bell contact with 0.28 rebound, and settles at y=602. ImpactRing is a damped spring: angular acceleration=-65*angle-9*velocity; strike adds 2.8 angular velocity. Change those for visual ringing, not score. The hammer's grip remains a fixed pivot in the controller drawing.

To increase attempts, update ATTEMPTS, controller turn/labels/results and AI's current length===3 guard. To widen the scoring sweet spot, edit strikeScore and update AI target interpretation/tests. To tune gauge frequency, edit needle. To tune strike height, edit StrikePhysics only. Unit checks separate rules, AI and presentation.

5.7 Light Fingers - folder games/light-fingers

Rules.ts owns waiting -> signal -> active -> between -> done. The gem initially waits 1.5..4.5s. The controller draws the signal, then calls presentSignal to make it legally claimable. This prevents AI responding to a hidden state before players could see it. tap during waiting/between is a false start and blocks only that player for 1.25s; signal-stage taps are rejected. Active taps go through FirstResponse. A same-instant tie leaves the gem available and locks both briefly. A successful steal gives one point to its owner, while an idle rival is not penalized. First to five wins; the between beat is 1.1s.

LightGame.ts wires taps/keys, updates the model/AI, presents the visible signal, follows reach/steal animation and reports a finished match once. LightArt.ts draws original gloves, forearm, plinth and gem. Reach animation follows the

outcome and never delays arbitration. AI.ts starts its delay from signalAt, with means 0.53/0.29/0.145s and a 0.12s floor; it uses the same tap method and can retry after a tied cooldown.

To change waiting suspense, edit the two waiting-duration expressions in Rules. To change false-start punishment, edit blockedUntil durations and related AI retry pacing. To change score target, update the score===5 terminal check and visible instructions. To change reach appearance, edit LightArt/controller animation without moving presentSignal after a long animation. Relevant tests cover false starts, visibility, ties, cooldown and completed matches.

5.8 Fruit Duel - folder games/fruit-duel

Rules.ts defines seven real fruits (IDs 0..6) and three decoys (7 gift, 8 medal, 9 flower). Every real fruit scores; there is no target-memory task. reveal guarantees a real fruit within three reveals, otherwise chooses fruit with probability 0.58. show waits for the controller's present call, then active lasts 0.85..1.25s. A legal fruit tap captures its ID immediately, preventing a later reveal from changing the judgment. FirstResponse resolves the race; ties award no point. First to ten wins.

SwordMotion.ts is shared by rule cooldowns and rendering. SWING_IMPACT=0.15, HIT_RECOVERY=0.72, MISS_RECOVERY=1.35 seconds. swordPose interpolates mirrored rest/windup/impact/follow-through/return poses with smooth easing. The sword returns to its original backed-away rest exactly when input unlocks. Current swordPose uses index0 for the base pose arrays and mirrors Blue by x=420-x, y=762-y, angle+=pi. Editing an unused second array entry alone has no effect; edit the base pose or the mirror transform. Empty/decoy swings cause recovery but do not give the rival a point.

FruitGame.ts owns retained graphics, bounded halves/droplets, swing start times, impact-delayed slice sound/visuals and result feedback. FruitArt.ts draws fruit interiors, decoys and the curved sword. AI.ts reacts only to presented objects, samples recognition error once per reveal, and calls the same tap. Impossible intentionally attacks no decoys; reaction means are 0.46/0.275/0.145s with a 0.12s minimum.

To add a fruit, change the ID arrays, isFruit, reveal's hardcoded 7/3 ranges, FruitArt and any AI/tests assuming ten kinds. To slow returns, edit SwordMotion durations so rendering and input remain synchronized. To alter reveal cadence, edit Rules.present and preview/between deadlines. To change the target, edit the scores>=10 terminal rule plus metadata. The retained target field stores the latest real fruit; it is not an instruction to match a remembered target.

5.9 Math Quiz - folder games/math-quiz

MathGame.ts constructs a QuizMatch and question provider, renders the DOM MathUI, maps Orange 1/2/3 and local Blue 7/8/9, calls present after rendering, and reports the winner once. Rules.ts arbitrates the first answer with FirstResponse: first correct earns one point, first wrong gives the rival one point, same-instant responses advance without a point, first to ten wins. The between-question beat is 1.4s.

questions/Question.ts defines a MathQuestion with ID, topic, difficulty, questionLatex, exactly three optionsLatex, correctIndex 0..2, explanationLatex and estimatedSeconds. It also owns fraction normalization, LaTeX formatting and answer shuffling that remaps correctIndex. Basic.ts generates seven basic arithmetic categories with exact integer/rational answers and plausible distractors. ProceduralUniversity.ts generates university families across eight topics. university.json is the static 100-question bank. Bank.ts combines and deduplicates static/procedural IDs, groups by topic, prioritizes unseen questions and interleaves a finite deck. History.ts persists a bounded 150-ID set and removes oldest entries. It is implemented as an insertion-ordered Set with FIFO eviction, not a separate array ring structure.

MathUI.ts creates KaTeX DOM content, player-local topic labels and three option buttons. It rotates the Blue section for local play and auto-fits both axes after font readiness and resize. quiz.css owns equation/button spacing and text layout. AI.ts chooses a difficulty-calibrated answer after visible question latency; it changes neither the bank nor score directly.

To add static questions, preserve the JSON shape, stable unique ID, three distinct options and exact correctIndex. Run math-quiz tests; use verify-math-bank.py only when intentionally regenerating the static bank, because it overwrites that JSON and its report. To add a procedural family, extend the corresponding generator and validity/uniqueness checks.

To change repetition memory, edit MAX_HISTORY in History. To improve readability, edit MathUI/quiz.css rather than shrinking physics coordinates. To change target points, update QuizMatch.update and instructions. The file atlas maps all generator helpers individually.

5.10 Lumberjack - folder games/lumberjack

Rules.ts is compact but owns all gameplay. branches uses a seeded linear congruential sequence, starts with two clear segments and prevents adjacent branches from occupying opposite sides. Both players consume the same pattern independently. safe checks both current and next branch before a chop. chop changes side, applies 0.085s cooldown, kills an unsafe chop or increments score/reset idle. limit=max(0.32,1.5-score*0.035) seconds is the allowed idle time. A round lasts at most 20s. After both are dead, scores add to totals; at least three rounds play, with extra rounds while cumulative totals tie. Between rounds lasts 1.4s.

LumberGame.ts maps left/right actions, drives model/AI and draws two independent trees with local Blue orientation. LumberArt.ts owns foliage, original characters, seven visible 28-unit segments, axes and edge meters. Chop displacement is capped and decays; character sides change immediately, avoiding motion through the trunk.

AI.ts selects a legal side from the same visible seven segments and returns an ordinary chop choice. Mean decision intervals are 0.42/0.245/0.135s with personality variation. Hard/Impossible bound decisions to the remaining idle budget; Impossible deliberately chooses no branch. Some Medium/Hard recognition errors remain intentional.

To make play easier, increase the idle budget or reduce its score slope, then retune AI pacing and edge-meter presentation. To lengthen rounds, change ROUND_SECONDS. To change stream density, edit branches while preserving a legal side. To change number of rounds, edit round>=3; ties still continue. To change visuals, edit LumberArt/controller offsets, keeping the seven visible segments consistent with AI lookahead.

5.11 Water Guess - folder games/water-guess

WaterGuessRules.ts stores percentages as integer hundredths of one percent: 5000 means 50.00%. MIN_PERCENT=1000 and MAX_PERCENT=9000 generate 10.00..90.00% targets. startNewRound avoids used values with a bounded 100-attempt fallback. submit rejects nonfinite input, clamps/rounds to 0..10000 and allows one guess per player. resolveRound compares absolute integer errors, awards the closer player one point, awards none on equal error, and caches the result so repeated calls cannot add points. Five rounds are followed by sudden death while scores tie.

WaterGuessGame.ts is the turn/fill/guess/reveal controller. setupKeypad builds the numeric buttons; setupInputs handles digits, decimal, backspace and submit. appendDigit/appendDecimal maintain the buffer; startRound animates fill; submitCurrentGuess commits; triggerReveal shows the answer/errors; advanceNext progresses. It uses TurnView for full local Blue field/keypad rotation and its inverse for pointer hits. It rejects actions while transitioning and waits for AI preparation using foreground update time.

Geometry.ts owns TUBE_HEIGHT=310, shared with WaterRenderer and AI. WaterRenderer.ts draws the outlined tube and liquid. WaterGuessAI.ts rounds the hidden level to the visible one-logical-pixel tube measurement, then applies difficulty/personality bias and noise. Spread is 550/170/0 hundredths for Medium/Hard/Impossible. Impossible therefore has pixel quantization rather than oracle-perfect hidden decimal access.

To change target range, edit MIN_PERCENT/MAX_PERCENT. To change rounds, edit TOTAL_ROUNDS and every controller label/terminal assumption. To change tube height, edit Geometry and renderer endpoints together. To change accepted precision, update parsing, integer units, display and comparisons consistently. parsePercentInput uses parseFloat and a forgiving fallback, while submit enforces finite numeric input; strict text validation would require an explicit parser change and tests. Changing display precision alone must not silently change legal error comparison.

5.12 Perfect Timer - folder games/perfect-timer

PerfectTimerRules.ts stores durations in integer centiseconds. TARGET_PRESETS=[500,743,1000,1267,1525] means 5.00, 7.43, 10.00, 12.67 and 15.25s. startNewRound prefers unused presets, then generates 500..1500 with a bounded deterministic uniqueness fallback. recordStop takes elapsed milliseconds, rejects invalid/negative values, rounds elapsedMs/10, and records each player once. resolveRound compares absolute errors, awards the closer player and caches its result. Five rounds extend into sudden death on a tie.

TimerClock.ts is the time authority. start records performance.now, elapsed returns carried plus foreground running time, pause banks elapsed, resume starts a fresh reference. PerfectTimerGame.ts uses it for both human events and AI deadlines, so background/pause time never leaks into a stop. Its controller phases include ready, countdown, timing and reveal. handlePlayerAction performs the phase-appropriate action; startTimingPhase starts the clock; triggerReveal resolves only after both stops; advanceNext opens the next round. Both players stop the same timer simultaneously; there is no per-stop rotation.

PerfectTimerAI.ts schedules a legal deadline in milliseconds from the same foreground elapsed clock. Error spreads are 72/21/0 centiseconds with persistent fast/slow personality bias. Impossible targets the exact visible duration. TimerRenderer.ts draws the stopwatch. The controller plays a quiet clockTick at most once per current half-second index, avoiding catch-up bursts, and clockStop on an actual stop.

To change durations, edit TARGET_PRESETS plus fallback range if desired. To change precision, update centisecond conversion, formatting, AI units and tests together. To tune AI error, edit spread in scheduleRound. To change timer art, edit TimerRenderer; do not replace TimerClock with raw elapsed from an unadjusted initial timestamp. To change round count, update TOTAL_ROUNDS and controller labels/assumptions.

5.13 Gravity Runner - folder games/gravity-runner

Rules.ts exports GravityMatch. Both runners share one endless seeded course and progress distance. A run ends on collision, not a time deadline. A sole survivor gets one point; simultaneous deaths replay. First to five wins. flip reverses gravity and applies a 160ms lock. speed(age) starts at 120, then six smooth +24 ramps, beginning after 8s and then every 20s over a 3s easing window, reaching 264. distance advances using that age-dependent speed.

RunnerPhysics.ts owns cross-track y, vy, gravity, alive and cooldown. advanceRunner uses gravity 900, supports intact outer walls at centers y 10/160, detects wall gaps with a 13-unit longitudinal reach, allows swept landings on runnable platform faces, rejects front penetration/late wall rescue, applies trap contact and kills escape beyond -22..192. The collision footprint is 26x18. There is no runner-to-runner collision.

CourseStream.ts selects weighted map-specific motifs excluding the previous two choices: single, switchback, cluster, gaps, slalom, rail, pinch and islands. It guarantees an early rail teaching section. Geometry is generated ahead about 2100 units, retired behind about 400, and history is capped at 16. Independent hazards retain at least 245 clearance. Rail gaps begin/end 180 units inside the supporting platform; pinches retain outer walls. Maps.ts is lightweight metadata/preview colors. TRAPS chooses ice icicle/block, green root/bramble, blue jet/buoy and purple crystal/phase families.

ObstacleGeometry.ts is shared by collision, art and AI. obstacleBounds computes timed jet depth and phase-gate activity/warning from the same round age. obstacleOutline returns canonical convex polygons. touchesTrap uses separating-axis tests for the runner rectangle against nonrectangular traps. AI.ts forecasts legal advanceRunner steps, tests bounded one/two-flip plans and uses difficulty-specific reaction/lookahead. GravityView interpolates display only; GravityArt/TrapArt draw the corridor, runners and canonical traps. GravityGame owns controls, fixed ticks, view capture and results.

To change difficulty ramp, edit speed, not a screen scroll coefficient. To change gravity/footprint, edit RunnerPhysics plus shared shape extents and revalidate forecasts/courses. To change obstacle density, edit CourseStream clearance/count/width rules, retaining reachability. To add a map, extend Maps and TRAPS, then tests. To change jet/gate timing, edit the generated period/phase and ObstacleGeometry formula. Never let AI use a different age from human collision or drawing.

5.14 Happy Birds - folder games/happy-birds

Rules.ts exports BirdsMatch: one shared 320x460 arena, two auto-flying birds, one star and first-to-15 scoring. There is a 1.1s opening beat, no match deadline or round reset, and individual recovery rather than rival survival points. speed=min(165,115+age*0.25). flap sets vy=-195, applies 140ms lock and increments a visual serial. ArenaPhysics uses gravity 650, radius 15, wall-triangle collisions, ceiling/floor hazards at 16-unit borders and safe horizontal bounces. Birds do not collide with each other.

Rules.wall creates a seeded three-row safe opening and four spike rows. schedule previews a change for 0.4s; clear defers committing it until both active birds are over 100 units away. A collision calls beginRecovery for that bird only. Recovery.ts freezes ordinary input/physics/collection for 3s, captures the impact, smoothly curves toward home (140/180,230), fades from gray and relaunches upward at -125. Recovery is model-time and pauses with gameplay.

Stars.ts sets STAR_GOAL=15 and STAR_RADIUS=10. spawnStar samples up to 64 arena candidates within x58..262 and y110..350, preferring at least 65 distance from active birds and 80 from the previous star; bounded failure uses the best-clearance candidate. pickupDistance computes distance from the star to the swept bird segment. Rules waits 0.18s birth time, awards the nearest eligible touching bird, shares credit for exactly equal distances, removes the collected star and spawns one replacement. Recovering birds cannot collect. Bursts expire after 0.5s.

AI.ts routes to real stars and safe wall openings through legal flaps with the same physics. BirdView handles interpolation, facing/wing motion and recovery reset. BirdArt draws original birds, hinged wings, spikes, stars and bursts. BirdsGame handles Orange lower/Blue upper inputs, AI scheduling, view capture and result reporting.

To change target points, edit STAR_GOAL and all target labels/metadata. To change flap behavior, edit ArenaPhysics constants and retest forecasts/reachability. To change recovery duration/path, edit Recovery's time/easing/home/relaunch consistently. To change star placement, edit spawnStar constraints and birth/pickup checks. To change wall density, edit Rules.wall/schedule while preserving safe openings and telegraph clearance.

5.15 Spike Attacks - folder games/spike-attacks

Maps.ts defines Courtyard, Split Steps and Crosswind plus seeded Random selection. Each map has three rectangular cover blocks and two spawns. MapPreview.ts draws metadata previews. ArenaPhysics.ts is authoritative for SIZE=320, RADIUS=9, SPEED=160, SHAFT=72, lane centers, legal points, cover-limited attackDepth, pistonHits and movement sliding around expanded blocks. Each edge has four 80-unit lanes; 72-wide shafts leave gaps too small to hide a radius-nine player. Both players face the same hazards and do not collide with one another.

Rules.ts owns warning -> extend -> hold -> retract -> recover. warning=max(1.05,1.85-(wave-1)*0.045); extensionSpeed=min(940,620+(wave-1)*18). Every piston extends only to the cover-limited attackDepth. Hold lasts 0.32+min(0.18,wave*0.008)s. Retraction uses smoothstep over 0.64s; recovery lasts 0.18s. nextWave chooses a seeded different direction. tick moves players and checks the maximum old/new extension to cover swept danger. Sole survival scores; same-step deaths replay; first to five wins.

The 120s match deadline is advanced separately by advanceClock using foreground dt and persists through round resets. Tied expiry draws. SpikeAI constructs bounded candidate safe points and a visibility graph around expanded map blocks at wave changes, then follows ordinary speed-limited targets. SpikeView interpolates position, shaft lengths and facing. SpikeArt owns themed arena, characters, warnings and joysticks. SpikeGame wires all control sources and model/view flow.

SpikeControls.ts prioritizes keyboard direction, normalizes gamepad axes/D-pad with dead zone, assigns stable player slots and requires neutral after pause. Gamepads are polled in the existing update; there are no global subscriptions. Only human players receive joysticks/drag subscriptions; single-player accepts arrows or WASD, local Orange WASD and Blue arrows. Cancel/release/pause clears movement.

To change movement speed/radius, edit ArenaPhysics and retest safe routes/cover geometry. To change warning tempo, edit Rules.warningDuration and preserve visible warnings. To change layouts, edit Maps so art, physics and AI share the selected blocks; avoid the default BLOCKS compatibility constant in new map-aware code. To tune controller dead zone/priority, edit SpikeControls. To change deadline, update remaining and advanceClock together.

6. Arcade games and Ultimate Golf

6.1 Classic Snake - folder arcade/snake

SnakeRules.ts owns SnakeWorld: 17x20 grid, a three-cell initial body, current direction, a two-action direction queue, food, score and accumulator. steer rejects duplicates, reverse turns and a third queued action. update caps catch-up at 0.5s and repeatedly calls tick using interval=max(0.065,0.18-score*0.0015). Each food is worth 10, so the speed changes by score rather than food count directly.

tick computes the new head and checks walls/body. When not eating, the old tail is excluded from collision because it moves away in the same step. Eating grows the body and spawns food from an enumerated free-cell list, avoiding rejection loops. Filling the grid sets won/dead and clears food. SnakeGame maps swipe, direction pad and keyboard, interpolates from previous positions and reports finishArcade. Pause clears queued directions/held controls.

To change grid size, edit width/height plus render spacing and starting body. To slow progression, edit interval's score coefficient or floor. To change food points, edit score+=10 and consider its speed effect. To allow wrapping, explicitly change tick's boundary rule and interpolation/tests; changing the drawing alone cannot wrap collisions.

6.2 Star Patrol - folder arcade/space-shooter

ShooterWorld.ts owns player motion, swept projectile collision, damage, pickups and bounded pools. Waves.ts now owns formation/enemy roles, speed and attack cadence; Powerups.ts owns upgrade metadata/rewards/life eligibility; ShooterArt.ts owns the original live artwork; SpaceShooterGame.ts owns host inputs, fixed stepping, HUD/audio and cleanup. These files were added/refined during the documentation review and this section follows the extracted snapshot rather than the older three-file description.

The player starts with three lives and moves at370 units/s within x44..376. There are80 pooled projectiles: slots0..51 reserved for player fire and52..79 for enemies, preventing spread fire from suppressing enemy attacks. Player bullets travel at vy=-490; enemies use speed min(340,155+wave*13), with straight, paired, aimed and boss patterns. contact computes swept segment-to-box entry time on both axes, including diagonal shots. The closest eligible enemy is hit first; laser projectiles track prior hits and can pierce up to three targets. The pool stores damage, pierce and laser flags per shot.

Waves.formation creates up to five rows, normally six columns, with five columns and an added boss every fourth wave. Roles include ordinary alternating types, tougher armored enemies, aimed attackers, a reward carrier and boss. waveSpeed=min(130,28+wave*6+killed*0.8). attackInterval=max(0.42,1.4-wave*0.065). Front-row attackers charge visibly for0.38s, or0.55s for a boss, before firing. Formations reverse/drop10 at edges, some enemies sway, and all descend slowly. Kill points are10+kind*5; wave clear adds wave*50 and a1.8s transition.

The pickup pool has eight slots and bursts are capped at24 with0.45s lifetime. Upgrades are spread cannon, rapid fire14s, piercing laser16s, wingmates18s, shield and life. Timed powers accumulate to at most24s. spread increments weaponLevel up to3; rapid changes player shot interval from0.23 to0.11s; drones add shots on alternating firing cycles; shield absorbs a hit and gives0.8s immunity. Ordinary damage removes a life, reduces weaponLevel by one and gives1.5s immunity. spawnWave grants at least1.2s immunity. Extra-life drops require a sixth-wave milestone,90s since the last life drop and fewer than four lives. A life pickup at the cap grants a shield instead. Reward carriers use the deterministic waveReward cycle; ordinary loot can drop every eighth kill.

Pickups fall at86 units/s, attract horizontally toward a nearby player below y450 at up to110 units/s, collect within24x25, and retire below640 or after9s. ShooterArt renders original craft/enemy types, charging telegraphs, upgrades, effects and gear indicators in one retained masked Graphics. The controller starts after a0.7s ready beat, fires while lower-field drag or Space is held, keeps keyboard/drag targets legal, rate-limits shot feedback and reports finishArcade once. Pause cancels dragging and fixed catch-up; disposal releases graphics, mask and text.

To change movement/fire, edit ShooterWorld's caps and shotClock. To change wave roles/health/bosses, edit Waves.formation; tune waveSpeed/attackInterval separately. To change power duration/name/color, edit POWERUPS;

to change rare-life policy, edit canDropLife and drop. To change pickup collection motion, edit the pickup loop in step. To change visuals, edit ShooterArt, preserving shared hit dimensions. Validate phase3 and star-patrol unit tests plus the real browser controls; effects/upgrades must never grow unbounded.

6.3 Ultimate Golf - active folder games/mini-golf/ultimate

The parent config keeps id mini-golf, display name Ultimate Golf, category arcade, custom/responsive presentation and tournament=false. The parent index lazily creates UltimateGolfGame. Everything in this subsection is active. The old parent-root duel files are explained separately in Chapter 8.

Controller and three modes

UltimateGolfGame.ts owns mode/course/challenge selection, help, input drag ownership, focus camera, result screens and lifecycle. initialize creates GolfUI/renderer, binds keys/drag and applies options. action handles menu controls. release converts one owned gesture into one shot only while shootable. attachWorld binds the current physics world to presentation. update advances the selected mode, AI world if applicable, camera/rendering and HUD. pause cancels aiming, resets physics catch-up and cancels worker planning; dispose removes DOM/graphics/masks/workers.

modes/CasualMode.ts starts a course/challenge and records casual bests through GolfProgress. LocalVersusMode.ts gives Orange then Blue identical seeded course/tour conditions with a handoff phase; a course has a 180s foreground cap and Easy stroke allowance. CompetitionRules.total/winner compare completion first, then total time, use a 0.1s tolerance and fewer strokes as the time tie-break. AIChallengeMode.ts advances two independent real worlds concurrently and records first completion/failure with the close-finish rule. CompetitiveStats.ts stores separate counters under golf-versus; these never modify casual records. Help, handoff, menus and pause do not consume competition time.

Rules and ball trajectory

GolfRules.ts defines three challenge records. Easy: obstacle speed 0.7, stroke factor 1.5, preview 0.6s, bonus 100 plus 10 per unused stroke. Normal: 1.0/1.1/0.3s/250+25. Hard: 1.3/0.75/no preview/500+50. strokeLimit rounds baseLimit*factor; scoreFor uses the remaining allowed strokes.

GolfCollision.ts defines RADIUS=7 and MAX_SPEED=650. shotVector in GolfInput.ts uses (dragStart-dragEnd)*3.125 in world units and caps magnitude to MAX_SPEED. GolfPhysics.shoot requires a stopped playing ball, remaining strokes, finite velocity and magnitude at least 43.75, equivalent to 14 world units of pullback. It saves lastShot and increments strokes exactly once.

GolfPhysics.advance caps render catch-up at 0.1s and simulates at 240Hz. tick first updates obstacle dynamics. UFO carrying suspends ground/hazard logic. Surfaces can replace normal friction 110 or apply push acceleration. Oil uses friction 20. Fields accelerate toward their center, optionally with quadratic falloff or only while moving. Speed is capped, position integrates, and friction reduces speed linearly by friction*dt, scaling vx/vy proportionally. A non-pushed ball stops below 6. This is not global exponential drag.

Then tick tracks bridge/plank/plate state, handles directed portals with 0.5s cooldown and minimum exit speed 80, recovers unsafe/lava/laser/quicksand states, resolves circles/movers/fans/diamond/piston/doors/cores/walls, and checks bounded contact oscillation or nonfinite/out-of-bounds state. Recovery returns to a legal last-shot position without an extra stroke. The cup captures inside abs(dx)<8.25 and abs(dy)<8.25 with no added speed test. A last-stroke cup win is checked before exhausted-stroke loss.

GolfCollision.hitRect/hitCircle project the ball outside overlap and reflect only approaching relative velocity; hitRotated transforms into an obstacle's local coordinates and accounts for contact-point angular velocity. Moving contact transfers legal momentum, then cap limits speed. Never enlarge the physical ball to match its visibility halo.

Dynamics and four course data files

GolfDynamics.ts uses difficulty-scaled obstacle time. mover traverses a triangular out-and-back path and returns real motion velocity. laser is active when (time+phase)%3>=1.6. bridgeSafe uses the 8s tidal cycle. piston implements fast extension, hold and slower retraction. safe checks ground, river/planks, bridges and ford. update advances doors, plates, planks, cooldowns and the Horizon UFO. recovery moves last-shot returns out of mover/fan/piston sweeps or chooses a safe start fallback.

Course data	Main data/mechanics to edit	Base stroke limit
courses/foundry.ts	Six serpentine lanes, crate/diamond, bumpers, belts, oil, moving beams, magnet/core, rotating cross, piston, lava and laser	18
courses/shoreline.ts	Ten ground islands/paths, sand/slick/ford, tidal bridges/current, crabs, palms/bumpers and whirlpool fields	25
courses/horizon.ts	Eleven platforms, slick strips, solar wind, fields/cores, directed portals, asteroids, lasers, rotating cross and UFO config	20
courses/temple.ts	Walls, river/two five-plank bridges, mud/slick/quicksand, boulders, dart gates, rotating cross, entrance plate and two latched plates/doors	18

courses/types.ts defines every data shape: Point, Rect, Circle, Mover, Surface, Field, Fan, Portal, UFOConfig and Course. p/r/outer are coordinate constructors/boundaries. courses/index.ts fixes course ordering. Keep original 1920x1080 source units. A course rect is x,y,w,h; a mover has from,to,seconds and radius or dimensions; a field has radius,strength,optional core/quadratic/movingOnly; a portal has from,to and exit angle in degrees. Course art and physics consume the same data.

Horizon's UFO values are first 11.5 obstacle seconds, interval 28, warning 1.5, speed 170, duration 13, hover 1, carry 0.8 and cooldown 8. Its seeded event hash makes competitors reproducible. Drop selection rejects hazards and falls back to lastShot. The third portal exits toward (960,310), then offsets 35 units along its angle. The course port is mechanically derived from an external source described in docs/ULTIMATE_GOLF.md; consult that file's provenance/license discussion before redistributing the derived course layouts. This handbook makes no new licensing claim.

Camera, aim preview and AI

GolfView uses one uniform scale. Portrait rotates clockwise; landscape is unrotated. project/unproject are exact inverses around the same center/origin/scale. Focus zoom is clamped to 2.2..4 and follows/clamps the ball neighborhood. UltimateGolfGame holds the camera during aiming so a stationary finger does not change the shot. GolfUI.bounds reserves actual responsive header/footer space.

GolfRenderer.ts owns retained masked Graphics and original vector course artwork. components/CourseArt.ts generates static SVG course overviews from real data without creating simulations. ModeArt.ts draws small mode icons. GolfUI.ts owns DOM menu/HUD/help/meter/rival elements; components/golf.css controls their responsive layout. GolfProgress.ts validates and stores four casual best scores.

GolfPhysics.preview makes a disposable ghost simulation for a bounded path and stops at impact/recovery/portal/terminal state. The renderer also contains the short preview drawing logic; moving hazards may change before release. Preview dots are guidance, not a guaranteed future path.

ai/GolfAI.ts owns planning scheduling/generation and calls the same world.shoot API. Current base decision delays are 2.2/1.6/1.2s for Medium/Hard/Impossible before personality variation; accelerated mode uses 0.25s. Older prose

mentioning 1.1/0.8/0.65s is stale. ShotPlanner.ts copies the actual world snapshot, records reached waypoints before forecasting reaction delay, simulates candidate angles/powers/waits in the actual 240Hz physics, refines the best result and selects only comparably good simulated landings. Budgets are 65/140/240ms and replay candidate limits 180/420/800. Each trial is bounded to ten simulated seconds. Penalties discourage recovery, ongoing motion and missed Temple plates. routes.ts supplies legal waypoint guidance and comparable route variants, not scripted awarded progress or shot velocities. planner.worker.ts runs the planner; cancellation terminates computation.

Golf change guide

Desired change	Edit	Related work
Stronger/weaker pullback	GolfInput.shotVector multiplier 3.125	shoot minimum 43.75, preview/power UI and touch tests
Faster maximum ball	GolfCollision.MAX_SPEED	planner powers/estimates, collision stress and route checks
More/less rolling	GolfPhysics.tick friction and course surface friction	Planner uses shared physics; legal route difficulty changes
More allowed strokes	Course.baseLimit or GolfRules factor	Score formula and tests use the same rounded limit
Easier moving obstacles	GolfRules speed multiplier	Dynamics contact velocity must remain scaled consistently
Move an obstacle	Corresponding course data	Course art updates; verify route clearance and AI waypoints
Change laser/tide/piston timing	GolfDynamics methods	Test timing and safe traversal at every challenge
Longer competition	CompetitionRules.COURSE_TIMEOUT and mode checks	Keep foreground time semantics
Faster AI thinking/action	ShotPlanner budget and GolfAI baseDelay separately	Search quality and reaction pacing are different settings
Better Temple route	ai/routes.ts waypoints and planner cost	Stay outside bumpers, quicksand and mover sweep
Different camera/UI size	GolfView, GolfUI.bounds and golf.css	Verify project/unproject after focus/resize

7. Preserved future games

These folders are outside both discovery globs and therefore absent from menus, global tournaments and production module chunks. Their unit tests remain active. To reactivate, move the entire module from src/future-games to src/games, restore its browser spec from tests/future to tests/browser, rebuild and run module/bundle/browser checks. Preserve stable IDs. Old unavailable tournament rounds cannot launch; create a new tournament rather than fabricating a result.

7.1 Flappy Jump - future-games/flappy-jump

Despite its name, this is a running basketball jumper. Rules.ts defines Jumper and BasketBall plus walk, projectedLanding and FlappyMatch. Both players receive identically seeded hoop positions. The runner moves at 70 units/s between x45 and195, reflecting at boundaries. jump applies vy=-310 under gravity750. At the jump apex, the ball releases at y-20 with vx225/vy-200. Ball gravity is also750. Rim-end circles use 10-unit combined contact radius and separating impulses; the backboard reflects with 0.65 horizontal restitution. Scoring requires a downward swept crossing of the hoop plane inside x+/-14 and a per-ball scored guard. Balls retire on bounds or after3s. First ten baskets wins; same-step finish draws; a240s cap compares score.

AI.ts uses projectedLanding to time the same jump action, plus seeded difficulty offset/delay. FlappyGame.ts maps input, advances120Hz, draws and reports. To change jump height, edit jump impulse/gravity and projectedLanding together. To change ball arc, edit release velocity and predictor. To change hoop range, edit hoop generation and prove reachability. The written implementation uses original balancing choices, not a claim of exact missing reference-PDF physics.

7.2 Car Duel - future-games/car-duel

Physics.ts contains a custom rigid-body approximation, not a second Phaser physics engine. Car has x/y/vx/vy/angle/omega/facing/alive/grounded. terrain is a compact raised middle bump. point rotates body-local coordinates; head and colliders share these transforms with rendering. INERTIA=560. integrate clamps input -1..1, applies grounded/air throttle145/35, gravity520, input torque1.1/3.2, exponential damping0.65, speed/rotation caps and three ground-contact projection passes. Ground normals derive from terrain slope; impulses include angular lever arm. Head contact with ground/walls kills. collide tests exposed heads against the opponent's body/head, projects chassis circles apart and applies an approaching relative impulse.

Rules.ts spawns cars and scores first-to-five rounds. Simultaneous deaths replay; stalled40s rounds replay; total240s compares scores. CarInput.ts owns per-pointer left/right button subscriptions and clears them on pause/dispose. CarAI drives ordinary throttle/torque-limited inputs. CarGame owns fixed120Hz update and original rendering. To change handling, edit integrate constants; to change collision shape, edit colliders/head and shared drawing transforms together. To change terrain, edit terrain and retest grounding/slope/head contact. Avoid modifying only the drawn wheels or body silhouette.

7.3 Archery Duel - future-games/archery-duel

Rules.ts owns ArcheryMatch, seeded target centers and an Archer three-tap state machine. Taps start aiming, lock horizontal crosshair, then lock vertical crosshair/fire. Lock debounce is120ms. The crosshair reflects at175 units/s. After380ms impact delay, the distance to the actual target awards1 or2 using the same visible circle radii. First to ten wins with180s safety cap; per-attempt target sequences are shared but consumed independently. The two target radii are28 and14, worth1 and2 points. Horizontal/vertical crosshair limits are340/170. AI.ts waits/locks the real moving crosshair instead of inventing impact coordinates. ArcheryGame.ts presents mirrored player fields, maps keys/taps, runs fixed updates and reports the result.

To change aim difficulty, edit crosshair speed, target radii or AI target offsets/delay. Keep drawing/rule radii synchronized. To change flight/impact delay, edit the Rules phase deadline and arrow presentation consistently. To

change target range or points, edit seeded targets/scoring and tests. Every attempt can resolve only once.

7.4 Chicken Jump - future-games/chicken-jump

Rules.ts owns two independent Chicken stacks and geometric descending landings. A sliding block moves horizontally; stop locks it and launches the chicken. The next height changes only after the descending chicken lands within the legal block interval. A miss retries the same height. First ten stacks wins, same-step finish draws, and180s expiry compares heights. speed, stop and step contain the sliding/bounce/jump/landing rules. ChickenAI samples an alignment offset per attempt/height and invokes the same stop action. ChickenGame handles input,120Hz updates and drawing.

Current defaults: block width65 shrinks by2.5 per successful stack to a38 minimum; slide speed is92+min(85,score*8). stop sets vy=-310; gravity is800. A descending crossing of y=-26 lands when abs(chicken.x-block.x)<=width/2+6. A miss falls beyond y60, waits0.65s and retries. To change stack target or timeout, edit step's terminal checks and metadata. To change alignment tolerance, edit geometric landing bounds together with block/chicken visible widths. To change block speed progression, edit speed; tune AI reaction without bypassing stop. To change jump shape, edit vertical velocity/gravity and camera/art follow behavior together.

8. Retained legacy golf and the example module

Legacy five-course duel - games/mini-golf root files

The active index does not import MiniGolfGame.ts, GolfMatch.ts, GolfPhysics.ts, GolfAI.ts, ai.worker.ts or courses.ts at this parent level. They remain source history with tests. Do not confuse their constants with Ultimate Golf's active constants.

courses.ts contains five small340x400 layouts. Legacy GolfPhysics.ts sets BALL_RADIUS7, FRICTION65 and MAX_POWER600. shoot requires a stationary/unholed ball and power>=20, then assigns angle-based velocity. golfStep subdivides travel to at most half a radius, uses separating-normal bounces with coefficient1.82, two projection passes, linear friction, stop threshold8 and circular cup capture distance<11 only below speed150. simulate copies a ball and advances up to1500 steps at120Hz.

GolfMatch.ts alternates players, counts strokes per hole, caps an unholed12th-stroke attempt as13 and marks it finished, totals the selected number of holes and chooses lower strokes. Saves contain validated stopped balls, hole/turn/count/history and totals. MiniGolfGame.ts owns the old drag-to-shoot, duel UI, fixed stepping, save points and worker lifecycle.

GolfAI.ts constructs a visibility route around expanded rectangles, derives stopping-power estimates sqrt(2*FRICTION*distance), simulates candidate angles/powers and bounds difficulty budgets45/140/300ms. Impossible considers a direct second shot from promising finalists. Its worker returns a normal legal shot. These algorithms are educational history; edits here do not change the current Ultimate Golf card.

examples/tap-duel

config.ts demonstrates metadata, assets/preview.svg provides its thumbnail, and index.ts contains a complete small MiniGame. Copy the whole folder into src/games/tap-duel to register it at build time. Use initialize to bind host-owned inputs and make display objects, update for foreground behavior, ctx.setScore for HUD, ctx.finish exactly once, and dispose for module resources. scripts/check-module.mjs proves folder-only discovery in a temporary copy rather than adding the example to the real collection.

9. Build, offline support, Android and assets

Commands and what each does

Command	Purpose
npm ci	Install exact package-lock dependency graph
npm run dev	Start Vite on127.0.0.1:5173
npm run build	Strict tsc check, then production Vite build to dist
npm run preview	Serve existing production dist on preview port
npm test	Active Vitest unit suites, including future-game units
npm run test:browser	Active Playwright tests under tests/browser
npm run test:modules	Temporary-copy example registration/build proof
npm run test:bundle	Verify shell budget and lazy module chunks
node scripts/measure-performance.mjs	Browser performance sampling; inspect required running server in script
node scripts/measure-expansion.mjs	Expansion performance sample using a running dev server
python3 scripts/verify-math-bank.py	Regenerate/verify 100 static questions, overwriting bank/report; requires SymPy
npm run android:sync	Build web assets, then cap sync android
npm run android:open	Open native wrapper in Android Studio
npm run android:debug	Sync then Gradle assembleDebug
npm run android:release	Sync then Gradle bundleRelease with local signing prerequisites

scripts/offline-plugin.ts emits sw.js with embedded FILES and SHELL lists from the build. Shell precaching is separate from caching game/worker assets on demand. core/OfflineManager.ts sends explicit cache-all work to the service worker and waits for its response. Settings exposes Save all games offline. main.ts registers the web worker only in production, outside native Capacitor. Android uses packaged assets. After source changes, rebuild the web output; after Android changes to web content, sync before compiling the native app.

scripts/bundle-report.mjs discovers registered folders/chunks and enforces lazy-loading/payload constraints. scripts/check-module.mjs validates automatic discovery with the example in a disposable checkout. measure-performance and measure-expansion are measurement tools, not gameplay code. verify-math-bank.py is a SymPy-based generator and symbolic verifier for 100 static questions. It WRITES src/games/math-quiz/questions/university.json and docs/math-validation.json; it is not a read-only audit of an edited bank and does not validate all procedural families. Run it only when intentionally regenerating the bank. Existing math-quiz unit tests provide separate validation. Their internal blocks are indexed in the atlas.

Android folder by folder

android/app/src/main/java/com/pocketrivals/games/MainActivity.java is the Capacitor activity. AndroidManifest.xml defines activity/orientation/theme and app entry. res/layout/activity_main.xml contains the native WebView wrapper layout. res/values holds strings, colors and theme/splash styles. res/xml holds provider paths and generated Capacitor/Cordova configuration. res/drawable and drawable-v24 contain vector branding, icon background/foreground and base splash resources. drawable-land-* and drawable-port-* hold density/orientation splash variants. mipmap-* contains raster launcher assets by density; mipmap-anydpi-v24/v26 contains adaptive icon resource definitions.

android/app/build.gradle configures application identity, SDK settings, build types, packaging and environment-based signing. proguard-rules.pro holds optional shrinking rules. app/capacitor.build.gradle and capacitor.settings.gradle connect generated plugin modules. android/build.gradle, settings.gradle, variables.gradle and gradle.properties configure Gradle/plugins/SDK values. gradle/wrapper holds the pinned wrapper properties and executable JAR; gradlew/gradlew.bat start it on Unix/Windows. capacitor-cordova-android-plugins is a generated compatibility module. ExampleUnitTest.java and ExampleInstrumentedTest.java are native template tests, not evidence of Android gameplay validation.

android/app/src/main/assets/public is a copied web build. Its hashed JS/CSS/font files, manifest, index, sw and compatibility stubs come from cap sync. Editing these copies will be overwritten and can leave Android out of sync with source. Rebuild/sync instead. Existing copied assets can lag behind src until synced; their presence alone does not prove the latest collection is installed on a device. local.properties, if present, is machine-local SDK configuration and deliberately excluded from the handbook inventory. No secrets or signing values are reproduced.

Static and generated artwork

public/icon.svg is the web app mark. Every game's assets/preview.svg is its card thumbnail. Some configs embed variant SVG strings or call a lightweight local map preview; changing a thumbnail does not alter live gameplay. Most live art is original procedural Phaser Graphics in Art/Renderer files, and some DOM/SVG helpers generate previews from game data. Fonts come from bundled @fontsource packages; npm/Vite emits hashed local font assets. KaTeX styles/fonts are bundled for Math Quiz. There is no remote asset dependency in normal play.

The docs PNGs are historical screenshots and visual evidence. Their filenames identify the game/state reviewed; they are not authoritative physics or current asset source. docs JSON reports record measured snapshots/audits, not live telemetry. docs/licenses contains bundled third-party license copies. docs/THIRD_PARTY_NOTICES.md and PRIVACY.md describe dependencies and local data behavior.

10. Testing, troubleshooting and safe modification

A practical edit workflow

1. Find the active file in this handbook and read its local AGENTS.md plus the shared contract.

2. Search for the symbol, then inspect all reads/writes and callers. Prefer a named constant over an unrelated UI literal.

3. Change one behavior at a time. Keep units explicit and update metadata/help only when actual behavior changes.

4. Run the game's targeted unit test(s). Run npm run build for TypeScript/import/bundle errors.

5. Exercise the real UI input path, pause/resume, restart and exit. For physics/input changes, run the related browser/framerate tests.

6. For metadata/module discovery changes, run test:modules and test:bundle. For Android delivery, rebuild/sync and validate on an actual device.

7. Update this guide and re-extract the source index when behavior or line locations change.

This documentation request did not require changing game code. The validation ledger at the end reports exactly which checks were run for this edition; it does not imply browser/native checks that were not executed.

Find the right test

Area	Main coverage to start with
Core saves/tokens, Pong/Hockey/boards	engine, phase2, phase3, app, precision/keyboard specs
Seeded AI identity and bounds	ai-refinement tests and browser spec
Dots/board sizes/captures	dots-and-boxes and phase suites
Hammer	hammer-hit, hammer-presentation, hammer-presentation-related browser checks
Light/Fruit/Lumber/Math	matching unit files, fruit-timer-effects, light-lumber-polish, math-quiz-visual
Water/Timer/rotation	water-guess, perfect-timer, turn-view, precision-games
Gravity/Birds/Spike	matching unit files, gravity-variation, spike-controls, premium-motion, expansion-framerate and matching browser specs
Active golf	ultimate-golf, golf-routes, golf-refinement, golf-temple-ai, browser ultimate-golf/golf-temple-ai
Optional full golf AI audit	fixtures/golf-ai-audit.test.ts via fixtures/golf-audit.config.ts
Future modules	matching unit files; specs in tests/future are excluded from normal browser discovery
Offline/lazy/discovery	offline browser spec, test:bundle and test:modules scripts

The atlas lists actual describe/test titles and line locations for every test file. Reading those titles identifies the assertions' intended contract; passing them does not establish every possible runtime path. fixtures/golf-solutions.ts contains legal Easy-mode shots for all four active courses. fixtures/golf-route-search.ts explains an optional search process and is not shipped gameplay. The expensive AI audit has a separate config and is not part of ordinary npm test.

Common symptoms and where to investigate

Symptom	Start here	Why
New game card absent	Registry globs, folder config, unique ID	Only games/* and arcade/* are discovered
Card opens old behavior	Active index.ts, rebuilt dist, Android sync	A retained legacy implementation may not be imported
Score appears twice	Controller reported/ended guard, GameHost.finish, attempt token	Terminal reporting must be idempotent
Timer advances during pause	TimerClock/GameHost resume reference, module update guard	Raw wall time can include background duration
Motion changes with frame rate	FixedStep usage and interpolation split	Simulation must not be frame-count based
AI hits invisible information	Controller present/presentSignal order and AI visibleAt	Model readiness must follow presentation
Ghost input after resume	InputManager.clear and module-specific held sets	Pointer/key/gamepad state must be canceled
Worker affects another match	WorkerTask.cancel and controller generation guard	Late asynchronous replies must be rejected
Art and collision disagree	Shared geometry/constants and transforms	Draw code may have independent literals
Golf aiming shifts on focus	project/unproject and camera freeze during drag	All coordinates must use one transform
Android seems outdated	cap sync copied public assets	Native assets are snapshots, not live src
Works online, worker fails offline	sw collection manifest/cache-all and worker chunks	Workers are separate lazy assets
Save corrupt/missing	Module restore validator and StorageManager persistent flag	Storage may be invalid or unavailable
Math repeated/too small	History/Bank and MathUI/quiz.css	Data scheduling and DOM fit are separate

Change requests mapped to owners

What you want to change	Where to start
App colors and default canvas	ThemeManager plus app.css and hardcoded local art
Menu card name/order/instructions	Game config.ts
Thumbnail	assets/preview.svg or config variant preview helper
Enable/disable tournament entry	config tournament/category and supportedModes
AI strength	The selected game's AI settings/search budgets
AI personality variety	MatchRandom identity and local comparable-choice policy
Win target	Local Rules/world plus any shared clock/controller hardcodes
Movement speed/gravity	Local Physics/world constants and legal controller
Collision shape	Canonical geometry plus matching renderer and forecasts
Animation speed	View/Art/controller age expressions, unless shared with rule cooldown
Keyboard mapping	Game initialize/setupInputs; host InputManager only for general mechanics
Shared sound effect	AudioManager.effects and Sound union
Per-game feedback event	The controller ctx.audio.play call at the legal event
Save schema	The save and restore pair, version and tests
Add math question/topic	questions JSON/generator, Question schema and bank tests
Add golf obstacle/course	Course data/types, dynamics/collision/renderer/planner as required
Reactivate future game	Move folder and restore its browser spec, then rebuild/check discovery
Add entirely new game	Copy tap-duel example, implement MiniGame, supply metadata/assets/tests
Change native app identity	capacitor.config.ts, native Gradle/manifest/resources consistently
Offline caching policy	scripts/offline-plugin.ts and OfflineManager

Constraints that protect correctness

Keep game rules inside their modules. Do not import one game into another or eager metadata into heavy physics/AI. Preserve legal human/AI actions and bounded search/physics. Preserve pointer identity and clear state on cancel/pause. Keep match deadlines on foreground monotonic time. Reject nonfinite input and validate restored data. Separate duel scores, Arcade bests, casual golf and competitive golf. Preserve stable ping-pong and mini-golf IDs. Own and dispose graphics, DOM, masks, workers and extra listeners. Use the existing Phaser loop. These are practical invariants needed for correct edits.

11. How to use the complete file atlas

The following reference follows the actual folders. For each maintained code file it states responsibility, dependencies, named blocks and source lines. Function entries explain the role, show selected decision guards, state writes, calls and return expressions extracted from the current AST. Those observations describe direct code; delegated helpers and callbacks may perform additional work. A displayed guard is an actual branch condition, not a claim that every branch rejects input. No arbitrary threshold is recommended as safe solely because it appears in the table.

State/constant tables distinguish module data from class initial state. A field initialized to zero can be a runtime counter, not a tuning knob. AI local settings and literals inside functions remain in the companion complete source. Long SVG/data literals are summarized in the PDF and reproduced in that source reference. CSS selectors and test titles have separate maps. The maintained-file inventory also covers JSON, XML, Gradle, shell, SVG, Markdown, licenses and image resources. Generated/native copied bundles are grouped with regeneration instructions.

12. Visual maps of execution and motion

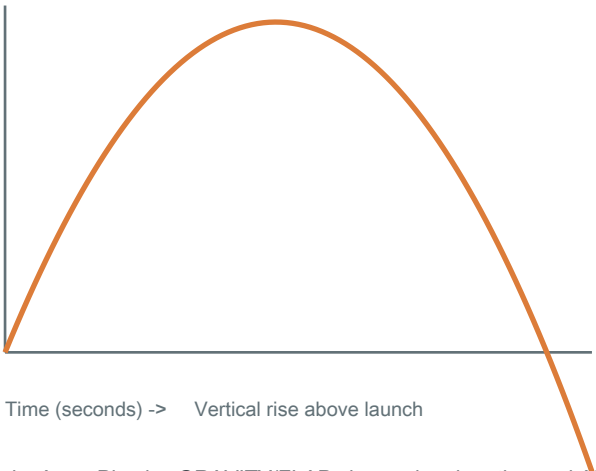
These are explanatory diagrams tied to the named source owners. Curves use the listed constants in free space; collisions and changing input alter the real path.



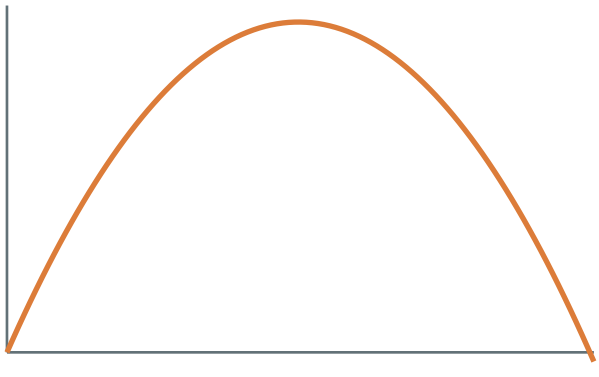
One Phaser instance and render loop. Workers return decisions; they do not run the UI.

Free-flight examples and what to change

Bird flap: v=-195, g=650



Hammer: score 100, g=2200



Birds: ArenaPhysics.GRAVITY/FLAP change legal motion and AI forecasts. Hammer: StrikePhysics changes feedback height after Rules has already assigned a timing score. The analytic curves above omit engine contacts and integration error.

Coordinate conversion for a golf drag

Stage	Source owner	Meaning
Screen point	InputManager / UltimateGolfGame	An owned touch or mouse gesture in the responsive canvas
World point	GolfView.unproject	Undo camera center, uniform scale and portrait rotation
Shot velocity	shotVector	(start-end)*3.125, magnitude limited to 650
Legal simulation	GolfPhysics.shoot / tick	Validate stopped state, consume one stroke, simulate at 240Hz
Display point	GolfView.project / GolfRenderer	Apply the matching transform to physics state

13. Complete source file and code-block atlas

The section order mirrors the project. Each file header gives its source fingerprint and status. A short exact block is reproduced when useful; longer blocks have extracted calls, state writes, return/guard expressions and a direct-statement map. Full unabridged sources are in Pocket_Rivals_Source_Reference.md.

src

src/main.ts

ACTIVE SOURCE / tool or application | 27 lines | SHA-256 8dd74f5b71c3fdc5...

Loads local fonts/CSS, creates GameManager, exposes development-only snapshots/HMR disposal and registers the production web service worker.

Imports: @fontsource/fredoka/latin-600.css, @fontsource/fredoka/latin-700.css, @fontsource/nunito-sans/latin-700.css, @fontsource/nunito-sans/latin-900.css, ./styles/app.css, @capacitor/core, ./core/GameManager

Module constants and data

L9-9 app = new GameManager()

Top-level block L8-8

```
await document.fonts.ready;
```

Top-level block L10-15

```
if (import.meta.env.DEV) {
  Object.defineProperty(window, "__POCKET__", {
    configurable: true,
    get: () => app.snapshot(),
  });
}
```

Top-level block L16-16

```
if (import.meta.hot) import.meta.hot.dispose(() => app.dispose());
```

Top-level block L18-26

```
if (
  import.meta.env.PROD &&
  !Capacitor.isNativePlatform() &&
  "serviceWorker" in navigator
) {
  void navigator.serviceWorker
    .register("./sw.js")
    .catch((error) => console.warn("Offline cache unavailable", error));
}
```

src/core

src/core/ArcadeScoreManager.ts

ACTIVE SOURCE / tool or application | 26 lines | SHA-256 8b9a57bc45797128...

Validates and stores separate per-game solo high scores.

Imports: ./StorageManager

ArcadeScoreManager [L2-25]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ArcadeScoreManager.constructor [L3-3]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private storage: StorageManager)

```
constructor(private storage: StorageManager) {}
```

ArcadeScoreManager.best [L4-13]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: best(id: string)

Calls / delegates: this.storage.readRecord

Return at L10: typeof n === "number" && Number.isFinite(n) && n >= 0 ? Math.floor(n) : 0

ArcadeScoreManager.record [L14-24]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: record(id: string, score: number)

Direct writes: data[id]

Calls / delegates: this.storage.readRecord, this.best, this.storage.writeRecord

Return at L23: best

src/core/AudioManager.ts

ACTIVE SOURCE / tool or application | 150 lines | SHA-256 bc1c825de1c05b55...

Synthesizes bounded short Web Audio effects, respecting persisted mute/volume and lifecycle.

Imports: ./StorageManager

State, resources and initial values

Line	Field / declaration
64	private ctx?: AudioContext;
65	private master?: GainNode;
66	private voices = new Set<OscillatorNode>();
67	private last = new Map<Sound, number>();

Module constants and data

L26-62 effects = { slice: { notes: [2400, 1100], wave: "sawtooth", duration: 0.045, gain: 0.035, }, thud: { notes: [110, 65], wave: "triangle", duration: 0.075, gain: 0.13 }, clockTick: { notes: [1550], wave: "triangle", duration: 0.018, gain: 0.045 }, clockStop: { notes: [820, 520], wave: "triangle", duration: 0.04, gain: 0.09, }, tap: { notes: [480], duration: 0.055 }, hit: { notes: [260], duration: 0.055 }, goal: { notes: [440, 660] }, win: { notes: [392, 494, 587, 784] }, draw: { notes: [392, 330] }, start: { notes: [330, 660], duration: 0.09 }, countdown: { notes: [620], duration: 0.05 }, pause: { notes: [400, 300], duration: 0.045 }, resume: { notes: [300, 400], duration: 0.045 }, fail: { notes: [180, 110], wave: "triangle", duration: 0.12 }, soft: { notes: [170], duration: 0.055, gain: 0.06 }, metal: { notes: [890, 1270], wave: "triangle", duration: 0.07 }, wood: { notes: [140, 90], wave: "triangle", duration: 0.045 }, shoot: { notes: [720], wave: "triangle", duration: 0.06 }, fall: { notes: [330], duration: 0.13 }, slide: { notes: [140], duration: 0.08, gain: 0.035 }, capture: { notes: [500, 750], duration: 0.065 }, perfect: { notes: [660, 880, 1100], duration: 0.09 }, }

Sound [L2-24]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Sound = | "tap" | "hit" | "goal" | "win" | "draw" | "start" | "countdown" | "pause" | "resume" | "fail" | "soft" | "metal" | "wood" | "shoot" | "fall" | "slide" | "capture" | "perfect" | "slice" | "thud" | "clockTick" | "clockStop";
```

AudioManager [L63-149]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

AudioManager.constructor [L68-68]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private storage: StorageManager)

```
constructor(private storage: StorageManager) {}
```

AudioManager.unlock [L69-82]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: unlock()

Direct writes: this.ctx, this.master

Calls / delegates: AudioContext, this.ctx.createGain, this.master.connect, this.syncSettings, this.ctx.resume().catch, this.ctx.resume

Decision branches: L71: !this.ctx; L77: this.ctx.state === "suspended"

AudioManager.syncSettings [L83-89]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: syncSettings()

Calls / delegates: this.master.gain.setValueAtTime

Decision branches: L84: this.master && this.ctx

AudioManager.play [L90-129]

Apply a requested legal move or start a selected application module, depending on the owner.

Signature: play(kind: Sound, strength = 1)

Direct writes: oscillator.type, oscillator.onended

Calls / delegates: this.unlock, this.last.get, this.last.set, e.notes.entries, ctx.createOscillator, ctx.createGain, this.voices.add, oscillator.frequency.setValueAtTime, oscillator.frequency.exponentialRampToValueAtTime, gain.gain.setValueAtTime, gain.gain.linearRampToValueAtTime, gain.gain.exponentialRampToValueAtTime, oscillator.connect, gain.connect, oscillator.disconnect, gain.disconnect, this.voices.delete, oscillator.start, oscillator.stop

Decision branches: L91: !this.storage.data.sound || this.storage.data.volume <= 0; L94: !ctx || ctx.state !== "running" || !this.master; L96: t - (this.last.get(kind) ?? -1) < (kind === "slide" ? 0.12 : 0.04); L101: this.voices.size >= 10

Execution order (direct statements; nested callbacks stay within their owner):

L91-91 IfStatement: if (!this.storage.data.sound || this.storage.data.volume <= 0) return;

L92-92 ExpressionStatement: this.unlock();

L93-93 FirstStatement: const ctx = this.ctx;

L94-94 IfStatement: if (!ctx || ctx.state !== "running" || !this.master) return;

L95-95 FirstStatement: const t = ctx.currentTime;

L96-97 IfStatement: if (t - (this.last.get(kind) ?? -1) < (kind === "slide" ? 0.12 : 0.04)) return;

L98-98 ExpressionStatement: this.last.set(kind, t);

L99-99 FirstStatement: const e = effects[kind];

L100-128 ForOfStatement: for (const [i, hz] of e.notes.entries()) { if (this.voices.size >= 10) break;

AudioManager.stop [L130-137]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private stop()

Calls / delegates: this.voices.forEach, v.stop, this.voices.clear

AudioManager.suspend [L138-141]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: suspend()

Calls / delegates: this.stop, this.ctx.suspend().catch, this.ctx.suspend

Decision branches: L140: this.ctx?.state === "running"

AudioManager.dispose [L142-145]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.stop, this.ctx?.close().catch, this.ctx?.close

AudioManager.activeVoices [L146-148]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get activeVoices()

Return at L147: this.voices.size

```
get activeVoices() { return this.voices.size; }
```

src/core/Diagnostics.ts

ACTIVE SOURCE / tool or application | 64 lines | SHA-256 f4e56f184e956310...

Collects development-only FPS/frame timing, loaded-module, scene/texture and AI computation metrics.

State, resources and initial values

Line	Field / declaration
3	private panel?: HTMLElement;
4	private frames = 0;
5	private seconds = 0;
6	private frameMs = 0;
7	private fps = 0;
8	private initializations: Record<string, number> = {};
9	private modules = new Set<string>();
10	private ai = 0;
11	private aiMax = 0;

Diagnostics [L2-63]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

Diagnostics.constructor [L12-21]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor()

Direct writes: this.panel, this.panel.className

Calls / delegates: new URLSearchParams(location.search).has, URLSearchParams, document.createElement, document.body.append

Decision branches: L13: import.meta.env.DEV && new URLSearchParams(location.search).has("diagnostics")

Diagnostics.loaded [L22-24]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: loaded(id: string)

Calls / delegates: this.modules.add

Diagnostics.initialized [L25-27]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: initialized(id: string, ms: number)

Direct writes: this.initializations[id]

Diagnostics.computation [L28-31]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: computation(ms: number)

Direct writes: this.ai, this.aiMax

Diagnostics.frame [L32-49]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: frame(dt: number, scenes: number, assets: number)

Direct writes: this.seconds, this.frameMs, this.fps, this.panel.textContent, this.frames

Calls / delegates: this.frameMs.toFixed, (memory.usedJSHeapSize / 1048576).toFixed, this.ai.toFixed, this.aiMax.toFixed, [...this.modules].join, t.toFixed

Decision branches: L33: !this.panel; L37: this.seconds < 0.5

Diagnostics.snapshot [L50-59]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Return at L51: { fps: this.fps, frameMs: this.frameMs, modules: [...this.modules], initializations: { ...this.initializations }, aiMs: this.ai, aiMaxMs: this.aiMax, }

```
snapshot() { return { fps: this.fps, frameMs: this.frameMs, modules: [...this.modules], initializations: {
...this.initializations }, aiMs: this.ai, aiMaxMs: this.aiMax, }; }
```

Diagnostics.dispose [L60-62]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.panel?.remove

src/core/GameLoader.ts

ACTIVE SOURCE / tool or application | 26 lines | SHA-256 1ba41d56887a84a2...

Resolves a registered module path and lazily loads its factory or optional tutorial.

Imports: ../shared/contracts/GameModule, ./GameRegistry

Module constants and data

L7-10 modules = import.meta.glob<GameExports>(["../games/*/index.ts", "../arcade/*/index.ts",])

exportsFor [L11-15]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: async function exportsFor(id: string)

Calls / delegates: modulePath, Error, load

Return at L14: load()

Decision branches: L13: !load

loadGame [L16-18]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export async function loadGame(id: string, options: MatchOptions)

Calls / delegates: (await exportsFor(id)).default, exportsFor

Return at L17: (await exportsFor(id)).default(options)

loadTutorial [L19-25]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export async function loadTutorial(id: string, variant: string, ui: TutorialUI,)

Calls / delegates: (await exportsFor(id)).tutorial, exportsFor

src/core/GameManager.ts

ACTIVE SOURCE / tool or application | 415 lines | SHA-256 41d72b24f9b424e4...

Coordinates the one Phaser instance, screen navigation, run guards, native lifecycle, pause, quick-match saves and duel/Arcade result routes.

Imports: ./Diagnostics, ./ArcadeScoreManager, phaser, @capacitor/app, @capacitor/core, @capacitor/splash-screen, ./StorageManager, ./ScoreManager, ./AudioManager, ./TournamentManager, ../scenes/GameHost, ../scenes/MainMenu, ../scenes/Settings, ../scenes/GameSetup, ../tournament/TournamentScene, ../shared/components/Modal, ../scenes/Results, ./GameRegistry, ../shared/contracts/GameModule

State, resources and initial values

Line	Field / declaration
27	readonly diagnostics = new Diagnostics();
28	readonly arcade: ArcadeScoreManager;
29	readonly storage: StorageManager;
30	readonly scores: ScoreManager;
31	readonly audio: AudioManager;
32	readonly tournament: TournamentManager;
33	readonly modal = new Modal();
34	readonly renderScale = Math.min(window.devicePixelRatio || 1, 2);
35	readonly host: GameHost;
36	readonly game: Phaser.Game;
37	private transition = document.createElement("div");
38	private menu: MainMenu;
39	private tournamentView: TournamentScene;
40	private active: MatchSpec | null = null;
41	private token: string | null = null;
42	private run = 0;
43	private state: "menu" | "playing" | "paused" | "results" | "loading" = "menu";
44	private menuRoot = document.querySelector<HTMLElement>("#menu")!;
45	private disposers: (() => void)[] = [];

Module constants and data

L25-25 MATCH_KEY = "pocket-rivals.match.v1"

GameManager [L26-414]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GameManager.constructor [L46-117]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor()

Direct writes: storage, this.storage, this.scores, this.arcade, this.audio, this.tournament, this.tournamentView, this.host, this.game, this.transition.className, this.transition.innerHTML, this.transition.hidden, this.menu

Calls / delegates: StorageManager, ScoreManager, ArcadeScoreManager, AudioManager, TournamentManager, TournamentScene, GameHost, Phaser.Game, game.scene.add, game.loop.sleep, ResizeObserver, this.resizeGame, this.host.controls?.clear, resize.observe, document.querySelector, this.disposers.push, resize.disconnect, document.querySelector("#app")!.append, MainMenu, this.menu.render, e.preventDefault, this.back, this.modal.trapFocus, this.background, window.addEventListener, document.addEventListener, window.removeEventListener, document.removeEventListener, Capacitor.isNativePlatform, this.nativeLifecycle

Decision branches: L78: this.state === "menu"; L83: this.game.isBooted; L98: e.key === "Escape"; L105: document.hidden; L116: Capacitor.isNativePlatform()

Execution order (direct statements; nested callbacks stay within their owner):

L47-47 FirstStatement: let storage: Storage | undefined;

L48-50 TryStatement: try { storage = localStorage;

L51-51 ExpressionStatement: this.storage = new StorageManager(storage);

L52-52 ExpressionStatement: this.scores = new ScoreManager(this.storage);

L53-53 ExpressionStatement: this.arcade = new ArcadeScoreManager(this.storage);

L54-54 ExpressionStatement: this.audio = new AudioManager(this.storage);

L55-55 ExpressionStatement: this.tournament = new TournamentManager(this.storage);

L56-56 ExpressionStatement: this.tournamentView = new TournamentScene(this);

L57-57 ExpressionStatement: this.host = new GameHost(this);

L58-81 ExpressionStatement: this.game = new Phaser.Game({ type: Phaser.AUTO,

L82-87 FirstStatement: const resize = new ResizeObserver(() => { if (this.game.isBooted) {

L88-88 ExpressionStatement: resize.observe(document.querySelector("#game-stage")!);

L89-89 ExpressionStatement: this.disposers.push(() => resize.disconnect());

L90-90 ExpressionStatement: this.transition.className = "game-transition";

L91-92 ExpressionStatement: this.transition.innerHTML = 'pr.One little moment…';

L93-93 ExpressionStatement: this.transition.hidden = true;

L94-94 ExpressionStatement: document.querySelector("#app")!.append(this.transition);

L95-95 ExpressionStatement: this.menu = new MainMenu(this, this.menuRoot);

L96-96 ExpressionStatement: this.menu.render();

L97-103 FirstStatement: const key = (e: KeyboardEvent) => { if (e.key === "Escape") {

L104-106 FirstStatement: const visibility = () => { if (document.hidden) this.background();

L107-107 FirstStatement: const blur = () => this.background();

L108-108 ExpressionStatement: window.addEventListener("keydown", key);

L109-109 ExpressionStatement: document.addEventListener("visibilitychange", visibility);

L110-110 ExpressionStatement: window.addEventListener("blur", blur);

L111-115 ExpressionStatement: this.disposers.push(() => window.removeEventListener("keydown", key),

L116-116 IfStatement: if (Capacitor.isNativePlatform()) void this.nativeLifecycle();

GameManager.constructor.key [L97-103]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: key = (e: KeyboardEvent) =>

Calls / delegates: e.preventDefault, this.back, this.modal.trapFocus

Decision branches: L98: e.key === "Escape"

GameManager.constructor.visibility [L104-106]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: visibility = () =>

Calls / delegates: this.background

Decision branches: L105: document.hidden

GameManager.constructor.blur [L107-107]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: blur = () =>

```
blur = () => this.background()
```

GameManager.nativeLifecycle [L118-135]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private async nativeLifecycle()

Calls / delegates: App.addListener, this.modal.show, this.modal.close, App.exitApp, this.back, this.background, this.disposers.push, back.remove, state.remove, SplashScreen.hide

Decision branches: L120: this.state === "menu" && !this.modal.isOpen; L128: !isActive

GameManager.selectGame [L136-148]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: selectGame(id: string)

Calls / delegates: getGame, this.play, this.audio.play, new GameSetup(this.modal, (options) => void this.play(id, options)).open, GameSetup

Decision branches: L137: getGame(id).category === "arcade" || getGame(id).presentation === "custom"

GameManager.play [L149-176]

Apply a requested legal move or start a selected application module, depending on the owner.

Signature: async play(id: string, options: MatchOptions = defaultOptions(), restoredState?: unknown, token: string | null = null,)

Direct writes: this.active, this.token, this.state, this.transition.hidden

Calls / delegates: getGame, meta.supportedModes.includes, Error, this.audio.play, this.modal.close, structuredClone, this.scores.beginRound, this.storage.writeRecord, this.game.loop.wake, this.resizeGame, this.game.scene.isActive, this.game.scene.stop, this.game.scene.start

Decision branches: L156: options.mode === "ai" && !meta.supportedModes.includes("single-player"); L165: !token && !restoredState; L169: this.game.scene.isActive("GameHost")

GameManager.resizeGame [L177-189]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private resizeGame()

Calls / delegates: getGame, document.querySelector("#game-stage")!.getBoundingClientRect, document.querySelector, this.game.scale.setGameSize, this.game.scale.getParentBounds, this.game.scale.refresh, this.host.module?.resize

GameManager.onReady [L190-199]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: onReady(run: number)

Direct writes: this.menuRoot.hidden, this.transition.hidden, this.state

Calls / delegates: document.querySelector("#app")!.classList.add, document.querySelector, this.background

Decision branches: L191: run !== this.run; L195: this.state === "loading"; L197: document.hidden

GameManager.checkpoint [L200-214]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: checkpoint(run: number, data: unknown)

Calls / delegates: ["playing", "loading"].includes, this.tournament.checkpoint, this.storage.writeRecord

Decision branches: L201: run !== this.run || !this.active || !["playing", "loading"].includes(this.state); L207: this.token

GameManager.savedMatch [L215-234]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: savedMatch()

Calls / delegates: this.storage.readRecord, ["local", "ai"].includes, ["medium", "hard", "impossible"].includes, getGame

Return at L230: d

Decision branches: L222: d?.version !== 1 || !d.spec || ![“local”, “ai”].includes(d.spec.options.mode) || ![“medium”, “hard”, “impossible”].includes(d.spec.options.difficulty); L229: getGame(d.spec.gameId).category === "arcade"

GameManager.resumeSaved [L235-238]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: resumeSaved()

Calls / delegates: this.savedMatch, this.play

Decision branches: L237: d

GameManager.finish [L239-266]

Resolve/report the owned terminal attempt/result using current guards.

Signature: finish(result: GameResult, run: number)

Direct writes: this.state

Calls / delegates: getGame, this.tournament.finish, this.scores.record, this.storage.writeRecord, this.audio.play, this.tournamentView.show, this.modal.show, resultsContent, this.play, this.goMenu

Decision branches: L240: run !== this.run || this.state === "results" || !this.active || getGame(this.active.gameId).category === "arcade"; L247: this.token && !this.tournament.finish(this.token, result); L248: !this.scores.record(result.winner); L249: !this.token; L252: this.token

GameManager.finishArcade [L267-287]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: finishArcade(score: number, detail: string, run: number)

Direct writes: this.state

Calls / delegates: getGame, this.arcade.record, this.audio.play, this.modal.show, this.play, this.goMenu

Decision branches: L268: run !== this.run || this.state === "results" || (getGame(this.active!.gameId).category !== "arcade" && getGame(this.active!.gameId).presentation !== "custom")

GameManager.pause [L288-303]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause(reason = "pause")

Direct writes: this.state

Calls / delegates: this.host.pauseGame, this.audio.play, this.modal.show, getGame, this.resume, this.restart, this.goMenu

Decision branches: L289: this.state !== "playing" && this.state !== "loading"

GameManager.restart [L304-316]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private restart()

Calls / delegates: this.tournament.restartRound, this.play

Decision branches: L305: this.token; L307: round; L314: this.active

GameManager.resume [L317-322]

Re-enable the component with a fresh/clean timing or input state.

Signature: private resume()

Direct writes: this.state

Calls / delegates: this.modal.close, this.audio.unlock, this.host.resumeGame

GameManager.background [L323-327]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private background()

Calls / delegates: this.pause, this.audio.suspend, this.host.controls?.clear

Decision branches: L324: this.state === "playing" || this.state === "loading"

GameManager.back [L328-333]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: back()

Calls / delegates: this.pause, this.modal.back, this.goMenu

Decision branches: L329: this.state === "playing" || this.state === "loading"; L331: this.modal.isOpen

GameManager.goMenu [L334-346]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: goMenu()

Direct writes: this.transition.hidden, this.active, this.token, this.state, this.menuRoot.hidden

Calls / delegates: this.modal.close, this.game.scene.stop, this.game.loop.sleep, document.querySelector("#app")!.classList.remove, document.querySelector, this.menu.render

GameManager.refreshMenu [L347-349]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: refreshMenu()

Calls / delegates: this.menu.render

GameManager.openTournament [L350-353]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: openTournament()

Calls / delegates: this.showTournament, this.tournamentView.setup

Decision branches: L351: this.tournament.state

GameManager.newTournament [L354-356]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: newTournament()

Calls / delegates: this.tournamentView.setup

GameManager.showTournament [L357-360]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: showTournament()

Calls / delegates: this.goMenu, this.tournamentView.show

GameManager.startTournamentRound [L361-377]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: startTournamentRound()

Calls / delegates: this.tournament.startRound, getGame, Error, this.play, this.showError

Decision branches: L363: !round; L366: meta.category === "arcade" || meta.tournament === false

GameManager.toggleSound [L378-383]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: toggleSound()

Direct writes: this.storage.data.sound

Calls / delegates: this.audio.syncSettings, this.storage.save, this.audio.play

GameManager.settings [L384-386]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: settings()

Calls / delegates: new Settings(this).show, Settings

GameManager.showError [L387-397]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: showError(error: unknown)

Direct writes: this.transition.hidden

Calls / delegates: console.error, this.host.pauseGame, this.modal.show, this.goMenu

GameManager.snapshot [L398-406]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.diagnostics.snapshot, structuredClone, this.host.snapshot

Return at L399: { state: this.state, diagnostics: this.diagnostics.snapshot(), saved: structuredClone(this.storage.data), tournament: structuredClone(this.tournament.state), scene: this.host.snapshot(), }

GameManager.dispose [L407-413]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.disposers.forEach, fn, this.audio.dispose, this.diagnostics.dispose, this.transition.remove, this.game.destroy

src/core/GameRegistry.ts

ACTIVE SOURCE / tool or application | 33 lines | SHA-256 cab8cad0bf77ee75...

Eagerly discovers lightweight game metadata, validates stable IDs, sorts cards and filters tournament eligibility.

Imports: ../shared/contracts/GameModule

Module constants and data

L2-5 configs = import.meta.glob<{ default: GameMetadata }>(["../games/*/config.ts", "../arcade/*/config.ts"], { eager: true },)

L6-9 entries = Object.entries(configs).map(([path, m]) => ({ path: path.replace("/config.ts", "/index.ts"), metadata: m.default, }))

L10-12 games = entries .map((e) => e.metadata) .sort((a, b) => a.order - b.order)

L13-13 ids = new Set<string>()

L30-32 tournamentGames = games.filter((g) => g.category !== "arcade" && g.tournament !== false,)

getGame [L19-23]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function getGame(id: string)

Calls / delegates: games.find, Error

Return at L22: game

Decision branches: L21: !game

modulePath [L24-28]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function modulePath(id: string)

Calls / delegates: entries.find, Error

Return at L27: entry.path

Decision branches: L26: !entry

Top-level block L14-18

```
for (const game of games) {
  if (!/^[a-z0-9-]+$/.test(game.id) || ids.has(game.id))
    throw Error(`Invalid/duplicate game ID: ${game.id}`);
  ids.add(game.id);
}
```

src/core/InputManager.ts

ACTIVE SOURCE / tool or application | 176 lines | SHA-256 4bae021c280b1857...

Owns scene-local pointer/key subscriptions, per-player pointer identity, cancellation, pause and cleanup.

Imports: phaser, ./ThemeManager, ../shared/contracts/GameModule

State, resources and initial values

Line	Field / declaration
12	readonly players: [TouchPoint, TouchPoint] = [{ x: 210, y: 600, down: false, id: null }, { x: 210, y: 160, down: false, id: null },];
16	private cleanups: (() => void)[] = [];
17	private enabled = true;
18	private clearHandlers: (() => void)[] = [];

TouchPoint [L4-9]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface TouchPoint { x: number; y: number; down: boolean; id: number | null; }
```

InputManager [L11-175]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

InputManager.constructor [L19-19]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private scene: Phaser.Scene)

```
constructor(private scene: Phaser.Scene) {}
```

InputManager.dragHalves [L20-65]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: dragHalves()

Direct writes: player.id, player.down, player.x, player.y

Calls / delegates: this.scene.cameras.main.getWorldPoint, this.players.find, this.on, this.clear, this.scene.game.canvas.addEventListener, this.cleanups.push, this.scene.game.canvas.removeEventListener

Decision branches: L23: !this.enabled || pos.y < FIELD.top || pos.y > FIELD.bottom || pos.x < FIELD.left || pos.x > FIELD.right; L32: player.id !== null; L39: !this.enabled; L42: player; L49: player

Execution order (direct statements; nested callbacks stay within their owner):

L21-37 FirstStatement: const down = (p: Phaser.Input.Pointer) => { const pos = this.scene.cameras.main.getWorldPoint(p.x, p.y);

L38-46 FirstStatement: const move = (p: Phaser.Input.Pointer) => { if (!this.enabled) return;

L47-53 FirstStatement: const up = (p: Phaser.Input.Pointer) => { const player = this.players.find((v) => v.id === p.id);

L54-54 ExpressionStatement: this.on("pointerdown", down);

L55-55 ExpressionStatement: this.on("pointermove", move);

L56-56 ExpressionStatement: this.on("pointerup", up);

L57-57 ExpressionStatement: this.on("pointerupoutside", up);

L58-58 FirstStatement: const cancel = () => this.clear();

L59-59 ExpressionStatement: this.scene.game.canvas.addEventListener("touchcancel", cancel);

L60-60 ExpressionStatement: this.scene.game.canvas.addEventListener("pointercancel", cancel);

L61-64 ExpressionStatement: this.cleanups.push(() => { this.scene.game.canvas.removeEventListener("touchcancel", cancel);

InputManager.dragHalves.down [L21-37]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: down = (p: Phaser.Input.Pointer) =>

Direct writes: player.id, player.down, player.x, player.y

Calls / delegates: this.scene.cameras.main.getWorldPoint

Decision branches: L23: !this.enabled || pos.y < FIELD.top || pos.y > FIELD.bottom || pos.x < FIELD.left || pos.x > FIELD.right; L32: player.id !== null

InputManager.dragHalves.move [L38-46]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: move = (p: Phaser.Input.Pointer) =>

Direct writes: player.x, player.y

Calls / delegates: this.scene.cameras.main.getWorldPoint, this.players.find

Decision branches: L39: !this.enabled; L42: player

InputManager.dragHalves.up [L47-53]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: up = (p: Phaser.Input.Pointer) =>

Direct writes: player.down, player.id

Calls / delegates: this.players.find

Decision branches: L49: player

InputManager.dragHalves.cancel [L58-58]

Invalidate/cancel a gesture or asynchronous computation and clear pending state.

Signature: cancel = () =>

```
cancel = () => this.clear()
```

InputManager.tap [L66-73]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: tap(fn: (x: number, y: number, player: Player) => void)

Calls / delegates: this.on, this.scene.cameras.main.getWorldPoint, fn

Decision branches: L68: this.enabled

InputManager.drag [L74-115]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: drag(start: (x: number, y: number) => boolean, move: (x: number, y: number) => void, end: (x: number, y: number) => void, cancel: () => void = () => {},)

Direct writes: id

Calls / delegates: this.scene.cameras.main.getWorldPoint, this.on, point, start, move, end, cancel, this.clearHandlers.push, this.scene.game.canvas.addEventListener, this.cleanups.push, this.scene.game.canvas.removeEventListener

Decision branches: L84: !this.enabled || id !== null; L86: start(v.x, v.y); L89: this.enabled && p.id === id; L95: p.id !== id; L97: this.enabled

Execution order (direct statements; nested callbacks stay within their owner):

L80-80 FirstStatement: let id: number | null = null;

L81-82 FirstStatement: const point = (p: Phaser.Input.Pointer) => this.scene.cameras.main.getWorldPoint(p.x, p.y);

L83-87 ExpressionStatement: this.on("pointerdown", (p: Phaser.Input.Pointer) => { if (!this.enabled || id !== null) return;

L88-93 ExpressionStatement: this.on("pointermove", (p: Phaser.Input.Pointer) => { if (this.enabled && p.id === id) {

L94-101 FirstStatement: const up = (p: Phaser.Input.Pointer) => { if (p.id !== id) return;

L102-102 ExpressionStatement: this.on("pointerup", up);

L103-103 ExpressionStatement: this.on("pointerupoutside", up);

L104-107 FirstStatement: const clear = () => { id = null;

L108-108 ExpressionStatement: this.clearHandlers.push(clear);

L109-109 ExpressionStatement: this.scene.game.canvas.addEventListener("touchcancel", clear);

L110-110 ExpressionStatement: this.scene.game.canvas.addEventListener("pointercancel", clear);

L111-114 ExpressionStatement: this.cleanups.push(() => { this.scene.game.canvas.removeEventListener("touchcancel", clear);

InputManager.drag.point [L81-82]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: point = (p: Phaser.Input.Pointer) =>

Calls / delegates: this.scene.cameras.main.getWorldPoint

InputManager.drag.up [L94-101]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: up = (p: Phaser.Input.Pointer) =>

Direct writes: id

Calls / delegates: point, end

Decision branches: L95: p.id !== id; L97: this.enabled

InputManager.drag.clear [L104-107]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clear = () =>

Direct writes: id

Calls / delegates: cancel

InputManager.press [L116-127]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: press(codes: string[], callback: (code: string) => void)

Calls / delegates: codes.includes, event.preventDefault, callback, keyboard.on, this.cleanups.push, keyboard.off

Decision branches: L118: !keyboard; L120: this.enabled && !event.repeat && codes.includes(event.code)

InputManager.press.handler [L119-124]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: handler = (event: KeyboardEvent) =>

Calls / delegates: codes.includes, event.preventDefault, callback

Decision branches: L120: this.enabled && !event.repeat && codes.includes(event.code)

InputManager.keys [L128-147]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: keys(codes: string[])

Calls / delegates: Set, codes.includes, e.preventDefault, held.add, held.delete, keyboard.on, this.cleanups.push, keyboard.off, this.clearHandlers.push, held.clear

Return at L131: held

Return at L146: held

Decision branches: L131: !keyboard; L133: this.enabled && codes.includes(e.code)

InputManager.keys.down [L132-137]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: down = (e: KeyboardEvent) =>

Calls / delegates: codes.includes, e.preventDefault, held.add

Decision branches: L133: this.enabled && codes.includes(e.code)

InputManager.keys.up [L138-138]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: up = (e: KeyboardEvent) =>

```
up = (e: KeyboardEvent) => held.delete(e.code)
```

InputManager.on [L148-151]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private on(event: string, handler: (...args: any[]) => void)

Calls / delegates: this.scene.input.on, this.cleanups.push, this.scene.input.off

InputManager.clear [L152-158]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clear()

Direct writes: p.down, p.id

Calls / delegates: this.clearHandlers.forEach, fn, this.players.forEach

InputManager.pause [L159-162]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.enabled

Calls / delegates: this.clear

InputManager.resume [L163-166]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.enabled

Calls / delegates: this.clear

InputManager.dispose [L167-171]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.clearHandlers

Calls / delegates: this.pause, this.cleanups.splice(0).forEach, this.cleanups.splice, fn

InputManager.listenerCount [L172-174]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get listenerCount()

Return at L173: this.cleanups.length

```
get listenerCount() { return this.cleanups.length; }
```

src/core/OfflineManager.ts

ACTIVE SOURCE / tool or application | 20 lines | SHA-256 22cc52b7aef59d07...

Requests explicit CACHE_ALL work from the current service worker through a MessageChannel and handles timeout/failure.

downloadCollection [L2-19]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export async function downloadCollection()

Direct writes: channel.port1.onmessage

Calls / delegates: Error, Promise, MessageChannel, setTimeout, channel.port1.close, reject, clearTimeout, resolve, controller.postMessage

Return at L5: new Promise<void>((resolve, reject) => { const channel = new MessageChannel(), timeout = setTimeout(() => { channel.port1.close(); reject(Error("Download timed out")); }, 60000); channel.port1.onmessage = ({ data }) => { clearTimeout(timeout); channel.port1.close(); if (data.ok) resolve(); else reject(Error("Download failed")); }; controller.postMe ... [continued in source reference]

Decision branches: L4: !controller; L14: data.ok

src/core/ScoreManager.ts

ACTIVE SOURCE / tool or application | 19 lines | SHA-256 6fe99d96c17f962d...

Records one overall duel result per begun round.

Imports: ../shared/contracts/GameModule, ./StorageManager

State, resources and initial values

Line	Field / declaration
4	private recorded = false;

ScoreManager [L3-18]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ScoreManager.constructor [L5-5]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private storage: StorageManager)

```
constructor(private storage: StorageManager) {}
```

ScoreManager.beginRound [L6-8]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: beginRound()

Direct writes: this.recorded

ScoreManager.record [L9-17]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: record(winner: Winner)

Direct writes: this.recorded

Calls / delegates: this.storage.save

Decision branches: L10: this.recorded; L12: winner === "draw"

src/core/StorageManager.ts

ACTIVE SOURCE / tool or application | 84 lines | SHA-256 bb0f468bb78d7e43...

Validates shared settings/results and reads/writes JSON records with in-memory fallback.

State, resources and initial values

Line	Field / declaration
22	static key = "pocket-rivals.v1";
23	data: SavedData = defaults();
24	persistent = true;
25	private records = new Map<string, string>();

SavedData [L1-9]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface SavedData { version: 1; sound: boolean; music: boolean; // Legacy field retained for saved-data compatibility; no music is played. volume: number; wins: [number, number]; draws: number; rounds: number; }
```

defaults [L10-18]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: defaults = (): SavedData =>

```
defaults = (): SavedData => ({ version: 1, sound: true, music: false, volume: 0.75, wins: [0, 0], draws: 0, rounds: 0, })
```

count [L19-20]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: count = (v: unknown) =>

StorageManager [L21-83]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

StorageManager.constructor [L26-51]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private storage: Pick<Storage, "getItem" | "setItem"> | undefined,)

Direct writes: this.persistent, this.data

Calls / delegates: storage?.getItem, count

Decision branches: L32: raw; L34: d?.version === 1

StorageManager.save [L52-59]

Create/persist the component checkpoint or record described by the returned expression.

Signature: save()

Direct writes: this.persistent

Calls / delegates: Error, this.storage.setItem

Decision branches: L54: !this.storage

StorageManager.readRecord [L60-67]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: readRecord(key: string): unknown

Calls / delegates: this.records.get, this.storage?.getItem

Return at L63: raw ? JSON.parse(raw) : null

StorageManager.writeRecord [L68-76]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: writeRecord(key: string, value: unknown)

Direct writes: this.persistent

Calls / delegates: this.records.set, Error, this.storage.setItem

Decision branches: L71: !this.storage

StorageManager.resetScores [L77-82]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: resetScores()

Direct writes: this.data.wins, this.data.draws, this.data.rounds

Calls / delegates: this.save

src/core/ThemeManager.ts

ACTIVE SOURCE / tool or application | 24 lines | SHA-256 2bffcd138882baa2...

Defines the shared palette, logical canvas dimensions and default FIELD geometry.

Module constants and data

L1-12 theme = { ink: 0x242b32, paper: 0xf8f5eb, orange: 0xf49b58, blue: 0x58addb, green: 0xb8d998, ice: 0xc8e7eb, peach: 0xf3bdab, white: 0xfffdf5, muted: 0x7d8583, line: 0x93ada5, }

L13-13 W = 420

L14-14 H = 760

L15-21 FIELD = { left: 24, right: 396, top: 112, bottom: 650, middle: 381, }

cssColor [L22-23]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: cssColor = (color: number) =>

Calls / delegates: color.toString(16).padStart, color.toString

src/core/TournamentManager.ts

ACTIVE SOURCE / tool or application | 158 lines | SHA-256 c9c19f188e57b017...

Owns serialized tournament phases, shuffled schedules, attempt tokens, checkpoints and exactly-once points.

Imports: ../shared/contracts/GameModule, ./StorageManager

State, resources and initial values

Line	Field / declaration
34	state: TournamentState | null = null;

Module constants and data

L14-14 KEY = "pocket-rivals.tournament.v1"

TournamentState [L3-13]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface TournamentState { version: 1; id: string; phase: "between" | "playing" | "complete"; plan: MatchSpec[]; results: GameResult[]; points: [number, number]; attempt: number; token: string | null; checkpoint?: unknown; }
```

shuffledSchedule [L15-32]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function shuffledSchedule(choices: MatchSpec[], rounds: number, random = Math.random,)

Direct writes: [bag[i], bag[j]]

Calls / delegates: Error, random, plan.push, bag.slice, structuredClone

Return at L31: structuredClone(plan)

Decision branches: L20: !choices.length || !Number.isInteger(rounds) || rounds < 1 || rounds > 25

TournamentManager [L33-157]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TournamentManager.constructor [L35-37]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private storage: StorageManager)

Direct writes: this.state

Calls / delegates: this.restore, storage.readRecord

TournamentManager.restore [L38-83]

Validate/replay saved state and fall back to a fresh legal instance when rejected.

Signature: private restore(data: unknown): TournamentState | null

Direct writes: d.points

Calls / delegates: ["between", "playing", "complete"].includes, d.plan.every, ["local", "ai"].includes, ["medium", "hard", "impossible"].includes, d.results.every, [1, 2, "draw"].includes, r.scores.every, this.addPoints

Return at L82: d

Decision branches: L40: !d || d.version !== 1 || typeof d.id !== "string" || !Array.isArray(d.plan) || d.plan.length < 1 || d.plan.length > 25 || !Array.isArray(d.results) || d.results.length > d.plan.length || ![“between”, “playing”, “complete ... [continued in source reference]; L54: !d.plan.every((m) => typeof m?.gameId === "string" && ["local", "ai"].includes(m.options?.mode) && ["medium", "hard", "impossible"].includes(m.options?.difficulty),); L63: !d.results.every((r) => [1, 2, "draw"].includes(r?.winner) && Array.isArray(r.scores) && r.scores.length === 2 && r.scores.every((n) => Number.isFinite(n) && n >= 0),); L73: (d.phase === "complete") !== (d.results.length === d.plan.length); L75: d.phase === "playing" && d.token !== `${d.id}:${d.results.length}:${d.attempt}`

Execution order (direct statements; nested callbacks stay within their owner):

L39-39 FirstStatement: const d = data as TournamentState;

L40-53 IfStatement: if (!d ||

L54-62 IfStatement: if (!d.plan.every(

L63-72 IfStatement: if (!d.results.every(

L73-74 IfStatement: if ((d.phase === "complete") !== (d.results.length === d.plan.length)) return null;

L75-79 IfStatement: if (d.phase === "playing" &&

L80-80 ExpressionStatement: d.points = [0, 0];

L81-81 ForOfStatement: for (const r of d.results) this.addPoints(d, r);

L82-82 ReturnStatement: return d;

TournamentManager.persist [L84-86]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private persist()

Calls / delegates: this.storage.writeRecord

TournamentManager.create [L87-101]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: create(plan: MatchSpec[])

Direct writes: this.state

Calls / delegates: Error, crypto.randomUUID, structuredClone, this.persist

Decision branches: L88: !plan.length || plan.length > 25

TournamentManager.current [L102-106]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: current()

Return at L105: s.plan[s.results.length]

Decision branches: L104: !s || s.phase === "complete"

TournamentManager.startRound [L107-116]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: startRound()

Direct writes: s.phase, s.token

Calls / delegates: this.persist, this.current

Return at L115: { spec: this.current()!, token: s.token!, checkpoint: s.checkpoint }

Decision branches: L109: !s || s.phase === "complete"; L110: s.phase === "between"

TournamentManager.restartRound [L117-125]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: restartRound()

Direct writes: s.phase, s.token

Calls / delegates: this.startRound

Return at L124: this.startRound()

Decision branches: L119: !s || s.phase !== "playing"

TournamentManager.checkpoint [L126-132]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: checkpoint(token: string, data: unknown)

Direct writes: s.checkpoint

Calls / delegates: this.persist

Decision branches: L128: s?.phase !== "playing" || s.token !== token

TournamentManager.finish [L133-143]

Resolve/report the owned terminal attempt/result using current guards.

Signature: finish(token: string, result: GameResult)

Direct writes: s.phase, s.token

Calls / delegates: s.results.push, structuredClone, this.addPoints, this.persist

Decision branches: L135: !s || s.phase !== "playing" || s.token !== token

TournamentManager.addPoints [L144-149]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private addPoints(s: TournamentState, r: GameResult)

Direct writes: s.points[0], s.points[1]

Decision branches: L145: r.winner === "draw"

TournamentManager.rematch [L150-152]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: rematch()

Calls / delegates: this.create

Decision branches: L151: this.state

TournamentManager.clear [L153-156]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clear()

Direct writes: this.state

Calls / delegates: this.persist

src/scenes

src/scenes/GameHost.ts

ACTIVE SOURCE / tool or application | 240 lines | SHA-256 bdd9cca51698fa7c...

Creates the selected module context/HUD, owns its InputManager, derives foreground dt and disposes the module on scene shutdown.

Imports: phaser, ../core/GameManager, ../shared/contracts/GameModule, ../core/GameRegistry, ../core/GameLoader, ../core/InputManager, ../core/ThemeManager, ../shared/components/drawing

State, resources and initial values

Line	Field / declaration
14	module?: MiniGame;
15	controls?: InputManager;
16	paused = false;
17	ready = false;
18	private ended = false;
19	private alive = false;
20	private generation = 0;
21	private id = "";
22	private options!: MatchOptions;
23	private restoredState?: unknown;
24	private run = 0;
25	private texts: Phaser.GameObjects.Text[] = [];
26	private lastTime = 0;
27	private clockText!: Phaser.GameObjects.Text;
28	private hint!: Phaser.GameObjects.Text;

GameHost [L13-239]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GameHost.constructor [L29-31]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private app: GameManager)

Calls / delegates: super

GameHost.init [L32-48]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: init(data: { id: string; options: MatchOptions; restoredState?: unknown; run: number; })

Direct writes: this.id, this.options, this.restoredState, this.run, this.paused, this.ready, this.ended, this.alive, this.texts

GameHost.create [L49-171]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: create()

Direct writes: this.texts, this.clockText, this.hint, this.controls, this.module, this.ended, this.ready, this.lastTime

Calls / delegates: performance.now, getGame, this.cameras.main .setBackgroundColor(theme.paper) .setOrigin(0, 0) .setZoom(this.app.renderScale) .setScroll, this.cameras.main .setBackgroundColor(theme.paper) .setOrigin(0, 0) .setZoom, this.cameras.main .setBackgroundColor(theme.paper) .setOrigin, this.cameras.main .setBackgroundColor, this.add.graphics, rounded, g.lineStyle(3, theme.ink) .lineBetween(44, 27, 35, 35) .lineBetween(35, 35, 44, 43) .lineBetween, g.lineStyle(3, theme.ink) .lineBetween(44, 27, 35, 35) .lineBetween, g.lineStyle(3, theme.ink) .lineBetween, g.lineStyle, g.fillStyle(theme.ink) .fillRoundedRect(373, 27, 4, 16, 2) .fillRoundedRect, g.fillStyle(theme.ink) .fillRoundedRect, g.fillStyle, label, this.options.variant.toUpperCase, this.add .zone(40, 35, 50, 48) .setInteractive({ useHandCursor: true }) .on, this.add .zone(40, 35, 50, 48) .setInteractive, this.add .zone, this.app.pause, this.add .zone(380, 35, 50, 48) .setInteractive({ useHandCursor: true }) .on, this.add .zone(380, 35, 50, 48) .setInteractive, label(this, 210, 86, this.options.mode === "ai" ? "BLUE AI 0" : "BLUE 0", 18,).setRotation, InputManager, this.events.once, this.cleanup, loadGame(this.id, this.options) .then((module) => { if (!this.alive || generation !== this.generation) { return; } this.module = module; module.initialize({ reportAI: (ms) => this.app.diagnostics.computation(ms), bestScore: this.app.arcade.best(this.id), setArcadeHUD: (score, detail) => { this.texts[0]?.setText(`SCORE ${score} BEST ${Math.max(score, this.app.arcade.best(this.id))}`,); this.texts[1]?.setText(detail); }, finishArcade: (score, detail) => { if (this.ended) return; this.ended = true; this.pauseGame(); this.app.finishArcade(score, detail, this.run); }, scene: this, navigation: { pause: () => this.app.pause(), exit: () => this.app.pause("leave"), }, options: this.options, restoredState: this.restoredState, saveState: (state) => this.app.checkpoint(this.run, state), input: this.controls!, audio: this.app.audio, setClock: (text, urgent) => this.clockText ?.setText(text) .setColor(urgent ? "#b2462c" : "#697474"), setScore: (a, b) => this.setScore(a, b), setHint: (text) => this.hint?.setText(text), finish: (result) => this.finish(result), }); this.ready = true; this.app.diagnostics.loaded(this.id); this.app.diagnostics.initialized(this.id, performance.now() - started); if (this.paused) { this.controls!.pause(); module.pause(); } this.lastTime = performance.now(); this.game.events.once(Phaser.Core.Events.POST_RENDER, () => { if (this.alive && generation === this.generation) this.app.onReady(this.run); }); }) .catch, loadGame(this.id, this.options) .then, loadGame, module.initialize, this.app.diagnostics.computation, this.app.arcade.best, this.texts[0]?.setText, this.texts[1]?.setText, this.pauseGame, this.app.finishArcade, this.app.checkpoint, this.clockText ?.setText(text) .setColor, this.clockText ?.setText, this.setScore, this.hint?.setText, this.finish, this.app.diagnostics.loaded, this.app.diagnostics.initialized, this.controls!.pause, module.pause, this.game.events.once, this.app.onReady, this.app.showError

Decision branches: L58: meta.presentation !== "custom"; L88: meta.category !== "arcade"; L110: meta.category !== "arcade"; L117: !this.alive || generation !== this.generation; L131: this.ended; L157: this.paused; L163: this.alive && generation === this.generation; L168: this.alive && generation === this.generation

Execution order (direct statements; nested callbacks stay within their owner):

L50-50 FirstStatement: const generation = this.generation;

L51-51 FirstStatement: const started = performance.now();

L52-52 FirstStatement: const meta = getGame(this.id);

L53-57 ExpressionStatement: this.cameras.main .setBackgroundColor(theme.paper)

L58-112 IfStatement: if (meta.presentation !== "custom") { const g = this.add.graphics();

L113-113 ExpressionStatement: this.controls = new InputManager(this);

L114-114 ExpressionStatement: this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => this.cleanup());

L115-170 ExpressionStatement: void loadGame(this.id, this.options) .then((module) => {

GameHost.setScore [L172-187]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private setScore(a: number, b: number)

Calls / delegates: [a, b].forEach, text.setText, this.tweens.add

Decision branches: L176: text.text !== next

GameHost.finish [L188-193]

Resolve/report the owned terminal attempt/result using current guards.

Signature: private finish(result: GameResult)

Direct writes: this.ended

Calls / delegates: this.pauseGame, this.app.finish

Decision branches: L189: this.ended

GameHost.update [L194-204]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update()

Direct writes: this.lastTime

Calls / delegates: performance.now, this.module?.update, this.app.diagnostics.frame, this.game.scene.getScenes, this.textures.getTextureKeys

Decision branches: L198: this.ready && !this.paused

GameHost.pauseGame [L205-209]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: pauseGame()

Direct writes: this.paused

Calls / delegates: this.controls?.pause, this.module?.pause

GameHost.resumeGame [L210-216]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: resumeGame()

Direct writes: this.paused, this.lastTime

Calls / delegates: performance.now, this.controls?.resume, this.module?.resume

Decision branches: L211: this.ended

GameHost.cleanup [L217-225]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private cleanup()

Direct writes: this.alive, this.ready, this.module, this.controls, this.texts

Calls / delegates: this.module?.dispose, this.controls?.dispose

GameHost.snapshot [L226-238]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.controls?.players.map, this.module?.snapshot

Return at L227: { id: this.id, options: this.options, ready: this.ready, paused: this.paused, ended: this.ended, inputListeners: this.controls?.listenerCount ?? 0, pointers: this.controls?.players.map((p) => ({ ...p })), game: this.module?.snapshot?.(), children: this.children.length, }

src/scenes/GameSetup.ts

ACTIVE SOURCE / tool or application | 64 lines | SHA-256 5c35feae9cd7d885...

Renders mode/difficulty/variant selection and starts tutorials or the chosen game.

Imports: ../shared/contracts/GameModule, ../shared/contracts/GameModule, ../core/GameLoader, ../shared/components/Modal

difficultyButtons [L8-9]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: difficultyButtons = (selected: string) =>

Calls / delegates: ["medium", "hard", "impossible"].map((d) => `<button role="radio" aria-checked="${selected === d}" class="${selected === d ? "selected" : ""}" data-action="difficulty-${d}">${d[0].toUpperCase() + d.slice(1)}</button>`).join, ["medium", "hard", "impossible"].map, d[0].toUpperCase, d.slice

GameSetup [L10-63]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GameSetup.constructor [L11-14]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private modal: Modal, private launch: (options: MatchOptions) => void,)

```
constructor( private modal: Modal, private launch: (options: MatchOptions) => void, ) {}
```

GameSetup.open [L15-62]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: open(meta: GameMetadata, options: MatchOptions = defaultOptions())

Direct writes: current.mode, actions[`difficulty-${d}`], current.difficulty, actions[`variant-${v.id}`], current.variant

Calls / delegates: meta.variants?.find, meta.supportedModes.includes, this.modal.close, render, this.launch, loadTutorial, this.modal.show, meta.variants.map((v) => `<button data-action="variant-${v.id}" class="variant-choice ${current.variant === v.id ? "selected" : ""}" aria-pressed="${current.variant === v.id}">${v.preview ? `${v.preview}` : ""}<b>${v.name}</b><small>${v.description}</small></button>`).join, meta.variants.map, difficultyButtons

Decision branches: L25: !ai; L33: ai

Execution order (direct statements; nested callbacks stay within their owner):

L16-19 FirstStatement: const current = { ...options,

L20-60 FirstStatement: const render = () => { const variant = meta.variants?.find((v) => v.id === current.variant),

L61-61 ExpressionStatement: render();

GameSetup.open.render [L20-60]

Build or refresh visual/DOM output from current state.

Signature: render = () =>

Direct writes: current.mode, actions[`difficulty-${d}`], current.difficulty, actions[`variant-${v.id}`], current.variant

Calls / delegates: meta.variants?.find, meta.supportedModes.includes, this.modal.close, render, this.launch, loadTutorial, this.modal.show, meta.variants.map((v) => `<button data-action="variant-${v.id}" class="variant-choice ${current.variant === v.id ? "selected" : ""}" aria-pressed="${current.variant === v.id}">${v.preview ? `${v.preview}` : ""}<b>${v.name}</b><small>${v.description}</small></button>`).join, meta.variants.map, difficultyButtons

Decision branches: L25: !ai; L33: ai

Execution order (direct statements; nested callbacks stay within their owner):

L21-24 FirstStatement: const variant = meta.variants?.find((v) => v.id === current.variant), ai =

L25-25 IfStatement: if (!ai) current.mode = "local";

L26-44 FirstStatement: const actions: Record<string, () => void> = { close: () => this.modal.close(),

L45-49 ForOfStatement: for (const d of ["medium", "hard", "impossible"] as const) actions[`difficulty-${d}`] = () => {

L50-54 ForOfStatement: for (const v of meta.variants ?? []) actions[`variant-${v.id}`] = () => {

L55-59 ExpressionStatement: this.modal.show(`<div class="eyebrow">PICK YOUR RIVAL</div><h2 id="dialog-title">${meta.name}</h2>${meta.variants ? ` ${meta.variants.map((v) => `<button data-action="vari

src/scenes/MainMenu.ts

ACTIVE SOURCE / tool or application | 54 lines | SHA-256 11d5d5d5de46a05b...

Renders metadata-driven collection sections and binds menu navigation.

Imports: ../core/GameManager, ../core/GameRegistry, ../shared/components/icons

State, resources and initial values

Line	Field / declaration
5	private section: "duel" | "arcade" = "duel";

MainMenu [L4-53]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

MainMenu.constructor [L6-9]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private app: GameManager, private root: HTMLElement,)

```
constructor( private app: GameManager, private root: HTMLElement, ) {}
```

MainMenu.render [L10-52]

Build or refresh visual/DOM output from current state.

Signature: render()

Direct writes: this.root.innerHTML, this.root.querySelector(".menu-scroll")!.scrollTop, b.onclick, this.section, this.root.querySelector<HTMLButtonElement>('[data-action="settings"]',)!.onclick, this.root.querySelector<HTMLButtonElement>('[data-action="sound"]',)!.onclick

Calls / delegates: this.root.querySelector, games.filter, this.app.savedMatch, available.map((g, i) => `<button class="game-card" data-game="${g.id}" style="--card:${g.color};--accent:${g.accent};--delay:${i * 60}ms" aria-label="Play ${g.name}">${g.tag}${g.name.split(/[-]+/).join("
")}${g.description}${g.preview}${icons.arrow}0${i + 1}</button>`).join, available.map, g.name.split(/[-]+/).join, g.name.split, this.render, this.root .querySelector(".collection-modes") ?.scrollIntoView, this.root .querySelector, bind, switchSection, this.app.openTournament, this.app.showTournament, this.app.resumeSaved, this.root .querySelectorAll<HTMLButtonElement>("[data-game]") .forEach, this.root .querySelectorAll, this.app.selectGame, this.app.settings, this.app.toggleSound

Decision branches: L26: b

Execution order (direct statements; nested callbacks stay within their owner):

L11-11 FirstStatement: const scroll = this.root.querySelector(".menu-scroll")?.scrollTop ?? 0;

L12-12 FirstStatement: const d = this.app.storage.data;

L13-15 FirstStatement: const available = games.filter((g) => (g.category ?? "duel") === this.section,

L16-20 ExpressionStatement: this.root.innerHTML = ` <header class="toolbar"> GOOD GAMES.
GREAT COMPANY. <div class="toolbar-actions"

L21-21 ExpressionStatement: this.root.querySelector(".menu-scroll")!.scrollTop = scroll;

L22-27 FirstStatement: const bind = (name: string, fn: () => void) => { const b = this.root.querySelector<HTMLButtonElement>(

L28-34 FirstStatement: const switchSection = (section: "duel" | "arcade") => { this.section = section;

L35-35 ExpressionStatement: bind("quick", () => switchSection("duel"));

L36-36 ExpressionStatement: bind("arcade", () => switchSection("arcade"));

L37-37 ExpressionStatement: bind("tournament", () => this.app.openTournament());

L38-38 ExpressionStatement: bind("resume-tournament", () => this.app.showTournament());

L39-39 ExpressionStatement: bind("resume-match", () => this.app.resumeSaved());

L40-42 ExpressionStatement: this.root .querySelectorAll<HTMLButtonElement>("[data-game]")

L43-45 ExpressionStatement: this.root.querySelector<HTMLButtonElement>('[data-action="settings"]',

L46-51 ExpressionStatement: this.root.querySelector<HTMLButtonElement>('[data-action="sound"]',

MainMenu.render.bind [L22-27]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: bind = (name: string, fn: () => void) =>

Direct writes: b.onclick

Calls / delegates: this.root.querySelector

Decision branches: L26: b

MainMenu.render.switchSection [L28-34]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: switchSection = (section: "duel" | "arcade") =>

Direct writes: this.section

Calls / delegates: this.render, this.root .querySelector(".collection-modes") ?.scrollIntoView, this.root .querySelector

src/scenes/Results.ts

ACTIVE SOURCE / tool or application | 8 lines | SHA-256 7e18c05de5c4f190...

Formats explicit duel winner and score data into result markup.

Imports: ../shared/contracts/GameModule, ../shared/components/icons

resultsContent [L3-7]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function resultsContent(result: GameResult, options?: MatchOptions)

Calls / delegates: name.toLowerCase, Number, result.scores[0].toFixed, result.scores[1].toFixed

Return at L6: `<div class="result-confetti"><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i></div><span class="result-emblem ${draw ? "draw" : name.toLowerCase()}">${draw ? "=" : icons.cup}</span><h2 id="dialog-title">${draw ? "Evenly matched." : `${name}${options?.mode === "ai" && result.winner === 2 ? " AI" : ""} takes it!`}</h2> ${Num ... [continued in source reference]

src/scenes/Settings.ts

ACTIVE SOURCE / tool or application | 66 lines | SHA-256 8d3ad1d6f600d1fc...

Presents sound/volume/offline download controls and confirmed score reset.

Imports: ../core/OfflineManager, ../core/GameManager, ../shared/components/icons

State, resources and initial values

Line	Field / declaration
6	private offlineStatus = "";
7	private downloading = false;

Settings [L4-65]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

Settings.constructor [L5-5]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private app: GameManager)

```
constructor(private app: GameManager) {}
```

Settings.show [L8-51]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: show()

Direct writes: this.downloading, this.offlineStatus, volume.oninput, d.volume, root.querySelector("#volume-label")!.textContent, volume.onchange

Calls / delegates: this.app.modal.show, this.app.modal.close, this.app.refreshMenu, this.app.toggleSound, this.show, downloadCollection, document.querySelector, this.confirmReset, root.querySelector, Number, this.app.storage.save, this.app.audio.syncSettings, this.app.audio.play

Decision branches: L32: this.app.modal.isOpen && document.querySelector('[data-action="offline"]')

Execution order (direct statements; nested callbacks stay within their owner):

L9-9 FirstStatement: const d = this.app.storage.data;

L10-40 FirstStatement: const root = this.app.modal.show(`<button class="modal-close icon-button" data-action="close" aria-label="Close settings">${icons.close}</button> MAKE YOURSELF AT HOME <h2 id

L41-43 FirstStatement: const volume = root.querySelector<HTMLInputElement>('input[aria-label="Effects volume"]',

L44-49 ExpressionStatement: volume.oninput = () => { d.volume = Number(volume.value) / 100;

L50-50 ExpressionStatement: volume.onchange = () => this.app.audio.play("tap");

Settings.confirmReset [L52-64]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private confirmReset()

Calls / delegates: this.app.modal.show, this.app.storage.resetScores, this.app.refreshMenu, this.show

src/shared/ai

src/shared/ai/MatchRandom.ts

ACTIVE SOURCE / tool or application | 87 lines | SHA-256 d07ade6b650939f4...

Provides isolated seeded identity, bounded decision jitter, deterministic rankings and replay support.

State, resources and initial values

Line	Field / declaration
33	readonly seed: number;
34	readonly profile: number;
35	readonly role: number;
36	readonly tempo: number;
37	readonly side: -1 | 1;
38	private state: number;

SeedInput [L2-2]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type SeedInput = number | (() => number);
```

mixSeed [L3-8]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function mixSeed(seed: number, salt: number)

Direct writes: n

Return at L7: (n ^ (n >>> 15)) >>> 0

freshSeed [L9-17]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function freshSeed(): number

Calls / delegates: new URLSearchParams(location.search).get, URLSearchParams, /^\d+$/.test, Number, crypto.getRandomValues, Uint32Array

Return at L12: Number(value) >>> 0

Return at L15: crypto.getRandomValues(new Uint32Array(1))[0]

Return at L16: (Math.random() * 4294967296) >>> 0

Decision branches: L10: import.meta.env.DEV && typeof location !== "undefined"; L12: value !== null && /^\d+$/.test(value); L14: typeof crypto !== "undefined" && crypto.getRandomValues

replayMode [L18-24]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function replayMode()

Calls / delegates: new URLSearchParams(location.search).has, URLSearchParams

Return at L19: (import.meta.env.DEV && typeof location !== "undefined" && new URLSearchParams(location.search).has("aiReplay"))

AIIdentity [L25-31]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface AIIdentity { seed: number; profile: number; role: number; tempo: number; side: -1 | 1; }
```

MatchRandom [L32-77]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

MatchRandom.constructor [L39-47]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(input: SeedInput = freshSeed())

Direct writes: this.seed, this.state, this.profile, this.role, this.tempo, this.side

Calls / delegates: input, mixSeed

MatchRandom.next [L48-54]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: next = () =>

Direct writes: this.state, t

Return at L53: ((t ^ (t >>> 14)) >>> 0) / 4294967296

MatchRandom.sample [L56-61]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: sample(mean: number, spread: number, min: number)

Calls / delegates: this.next

Return at L57: Math.max(min, mean + (spread * (this.next() + this.next() + this.next() - 1.5)) / 1.5,)

MatchRandom.rank [L62-64]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: rank(action: number, salt = 0)

Calls / delegates: mixSeed

Return at L63: mixSeed(this.seed, action + salt * 97) / 4294967296

MatchRandom.turnSeed [L65-67]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: turnSeed(turn: number)

Calls / delegates: mixSeed

Return at L66: mixSeed(this.seed, turn + 101)

MatchRandom.snapshot [L68-76]

Return diagnostic copies/observations for development and tests.

Signature: snapshot(): AIIdentity

Return at L69: { seed: this.seed, profile: this.profile, role: this.role, tempo: this.tempo, side: this.side, }

```
snapshot(): AIIdentity { return { seed: this.seed, profile: this.profile, role: this.role, tempo: this.tempo, side: this.side, }; }
```

restoredAISeed [L78-86]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function restoredAISeed(state: unknown): number | undefined

Return at L80: typeof value === "number" && Number.isInteger(value) && value >= 0 && value <= 4294967295 ? value : undefined

src/shared/components

src/shared/components/Modal.ts

ACTIVE SOURCE / tool or application | 79 lines | SHA-256 3bfa2f1968e56b14...

Owns overlay content, action binding, back handling, focus trapping and focus restoration.

State, resources and initial values

Line	Field / declaration
2	private root = document.querySelector<HTMLElement>("#overlays")!;
3	private previous: HTMLElement | null = null;
4	private backAction?: () => void;

Modal [L1-78]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

Modal.isOpen [L5-7]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get isOpen()

Return at L6: this.root.childElementCount > 0

```
get isOpen() { return this.root.childElementCount > 0; }
```

Modal.show [L8-44]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: show(content: string, actions: Record<string, () => void>, className = "", back?: () => void,)

Direct writes: this.previous, this.backAction, node.inert, this.root.innerHTML, panel, panel.className, panel.innerHTML, b.onclick

Calls / delegates: document.querySelector, node.setAttribute, this.root.querySelector, this.root .querySelectorAll<HTMLButtonElement>("[data-action]") .forEach, this.root .querySelectorAll, actions[b.dataset.action!], (focused ? this.root.querySelector<HTMLButtonElement>(`[data-action="${CSS.escape(focused)}"]`,) : null)?.focus, CSS.escape, this.root.contains, this.root .querySelector<HTMLButtonElement>("button:not([disabled])") ?.focus, this.root .querySelector

Return at L43: this.root

Decision branches: L14: !this.isOpen; L23: !panel; L39: !this.root.contains(document.activeElement)

Execution order (direct statements; nested callbacks stay within their owner):

L14-14 IfStatement: if (!this.isOpen) this.previous = document.activeElement as HTMLElement;

L15-15 ExpressionStatement: this.backAction = back;

L16-20 ForOfStatement: for (const id of ["#menu", "#game-stage"]) { const node = document.querySelector<HTMLElement>(id)!;

L21-21 FirstStatement: const focused = (document.activeElement as HTMLElement)?.dataset.action;

L22-22 FirstStatement: let panel = this.root.querySelector<HTMLElement>(".modal");

L23-27 IfStatement: if (!panel) { this.root.innerHTML =

L28-28 ExpressionStatement: panel.className = `modal ${className}`;

L29-29 ExpressionStatement: panel.innerHTML = content;

L30-32 ExpressionStatement: this.root .querySelectorAll<HTMLButtonElement>("[data-action]")

L33-38 ExpressionStatement: (focused ? this.root.querySelector<HTMLButtonElement>(

L39-42 IfStatement: if (!this.root.contains(document.activeElement)) this.root

L43-43 ReturnStatement: return this.root;

Modal.back [L45-48]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: back()

Calls / delegates: this.backAction, this.close

Decision branches: L46: this.backAction

Modal.close [L49-60]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: close()

Direct writes: this.backAction, node.inert, this.previous

Calls / delegates: this.root.replaceChildren, document.querySelector, node.removeAttribute, this.previous.focus

Decision branches: L57: this.previous?.isConnected

Modal.trapFocus [L61-77]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: trapFocus(e: KeyboardEvent)

Calls / delegates: this.root.querySelectorAll, list.at, e.preventDefault, last?.focus, first?.focus

Decision branches: L62: e.key !== "Tab" || !this.isOpen; L70: e.shiftKey && document.activeElement === first; L73: !e.shiftKey && document.activeElement === last

src/shared/components/TurnView.ts

ACTIVE SOURCE / tool or application | 48 lines | SHA-256 887c415958020461...

Rotates hot-seat module-owned content, blocks transition input and maps pointer coordinates back.

Imports: phaser

State, resources and initial values

Line	Field / declaration
4	private root: Phaser.GameObjects.Container;
5	private target = false;
6	private flipped = false;
7	private age = 0;

TurnView [L3-45]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TurnView.constructor [L8-17]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(scene: Phaser.Scene, children: (Phaser.GameObjects.Graphics | Phaser.GameObjects.Text)[],)

Direct writes: this.root

Calls / delegates: scene.add.container, child.setPosition, this.root.add

TurnView.set [L18-23]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: set(flipped: boolean)

Direct writes: this.target, this.age

Decision branches: L19: flipped !== this.target

TurnView.update [L24-32]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.age, this.flipped

Calls / delegates: this.root.setRotation, this.root.setAlpha

Decision branches: L25: this.age <= 0; L27: this.age <= 0.14 && this.flipped !== this.target

TurnView.ready [L33-35]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get ready()

Return at L34: this.age === 0

```
get ready() { return this.age === 0; }
```

TurnView.point [L36-38]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: point(x: number, y: number)

Return at L37: this.flipped ? { x: 420 - x, y: 762 - y } : { x, y }

```
point(x: number, y: number) { return this.flipped ? { x: 420 - x, y: 762 - y } : { x, y }; }
```

TurnView.rotation [L39-41]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get rotation()

Return at L40: this.flipped ? Math.PI : 0

```
get rotation() { return this.flipped ? Math.PI : 0; }
```

TurnView.destroy [L42-44]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: destroy()

Calls / delegates: this.root.destroy

turnBackground [L46-47]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: turnBackground = (player: 1 | 2) =>

```
turnBackground = (player: 1 | 2) => player === 1 ? 0xf7e1cc : 0xd9eaf4
```

src/shared/components/drawing.ts

ACTIVE SOURCE / tool or application | 67 lines | SHA-256 37a93ac8bbd4df1c...

Creates reusable outlined shapes, labels and game symbols.

Imports: phaser, ../../core/ThemeManager

rounded [L3-15]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function rounded(g: Phaser.GameObjects.Graphics, x: number, y: number, w: number, h: number, r: number, color: number, line = 3,)

Calls / delegates: g.fillStyle(color).fillRoundedRect, g.fillStyle, g.lineStyle(line, theme.ink).strokeRoundedRect, g.lineStyle

Decision branches: L14: line

circle [L16-26]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function circle(g: Phaser.GameObjects.Graphics, x: number, y: number, r: number, color: number, line = 3,)

Calls / delegates: g.fillStyle(color).fillCircle, g.fillStyle, g.lineStyle(line, theme.ink).strokeCircle, g.lineStyle

Decision branches: L25: line

label [L27-44]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function label(scene: Phaser.Scene, x: number, y: number, text: string, size = 20, color = "#242b32",)

Calls / delegates: scene.add .text(x, y, text, { fontFamily: "Fredoka, sans-serif", fontSize: `${size}px`, fontStyle: "600", color, }) .setResolution(Math.min(window.devicePixelRatio || 1, 2)) .setOrigin, scene.add .text(x, y, text, { fontFamily: "Fredoka, sans-serif", fontSize: `${size}px`, fontStyle: "600", color, }) .setResolution, scene.add .text

Return at L35: scene.add .text(x, y, text, { fontFamily: "Fredoka, sans-serif", fontSize: `${size}px`, fontStyle: "600", color, }) .setResolution(Math.min(window.devicePixelRatio || 1, 2)) .setOrigin(0.5)

symbol [L45-66]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function symbol(g: Phaser.GameObjects.Graphics, x: number, y: number, player: 1 | 2, size = 22, scale = 1,)

Calls / delegates: g.lineStyle, g.lineBetween, g.strokeCircle

Decision branches: L55: player === 1

src/shared/components/icons.ts

ACTIVE SOURCE / tool or application | 26 lines | SHA-256 9c4b3fac170a0ee4...

Defines small inline SVG UI icons through a common SVG wrapper.

Module constants and data

L3-25 icons = { settings: svg('<path d="m9 3-.6 2.2-2 .9-2-.6-2 3.5 1.5 1.6v2.3l-1.5 1.6 2 3.5 2-.6 2 .9L9 21h4l.6-2.2 2-.9 2 .6 2-3.5-1.5-1.6v-2.3L19.6 9l-2-3.5-2 .6-2-.9L13 3Z"/><circle cx="11" cy="12" r="3"/>',), sound: svg('<path d="m11 4-5 4H3v8h3l5 4V4Z"/><path d="M15 8a6 6 0 0 1 0 8m3-11a10 10 0 0 1 0 14"/>',), muted: svg('<path d="m11 4-5 4H3v8h3l5 4V4Z"/><path d="m16 9 6 6m0-6-6 6"/>'), arrow: svg('<path d="M5 12h14m-6-6 6 6-6 6"/>'), close: svg('<path d="m6 6 12 12m0-12L6 18"/>'), cup: svg('<path d="M7 3h10v7a5 5 0 0 1-10 0V3Zm0 2H3v3a4 4 0 0 0 4 4m10-7h4v3a4 4 0 0 1-4 4m-5 3v6m-4 0h8"/>',), people: svg('<circle cx="8" cy="7" r="3"/><circle cx="17" cy="8" r="2.5"/><path d="M2 20v-3a6 6 0 0 1 12 0v3m2-7a5 5 0 0 1 6 5v2"/>',), offline: svg('<path d="M4 4 20 20M2 8a17 17 0 0 1 5-3m4-1a17 17 0 0 1 11 4M5 12a12 12 0 0 1 5-2m5 1a12 12 0 0 1 4 1m-11 4a6 6 0 0 1 7-1"/><circle cx="12" cy="20" r="1"/>',), hand: svg('<path d="M8 12V5a2 2 0 0 1 4 0v6-1a2 2 0 0 1 4 0v1a2 2 0 0 1 4 0v6c0 4-3 6-6 6-3 0-5-2-7-5l-3-4a2 2 0 0 1 3-2l1 1Z"/>',), }

svg [L1-2]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: svg = (body: string) =>

```
svg = (body: string) => `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" aria-hidden="true">${body}</svg>`
```

src/shared/contracts

src/shared/contracts/AIController.ts

ACTIVE SOURCE / tool or application | 8 lines | SHA-256 4a1c0470d4c7f834...

Defines the three AI difficulty IDs and generic observation-to-action controller contract.

Difficulty [L1-1]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Difficulty = "medium" | "hard" | "impossible";
```

AIController [L3-7]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface AIController<Observation, Action> { update(dt: number, observation: Observation): Action; reset(): void; dispose?(): void; }
```

src/shared/contracts/GameModule.ts

ACTIVE SOURCE / tool or application | 82 lines | SHA-256 70c564977bdd7467...

Defines metadata, options, module lifecycle, host context, factory and tutorial contracts.

Imports: phaser, ../../core/AudioManager, ../../core/InputManager, ./AIController, ./GameResult

MatchOptions [L7-11]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface MatchOptions { mode: "local" | "ai"; difficulty: Difficulty; variant?: string; }
```

MatchSpec [L12-15]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface MatchSpec { gameId: string; options: MatchOptions; }
```

GameVariant [L16-23]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface GameVariant { id: string; name: string; description: string; preview?: string; hasTutorial?: boolean; ai?: boolean; }
```

GameMetadata [L24-40]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface GameMetadata { id: string; name: string; description: string; instructions: string; color: string; accent: string; preview: string; order: number; tag: string; category?: "duel" | "arcade"; tournament?: boolean;
```

```
presentation?: "custom"; responsive?: boolean; supportedModes: readonly ("local-two-player" | "single-player")[];
variants?: readonly GameVariant[]; }
```

GameContext [L41-57]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface GameContext { reportAI: (milliseconds: number) => void; bestScore: number; setArcadeHUD: (score:
number, detail: string) => void; finishArcade: (score: number, detail: string) => void; scene: Phaser.Scene;
navigation?: { pause: () => void; exit: () => void }; input: InputManager; audio: AudioManager; options:
MatchOptions; restoredState?: unknown; saveState: (state: unknown) => void; setClock: (text: string, urgent:
boolean) => void; setScore: (one: number, two: number) => void; setHint: (text: string) => void; finish: (result:
GameResult) => void; }
```

MiniGame [L58-66]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface MiniGame { initialize(context: GameContext): void; update(deltaSeconds: number): void; pause():
void; resume(): void; dispose(): void; resize?(width: number, height: number): void; snapshot?(): unknown; }
```

GameFactory [L67-69]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type GameFactory = ( options: MatchOptions, ) => MiniGame | Promise<MiniGame>;
```

TutorialUI [L70-73]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface TutorialUI { show(content: string, actions: Record<string, () => void>): void; done(): void; }
```

GameExports [L74-77]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface GameExports { default: GameFactory; tutorial?: (variant: string, ui: TutorialUI) => void |
Promise<void>; }
```

defaultOptions [L78-81]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: defaultOptions = (): MatchOptions =>

```
defaultOptions = (): MatchOptions => ({ mode: "local", difficulty: "medium", })
```

Top-level block L5-5

```
export type { Player, Winner, GameResult } from "./GameResult";
```

src/shared/contracts/GameResult.ts

ACTIVE SOURCE / tool or application | 7 lines | SHA-256 12a25417c8a15ca9...

Defines Player, Winner and explicit two-score duel result types.

Player [L1-1]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Player = 1 | 2;
```

Winner [L2-2]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Winner = Player | "draw";
```

GameResult [L3-6]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface GameResult { winner: Winner; scores: readonly [number, number]; }
```

src/shared/effects

src/shared/effects/Impacts.ts

ACTIVE SOURCE / tool or application | 32 lines | SHA-256 d37892811d1e16fd...

Retains a fixed pool of short impact pulses for bounded feedback.

Imports: phaser

State, resources and initial values

Line	Field / declaration
9	private pool: Pulse[] = Array.from({ length: 14 }, () => ({ x: 0, y: 0, color: 0, life: 0, }));
15	private cursor = 0;

Pulse [L2-7]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
interface Pulse { x: number; y: number; color: number; life: number; }
```

Impacts [L8-31]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

Impacts.burst [L16-22]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: burst(x: number, y: number, color: number)

Direct writes: p.x, p.y, p.color, p.life

Impacts.draw [L23-30]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: draw(g: Phaser.GameObjects.Graphics, dt: number)

Direct writes: p.life

Calls / delegates: g.lineStyle(3 * t, p.color, t).strokeCircle, g.lineStyle

Decision branches: L25: p.life <= 0

src/shared/interfaces

src/shared/interfaces/MiniGame.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 f5d8458cf933c7e3...

Compatibility re-export of the current shared module interfaces.

Top-level block L2-2

```
export * from "../contracts/GameModule";
```

src/shared/physics

src/shared/physics/FixedStep.ts

ACTIVE SOURCE / tool or application | 24 lines | SHA-256 b489bc600cf248e8...

Advances a bounded 120Hz simulation and exposes remainder/fraction for presentation.

State, resources and initial values

Line	Field / declaration
2	private accumulator = 0;
3	readonly step = 1 / 120;

FixedStep [L1-21]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FixedStep.advance [L4-11]

Consume elapsed time using this helper's fixed-step/clock policy.

Signature: advance(deltaSeconds: number, tick: (dt: number) => void)

Direct writes: this.accumulator

Calls / delegates: tick

FixedStep.reset [L12-14]

Reset this component's transient state for a new round/attempt.

Signature: reset()

Direct writes: this.accumulator

FixedStep.remainder [L15-17]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get remainder()

Return at L16: this.accumulator

```
get remainder() { return this.accumulator; }
```

FixedStep.fraction [L18-20]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get fraction()

Return at L19: Math.min(1, Math.max(0, this.accumulator / this.step))

clamp [L22-23]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clamp = (v: number, min: number, max: number) =>

src/shared/physics/MotionController.ts

ACTIVE SOURCE / tool or application | 45 lines | SHA-256 dfc05c338c389f95...

Converts target positions into continuous speed/acceleration-limited damped motion.

State, resources and initial values

Line	Field / declaration
3	vx = 0;
4	vy = 0;
5	private tx: number;
6	private ty: number;

MotionController [L2-44]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

MotionController.constructor [L7-10]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(x: number, y: number)

Direct writes: this.tx, this.ty

MotionController.reset [L11-15]

Reset this component's transient state for a new round/attempt.

Signature: reset(x: number, y: number)

Direct writes: this.vx, this.vy, this.tx, this.ty

MotionController.move [L16-43]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: move(dt: number, x: number, y: number, targetX: number, targetY: number, speed: number, acceleration = speed * 7,)

Direct writes: this.tx, this.ty, this.vx, this.vy

Return at L42: { x: x + this.vx * dt, y: y + this.vy * dt }

Decision branches: L41: d < 0.3 && Math.hypot(this.vx, this.vy) < 3

src/shared/rules

src/shared/rules/FirstResponse.ts

ACTIVE SOURCE / tool or application | 20 lines | SHA-256 33a961e55821c230...

Arbitrates the first queued response per player with model-time tie tolerance.

State, resources and initial values

Line	Field / declaration
3	private entries: { player: 0 | 1; value: T; time: number }[] = [];

FirstResponse [L2-19]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FirstResponse.add [L4-7]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: add(player: 0 | 1, value: T, time: number)

Calls / delegates: this.entries.some, this.entries.push

Decision branches: L5: this.entries.some((e) => e.player === player)

FirstResponse.resolve [L8-15]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: resolve(now: number)

Direct writes: this.entries

Return at L14: { first, tie }

Decision branches: L10: !first || now - first.time < 0.009

FirstResponse.clear [L16-18]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clear()

Direct writes: this.entries

src/shared/rules/MatchClock.ts

ACTIVE SOURCE / tool or application | 34 lines | SHA-256 a791304380e72c6f...

Owns Pong/Hockey regulation, point target, sudden death, expiry winner and clock text.

Imports: ../contracts/GameResult

State, resources and initial values

Line	Field / declaration
4	remaining = 120;
5	overtime = false;
6	winner: Winner | null = null;

MatchClock [L3-33]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

MatchClock.score [L7-14]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: score(scores: readonly number[])

Direct writes: this.winner

Return at L8: this.winner

Return at L13: this.winner

Decision branches: L8: this.winner !== null; L9: scores[0] >= 5 || scores[1] >= 5; L11: this.overtime && scores[0] !== scores[1]

MatchClock.advance [L15-28]

Consume elapsed time using this helper's fixed-step/clock policy.

Signature: advance(dt: number, scores: readonly number[])

Direct writes: this.remaining, this.winner, this.overtime

Calls / delegates: this.score

Return at L16: this.winner

Return at L27: this.winner

Decision branches: L16: this.score(scores) !== null; L18: this.remaining <= 0; L19: scores[0] !== scores[1]; L20: !this.overtime && scores[0] > 0; L23: this.remaining <= 0; L26: this.winner !== null

MatchClock.text [L29-32]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get text()

Return at L31: `${this.overtime ? "OT " : ""}${Math.floor(t / 60)}:${String(t % 60).padStart(2, "0")}`

src/shared/utilities

src/shared/utilities/WorkerTask.ts

ACTIVE SOURCE / tool or application | 43 lines | SHA-256 7bb5b1de6a145686...

Runs one bounded worker request and terminates CPU work on cancellation/watchdog.

State, resources and initial values

Line	Field / declaration
3	private worker?: Worker;
4	private resolve?: (value: Output | null) => void;
5	private timeout?: ReturnType<typeof setTimeout>;

WorkerTask [L2-42]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

WorkerTask.constructor [L6-9]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private create: () => Worker, private report: (ms: number) => void = () => {},)

```
constructor( private create: () => Worker, private report: (ms: number) => void = () => {}, ) {}
```

WorkerTask.run [L10-33]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: run(input: Input): Promise<Output | null>

Direct writes: this.resolve, this.worker, this.worker.onmessage, this.worker.onerror, this.timeout

Calls / delegates: this.cancel, performance.now, Promise, this.report, this.create, finish, this.worker.postMessage, setTimeout

Return at L13: new Promise((resolve) => { this.resolve = resolve; const finish = (value: Output | null) => { const reported = (value as { milliseconds?: number } | null) ?.milliseconds; this.report(typeof reported === "number" ? reported : performance.now() - start,); this.cancel(value); }; try { this.worker = this.create(); this.worker.onmessage = (e) => finis ... [continued in source reference]

WorkerTask.run.finish [L15-22]

Resolve/report the owned terminal attempt/result using current guards.

Signature: finish = (value: Output | null) =>

Calls / delegates: this.report, performance.now, this.cancel

WorkerTask.cancel [L34-41]

Invalidate/cancel a gesture or asynchronous computation and clear pending state.

Signature: cancel(value: Output | null = null)

Direct writes: this.worker, this.resolve

Calls / delegates: clearTimeout, this.worker?.terminate, resolve

src/styles

src/styles/app.css

ACTIVE SOURCE / tool or application | 1253 lines | SHA-256 f5ddb0f06d16a381...

Presentation selectors and responsive layout rules; detailed selector/property blocks appear below.

Lines	Selector or rule	Declarations
1-11	:root	font-family: "Nunito Sans", sans-serif; color: #242b32; background: #eeece3; font-synthesis: none; -webkit-font-smoothing: antialiased; --ink: #242b32; --paper: #f8f5eb; --orange: #f49b58; --blue: #58addb;
11-14	*	box-sizing: border-box;
14-22	html, body	margin: 0; width: 100%; height: 100%; overflow: hidden; overscroll-behavior: none;
22-28	button	font: inherit; color: inherit; cursor: pointer; -webkit-tap-highlight-color: transparent;
28-32	button:focus-visible	outline: 3px solid #286e9c; outline-offset: 5px;
32-37	button, svg	-webkit-user-select: none; user-select: none;
37-40	button	touch-action: manipulation;
40-43	button:active	transform: translateY(2px) scale(0.98);
43-51	#app	position: relative; width: 100%; height: 100dvh; margin: auto; background: var(--paper); overflow: hidden;
51-62	#game-stage	position: absolute; inset: max(var(--safe-area-inset-top, env(safe-area-inset-top, 0px)), 6px) max(var(--safe-area-inset-right, env(safe-area-inset-right, 0px)), 0px) max(var(--safe-area-inset-bottom, env(safe-area-inset-bottom, 0px)), 6px) max(var(--safe-area-inset-left, env(safe-area-inset-left, 0px)), 0px); visibility: hidden; touch-action: none; user-select: none; -webkit-user-select: none;
62-66	#game-stage canvas	display: block; touch-action: none !important;
66-69	#app.in-game #game-stage	visibility: visible;
69-73	#menu	height: 100%; position: relative;
73-76	#menu[hidden]	display: none;
76-91	.menu-scroll	height: 100%; overflow-y: auto; overscroll-behavior: contain; padding: calc(20px + var(--safe-area-inset-top, env(safe-area-inset-top, 0px))) 24px calc(148px + var(--safe-area-inset-bottom, env(safe-area-inset-bottom, 0px))); scrollbar-width: none; background-image: radial-gradient(#d9d9c8 0.85px, transparent 0.85px); background-size: 17px 17px;
91-94	.menu-scroll::-webkit-scrollbar	display: none;
94-100	.toolbar	display: flex; align-items: center; justify-content: space-between; height: 42px;

Lines	Selector or rule	Declarations
100-105	.wordmark	display: flex; gap: 9px; align-items: center;
105-109	.wordmark img	width: 36px; height: 36px;
109-115	.wordmark span	font-size: 8px; font-weight: 900; letter-spacing: 1px; line-height: 1.4;
115-119	.toolbar-actions	display: flex; gap: 8px;
119-131	.icon-button	border: 2px solid var(--ink); border-radius: 13px; width: 40px; height: 40px; display: grid; place-items: center; background: #fffdf5; box-shadow: 0 2px 0 var(--ink); padding: 9px; transition: transform 0.1s;
131-135	.icon-button svg	width: 20px; height: 20px;
135-140	.hero	position: relative; text-align: center; padding: 29px 0 26px;
140-145	.eyebrow	font-size: 9px; letter-spacing: 1.8px; font-weight: 900;
145-153	.hero h1	font-family: "Fredoka", sans-serif; font-weight: 700; letter-spacing: -3.5px; line-height: 0.83; margin: 13px 0 17px; transform: rotate(-3deg);
153-161	.pocket	display: block; font-size: 62px; color: var(--orange); -webkit-text-stroke: 2.5px var(--ink); paint-order: stroke fill; text-shadow: 2px 3px 0 var(--ink);
161-169	.rivals	display: block; font-size: 85px; color: var(--blue); -webkit-text-stroke: 3px var(--ink); paint-order: stroke fill; text-shadow: 2px 4px 0 var(--ink);
169-172	.title-dot	color: var(--orange);
172-177	.hero p	margin: 0; font-size: 13px; font-weight: 700;
177-183	.hero-spark	position: absolute; font-size: 37px; font-family: Arial, sans-serif; line-height: 1;
183-190	.spark-one	left: 24px; top: 77px; color: var(--orange); transform: rotate(16deg); -webkit-text-stroke: 1px var(--ink);
190-197	.spark-two	right: 25px; top: 137px; color: var(--blue); -webkit-text-stroke: 1px var(--ink); font-size: 28px;
197-206	.hero-badges	display: flex; justify-content: center; align-items: center; gap: 10px; margin: 20px -10px 0; font-size: 10px; font-weight: 900;
206-211	.hero-badges span	display: flex; align-items: center; gap: 5px;
211-215	.hero-badges svg	width: 14px; height: 14px;
215-221	.hero-badges i	height: 3px; width: 3px; border-radius: 50%; background: #a5a89a;

Lines	Selector or rule	Declarations
221-227	.section-label	display: flex; justify-content: space-between; align-items: center; margin: 0 1px 12px;
227-233	.section-label h2	font-family: "Fredoka"; font-size: 21px; letter-spacing: -0.5px; margin: 0;
233-239	.section-label > span	font-size: 8px; letter-spacing: 0.75px; font-weight: 900; color: #778074;
239-244	.section-label b	font-size: 17px; margin-left: 5px; color: var(--ink);
244-249	.game-list	display: flex; flex-direction: column; gap: 17px;
249-267	.game-card	isolation: isolate; text-align: left; position: relative; display: block; width: 100%; height: 181px; border: 3px solid var(--ink); border-radius: 24px; background: var(--card); box-shadow: 0 5px 0 var(--ink); padding: 18px 20px; overflow: hidden; transition: transform 0.13s, box-shadow 0.13s; animation: card-in 0.14s ease-out;
267-271	.game-card:hover	transform: translateY(-2px); box-shadow: 0 7px 0 var(--ink);
271-275	.game-card:active	transform: translateY(4px); box-shadow: 0 1px 0 var(--ink);
275-287	.game-card:after	content: ""; position: absolute; width: 230px; height: 230px; border-radius: 50%; right: -70px; top: -13px; background: var(--accent); opacity: 0.45; z-index: -1;
287-293	.card-copy	position: relative; z-index: 2; display: block; pointer-events: none;
293-302	.card-tag	display: flex; align-items: center; gap: 4px; font-size: 7px; letter-spacing: 1px; font-weight: 900; margin-bottom: 11px;
302-308	.tiny-dot	height: 4px; width: 4px; background: var(--ink); border-radius: 50%;
308-317	.card-copy strong	display: block; font-family: "Fredoka"; font-weight: 700; font-size: 34px; line-height: 0.94; letter-spacing: -0.7px; text-shadow: 0 1px 0 #ffffff38;
317-325	.card-description	display: block; font-size: 9px; max-width: 128px; line-height: 1.3; margin-top: 12px; font-weight: 700;
325-334	.card-art	position: absolute; right: -10px; top: 0; width: 66%; height: 100%; pointer-events: none; z-index: 1;
334-338	.card-art svg	width: 100%; height: 100%;
338-352	.play-circle	position: absolute; right: 14px; bottom: 14px; width: 32px; height: 32px; display: grid; place-items: center; border: 2px solid var(--ink); border-radius: 50%; background: #fffdf5; z-index: 3; box-shadow: 0 2px 0 var(--ink);
352-356	.play-circle svg	width: 18px; height: 18px;
356-366	.card-number	position: absolute; right: 13px; top: 12px; font-size: 8px; font-weight: 900; letter-spacing: 1px; opacity: 0.5; z-index: 4;

Lines	Selector or rule	Declarations
366-370	.game-card:last-child .card-copy strong	font-size: 30px; line-height: 0.95;
370-374	.game-card:last-child .card-description	margin-top: 8px; max-width: 100px;
374-381	.menu-note	text-align: center; font-size: 11px; margin: 29px 0 0; font-weight: 900; color: #677568;
381-388	.menu-note span	display: block; font-size: 10px; font-weight: 700; color: #879183; margin-top: 5px;
388-400	.match-dock	position: absolute; bottom: 0; left: 0; right: 0; padding: 11px 22px calc(14px + var(--safe-area-inset-bottom, env(safe-area-inset-bottom, 0px))); background: #fffdf5; border-top: 2px solid var(--ink); box-shadow: 0 -8px 18px #f8f5ebbd; z-index: 10;
400-410	.dock-caption	font-size: 7px; letter-spacing: 1px; font-weight: 900; display: flex; align-items: center; gap: 5px; margin-bottom: 11px; color: #788172;
410-416	.live-dot	width: 5px; height: 5px; border-radius: 50%; background: #85ac72;
416-421	.match-count	margin-left: auto; font-size: 7px; letter-spacing: 0.6px;
421-427	.match-scores	display: flex; align-items: center; justify-content: space-between; gap: 9px;
427-433	.player-block	display: flex; align-items: center; gap: 8px; flex: 1;
433-437	.player-block.blue	justify-content: flex-end; text-align: right;
437-450	.avatar	border: 2px solid var(--ink); border-radius: 13px; display: grid; place-items: center; width: 34px; height: 37px; font-size: 30px; font-weight: 900; line-height: 1; font-family: Arial; flex-shrink: 0;
450-453	.orange .avatar	background: var(--orange);
453-456	.blue .avatar	background: var(--blue);
456-461	.player-block div strong	display: block; font-size: 8px; letter-spacing: 1px;
461-467	.player-block div span	font-size: 9px; color: #929283; display: block; margin-top: 2px;
467-473	.player-block b	font-family: "Fredoka"; font-size: 29px; font-weight: 600; margin-left: auto;
473-477	.player-block.blue b	margin-right: auto; margin-left: 0;
477-483	.versus	font-size: 8px; color: #979b8e; padding: 0 8px; font-weight: 900;

Lines	Selector or rule	Declarations
483-499	.modal-backdrop	position: absolute; inset: 0; background: #242b3270; backdrop-filter: blur(5px); display: flex; align-items: center; justify-content: center; padding: calc(22px + var(--safe-area-inset-top, env(safe-area-inset-top, 0px))) 22px calc(22px + var(--safe-area-inset-bottom, env(safe-area-inset-bottom, 0px))); z-index: 20; animation: fade-in 0.14s;
499-513	.modal	position: relative; width: 100%; max-width: 370px; max-height: 100%; overflow-y: auto; background: var(--paper); border: 3px solid var(--ink); border-radius: 28px; box-shadow: 0 7px 0 var(--ink); padding: 34px 25px 22px; text-align: center; animation: pop-in 0.22s cubic-bezier(0.15, 0.7, 0.3, 1.15);
513-518	.modal .eyebrow	font-size: 8px; letter-spacing: 1.25px; margin-top: 9px;
518-525	.modal h2	font-family: "Fredoka"; font-size: 31px; line-height: 1.05; letter-spacing: -0.8px; margin: 12px 0;
525-531	.modal p	font-size: 12px; line-height: 1.65; color: #6e7a73; margin: 10px 0 24px;
531-548	.primary-button, .secondary-button	border: 2px solid var(--ink); border-radius: 14px; min-height: 49px; width: 100%; font-size: 14px; font-weight: 900; background: var(--orange); box-shadow: 0 3px 0 var(--ink); padding: 10px 17px; display: flex; align-items: center; justify-content: center; gap: 15px; transition: transform 0.1s;
548-552	.primary-button svg	width: 21px; height: 21px;
552-558	.secondary-button	background: var(--paper); box-shadow: 0 2px 0 var(--ink); font-size: 12px; margin-top: 13px;
558-566	.text-button	border: 0; background: transparent; padding: 19px 8px 3px; font-size: 12px; font-weight: 900; color: #5c6c62;
566-577	.modal-close	position: absolute; right: 15px; top: 15px; width: 30px; height: 30px; border: 0; box-shadow: none; padding: 5px; background: transparent;
577-580	.settings-rows	margin: 20px 0 0;
580-592	.setting-row	display: flex; align-items: center; justify-content: space-between; gap: 10px; width: 100%; background: transparent; border: 0; border-bottom: 1px solid #d8dfd1; text-align: left; padding: 17px 0;
592-596	.setting-row b	display: block; font-size: 13px;
596-602	.setting-row small	display: block; color: #859082; font-size: 9px; margin-top: 4px;
602-611	.toggle	width: 40px; height: 23px; border: 2px solid var(--ink); border-radius: 20px; background: #d5dacb; position: relative; flex-shrink: 0;
611-623	.toggle:after	content: ""; position: absolute; width: 15px; height: 15px; top: 2px; left: 2px; background: var(--paper); border: 1.5px solid var(--ink); border-radius: 50%; transition: transform 0.15s;
623-626	.toggle.on	background: var(--blue);

Lines	Selector or rule	Declarations
626-629	.toggle.on:after	transform: translateX(16px);
629-634	.modal .settings-note	font-size: 10px; text-align: left; margin: 18px 0;
634-639	.modal .version	font-size: 8px; letter-spacing: 1px; margin: 21px 0 0;
639-654	.pause-symbol	font-family: "Fredoka"; font-size: 41px; font-weight: 700; background: #b8d998; border: 3px solid var(--ink); border-radius: 23px; width: 75px; height: 75px; display: grid; place-items: center; margin: 0 auto 20px; transform: rotate(-6deg); box-shadow: 3px 4px 0 var(--ink);
654-665	.result-emblem	width: 94px; height: 94px; display: grid; place-items: center; border-radius: 30px; border: 3px solid var(--ink); margin: 4px auto 24px; transform: rotate(-7deg); box-shadow: 4px 5px 0 var(--ink);
665-668	.result-emblem.orange	background: var(--orange);
668-671	.result-emblem.blue	background: var(--blue);
671-675	.result-emblem.draw	background: #b8d998; font-size: 50px;
675-679	.result-emblem svg	width: 49px; height: 49px;
679-688	.result-score	display: flex; justify-content: center; align-items: center; gap: 25px; font-family: "Fredoka"; font-size: 55px; margin: 10px 0 24px;
688-691	.result-score span:first-child	color: #dc8442;
691-694	.result-score span:last-child	color: #338fbf;
694-698	.result-score small	font-size: 20px; color: #a4ac9a;
698-701	.result-modal	overflow: hidden;
701-709	.result-confetti	position: absolute; top: 22px; left: 20px; right: 20px; height: 150px; pointer-events: none;
709-719	.result-confetti i	position: absolute; width: 9px; height: 16px; left: 10%; top: 30%; background: var(--orange); transform: rotate(-20deg); animation: confetti 1s ease-out both;
719-725	.result-confetti i:nth-child(2)	left: 85%; top: 15%; background: var(--blue); transform: rotate(30deg);
725-731	.result-confetti i:nth-child(3)	left: 20%; top: 70%; background: #b8d998; transform: rotate(60deg);
731-737	.result-confetti i:nth-child(4)	left: 75%; top: 80%; background: var(--orange); transform: rotate(-45deg);

Lines	Selector or rule	Declarations
737-743	.result-confetti i:nth-child(5)	left: 30%; top: 0; background: var(--blue); transform: rotate(70deg);
743-749	.result-confetti i:nth-child(6)	left: 95%; top: 60%; background: #b8d998; transform: rotate(-20deg);
750-753	from	translate: 0 3px;
753-756	to	translate: 0 0;
758-761	from	opacity: 0;
761-764	to	opacity: 1;
766-770	from	opacity: 0; transform: scale(0.95) translateY(9px);
770-774	to	opacity: 1; transform: scale(1) translateY(0);
776-780	from	opacity: 0; translate: 0 -20px;
780-783	to	translate: 0 0;
785-792	body	display: grid; place-items: center; background-color: #e7e9de; background-image: radial-gradient(#c8cfc2 1.2px, transparent 1.2px); background-size: 23px 23px;
792-799	#app	max-width: 440px; height: min(960px, calc(100dvh - 40px)); border: 3px solid var(--ink); border-radius: 32px; box-shadow: 9px 10px 0 #242b3220;
799-802	.menu-scroll	padding-top: 22px;
802-806	.hero	padding-top: 34px; padding-bottom: 29px;
808-812	.menu-scroll	padding-left: 17px; padding-right: 17px;
812-815	.wordmark span	font-size: 7px;
815-819	.hero-badges	gap: 7px; font-size: 9px;
819-822	.card-art	right: -24px;
822-825	.card-copy strong	font-size: 31px;
825-829	.match-dock	padding-left: 15px; padding-right: 15px;

Lines	Selector or rule	Declarations
829-832	.player-block	gap: 5px;
832-835	.versus	padding: 0 4px;
835-838	.spark-one	left: 7px;
838-841	.spark-two	right: 8px;
841-845	.modal	padding-left: 19px; padding-right: 19px;
847-853	#app	border: 0; border-radius: 0; max-width: none; height: 100dvh;
853-857	.menu-scroll	max-width: 460px; margin: auto;
857-862	.match-dock	max-width: 460px; left: 50%; transform: translateX(-50%);
862-866	.modal	max-width: 460px; padding-top: 18px;
866-870	.pause-symbol, .result-emblem	display: none;
870-873	.modal p	margin-bottom: 12px;
873-877	.result-score	font-size: 32px; margin: 4px 0 10px;
879-885	*, *:before, *:after	animation-duration: 0.01ms !important; transition-duration: 0.01ms !important;
886-892	.collection-modes	display: flex; gap: 8px; margin: 0 0 21px;
892-906	.collection-modes button	flex: 1; display: flex; align-items: center; justify-content: center; gap: 7px; padding: 12px 8px; background: #fffdf5; border: 2px solid var(--ink); border-radius: 13px; font-size: 12px; font-weight: 900; box-shadow: 0 2px 0 var(--ink);
906-909	.collection-modes button.selected	background: #f6d37f;
909-913	.collection-modes svg	width: 17px; height: 17px;
913-925	.resume-banner	display: flex; align-items: center; justify-content: space-between; width: 100%; padding: 13px 10px; border: 2px solid var(--ink); border-radius: 12px; background: #e0ebce; font-size: 11px; margin: -5px 0 16px;
925-928	.resume-banner b	font-size: 9px;

Lines	Selector or rule	Declarations
928-937	.segmented	display: flex; background: #e9e9da; border: 2px solid var(--ink); border-radius: 14px; padding: 4px; gap: 3px; margin: 14px 0;
937-947	.segmented button	flex: 1; border: 0; background: transparent; border-radius: 9px; padding: 10px 4px; font-size: 12px; font-weight: 900; min-height: 40px;
947-951	.segmented button.selected	background: #fffdf5; box-shadow: 0 2px 0 #242b3230;
951-958	.segmented button small	display: block; font-size: 9px; font-weight: 700; color: #7e8678; margin-top: 3px;
958-961	.players button	padding: 12px 4px;
961-964	.difficulty button	font-size: 10px;
964-971	.choice-label	text-align: left; font-size: 8px; letter-spacing: 1px; font-weight: 900; margin-top: 20px;
971-976	.modal .setup-note	font-size: 11px; line-height: 1.6; margin: 17px 0 21px;
976-981	.variant-choices	display: flex; gap: 8px; margin: 20px 0;
981-994	.variant-choice	flex: 1; min-height: 75px; border: 2px solid var(--ink); border-radius: 13px; background: transparent; padding: 10px 7px; display: flex; flex-direction: column; align-items: center; justify-content: center; text-align: center;
994-998	.variant-choice.selected	background: var(--blue); box-shadow: 0 3px 0 var(--ink);
998-1004	.variant-choice .variant-preview-icon	display: flex; align-items: center; justify-content: center; margin-bottom: 5px;
1004-1007	.variant-choice .variant-preview-icon svg	display: block;
1007-1011	.variant-choice b	font-size: 12px; display: block;
1011-1017	.variant-choice small	font-size: 9px; display: block; margin-top: 6px; line-height: 1.3;
1017-1020	.setup-modal .text-button	margin: 0 7px;
1020-1028	.round-control	display: flex; align-items: center; justify-content: space-between; font-size: 12px; font-weight: 900; margin: 18px 0;
1028-1037	.round-control select	width: 67px; border: 2px solid var(--ink); border-radius: 10px; background: #fffdf5; padding: 9px; font: inherit; color: inherit;
1037-1042	.game-picks	display: grid; grid-template-columns: 1fr 1fr; gap: 6px;
1042-1051	.game-picks button	border: 2px solid #b4bdac; border-radius: 10px; background: #fffdf5; text-align: left; font-size: 10px; min-height: 43px; padding: 7px;

Lines	Selector or rule	Declarations
1051-1055	.game-picks button.selected	border-color: var(--ink); background: #cfe3b7;
1055-1067	.tournament-cup	display: grid; place-items: center; margin: 0 auto 20px; background: #f6d37f; width: 65px; height: 65px; border-radius: 21px; border: 3px solid var(--ink); box-shadow: 3px 4px 0 var(--ink); transform: rotate(-6deg);
1067-1071	.tournament-cup svg	width: 35px; height: 35px;
1071-1078	.tournament-score	display: flex; align-items: center; justify-content: center; gap: 25px; margin: 20px 0;
1078-1083	.tournament-score div small	display: block; font-size: 9px; letter-spacing: 1px;
1083-1086	.tournament-score b	font: 600 47px Fredoka;
1086-1089	.tournament-score div:first-child b	color: #d57e3d;
1089-1092	.tournament-score div:last-child b	color: #2f8bb9;
1092-1095	.tournament-score > span	color: #a0a892;
1095-1102	.round-dots	display: flex; justify-content: center; flex-wrap: wrap; gap: 7px; margin-bottom: 14px;
1102-1108	.round-dots i	width: 9px; height: 9px; border-radius: 50%; background: #dde0d2;
1108-1111	.round-dots i.done	background: #8aad74;
1111-1115	.round-dots i.current	background: var(--orange); outline: 2px solid var(--ink);
1115-1122	.round-history	max-height: 100px; overflow-y: auto; text-align: left; margin: 16px 0; font-size: 10px;
1122-1128	.round-history span	display: flex; justify-content: space-between; padding: 6px 0; border-top: 1px solid #d8decb;
1128-1132	.tutorial-diagram	max-width: 170px; margin: 15px auto;
1132-1139	.tutorial-route	display: flex; align-items: center; justify-content: center; gap: 12px; margin: 22px 0;
1139-1142	.tutorial-route > svg	width: 125px;
1142-1148	.tutorial-cells	width: 115px; display: grid; grid-template-columns: repeat(3, 1fr); gap: 3px;
1148-1156	.tutorial-cells button	height: 38px; border: 2px solid var(--ink); border-radius: 5px; background: #fffdf5; font-size: 24px; line-height: 1;

Lines	Selector or rule	Declarations
1156-1159	.tutorial-cells button.selected	background: var(--orange);
1159-1164	button:disabled	opacity: 0.4; cursor: not-allowed; transform: none;
1164-1167	.tournament-setup h2	font-size: 29px;
1167-1170	.tournament-results h2	font-size: 29px;
1170-1183	.game-transition	position: absolute; inset: 0; z-index: 15; background: #f8f5eb; display: flex; flex-direction: column; align-items: center; justify-content: center; gap: 12px; color: #697474; font-size: 13px;
1183-1186	.game-transition[hidden]	display: none;
1186-1191	.transition-mark	font: 700 72px Fredoka; color: #f49b58; letter-spacing: -7px;
1191-1194	.transition-mark span	color: #58addb;
1194-1198	.collection-modes button	font-size: 12px; min-height: 44px;
1198-1201	.collection-modes	gap: 5px;
1201-1208	.arcade-dock	display: flex; align-items: center; justify-content: space-between; padding: 22px 26px; gap: 16px;
1208-1212	.arcade-dock strong	font-family: Fredoka; font-size: 20px;
1212-1215	.arcade-dock small	color: #697474;
1215-1229	.diagnostics	position: fixed; top: 8px; left: 8px; max-width: 320px; white-space: pre-wrap; z-index: 100; pointer-events: none; font: 10px/1.4 monospace; background: #242b32e8; color: #fff9ee; border-radius: 8px; padding: 10px;
1229-1234	.arcade-result-score	font: 700 80px Fredoka; color: #58addb; margin: 10px;
1234-1237	.result-modal:has(.result-emblem.orange)	background: #f7e1cc;
1237-1238	.result-modal:has(.result-emblem.blue)	background: #d9eaf4;
1238-1239	.result-modal:has(.result-emblem.orange) .result-emblem	box-shadow: 0 0 0 12px #f5a05a55, 4px 5px 0 var(--ink);
1239-1240	.result-modal:has(.result-emblem.blue) .result-emblem	box-shadow: 0 0 0 12px #58add555, 4px 5px 0 var(--ink);
1240-1248	.variant-choices:has(.map-preview)	display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 10px; margin: 14px 0;

Lines	Selector or rule	Declarations
1248-1249	.variant-choice:has(.map-preview)	min-width: 0; padding: 9px;
1249-1250	.variant-choice:has(.map-preview) b	font-size: 14px;
1250-1251	.variant-choice:has(.map-preview) small	font-size: 10px;
1251-1252	.variant-preview-icon .map-preview	width: 72px; height: 72px;

src/tournament

src/tournament/TournamentScene.ts

ACTIVE SOURCE / tool or application | 127 lines | SHA-256 680a52038e3f76d6...

Presents tournament pool/setup, progress, champion and rematch UI.

Imports: ../core/GameManager, ../core/GameRegistry, ../core/TournamentManager, ../scenes/GameSetup, ../shared/contracts/AIController, ../shared/contracts/GameModule, ../shared/components/icons

TournamentScene [L8-126]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TournamentScene.constructor [L9-9]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private app: GameManager)

```
constructor(private app: GameManager) {}
```

TournamentScene.setup [L10-87]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: setup()

Direct writes: mode, manual, actions[`difficulty-${d}`], difficulty, actions[`choose-${e.key}`], root.querySelector("select")!.onchange, rounds

Calls / delegates: Set, tournamentGames.flatMap, (g.variants ?? [{ id: undefined, name: g.name }]) .filter((v) => mode === "local" || (g.supportedModes.includes("single-player") && (!("ai" in v) || v.ai !== false)),) .map, (g.variants ?? [{ id: undefined, name: g.name }]) .filter, g.supportedModes.includes, v.name.split, entries, this.app.modal.close, render, pool.filter, selected.has, this.app.tournament.create, shuffledSchedule, chosen.map, this.app.showTournament, selected.delete, selected.add, this.app.modal.show, difficultyButtons, pool.map((e) => `<button role="checkbox" aria-checked="${selected.has(e.key)}" data-action="choose-${e.key}" class="${selected.has(e.key) ? "selected" : ""}">${selected.has(e.key) ? "✓" : "○"} ${e.name}</button>`).join, pool.map, pool.some, root.querySelector, Number

Decision branches: L56: !chosen.length; L73: selected.has(e.key)

Execution order (direct statements; nested callbacks stay within their owner):

L11-14 FirstStatement: let mode: "local" | "ai" = "local", difficulty: Difficulty = "medium",

L15-15 FirstStatement: const selected = new Set<string>();

L16-33 FirstStatement: const entries = () => tournamentGames.flatMap((g) =>

L34-85 FirstStatement: const render = () => { const pool = entries();

L86-86 ExpressionStatement: render();

TournamentScene.setup.entries [L16-33]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: entries = () =>

Calls / delegates: tournamentGames.flatMap, (g.variants ?? [{ id: undefined, name: g.name }]) .filter((v) => mode === "local" || (g.supportedModes.includes("single-player") && (!("ai" in v) || v.ai !== false)),) .map, (g.variants ?? [{ id: undefined, name: g.name }]) .filter, g.supportedModes.includes, v.name.split

TournamentScene.setup.render [L34-85]

Build or refresh visual/DOM output from current state.

Signature: render = () =>

Direct writes: mode, manual, actions[`difficulty-${d}`], difficulty, actions[`choose-${e.key}`], root.querySelector("select")!.onchange, rounds

Calls / delegates: entries, this.app.modal.close, render, pool.filter, selected.has, this.app.tournament.create, shuffledSchedule, chosen.map, this.app.showTournament, selected.delete, selected.add, this.app.modal.show, difficultyButtons, pool.map((e) => `<button role="checkbox" aria-checked="${selected.has(e.key)}" data-action="choose-${e.key}" class="${selected.has(e.key) ? "selected" : ""}">${selected.has(e.key) ? "✓" : "○"} ${e.name}</button>`).join, pool.map, pool.some, root.querySelector, Number

Decision branches: L56: !chosen.length; L73: selected.has(e.key)

Execution order (direct statements; nested callbacks stay within their owner):

L35-35 FirstStatement: const pool = entries();

L36-65 FirstStatement: const actions: Record<string, () => void> = { close: () => this.app.modal.close(),

L66-70 ForOfStatement: for (const d of ["medium", "hard", "impossible"] as const) actions[`difficulty-${d}`] = () => {

L71-76 ForOfStatement: for (const e of pool) actions[`choose-${e.key}`] = () => {

L77-81 FirstStatement: const root = this.app.modal.show(` A PROPER LITTLE SHOWDOWN <h2 id="dialog-title">Make it a tournament.</h2> <button data-action="local" class="

L82-84 ExpressionStatement: root.querySelector("select")!.onchange = (e) => { rounds = Number((e.target as HTMLSelectElement).value);

TournamentScene.show [L88-125]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: show()

Direct writes: nextName

Calls / delegates: this.setup, this.app.tournament.current, getGame, s.results.at, this.app.startTournamentRound, this.app.goMenu, this.app.tournament.rematch, this.app.showTournament, this.app.modal.show, s.plan.map((_, i) => `<i class="${i < s.results.length ? "done" : i === s.results.length ? "current" : ""}"></i>`).join, s.plan.map, s.results.map((r, i) => `<span>Round ${i + 1}<b>${r.winner === "draw" ? "Draw · ½ each" : r.winner === 1 ? "Orange +1" : blue + " +1"}</b></span>`).join, s.results.map

Decision branches: L90: !s; L105: next

Execution order (direct statements; nested callbacks stay within their owner):

L89-89 FirstStatement: const s = this.app.tournament.state;

L90-93 IfStatement: if (!s) { this.setup();

L94-95 FirstStatement: const complete = s.phase === "complete", blue = s.plan[0].options.mode === "ai" ? "BLUE AI" : "BLUE";

L96-101 FirstStatement: const champion = s.points[0] === s.points[1]

L102-102 FirstStatement: const next = this.app.tournament.current();

L103-103 FirstStatement: let nextName = "";

L104-108 TryStatement: try { if (next) nextName = getGame(next.gameId).name;

L109-109 FirstStatement: const last = s.results.at(-1);

L110-118 FirstStatement: const actions = { play: () => this.app.startTournamentRound(),

L119-124 ExpressionStatement: this.app.modal.show(`<span class="tournament-cup">${icons.cup}</span><div class="eyebrow">${complete ? "THE FINAL SCORE" : `ROUND ${s.results.length + 1} OF ${s.plan.length}`} <h2 id="dialog-tit

src/games/air-hockey

src/games/air-hockey/AirHockeyAI.ts

ACTIVE SOURCE / tool or application | 123 lines | SHA-256 e00f991397025d33...

Selects legal interception and behind-puck attack targets through the shared motion controller.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./AirHockeyPhysics, ../../shared/physics/MotionController, ../../shared/physics/FixedStep, ../../core/ThemeManager

State, resources and initial values

Line	Field / declaration
16	readonly personality: MatchRandom;
17	private approachSide = 1;
18	private observedOpponent = 210;
26	private motion = new MotionController(210, 183);
27	private attacking = false;
28	private error = 0;
29	private reaction = 0;
30	private target: Vec = { x: 210, y: 180 };

reflect [L10-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: function reflect(x: number)

Return at L13: 35 + (phase <= width ? phase : 2 * width - phase)

```
function reflect(x: number) { const width = 350, phase = (((x - 35) % (2 * width)) + 2 * width) % (2 * width); return 35 + (phase <= width ? phase : 2 * width - phase); }
```

AirHockeyAI [L15-122]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

AirHockeyAI.snapshot [L19-25]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L20: { ...this.personality.snapshot(), target: { ...this.target }, attacking: this.attacking, }

AirHockeyAI.constructor [L31-37]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: SeedInput,)

Direct writes: this.personality, this.approachSide

Calls / delegates: MatchRandom

AirHockeyAI.reset [L38-43]

Reset this component's transient state for a new round/attempt.

Signature: reset()

Direct writes: this.reaction, this.target, this.attacking

Calls / delegates: this.motion.reset

AirHockeyAI.update [L44-121]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, w: HockeyWorld)

Direct writes: this.reaction, this.target, this.attacking, this.observedOpponent, this.approachSide, this.error, this.target.x, this.target.y

Calls / delegates: this.personality.sample, reflect, clamp, this.motion.move

Return at L45: { x: w.mallets[1].x, y: w.mallets[1].y }

Return at L120: { x: clamp(next.x, 53, 367), y: clamp(next.y, 141, 352) }

Decision branches: L45: w.done; L54: w.cooldown > 0; L58: this.reaction <= 0 && w.cooldown <= 0; L67: p.vy < -60 && p.y < 480; L70: p.y < FIELD.middle + 10; L72: m.y < p.y - 43; L73: m.y > p.y + 30; L74: this.attacking; L83: !this.attacking

Execution order (direct statements; nested callbacks stay within their owner):

L45-45 IfStatement: if (w.done) return { x: w.mallets[1].x, y: w.mallets[1].y };

L46-50 FirstStatement: const cfg = { medium: { delay: 0.19, error: 24, speed: 530 },

L51-52 FirstStatement: const p = w.puck, m = w.mallets[1];

L53-53 ExpressionStatement: this.reaction -= dt;

L54-57 IfStatement: if (w.cooldown > 0) { this.target = { x: 210, y: 183 };

L58-111 IfStatement: if (this.reaction <= 0 && w.cooldown <= 0) { this.reaction = this.personality.sample(

L112-119 FirstStatement: const next = this.motion.move(dt,

L120-120 ReturnStatement: return { x: clamp(next.x, 53, 367), y: clamp(next.y, 141, 352) };

src/games/air-hockey/AirHockeyGame.ts

ACTIVE SOURCE / tool or application | 226 lines | SHA-256 4bcb9ae80da17097...

Air Hockey lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/ai/MatchRandom, ./HockeyState, ./PossessionClock, ../../shared/rules/MatchClock, ./AirHockeyAI, phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/physics/FixedStep, ../../shared/components/drawing, ../../shared/effects/Impacts, ./AirHockeyPhysics

State, resources and initial values

Line	Field / declaration
18	private ctx!: GameContext;
19	private world!: HockeyWorld;
20	private clock = new FixedStep();
21	private matchClock = new MatchClock();
22	private graphics!: Phaser.GameObjects.Graphics;
23	private bg!: Phaser.GameObjects.Graphics;
24	private status!: Phaser.GameObjects.Text;
25	private effects = new Impacts();
26	private possession = new PossessionClock();
27	private possessionLabel!: Phaser.GameObjects.Text;
28	private violation = false;
29	private sinceSave = 0;
30	private paused = false;
31	private ai?: AirHockeyAI;
32	private keys = new Set<string>();

AirHockey [L17-225]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

AirHockey.initialize [L33-81]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.ai, this.keys, this.bg, this.graphics, this.possessionLabel, this.status, this.world

Calls / delegates: AirHockeyAI, restoredAISeed, ctx.input.dragHalves, ctx.input.keys, ctx.setHint, ctx.scene.add.graphics, rounded, g.lineStyle(2, 0x90b9bf).strokeRoundedRect, g.lineStyle, g.lineStyle(3, 0x90b9bf).lineBetween, g.strokeCircle, g.fillStyle(theme.blue, 0.15).fillCircle, g.fillStyle, g.fillStyle(theme.orange, 0.15).fillCircle, label, HockeyWorld, this.effects.burst, ctx.audio.play, this.goal, restoreHockey, ctx.setScore, this.save

Decision branches: L35: ctx.options.mode === "ai"

Execution order (direct statements; nested callbacks stay within their owner):

L34-34 ExpressionStatement: this.ctx = ctx;

L35-39 IfStatement: if (ctx.options.mode === "ai") this.ai = new AirHockeyAI(

L40-40 ExpressionStatement: ctx.input.dragHalves();

L41-50 ExpressionStatement: this.keys = ctx.input.keys(["KeyW",

L51-51 ExpressionStatement: ctx.setHint("");

L52-52 ExpressionStatement: this.bg = ctx.scene.add.graphics();

L53-53 FirstStatement: const g = this.bg;

L54-54 ExpressionStatement: rounded(g, 24, 112, 372, 538, 44, theme.ice, 4);

L55-55 ExpressionStatement: g.lineStyle(2, 0x90b9bf).strokeRoundedRect(40, 128, 340, 506, 12);

L56-56 ExpressionStatement: g.lineStyle(3, 0x90b9bf).lineBetween(25, 381, 395, 381);

L57-57 ExpressionStatement: g.strokeCircle(210, 381, 55);

L58-58 ExpressionStatement: g.strokeCircle(210, 381, 6);

L59-59 ExpressionStatement: g.fillStyle(theme.blue, 0.15).fillCircle(210, 145, 52);

L60-60 ExpressionStatement: g.fillStyle(theme.orange, 0.15).fillCircle(210, 617, 52);

L61-61 ExpressionStatement: rounded(g, 147, 102, 126, 18, 7, theme.blue, 3);

L62-62 ExpressionStatement: rounded(g, 147, 642, 126, 18, 7, theme.orange, 3);

L63-63 ExpressionStatement: this.graphics = ctx.scene.add.graphics();

L64-64 ExpressionStatement: this.possessionLabel = label(ctx.scene, 340, 140, "", 12);

L65-65 ExpressionStatement: this.status = label(ctx.scene, 210, 300, "READY?", 28);

L66-72 ExpressionStatement: this.world = new HockeyWorld((x, y) => {

L73-78 ExpressionStatement: restoreHockey(this.world,

L79-79 ExpressionStatement: ctx.setScore(...this.world.scores);

L80-80 ExpressionStatement: this.save();

AirHockey.goal [L82-92]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private goal(p: Player)

Direct writes: this.world.done

Calls / delegates: this.possession.reset, this.ai?.reset, this.ctx.setScore, this.ctx.audio.play, this.matchClock.score, this.ctx.finish

Decision branches: L88: winner !== null

AirHockey.update [L93-156]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: w.done, this.violation, this.sinceSave

Calls / delegates: this.matchClock.advance, this.ctx.setClock, this.ctx.setHint, this.ctx.finish, this.clock.advance, w.step, this.malletTarget, this.ai.update, this.possession.advance, w.awardPoint, this.possessionLabel .setText(this.possession.side ? `${Math.ceil(this.possession.remaining)}s` : "",) .setPosition(348, this.possession.side === 1 ? 619 : 143) .setColor(this.possession.remaining <= 5 ? "#ae432c" : "#62858b") .setAlpha, this.possessionLabel .setText(this.possession.side ? `${Math.ceil(this.possession.remaining)}s` : "",) .setPosition(348, this.possession.side === 1 ? 619 : 143) .setColor, this.possessionLabel .setText(this.possession.side ? `${Math.ceil(this.possession.remaining)}s` : "",) .setPosition, this.possessionLabel .setText, this.save, g.clear, w.mallets.forEach, g.fillStyle(theme.ink, 0.12).fillCircle, g.fillStyle, circle, g.lineStyle(3, theme.ink, 0.2).strokeCircle, g.lineStyle, g.lineStyle(2, theme.white, 0.7) .beginPath() .arc(m.x, m.y - 2, 8, 3.5, 5) .strokePath, g.lineStyle(2, theme.white, 0.7) .beginPath() .arc, g.lineStyle(2, theme.white, 0.7) .beginPath, g.fillStyle(0x5b6770).fillCircle, this.effects.draw, this.status.setText

Decision branches: L94: this.paused; L98: this.matchClock.overtime; L100: winner !== null; L110: w.cooldown <= 0 && !w.done; L112: penalty; L118: w.cooldown <= 0; L127: this.sinceSave >= 1

Execution order (direct statements; nested callbacks stay within their owner):

L94-94 IfStatement: if (this.paused) return;

L95-95 FirstStatement: const w = this.world;

L96-96 FirstStatement: const winner = this.matchClock.advance(dt, w.scores);

L97-97 ExpressionStatement: this.ctx.setClock(this.matchClock.text, this.matchClock.remaining <= 10);

L98-99 IfStatement: if (this.matchClock.overtime) this.ctx.setHint("SUDDEN DEATH · NEXT POINT WINS");

L100-104 IfStatement: if (winner !== null) { w.done = true;

L105-117 ExpressionStatement: this.clock.advance(dt, (step) => { w.step(step, [

L118-118 IfStatement: if (w.cooldown <= 0) this.violation = false;

L119-125 ExpressionStatement: this.possessionLabel .setText(

L126-126 ExpressionStatement: this.sinceSave += dt;

L127-130 IfStatement: if (this.sinceSave >= 1) { this.sinceSave = 0;

L131-131 FirstStatement: const g = this.graphics;

L132-132 ExpressionStatement: g.clear();

L133-142 ExpressionStatement: w.mallets.forEach((m, i) => { g.fillStyle(theme.ink, 0.12).fillCircle(m.x, m.y + 5, 30);

L143-143 ExpressionStatement: g.fillStyle(theme.ink, 0.12).fillCircle(w.puck.x, w.puck.y + 4, 12);

L144-144 ExpressionStatement: circle(g, w.puck.x, w.puck.y, 11, theme.ink, 0);

L145-145 ExpressionStatement: g.fillStyle(0x5b6770).fillCircle(w.puck.x - 2, w.puck.y - 3, 4);

L146-146 ExpressionStatement: this.effects.draw(g, dt);

L147-155 ExpressionStatement: this.status.setText(w.cooldown > 0.5

AirHockey.save [L157-163]

Create/persist the component checkpoint or record described by the returned expression.

Signature: private save()

Calls / delegates: this.ctx.saveState, saveHockey, this.ai?.snapshot

Decision branches: L158: !this.world.done

AirHockey.malletTarget [L164-193]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private malletTarget(player: 0 | 1, step: number): Vec

Direct writes: vx, vy

Calls / delegates: this.keys.has, Number

Return at L166: pointer

Return at L183: m

Return at L189: { x: m.x + vx * speed * step, y: m.y + vy * speed * step, }

Decision branches: L166: pointer.down; L183: vx === 0 && vy === 0

AirHockey.pause [L194-198]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.save, this.clock.reset

AirHockey.resume [L199-202]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

Calls / delegates: this.clock.reset

AirHockey.dispose [L203-208]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.bg.destroy, this.graphics.destroy, this.status.destroy, this.possessionLabel.destroy

AirHockey.snapshot [L209-224]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot, this.world.mallets.map

Return at L210: { ai: this.ai?.snapshot(), possession: { side: this.possession.side, remaining: this.possession.remaining, }, puck: { ...this.world.puck }, mallets: this.world.mallets.map((m) => ({ ...m })), timer: { remaining: this.matchClock.remaining, overtime: this.matchClock.overtime, }, scores: [...this.world.scores], }

src/games/air-hockey/AirHockeyPhysics.ts

ACTIVE SOURCE / tool or application | 227 lines | SHA-256 0d307519d23f6c04...

Simulates puck/mallet contacts, substeps, posts, pinches, speed cap and unreachable-state repairs.

Imports: ../../core/ThemeManager, ../../shared/physics/FixedStep, ../../shared/contracts/GameResult, ./rink

State, resources and initial values

Line	Field / declaration
17	puck: Body = { x: 210, y: 381, vx: 75, vy: 220, r: 11 };
18	mallets: [Body, Body] = [{ x: 210, y: 579, vx: 0, vy: 0, r: 29 }, { x: 210, y: 183, vx: 0, vy: 0, r: 29 },];
22	scores: [number, number] = [0, 0];
23	cooldown = 1.35;
24	done = false;
25	readonly goalHalf = GOAL_HALF;
26	recoveries = 0;
27	private invalidFor = 0;

Vec [L12-15]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Vec { x: number; y: number; }
```

HockeyWorld [L16-226]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

HockeyWorld.constructor [L28-31]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private impact: (x: number, y: number) => void = () => {}, private scored: (p: Player) => void = () => {},)

```
constructor( private impact: (x: number, y: number) => void = () => {}, private scored: (p: Player) => void = () => {}, ) {}
```

HockeyWorld.step [L32-97]

Interpolate mallets across three substeps, damp/move the puck, project contacts, score goals and inspect unreachable states.

Signature: step(dt: number, targets: [Vec, Vec])

Direct writes: m.x, m.y, m.vx, m.vy, p.vx, p.vy, p.x, p.y, this.cooldown

Calls / delegates: this.checkUnplayable, this.limitSpeed, this.mallets.map, clamp, constrainRink, this.mallets.forEach, this.malletContact, this.impact, this.posts, this.solvePinch, inMouth, this.awardPoint

Return at L56: end

Decision branches: L33: this.done; L34: !Number.isFinite(this.puck.x + this.puck.y + this.puck.vx + this.puck.vy); L70: this.cooldown > 0; L81: constrainRink(p, true) && iteration === 0; L86: inMouth(p.x, p.r) && p.y < FIELD.top - p.r; L90: inMouth(p.x, p.r) && p.y > FIELD.bottom + p.r; L95: this.cooldown > 0

Execution order (direct statements; nested callbacks stay within their owner):

L33-33 IfStatement: if (this.done) return;

L34-39 IfStatement: if (!Number.isFinite(this.puck.x + this.puck.y + this.puck.vx + this.puck.vy)

L40-40 ExpressionStatement: this.limitSpeed();

L41-41 FirstStatement: const sub = dt / 3;

L42-42 FirstStatement: const starts = this.mallets.map((m) => ({ x: m.x, y: m.y }));

L43-57 FirstStatement: const destinations = this.mallets.map((m, i) => { const tx = clamp(targets[i].x, FIELD.left + m.r, FIELD.right - m.r),

L58-94 ForStatement: for (let n = 0; n < 3; n++) { this.mallets.forEach((m, i) => {

L95-96 IfStatement: if (this.cooldown > 0) this.cooldown = Math.max(0, this.cooldown - dt); else this.checkUnplayable(dt);

HockeyWorld.malletContact [L98-115]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private malletContact(m: Body, impulse: boolean)

Direct writes: p.x, p.y, p.vx, p.vy

Calls / delegates: this.impact

Decision branches: L104: d >= r; L110: impulse && approach < 0

HockeyWorld.posts [L116-136]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private posts(impulse: boolean)

Direct writes: p.x, p.y, p.vx, p.vy

Calls / delegates: this.impact

Decision branches: L124: d >= r; L130: impulse && approach < 0

HockeyWorld.solvePinch [L137-177]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private solvePinch()

Direct writes: best, bx, by, p.x, p.y, p.vx, p.vy

Calls / delegates: validPuckPosition, this.mallets.some

Decision branches: L141: Math.hypot(p.x - m.x, p.y - m.y) >= r - 0.08; L149: !validPuckPosition(x, y, p.r) || this.mallets.some((other) => other !== m && Math.hypot(x - other.x, y - other.y) < other.r + p.r,); L159: d < best; L165: best < Infinity; L171: dot < 0

Execution order (direct statements; nested callbacks stay within their owner):

L138-138 FirstStatement: const p = this.puck;

L139-176 ForOfStatement: for (const m of this.mallets) { const r = m.r + p.r + 0.05;

HockeyWorld.checkUnplayable [L178-200]

Repair only invalid/unreachable puck states after sustained detection; reachable stationary pucks remain legal.

Signature: private checkUnplayable(dt: number)

Direct writes: this.invalidFor, p.x, p.y, p.vx, p.vy

Calls / delegates: validPuckPosition, reachablePuck, constrainRink, this.solvePinch, clamp

Decision branches: L184: this.invalidFor < 0.75; L188: !Number.isFinite(p.x + p.y + p.vx + p.vy); L196: !reachablePuck(p.x, p.y, p.r)

HockeyWorld.limitSpeed [L201-208]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private limitSpeed()

Direct writes: p.vx, p.vy

Decision branches: L204: s > 1100

HockeyWorld.awardPoint [L209-225]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: awardPoint(player: Player)

Direct writes: this.done, this.cooldown, this.invalidFor

Calls / delegates: this.mallets.forEach, this.scored

Decision branches: L210: this.done

src/games/air-hockey/HockeyState.ts

ACTIVE SOURCE / tool or application | 89 lines | SHA-256 a3134d50d878c111...

Serializes and validates puck/mallet geometry, scores, match/possession clocks and AI seed.

Imports: ./AirHockeyPhysics, ../../shared/rules/MatchClock, ./PossessionClock, ./rink

saveHockey [L5-19]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function saveHockey(w: HockeyWorld, clock: MatchClock, poss: PossessionClock,)

Calls / delegates: w.mallets.map

Return at L10: { version: 1, puck: { ...w.puck }, mallets: w.mallets.map((m) => ({ ...m })), scores: [...w.scores], cooldown: w.cooldown, clock: { remaining: clock.remaining, overtime: clock.overtime }, possession: { side: poss.side, elapsed: poss.elapsed }, }

restoreHockey [L20-88]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function restoreHockey(w: HockeyWorld, clock: MatchClock, poss: PossessionClock, value: unknown,)

Direct writes: w.scores, w.cooldown

Calls / delegates: d.scores.every, [null, 1, 2].includes, validPuckPosition, d.mallets.every, w.mallets.forEach

Decision branches: L27: !d || d.version !== 1 || !d.puck || !d.clock || !d.possession || !Array.isArray(d.mallets) || d.mallets.length !== 2 || !Array.isArray(d.scores) || d.scores.length !== 2; L39: !d.scores.every((n) => Number.isInteger(n) && n >= 0 && n < 5) || !Number.isFinite(d.cooldown) || d.cooldown < 0 || d.cooldown > 1.35; L46: typeof d.clock.overtime !== "boolean" || !Number.isFinite(d.clock.remaining) || d.clock.remaining <= 0 || d.clock.remaining > (d.clock.overtime ? 60 : 120); L53: ![null, 1, 2].includes(d.possession.side) || !Number.isFinite(d.possession.elapsed) || d.possession.elapsed < 0 || d.possession.elapsed >= 15; L61: !Number.isFinite(p.x + p.y + p.vx + p.vy) || !validPuckPosition(p.x, p.y, 11) || Math.hypot(p.vx, p.vy) > 1100.01; L67: !d.mallets.every((m, i) => m && Number.isFinite(m.x + m.y) && m.x >= 53 && m.x <= 367 && m.y >= (i === 0 ? 410 : 141) && m.y <= (i === 0 ? 621 : 352),)

Execution order (direct statements; nested callbacks stay within their owner):

L26-26 FirstStatement: const d = value as ReturnType<typeof saveHockey>;

L27-38 IfStatement: if (!d ||

L39-45 IfStatement: if (!d.scores.every((n) => Number.isInteger(n) && n >= 0 && n < 5) ||

L46-52 IfStatement: if (typeof d.clock.overtime !== "boolean" ||

L53-59 IfStatement: if (![null, 1, 2].includes(d.possession.side) ||

L60-60 FirstStatement: const p = d.puck;

L61-66 IfStatement: if (!Number.isFinite(p.x + p.y + p.vx + p.vy) ||

L67-78 IfStatement: if (!d.mallets.every(

L79-79 ExpressionStatement: Object.assign(w.puck, p, { r: 11 });

L80-82 ExpressionStatement: w.mallets.forEach((m, i) => Object.assign(m, { x: d.mallets[i].x, y: d.mallets[i].y, vx: 0, vy: 0 }),

L83-83 ExpressionStatement: w.scores = [...d.scores] as [number, number];

L84-84 ExpressionStatement: w.cooldown = d.cooldown;

L85-85 ExpressionStatement: Object.assign(clock, d.clock);

L86-86 ExpressionStatement: Object.assign(poss, d.possession);

L87-87 ReturnStatement: return true;

src/games/air-hockey/PossessionClock.ts

ACTIVE SOURCE / tool or application | 26 lines | SHA-256 7d6fed3c6d541974...

Counts continuous half-table possession with center hysteresis and a 15-second violation.

Imports: ../../shared/contracts/GameResult

State, resources and initial values

Line	Field / declaration
3	side: Player | null = null;
4	elapsed = 0;

PossessionClock [L2-25]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

PossessionClock.reset [L5-8]

Reset this component's transient state for a new round/attempt.

Signature: reset()

Direct writes: this.side, this.elapsed

PossessionClock.advance [L9-21]

Consume elapsed time using this helper's fixed-step/clock policy.

Signature: advance(dt: number, y: number): Player | null

Direct writes: this.side, this.elapsed

Calls / delegates: this.reset

Return at L20: opponent

Decision branches: L11: next !== this.side; L15: this.side === null; L17: this.elapsed < 15

PossessionClock.remaining [L22-24]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get remaining()

Return at L23: Math.max(0, 15 - this.elapsed)

src/games/air-hockey/config.ts

ACTIVE SOURCE / tool or application | 16 lines | SHA-256 6e570dc87d46ea40...

Eager lightweight metadata for Air Hockey: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-15

```
export default {
  id: "air-hockey",
  name: "Air Hockey",
  description: "Slide. Strike. Settle the score.",
  instructions:
    "Drag your half to move your mallet. Keyboard: Orange WASD, Blue arrow keys. Stay on your side and shoot the puck into the opposite goal. First to 5 or the lead after 2 minutes wins; a nonzero tie gets sudden death. Keep the puck moving: 15 seconds on your half gives your rival a point.",
  color: "#a8d6e1",
  accent: "#e0f0f2",
  tag: "FAST & FURIOUS",
  order: 2,
  preview,
  supportedModes: ["local-two-player", "single-player"],
} satisfies GameMetadata;
```

src/games/air-hockey/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 a04362b1ecaabb84...

Side-effect-free lazy factory for Air Hockey; implementation initializes only after host selection.

Imports: ./AirHockeyGame

Top-level block L2-2

```
export default () => new AirHockey();
```

src/games/air-hockey/rink.ts

ACTIVE SOURCE / tool or application | 108 lines | SHA-256 ec220caa279a5013...

Defines rounded rail/goal geometry, projection normals and puck reachability.

Imports: ../../core/ThemeManager, ../../shared/physics/FixedStep

Module constants and data

L10-10 CORNER = 44

L11-11 GOAL_HALF = 63

Body [L3-9]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Body { x: number; y: number; vx: number; vy: number; r: number; }
```

inMouth [L12-13]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: inMouth = (x: number, r: number) =>

contact [L15-30]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function contact(b: Body, x: number, y: number, nx: number, ny: number, restitution: number,)

Direct writes: b.x, b.y, b.vx, b.vy

Decision branches: L26: approach > 0

constrainRink [L31-74]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function constrainRink(b: Body, goals: boolean, restitution = 0.96)

Direct writes: hit

Calls / delegates: contact, inMouth, clamp

Return at L73: hit

Decision branches: L37: b.x < left; L40: b.x > right; L44: !(goals && inMouth(b.x, b.r)); L45: b.y < top; L48: b.y > bottom; L56: b.x !== cx && b.y !== cy; L61: d > allowed

Execution order (direct statements; nested callbacks stay within their owner):

L32-32 FirstStatement: let hit = false;

L33-36 FirstStatement: const left = FIELD.left + b.r, right = FIELD.right - b.r,

L37-43 IfStatement: if (b.x < left) { contact(b, left, b.y, -1, 0, restitution);

L44-52 IfStatement: if (!(goals && inMouth(b.x, b.r))) { if (b.y < top) {

L54-55 FirstStatement: const cx = clamp(b.x, FIELD.left + CORNER, FIELD.right - CORNER), cy = clamp(b.y, FIELD.top + CORNER, FIELD.bottom - CORNER);

L56-72 IfStatement: if (b.x !== cx && b.y !== cy) { const dx = b.x - cx,

L73-73 ReturnStatement: return hit;

validPuckPosition [L75-89]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function validPuckPosition(x: number, y: number, r: number)

Calls / delegates: inMouth, clamp

Return at L82: y >= FIELD.top - r && y <= FIELD.bottom + r

Return at L86: (x === cx || y === cy || Math.hypot(x - cx, y - cy) <= CORNER - r + 0.01)

Decision branches: L76: !Number.isFinite(x + y) || x < FIELD.left + r - 0.01 || x > FIELD.right - r + 0.01; L82: inMouth(x, r); L83: y < FIELD.top + r - 0.01 || y > FIELD.bottom - r + 0.01

reachablePuck [L90-107]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function reachablePuck(x: number, y: number, r: number)

Calls / delegates: [0, 1].some, clamp, constrainRink

Return at L92: [0, 1].some((i) => { const m = { x: clamp(x, FIELD.left + 29, FIELD.right - 29), y: clamp(y, i === 0 ? FIELD.middle + 29 : FIELD.top + 29, i === 0 ? FIELD.bottom - 29 : FIELD.middle - 29,), vx: 0, vy: 0, r: 29, }; constrainRink(m, false, 0); return Math.hypot(x - m.x, y - m.y) <= 29 + r + 0.5; })

Return at L105: Math.hypot(x - m.x, y - m.y) <= 29 + r + 0.5

src/games/connect-four

src/games/connect-four/ConnectFourAI.ts

ACTIVE SOURCE / tool or application | 164 lines | SHA-256 158756c451c76913...

Evaluates board windows and searches a private mutable/undo board under bounded alpha-beta budgets.

Imports: ../../shared/ai/MatchRandom, ./ConnectFourRules, ../../shared/contracts/AIController, ../../shared/contracts/GameResult

SearchResult [L5-10]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface SearchResult { column: number; depth: number; nodes: number; milliseconds: number; }
```

evaluate [L11-34]

Score a board/candidate state for the surrounding AI search.

Signature: function evaluate(b: ConnectFourBoard, root: Player, style: MatchRandom)

Direct writes: score

Return at L33: score

Decision branches: L18: b.cells[i] === root; L19: b.cells[i] !== null; L21: !them; L22: !us; L26: p; L31: p

searchConnectFour [L35-163]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function searchConnectFour(b: ConnectFourBoard, difficulty: Difficulty, random = Math.random, options: { seed?: number; replay?: boolean } = {},): SearchResult

Direct writes: alpha, beta, score, value, move, s, candidate, best, completed

Calls / delegates: MatchRandom, performance.now, b.legal, Map, evaluate, b.cells.map((c) => c ?? 0).join, b.cells.map, table.get, moves.indexOf, moves.unshift, moves.splice, b.drop, search, b.undo, table.set, b.legal().sort, style.rank, legal.filter, style.next

Return at L62: b.winner === "draw" ? 0 : b.winner === root ? 100000 + depth : -100000 - depth

Return at L67: evaluate(b, root, style)

Return at L73: entry.score

Return at L76: entry.score

Return at L109: value

Return at L112: { column: -1, depth: 0, nodes: 0, milliseconds: 0 }

Return at L119: { column: c, depth: 1, nodes, milliseconds: performance.now() - start, }

Return at L157: { column: best, depth: completed, nodes, milliseconds: performance.now() - start, }

Decision branches: L59: (++nodes & 63) === 0 && (performance.now() > end || nodes > cfg.nodes); L61: b.winner; L67: depth === 0; L72: entry && entry.depth >= depth; L73: entry.flag === 0; L74: entry.flag === 1; L76: alpha >= beta; L80: entry; L82: at > 0; L94: maximizing ? score > value : score < value; L98: maximizing; L100: alpha >= beta; L102: table.size < 40000; L112: !legal.length; L118: won; L139: s > value || (s === value && style.rank(c) < style.rank(candidate)); L150: e !== abort; L153: Math.abs(value) >= 100000 || performance.now() >= end; L155: difficulty === "medium" && style.next() < 0.16

Execution order (direct statements; nested callbacks stay within their owner):

L41-49 FirstStatement: const style = new MatchRandom(options.seed ?? random), start = performance.now(),

L50-52 FirstStatement: let nodes = 0, best = b.legal()[0] ?? -1,

L53-56 FirstStatement: const table = new Map< string,

L57-57 FirstStatement: const abort = {};

L58-110 FunctionDeclaration: function search(depth: number, alpha: number, beta: number): number { if ((++nodes & 63) === 0 && (performance.now() > end || nodes > cfg.nodes))

L111-111 FirstStatement: const legal = b.legal().sort((a, z) => style.rank(a) - style.rank(z));

L112-112 IfStatement: if (!legal.length) return { column: -1, depth: 0, nodes: 0, milliseconds: 0 };

L114-125 ForOfStatement: for (const c of legal) { b.drop(c);

L126-154 ForStatement: for (let depth = 1; depth <= cfg.depth; depth++) { let candidate = best,

L155-156 IfStatement: if (difficulty === "medium" && style.next() < 0.16) best = legal[Math.floor(style.next() * legal.length)];

L157-162 ReturnStatement: return { column: best,

searchConnectFour.search [L58-110]

Explore a bounded game tree with the local turn/evaluation policy.

Signature: function search(depth: number, alpha: number, beta: number): number

Direct writes: alpha, beta, score, value, move

Calls / delegates: performance.now, evaluate, b.cells.map((c) => c ?? 0).join, b.cells.map, table.get, b.legal, moves.indexOf, moves.unshift, moves.splice, b.drop, search, b.undo, table.set

Return at L62: b.winner === "draw" ? 0 : b.winner === root ? 100000 + depth : -100000 - depth

Return at L67: evaluate(b, root, style)

Return at L73: entry.score

Return at L76: entry.score

Return at L109: value

Decision branches: L59: (++nodes & 63) === 0 && (performance.now() > end || nodes > cfg.nodes); L61: b.winner; L67: depth === 0; L72: entry && entry.depth >= depth; L73: entry.flag === 0; L74: entry.flag === 1; L76: alpha >= beta; L80: entry; L82: at > 0; L94: maximizing ? score > value : score < value; L98: maximizing; L100: alpha >= beta; L102: table.size < 40000

Execution order (direct statements; nested callbacks stay within their owner):

L59-60 IfStatement: if ((++nodes & 63) === 0 && (performance.now() > end || nodes > cfg.nodes)) throw abort;

L61-66 IfStatement: if (b.winner) return b.winner === "draw"

L67-67 IfStatement: if (depth === 0) return evaluate(b, root, style);

L68-71 FirstStatement: const key = b.cells.map((c) => c ?? 0).join("") + b.turn, entry = table.get(key),

L72-77 IfStatement: if (entry && entry.depth >= depth) { if (entry.flag === 0) return entry.score;

L78-79 FirstStatement: const maximizing = b.turn === root, moves = b.legal();

L80-83 IfStatement: if (entry) { const at = moves.indexOf(entry.move);

L84-85 FirstStatement: let value = maximizing ? -Infinity : Infinity, move = moves[0];

L86-101 ForOfStatement: for (const c of moves) { b.drop(c);

L102-108 IfStatement: if (table.size < 40000) table.set(key, {

L109-109 ReturnStatement: return value;

src/games/connect-four/ConnectFourGame.ts

ACTIVE SOURCE / tool or application | 187 lines | SHA-256 e31d48d470f94629...

Connect Four lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/components/TurnView, ../../shared/ai/MatchRandom, phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ../../shared/utilities/WorkerTask, ./ConnectFourRules, ./ConnectFourAI

State, resources and initial values

Line	Field / declaration
15	private ctx!: GameContext;
16	private board = new ConnectFourBoard();
17	private g!: Phaser.GameObjects.Graphics;
18	private heading!: Phaser.GameObjects.Text;
19	private hint!: Phaser.GameObjects.Text;
20	private ai?: WorkerTask< { state: unknown; difficulty: GameContext["options"]["difficulty"]; seed: number; replay: boolean; }, SearchResult >;
29	private personality!: MatchRandom;
30	private pending = false;
31	private paused = false;
32	private disposed = false;
33	private generation = 0;
34	private age = 1;
35	private last = -1;
36	private thinking = 0;
37	private reported = false;

ConnectFourGame [L14-186]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ConnectFourGame.initialize [L38-69]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.personality, this.board, this.g, this.heading, this.hint, this.ai

Calls / delegates: MatchRandom, restoredAISeed, ConnectFourBoard.restore, ctx.scene.add.graphics, label, ctx.setHint, ctx.setScore, WorkerTask, Worker, URL, ctx.input.tap, this.place

Decision branches: L47: ctx.options.mode === "ai"; L56: this.paused || this.age < 0.32 || this.board.winner || (this.ai && this.board.turn === 2) || x < 35 || x >= 385 || y < 200 || y > 560

ConnectFourGame.place [L70-81]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private place(c: number)

Direct writes: this.last, this.age, this.thinking

Calls / delegates: this.board.drop, this.ctx.saveState, this.board.save, this.personality.snapshot, this.ctx.audio.play

Decision branches: L72: at < 0

ConnectFourGame.update [L82-157]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.age, this.thinking, this.pending, this.reported

Calls / delegates: this.ai .run({ state: this.board.save(), difficulty: this.ctx.options.difficulty, seed: this.personality.seed, replay: replayMode(), }) .then, this.ai .run, this.board.save, replayMode, this.place, this.board.legal, this.heading.setText, g.clear, rounded, turnBackground, circle, g.lineStyle(2, theme.white,

0.55).strokeCircle, g.lineStyle, this.board.line?.includes, g.lineStyle(3, theme.white).strokeCircle, g.fillStyle(this.board.turn === 1 ? theme.orange : theme.blue,).fillTriangle, g.fillStyle, this.heading.setDepth, this.hint.setDepth, this.ctx.finish

Decision branches: L83: this.paused; L85: this.ai && !this.board.winner && this.board.turn === 2 && !this.pending && this.age > 0.32; L93: this.thinking > 0.75; L104: this.disposed || generation !== this.generation; L106: !this.paused; L131: mark; L136: this.board.line?.includes(i) && this.age > 0.3; L140: !this.board.winner; L149: this.board.winner && this.age > 0.9 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L83-83 IfStatement: if (this.paused) return;

L84-84 ExpressionStatement: this.age += dt;

L85-110 IfStatement: if (this.ai &&

L111-121 ExpressionStatement: this.heading.setText(this.board.winner

L122-122 FirstStatement: const g = this.g;

L123-123 ExpressionStatement: g.clear();

L124-124 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, turnBackground(this.board.turn), 4);

L125-125 ExpressionStatement: rounded(g, 28, 210, 364, 332, 18, 0x82b4c3, 4);

L126-139 ForStatement: for (let i = 0; i < 42; i++) { const x = 60 + (i % 7) * 50,

L140-145 IfStatement: if (!this.board.winner) for (const c of this.board.legal()) {

L147-147 ExpressionStatement: this.heading.setDepth(1);

L148-148 ExpressionStatement: this.hint.setDepth(1);

L149-156 IfStatement: if (this.board.winner && this.age > 0.9 && !this.reported) { this.reported = true;

ConnectFourGame.pause [L158-163]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused, this.pending

Calls / delegates: this.ai?.cancel

ConnectFourGame.resume [L164-167]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused, this.thinking

ConnectFourGame.dispose [L168-175]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.disposed

Calls / delegates: this.ai?.cancel, this.g.destroy, this.heading.destroy, this.hint.destroy

ConnectFourGame.snapshot [L176-185]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L177: { ai: this.ai ? this.personality.snapshot() : undefined, cells: [...this.board.cells], turn: this.board.turn, winner: this.board.winner, moves: this.board.history.length, pending: this.pending, }

src/games/connect-four/ConnectFourRules.ts

ACTIVE SOURCE / tool or application | 78 lines | SHA-256 1dfb4ca44a898c7e...

Owns a 7x6 gravity board, 69 winning windows, legal drops, undo and history restore.

Imports: ../../shared/contracts/GameResult

State, resources and initial values

Line	Field / declaration
19	cells: (Player | null)[] = Array(42).fill(null);
20	turn: Player = 1;
21	winner: Winner | null = null;
22	line: number[] | null = null;
23	history: number[] = [];

Module constants and data

L2-2 FOUR_LINES = []

ConnectFourBoard [L18-77]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ConnectFourBoard.legal [L24-28]

Enumerate/check legal actions under the current board and terminal state.

Signature: legal()

Calls / delegates: [3, 2, 4, 1, 5, 0, 6].filter

Return at L25: this.winner ? [] : [3, 2, 4, 1, 5, 0, 6].filter((c) => this.cells[c] === null)

ConnectFourBoard.drop [L29-50]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: drop(column: number)

Direct writes: this.cells[index], this.line, this.winner, this.turn

Calls / delegates: this.history.push, FOUR_LINES.find, l.every

Return at L37: -1

Return at L49: index

Decision branches: L30: this.winner || !Number.isInteger(column) || column < 0 || column > 6 || this.cells[column] !== null; L46: this.line; L47: this.history.length === 42

ConnectFourBoard.undo [L51-62]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: undo()

Direct writes: this.cells[r * 7 + c], this.turn, this.winner, this.line

Calls / delegates: this.history.pop

Decision branches: L53: c === undefined; L55: this.cells[r * 7 + c] !== null

ConnectFourBoard.save [L63-65]

Create/persist the component checkpoint or record described by the returned expression.

Signature: save()

Return at L64: { version: 1, moves: [...this.history] }

```
save() { return { version: 1, moves: [...this.history] }; }
```

ConnectFourBoard.restore [L66-76]

Validate/replay saved state and fall back to a fresh legal instance when rejected.

Signature: static restore(data: unknown)

Calls / delegates: ConnectFourBoard, b.drop

Return at L68: fresh

Return at L71: fresh

Return at L74: fresh

Return at L75: b

Decision branches: L68: !data || typeof data !== "object"; L70: d.version !== 1 || !Array.isArray(d.moves) || d.moves.length > 42; L74: typeof c !== "number" || b.drop(c) < 0

Top-level block L3-17

```
for (let y = 0; y < 6; y++)
  for (let x = 0; x < 7; x++)
    for (const [dx, dy] of [
      [1, 0],
      [0, 1],
      [1, 1],
      [-1, 1],
    ]) {
      const xx = x + 3 * dx,
        yy = y + 3 * dy;
      if (xx >= 0 && xx < 7 && yy < 6)
        FOUR_LINES.push(
          Array.from({ length: 4 }, (_, i) => (y + i * dy) * 7 + x + i * dx),
        );
    }
```

src/games/connect-four/ai.worker.ts

ACTIVE SOURCE / tool or application | 20 lines | SHA-256 cc393a374d4f7d27...

Worker message entry for Connect Four. Restores/copies request state, runs the bounded local search and posts its result; cancellation is owned by WorkerTask/controller.

Imports: ./ConnectFourRules, ./ConnectFourAI, ../../shared/contracts/AIController

Top-level block L4-19

```
self.onmessage = (
  e: MessageEvent<{
    state: unknown;
    difficulty: Difficulty;
    seed?: number;
    replay?: boolean;
  }>,
) =>
  self.postMessage(
    searchConnectFour(
      ConnectFourBoard.restore(e.data.state),
      e.data.difficulty,
      undefined,
      e.data,
    ),
  );
```

src/games/connect-four/config.ts

ACTIVE SOURCE / tool or application | 17 lines | SHA-256 53cd9920f25b5e0d...

Eager lightweight metadata for Connect Four: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "connect-four",
  name: "Connect Four",
  description: "Drop a piece. Spot a plan.",
  instructions:
    "Tap a column to drop your piece. Connect four horizontally, vertically or diagonally.",
  color: "#c8e7eb",
  accent: "#82b4c3",
  preview,
  order: 4,
  tag: "AHEAD OF THE CURVE",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/connect-four/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 6f72fe88595ae1b5...

Side-effect-free lazy factory for Connect Four; implementation initializes only after host selection.

Imports: ./ConnectFourGame

Top-level block L2-2

```
export default () => new ConnectFourGame();
```

src/games/dots-and-boxes

src/games/dots-and-boxes/AI.ts

ACTIVE SOURCE / tool or application | 223 lines | SHA-256 3b810e6682b0eefb...

Orders captures/safe edges/chains and performs bounded extra-turn-aware alpha-beta/endgame search.

Imports: ../../shared/ai/MatchRandom, ./Rules, ../../shared/contracts/AIController

sacrificeDamage [L5-35]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function sacrificeDamage(board: DotsBoard, edge: number): number

Direct writes: claimed[edge], claimed[remaining]

Calls / delegates: board.edges.slice, queue.push, queue.shift, board.boxes[b].find, queue.includes

Return at L34: total

Decision branches: L10: !board.owners[b]; L12: claimed[e]; L13: drawn === 3; L21: remaining !== undefined; L24: adj !== b && !board.owners[adj]; L26: claimed[e]; L27: drawn === 3 && !queue.includes(adj)

priority [L37-64]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function priority(b: DotsBoard, e: number): number

Direct writes: safeQuality

Calls / delegates: sacrificeDamage

Return at L54: 30000

Return at L57: hasTwo === 1 ? 25000 : 20000

Return at L61: -20000 - damage * 1000

Return at L63: 1000 + safeQuality

Decision branches: L45: b.edges[edge]; L47: drawn === 3; L48: drawn === 2; L49: drawn === 1; L53: hasThree === 2; L56: hasThree === 1; L59: hasTwo > 0

getBoardKey [L66-81]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function getBoardKey(b: DotsBoard): string

Direct writes: m0, m1, m2

Return at L80: `${m0}_${m1}_${m2}_${b.turn}_${b.scores[0]}`

Decision branches: L72: b.edges[i]; L75: b.edges[i]; L78: b.edges[i]

chooseEdge [L83-222]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function chooseEdge(board: DotsBoard, difficulty: Difficulty, options: { seed?: number; replay?: boolean } = {},)

Direct writes: alpha, beta, value, score, candidate, best, completedDepth

Calls / delegates: MatchRandom, style.rank, priority, preference, performance.now, board.legal, legal.sort, order, Map, Symbol, evaluate, getBoardKey, cache.get, b.legal().sort, b.legal, b.clone, child.play, search, cache.set, legal.filter, board.clone, b.play

Return at L94: ((style.role === 0 ? Math.abs(row - center) + Math.abs(col - center) : style.role === 1 ? -Math.abs(row - center) - Math.abs(col - center) : style.role === 2 ? e < board.horizontalCount ? 1 : -1 : col * style.side) + style.rank(e) * (1 + style.tempo))

Return at L117: { edge: best, milliseconds: performance.now() - start, depth: 0, nodes, }

Return at L131: 100000 + (b.scores[root - 1] - b.scores[2 - root]) * 1000

Return at L133: -100000 + (b.scores[root - 1] - b.scores[2 - root]) * 1000

Return at L145: scoreDiff + pendingCaptures * turnMultiplier * 900

Return at L159: evaluate(b)

Return at L165: memo.value

Return at L168: memo.value

Return at L189: value

Return at L216: { edge: best, milliseconds: performance.now() - start, depth: completedDepth, nodes, }

Decision branches: L116: difficulty === "medium"; L129: b.winner; L130: b.winner === root; L132: b.winner === "draw"; L138: !b.owners[i]; L140: b.edges[e]; L141: drawn === 3; L154: ++nodes > 50000 || (!options.replay && performance.now() - start > budget); L159: b.winner || depth === 0; L164: memo && memo.depth >= depth; L165: memo.flag === 0; L166: memo.flag === 1; L168: alpha >= beta; L177: maximize; L179: beta <= alpha; L183: cache.size < 20000; L201: v > score || (v === score && preference(e) > preference(candidate)); L212: error !== timeout

Execution order (direct statements; nested callbacks stay within their owner):

L88-88 FirstStatement: const style = new MatchRandom(options.seed);

L89-106 FirstStatement: const preference = (e: number) => { const box = board.adjacent[e]?.[0] ?? 0,

L107-108 FirstStatement: const order = (b: DotsBoard, a: number, c: number) => priority(b, c) - priority(b, a) || preference(c) - preference(a);

L109-111 FirstStatement: const start = performance.now(), budget = { medium: 12, hard: 80, impossible: 350 }[difficulty],

L112-114 FirstStatement: let nodes = 0, best = legal.sort((a, b) => order(board, a, b))[0] ?? -1,

L115-115 FirstStatement: const root = board.turn;

L116-123 IfStatement: if (difficulty === "medium") { return {

L124-127 FirstStatement: const cache = new Map< string,

L128-146 FirstStatement: const evaluate = (b: DotsBoard) => { if (b.winner) {

L147-147 FirstStatement: const timeout = Symbol("search budget");

L148-190 FirstStatement: const search = (b: DotsBoard,

L191-192 FirstStatement: const maxSearchDepth = difficulty === "hard" ? 6 : Math.min(legal.length, 30);

L193-215 ForStatement: for (let depth = 1; depth <= maxSearchDepth; depth++) { let candidate = best,

L216-221 ReturnStatement: return { edge: best,

chooseEdge.preference [L89-106]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: preference = (e: number) =>

Calls / delegates: style.rank

Return at L94: ((style.role === 0 ? Math.abs(row - center) + Math.abs(col - center) : style.role === 1 ? -Math.abs(row - center) - Math.abs(col - center) : style.role === 2 ? e < board.horizontalCount ? 1 : -1 : col * style.side) + style.rank(e) * (1 + style.tempo))

chooseEdge.order [L107-108]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: order = (b: DotsBoard, a: number, c: number) =>

Calls / delegates: priority, preference

chooseEdge.evaluate [L128-146]

Score a board/candidate state for the surrounding AI search.

Signature: evaluate = (b: DotsBoard) =>

Return at L131: 100000 + (b.scores[root - 1] - b.scores[2 - root]) * 1000

Return at L133: -100000 + (b.scores[root - 1] - b.scores[2 - root]) * 1000

Return at L145: scoreDiff + pendingCaptures * turnMultiplier * 900

Decision branches: L129: b.winner; L130: b.winner === root; L132: b.winner === "draw"; L138: !b.owners[i]; L140: b.edges[e]; L141: drawn === 3

chooseEdge.search [L148-190]

Explore a bounded game tree with the local turn/evaluation policy.

Signature: search = (b: DotsBoard, depth: number, alpha: number, beta: number,): number =>

Direct writes: alpha, beta, value

Calls / delegates: performance.now, evaluate, getBoardKey, cache.get, b.legal().sort, b.legal, order, b.clone, child.play, search, cache.set

Return at L159: evaluate(b)

Return at L165: memo.value

Return at L168: memo.value

Return at L189: value

Decision branches: L154: ++nodes > 50000 || (!options.replay && performance.now() - start > budget); L159: b.winner || depth === 0; L164: memo && memo.depth >= depth; L165: memo.flag === 0; L166: memo.flag === 1; L168: alpha >= beta; L177: maximize; L179: beta <= alpha; L183: cache.size < 20000

Execution order (direct statements; nested callbacks stay within their owner):

L154-158 IfStatement: if (++nodes > 50000 ||

L159-159 IfStatement: if (b.winner || depth === 0) return evaluate(b);

L160-160 FirstStatement: const key = getBoardKey(b);

L161-163 FirstStatement: const originalAlpha = alpha, originalBeta = beta,

L164-169 IfStatement: if (memo && memo.depth >= depth) { if (memo.flag === 0) return memo.value;

L170-170 FirstStatement: const maximize = b.turn === root;

L171-171 FirstStatement: let value = maximize ? -Infinity : Infinity;

L172-182 ForOfStatement: for (const e of b.legal().sort((a, c) => order(b, a, c))) { const child = b.clone();

L183-188 IfStatement: if (cache.size < 20000) cache.set(key, {

L189-189 ReturnStatement: return value;

src/games/dots-and-boxes/DotsGame.ts

ACTIVE SOURCE / tool or application | 244 lines | SHA-256 643499bb1b5d8002...

Dots and Boxes lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/components/TurnView, ../../shared/ai/MatchRandom, phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ../../shared/utilities/WorkerTask, ./Rules, ./AI

State, resources and initial values

Line	Field / declaration
15	private ctx!: GameContext;
16	private board = new DotsBoard();
17	private g!: Phaser.GameObjects.Graphics;
18	private title!: Phaser.GameObjects.Text;
19	private detail!: Phaser.GameObjects.Text;
20	private ai?: WorkerTask< { state: unknown; difficulty: GameContext["options"]["difficulty"]; seed: number; replay: boolean; }, ReturnType<typeof chooseEdge> >;
29	private personality!: MatchRandom;
30	private pending = false;
31	private generation = 0;
32	private paused = false;
33	private age = 1;
34	private flashAge = 1;
35	private aiDelay = 0;
36	private last = -1;
37	private selected = 0;
38	private reported = false;

DotsGame [L14-243]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

DotsGame.initialize [L39-83]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.personality, this.board, this.aiDelay, this.g, this.title, this.detail, this.ai, this.selected

Calls / delegates: MatchRandom, restoredAISeed, DotsBoard.restore, replayMode, ctx.scene.add.graphics, label, ctx.setHint, ctx.setScore, WorkerTask, Worker, URL, ctx.input.tap, this.place, pickEdge, ctx.input.press, this.board.legal, legal.indexOf

Decision branches: L62: ctx.options.mode === "ai"; L74: code === "Enter" || code === "Space"

Execution order (direct statements; nested callbacks stay within their owner):

L40-40 ExpressionStatement: this.ctx = ctx;

L41-41 ExpressionStatement: this.personality = new MatchRandom(restoredAISeed(ctx.restoredState));

L42-42 FirstStatement: const variant = ctx.options.variant;

L43-43 FirstStatement: const size = variant === "6x6" ? 6 : variant === "5x5" ? 5 : 4;

L44-44 ExpressionStatement: this.board = DotsBoard.restore(ctx.restoredState, size);

L45-50 ExpressionStatement: this.aiDelay = ctx.options.mode === "ai" && this.board.turn === 2

L51-51 ExpressionStatement: this.g = ctx.scene.add.graphics();

L52-58 ExpressionStatement: this.title = label(ctx.scene,

L59-59 ExpressionStatement: this.detail = label(ctx.scene, 210, 602, "TAP BETWEEN TWO DOTS", 13);

L60-60 ExpressionStatement: ctx.setHint("");

L61-61 ExpressionStatement: ctx.setScore(...this.board.scores);

L62-69 IfStatement: if (ctx.options.mode === "ai") this.ai = new WorkerTask(

L70-70 ExpressionStatement: ctx.input.tap((x, y) => this.place(pickEdge(x, y, this.board.size)));

L71-82 ExpressionStatement: ctx.input.press(["ArrowLeft", "ArrowRight", "ArrowUp", "ArrowDown", "Enter", "Space"],

DotsGame.place [L84-115]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private place(edge: number, ai = false)

Direct writes: this.last, this.flashAge, this.age, this.aiDelay

Calls / delegates: this.board.play, this.ctx.setScore, this.ctx.saveState, this.board.save, this.personality.snapshot, this.ctx.audio.play, this.detail.setText, replayMode, this.personality.next

Decision branches: L85: this.paused || this.age < 0.12 || this.board.winner || (!ai && this.ai && this.board.turn === 2); L93: gain < 0; L108: this.ai && !this.board.winner && this.board.turn === 2

DotsGame.update [L116-212]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.age, this.flashAge, this.aiDelay, this.pending, this.reported

Calls / delegates: this.ai .run({ state: this.board.save(), difficulty: this.ctx.options.difficulty, seed: this.personality.seed, replay: replayMode(), }) .then, this.ai .run, this.board.save, replayMode, this.place, this.board.legal, g.clear, rounded, turnBackground, g.fillStyle(theme.ink, alpha).fillRoundedRect, g.fillStyle, this.title .setText(this.board.winner ? "BOXED UP!" : this.age < 1.0 && !this.board.history.length ? `${size}x${size} · ${this.board.totalBoxes} BOXES` : this.board.turn === 1 ? "" : this.ai ? "…" : "",) .setDepth, this.title .setText, this.detail.setDepth(1).setFontSize, this.detail.setDepth, this.title.setFontSize, g.fillStyle(p === 1 ? theme.orange : theme.blue, 0.7).fillRoundedRect, edgeEnds, g.lineStyle(owner ? edgeWidth : e === this.selected ? edgeWidth - 1 : unownedWidth, owner ? (owner === 1 ? theme.orange : theme.blue) : theme.ink, owner ? 1 : e === this.selected ? 0.35 : 0.12,).lineBetween, g.lineStyle, g.lineStyle(2, theme.white).lineBetween, circle, this.ctx.finish

Decision branches: L117: this.paused; L120: this.aiDelay > 0; L121: this.ai && !this.pending && !this.board.winner && this.board.turn === 2 && this.aiDelay <= 0; L138: token !== this.generation; L153: this.age < 0.9 && !this.board.history.length; L175: this.board.owners[i]; L199: e === this.last && this.flashAge < 0.35; L208: this.board.winner && this.age > 0.7 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L117-117 IfStatement: if (this.paused) return;

L118-118 ExpressionStatement: this.age += dt;

L119-119 ExpressionStatement: this.flashAge += dt;

L120-120 IfStatement: if (this.aiDelay > 0) this.aiDelay -= dt;

L121-142 IfStatement: if (this.ai &&

L143-143 FirstStatement: const g = this.g;

L144-144 ExpressionStatement: g.clear();

L145-145 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, turnBackground(this.board.turn), 4);

L147-147 FirstStatement: const size = this.board.size;

L148-148 FirstStatement: const spacing = 300 / size;

L149-149 FirstStatement: const pad = size === 6 ? 4 : size === 5 ? 5 : 6;

L150-150 FirstStatement: const boxSize = spacing - 2 * pad;

L151-151 FirstStatement: const cornerRadius = size === 6 ? 6 : size === 5 ? 8 : 10;

L153-156 IfStatement: if (this.age < 0.9 && !this.board.history.length) { const alpha = Math.sin((this.age / 0.9) * Math.PI) * 0.15;

L158-170 ExpressionStatement: this.title .setText(

L171-171 ExpressionStatement: this.detail.setDepth(1).setFontSize(11);

L172-172 ExpressionStatement: this.title.setFontSize(12);

L174-186 ForStatement: for (let i = 0; i < this.board.totalBoxes; i++) if (this.board.owners[i]) {

L188-188 FirstStatement: const edgeWidth = size === 6 ? 5 : size === 5 ? 6 : 7;

L189-189 FirstStatement: const unownedWidth = size === 6 ? 2.5 : 3;

L191-201 ForStatement: for (let e = 0; e < this.board.totalEdges; e++) { const p = edgeEnds(e, size),

L203-203 FirstStatement: const dotRadius = size === 6 ? 4.5 : size === 5 ? 5.2 : 6;

L204-206 ForStatement: for (let r = 0; r <= size; r++) for (let c = 0; c <= size; c++)

L208-211 IfStatement: if (this.board.winner && this.age > 0.7 && !this.reported) { this.reported = true;

DotsGame.pause [L213-219]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused, this.pending, this.aiDelay

Calls / delegates: this.ai?.cancel

DotsGame.resume [L220-224]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused, this.age, this.aiDelay

DotsGame.dispose [L225-230]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.pause, this.g.destroy, this.title.destroy, this.detail.destroy

DotsGame.snapshot [L231-242]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot, this.board.save

Return at L232: { ai: this.ai ? this.personality.snapshot() : undefined, ...this.board.save(), edges: [...this.board.edges], owners: [...this.board.owners], turn: this.board.turn, winner: this.board.winner, scores: [...this.board.scores], pending: this.pending, }

src/games/dots-and-boxes/Rules.ts

ACTIVE SOURCE / tool or application | 171 lines | SHA-256 7c81614b1cc0a15e...

Builds size-dependent edges/boxes and owns legal edge capture, extra turns, score, history and hit geometry.

State, resources and initial values

Line	Field / declaration
37	readonly size: number;
38	readonly horizontalCount: number;
39	readonly totalEdges: number;
40	readonly totalBoxes: number;
41	readonly boxes: number[][];
42	readonly adjacent: number[][];
43	edges: number[];
44	owners: number[];
45	turn: 1 | 2 = 1;
46	history: number[] = [];
47	scores: [number, number] = [0, 0];
48	winner: 1 | 2 | "draw" | null = null;

Module constants and data

L21-25 BOXES_BY_SIZE = { 4: createBoxes(4), 5: createBoxes(5), 6: createBoxes(6), }

L27-31 ADJACENT_BY_SIZE = { 4: createAdjacent(40, BOXES_BY_SIZE[4]), 5: createAdjacent(60, BOXES_BY_SIZE[5]), 6: createAdjacent(84, BOXES_BY_SIZE[6]), }

L33-33 boxes = BOXES_BY_SIZE[4]

L34-34 adjacent = ADJACENT_BY_SIZE[4]

createBoxes [L1-13]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function createBoxes(size: number): number[][]

Return at L3: Array.from({ length: size * size }, (_, i) => { const r = Math.floor(i / size), c = i % size; return [r * size + c, (r + 1) * size + c, H + r * (size + 1) + c, H + r * (size + 1) + c + 1,]; })

Return at L6: [r * size + c, (r + 1) * size + c, H + r * (size + 1) + c, H + r * (size + 1) + c + 1,]

createAdjacent [L15-19]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function createAdjacent(totalEdges: number, boxesList: number[][]): number[][]

Calls / delegates: boxesList.flatMap, b.includes

Return at L16: Array.from({ length: totalEdges }, (_, i) => boxesList.flatMap((b, j) => (b.includes(i) ? [j] : [])),)

DotsBoard [L36-130]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

DotsBoard.constructor [L50-61]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(size: number = 4)

Direct writes: this.size, this.horizontalCount, this.totalEdges, this.totalBoxes, this.boxes, this.adjacent, this.edges, this.owners

Calls / delegates: createBoxes, createAdjacent, Array<number>(this.totalEdges).fill, Array, Array<number>(this.totalBoxes).fill

DotsBoard.legal [L63-65]

Enumerate/check legal actions under the current board and terminal state.

Signature: legal()

Calls / delegates: this.edges.flatMap

Return at L64: this.edges.flatMap((v, i) => (v === 0 ? [i] : []))

DotsBoard.play [L67-94]

Apply a requested legal move or start a selected application module, depending on the owner.

Signature: play(edge: number)

Direct writes: this.edges[edge], this.owners[box], this.winner, this.turn

Calls / delegates: this.history.push, this.boxes[box].every

Return at L75: -1

Return at L93: gained

Decision branches: L68: this.winner || !Number.isInteger(edge) || edge < 0 || edge >= this.totalEdges || this.edges[edge]; L80: !this.owners[box] && this.boxes[box].every((e) => this.edges[e]); L85: this.scores[0] + this.scores[1] === this.totalBoxes; L92: !gained

DotsBoard.clone [L96-105]

Create independent state for simulation/search without mutating the live instance.

Signature: clone()

Direct writes: b.edges, b.owners, b.turn, b.history, b.scores, b.winner

Calls / delegates: DotsBoard

Return at L104: b

DotsBoard.save [L107-109]

Create/persist the component checkpoint or record described by the returned expression.

Signature: save()

Return at L108: { history: [...this.history], size: this.size }

```
save() { return { history: [...this.history], size: this.size }; }
```

DotsBoard.restore [L111-129]

Validate/replay saved state and fall back to a fresh legal instance when rejected.

Signature: static restore(value: unknown, fallbackSize = 4)

Calls / delegates: DotsBoard, b.play

Return at L124: b

Return at L126: b

Return at L127: new DotsBoard(size)

Return at L128: b

Decision branches: L119: !value || typeof value !== "object" || !Array.isArray((value as any).history); L126: h.length > b.totalEdges; L127: b.play(e) < 0

edgeEnds [L132-145]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function edgeEnds(edge: number, size = 4)

Return at L138: { x, y, x2: x + spacing, y2: y }

Return at L144: { x, y, x2: x, y2: y + spacing }

Decision branches: L135: edge < H

pickEdge [L147-170]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function pickEdge(x: number, y: number, size = 4)

Direct writes: best, distance

Calls / delegates: edgeEnds

Return at L169: best

Decision branches: L164: d < distance

src/games/dots-and-boxes/ai.worker.ts

ACTIVE SOURCE / tool or application | 7 lines | SHA-256 8362eaed4950c2c2...

Worker message entry for Dots and Boxes. Restores/copies request state, runs the bounded local search and posts its result; cancellation is owned by WorkerTask/controller.

Imports: ./Rules, ./AI

Top-level block L3-6

```
self.onmessage = (e) =>
  self.postMessage(
    chooseEdge(DotsBoard.restore(e.data.state), e.data.difficulty, e.data),
  );
```

src/games/dots-and-boxes/config.ts

ACTIVE SOURCE / tool or application | 44 lines | SHA-256 abbd624ce0b277bd...

Eager lightweight metadata for Dots and Boxes: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Module constants and data

L4-4 preview4x4 = Structured literal/expression (457 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

L6-6 preview5x5 = Structured literal/expression (527 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

L8-8 preview6x6 = Structured literal/expression (477 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

Top-level block L10-43

```
export default {
  id: "dots-and-boxes",
  name: "Dots and Boxes",
  description: "Join the dots. Claim your corner.",
  instructions:
    "Tap between two neighboring dots to draw a line. Complete a box to claim it and play again. One line can claim
two boxes. Most boxes wins. On desktop, use arrows and Enter to select a line.",
  color: "#e7ddef",
  accent: "#b6a2c5",
  preview,
  order: 8,
  tag: "CONNECT. COLLECT.",
  variants: [
    {
      id: "4x4",
      name: "4x4",
      description: "16 boxes · Fast",
      preview: preview4x4,
    },
    {
      id: "5x5",
      name: "5x5",
      description: "25 boxes · Classic",
      preview: preview5x5,
    },
    {
      id: "6x6",
      name: "6x6",
      description: "36 boxes · Epic",
      preview: preview6x6,
    },
  ],
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/dots-and-boxes/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 c9fec54fca5af647...

Side-effect-free lazy factory for Dots and Boxes; implementation initializes only after host selection.

Imports: ./DotsGame

Top-level block L2-2

```
export default () => new DotsGame();
```

src/games/fruit-duel

src/games/fruit-duel/AI.ts

ACTIVE SOURCE / tool or application | 48 lines | SHA-256 1c89613d73e09b60...

Game-local Fruit Duel observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
9	private version = -1;
10	private tapped = false;
11	private delay = 1;
12	private mistake = false;

FruitAI [L4-47]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FruitAI.snapshot [L6-8]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L7: { ...this.personality.snapshot(), reaction: this.delay }

FruitAI.constructor [L13-18]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: SeedInput,)

Direct writes: this.personality

Calls / delegates: MatchRandom

FruitAI.update [L19-46]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(m: FruitMatch)

Direct writes: this.version, this.tapped, this.delay, this.mistake

Calls / delegates: style.sample, style.next, isFruit, m.tap

Decision branches: L20: m.version !== this.version; L38: !this.tapped && m.phase === "active" && m.time - m.visibleAt >= this.delay && (isFruit(m.current) || this.mistake); L44: m.tap(1)

src/games/fruit-duel/FruitArt.ts

ACTIVE SOURCE / tool or application | 229 lines | SHA-256 34735132d09876de...

Fruit Duel original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: phaser

Module constants and data

L2-2 ink = 0x242b32

L3-5 skin = [0xef7168, 0xf6a14c, 0x73b878, 0xf2ce64, 0xc76885, 0xeac56c, 0xeb747b,]

fruit [L7-149]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function fruit(g: Phaser.GameObjects.Graphics, id: number, x: number, y: number, r: number,)

Calls / delegates: g.save, g.translateCanvas, g.scaleCanvas, g.fillStyle(c).fillEllipse, g.fillStyle, g.lineStyle(3, ink).strokeEllipse, g.lineStyle, g.lineStyle(4, ink).lineBetween, g.fillStyle(0x80b66d).fillEllipse, g.lineStyle(2, ink).strokeEllipse, g.fillStyle(skin[0]).fillPoints, g.lineStyle(3, ink).strokePoints, leaf, g.lineStyle(4, 0xffd0b9, 0.8).lineBetween, oval, g.fillStyle(0xd57e38, 0.7).fillCircle, g.fillStyle(0xffd8a1).fillEllipse, g.beginPath, g.moveTo, g.lineTo, g.strokePath, g.lineStyle(24, ink).beginPath().arc(-18, -27, 57, 0.1, 1.8).strokePath, g.lineStyle(24, ink).beginPath().arc, g.lineStyle(24, ink).beginPath, g.lineStyle(17, skin[3]) .beginPath() .arc(-18, -27, 57, 0.1, 1.8) .strokePath, g.lineStyle(17, skin[3]) .beginPath() .arc, g.lineStyle(17, skin[3]) .beginPath, g.lineStyle(3, 0xbf9343) .beginPath() .arc(-18, -27, 57, 0.1, 1.8) .strokePath, g.lineStyle(3, 0xbf9343) .beginPath() .arc, g.lineStyle(3, 0xbf9343) .beginPath, g.lineStyle(6, ink).lineBetween, g.fillStyle(skin[4]).fillTriangle, g.lineStyle(3, ink).strokeTriangle, g.fillStyle(0xb34e70).fillCircle, g.lineBetween, g.fillTriangle, g.fillStyle(skin[6]).fillPoints, g.fillStyle(0x80b66d).fillTriangle, g.fillStyle(0xffe49a).fillEllipse, g.lineStyle(9, 0xffe09c) .lineBetween(-40, 0, 40, 0) .lineBetween, g.lineStyle(9, 0xffe09c) .lineBetween, g.fillStyle(0xf6b765) .fillEllipse(-15, -7, 30, 20) .fillEllipse, g.fillStyle(0xf6b765) .fillEllipse, g.lineStyle(13, 0xb37a52) .lineBetween(-20, -35, 18, 35) .lineBetween, g.lineStyle(13, 0xb37a52) .lineBetween, g.fillStyle(0xe4ac41).fillPoints, g.restore

Return at L29: { x: 42 * Math.sin(t) ** 3, y: -(32 * Math.cos(t) - 13 * Math.cos(2 * t) - 6 * Math.cos(3 * t) - 2 * Math.cos(4 * t)) - 4, }

Decision branches: L26: id === 0; L44: id === 1; L54: id === 2; L66: id === 3; L77: id === 4; L88: id === 5; L99: id === 6; L121: id === 7; L129: id === 8; L140: id === 9

Execution order (direct statements; nested callbacks stay within their owner):

L14-14 ExpressionStatement: g.save();

L15-15 ExpressionStatement: g.translateCanvas(x, y);

L16-16 ExpressionStatement: g.scaleCanvas(r / 50, r / 50);

L17-20 FirstStatement: const oval = (x: number, y: number, w: number, h: number, c: number) => { g.fillStyle(c).fillEllipse(x, y, w, h);

L21-25 FirstStatement: const leaf = () => { g.lineStyle(4, ink).lineBetween(0, -32, 3, -53);

L26-147 IfStatement: if (id === 0) { const points = Array.from({ length: 64 }, (_, i) => {

L148-148 ExpressionStatement: g.restore();

fruit.oval [L17-20]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: oval = (x: number, y: number, w: number, h: number, c: number) =>

Calls / delegates: g.fillStyle(c).fillEllipse, g.fillStyle, g.lineStyle(3, ink).strokeEllipse, g.lineStyle

fruit.leaf [L21-25]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: leaf = () =>

Calls / delegates: g.lineStyle(4, ink).lineBetween, g.lineStyle, g.fillStyle(0x80b66d).fillEllipse, g.fillStyle, g.lineStyle(2, ink).strokeEllipse

half [L150-182]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function half(g: Phaser.GameObjects.Graphics, id: number, x: number, y: number, r: number, left: boolean, angle: number,)

Calls / delegates: g.save, g.translateCanvas, g.rotateCanvas, g.fillStyle, g.beginPath, g.arc, g.closePath, g.fillPath, g.lineStyle(3, ink).strokePath, g.lineStyle, g.fillStyle(id === 2 ? ink : 0xb87350).fillEllipse, g.restore

Execution order (direct statements; nested callbacks stay within their owner):

L159-159 ExpressionStatement: g.save();

L160-160 ExpressionStatement: g.translateCanvas(x, y);

L161-161 ExpressionStatement: g.rotateCanvas(angle);

L162-162 FirstStatement: const a = left ? Math.PI / 2 : -Math.PI / 2;

L163-163 ExpressionStatement: g.fillStyle(skin[id] ?? skin[0]);

L164-164 ExpressionStatement: g.beginPath();

L165-165 ExpressionStatement: g.arc(0, 0, r, a, a + Math.PI);

L166-166 ExpressionStatement: g.closePath();

L167-167 ExpressionStatement: g.fillPath();

L168-168 ExpressionStatement: g.lineStyle(3, ink).strokePath();

L169-169 ExpressionStatement: g.fillStyle(id === 2 ? 0xf48e8d : id === 1 ? 0xffcf82 : 0xffe4b2);

L170-170 ExpressionStatement: g.beginPath();

L171-171 ExpressionStatement: g.arc(0, 0, r * 0.82, a, a + Math.PI);

L172-172 ExpressionStatement: g.closePath();

L173-173 ExpressionStatement: g.fillPath();

L174-180 ForStatement: for (let i = 0; i < 3; i++) g.fillStyle(id === 2 ? ink : 0xb87350).fillEllipse(

L181-181 ExpressionStatement: g.restore();

sword [L183-228]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function sword(g: Phaser.GameObjects.Graphics, x: number, y: number, rotation: number, color: number, stunned: boolean,)

Calls / delegates: g.save, g.translateCanvas, g.rotateCanvas, g.fillStyle(ink, 0.1).fillEllipse, g.fillStyle, bezier, g.fillStyle(stunned ? 0xb3b4bc : 0xedf5f4).fillPoints, g.lineStyle(5, ink).strokePoints, g.lineStyle, g.lineStyle(5, 0xc4d3d6) .lineBetween(-22, -28, -61, -97) .lineBetween, g.lineStyle(5, 0xc4d3d6) .lineBetween, g.lineStyle(16, ink).lineBetween, g.lineStyle(9, color).lineBetween, g.lineStyle(7, ink) .beginPath() .arc(0, 0, 24, -0.4, Math.PI + 1) .strokePath, g.lineStyle(7, ink) .beginPath() .arc, g.lineStyle(7, ink) .beginPath, g.lineStyle(3, color) .beginPath() .arc(0, 0, 24, -0.4, Math.PI + 1) .strokePath, g.lineStyle(3, color) .beginPath() .arc, g.lineStyle(3, color) .beginPath, g.restore

Return at L203: { x: u * u * a.x + 2 * u * t * b.x + t * t * c.x, y: u * u * a.y + 2 * u * t * b.y + t * t * c.y, }

Execution order (direct statements; nested callbacks stay within their owner):

L191-191 ExpressionStatement: g.save();

L192-192 ExpressionStatement: g.translateCanvas(x, y);

L193-193 ExpressionStatement: g.rotateCanvas(rotation);

L194-194 ExpressionStatement: g.fillStyle(ink, 0.1).fillEllipse(-25, -62, 58, 150);

L195-207 FirstStatement: const bezier = (a: { x: number; y: number },

L208-211 FirstStatement: const blade = [...bezier({ x: -8, y: -18 }, { x: -45, y: -119 }, { x: -139, y: -154 }),

L212-212 ExpressionStatement: g.fillStyle(stunned ? 0xb3b4bc : 0xedf5f4).fillPoints(blade, true);

L213-213 ExpressionStatement: g.lineStyle(5, ink).strokePoints(blade, true);

L214-216 ExpressionStatement: g.lineStyle(5, 0xc4d3d6) .lineBetween(-22, -28, -61, -97)

L217-217 ExpressionStatement: g.lineStyle(16, ink).lineBetween(-7, -7, 12, 20);

L218-218 ExpressionStatement: g.lineStyle(9, color).lineBetween(-7, -7, 12, 20);

L219-222 ExpressionStatement: g.lineStyle(7, ink) .beginPath()

L223-226 ExpressionStatement: g.lineStyle(3, color) .beginPath()

L227-227 ExpressionStatement: g.restore();

sword.bezier [L195-207]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: bezier = (a: { x: number; y: number }, b: { x: number; y: number }, c: { x: number; y: number },) =>

Return at L203: { x: u * u * a.x + 2 * u * t * b.x + t * t * c.x, y: u * u * a.y + 2 * u * t * b.y + t * t * c.y, }

src/games/fruit-duel/FruitGame.ts

ACTIVE SOURCE / tool or application | 206 lines | SHA-256 e794313147728683...

Fruit Duel lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/ai/MatchRandom, phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ./SwordMotion, ./Rules, ./AI, ./FruitArt

State, resources and initial values

Line	Field / declaration
11	private ctx!: GameContext;
12	private m = new FruitMatch();
13	private ai?: FruitAI;
14	private g!: Phaser.GameObjects.Graphics;
15	private paused = false;
16	private round = 0;
17	private done = 0;
18	private reported = false;
19	private seen = [0, 0];
20	private swing = [1, 1];
21	private cutAge = 2;
22	private cutSound = false;
23	private revealAge = 0;
24	private version = -1;

FruitGame [L10-205]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FruitGame.initialize [L25-44]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.ai, this.m

Calls / delegates: ctx.scene.add.graphics, FruitAI, FruitMatch, MatchRandom, mixSeed, freshSeed, ctx.input.tap, this.tap, ctx.input.press, ctx.setScore, ctx.setHint, ctx.audio.play

Decision branches: L28: ctx.options.mode === "ai"; L36: x > 24 && x < 396 && y > 112 && y < 650

FruitGame.tap [L45-47]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private tap(p: 0 | 1)

Calls / delegates: this.m.tap

Decision branches: L46: !this.paused && !this.reported && !(p === 1 && this.ai)

FruitGame.update [L48-168]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.version, this.revealAge, this.cutAge, this.seen[p], this.swing[p], this.round, this.cutSound, this.done, this.reported

Calls / delegates: this.m.update, this.ai?.update, this.ctx.audio.play, this.ctx.setScore, g.clear, rounded, fruit, half, g.fillStyle(i % 2 ? 0xffda85 : 0xf08d86, 1 - t / 0.6).fillEllipse, g.fillStyle, g.lineStyle(4, p === 0 ? theme.orange : theme.blue, Math.max(0, 1 - this.cutAge * 4),).lineBetween, g.lineStyle, g.fillStyle(0x51465c, 0.12).fillEllipse, this.m.missed.some, g.save, g.translateCanvas, g.rotateCanvas, g.restore, g.lineStyle(2, 0xffffff, 0.35).strokeCircle, swordPose, sword, this.m.present, this.ctx.finish

Decision branches: L49: this.paused || this.reported; L52: this.version !== this.m.version; L59: this.seen[p] !== this.m.attemptSerial[p]; L69: this.round !== this.m.round; L72: this.m.capturedBy !== null; L84: slicing && !this.cutSound && this.cutAge >= SWING_IMPACT; L88: slicing && this.cutAge < SWING_IMPACT; L90: slicing; L106: t < 0.6; L121: this.m.current >= 0; L160: this.m.phase === "show"; L161: this.m.winner; L163: this.done > 0.9

Execution order (direct statements; nested callbacks stay within their owner):

L49-49 IfStatement: if (this.paused || this.reported) return;

L50-50 ExpressionStatement: this.m.update(dt);

L51-51 ExpressionStatement: this.ai?.update(this.m);

L52-55 IfStatement: if (this.version !== this.m.version) { this.version = this.m.version;

L56-56 ExpressionStatement: this.revealAge += dt;

L57-57 ExpressionStatement: this.cutAge += dt;

L58-68 ForOfStatement: for (const p of [0, 1] as const) { if (this.seen[p] !== this.m.attemptSerial[p]) {

L69-77 IfStatement: if (this.round !== this.m.round) { this.round = this.m.round;

L78-78 FirstStatement: const g = this.g;

L79-79 ExpressionStatement: g.clear();

L80-80 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xc5a5ce, 4);

L81-83 FirstStatement: const slicing = this.m.capturedBy !== null &&

L84-87 IfStatement: if (slicing && !this.cutSound && this.cutAge >= SWING_IMPACT) { this.cutSound = true;

L88-143 IfStatement: if (slicing && this.cutAge < SWING_IMPACT) { fruit(g, this.m.capturedKind, 210, 381, 49);

L144-159 ForOfStatement: for (const p of [0, 1] as const) { const pose = swordPose(

L160-160 IfStatement: if (this.m.phase === "show") this.m.present();

L161-167 IfStatement: if (this.m.winner) { this.done += dt;

FruitGame.pause [L169-171]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

FruitGame.resume [L172-174]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

FruitGame.dispose [L175-178]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.reported

Calls / delegates: this.g.destroy

FruitGame.snapshot [L179-204]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot, isFruit, this.m.blockedUntil.map, ([0, 1] as const).map, swordPose

Return at L180: { ai: this.ai?.snapshot(), phase: this.m.phase, target: this.m.target, current: this.m.current, isFruit: isFruit(this.m.current), time: this.m.time, round: this.m.round, scores: [...this.m.scores], winner: this.m.winner, cooldowns: this.m.blockedUntil.map((t) => Math.max(0, t - this.m.time)), attempts: [...this.m.attemptSerial], swings: [...this.sw ... [continued in source reference]

src/games/fruit-duel/Rules.ts

ACTIVE SOURCE / tool or application | 114 lines | SHA-256 a84d86f80a1ec34f...

Owns real-fruit/decoy reveal, captured-at-input judgment, shared recovery timing and first-to-ten.

Imports: ./SwordMotion, ../../shared/rules/FirstResponse

State, resources and initial values

Line	Field / declaration
16	scores: [number, number] = [0, 0];
17	target = 0;
18	current = -1;
19	phase: "preview" | "show" | "active" | "between" | "done" = "preview";
20	time = 0;
21	visibleAt = 0;
22	version = 0;
23	round = 0;
24	message = "";
25	winner: 1 | 2 | null = null;
26	blockedUntil = [0, 0];
27	attemptSerial = [0, 0];
28	missed = [false, false];
29	swingStartedAt = [-100, -100];
30	recoveryDuration = [HIT_RECOVERY, HIT_RECOVERY];
31	capturedBy: 0 | 1 | null = null;
32	capturedKind = -1;
33	private until = 0.7;
34	private gap = 0;
35	private queue = new FirstResponse<{ kind: number }>();

Module constants and data

L3-11 fruits = ["Apple", "Orange", "Watermelon", "Banana", "Pomegranate", "Pineapple", "Strawberry",]

L12-12 decoys = ["Gift", "Medal", "Flower"]

isFruit [L13-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: isFruit = (id: number) =>

FruitMatch [L15-113]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FruitMatch.constructor [L36-36]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private random = Math.random)

```
constructor(private random = Math.random) {}
```

FruitMatch.canTap [L37-39]

Check the player's current action/cooldown eligibility.

Signature: canTap(p: 0 | 1)

Return at L38: this.phase !== "done" && this.time >= this.blockedUntil[p]

```
canTap(p: 0 | 1) { return this.phase !== "done" && this.time >= this.blockedUntil[p]; }
```

FruitMatch.tap [L40-56]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: tap(p: 0 | 1)

Direct writes: this.swingStartedAt[p], this.missed[p], this.recoveryDuration[p], this.blockedUntil[p]

Calls / delegates: this.canTap, isFruit, this.queue.add

Decision branches: L41: !this.canTap(p) || this.phase === "show"; L46: !valid

FruitMatch.present [L57-63]

Mark the already-rendered question/object legally visible for input/AI.

Signature: present()

Direct writes: this.phase, this.visibleAt, this.until

Calls / delegates: this.random

Decision branches: L58: this.phase === "show"

FruitMatch.reveal [L64-75]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private reveal()

Direct writes: this.current, this.gap, this.target, this.phase

Calls / delegates: this.random, isFruit

Decision branches: L69: isFruit(this.current)

FruitMatch.update [L76-112]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.time, this.capturedBy, this.capturedKind, this.winner, this.phase, this.until, this.current

Calls / delegates: this.queue.resolve, this.queue.clear, this.reveal

Decision branches: L78: this.phase === "done"; L80: response; L81: response.tie; L89: this.scores[p] >= 10; L101: this.phase === "between" && this.time >= this.until; L107: (this.phase === "preview" || this.phase === "active") && this.time >= this.until

Execution order (direct statements; nested callbacks stay within their owner):

L77-77 ExpressionStatement: this.time += Math.max(0, dt);

L78-78 IfStatement: if (this.phase === "done") return;

L79-79 FirstStatement: const response = this.queue.resolve(this.time);

L80-100 IfStatement: if (response) { if (response.tie) {

L101-111 IfStatement: if (this.phase === "between" && this.time >= this.until) { this.current = -1;

src/games/fruit-duel/SwordMotion.ts

ACTIVE SOURCE / tool or application | 64 lines | SHA-256 c58b6b0f607c85f7...

Defines shared sword impact/recovery times and mirrored eased rest/windup/strike/return poses.

Module constants and data

L2-2 SWING_IMPACT = 0.15

L3-3 HIT_RECOVERY = 0.72

L4-4 MISS_RECOVERY = 1.35

L10-13 rest = [{ x: 322, y: 615, angle: -0.25 }, { x: 98, y: 615, angle: 1.6 },]

L14-17 wind = [{ x: 338, y: 624, angle: -0.4 }, { x: 82, y: 624, angle: 1.75 },]

L18-21 impact = [{ x: 300, y: 575, angle: 0.4 }, { x: 120, y: 575, angle: 1.15 },]

L22-25 follow = [{ x: 180, y: 560, angle: 1.25 }, { x: 240, y: 560, angle: 0.1 },]

smooth [L5-8]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: smooth = (t: number) =>

Return at L7: u * u * (3 - 2 * u)

Pose [L9-9]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
type Pose = { x: number; y: number; angle: number };
```

mix [L26-35]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function mix(a: Pose, b: Pose, t: number): Pose

Calls / delegates: smooth

Return at L27: { ...a }

Return at L28: { ...b }

Return at L30: { x: a.x + (b.x - a.x) * s, y: a.y + (b.y - a.y) * s, angle: a.angle + (b.angle - a.angle) * s, }

Decision branches: L27: t <= 0; L28: t >= 1

swordPose [L36-63]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function swordPose(player: 0 | 1, attempt: number, age: number, duration: number,): Pose

Direct writes: pose

Calls / delegates: mix

Return at L60: player === 0 ? { ...pose } : { x: 420 - pose.x, y: 762 - pose.y, angle: pose.angle + Math.PI }

Decision branches: L45: attempt === 0; L46: age < 0.065; L47: age < SWING_IMPACT; L53: age < 0.26

src/games/fruit-duel/config.ts

ACTIVE SOURCE / tool or application | 17 lines | SHA-256 507476a33d8f4b8b...

Eager lightweight metadata for Fruit Duel: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "fruit-duel",
  name: "Fruit Duel",
  description: "Slice the fruit. Dodge the decoys.",
  instructions:
    "Tap your half to slice real fruit. Gifts, medals, flowers and empty swings cause a 1.35s cooldown. Simultaneous
slices replay. First to 10. Orange: Space; Blue: Enter.",
  color: "#d6bddb",
  accent: "#b693be",
  preview,
  order: 11,
  tag: "A FRUITY FACE-OFF",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/fruit-duel/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 a20acc6d72771750...

Side-effect-free lazy factory for Fruit Duel; implementation initializes only after host selection.

Imports: ./FruitGame

Top-level block L2-2

```
export default () => new FruitGame();
```

src/games/gravity-runner

src/games/gravity-runner/AI.ts

ACTIVE SOURCE / tool or application | 80 lines | SHA-256 03c4b189075db373...

Game-local Gravity Runner observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules, ./RunnerPhysics

State, resources and initial values

Line	Field / declaration
7	readonly personality: MatchRandom;
8	private wait = 0;

GravityAI [L6-79]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GravityAI.constructor [L9-14]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

GravityAI.update [L15-78]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(m: GravityMatch, dt = 1 / 120)

Direct writes: this.wait, positions[0], positions[i], p.gravity, p.cooldown, hold, flip

Calls / delegates: this.personality.sample, m.speed, Float64Array, m.gaps.filter, m.obstacles.filter, advanceRunner, survival

Return at L56: i * h

Return at L58: steps * h

Return at L77: flip > hold + 0.035

Decision branches: L16: m.between || m.winner || !m.players[1].alive; L18: this.wait > 0; L24: m.players[1].cooldown > 0; L43: !gaps.length && !obstacles.length; L46: flipNow; L51: i === secondAt && p.cooldown <= 0; L56: !p.alive; L62: stay >= horizon; L65: this.difficulty !== "medium"; L71: delay >= horizon

Execution order (direct statements; nested callbacks stay within their owner):

L16-16 IfStatement: if (m.between || m.winner || !m.players[1].alive) return false;

L17-17 ExpressionStatement: this.wait -= dt;

L18-18 IfStatement: if (this.wait > 0) return false;

L19-23 ExpressionStatement: this.wait = this.personality.sample({ medium: 0.23, hard: 0.085, impossible: 0.04 }[this.difficulty],

L24-24 IfStatement: if (m.players[1].cooldown > 0) return false;

L25-29 FirstStatement: const horizon = Math.min(410 / m.speed(),

L30-31 FirstStatement: const h = 1 / 120, steps = Math.ceil(horizon / h);

L32-32 FirstStatement: const positions = new Float64Array(steps + 1);

L33-33 ExpressionStatement: positions[0] = m.distance;

L34-35 ForStatement: for (let i = 1; i <= steps; i++) positions[i] = positions[i - 1] + m.speed(m.age + i * h) * h;

L36-36 FirstStatement: const end = positions[steps] + 18;

L37-39 FirstStatement: const gaps = m.gaps.filter((g) => g.x + g.width >= m.distance - 15 && g.x <= end,

L40-42 FirstStatement: const obstacles = m.obstacles.filter((o) => o.x + o.width >= m.distance - 15 && o.x <= end,

L43-43 IfStatement: if (!gaps.length && !obstacles.length) return false;

L44-59 FirstStatement: const survival = (flipNow: boolean, secondAt = Infinity) => { const p = { ...m.players[1] };

L60-60 FirstStatement: const stay = survival(false);

L62-62 IfStatement: if (stay >= horizon) return false;

L63-64 FirstStatement: let hold = stay, flip = survival(true);

L65-76 IfStatement: if (this.difficulty !== "medium") { const delays =

L77-77 ReturnStatement: return flip > hold + 0.035;

GravityAI.update.survival [L44-59]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: survival = (flipNow: boolean, secondAt = Infinity) =>

Direct writes: p.gravity, p.cooldown

Calls / delegates: advanceRunner

Return at L56: i * h

Return at L58: steps * h

Decision branches: L46: flipNow; L51: i === secondAt && p.cooldown <= 0; L56: !p.alive

src/games/gravity-runner/CourseStream.ts

ACTIVE SOURCE / tool or application | 178 lines | SHA-256 b00b3a951f6a6ff0...

Generates and retires bounded seeded weighted endless-course motifs using map-specific trap families.

Imports: ../../shared/ai/MatchRandom, ./RunnerPhysics

State, resources and initial values

Line	Field / declaration
73	next = 350;
74	readonly history: Pattern[] = [];
75	private random: MatchRandom;

Module constants and data

L17-70 TRAPS = { ice: { side: ["icicle", "ice-block"], center: "ice-block", patterns: ["single", "switchback", "gaps", "gaps", "rail", "pinch", "islands",], }, green: { side: ["root", "bramble"], center: "bramble", patterns: ["cluster", "switchback", "cluster", "gaps", "rail", "pinch", "slalom",], }, blue: { side: ["jet"], center: "buoy", patterns: ["single", "switchback", "cluster", "slalom", "rail", "pinch", "islands",], }, purple: { side: ["crystal"], center: "phase", patterns: ["switchback", "cluster", "slalom", "slalom", "rail", "pinch", "gaps",], }, }

Pattern [L3-11]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Pattern = | "single" | "switchback" | "cluster" | "gaps" | "slalom" | "rail" | "pinch" | "islands";
```

Palette [L12-16]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
interface Palette { side: Obstacle["kind"][]; center: Obstacle["kind"]; patterns: Pattern[]; }
```

CourseStream [L72-177]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

CourseStream.constructor [L76-81]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(seed: number, readonly map: string,)

Direct writes: this.random

Calls / delegates: MatchRandom

CourseStream.append [L82-176]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: append(gaps: Gap[], obstacles: Obstacle[])

Direct writes: pattern, this.next, at

Calls / delegates: palette.patterns.filter, this.history.slice(-2).includes, this.history.slice, r.next, this.history.includes, this.history.push, this.history.shift, obstacles.push, gaps.push, hazard, obstacles.at

Decision branches: L90: this.history.length === 2 && !this.history.includes("rail"); L93: this.history.length > 16; L109: pattern === "rail" || pattern === "pinch"; L119: pattern === "rail"; L126: pattern === "islands"; L152: pattern === "gaps"

Execution order (direct statements; nested callbacks stay within their owner):

L83-84 FirstStatement: const r = this.random, palette = TRAPS[this.map] ?? TRAPS.green;

L85-87 FirstStatement: const options = palette.patterns.filter((p) => !this.history.slice(-2).includes(p),

L88-88 FirstStatement: let pattern = options[Math.floor(r.next() * options.length)];

L90-91 IfStatement: if (this.history.length === 2 && !this.history.includes("rail")) pattern = "rail";

L92-92 ExpressionStatement: this.history.push(pattern);

L93-93 IfStatement: if (this.history.length > 16) this.history.shift();

L94-95 FirstStatement: const x = this.next, side = r.next() < 0.5 ? 1 : -1;

L96-96 FirstStatement: const intensity = Math.min(1, Math.max(0, (x - 1200) / 11000));

L97-97 FirstStatement: const clearance = 245 + (1 - intensity) * 80 + r.next() * 55;

L98-108 FirstStatement: const hazard = (at: number, s: -1 | 0 | 1, center = false): Obstacle => ({ x: at,

L109-175 IfStatement: if (pattern === "rail" || pattern === "pinch") { const width =

CourseStream.append.hazard [L98-108]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: hazard = (at: number, s: -1 | 0 | 1, center = false): Obstacle =>

Calls / delegates: r.next

src/games/gravity-runner/GravityArt.ts

ACTIVE SOURCE / tool or application | 246 lines | SHA-256 b64ee22f165cc940...

Gravity Runner original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: ./TrapArt, phaser, ../../core/ThemeManager, ../../shared/components/drawing, ./Maps, ./Rules

Module constants and data

L7-7 left = 84

L8-8 right = 336

L9-9 scale = (right - left) / 170

trackY [L10-10]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: trackY = (x: number, distance: number) =>

```
trackY = (x: number, distance: number) => 548 - (x - distance) * 1.05
```

wall [L11-56]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function wall(g: Phaser.GameObjects.Graphics, side: -1 | 1, top: number, bottom: number, distance: number, map: RunnerMap,)

Direct writes: y

Calls / delegates: g.fillStyle(map.wall).fillRect, g.fillStyle, g.fillStyle(map.highlight).fillRect, g.lineStyle(3, theme.ink) .lineBetween(edge, top, edge, bottom) .lineBetween(x, top, x + 60, top) .lineBetween, g.lineStyle(3, theme.ink) .lineBetween(edge, top, edge, bottom) .lineBetween, g.lineStyle(3, theme.ink) .lineBetween, g.lineStyle, g.fillStyle(map.edge).fillRect, g.fillStyle(map.detail).fillEllipse, g.lineBetween

Decision branches: L19: bottom <= top

Execution order (direct statements; nested callbacks stay within their owner):

L19-19 IfStatement: if (bottom <= top) return;

L20-21 FirstStatement: const x = side === -1 ? 24 : right, edge = side === -1 ? left : right;

L22-22 ExpressionStatement: g.fillStyle(map.wall).fillRect(x, top, 60, bottom - top);

L23-28 ExpressionStatement: g.fillStyle(map.highlight).fillRect(x + (side === -1 ? 6 : 18),

L29-32 ExpressionStatement: g.lineStyle(3, theme.ink) .lineBetween(edge, top, edge, bottom)

L33-38 ExpressionStatement: g.fillStyle(map.edge).fillRect(side === -1 ? edge - 10 : edge,

L40-40 FirstStatement: const phase = (548 + distance * 1.05) % 13;

L41-51 ForStatement: for (let y = Math.ceil((top + 3 - phase) / 13) * 13 + phase;

L52-52 ExpressionStatement: g.lineStyle(1, map.detail, 0.35);

L53-53 FirstStatement: const soil = (548 + distance * 1.05) % 39;

L54-55 ForStatement: for (let y = Math.ceil((top - soil) / 39) * 39 + soil; y < bottom; y += 39) g.lineBetween(x + 8, y, x + 48, y);

course [L57-194]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function course(g: Phaser.GameObjects.Graphics, m: GravityMatch, distance = m.distance,)

Direct writes: edge, d, y

Calls / delegates: rounded, g.fillGradientStyle(map.sky, map.sky, map.haze, map.haze).fillRect, g.fillGradientStyle, g.fillStyle(map.haze, 0.32).fillEllipse, g.fillStyle, g.fillStyle(0xffffff, 0.65) .fillEllipse(i % 2 ? 160 : 270, y + 50, 46, 12) .fillCircle, g.fillStyle(0xffffff, 0.65) .fillEllipse, g.lineStyle, g.lineBetween(x - 4, y, x + 4, y) .lineBetween(x, y - 4, x, y + 4) .lineBetween, g.lineBetween(x - 4, y, x + 4, y) .lineBetween, g.lineBetween, g.strokeCircle, g.lineBetween(x - 3, y, x + 3, y).lineBetween, g.fillStyle(map.detail, 0.45).fillEllipse, trackY, wall, g.fillStyle(map.haze, 0.45).fillRect, ["ledge", "spikes", "beam", "bridge"].includes, drawTrap, g.fillStyle(color).fillRect, g.lineStyle(3, theme.ink).strokeRect, g.fillStyle(map.edge) .fillRect(x + 2, top + 2, 4, bottom - top - 4) .fillRect, g.fillStyle(map.edge)

.fillRect, g.lineStyle(2, map.detail, 0.65).lineBetween, g.fillStyle(0xa7c87f).fillRect, g.fillStyle(0xffd7a1).fillTriangle, g.lineStyle(2, 0xf1dbc2).lineBetween, g.lineStyle(2, theme.ink).strokePoints, g.lineStyle(3, 0xb9d1d6).lineBetween, circle, g.lineStyle(4, theme.ink).strokeRoundedRect

Decision branches: L82: map.id === "ice"; L86: map.id === "blue"; L87: map.id === "purple"; L95: gap.side !== side; L98: b <= 116 || a >= 646; L113: bottom < 112 || top > 650; L114: o.kind !== "platform" && !["ledge", "spikes", "beam", "bridge"].includes(o.kind); L140: o.kind === "platform"; L148: y < 114 || y > 648; L151: o.kind === "ledge"; L158: o.kind === "spikes"; L169: o.kind === "bridge"

Execution order (direct statements; nested callbacks stay within their owner):

L62-62 FirstStatement: const map = m.map;

L63-63 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, map.sky, 4);

L64-69 ExpressionStatement: g.fillGradientStyle(map.sky, map.sky, map.haze, map.haze).fillRect(24,

L70-76 ForStatement: for (let i = 0; i < 3; i++) { const y = 170 + ((i * 195 + distance * 0.09) % 580) - 80;

L78-90 ForStatement: for (let i = 0; i < 9; i++) { const x = 106 + ((i * 71) % 208),

L91-109 ForOfStatement: for (const side of [-1, 1] as const) { let edge = 116;

L110-192 ForOfStatement: for (const o of m.obstacles) { const top = trackY(o.x + o.width, distance),

L193-193 ExpressionStatement: g.lineStyle(4, theme.ink).strokeRoundedRect(24, 112, 372, 538, 20);

runners [L195-245]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function runners(g: Phaser.GameObjects.Graphics, m: GravityMatch, view?: { age: number; players: { y: number; vy: number; angle: number; landing: number }[]; },)

Calls / delegates: g.fillStyle(color, 0.15 / j).fillCircle, g.fillStyle, g.save, g.translateCanvas, g.rotateCanvas, g.scaleCanvas, g.lineStyle(4, theme.ink) .lineBetween(-4, 6, -5 + stride, 11) .lineBetween, g.lineStyle(4, theme.ink) .lineBetween, g.lineStyle, rounded, circle, g.fillStyle(theme.ink).fillCircle, g.lineStyle(2, color).lineBetween, g.lineStyle(3, theme.ink).lineBetween, g.restore, g.lineStyle(3, color, 0.7).strokeCircle

Decision branches: L211: p.alive; L241: !p.alive

Execution order (direct statements; nested callbacks stay within their owner):

L203-244 ForOfStatement: for (const i of [0, 1] as const) { const p = m.players[i],

src/games/gravity-runner/GravityGame.ts

ACTIVE SOURCE / tool or application | 132 lines | SHA-256 38c1082419110244...

Gravity Runner lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../shared/physics/FixedStep, ../../shared/ai/MatchRandom, ./Rules, ./GravityView, ./GravityArt, ./AI

State, resources and initial values

Line	Field / declaration
10	private ctx!: GameContext;
11	private m = new GravityMatch(freshSeed());
12	private ai?: GravityAI;
13	private g!: Phaser.GameObjects.Graphics;
14	private clip!: Phaser.GameObjects.Graphics;
15	private mask!: Phaser.Display.Masks.GeometryMask;
16	private fixed = new FixedStep();
17	private view = new GravityView();
18	private renderState?: ReturnType<GravityView["sample"]>;
19	private paused = false;
20	private done = false;
21	private score = "";
22	private finishAge = 0;

GravityGame [L9-131]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GravityGame.initialize [L23-47]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.clip, this.mask, this.m, this.ai

Calls / delegates: ctx.scene.add.graphics, ctx.scene.make.graphics, this.clip.fillStyle(0xffffff).fillRoundedRect, this.clip.fillStyle, this.clip.createGeometryMask, this.g.setMask, GravityMatch, GravityAI, ctx.input.tap, this.flip, ctx.input.press, ctx.setHint, ctx.setScore, this.view.reset, ctx.setClock, ctx.audio.play, this.draw

Decision branches: L31: ctx.options.mode === "ai"; L36: x >= 24 && x <= 396 && y >= 112 && y <= 650

GravityGame.flip [L48-54]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private flip(p: 0 | 1, ai = false)

Calls / delegates: this.m.flip, this.view.flip, this.ctx.audio.play

Decision branches: L49: this.paused || this.done || (p === 1 && this.ai && !ai); L50: this.m.flip(p)

GravityGame.update [L55-92]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.score, this.renderState, this.finishAge, this.done

Calls / delegates: this.fixed.advance, this.view.capture, this.ai?.update, this.flip, this.m.players.map, this.m.step, this.view.reset, this.ctx.audio.play, this.view.land, this.m.scores.join, this.ctx.setScore, this.view.sample, this.draw, this.ctx.finish

Decision branches: L56: this.paused || this.done; L59: this.ai?.update(this.m); L63: round !== this.m.round; L69: before[i].alive && !p.alive; L70: Math.abs(before[i].vy) > 80 && p.vy === 0 && p.alive; L76: this.m.scores.join() !== this.score; L77: this.score; L87: this.m.winner; L88: this.m.winner && this.finishAge > 0.65

Execution order (direct statements; nested callbacks stay within their owner):

L56-56 IfStatement: if (this.paused || this.done) return;

L57-75 ExpressionStatement: this.fixed.advance(dt, (h) => { this.view.capture(this.m);

L76-80 IfStatement: if (this.m.scores.join() !== this.score) { if (this.score) this.ctx.audio.play("goal");

L81-85 ExpressionStatement: this.renderState = this.view.sample(this.m,

L86-86 ExpressionStatement: this.draw();

L87-87 IfStatement: if (this.m.winner) this.finishAge += dt;

L88-91 IfStatement: if (this.m.winner && this.finishAge > 0.65) { this.done = true;

GravityGame.draw [L93-98]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Calls / delegates: g.clear, course, runners

GravityGame.pause [L99-102]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.fixed.reset

GravityGame.resume [L103-105]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

GravityGame.dispose [L106-111]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.done

Calls / delegates: this.g.destroy, this.mask.destroy, this.clip.destroy

GravityGame.snapshot [L112-130]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.m.speed, this.ai?.personality.snapshot

Return at L113: { players: this.m.players, gaps: this.m.gaps, obstacles: this.m.obstacles, sharedCourse: true, map: this.m.map.id, speed: this.m.speed(), age: this.m.age, patterns: [...this.m.patterns], renderState: this.renderState, distance: this.m.distance, scores: this.m.scores, round: this.m.round, between: this.m.between, winner: this.m.winner, ai: this.ai?. ... [continued in source reference]

src/games/gravity-runner/GravityView.ts

ACTIVE SOURCE / tool or application | 53 lines | SHA-256 db5943ae6711e104...

Captures/interpolates runner presentation without changing legal physics.

Imports: ./Rules

State, resources and initial values

Line	Field / declaration
4	private distance = 0;
5	private age = 0;
6	private y = [160, 160];
7	private vy = [0, 0];
8	readonly angles = [-Math.PI / 2, -Math.PI / 2];
9	private targets = [-Math.PI / 2, -Math.PI / 2];
10	readonly landing = [0, 0];

GravityView [L3-52]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GravityView.capture [L11-18]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: capture(m: GravityMatch)

Direct writes: this.distance, this.age, this.y[i], this.vy[i]

GravityView.reset [L19-26]

Reset this component's transient state for a new round/attempt.

Signature: reset(m: GravityMatch)

Direct writes: this.angles[i], this.targets[i], this.landing[i]

Calls / delegates: this.capture

GravityView.flip [L27-29]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: flip(p: number)

Direct writes: this.targets[p]

GravityView.land [L30-32]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: land(p: number)

Direct writes: this.landing[p]

GravityView.sample [L33-51]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: sample(m: GravityMatch, alpha: number, dt: number)

Direct writes: this.angles[i], this.landing[i]

Calls / delegates: m.players.map

Return at L40: { distance: this.distance + (m.distance - this.distance) * t, age: this.age + (m.age - this.age) * t, players: m.players.map((p, i) => ({ ...p, y: this.y[i] + (p.y - this.y[i]) * t, vy: this.vy[i] + (p.vy - this.vy[i]) * t, angle: this.angles[i], landing: this.landing[i], })), }

src/games/gravity-runner/Maps.ts

ACTIVE SOURCE / tool or application | 66 lines | SHA-256 040181d732bd7933...

Lightweight map definitions and preview data; canonical selected geometry is shared by rules, AI and rendering.

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
/** Lightweight map metadata; no renderer or simulation enters the menu bundle. */
export const MAPS = [
  {
    id: "ice",
    name: "Icy Heights",
    traps: "Icicles, frost blocks & fractured paths",
    sky: 0xdceff7,
    haze: 0xb4d7ed,
    wall: 0x8eaec6,
    highlight: 0xbad6e8,
    edge: 0xeefaff,
    rail: 0x7899b4,
    detail: 0xffffff,
  },
  {
    id: "green",
    name: "Green Valley",
    traps: "Roots, brambles & fallen log rails",
    sky: 0xe6f1d6,
    haze: 0xbcdcca,
    wall: 0xb98265,
    highlight: 0xc89272,
    edge: 0x91bb72,
    rail: 0x879c74,
    detail: 0xcfe8a4,
  },
  {
    id: "blue",
    name: "Blue Lagoon",
    traps: "Pulsing water jets & floating buoys",
    sky: 0xcbe8f6,
    haze: 0x9bc9e8,
    wall: 0x698cac,
    highlight: 0x8eb6cf,
    edge: 0x80d6e1,
    rail: 0x6087a2,
    detail: 0xd1f3f5,
  },
  {
    id: "purple",
    name: "Violet Night",
    traps: "Crystal teeth & timed phase gates",
    sky: 0xe6def3,
    haze: 0xc0b0db,
    wall: 0x8d7ba8,
    highlight: 0xab98c1,
    edge: 0xc3b4e2,
    rail: 0x807498,
    detail: 0xeee2ff,
  },
] as const;
export type RunnerMap = (typeof MAPS)[number];
export function mapPreview(map?: RunnerMap) {
  const m = map ?? MAPS[0],
    hex = (v: number) => `#${v.toString(16).padStart(6, "0")}`;
  const traps =
    m.id === "ice"
      ? '<path d="M28 24l10 28 10-28M204 114l8-28 8 28" fill="#b6e3ef"/>'
      : m.id === "green"
```

```
60 |        ? '<path d="M28 24l4 24h12l4-24M204 114l4-22h8l4 22" fill="#7d9e66"/>'
61 |        : m.id === "blue"
62 |          ? '<path d="M28 24v24h20V24M204 114V92h16v22" fill="#56b5d5"/><path d="M30 35h16m60 35h18" fill="none"
stroke="#fff"/>'
63 |          : '<path d="M38 24l10 13-10 13-10-13zM212 114l8-13-8-13-8 13z" fill="#b29ace"/>';
64 |  return `<svg class="map-preview" aria-hidden="true" width="80" height="80" xmlns="http://www.w3.org/2000/svg"
viewBox="0 0 240 138"><rect width="240" height="138" rx="16" fill="${hex(m.sky)}"/><g stroke="#252d32"
stroke-width="4" stroke-linejoin="round"><path d="M0 0H240V24H165V10H95V24H0Z" fill="${hex(m.wall)}"/><path d="M0
138H240V114H165V128H95V114H0Z" fill="${hex(m.wall)}"/><rect x="76" y="58" width="110" height="22" rx="4"
fill="${hex(m.rail)}"/>${traps}<circle cx="111" cy="94" r="10" fill="#f5a052"/><circle cx="152" cy="44" r="10"
fill="#50abd6"/></g><path d="M20 67H51m-8-7 8 7-8 7" fill="none" stroke="#252d32" stroke-width="3"
stroke-linecap="round"/>${map ? "" : '<text x="218" y="55" font-size="22" text-anchor="middle"
fill="#252d32">?</text>'}</svg>`;
65 | }
```

Module constants and data

L2-51 MAPS = [{ id: "ice", name: "Icy Heights", traps: "Icicles, frost blocks & fractured paths", sky: 0xdceff7, haze: 0xb4d7ed, wall: 0x8eaec6, highlight: 0xbad6e8, edge: 0xeefaff, rail: 0x7899b4, detail: 0xffffff, }, { id: "green", name: "Green Valley", traps: "Roots, brambles & fallen log rails", sky: 0xe6f1d6, haze: 0xbcdcca, wall: 0xb98265, highlight: 0xc89272, edge: 0x91bb72, rail: 0x879c74, detail: 0xcfe8a4, }, { id: "blue", name: "Blue Lagoon", traps: "Pulsing water jets & floating buoys", sky: 0xcbe8f6, haze: 0x9bc9e8, wall: 0x698cac, highlight: 0x8eb6cf, edge: 0x80d6e1, rail: 0x6087a2, detail: 0xd1f3f5, }, { id: "purple", name: "Violet Night", traps: "Crystal teeth & timed phase gates", sky: 0xe6def3, haze: 0xc0b0db, wall: 0x8d7ba8, highlight: 0xab98c1, edge: 0xc3b4e2, rail: 0x807498, detail: 0xeee2ff, },] as const

RunnerMap [L52-52]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type RunnerMap = (typeof MAPS)[number];
```

mapPreview [L53-65]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function mapPreview(map?: RunnerMap)

Calls / delegates: v.toString(16).padStart, v.toString, hex

Return at L64: `<svg class="map-preview" aria-hidden="true" width="80" height="80" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 240 138"><rect width="240" height="138" rx="16" fill="${hex(m.sky)}"/><g stroke="#252d32" stroke-width="4" stroke-linejoin="round"><path d="M0 0H240V24H165V10H95V24H0Z" fill="${hex(m.wall)}"/><path d="M0 138H240V114H165V128H95V114H0Z" ... [continued in source reference]

mapPreview.hex [L55-55]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: hex = (v: number) =>

```
hex = (v: number) => `#${v.toString(16).padStart(6, "0")}`
```

src/games/gravity-runner/ObstacleGeometry.ts

ACTIVE SOURCE / tool or application | 117 lines | SHA-256 fefc82f2016b957c...

Defines timed trap bounds, convex outlines and SAT contact for the canonical runner footprint.

Imports: ./RunnerPhysics

Point [L2-5]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Point { x: number; y: number; }
```

obstacleBounds [L7-24]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function obstacleBounds(o: Obstacle, age: number)

Direct writes: depth

Return at L23: { low, high: low + depth, active, warning }

Decision branches: L15: o.kind === "jet"

obstacleOutline [L25-83]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function obstacleOutline(o: Obstacle, age: number): Point[]

Calls / delegates: obstacleBounds

Return at L33: [{ x, y: (low + high) / 2 }, { x: mid, y: tip }, { x: end, y: (low + high) / 2 }, { x: mid, y: base },]

Return at L43: [{ x, y: base }, { x: mid, y: tip }, { x: end, y: base },]

Return at L52: [{ x, y: base }, { x: x + o.width * 0.18, y: tip }, { x: x + o.width * 0.82, y: tip }, { x: end, y: base },]

Return at L63: [{ x, y: base }, { x: x + o.width * 0.12, y: base + sign * depth * 0.65 }, { x: mid, y: base + sign * depth }, { x: end - o.width * 0.12, y: base + sign * depth * 0.65 }, { x: end, y: base },]

Return at L72: Array.from({ length: 8 }, (_, i) => ({ x: mid + (Math.cos((i * Math.PI) / 4) * o.width) / 2, y: (low + high) / 2 + (Math.sin((i * Math.PI) / 4) * (high - low)) / 2, }))

Return at L77: [{ x, y: low }, { x: end, y: low }, { x: end, y: high }, { x, y: high },]

Decision branches: L30: o.kind === "crystal"; L40: o.kind === "icicle"; L49: o.kind === "root"; L59: o.kind === "bramble" && o.side !== 0; L71: o.kind === "buoy" || o.kind === "bramble"

Execution order (direct statements; nested callbacks stay within their owner):

L26-29 FirstStatement: const { low, high } = obstacleBounds(o, age), x = o.x,

L30-39 IfStatement: if (o.kind === "crystal") { const base = o.side === 1 ? high : low,

L40-48 IfStatement: if (o.kind === "icicle") { const base = o.side === 1 ? high : low,

L49-58 IfStatement: if (o.kind === "root") { const base = o.side === 1 ? high : low,

L59-70 IfStatement: if (o.kind === "bramble" && o.side !== 0) { const base = o.side === 1 ? high : low,

L71-76 IfStatement: if (o.kind === "buoy" || o.kind === "bramble") { return Array.from({ length: 8 }, (_, i) => ({

L77-82 ReturnStatement: return [{ x, y: low },

touchesTrap [L85-116]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function touchesTrap(o: Obstacle, distance: number, y: number, age: number,)

Direct writes: lo, hi

Calls / delegates: obstacleBounds, ["icicle", "crystal", "root", "buoy", "bramble"].includes, obstacleOutline

Decision branches: L92: distance + 13 < o.x || distance - 13 > o.x + o.width; L93: !b.active || y + 9 <= b.low + 0.001 || y - 9 >= b.high - 0.001; L95: !["icicle", "crystal", "root", "buoy", "bramble"].includes(o.kind); L112: center + radius <= lo + 0.001 || center - radius >= hi - 0.001

src/games/gravity-runner/Rules.ts

ACTIVE SOURCE / tool or application | 105 lines | SHA-256 87352faa57d8bf27...

Owns shared endless course lifecycle, age-dependent speed, legal flips, sole-survival points and first-to-five.

Imports: ../../shared/ai/MatchRandom, ./RunnerPhysics, ./Maps, ./CourseStream

State, resources and initial values

Line	Field / declaration
12	players: [Runner, Runner] = [this.runner(), this.runner()];
13	scores: [number, number] = [0, 0];
14	gaps: Gap[] = [];
15	obstacles: Obstacle[] = [];
16	distance = 0;
17	round = 1;
18	age = 0;
19	total = 0;
20	between = 0;
21	winner: 1 | 2 | "draw" | null = null;
22	readonly map: RunnerMap;
23	private stream!: CourseStream;
24	private trimmed = -200;

GravityMatch [L11-104]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GravityMatch.constructor [L25-32]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly seed: number, mapId?: string,)

Direct writes: this.map

Calls / delegates: MAPS.find, mixSeed, this.course

GravityMatch.runner [L33-35]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private runner(): Runner

Return at L34: { y: 160, vy: 0, gravity: 1, alive: true, cooldown: 0 }

```
private runner(): Runner { return { y: 160, vy: 0, gravity: 1, alive: true, cooldown: 0 }; }
```

GravityMatch.course [L36-42]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private course()

Direct writes: this.stream, this.trimmed, this.gaps, this.obstacles

Calls / delegates: CourseStream, mixSeed, this.extend

GravityMatch.extend [L43-57]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private extend()

Direct writes: this.trimmed, this.gaps, this.obstacles

Calls / delegates: this.gaps.filter, this.obstacles.filter, this.stream.append

Decision branches: L45: this.distance - this.trimmed >= 180

GravityMatch.patterns [L58-60]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get patterns()

Return at L59: this.stream.history

```
get patterns() { return this.stream.history; }
```

GravityMatch.speed [L61-69]

Apply six eased +24 speed ramps from 120 toward264, with an 8s opening and20s stage spacing.

Signature: speed(age = this.age)

Direct writes: speed

Return at L68: speed

GravityMatch.flip [L70-76]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: flip(i: 0 | 1)

Direct writes: p.gravity, p.cooldown

Decision branches: L72: this.winner || this.between || !p.alive || p.cooldown > 0

GravityMatch.step [L77-103]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number)

Direct writes: this.total, this.between, this.players, this.age, this.distance, this.winner

Calls / delegates: this.runner, this.course, this.speed, this.extend, advanceRunner, this.players.some, this.scores.some

Decision branches: L78: this.winner; L80: this.between; L82: !this.between; L96: this.players.some((p) => !p.alive); L97: this.players[0].alive !== this.players[1].alive; L100: this.scores.some((s) => s >= 5)

Top-level block L10-10

```
export type { Gap, Obstacle, Runner } from "./RunnerPhysics";
```

src/games/gravity-runner/RunnerPhysics.ts

ACTIVE SOURCE / tool or application | 103 lines | SHA-256 fc546d85cf3d7977...

Integrates cross-track gravity/momentum, wall gaps, swept platform landings and shared trap contact.

Imports: ./ObstacleGeometry

Module constants and data

L36-36 RUNNER_REACH = 13

Gap [L2-6]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Gap { x: number; width: number; side: -1 | 1; }
```

Obstacle [L7-28]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Obstacle { x: number; width: number; side: -1 | 0 | 1; depth: number; kind: | "ledge" | "spikes" | "beam" | "bridge" | "platform" | "icicle" | "ice-block" | "root" | "bramble" | "jet" | "buoy" | "crystal" | "phase"; phase?: number; period?: number; }
```

Runner [L29-35]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Runner { y: number; vy: number; gravity: -1 | 1; alive: boolean; cooldown: number; }
```

advanceRunner [L38-102]

Integrate legal gravity momentum, wall support/gaps, runnable platform crossings and canonical trap collisions.

Signature: export function advanceRunner(p: Runner, distance: number, dt: number, gaps: readonly Gap[], obstacles: readonly Obstacle[], age = 0,)

Direct writes: p.cooldown, p.vy, p.y, p.alive

Calls / delegates: gaps.some, gap, obstacleBounds, touchesTrap

Decision branches: L46: !p.alive; L58: p.y >= 160 && !gap(1); L59: previousY > 160.001; L66: p.y <= 10 && !gap(-1); L67: previousY < 9.999; L75: distance + RUNNER_REACH < o.x || distance - RUNNER_REACH > o.x + o.width; L81: !active; L82: o.kind === "platform"; L85: previousY + 9 <= low + 0.001 && p.y + 9 >= low && p.vy >= 0; L90: previousY - 9 >= high - 0.001 && p.y - 9 <= high && p.vy <= 0; L96: touchesTrap(o, distance, p.y, age); L101: p.y < -22 || p.y > 192

Execution order (direct statements; nested callbacks stay within their owner):

L46-46 IfStatement: if (!p.alive) return;

L47-47 ExpressionStatement: p.cooldown = Math.max(0, p.cooldown - dt);

L48-48 FirstStatement: const previousY = p.y;

L49-49 ExpressionStatement: p.vy += p.gravity * 900 * dt;

L50-50 ExpressionStatement: p.y += p.vy * dt;

L51-57 FirstStatement: const gap = (side: number) => gaps.some(

L58-65 IfStatement: if (p.y >= 160 && !gap(1)) { if (previousY > 160.001) {

L66-73 IfStatement: if (p.y <= 10 && !gap(-1)) { if (previousY < 9.999) {

L74-100 ForOfStatement: for (const o of obstacles) { if (

L101-101 IfStatement: if (p.y < -22 || p.y > 192) p.alive = false;

advanceRunner.gap [L51-57]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: gap = (side: number) =>

Calls / delegates: gaps.some

src/games/gravity-runner/TrapArt.ts

ACTIVE SOURCE / tool or application | 87 lines | SHA-256 d447509d1036699a...

Gravity Runner original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: phaser, ../../core/ThemeManager, ./RunnerPhysics, ./Maps, ./ObstacleGeometry

drawTrap [L7-86]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function drawTrap(g: Phaser.GameObjects.Graphics, o: Obstacle, map: RunnerMap, distance: number, age: number,)

Direct writes: y

Calls / delegates: obstacleBounds, sy, sx, obstacleOutline(o, age).map, obstacleOutline, g.fillStyle(o.kind === "root" ? 0x9c7960 : color).fillPoints, g.fillStyle, g.lineStyle(3, bounds.warning ? 0xd4934f : theme.ink, bounds.active || bounds.warning ? 1 : 0.25,).strokePoints, g.lineStyle, g.lineStyle(2, map.detail, 0.9) .lineBetween(polygon[0].x, polygon[0].y, polygon[2].x, polygon[2].y) .lineBetween, g.lineStyle(2, map.detail, 0.9) .lineBetween, g.lineStyle(2, map.detail, 0.85) .lineBetween(base, top + 4, tip + (base - tip) * 0.25, mid) .lineBetween(base, mid, tip, mid) .lineBetween, g.lineStyle(2, map.detail, 0.85) .lineBetween(base, top + 4, tip + (base - tip) * 0.25, mid) .lineBetween, g.lineStyle(2, map.detail, 0.85) .lineBetween, g.lineStyle(2, 0xf1fcff, 0.85) .strokeRect(x + 5, top + 5, w - 10, bottom - top - 10) .lineBetween, g.lineStyle(2, 0xf1fcff, 0.85) .strokeRect, g.lineStyle(3, 0x4b6a49).lineBetween, g.lineStyle(2, 0x4b6a49).lineBetween, g.fillStyle(0xc8dc8a).fillEllipse, g.lineBetween, g.lineStyle(5, map.rail).lineBetween, g.lineStyle(4, 0xfaf3d7).lineBetween, g.fillStyle(map.rail).fillCircle, g.fillStyle(0xf4d8ff, 0.9).fillCircle

Decision branches: L34: bounds.active; L41: o.kind === "crystal"; L45: o.kind === "icicle"; L52: o.kind === "ice-block"; L56: o.kind === "root" || o.kind === "bramble"; L67: o.kind === "jet"; L73: o.kind === "buoy"; L76: o.kind === "phase"; L84: bounds.active

Execution order (direct statements; nested callbacks stay within their owner):

L14-15 FirstStatement: const sy = (x: number) => 548 - (x - distance) * 1.05, sx = (y: number) => 84 + (y * 252) / 170;

L16-18 FirstStatement: const bounds = obstacleBounds(o, age), top = sy(o.x + o.width),

L19-21 FirstStatement: const x = sx(bounds.low), w = ((bounds.high - bounds.low) * 252) / 170,

L22-25 FirstStatement: const polygon = obstacleOutline(o, age).map((p) => ({ x: sx(p.y),

L26-33 FirstStatement: const color = map.id === "ice"

L34-35 IfStatement: if (bounds.active) g.fillStyle(o.kind === "root" ? 0x9c7960 : color).fillPoints(polygon, true);

L36-40 ExpressionStatement: g.lineStyle(3,

L41-85 IfStatement: if (o.kind === "crystal") { g.lineStyle(2, map.detail, 0.9)

drawTrap.sy [L14-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: sy = (x: number) =>

```
sy = (x: number) => 548 - (x - distance) * 1.05
```

drawTrap.sx [L15-15]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: sx = (y: number) =>

```
sx = (y: number) => 84 + (y * 252) / 170
```

src/games/gravity-runner/config.ts

ACTIVE SOURCE / tool or application | 32 lines | SHA-256 b6de784bbabaac7a...

Eager lightweight metadata for Gravity Runner: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./Maps, ./assets/preview.svg?raw

Top-level block L4-31

```
export default {
  id: "gravity-runner",
  name: "Gravity Runner",
  description: "The ceiling is your escape route.",
  instructions:
    "Tap your half to flip between the corridor walls: orange below, blue above. Avoid spikes and gaps. When both
walls disappear, land on either face of the middle rail before the gap. Each landscape has its own traps. After 8
seconds, speed rises in smooth stages every 20 seconds. Runs continue until a collision. Last survivor scores; ties
replay. First to 5. Orange: Space/A. Blue: Up arrow/Enter/L.",
  color: "#e3edc6",
  accent: "#92b875",
  preview,
  order: 18,
  tag: "FLIP THE WORLD",
  variants: [
    {
      id: "random",
      name: "Random map",
      description: "A new landscape each match.",
      preview: mapPreview(),
    },
    ...MAPS.map((m) => ({
      id: m.id,
      name: m.name,
      description: m.traps,
      preview: mapPreview(m),
    })),
  ],
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/gravity-runner/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 6fb2d8e7de12fabb...

Side-effect-free lazy factory for Gravity Runner; implementation initializes only after host selection.

Imports: ./GravityGame

Top-level block L2-2

```
export default () => new GravityGame();
```

src/games/hammer-hit

src/games/hammer-hit/AI.ts

ACTIVE SOURCE / tool or application | 59 lines | SHA-256 0a283440e25f9e42...

Game-local Hammer Hit observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private threshold = 0;
7	private previous = 1;
8	private attempt = -1;

HammerAI [L4-58]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

HammerAI.constructor [L9-14]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: SeedInput,)

Direct writes: this.personality

Calls / delegates: MatchRandom

HammerAI.snapshot [L15-17]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L16: { ...this.personality.snapshot(), threshold: this.threshold }

HammerAI.update [L18-57]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(m: HammerMatch)

Direct writes: this.previous, this.attempt, this.threshold

Calls / delegates: p.next, m.position

Return at L56: hit

Decision branches: L19: m.cooldowns[1] > 0 || m.scores[1].length === 3; L23: this.attempt !== m.scores[1].length

Execution order (direct statements; nested callbacks stay within their owner):

L19-22 IfStatement: if (m.cooldowns[1] > 0 || m.scores[1].length === 3) { this.previous = 1;

L23-49 IfStatement: if (this.attempt !== m.scores[1].length) { this.attempt = m.scores[1].length;

L50-50 FirstStatement: const distance = Math.abs(m.position(1));

L51-54 FirstStatement: const hit = m.clocks[1] > 0.3 &&

L55-55 ExpressionStatement: this.previous = distance;

L56-56 ReturnStatement: return hit;

src/games/hammer-hit/HammerGame.ts

ACTIVE SOURCE / tool or application | 669 lines | SHA-256 4a6247d8c4d37d60...

Hammer Hit lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/physics/FixedStep, ./StrikePhysics, phaser, ../../shared/components/TurnView, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ./Rules, ./AI

State, resources and initial values

Line	Field / declaration
34	private ctx!: GameContext;
35	private view!: TurnView;
36	private fixed = new FixedStep();
37	private animationFixed = new FixedStep();
38	private ring = new ImpactRing();
39	private turnWait = 0.7;
40	private model = new HammerMatch();
41	private ai?: HammerAI;
42	private g!: Phaser.GameObjects.Graphics;
43	private numberTexts: Phaser.GameObjects.Text[] = [];
44	private turnBadgeText!: Phaser.GameObjects.Text;
45	private popupText!: Phaser.GameObjects.Text;
46	private paused = false;
47	private finishAge = 0;
48	private reported = false;
51	private currentTurn: 0 | 1 = 0;
54	private strikeAnim?: StrikeAnimation;
55	private hammerAngle = 0.32;
56	private hammerRestAngle = 0.32;
57	private sparks: Spark[] = [];
58	private bellFlashAge = 0;
59	private cameraShake = 0;
60	private totalElapsed = 0;

Spark [L11-20]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
interface Spark { x: number; y: number; vx: number; vy: number; life: number; maxLife: number; color: number; size: number; }
```

StrikeAnimation [L22-31]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
interface StrikeAnimation { physics: StrikePhysics; player: 0 | 1; score: number; elapsed: number; peakY: number; puckY: number; hitTriggered: boolean; bellTriggered: boolean; }
```

HammerGame [L33-668]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

HammerGame.initialize [L62-138]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.turnBadgeText, this.popupText, this.ai, this.view

Calls / delegates: ctx.scene.add.graphics, label, t.setFontStyle("700").setDepth, t.setFontStyle, this.numberTexts.push, bellText.setFontStyle("800").setDepth, bellText.setFontStyle, this.turnBadgeText.setFontStyle("700").setDepth, this.turnBadgeText.setFontStyle, this.popupText .setFontStyle("900") .setStroke("#fffaf0", 3) .setDepth(4) .setAlpha, this.popupText .setFontStyle("900") .setStroke("#fffaf0", 3) .setDepth, this.popupText .setFontStyle("900") .setStroke, this.popupText .setFontStyle, HammerAI, TurnView, ctx.input.tap, this.hit, ctx.input.press, ctx.setScore, ctx.setHint, ctx.audio.play

Decision branches: L112: ctx.options.mode === "ai"; L124: x > 20 && x < 400 && y > 100 && y < 700; L131: p === this.currentTurn

Execution order (direct statements; nested callbacks stay within their owner):

L63-63 ExpressionStatement: this.ctx = ctx;

L64-64 ExpressionStatement: this.g = ctx.scene.add.graphics();

L69-80 FirstStatement: const bandNumbers = ["0",

L81-93 ForStatement: for (let i = 0; i < 10; i++) { const y = 606 - (i + 0.5) * 36;

L96-96 FirstStatement: const bellText = label(ctx.scene, 210, 212, "100", 22, "#1d2328");

L97-97 ExpressionStatement: bellText.setFontStyle("800").setDepth(2);

L98-98 ExpressionStatement: this.numberTexts.push(bellText);

L101-101 ExpressionStatement: this.turnBadgeText = label(ctx.scene, 342, 136, "1 / 3", 12, "#596361");

L102-102 ExpressionStatement: this.turnBadgeText.setFontStyle("700").setDepth(3);

L105-105 ExpressionStatement: this.popupText = label(ctx.scene, 88, 380, "", 24, "#242b32");

L106-110 ExpressionStatement: this.popupText .setFontStyle("900")

L112-114 IfStatement: if (ctx.options.mode === "ai") { this.ai = new HammerAI(ctx.options.difficulty);

L116-121 ExpressionStatement: this.view = new TurnView(ctx.scene, [this.g,

L123-128 ExpressionStatement: ctx.input.tap((x, y) => { if (x > 20 && x < 400 && y > 100 && y < 700) {

L129-132 ExpressionStatement: ctx.input.press(["Space", "Enter"], (code) => { const p = code === "Space" ? 0 : 1;

L135-135 ExpressionStatement: ctx.setScore(0, 0);

L136-136 ExpressionStatement: ctx.setHint("");

L137-137 ExpressionStatement: ctx.audio.play("start");

HammerGame.hit [L140-169]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private hit(p: 0 | 1, ai = false)

Direct writes: this.strikeAnim

Calls / delegates: this.model.hit, StrikePhysics, this.ctx.setScore

Decision branches: L141: this.paused || this.reported; L142: !this.view.ready || this.turnWait > 0; L144: p !== this.currentTurn; L146: this.ai && p === 1 && !ai; L148: this.strikeAnim; L151: score !== null

HammerGame.update [L171-224]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.totalElapsed, this.turnWait, this.cameraShake, this.bellFlashAge, this.finishAge, this.reported

Calls / delegates: this.view.update, this.fixed.advance, this.model.update, this.ai?.update, this.hit, this.fixed.reset, this.animationFixed.advance, this.ring.step, this.updateStrike, this.updateSparks, this.render, this.updateUI, this.ctx.finish

Decision branches: L172: this.paused || this.reported; L176: this.turnWait === 0 && !this.strikeAnim; L181: this.currentTurn === 1 && this.ai?.update(this.model); L188: this.cameraShake > 0; L193: this.bellFlashAge > 0; L213: this.model.done; L215: this.finishAge > 1.4 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L172-172 IfStatement: if (this.paused || this.reported) return;

L173-173 ExpressionStatement: this.totalElapsed += dt;

L174-174 ExpressionStatement: this.view.update(dt);

L175-175 ExpressionStatement: this.turnWait = Math.max(0, this.turnWait - dt);

L176-185 IfStatement: if (this.turnWait === 0 && !this.strikeAnim) { this.fixed.advance(dt, (step) => {

L188-190 IfStatement: if (this.cameraShake > 0) { this.cameraShake = Math.max(0, this.cameraShake - dt * 18);

L193-195 IfStatement: if (this.bellFlashAge > 0) { this.bellFlashAge = Math.max(0, this.bellFlashAge - dt);

L198-201 ExpressionStatement: this.animationFixed.advance(dt, (step) => { this.ring.step(step);

L204-204 ExpressionStatement: this.updateSparks(dt);

L207-207 ExpressionStatement: this.render();

L210-210 ExpressionStatement: this.updateUI();

L213-223 IfStatement: if (this.model.done) { this.finishAge += dt;

HammerGame.updateStrike [L226-297]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private updateStrike(dt: number)

Direct writes: this.hammerAngle, anim.elapsed, anim.hitTriggered, this.cameraShake, anim.puckY, anim.bellTriggered, anim.peakY, this.bellFlashAge, this.strikeAnim, this.currentTurn, this.turnWait, this.model.clocks[this.currentTurn], this.model.cooldowns[this.currentTurn]

Calls / delegates: this.ctx.audio.play, anim.physics.step, this.ring.strike, this.spawnSparks, this.popupText .setText(String(anim.score)) .setColor, this.popupText .setText, this.popupText .setPosition(88 - 210, Math.max(220, anim.peakY) - 381 - Math.min(24, age * 24),) .setAlpha(Math.min(1, Math.max(0, (1.25 - t) / 0.35))) .setScale, this.popupText .setPosition(88 - 210, Math.max(220, anim.peakY) - 381 - Math.min(24, age * 24),) .setAlpha, this.popupText .setPosition, this.popupText.setAlpha, this.view.set

Decision branches: L227: !this.strikeAnim; L237: t < 0.07; L238: t < 0.16; L242: !anim.hitTriggered; L251: anim.physics.peaked && !anim.bellTriggered; L254: anim.physics.contacted; L264: anim.bellTriggered; L275: t >= 1.25 && anim.physics.settled; L280: !this.model.done; L283: this.currentTurn === 0; L284: s1 < 3; L286: s0 < 3

Execution order (direct statements; nested callbacks stay within their owner):

L227-231 IfStatement: if (!this.strikeAnim) { // Idle hammer breathing

L233-233 FirstStatement: const anim = this.strikeAnim;

L234-234 ExpressionStatement: anim.elapsed += dt;

L235-235 FirstStatement: const t = anim.elapsed;

L237-274 IfStatement: if (t < 0.07) this.hammerAngle = this.hammerRestAngle - (t / 0.07) * 0.42; else if (t < 0.16)

L275-296 IfStatement: if (t >= 1.25 && anim.physics.settled) { this.popupText.setAlpha(0);

HammerGame.spawnSparks [L299-314]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private spawnSparks(cx: number, cy: number, color: number, count = 8)

Calls / delegates: this.sparks.push

HammerGame.updateSparks [L316-328]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private updateSparks(dt: number)

Direct writes: s.life, s.x, s.y, s.vy

Calls / delegates: this.sparks.splice

Decision branches: L320: s.life <= 0

HammerGame.render [L330-482]

Build or refresh visual/DOM output from current state.

Signature: private render()

Calls / delegates: g.clear, rounded, turnBackground, circle, this.drawStar, g.fillStyle, g.beginPath, g.arc, g.closePath, g.fillPath, g.fillRect, g.fillStyle(fill).fillRect, g.lineStyle, g.moveTo, g.lineTo, g.strokePath, g.lineStyle(6, 0xffffff).lineBetween, g.lineStyle(6, 0xffea00, auraAlpha).strokeCircle, this.model.position, g.lineBetween, this.drawHammer, g.fillStyle(s.color, alpha).fillCircle

Decision branches: L413: this.bellFlashAge > 0

Execution order (direct statements; nested callbacks stay within their owner):

L331-331 FirstStatement: const g = this.g;

L332-332 ExpressionStatement: g.clear();

L334-334 FirstStatement: const shakeX = (Math.random() - 0.5) * this.cameraShake;

L335-335 FirstStatement: const shakeY = (Math.random() - 0.5) * this.cameraShake;

L338-347 ExpressionStatement: rounded(g,

L350-350 FirstStatement: const towerLeft = 147 + shakeX;

L351-351 FirstStatement: const towerRight = 273 + shakeX;

L352-352 FirstStatement: const towerWidth = 126;

L353-353 FirstStatement: const towerTop = 246 + shakeY;

L354-354 FirstStatement: const towerBottom = 606 + shakeY;

L355-355 FirstStatement: const segmentHeight = 36;

L356-356 FirstStatement: const domeCX = 210 + shakeX;

L357-357 FirstStatement: const domeCY = 212 + shakeY;

L360-360 ExpressionStatement: circle(g, domeCX, domeCY, 83, 0x414b48, 4);

L361-368 ExpressionStatement: circle(g,

L369-379 ForOfStatement: for (const angle of [-2.95, -2.55, -2.15, -1.57, -0.99, -0.59, -0.19]) { const a = angle + this.ring.angle;
L382-382 ExpressionStatement: g.fillStyle(0xfec32e);
L383-383 ExpressionStatement: g.beginPath();
L384-384 ExpressionStatement: g.arc(domeCX, domeCY, 63, Math.PI, 0, false);
L385-385 ExpressionStatement: g.closePath();
L386-386 ExpressionStatement: g.fillPath();
L387-387 ExpressionStatement: g.fillRect(towerLeft, domeCY, towerWidth, towerTop - domeCY);
L390-394 ForStatement: for (let i = 0; i < 10; i++) { const y = towerBottom - (i + 1) * segmentHeight;
L397-397 ExpressionStatement: g.lineStyle(4, theme.ink);
L398-398 ExpressionStatement: g.beginPath();
L399-399 ExpressionStatement: g.moveTo(towerLeft, towerBottom);
L400-400 ExpressionStatement: g.lineTo(towerLeft, domeCY);
L401-401 ExpressionStatement: g.arc(domeCX, domeCY, 63, Math.PI, 0, false);
L402-402 ExpressionStatement: g.lineTo(towerRight, towerBottom);
L403-403 ExpressionStatement: g.lineTo(towerLeft, towerBottom);
L404-404 ExpressionStatement: g.strokePath();
L407-407 ExpressionStatement: g.lineStyle(6, 0xffffff).lineBetween(domeCX, domeCY, domeCX, towerBottom);
L410-410 ExpressionStatement: circle(g, domeCX, domeCY, 32, 0x2ecc71, 4);
L413-420 IfStatement: if (this.bellFlashAge > 0) { const auraAlpha = this.bellFlashAge / 0.6;
L424-424 ExpressionStatement: rounded(g, 120 + shakeX, 606 + shakeY, 180, 40, 10, 0x353b42, 4);
L427-427 ExpressionStatement: rounded(g, 197 + shakeX, 592 + shakeY, 26, 22, 6, 0xe74c3c, 3);
L430-430 ExpressionStatement: circle(g, 210 + shakeX, 614 + shakeY, 9, 0x2b3036, 3);
L431-431 ExpressionStatement: circle(g, 210 + shakeX, 614 + shakeY, 4, theme.ink, 0);
L434-434 FirstStatement: const activePlayer = this.currentTurn;
L435-437 FirstStatement: const isHold = this.model.cooldowns[activePlayer] > 0 &&
L438-440 FirstStatement: const needlePos = isHold ? this.model.locked[activePlayer]
L442-442 FirstStatement: const needleAngle = needlePos * 0.64;
L443-443 FirstStatement: const pivotX = 210 + shakeX;
L444-444 FirstStatement: const pivotY = 614 + shakeY;
L445-445 FirstStatement: const needleLen = 70;
L446-446 FirstStatement: const tipX = pivotX + Math.sin(needleAngle) * needleLen;
L447-447 FirstStatement: const tipY = pivotY - Math.cos(needleAngle) * needleLen;
L450-450 ExpressionStatement: g.lineStyle(4, theme.ink);
L451-451 ExpressionStatement: g.lineBetween(pivotX, pivotY, tipX, tipY);
L452-452 ExpressionStatement: circle(g, tipX, tipY, 3, theme.ink, 0);
L455-457 FirstStatement: const puckY = this.strikeAnim ? this.strikeAnim.puckY + shakeY
L458-458 ExpressionStatement: rounded(g, 198 + shakeX, puckY - 10, 24, 20, 6, 0xe74c3c, 3);
L461-461 ExpressionStatement: this.drawHammer(g, shakeX, shakeY);
L464-467 ForOfStatement: for (const s of this.sparks) { const alpha = s.life / s.maxLife;
L471-475 ForStatement: for (let i = 0; i < 3; i++) { const s = this.model.scores[0][i];
L477-481 ForStatement: for (let i = 0; i < 3; i++) { const s = this.model.scores[1][i];

HammerGame.drawStar [L484-514]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private drawStar(g: Phaser.GameObjects.Graphics, cx: number, cy: number, r: number, color: number, line = 2,)

Calls / delegates: points.push, g.fillStyle, g.beginPath, g.moveTo, g.lineTo, g.closePath, g.fillPath, g.lineStyle, g.strokePath

Decision branches: L510: line

HammerGame.drawHammer [L516-609]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private drawHammer(g: Phaser.GameObjects.Graphics, shakeX: number, shakeY: number,)

Calls / delegates: rot, g.fillStyle, g.beginPath, g.moveTo, g.lineTo, g.closePath, g.fillPath, g.lineStyle, g.strokePath, circle

Execution order (direct statements; nested callbacks stay within their owner):

L522-524 FirstStatement: const activePlayer = this.strikeAnim ? this.strikeAnim.player
L525-525 FirstStatement: const gripColor = activePlayer === 0 ? theme.orange : theme.blue;
L528-528 FirstStatement: const pivotX = 300 + shakeX;

L529-529 FirstStatement: const pivotY = 600 + shakeY;
L530-530 FirstStatement: const angle = 0.32 - (this.hammerAngle - 0.32) * 2.32;
L531-531 FirstStatement: const cos = Math.cos(angle);
L532-532 FirstStatement: const sin = Math.sin(angle);
L534-537 FirstStatement: const rot = (dx: number, dy: number) => ({ x: pivotX + dx * cos - (dy - 86) * sin,
L541-541 FirstStatement: const p1 = rot(-6, 0);
L542-542 FirstStatement: const p2 = rot(6, 0);
L543-543 FirstStatement: const p3 = rot(6, 84);
L544-544 FirstStatement: const p4 = rot(-6, 84);
L546-546 ExpressionStatement: g.fillStyle(0x8c502b);
L547-547 ExpressionStatement: g.beginPath();
L548-548 ExpressionStatement: g.moveTo(p1.x, p1.y);
L549-549 ExpressionStatement: g.lineTo(p2.x, p2.y);
L550-550 ExpressionStatement: g.lineTo(p3.x, p3.y);
L551-551 ExpressionStatement: g.lineTo(p4.x, p4.y);
L552-552 ExpressionStatement: g.closePath();
L553-553 ExpressionStatement: g.fillPath();
L554-554 ExpressionStatement: g.lineStyle(3, theme.ink);
L555-555 ExpressionStatement: g.strokePath();
L558-558 FirstStatement: const g1 = rot(-6, 38);
L559-559 FirstStatement: const g2 = rot(6, 38);
L560-560 FirstStatement: const g3 = rot(6, 70);
L561-561 FirstStatement: const g4 = rot(-6, 70);
L562-562 ExpressionStatement: g.fillStyle(gripColor);
L563-563 ExpressionStatement: g.beginPath();
L564-564 ExpressionStatement: g.moveTo(g1.x, g1.y);
L565-565 ExpressionStatement: g.lineTo(g2.x, g2.y);
L566-566 ExpressionStatement: g.lineTo(g3.x, g3.y);
L567-567 ExpressionStatement: g.lineTo(g4.x, g4.y);
L568-568 ExpressionStatement: g.closePath();
L569-569 ExpressionStatement: g.fillPath();
L572-572 FirstStatement: const pommel = rot(0, 86);
L573-573 ExpressionStatement: circle(g, pommel.x, pommel.y, 6, 0x8c502b, 3);
L577-582 FirstStatement: const headCorners = [rot(-32, -18),
L583-583 ExpressionStatement: g.fillStyle(0x41474d);
L584-584 ExpressionStatement: g.beginPath();
L585-585 ExpressionStatement: g.moveTo(headCorners[0].x, headCorners[0].y);
L586-586 ExpressionStatement: g.lineTo(headCorners[1].x, headCorners[1].y);
L587-587 ExpressionStatement: g.lineTo(headCorners[2].x, headCorners[2].y);
L588-588 ExpressionStatement: g.lineTo(headCorners[3].x, headCorners[3].y);
L589-589 ExpressionStatement: g.closePath();
L590-590 ExpressionStatement: g.fillPath();
L591-591 ExpressionStatement: g.lineStyle(4, theme.ink);
L592-592 ExpressionStatement: g.strokePath();
L595-595 FirstStatement: const pin1 = rot(-6, -24);
L596-596 FirstStatement: const pin2 = rot(6, -24);
L597-597 FirstStatement: const pin3 = rot(6, -18);
L598-598 FirstStatement: const pin4 = rot(-6, -18);
L599-599 ExpressionStatement: g.fillStyle(0x2b3036);
L600-600 ExpressionStatement: g.beginPath();
L601-601 ExpressionStatement: g.moveTo(pin1.x, pin1.y);
L602-602 ExpressionStatement: g.lineTo(pin2.x, pin2.y);
L603-603 ExpressionStatement: g.lineTo(pin3.x, pin3.y);
L604-604 ExpressionStatement: g.lineTo(pin4.x, pin4.y);
L605-605 ExpressionStatement: g.closePath();
L606-606 ExpressionStatement: g.fillPath();
L607-607 ExpressionStatement: g.lineStyle(2, theme.ink);

L608-608 ExpressionStatement: g.strokePath();

HammerGame.drawHammer.rot [L534-537]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: rot = (dx: number, dy: number) =>

```
rot = (dx: number, dy: number) => ({ x: pivotX + dx * cos - (dy - 86) * sin, y: pivotY + dx * sin + (dy - 86) * cos, })
```

HammerGame.updateUI [L611-634]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private updateUI()

Direct writes: roundNum

Calls / delegates: this.turnBadgeText.setText

Decision branches: L615: this.model.done; L622: this.strikeAnim && this.strikeAnim.player === this.currentTurn; L628: this.currentTurn === 0

HammerGame.pause [L636-640]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.fixed.reset, this.animationFixed.reset

HammerGame.resume [L642-644]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

HammerGame.dispose [L646-649]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.reported

Calls / delegates: this.view.destroy

HammerGame.snapshot [L651-667]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot, this.model.scores.map, this.model.position

Return at L652: { ai: this.ai?.snapshot(), scores: this.model.scores.map((s) => [...s]), totals: this.model.totals, averages: this.model.averages, clocks: [...this.model.clocks], positions: [this.model.position(0), this.model.position(1)], cooldowns: [...this.model.cooldowns], winner: this.model.winner, turn: this.currentTurn, turnWait: this.turnWait, rotation: th ... [continued in source reference]

src/games/hammer-hit/Rules.ts

ACTIVE SOURCE / tool or application | 53 lines | SHA-256 11724c721dde8aff...

Owns sine-gauge timing, three integer strikes per player, legal cooldown and total-based winner.

State, resources and initial values

Line	Field / declaration
9	scores: [number[], number[]] = [[], []];
10	clocks = [0, 0];
11	cooldowns = [0.6, 0.6];
12	locked = [-1, -1];

Module constants and data

L1-1 ATTEMPTS = 3

needle [L2-4]

Compute the sinusoidal gauge from elapsed clock and attempt-dependent frequency.

Signature: export function needle(time: number, attempt: number)

Return at L3: Math.sin(time * (2.6 + attempt * 0.35) - Math.PI / 2)

strikeScore [L5-7]

Convert the gauge distance from center into a rounded integer0..100 timing score.

Signature: export function strikeScore(position: number)

Return at L6: Math.round(100 * (1 - Math.min(1, Math.abs(position))))

HammerMatch [L8-52]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

HammerMatch.update [L13-21]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.cooldowns[p], this.clocks[p]

Decision branches: L15: this.cooldowns[p] > 0; L19: this.scores[p].length < ATTEMPTS

HammerMatch.position [L22-24]

Calculate the current logical gauge position used for scoring.

Signature: position(p: number)

Calls / delegates: needle

Return at L23: needle(this.clocks[p], this.scores[p].length)

HammerMatch.hit [L25-34]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: hit(p: 0 | 1)

Direct writes: this.locked[p], this.clocks[p], this.cooldowns[p]

Calls / delegates: this.position, strikeScore, this.scores[p].push

Return at L33: score

Decision branches: L26: this.cooldowns[p] > 0 || this.scores[p].length >= ATTEMPTS

HammerMatch.totals [L35-40]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get totals()

Calls / delegates: this.scores.map, s.reduce

Return at L36: this.scores.map((s) => s.reduce((a, b) => a + b, 0)) as [number, number,]

HammerMatch.done [L41-43]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get done()

Calls / delegates: this.scores.every

Return at L42: this.scores.every((s) => s.length === ATTEMPTS)

HammerMatch.winner [L44-48]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get winner(): 1 | 2 | "draw" | null

Return at L47: a === b ? "draw" : a > b ? 1 : 2

Decision branches: L45: !this.done

HammerMatch.averages [L49-51]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get averages()

Calls / delegates: this.totals.map

Return at L50: this.totals.map((n) => n / ATTEMPTS) as [number, number]

src/games/hammer-hit/StrikePhysics.ts

ACTIVE SOURCE / tool or application | 44 lines | SHA-256 d273da2efaec58dd...

Simulates presentation-only hammer puck flight and damped bell-ring motion; does not award scores.

State, resources and initial values

Line	Field / declaration
3	y = 602;
4	velocity: number;
5	settled = false;
6	contacted = false;
7	peaked = false;
34	angle = 0;
35	velocity = 0;

StrikePhysics [L2-32]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

StrikePhysics.constructor [L8-12]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(score: number)

Direct writes: this.velocity

StrikePhysics.step [L13-31]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number)

Direct writes: this.y, this.velocity, this.contacted, this.peaked, this.settled

Decision branches: L14: this.settled; L19: this.y <= 254 && this.velocity < 0; L24: previous <= 0 && this.velocity >= 0; L25: this.y >= 602 && this.velocity >= 0

ImpactRing [L33-43]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ImpactRing.strike [L36-38]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: strike()

Direct writes: this.velocity

ImpactRing.step [L39-42]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number)

Direct writes: this.velocity, this.angle

src/games/hammer-hit/config.ts

ACTIVE SOURCE / tool or application | 17 lines | SHA-256 8fb1d804153cb658...

Eager lightweight metadata for Hammer Hit: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "hammer-hit",
  name: "Hammer Hit",
  description: "Three swings. Find your rhythm.",
  instructions:
    "Take turns tapping when the needle reaches the center. The playfield faces the active local player. Each player
gets exactly three strikes. The arithmetic average of all three scores decides the winner. Orange: Space. Blue:
Enter.",
  color: "#f7d6ad",
  accent: "#d9a975",
  preview,
  order: 9,
  tag: "MAKE IT COUNT",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/hammer-hit/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 289b53812bae0e79...

Side-effect-free lazy factory for Hammer Hit; implementation initializes only after host selection.

Imports: ./HammerGame

Top-level block L2-2

```
export default () => new HammerGame();
```

src/games/happy-birds

src/games/happy-birds/AI.ts

ACTIVE SOURCE / tool or application | 62 lines | SHA-256 85e9917ad9f9a6f1...

Game-local Happy Birds observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private think = 0;

BirdsAI [L4-61]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

BirdsAI.constructor [L7-12]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

BirdsAI.ageReady [L13-15]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private ageReady(m: BirdsMatch, born: number)

Return at L14: m.age - born > 0.1

```
private ageReady(m: BirdsMatch, born: number) { return m.age - born > 0.1; }
```

BirdsAI.update [L16-60]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, m: BirdsMatch, player: 0 | 1 = 1)

Direct writes: this.think, target

Calls / delegates: this.personality.sample, m.speed, m.safeCenter, m.stars .filter((s) => this.ageReady(m, s.born) && (s.x - p.x) * p.direction > -20,) .sort, m.stars .filter, this.ageReady

Return at L54: (climb || (p.vy >= -20 && future > target + { medium: 20, hard: 10, impossible: 10 }[this.difficulty]))

Decision branches: L17: m.winner || m.between || !m.players[player].alive; L19: this.think > 0; L26: p.cooldown > 0; L40: ahead && eta > 0.25; L52: p.y < 48 && p.vy < 0

Execution order (direct statements; nested callbacks stay within their owner):

L17-17 IfStatement: if (m.winner || m.between || !m.players[player].alive) return false;

L18-18 ExpressionStatement: this.think -= dt;

L19-19 IfStatement: if (this.think > 0) return false;

L20-24 ExpressionStatement: this.think = this.personality.sample({ medium: 0.14, hard: 0.06, impossible: 0.024 }[this.difficulty],

L25-25 FirstStatement: const p = m.players[player];

L26-26 IfStatement: if (p.cooldown > 0) return false;

L27-28 FirstStatement: const side = p.direction === 1 ? 1 : 0, eta = (side === 0 ? p.x : 320 - p.x) / m.speed();

L29-32 FirstStatement: const desired = m.safeCenter(side) +

L33-33 FirstStatement: const mix = Math.max(0, Math.min(1, (1.9 - eta) / 0.8));

L34-34 FirstStatement: let target = 230 + (desired - 230) * mix;

L35-39 FirstStatement: const ahead = m.stars .filter(

L40-47 IfStatement: if (ahead && eta > 0.25) { // Route toward visible collectibles, then leave time to line up with the safe wall opening.

L48-50 FirstStatement: const look = { medium: 0.05, hard: 0.11, impossible: 0.11 }[this.difficulty

L51-51 FirstStatement: const future = p.y + p.vy * look + 325 * look * look;

L52-52 IfStatement: if (p.y < 48 && p.vy < 0) return false;

L53-53 FirstStatement: const climb = p.y > target + 35 && p.vy > -145;

L54-59 ReturnStatement: return (climb ||

src/games/happy-birds/ArenaPhysics.ts

ACTIVE SOURCE / tool or application | 91 lines | SHA-256 91f02064aa3e8b50...

Maintained implementation/configuration file. Use the source-grounded block map and complete companion listing for its exact control flow.

Module constants and data

L1-1 WIDTH = 320

L2-2 HEIGHT = 460

L3-3 RADIUS = 15

L4-4 GRAVITY = 650

L5-5 FLAP = -195

Bird [L6-18]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Bird { x: number; y: number; vy: number; direction: -1 | 1; alive: boolean; cooldown: number; bounces: number; flapSerial: number; recovery: number; hitX: number; hitY: number; }
```

Wall [L19-24]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Wall { rows: number[]; window: number; version: number; pending?: { rows: number[]; window: number; remaining: number }; }
```

rowY [L25-25]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: rowY = (index: number) =>

```
rowY = (index: number) => 44 + index * 44
```

segmentDistance [L26-41]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function segmentDistance(x: number, y: number, ax: number, ay: number, bx: number, by: number,)

Return at L40: (x - ax - t * dx) **2 + (y - ay - t * dy)** 2

spikeCollision [L42-55]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function spikeCollision(x: number, y: number, side: 0 | 1, row: number)

Calls / delegates: rowY, segmentDistance

Return at L47: (Math.min(segmentDistance(cx, cy, 0, -16, 18, 0), segmentDistance(cx, cy, 18, 0, 0, 16), segmentDistance(cx, cy, 0, 16, 0, -16),) <= RADIUS * RADIUS)

Decision branches: L45: cx > 18 + RADIUS || Math.abs(cy) > 16 + RADIUS; L46: cx >= 0 && cx <= 18 && Math.abs(cy) <= 16 * (1 - cx / 18)

advanceBird [L57-90]

Integrate auto-flight and flap gravity, collide with triangle hazards and reflect only safe side-wall contacts.

Signature: export function advanceBird(p: Bird, dt: number, speed: number, walls: readonly Wall[],): 0 | 1 | null

Direct writes: p.cooldown, p.x, p.vy, p.y, p.alive, p.direction

Calls / delegates: walls[side].rows.some, spikeCollision

Decision branches: L63: !p.alive; L68: p.y - RADIUS < 16 || p.y + RADIUS > HEIGHT - 16; L73: walls[side].rows.some((row) => spikeCollision(p.x, p.y, side, row)); L77: p.x <= RADIUS; L83: p.x >= WIDTH - RADIUS

Execution order (direct statements; nested callbacks stay within their owner):

L63-63 IfStatement: if (!p.alive) return null;

L64-64 ExpressionStatement: p.cooldown = Math.max(0, p.cooldown - dt);

L65-65 ExpressionStatement: p.x += p.direction * speed * dt;

L66-66 ExpressionStatement: p.vy += GRAVITY * dt;

L67-67 ExpressionStatement: p.y += p.vy * dt;

L68-71 IfStatement: if (p.y - RADIUS < 16 || p.y + RADIUS > HEIGHT - 16) { p.alive = false;

L72-76 ForOfStatement: for (const side of [0, 1] as const) if (walls[side].rows.some((row) => spikeCollision(p.x, p.y, side, row))) {

L77-82 IfStatement: if (p.x <= RADIUS) { p.x = RADIUS;

L83-88 IfStatement: if (p.x >= WIDTH - RADIUS) { p.x = WIDTH - RADIUS;

L89-89 ReturnStatement: return null;

src/games/happy-birds/BirdArt.ts

ACTIVE SOURCE / tool or application | 192 lines | SHA-256 89583e38584f9078...

Happy Birds original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: ./Recovery, phaser, ../../core/ThemeManager, ../../shared/components/drawing, ./ArenaPhysics, ./Rules, ./BirdView

Module constants and data

L8-8 scale = 372 / WIDTH

L9-9 ox = 24

L10-10 oy = 113

spikes [L11-40]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function spikes(g: Phaser.GameObjects.Graphics, side: 0 | 1, rows: readonly number[], ghost = false,)

Calls / delegates: rowY, g.fillStyle(0x787088).fillPoints, g.fillStyle, g.lineStyle(ghost ? 1.5 : 2.2, ghost ? 0xe6b966 : theme.ink, ghost ? 0.5 : 1,).strokePoints, g.lineStyle, g.lineStyle(1, 0xb2a6bf).lineBetween

Decision branches: L26: !ghost; L32: !ghost

star [L41-57]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function star(g: Phaser.GameObjects.Graphics, x: number, y: number, r: number, color: number, alpha = 1, angle = 0,)

Calls / delegates: g.fillStyle(color, alpha).fillPoints, g.fillStyle, g.lineStyle(1.5, theme.ink, alpha).strokePoints, g.lineStyle

Return at L53: { x: x + Math.cos(a) * size, y: y + Math.sin(a) * size }

arena [L58-191]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function arena(g: Phaser.GameObjects.Graphics, m: BirdsMatch, view: ReturnType<BirdView["sample"]>, trails: BirdView["trails"],)

Direct writes: x

Calls / delegates: g.fillStyle(0xc8e8ec).fillRect, g.fillStyle, g.save, g.translateCanvas, g.scaleCanvas, g.fillGradientStyle(0xdff1f2, 0xdff1f2, 0xbfe1e4, 0xbfe1e4).fillRect, g.fillGradientStyle, g.fillStyle(0xffffff, 0.22) .fillEllipse(x, y, 62, 17) .fillCircle(x - 8, y - 7, 14) .fillCircle, g.fillStyle(0xffffff, 0.22) .fillEllipse(x, y, 62, 17) .fillCircle, g.fillStyle(0xffffff, 0.22) .fillEllipse, g.fillStyle(0x787088).fillPoints, g.lineStyle(1.5, theme.ink).strokePoints, g.lineStyle, pts.map, g.strokePoints, spikes, g.fillStyle(0xffdc7b, 0.14 * appear).fillCircle, star, recoveryPose, recoveryColor, g.fillStyle(color, 0.1 + (5 - j) * 0.035).fillCircle,

g.rotateCanvas, g.fillStyle(color).fillTriangle, g.lineStyle(2.4, theme.ink).strokeTriangle, circle, g.fillStyle(0xfff7dc, 0.32).fillEllipse, g.fillStyle(recoveryColor(0xf6ce78, tint)).fillTriangle, g.lineStyle(2, theme.ink).strokeTriangle, g.fillStyle(theme.ink) .fillCircle(8.5, -5, 3) .fillStyle(0xffffff) .fillCircle, g.fillStyle(theme.ink) .fillCircle(8.5, -5, 3) .fillStyle, g.fillStyle(theme.ink) .fillCircle, g.fillStyle(recoveryColor(0xfff8df, tint)).fillEllipse, g.lineStyle(1.7, theme.ink).strokeEllipse, g.restore, g.lineStyle(1.5, color, 0.35 * (1 - recovery.progress)).strokeCircle, g.fillStyle(theme.ink, Math.ceil(m.between * 3) > i ? 0.45 : 0.12,).fillCircle

Decision branches: L98: m.walls[side].pending; L175: !p.alive; L184: m.between

Execution order (direct statements; nested callbacks stay within their owner):

L64-64 ExpressionStatement: g.fillStyle(0xc8e8ec).fillRect(24, 112, 372, 538);

L65-65 ExpressionStatement: g.save();

L66-66 ExpressionStatement: g.translateCanvas(ox, oy);

L67-67 ExpressionStatement: g.scaleCanvas(scale, scale);

L68-73 ExpressionStatement: g.fillGradientStyle(0xdff1f2, 0xdff1f2, 0xbfe1e4, 0xbfe1e4).fillRect(0,

L75-83 ForOfStatement: for (const [x, y] of [[65, 105],

L84-95 ForStatement: for (let x = 0; x < WIDTH; x += 24) { const pts = [

L96-100 ForOfStatement: for (const side of [0, 1] as const) { spikes(g, side, m.walls[side].rows);

L101-114 ForOfStatement: for (const s of m.stars) { const appear = Math.min(1, (m.age - s.born) / 0.18);

L115-129 ForOfStatement: for (const b of m.bursts) { const t = b.age / 0.5,

L130-183 ForOfStatement: for (const i of [0, 1] as const) { const p = view[i],

L184-189 IfStatement: if (m.between) for (let i = 0; i < 3; i++)

L190-190 ExpressionStatement: g.restore();

src/games/happy-birds/BirdView.ts

ACTIVE SOURCE / tool or application | 69 lines | SHA-256 ad47fe027d1d819c...

Interpolates bird positions/facing/wing motion, bounded trails and recovery tint.

Imports: ./Recovery, ./Rules

State, resources and initial values

Line	Field / declaration
4	private previous = [{ x: 100, y: 230, vy: 0 }, { x: 220, y: 230, vy: 0 },];
8	private faces = [1, -1];
9	private angles = [0, 0];
10	readonly wings = [0, 0];
11	readonly bumps = [0, 0];
12	readonly trails = [Array.from({ length: 6 }, () => ({ x: 100, y: 230 })), Array.from({ length: 6 }, () => ({ x: 220, y: 230 })),];
16	private trailAge = 0;

BirdView [L3-68]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

BirdView.capture [L17-22]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: capture(m: BirdsMatch)

BirdView.reset [L23-26]

Reset this component's transient state for a new round/attempt.

Signature: reset(m: BirdsMatch)

Calls / delegates: this.capture, this.resetBird

BirdView.resetBird [L27-35]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: resetBird(m: BirdsMatch, i: 0 | 1)

Direct writes: this.faces[i], this.angles[i], this.wings[i], this.bumps[i]

BirdView.sample [L36-67]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: sample(m: BirdsMatch, t: number, dt: number)

Direct writes: this.faces[i], this.angles[i], this.wings[i], this.bumps[i], this.trailAge

Calls / delegates: m.players.map, recoveryPose

Return at L46: { ...p, saturation: p.alive ? 1 : recoveryPose(p.recovery).tint, x: prev.x + (p.x - prev.x) * alpha, y: prev.y + (p.y - prev.y) * alpha, face: this.faces[i], angle: this.angles[i], wing: this.wings[i], bump: this.bumps[i], }

Return at L66: result

Decision branches: L58: this.trailAge >= 0.045

src/games/happy-birds/BirdsGame.ts

ACTIVE SOURCE / tool or application | 154 lines | SHA-256 3641267a682a9ee2...

Happy Birds lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../shared/physics/FixedStep, ../../shared/ai/MatchRandom, ./Rules, ./AI, ./BirdView, ../../shared/components/drawing, ./BirdArt

State, resources and initial values

Line	Field / declaration
11	private ctx!: GameContext;
12	private m = new BirdsMatch(freshSeed());
13	private ai?: BirdsAI;
14	private g!: Phaser.GameObjects.Graphics;
15	private clip!: Phaser.GameObjects.Graphics;
16	private mask!: Phaser.Display.Masks.GeometryMask;
17	private goalLabels: Phaser.GameObjects.Text[] = [];
18	private fixed = new FixedStep();
19	private view = new BirdView();
20	private renderState?: ReturnType<BirdView["sample"]>;
21	private paused = false;
22	private done = false;
23	private scores = "";
24	private finishAge = 0;

BirdsGame [L10-153]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

BirdsGame.initialize [L25-57]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.clip, this.mask, this.ai, this.m, this.goalLabels, this.renderState

Calls / delegates: ctx.scene.add.graphics, ctx.scene.make.graphics, this.clip.fillStyle(0xffffff).fillRoundedRect, this.clip.fillStyle, this.clip.createGeometryMask, this.g.setMask, BirdsAI, BirdsMatch, ctx.input.tap, this.flap, ctx.input.press, ctx.setHint, ctx.setScore, ctx.setClock, label, this.goalLabels[0].setRotation,

this.view.reset, this.view.sample, this.draw, ctx.audio.play

Decision branches: L32: ctx.options.mode === "ai"; L37: x >= 24 && x <= 396 && y >= 112 && y <= 650; L52: !this.ai

Execution order (direct statements; nested callbacks stay within their owner):

L26-26 ExpressionStatement: this.ctx = ctx;

L27-27 ExpressionStatement: this.g = ctx.scene.add.graphics();

L28-28 ExpressionStatement: this.clip = ctx.scene.make.graphics({ x: 0, y: 0 });

L29-29 ExpressionStatement: this.clip.fillStyle(0xffffff).fillRoundedRect(22, 110, 376, 542, 22);

L30-30 ExpressionStatement: this.mask = this.clip.createGeometryMask();

L31-31 ExpressionStatement: this.g.setMask(this.mask);

L32-35 IfStatement: if (ctx.options.mode === "ai") { this.ai = new BirdsAI(ctx.options.difficulty);

L36-39 ExpressionStatement: ctx.input.tap((x, y) => { if (x >= 24 && x <= 396 && y >= 112 && y <= 650)

L40-40 ExpressionStatement: ctx.input.press(["Space", "KeyA"], () => this.flap(0));

L41-43 ExpressionStatement: ctx.input.press(["ArrowUp", "ArrowDown", "ArrowLeft", "ArrowRight"], () => this.flap(this.ai ? 0 : 1),

L44-44 ExpressionStatement: ctx.input.press(["Enter", "KeyL"], () => this.flap(1));

L45-45 ExpressionStatement: ctx.setHint("");

L46-46 ExpressionStatement: ctx.setScore(0, 0);

L47-47 ExpressionStatement: ctx.setClock("", false);

L48-51 ExpressionStatement: this.goalLabels = [label(ctx.scene, 73, 86, "★ / 15", 12, "#a17e2d"),

L52-52 IfStatement: if (!this.ai) this.goalLabels[0].setRotation(Math.PI);

L53-53 ExpressionStatement: this.view.reset(this.m);

L54-54 ExpressionStatement: this.renderState = this.view.sample(this.m, 1, 0);

L55-55 ExpressionStatement: this.draw();

L56-56 ExpressionStatement: ctx.audio.play("start");

BirdsGame.flap [L58-64]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private flap(p: 0 | 1, ai = false)

Direct writes: this.view.wings[p]

Calls / delegates: this.m.flap, this.ctx.audio.play

Decision branches: L59: this.paused || this.done || (p === 1 && this.ai && !ai); L60: this.m.flap(p)

BirdsGame.update [L65-117]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.view.wings[i], this.view.bumps[i], this.renderState, this.scores, this.finishAge, this.done

Calls / delegates: this.fixed.advance, this.view.capture, this.ai?.update, this.flap, this.m.players.map, this.m.step, this.view.reset, this.ctx.audio.play, this.view.resetBird, this.view.sample, this.m.scores.join, this.ctx.setScore, this.draw, this.ctx.finish

Decision branches: L66: this.paused || this.done; L69: this.ai?.update(h, this.m); L78: this.m.round !== oldRound || (oldBetween > 0 && this.m.between === 0); L87: old[i].recovery > 0 && p.recovery === 0; L92: old[i].alive && !p.alive; L93: p.bounces > old[i].bounces; L104: this.m.scores.join() !== this.scores; L105: this.scores; L110: this.m.winner; L112: this.finishAge > 0.75

Execution order (direct statements; nested callbacks stay within their owner):

L66-66 IfStatement: if (this.paused || this.done) return;

L67-98 ExpressionStatement: this.fixed.advance(dt, (h) => { this.view.capture(this.m);

L99-103 ExpressionStatement: this.renderState = this.view.sample(this.m,

L104-108 IfStatement: if (this.m.scores.join() !== this.scores) { if (this.scores) this.ctx.audio.play("capture", 0.55);

L109-109 ExpressionStatement: this.draw();

L110-116 IfStatement: if (this.m.winner) { this.finishAge += dt;

BirdsGame.draw [L118-121]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Calls / delegates: this.g.clear, arena

BirdsGame.pause [L122-125]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.fixed.reset

BirdsGame.resume [L126-128]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

BirdsGame.dispose [L129-136]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.done, this.goalLabels

Calls / delegates: this.g.destroy, this.mask.destroy, this.clip.destroy, this.goalLabels.forEach, t.destroy

BirdsGame.snapshot [L137-152]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.personality.snapshot

Return at L138: { players: this.m.players, walls: this.m.walls, sharedArena: true, scores: this.m.scores, round: this.m.round, between: this.m.between, winner: this.m.winner, stars: this.m.stars, bursts: this.m.bursts, goal: 15, renderState: this.renderState, ai: this.ai?.personality.snapshot(), }

src/games/happy-birds/Recovery.ts

ACTIVE SOURCE / tool or application | 47 lines | SHA-256 db4536d01b3d34fb...

Owns the three-second individual bird recovery, curved return, tint and legal relaunch.

Imports: ./ArenaPhysics

Module constants and data

L2-2 RECOVERY_SECONDS = 3

home [L3-6]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: home = (player: 0 | 1) =>

```
home = (player: 0 | 1) => ({ x: player === 0 ? 140 : 180, y: 230, })
```

beginRecovery [L7-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function beginRecovery(p: Bird)

Direct writes: p.alive, p.recovery, p.hitX, p.hitY, p.vy, p.cooldown

clamp [L15-15]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clamp = (n: number) =>

```
clamp = (n: number) => Math.max(0, Math.min(1, n))
```

recoveryPose [L16-22]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function recoveryPose(remaining: number)

Calls / delegates: clamp

Return at L21: { progress, travel, tint: tint * tint * (3 - 2 * tint) }

advanceRecovery [L23-37]

Advance the eased curved return to a per-player home and re-enable legal flight at recovery completion.

Signature: export function advanceRecovery(p: Bird, player: 0 | 1, dt: number)

Direct writes: p.recovery, p.x, p.y, p.alive, p.direction, p.vy

Calls / delegates: recoveryPose, home

Decision branches: L29: p.recovery < 1e-8

recoveryColor [L39-46]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function recoveryColor(color: number, t: number)

Calls / delegates: channel

Return at L45: (channel(16) << 16) | (channel(8) << 8) | channel(0)

recoveryColor.channel [L41-44]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: channel = (shift: number) =>

src/games/happy-birds/Rules.ts

ACTIVE SOURCE / tool or application | 178 lines | SHA-256 f50502982e332775...

Owns shared wall telegraphs, star-contact points, independent recovery and first-to-15 continuous race.

Imports: ../../shared/ai/MatchRandom, ./ArenaPhysics, ./Recovery, ./Stars

State, resources and initial values

Line	Field / declaration
21	players: [Bird, Bird] = [this.bird(0), this.bird(1)];
22	walls: [Wall, Wall];
23	scores: [number, number] = [0, 0];
24	winner: 1 | 2 | "draw" | null = null;
25	age = 0;
26	total = 0;
27	round = 1;
28	between = 1.1;
29	stars: Star[] = [];
30	bursts: StarBurst[] = [];
31	private random: MatchRandom;
32	private nextStar = 0;

BirdsMatch [L20-177]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

BirdsMatch.constructor [L33-39]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly seed: number)

Direct writes: this.walls, this.random, this.stars

Calls / delegates: this.wall, MatchRandom, mixSeed, spawnStar

BirdsMatch.bird [L40-54]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private bird(p: number): Bird

Return at L41: { x: p === 0 ? 100 : 220, y: 230, vy: 0, direction: p === 0 ? 1 : -1, alive: true, cooldown: 0, bounces: 0, flapSerial: 0, recovery: 0, hitX: 0, hitY: 0, }

```
private bird(p: number): Bird { return { x: p === 0 ? 100 : 220, y: 230, vy: 0, direction: p === 0 ? 1 : -1, alive: true, cooldown: 0, bounces: 0, flapSerial: 0, recovery: 0, hitX: 0, hitY: 0, }; }
```

BirdsMatch.wall [L55-65]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private wall(side: number, window: number, version: number): Wall

Calls / delegates: MatchRandom, mixSeed, r.rank

Return at L64: { rows, window, version }

BirdsMatch.speed [L66-68]

Calculate the current legal movement speed from stage/age/score.

Signature: speed()

Return at L67: Math.min(165, 115 + this.age * 0.25)

BirdsMatch.flap [L69-76]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: flap(i: 0 | 1)

Direct writes: p.vy, p.cooldown

Decision branches: L71: this.winner || this.between || !p.alive || p.cooldown > 0

BirdsMatch.safeCenter [L77-79]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: safeCenter(side: 0 | 1)

Calls / delegates: rowY

Return at L78: rowY(this.walls[side].window + 1)

BirdsMatch.clear [L80-84]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private clear(side: 0 | 1)

Calls / delegates: this.players.every

Return at L81: this.players.every((p) => !p.alive || (side === 0 ? p.x : 320 - p.x) > 100,)

BirdsMatch.schedule [L85-97]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private schedule(side: 0 | 1)

Direct writes: wall.pending

Calls / delegates: MatchRandom, mixSeed, r.next, this.wall

Decision branches: L87: wall.pending

BirdsMatch.step [L98-176]

Advance wall telegraphs, individual flight/recovery and physical shared-star collection, then check the15-star finish.

Signature: step(dt: number)

Direct writes: this.total, this.between, this.age, w.pending.remaining, w.rows, w.window, w.pending, burst.age, this.bursts, this.scores[i], this.stars, this.winner

Calls / delegates: this.clear, this.bursts.filter, this.players.map, advanceRecovery, advanceBird, this.speed, beginRecovery, this.schedule, pickupDistance, this.bursts.push, collected.push, this.stars.filter, collected.includes, this.scores.some, spawnStar

Decision branches: L99: this.winner; L101: this.between; L108: w.pending; L110: w.pending.remaining <= 0 && this.clear(side); L127: p.recovery > 0; L132: !p.alive; L136: bounce !== null; L140: this.age - star.born < 0.18; L147: nearest > RADIUS + STAR_RADIUS; L149: Math.abs(distances[i] - nearest) < 1e-6; L155: collected.length; L157: this.scores.some((s) => s >= STAR_GOAL); L166: collected.length

Execution order (direct statements; nested callbacks stay within their owner):

L99-99 IfStatement: if (this.winner) return;

L100-100 ExpressionStatement: this.total += dt;

L101-104 IfStatement: if (this.between) { this.between = Math.max(0, this.between - dt);

L105-105 ExpressionStatement: this.age += dt;

L106-117 ForOfStatement: for (const side of [0, 1] as const) { const w = this.walls[side];

L118-118 ForOfStatement: for (const burst of this.bursts) burst.age += dt;

L119-119 ExpressionStatement: this.bursts = this.bursts.filter((b) => b.age < 0.5);

L120-124 FirstStatement: const before = this.players.map((p) => ({ x: p.x,

L125-137 ForOfStatement: for (const i of [0, 1] as const) { const p = this.players[i];

L138-138 FirstStatement: const collected: Star[] = [];

L139-154 ForOfStatement: for (const star of this.stars) { if (this.age - star.born < 0.18) continue;

L155-156 IfStatement: if (collected.length) this.stars = this.stars.filter((s) => !collected.includes(s));

L157-165 IfStatement: if (this.scores.some((s) => s >= STAR_GOAL)) { this.winner =

L166-175 IfStatement: if (collected.length) this.stars = [

Top-level block L19-19

```
export type { Bird, Wall } from "./ArenaPhysics";
```

src/games/happy-birds/Stars.ts

ACTIVE SOURCE / tool or application | 61 lines | SHA-256 b6571079f07f5215...

Defines star target/radius, bounded fair placement and swept bird-to-star pickup distance.

Imports: ../../shared/ai/MatchRandom, ./ArenaPhysics

Module constants and data

L3-3 STAR_GOAL = 15

L4-4 STAR_RADIUS = 10

Star [L5-10]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Star { id: number; x: number; y: number; born: number; }
```

StarBurst [L11-16]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface StarBurst { x: number; y: number; player: 0 | 1; age: number; }
```

spawnStar [L18-44]

Try up to64 seeded open-arena candidates and retain a clearance-respecting or farthest fallback star.

Signature: export function spawnStar(random: MatchRandom, id: number, age: number, players: readonly Bird[], previous?: { x: number; y: number },): Star

Direct writes: best, bestClearance

Calls / delegates: random.next, players .filter((p) => p.alive) .map, players .filter

Return at L43: { id, ...best, born: age }

Decision branches: L37: clearance > bestClearance; L41: clearance >= 1

pickupDistance [L45-60]

Return the shortest star distance to the bird's swept previous/current movement segment.

Signature: export function pickupDistance(s: Star, from: { x: number; y: number }, to: { x: number; y: number },)

Return at L59: Math.hypot(from.x + t * dx - s.x, from.y + t * dy - s.y)

src/games/happy-birds/config.ts

ACTIVE SOURCE / tool or application | 17 lines | SHA-256 95aaeceed8204529...

Eager lightweight metadata for Happy Birds: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "happy-birds",
  name: "Happy Birds",
  description: "Tiny wings. Fifteen golden stars.",
  instructions:
    "Tap to flap in the shared arena. Birds bounce off safe walls. Avoid spikes; outlined spikes warn before
changes. Collect the golden stars: first to 15 wins. A spike hit costs no stars: your bird turns gray, glides back
to the middle and regains its color over 3 seconds, then flies again. The other bird keeps playing. Orange: Space/A.
Blue: arrow keys (Enter/L also work). Solo: Space or arrows.",
  color: "#c8e7eb",
  accent: "#9ebcc2",
  preview,
  order: 19,
  tag: "CHASE THE STARS",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/happy-birds/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 5475372adca45a4a...

Side-effect-free lazy factory for Happy Birds; implementation initializes only after host selection.

Imports: ./BirdsGame

Top-level block L2-2

```
export default () => new BirdsGame();
```

src/games/light-fingers

src/games/light-fingers/AI.ts

ACTIVE SOURCE / tool or application | 58 lines | SHA-256 08525946456e7fe1...

Game-local Light Fingers observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
9	private serial = -1;
10	private delay = 1;
11	private tapped = false;
12	private retryAt = 0;

LightAI [L4-57]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

LightAI.snapshot [L6-8]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L7: { ...this.personality.snapshot(), reaction: this.delay }

LightAI.constructor [L13-18]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: SeedInput,)

Direct writes: this.personality

Calls / delegates: MatchRandom

LightAI.update [L19-56]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(m: LightMatch)

Direct writes: this.serial, this.tapped, this.delay, this.retryAt

Calls / delegates: style.sample, m.canTap, m.tap

Decision branches: L20: this.serial !== m.serial; L39: this.tapped && m.phase === "active" && m.time >= this.retryAt && m.canTap(1); L46: !this.tapped && m.phase === "active" && m.time - m.signalAt >= this.delay; L51: m.tap(1)

Execution order (direct statements; nested callbacks stay within their owner):

L20-38 IfStatement: if (this.serial !== m.serial) { this.serial = m.serial;

L39-45 IfStatement: if (this.tapped &&

L46-55 IfStatement: if (!this.tapped &&

src/games/light-fingers/LightArt.ts

ACTIVE SOURCE / tool or application | 75 lines | SHA-256 89fa4e9c61c6d57b...

Light Fingers original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: phaser, ../../core/ThemeManager, ../../shared/components/drawing

hand [L5-39]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function hand(g: Phaser.GameObjects.Graphics, x: number, y: number, rotation: number, color: number, reach: number,)

Calls / delegates: g.save, g.translateCanvas, g.rotateCanvas, rounded, g.restore, g.lineStyle(2, 0x81918b, 0.65).lineBetween, g.lineStyle

Execution order (direct statements; nested callbacks stay within their owner):

L13-13 ExpressionStatement: g.save();

L14-14 ExpressionStatement: g.translateCanvas(x, y);

L15-15 ExpressionStatement: g.rotateCanvas(rotation);

L16-16 ExpressionStatement: g.translateCanvas(0, -reach);

L17-17 ExpressionStatement: rounded(g, -19, -2, 38, 91, 10, 0x414b48, 4);

L18-18 ExpressionStatement: rounded(g, -20, 12, 40, 13, 4, color, 3);

L19-19 ExpressionStatement: rounded(g, -29, -51, 58, 61, 17, 0x414b48, 4);

L21-31 ForStatement: for (let i = 0; i < 4; i++) rounded(

L32-32 ExpressionStatement: g.save();

L33-33 ExpressionStatement: g.translateCanvas(-29, -24);

L34-34 ExpressionStatement: g.rotateCanvas(-0.7);

L35-35 ExpressionStatement: rounded(g, -10, -24, 18, 40, 8, 0x414b48, 4);

L36-36 ExpressionStatement: g.restore();

L37-37 ExpressionStatement: g.lineStyle(2, 0x81918b, 0.65).lineBetween(-17, -35, 17, -35);

L38-38 ExpressionStatement: g.restore();

pedestal [L40-52]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function pedestal(g: Phaser.GameObjects.Graphics, active: boolean, time: number,)

Calls / delegates: g.fillStyle, g.fillTriangle, rounded, circle, gem

Decision branches: L50: !active

gem [L53-74]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function gem(g: Phaser.GameObjects.Graphics, x: number, y: number, scale = 1, time = 0,)

Calls / delegates: g.save, g.translateCanvas, g.scaleCanvas, g.fillStyle(theme.blue).fillPoints, g.fillStyle, g.lineStyle(4, theme.ink).strokePoints, g.lineStyle, g.fillStyle(0xa9e6ee).fillTriangle, g.restore

src/games/light-fingers/LightGame.ts

ACTIVE SOURCE / tool or application | 158 lines | SHA-256 e1fb3579e61437a8...

Light Fingers lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/ai/MatchRandom, phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ./Rules, ./LightArt, ../../shared/components/TurnView, ./AI

State, resources and initial values

Line	Field / declaration
11	private ctx!: GameContext;
12	private m = new LightMatch();
13	private ai?: LightAI;
14	private g!: Phaser.GameObjects.Graphics;
15	private text!: Phaser.GameObjects.Text;
16	private prompts: Phaser.GameObjects.Text[] = [];
17	private paused = false;
18	private serial = 0;
19	private reported = false;
20	private finishAge = 0;
21	private celebration: 1 | 2 | null = null;
22	private previousScores = [0, 0];
23	private reach = [0, 0];

LightGame [L10-157]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

LightGame.initialize [L24-48]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.text, this.prompts, this.ai, this.m

Calls / delegates: ctx.scene.add.graphics, label, label(ctx.scene, 210, 137, "", 11).setRotation, LightAI, LightMatch, MatchRandom, mixSeed, ctx.input.tap, this.tap, ctx.input.press, ctx.setScore, ctx.setHint, ctx.audio.play

Decision branches: L33: ctx.options.mode === "ai"; L35: this.ai; L40: x > 24 && x < 396 && y > 112 && y < 650

LightGame.tap [L49-54]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private tap(p: 0 | 1)

Direct writes: this.reach[p]

Calls / delegates: this.m.tap

Decision branches: L50: !this.paused && !(p === 1 && this.ai); L52: accepted

LightGame.update [L55-131]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.serial, this.celebration, this.previousScores, this.reach[this.celebration - 1], this.reach, this.finishAge, this.reported

Calls / delegates: this.m.update, this.ai?.update, this.ctx.setScore, this.ctx.audio.play, this.m.message.includes, g.clear, rounded, turnBackground, pedestal, this.reach.map, hand, gem, this.text.setText, cooldown.toFixed, this.prompts[0].setText, blueCooldown.toFixed, this.m.presentSignal, this.ctx.finish

Decision branches: L56: this.paused; L59: this.serial !== this.m.serial; L68: this.celebration; L102: celebrating && this.celebration; L120: this.m.phase === "signal"; L124: this.m.winner; L126: this.finishAge > 0.75 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L56-56 IfStatement: if (this.paused) return;

L57-57 ExpressionStatement: this.m.update(dt);

L58-58 ExpressionStatement: this.ai?.update(this.m);

L59-71 IfStatement: if (this.serial !== this.m.serial) { this.serial = this.m.serial;

L72-72 FirstStatement: const g = this.g;

L73-73 ExpressionStatement: g.clear();

L74-74 FirstStatement: const celebrating = this.m.phase === "between" || this.m.phase === "done";

L75-86 ExpressionStatement: rounded(g,

L87-87 FirstStatement: const active = this.m.phase === "signal" || this.m.phase === "active";

L88-88 ExpressionStatement: pedestal(g, active, this.m.time);

L89-89 ExpressionStatement: this.reach = this.reach.map((age) => Math.max(0, age - dt));

L90-101 ForOfStatement: for (const p of [0] as const) { const travel =

L102-113 IfStatement: if (celebrating && this.celebration) { const age = 1 - this.reach[this.celebration - 1];

L114-114 FirstStatement: const cooldown = Math.max(0, this.m.blockedUntil[0] - this.m.time);

L115-115 ExpressionStatement: this.text.setText(cooldown > 0 ? `${cooldown.toFixed(1)}s` : "");

L116-116 FirstStatement: const blueCooldown = Math.max(0, this.m.blockedUntil[1] - this.m.time);

L117-119 ExpressionStatement: this.prompts[0].setText(blueCooldown > 0 ? `${blueCooldown.toFixed(1)}s` : "",

L120-123 IfStatement: if (this.m.phase === "signal") { this.ctx.audio.play("countdown");

L124-130 IfStatement: if (this.m.winner) { this.finishAge += dt;

LightGame.pause [L132-134]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

LightGame.resume [L135-137]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

LightGame.dispose [L138-142]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.g.destroy, this.text.destroy, this.prompts.forEach, p.destroy

LightGame.snapshot [L143-156]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot, this.m.blockedUntil.map

Return at L144: { ai: this.ai?.snapshot(), phase: this.m.phase, scores: [...this.m.scores], time: this.m.time, signalAt: this.m.signalAt, round: this.m.serial, winner: this.m.winner, hands: [...this.reach], cooldowns: this.m.blockedUntil.map((t) => Math.max(0, t - this.m.time)), capturedBy: this.m.capturedBy, }

src/games/light-fingers/Rules.ts

ACTIVE SOURCE / tool or application | 83 lines | SHA-256 21f1379025698deb...

Owns visible-signal gating, false starts, first-response arbitration, cooldown and first-to-five.

Imports: ../../shared/rules/FirstResponse

State, resources and initial values

Line	Field / declaration
3	scores: [number, number] = [0, 0];
4	phase: "waiting" | "signal" | "active" | "between" | "done" = "waiting";
5	time = 0;
6	signalAt = 0;
7	blockedUntil = [0, 0];
8	capturedBy: 0 | 1 | null = null;
9	serial = 0;
10	message = "WAIT FOR THE GEM";
11	winner: 1 | 2 | null = null;
12	private until: number;
13	private attempted = [false, false];
14	private input = new FirstResponse<boolean>();

LightMatch [L2-82]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

LightMatch.constructor [L15-17]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private random = Math.random)

Direct writes: this.until

Calls / delegates: random

LightMatch.canTap [L18-20]

Check the player's current action/cooldown eligibility.

Signature: canTap(player: 0 | 1)

Return at L19: this.time >= this.blockedUntil[player]

```
canTap(player: 0 | 1) { return this.time >= this.blockedUntil[player]; }
```

LightMatch.tap [L21-35]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: tap(player: 0 | 1)

Direct writes: this.blockedUntil[player], this.attempted[player]

Calls / delegates: this.canTap, this.input.add

Decision branches: L22: this.phase === "done" || this.phase === "signal" || !this.canTap(player); L28: this.phase !== "active"

LightMatch.presentSignal [L36-41]

Make the gem claimable only after its signal has been drawn.

Signature: presentSignal()

Direct writes: this.phase, this.signalAt

Decision branches: L37: this.phase === "signal"

LightMatch.update [L42-81]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.time, this.attempted, this.blockedUntil, this.message, this.capturedBy, this.blockedUntil[1 - p], this.winner, this.phase, this.until

Calls / delegates: this.input.resolve, this.random, this.input.clear

Decision branches: L44: this.phase === "done"; L46: response; L50: tie; L57: attempted[1 - p]; L60: this.scores[p] === 5; L71: this.phase === "waiting" && this.time >= this.until; L74: this.phase === "between" && this.time >= this.until

Execution order (direct statements; nested callbacks stay within their owner):

L43-43 ExpressionStatement: this.time += dt;

L44-44 IfStatement: if (this.phase === "done") return;

L45-45 FirstStatement: const response = this.input.resolve(this.time);

L46-70 IfStatement: if (response) { const { first, tie } = response;

L71-80 IfStatement: if (this.phase === "waiting" && this.time >= this.until) { this.phase = "signal";

src/games/light-fingers/config.ts

ACTIVE SOURCE / tool or application | 17 lines | SHA-256 c3bc042ad3301f88...

Eager lightweight metadata for Light Fingers: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "light-fingers",
  name: "Light Fingers",
  description: "Hold your nerve. Grab the gem.",
  instructions:
    "Wait until the diamond appears, then tap your side. First response steals the gem. Early or unsuccessful grabs
impose a short cooldown, never a score penalty. Simultaneous grabs leave the gem available after both cooldowns.
First to five. Orange: Space. Blue: Enter.",
  color: "#d4e7d8",
  accent: "#9dbbaa",
  preview,
  order: 10,
  tag: "READY. STEADY…",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/light-fingers/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 b0aca220512e1c60...

Side-effect-free lazy factory for Light Fingers; implementation initializes only after host selection.

Imports: ./LightGame

Top-level block L2-2

```
export default () => new LightGame();
```

src/games/lumberjack

src/games/lumberjack/AI.ts

ACTIVE SOURCE / tool or application | 57 lines | SHA-256 70d549a604da2062...

Game-local Lumberjack observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/contracts/AIController, ../../shared/ai/MatchRandom, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private wait: number;

LumberAI [L4-56]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

LumberAI.constructor [L7-13]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: SeedInput,)

Direct writes: this.personality, this.wait

Calls / delegates: MatchRandom, this.personality.sample

LumberAI.snapshot [L14-16]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L15: { ...this.personality.snapshot(), wait: this.wait }

LumberAI.update [L17-55]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, m: LumberMatch): Side | null

Direct writes: this.wait, side

Calls / delegates: p.sample, m.limit, ([-1, 1] as Side[]).filter, m.safe, room, p.next

Return at L35: i

Return at L36: 7

Return at L54: side ?? null

Decision branches: L18: m.phase !== "playing" || !m.players[1].alive; L23: this.wait > 0; L28: this.difficulty !== "medium"; L32: legal.length === 2; L35: m.pattern[v.score + i] === s; L52: p.next() < { medium: 0.1, hard: 0.012, impossible: 0 }[this.difficulty]

Execution order (direct statements; nested callbacks stay within their owner):

L18-21 IfStatement: if (m.phase !== "playing" || !m.players[1].alive) { this.wait = 0.35;

L22-22 ExpressionStatement: this.wait -= dt;

L23-23 IfStatement: if (this.wait > 0) return null;

L24-26 FirstStatement: const p = this.personality, v = m.players[1],

L27-27 ExpressionStatement: this.wait = p.sample(base * (0.88 + p.tempo * 0.12), base * 0.12, 0.095);

L28-29 IfStatement: if (this.difficulty !== "medium") this.wait = Math.min(this.wait, m.limit(1) * 0.8);

L30-30 FirstStatement: const legal = ([-1, 1] as Side[]).filter((side) => m.safe(1, side));

L31-31 FirstStatement: let side = legal[0];

L32-50 IfStatement: if (legal.length === 2) { // Only inspect the seven visible segments. Plan which side stays safe longest.

L52-53 IfStatement: if (p.next() < { medium: 0.1, hard: 0.012, impossible: 0 }[this.difficulty]) side = side === 1 ? -1 : 1;

L54-54 ReturnStatement: return side ?? null;

LumberAI.update.room [L34-37]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: room = (s: Side) =>

Return at L35: i

Return at L36: 7

Decision branches: L35: m.pattern[v.score + i] === s

src/games/lumberjack/LumberArt.ts

ACTIVE SOURCE / tool or application | 76 lines | SHA-256 7adb2a4b4f179313...

Lumberjack original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: phaser, ../../core/ThemeManager, ../../shared/components/drawing, ./Rules

forest [L6-75]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function forest(g: Phaser.GameObjects.Graphics, m: LumberMatch, p: 0 | 1, flip: boolean, scroll: number, swing: number, side: number,)

Calls / delegates: g.save, g.translateCanvas, g.rotateCanvas, rounded, g.fillStyle(0xd79e6a).fillRect, g.fillStyle, g.lineStyle(2, 0x895c40, 0.6).lineBetween, g.lineStyle, g.lineStyle(10, theme.ink).lineBetween, g.lineStyle(6, 0xb77b50).lineBetween, circle, g.scaleCanvas, g.lineStyle(5, theme.ink) .lineBetween(-9, -14, -11, 0) .lineBetween, g.lineStyle(5, theme.ink) .lineBetween, g.fillStyle(theme.ink).fillCircle, g.fillStyle(0x704a36).fillTriangle, g.lineStyle(7, theme.ink).lineBetween, g.lineStyle(4, 0xe0b77e).lineBetween, g.restore, m.limit

Decision branches: L19: flip; L28: !branch; L40: side > 0; L63: fraction > 0

Execution order (direct statements; nested callbacks stay within their owner):

L15-16 FirstStatement: const v = m.players[p], color = p === 0 ? theme.orange : theme.blue;
L17-17 ExpressionStatement: g.save();
L18-18 ExpressionStatement: g.translateCanvas(210, p === 0 ? 625 : flip ? 137 : 359);
L19-19 IfStatement: if (flip) g.rotateCanvas(Math.PI);
L21-21 ExpressionStatement: rounded(g, -166, -2, 332, 16, 7, 0xa6c982, 0);
L22-22 ExpressionStatement: rounded(g, -24, -226, 48, 228, 7, 0xb77b50, 3);
L23-23 ExpressionStatement: g.fillStyle(0xd79e6a).fillRect(-18, -222, 12, 218);
L24-36 ForStatement: for (let j = 0; j < 7; j++) { const y = -j * 28 - scroll,
L38-38 ExpressionStatement: g.save();
L39-39 ExpressionStatement: g.translateCanvas(side * 66, 3);
L40-40 IfStatement: if (side > 0) g.scaleCanvas(-1, 1);
L41-41 FirstStatement: const alive = v.alive ? color : 0x9ba39a;
L42-44 ExpressionStatement: g.lineStyle(5, theme.ink) .lineBetween(-9, -14, -11, 0)
L45-45 ExpressionStatement: rounded(g, -16, -40, 32, 27, 7, alive, 3);
L46-46 ExpressionStatement: circle(g, 0, -53, 13, 0xf2c395, 3);
L47-47 ExpressionStatement: rounded(g, -16, -67, 33, 12, 4, alive, 3);
L48-48 ExpressionStatement: g.fillStyle(theme.ink).fillCircle(6, -54, 2);
L50-50 ExpressionStatement: g.fillStyle(0x704a36).fillTriangle(-9, -48, 11, -48, 3, -37);
L51-51 ExpressionStatement: g.save();
L52-52 ExpressionStatement: g.translateCanvas(12, -29);
L53-53 ExpressionStatement: g.rotateCanvas(-0.7 + Math.sin(swing * Math.PI) * 1.1);
L54-54 ExpressionStatement: g.lineStyle(7, theme.ink).lineBetween(0, 0, 27, -23);
L55-55 ExpressionStatement: g.lineStyle(4, 0xe0b77e).lineBetween(0, 0, 27, -23);
L56-56 ExpressionStatement: rounded(g, 19, -34, 20, 15, 3, 0xd5e3df, 3);
L57-57 ExpressionStatement: g.restore();
L58-58 ExpressionStatement: circle(g, 13, -29, 5, 0xf2c395, 2);
L59-59 ExpressionStatement: g.restore();
L61-61 FirstStatement: const fraction = v.alive ? Math.max(0, 1 - v.idle / m.limit(p)) : 0;
L62-62 ExpressionStatement: rounded(g, -160, -195, 8, 156, 4, theme.paper, 2);
L63-73 IfStatement: if (fraction > 0) rounded(
L74-74 ExpressionStatement: g.restore();

src/games/lumberjack/LumberGame.ts

ACTIVE SOURCE / tool or application | 133 lines | SHA-256 ae9c1168aa23d28f...

Lumberjack lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ./Rules, ./LumberArt, ./AI

State, resources and initial values

Line	Field / declaration
9	private ctx!: GameContext;
10	private m = new LumberMatch(crypto.getRandomValues(new Uint32Array(1))[0]);
11	private ai?: LumberAI;
12	private g!: Phaser.GameObjects.Graphics;
13	private text!: Phaser.GameObjects.Text;
14	private paused = false;
15	private done = 0;
16	private reported = false;
17	private previousScores = "";
18	private scroll = [0, 0];
19	private swing = [0, 0];
20	private sides = [-1, 1];
21	private round = 1;

LumberGame [L8-132]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

LumberGame.initialize [L22-46]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.text, this.ai, this.m

Calls / delegates: ctx.scene.add.graphics, label, LumberAI, LumberMatch, ctx.input.tap, this.chop, ctx.input.press, key.startsWith, ctx.setHint, ctx.audio.play

Decision branches: L26: ctx.options.mode === "ai"; L28: this.ai; L30: x < 30 || x > 390 || y < 120 || y > 650

LumberGame.chop [L47-55]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private chop(p: 0 | 1, side: Side, ai = false)

Direct writes: this.swing[p], this.scroll[p]

Calls / delegates: this.m.chop, this.ctx.audio.play

Decision branches: L48: this.paused || (p === 1 && this.ai && !ai); L50: result !== null; L53: result

LumberGame.update [L56-109]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.round, this.scroll, this.swing, this.scroll[p], this.swing[p], this.sides[p], this.previousScores, this.done, this.reported

Calls / delegates: this.m.update, this.ai?.update, this.chop, g.clear, rounded, forest, this.m.totals.map, scores.join, this.ctx.setScore, this.text .setDepth(1) .setText, this.text .setDepth, this.ctx.finish

Decision branches: L57: this.paused; L60: side; L67: this.round !== this.m.round; L89: scores.join(",") !== this.previousScores; L102: this.m.winner; L104: this.done > 0.8 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L57-57 IfStatement: if (this.paused) return;

L58-58 ExpressionStatement: this.m.update(dt);

L59-59 FirstStatement: const side = this.ai?.update(dt, this.m);

L60-60 IfStatement: if (side) this.chop(1, side, true);

L61-61 FirstStatement: const g = this.g;

L62-62 ExpressionStatement: g.clear();

L63-63 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xdbe4c2, 4);

L65-65 ExpressionStatement: rounded(g, 30, 118, 360, 250, 16, 0xd8ece4, 0);

L66-66 ExpressionStatement: rounded(g, 30, 394, 360, 250, 16, 0xe3ead0, 0);

L67-71 IfStatement: if (this.round !== this.m.round) { this.round = this.m.round;

L72-85 ForOfStatement: for (const p of [0, 1] as const) { this.scroll[p] *= Math.exp(-dt * 26);

L86-88 FirstStatement: const scores = this.m.totals.map((s, i) => s + (this.m.phase === "playing" ? this.m.players[i].score : 0),

L89-92 IfStatement: if (scores.join(",") !== this.previousScores) { this.previousScores = scores.join(",");

L93-101 ExpressionStatement: this.text .setDepth(1)

L102-108 IfStatement: if (this.m.winner) { this.done += dt;

LumberGame.pause [L110-112]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

LumberGame.resume [L113-115]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

LumberGame.dispose [L116-119]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.g.destroy, this.text.destroy

LumberGame.snapshot [L120-131]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot, this.m.players.map, this.m.pattern.slice

Return at L121: { ai: this.ai?.snapshot(), phase: this.m.phase, round: this.m.round, players: this.m.players.map((p) => ({ ...p })), pattern: [...this.m.pattern.slice(0, 40)], totals: [...this.m.totals], time: this.m.time, winner: this.m.winner, }

src/games/lumberjack/Rules.ts

ACTIVE SOURCE / tool or application | 12 lines | SHA-256 3684347bfa600328...

Owns shared seeded branches, safe chops, shrinking idle budget, round caps and cumulative totals.

State, resources and initial values

Line	Field / declaration
5	pattern:number[];
5	players=[{score:0,side:-1 as Side,alive:true,idle:0,cooldown:0},{score:0,side:1 as Side,alive:true,idle:0,cooldown:0}];
5	totals:[number,number]=[0,0];
5	round=1;
5	phase:'playing'|'between'|'done'='playing';
5	time=0;
5	between=0;
5	winner:1|2|null=null;

Module constants and data

L2-2 ROUND_SECONDS = 20

Side [L1-1]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Side=-1|1;
```

branches [L3-3]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function branches(seed:number,count=600)

```
export function branches(seed:number,count=600){let s=seed>>>0,last=0;return
Array.from({length:count},(_,i)=>{s=(Math.imul(s,1664525)+1013904223)>>>0;const
value=i<2||last!==0?0:s%3-1;last=value;return value;});}
```

LumberMatch [L4-11]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

LumberMatch.constructor [L6-6]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly seed=12345)

```
constructor(readonly seed=12345){this.pattern=branches(seed);}
```

LumberMatch.limit [L7-7]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: limit(p:number)

```
limit(p:number){return Math.max(.32,1.5-this.players[p].score*.035);}
```

LumberMatch.safe [L8-8]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: safe(p:number,side:Side)

```
safe(p:number,side:Side){const index=this.players[p].score;return
this.pattern[index]!==side&&this.pattern[index+1]!==side;}
```

LumberMatch.chop [L9-9]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: chop(p:0|1,side:Side)

```
chop(p:0|1,side:Side){const v=this.players[p];if(this.phase!=='playing'||!v.alive||v.cooldown>0)return
null;v.side=side;v.cooldown=.085;if(!this.safe(p,side)){v.alive=false;return false;}v.score++;v.idle=0;return
true;}
```

LumberMatch.update [L10-10]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt:number)

Direct writes: this.between, this.time, this.pattern, p.score, p.alive, p.idle, p.cooldown, this.phase, this.totals[i], this.winner

Calls / delegates: branches, this.players.forEach, this.players.every

Decision branches: L10: this.phase==='done'; L10: this.phase==='between'; L10: this.between<=0; L10: p.alive; L10: p.idle>Math.max(.32,1.5-p.score*.035); L10: this.time>=ROUND_SECONDS; L10: this.players.every(p=>!p.alive); L10: this.round>=3&&this.totals[0]!==this.totals[1]

src/games/lumberjack/config.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 b1dcb25e57c9623d...

Eager lightweight metadata for Lumberjack: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L2-2

```
export default {id:'lumberjack',name:'Lumberjack',description:'Chop quickly. Mind the branches.',instructions:'Tap
left or right to chop from that side. Avoid branches at your feet and the next falling segment. Keep chopping before
the green timer empties. Three 20-second rounds; highest total chops wins. Ties add sudden-death rounds. Orange:
A/D. Blue:  <- / -> .',color:'#dbe4c2',accent:'#a4b389',preview,order:13,tag:'A CUT
ABOVE',supportedModes:['local-two-player','single-player'],tournament:true} satisfies GameMetadata;
```

src/games/lumberjack/index.ts

ACTIVE SOURCE / tool or application | 2 lines | SHA-256 0944dde60f12ce19...

Side-effect-free lazy factory for Lumberjack; implementation initializes only after host selection.

Imports: ./LumberGame

Top-level block L1-1

```
export default ()=>new LumberGame();
```

src/games/math-quiz

src/games/math-quiz/AI.ts

ACTIVE SOURCE / tool or application | 4 lines | SHA-256 e7e83f942be7171e...

Game-local Math Quiz observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
3	private id='';
3	private tapped=false;
3	private delay=1;
3	private answer=0;

QuizAI [L3-3]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

QuizAI.constructor [L3-3]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty:Difficulty,private random=Math.random)

```
constructor(private difficulty:Difficulty,private random=Math.random){}
```

QuizAI.update [L3-3]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(m:QuizMatch)

Direct writes: this.id, this.tapped, this.delay, this.answer

Calls / delegates: this.random, m.answer

Decision branches: L3: m.question.id!==this.id; L3: !this.tapped&&m.phase==='active'&&m.time-m.visibleAt>=this.delay

src/games/math-quiz/MathGame.ts

ACTIVE SOURCE / tool or application | 8 lines | SHA-256 15e04da886c51962...

Math Quiz lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/contracts/GameModule, ./Rules, ./AI, ./MathUI, ./questions/Basic, ./questions/Bank, ./questions/History

State, resources and initial values

Line	Field / declaration
3	private ctx!:GameContext;
3	private m!:QuizMatch;
3	private ai?:QuizAI;
3	private ui!:MathUI;
3	private paused=false;
3	private round=0;
3	private finished=0;
3	private reported=false;

MathGame [L2-7]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

MathGame.initialize [L4-4]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx:GameContext)

Direct writes: this.ctx, q, deck, last, this.m, this.ai, this.ui

Calls / delegates: universityDeck, basicQuestion, deck.push, deck.shift, globalQuestionHistory.record, QuizMatch, QuizAI, MathUI, this.m.answer, ctx.input.press, Number, code.at, ctx.setScore, ctx.setHint, ctx.audio.play, this.ui.render, this.m.present

Return at L4: q

Decision branches: L4: ctx.options.variant!=='university'; L4: !deck.length; L4: deck[0].id===last; L4: ctx.options.mode==='ai'; L4: !this.paused&&!(p===1&&this.ai); L4: !this.paused&&!(n>=7&&this.ai)

MathGame.initialize.next [L4-4]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: next=()=>

```
next=()=>{let q;if(ctx.options.variant!=='university')q=basicQuestion(serial++);else{if(!deck.length){deck=universityDeck();if(deck[0].id===last)deck.push(deck.shift()!);}q=deck.shift()!;}last=q.id;globalQuestionHistory.record(q.id);return q;}
```

MathGame.update [L5-5]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt:number)

Direct writes: this.round, this.finished, this.reported

Calls / delegates: this.m.update, this.ai?.update, this.ctx.setScore, this.ctx.audio.play, this.ui.render, this.m.present, this.ctx.finish

Decision branches: L5: this.paused; L5: this.round!==this.m.round; L5: this.m.winner; L5: this.finished>1.4&&!this.reported

MathGame.resize [L6-6]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: resize()

```
resize(){this.ui?.resize();}
```

MathGame.pause [L6-6]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

```
pause(){this.paused=true;this.ui.pause(true);}
```

MathGame.resume [L6-6]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

```
resume(){this.paused=false;this.ui.pause(false);this.resize();}
```

MathGame.dispose [L6-6]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

```
dispose(){this.ui.dispose();}
```

MathGame.snapshot [L6-6]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

```
snapshot(){return {phase:this.m.phase,question:this.m.question,scores:[...this.m.scores],round:this.m.round,time:this.m.time,winner:this.m.winner};}
```

src/games/math-quiz/MathUI.ts

ACTIVE SOURCE / tool or application | 138 lines | SHA-256 987f5bb2d5254555...

Owns the KaTeX DOM overlay, player-local topics/options, Blue rotation and dual-axis resize/font fitting.

Imports: katex, katex/dist/katex.min.css, ./quiz.css, ./Rules

State, resources and initial values

Line	Field / declaration
15	readonly root = document.createElement('div');
16	private id = '';
17	private phase = '';

renderMath [L6-12]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: renderMath = (value: string, node: HTMLElement) =>

Calls / delegates: katex.render

MathUI [L14-137]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

MathUI.constructor [L19-31]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(local: boolean, answer: (p: 0 | 1, index: number) => void)

Direct writes: this.root.className, this.root.innerHTML

Calls / delegates: this.side, this.root.addEventListener, (e.target as HTMLElement).closest, e.preventDefault, answer, Number, document.querySelector('#game-stage')!.append, document.querySelector, this.resize

Decision branches: L24: b && !b.disabled

MathUI.side [L33-39]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private side(p: number, local: boolean)

Calls / delegates: [0, 1, 2].map((i) => `<button data-player="${p}" data-answer="${i}" aria-label="${p === 1 ? 'Blue' : 'Orange'} answer ${i + 1}"></button>`).join, [0, 1, 2].map

Return at L34: `<section class="quiz-side ${p === 1 ? 'blue' : 'orange'} ${local ? 'local' : ''}" aria-label="${p === 1 ? 'Blue' : 'Orange'} answers"> <div class="quiz-topic"></div> <div class="quiz-formula"></div> <div class="quiz-options">${[0, 1, 2].map((i) => `<button data-player="${p}" data-answer="${i}" aria-label="${p === 1 ? 'Blue' : 'Orange'} answer ${i ... [continued in source reference]

MathUI.render [L41-74]

Build or refresh visual/DOM output from current state.

Signature: render(m: QuizMatch, ai: boolean)

Direct writes: this.id, t.textContent, this.phase, this.root.querySelector('.quiz-middle span')!.textContent, b.disabled

Calls / delegates: this.root.querySelectorAll<HTMLElement>('.quiz-topic').forEach, this.root.querySelectorAll, this.root.querySelectorAll<HTMLElement>('.quiz-formula').forEach, renderMath, this.root.querySelectorAll<HTMLButtonElement>('button').forEach, Number, this.fit, document.fonts?.ready?.then, this.root.querySelector, explain.replaceChildren, b.classList.toggle

Decision branches: L43: changed; L58: this.phase !== m.phase || changed; L63: m.phase === 'between' || m.phase === 'done'

Execution order (direct statements; nested callbacks stay within their owner):

L42-42 FirstStatement: const changed = this.id !== m.question.id;

L43-56 IfStatement: if (changed) { this.id = m.question.id;

L58-73 IfStatement: if (this.phase !== m.phase || changed) { this.phase = m.phase;

MathUI.fit [L76-116]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private fit()

Direct writes: n.style.fontSize, b.style.fontSize, explain.style.fontSize

Calls / delegates: this.root.querySelectorAll, n.querySelector, b.querySelector, this.root.querySelector, explain.querySelector

Decision branches: L81: !math; L87: scale < 1; L96: !math; L102: scale < 1; L109: explain; L112: math && math.offsetWidth > 338

Execution order (direct statements; nested callbacks stay within their owner):

L78-90 ForOfStatement: for (const n of this.root.querySelectorAll<HTMLElement>('.quiz-formula')) { n.style.fontSize = '18px';

L93-105 ForOfStatement: for (const b of this.root.querySelectorAll<HTMLButtonElement>('.quiz-options button')) { b.style.fontSize = '18px';

L108-108 FirstStatement: const explain = this.root.querySelector<HTMLElement>('.quiz-explanation');

L109-115 IfStatement: if (explain) { explain.style.fontSize = '12px';

MathUI.resize [L118-128]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: resize()

Direct writes: this.root.style.left, this.root.style.top, this.root.style.transform

Calls / delegates: document.querySelector, stage.getBoundingClientRect, canvas.getBoundingClientRect, this.fit

Decision branches: L121: !stage || !canvas

MathUI.pause [L130-132]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause(p: boolean)

Direct writes: this.root.hidden

MathUI.dispose [L134-136]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.root.remove

src/games/math-quiz/Rules.ts

ACTIVE SOURCE / tool or application | 8 lines | SHA-256 c9244ae5988e78eb...

Arbitrates first answers, correct/wrong point assignment, question progression and first-to-ten.

Imports: ../../shared/rules/FirstResponse, ./questions/Question

State, resources and initial values

Line	Field / declaration
2	scores:[number,number]=[0,0];
2	question!:MathQuestion;
2	phase:'show'|'active'|'between'|'done'='show';
2	time=0;
2	visibleAt=0;
2	round=0;
2	winner:1|2|null=null;
2	message='CHOOSE ONE';
2	private until=0;
2	private queue=new FirstResponse<boolean>();

QuizMatch [L2-7]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

QuizMatch.constructor [L3-3]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private next:()=>MathQuestion)

```
constructor(private next:()=>MathQuestion){this.question=next();}
```

QuizMatch.present [L4-4]

Mark the already-rendered question/object legally visible for input/AI.

Signature: present()

```
present(){if(this.phase==='show'){this.phase='active';this.visibleAt=this.time;}}
```

QuizMatch.answer [L5-5]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: answer(p:0|1,index:number)

```
answer(p:0|1,index:number){if(this.phase==='active'&&Number.isInteger(index)&&index>=0&&index<3)this.queue.add(p,index===this.question.correctIndex,this.time);}
```

QuizMatch.update [L6-6]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt:number)

Direct writes: this.time, this.message, this.winner, this.phase, this.until, this.question

Calls / delegates: this.queue.resolve, this.next

Decision branches: L6: this.phase==='done'; L6: result; L6: result.tie; L6: this.scores[p]===10; L6: this.phase==='between'&&this.time>=this.until

src/games/math-quiz/config.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 f9a8dcd3bcdfafd7...

Eager lightweight metadata for Math Quiz: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L2-2

```
export default {id:'math-quiz',name:'Math Quiz',description:'Quick minds. Friendly competition.',instructions:'Both players see the same question and three answers. First correct answer earns a point; a wrong answer gives your rival a point. Same-frame answers replay. First to ten. Orange: 1/2/3. Blue: 7/8/9. University answers use a validated question bank, not a symbolic AI.',color:'#dce7f2',accent:'#a4bad3',preview,order:12,tag:'THINK FAST',variants:[{id:'basic',name:'Basic Arithmetic',description:'Quick sums and number sense.'},{id:'university',name:'University Challenge',description:'100 clever mental-math questions.'}],supportedModes:['local-two-player','single-player'],tournament:true} satisfies GameMetadata;
```

src/games/math-quiz/index.ts

ACTIVE SOURCE / tool or application | 2 lines | SHA-256 83cdbad2cc3a7109...

Side-effect-free lazy factory for Math Quiz; implementation initializes only after host selection.

Imports: ./MathGame

Top-level block L1-1

```
export default ()=>new MathGame();
```

src/games/math-quiz/questions

src/games/math-quiz/questions/Bank.ts

ACTIVE SOURCE / tool or application | 49 lines | SHA-256 fb73ae6aadc788e0...

Combines/deduplicates static and procedural questions and creates bounded topic-interleaved decks favoring unseen IDs.

Imports: ./Question, ./Question, ./ProceduralUniversity, ./History, ./university.json

Module constants and data

L7-7 university = data as MathQuestion[]

L10-10 rawCombined = [...university, ...getAllProceduralQuestions()]

L13-13 uniqueMap = new Map<string, MathQuestion>()

L18-18 allUniversityQuestions = Array.from(uniqueMap.values())

universityDeck [L25-48]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function universityDeck(random = Math.random): MathQuestion[]

Calls / delegates: shuffled, Set, allUniversityQuestions.map, topics.map, allUniversityQuestions.filter, topicPool.filter, globalQuestionHistory.has, groups.some, g.pop, deck.push, shuffleAnswers

Return at L34: shuffled(candidates, random)

Return at L47: deck

Decision branches: L41: q

Top-level block L14-16

```
for (const q of rawCombined) {
  uniqueMap.set(q.id, q);
}
```

src/games/math-quiz/questions/Basic.ts

ACTIVE SOURCE / tool or application | 195 lines | SHA-256 ba58fc968b084708...

Generates arithmetic categories using exact integer/rational answers and misconception distractors.

Imports: ./Question

basicQuestion [L3-194]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function basicQuestion(serial: number, random = Math.random): MathQuestion

Calls / delegates: random, shuffleAnswers, fracLatex, Set, optSet.has, optSet.add

Return at L32: shuffleAnswers(q, random)

Return at L57: shuffleAnswers(q, random)

Return at L83: shuffleAnswers(q, random)

Return at L108: shuffleAnswers(q, random)

Return at L137: shuffleAnswers(q, random)

Return at L167: shuffleAnswers(q, random)

Return at L193: shuffleAnswers(q, random)

Decision branches: L9: cat === 0; L35: cat === 1; L60: cat === 2; L86: cat === 3; L111: cat === 4; L140: cat === 5

Execution order (direct statements; nested callbacks stay within their owner):

L7-7 FirstStatement: const cat = Math.floor(random() * 7);

L9-33 IfStatement: if (cat === 0) { // Addition: 2-digit sums

L35-58 IfStatement: if (cat === 1) { // Subtraction

L60-84 IfStatement: if (cat === 2) { // Multiplication

L86-109 IfStatement: if (cat === 3) { // Division: exact integer

L111-138 IfStatement: if (cat === 4) { // Fractions: Addition with common denominator

L140-168 IfStatement: if (cat === 5) { // Percentages: 10%, 20%, 25%, 50%, 75%

L171-171 FirstStatement: const a = 2 + Math.floor(random() * 8);

L172-172 FirstStatement: const b = 2 + Math.floor(random() * 6);

L173-173 FirstStatement: const c = 2 + Math.floor(random() * 6);

L174-174 FirstStatement: const correct = a + b * c;

L176-176 FirstStatement: const dPrecedence = (a + b) * c;

L177-177 FirstStatement: const dOff = correct + 1;

L178-182 FirstStatement: const options: [string, string, string] = [String(correct),

L183-192 FirstStatement: const q: MathQuestion = { id: `basic-prec-${a}-${b}-${c}`,

L193-193 ReturnStatement: return shuffleAnswers(q, random);

src/games/math-quiz/questions/History.ts

ACTIVE SOURCE / tool or application | 77 lines | SHA-256 d7a4713b73254cbf...

Persists a bounded insertion-ordered recent-question Set with FIFO eviction and storage fallback.

State, resources and initial values

Line	Field / declaration
5	private memory = new Set<string>();

Module constants and data

L1-1 STORAGE_KEY = 'pocket-rivals.math-history.v1'

L2-2 MAX_HISTORY = 150

L76-76 globalQuestionHistory = new QuestionHistory()

QuestionHistory [L4-74]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

QuestionHistory.constructor [L7-9]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor()

Calls / delegates: this.load

QuestionHistory.storage [L11-18]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: private get storage(): Storage | null

Return at L13: window.localStorage

Decision branches: L13: typeof window !== 'undefined' && window.localStorage

QuestionHistory.load [L20-33]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private load(): void

Direct writes: this.memory

Calls / delegates: store.getItem, Set, arr.slice

Decision branches: L23: !store; L25: !raw; L27: Array.isArray(arr)

QuestionHistory.save [L35-44]

Create/persist the component checkpoint or record described by the returned expression.

Signature: private save(): void

Calls / delegates: store.setItem

Decision branches: L38: !store

QuestionHistory.has [L46-48]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: has(id: string): boolean

Calls / delegates: this.memory.has

Return at L47: this.memory.has(id)

QuestionHistory.record [L50-58]

Evict the oldest ID when at150 entries, add the current ID and persist the bounded recent set.

Signature: record(id: string): void

Calls / delegates: this.memory.values().next, this.memory.values, this.memory.delete, this.memory.add, this.save

Decision branches: L51: this.memory.size >= MAX_HISTORY; L54: first !== undefined

QuestionHistory.clear [L60-69]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clear(): void

Calls / delegates: this.memory.clear, localStorage.removeItem

Decision branches: L63: typeof localStorage !== 'undefined'

QuestionHistory.size [L71-73]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: size(): number

Return at L72: this.memory.size

```
size(): number { return this.memory.size; }
```

src/games/math-quiz/questions/ProceduralUniversity.ts

ACTIVE SOURCE / tool or application | 791 lines | SHA-256 2dc2d70410e459ec...

Generates university-level question families with unique IDs, answers, distractors and estimated timing.

Imports: ./Question

buildUniqueOptions [L3-21]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function buildUniqueOptions(correct: string, distractors: string[], fallbackOffset = 1): [string, string, string]

Direct writes: step

Calls / delegates: chosen.includes, chosen.push, Number, isNaN

Return at L20: [chosen[0], chosen[1], chosen[2]]

Decision branches: L6: !chosen.includes(d); L8: chosen.length === 3; L15: !chosen.includes(candidate)

generateLimitsTemplates [L26-141]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateLimitsTemplates(): MathQuestion[]

Calls / delegates: simplifyFrac, fracLatex, buildUniqueOptions, list.push

Return at L140: list

Decision branches: L32: a === b; L75: a === b

Execution order (direct statements; nested callbacks stay within their owner):

L27-27 FirstStatement: const list: MathQuestion[] = [];

L30-50 ForStatement: for (let a = 2; a <= 7; a++) { for (let b = 2; b <= 7; b++) {

L53-70 ForStatement: for (let a = 2; a <= 6; a++) { const a2 = a * a;

L73-93 ForStatement: for (let a = 2; a <= 5; a++) { for (let b = 2; b <= 5; b++) {

L96-115 ForStatement: for (let a = 1; a <= 3; a++) { for (let b = 1; b <= 3; b++) {

L118-127 ExpressionStatement: list.push({ id: 'limits-hard-taylor-1',

L129-138 ExpressionStatement: list.push({ id: 'limits-hard-taylor-2',

L140-140 ReturnStatement: return list;

generateDerivativesTemplates [L146-255]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateDerivativesTemplates(): MathQuestion[]

Calls / delegates: buildUniqueOptions, list.push, fracLatex

Return at L254: list

Execution order (direct statements; nested callbacks stay within their owner):

L147-147 FirstStatement: const list: MathQuestion[] = [];

L150-163 ForStatement: for (let a = 2; a <= 7; a++) { const correct = String(a);

L166-177 ForStatement: for (let a = 2; a <= 6; a++) { list.push({

L180-193 ForStatement: for (let a = 2; a <= 6; a++) { const correct = String(a);

L196-205 ExpressionStatement: list.push({ id: 'deriv-ln-sin-pi4',

L208-217 ExpressionStatement: list.push({ id: 'deriv-ln-cos-pi4',

L220-236 ForStatement: for (let a = 2; a <= 6; a++) { const a2 = a * a;

L239-252 ForStatement: for (let a = 2; a <= 6; a++) { const correct = String(a);

L254-254 ReturnStatement: return list;

generateIntegralsTemplates [L260-356]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateIntegralsTemplates(): MathQuestion[]

Calls / delegates: fracLatex, buildUniqueOptions, list.push

Return at L355: list

Execution order (direct statements; nested callbacks stay within their owner):

L261-261 FirstStatement: const list: MathQuestion[] = [];

L264-278 ForStatement: for (let k = 2; k <= 6; k++) { const d1 = String(k);

L281-290 ExpressionStatement: list.push({ id: 'int-odd-trig-pi',

L293-304 ForStatement: for (let k = 1; k <= 4; k++) { list.push({

L307-319 ForStatement: for (let k = 2; k <= 6; k++) { const options = buildUniqueOptions('1', [String(k), fracLatex(1, k), String(k + 1)]);

L322-335 ForStatement: for (let k = 2; k <= 6; k++) { const correct = String(k - 1);

L338-353 ForStatement: for (let k = 2; k <= 5; k++) { const correct = fracLatex(1, k + 1);

L355-355 ReturnStatement: return list;

generateTrigTemplates [L361-425]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateTrigTemplates(): MathQuestion[]

Calls / delegates: list.push

Return at L424: list

Execution order (direct statements; nested callbacks stay within their owner):

L362-362 FirstStatement: const list: MathQuestion[] = [];

L365-374 ExpressionStatement: list.push({ id: 'trig-cos2-sin2-pi8',

L377-386 ExpressionStatement: list.push({ id: 'trig-double-sin-pi12',

L389-398 ExpressionStatement: list.push({ id: 'trig-double-sin-pi8',

L401-410 ExpressionStatement: list.push({ id: 'trig-cos2-sin2-pi6',

L413-422 ExpressionStatement: list.push({ id: 'trig-identity-sum-pi4',

L424-424 ReturnStatement: return list;

generateAlgebraTemplates [L430-503]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateAlgebraTemplates(): MathQuestion[]

Calls / delegates: buildUniqueOptions, list.push, fracLatex

Return at L502: list

Execution order (direct statements; nested callbacks stay within their owner):

L431-431 FirstStatement: const list: MathQuestion[] = [];

L434-448 ForStatement: for (let k = 3; k <= 7; k++) { const k2 = k * k;

L451-466 ForStatement: for (let k = 2; k <= 6; k++) { const correct = fracLatex(1, k - 1);

L469-484 ForStatement: for (let k = 2; k <= 5; k++) { const correct = fracLatex(k, k - 1);

L487-500 ForStatement: for (let k = 2; k <= 6; k++) { const correct = String(k + 1);

L502-502 ReturnStatement: return list;

generateMatrixTemplates [L508-592]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateMatrixTemplates(): MathQuestion[]

Calls / delegates: buildUniqueOptions, list.push

Return at L591: list

Execution order (direct statements; nested callbacks stay within their owner):

L509-509 FirstStatement: const list: MathQuestion[] = [];

L512-530 ForStatement: for (let c = 2; c <= 3; c++) { for (let detA = 2; detA <= 5; detA++) {

L533-544 ForStatement: for (let k = 2; k <= 6; k++) { list.push({

L547-565 ForStatement: for (let a = 2; a <= 5; a++) { for (let d = 2; d <= 5; d++) {

L568-573 FirstStatement: const cases = [{ a: 3, b: 1, c: 4, d: 2, exp: 2 },

L574-589 ForOfStatement: for (const item of cases) { const correct = String(item.exp);

L591-591 ReturnStatement: return list;

generateComplexTemplates [L597-666]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateComplexTemplates(): MathQuestion[]

Calls / delegates: list.push, buildUniqueOptions

Return at L665: list

Execution order (direct statements; nested callbacks stay within their owner):

L598-598 FirstStatement: const list: MathQuestion[] = [];

L601-606 FirstStatement: const rCases = [{ r: 0, ans: '1', d1: '-1', d2: 'i' },

L607-621 ForStatement: for (let k = 2; k <= 5; k++) { for (const rc of rCases) {

L624-639 ForStatement: for (let a = 2; a <= 6; a++) { const correct = String(2 * a * a);

L642-651 ExpressionStatement: list.push({ id: 'cpx-one-plus-i-sq',

L654-663 ExpressionStatement: list.push({ id: 'cpx-euler-pi',

L665-665 ReturnStatement: return list;

generateProbabilityTemplates [L671-774]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function generateProbabilityTemplates(): MathQuestion[]

Calls / delegates: buildUniqueOptions, list.push, fracLatex

Return at L773: list

Execution order (direct statements; nested callbacks stay within their owner):

L672-672 FirstStatement: const list: MathQuestion[] = [];

L675-691 ForStatement: for (let n = 4; n <= 10; n++) { const correct = String((n * (n - 1)) / 2);

L694-709 ForStatement: for (let n = 4; n <= 8; n++) { const correct = String(n * (n - 1));

L712-729 ForStatement: for (let n = 3; n <= 6; n++) { const den = Math.pow(2, n);

L732-748 ForStatement: for (let n = 3; n <= 5; n++) { const den = Math.pow(2, n);

L751-760 ExpressionStatement: list.push({ id: 'prob-dice-sum-7',

L762-771 ExpressionStatement: list.push({ id: 'prob-dice-sum-2',

L773-773 ReturnStatement: return list;

getAllProceduralQuestions [L779-790]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function getAllProceduralQuestions(): MathQuestion[]

Calls / delegates: generateLimitsTemplates, generateDerivativesTemplates, generateIntegralsTemplates, generateTrigTemplates, generateAlgebraTemplates, generateMatrixTemplates, generateComplexTemplates, generateProbabilityTemplates

Return at L780: [...generateLimitsTemplates(), ...generateDerivativesTemplates(), ...generateIntegralsTemplates(), ...generateTrigTemplates(), ...generateAlgebraTemplates(), ...generateMatrixTemplates(), ...generateComplexTemplates(), ...generateProbabilityTemplates(),]

src/games/math-quiz/questions/Question.ts

ACTIVE SOURCE / tool or application | 53 lines | SHA-256 f3d0a2281cb96c77...

Defines the math question schema, exact fraction helpers and correct-index-preserving answer shuffle.

MathQuestion [L1-10]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface MathQuestion { id: string; topic: string; difficulty: 'easy' | 'medium' | 'hard'; questionLatex: string; optionsLatex: [string, string, string]; correctIndex: 0 | 1 | 2; explanationLatex: string; estimatedSeconds: number; }
```

gcd [L12-20]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function gcd(a: number, b: number): number

Direct writes: y, x

Return at L19: x || 1

simplifyFrac [L22-27]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function simplifyFrac(num: number, den: number): [number, number]

Calls / delegates: gcd

Return at L23: [num, 1]

Return at L26: [(Math.abs(num) / d) * sign, Math.abs(den) / d]

Decision branches: L23: den === 0

fracLatex [L29-34]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function fracLatex(num: number, den: number): string

Calls / delegates: simplifyFrac

Return at L31: String(n)

Return at L32: String.raw`-\frac{${Math.abs(n)}}{${d}}`

Return at L33: String.raw`\frac{${n}}{${d}}`

Decision branches: L31: d === 1; L32: n < 0

shuffled [L36-43]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function shuffled<T>(items: readonly T[], random = Math.random): T[]

Direct writes: [a[i], a[j]]

Calls / delegates: random

Return at L42: a

shuffleAnswers [L45-52]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function shuffleAnswers(q: MathQuestion, random = Math.random): MathQuestion

Calls / delegates: shuffled, order.map, order.indexOf

Return at L47: { ...q, optionsLatex: order.map(i => q.optionsLatex[i]) as MathQuestion['optionsLatex'], correctIndex: order.indexOf(q.correctIndex) as 0 | 1 | 2, }

src/games/math-quiz

src/games/math-quiz/quiz.css

ACTIVE SOURCE / tool or application | 146 lines | SHA-256 4a153ea4dc53a7fa...

Presentation selectors and responsive layout rules; detailed selector/property blocks appear below.

Lines	Selector or rule	Declarations
1-8	.quiz-overlay	position: absolute; width: 420px; height: 760px; transform-origin: 0 0; pointer-events: none; color: #242b32;
8-25	.quiz-panel	position: absolute; left: 24px; top: 114px; width: 372px; height: 536px; border: 4px solid #242b32; border-radius: 20px; background: #dce7f2; display: flex; flex-direction: column; padding: 10px 14px; gap: 6px; box-sizing: border-box; justify-content: space-between; overflow: hidden;
25-35	.quiz-side	display: flex; flex-direction: column; align-items: center; justify-content: space-between; flex: 1; min-height: 0; padding: 4px 0; box-sizing: border-box;
35-38	.quiz-side.blue.local	transform: rotate(180deg);
38-48	.quiz-topic	font: 800 11px 'Nunito Sans', sans-serif; text-transform: uppercase; letter-spacing: 1.2px; color: #242b32; opacity: 0.72; line-height: 1; margin-bottom: 2px; flex-shrink: 0;
48-61	.quiz-formula	width: 100%; flex: 1; min-height: 0; display: flex; align-items: center; justify-content: center; font-size: 18px; text-align: center; padding: 2px 0; box-sizing: border-box; overflow: hidden;
61-66	.quiz-formula .katex-display	margin: 0; padding: 0; max-width: 100%;
66-72	.quiz-options	display: flex; gap: 8px; width: 100%; flex-shrink: 0;
72-90	.quiz-options button	pointer-events: auto; flex: 1; min-width: 0; min-height: 56px; height: 56px; padding: 4px 6px; border: 2px solid #242b32; border-radius: 14px; background: #f49b58; font-size: 18px; touch-action: manipulation; display: flex; align-items: center; justify-content: center; box-sizing: border-box; overflow: hidden;
90-93	.quiz-side.blue button	background: #58addb;
93-99	.quiz-options button .katex-display	margin: 0; padding: 0; display: inline-block; max-width: 100%;
99-104	.quiz-options button .katex	font-size: 1em; line-height: 1.1; text-align: center;
104-108	.quiz-options button:disabled	opacity: 0.6; pointer-events: none;
108-112	.quiz-options button.correct	background: #b8d998 !important; opacity: 1;
112-127	.quiz-middle	font: 800 12px 'Nunito Sans', sans-serif; letter-spacing: 0.8px; text-align: center; border-top: 2px dashed rgba(36, 43, 50, 0.35); border-bottom: 2px dashed rgba(36, 43, 50, 0.35); padding: 6px 4px; min-height: 34px; box-sizing: border-box; display: flex; flex-direction: column; align-items: center; justify-content: center; flex-shrink: 0;
127-134	.quiz-explanation	font-size: 12px; margin-top: 4px; max-width: 100%; line-height: 1.1; overflow: hidden;
134-139	.quiz-explanation .katex-display	margin: 0; padding: 0; max-width: 100%;
139-142	.quiz-explanation:empty	display: none;

Lines	Selector or rule	Declarations
142-145	.quiz-overlay[hidden]	display: none;

src/games/mini-golf

src/games/mini-golf/GolfAI.ts

RETAINED LEGACY / inactive factory | 133 lines | SHA-256 545a562bbe550949...

Retained legacy visibility-route and bounded candidate-shot simulation planner; not active in the current golf card.

Imports: ../../shared/contracts/AIController, ./courses, ./GolfPhysics

Point [L4-7]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
interface Point { x: number; y: number; }
```

GolfShot [L8-13]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface GolfShot { angle: number; power: number; milliseconds: number; candidates: number; }
```

clear [L14-28]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function clear(a: Point, b: Point, c: Course)

Calls / delegates: c.obstacles.some

Decision branches: L19: c.obstacles.some((o) => x > o.x - 9 && x < o.x + o.w + 9 && y > o.y - 9 && y < o.y + o.h + 9,)

route [L29-63]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function route(c: Course)

Direct writes: at, distances[i]

Calls / delegates: points.push, points.map, Set, visited.has, visited.add, points.forEach, clear

Return at L52: { points, cost: (p: Point) => Math.min(...points.map((q, i) => clear(p, q, c) ? Math.hypot(p.x - q.x, p.y - q.y) + distances[i] : Infinity,),), }

Decision branches: L34: x > 8 && x < 332 && y > 8 && y < 392; L40: !visited.has(i) && (at === -1 || distances[i] < distances[at]); L42: at < 0; L45: clear(points[at], p, c)

Execution order (direct statements; nested callbacks stay within their owner):

L30-30 FirstStatement: const points: Point[] = [c.hole];

L31-34 ForOfStatement: for (const o of c.obstacles) for (const x of [o.x - 12, o.x + o.w + 12])

L35-36 FirstStatement: const distances = points.map((_, i) => (i ? Infinity : 0)), visited = new Set<number>();

L37-51 ForStatement: for (let k = 0; k < points.length; k++) { let at = -1;

L52-62 ReturnStatement: return { points,

planGolfShot [L64-132]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function planGolfShot(c: Course, ball: GolfBall, difficulty: Difficulty, random = Math.random,): GolfShot

Direct writes: value, best

Calls / delegates: performance.now, route, candidates.push, simulate, routing.cost, finalists.push, finalists.sort((a, b) => a.cost - b.cost).slice, finalists.sort, random

Return at L123: { angle: best.angle + (random() - 0.5) * cfg.error, power: Math.max(20, Math.min(MAX_POWER, best.power * (1 + (random() - 0.5) * cfg.error)),), milliseconds: performance.now() - start, candidates: n, }

Decision branches: L95: n > 8 && performance.now() - start > cfg.ms * 0.75; L101: cost < value; L105: difficulty === "impossible" && !next.holed && Number.isFinite(cost); L107: next.holed; L110: difficulty === "impossible" && value > 0; L112: performance.now() - start > cfg.ms; L118: end.holed && f.cost < value + 60

Execution order (direct statements; nested callbacks stay within their owner):

L70-77 FirstStatement: const start = performance.now(), cfg = {

L78-83 ForOfStatement: for (const p of routing.points) { const angle = Math.atan2(p.y - ball.y, p.x - ball.x),
L84-89 ForStatement: for (let i = 0; i < cfg.angles; i++) for (let j = 1; j <= cfg.powers; j++)
L90-92 FirstStatement: let best = candidates[0], value = Infinity,
L93-93 FirstStatement: const finalists: { shot: typeof best; ball: GolfBall; cost: number }[] = [];
L94-108 ForOfStatement: for (const shot of candidates) { if (n > 8 && performance.now() - start > cfg.ms * 0.75) break;
L110-122 IfStatement: if (difficulty === "impossible" && value > 0) for (const f of finalists.sort((a, b) => a.cost - b.cost).slice(0, 6)) {
L123-131 ReturnStatement: return { angle: best.angle + (random() - 0.5) * cfg.error,

src/games/mini-golf/GolfMatch.ts

RETAINED LEGACY / inactive factory | 154 lines | SHA-256 ab43f439ff703934...

Retained legacy duel: alternating stopped shots, per-hole stroke cap, cumulative totals and stopped-state validation.

Imports: ../../shared/contracts/GameResult, ./courses, ./GolfPhysics

State, resources and initial values

Line	Field / declaration
5	hole = 0;
6	turn: Player = 1;
7	balls: [GolfBall, GolfBall];
8	strokes: [number, number] = [0, 0];
9	totals: [number, number] = [0, 0];
10	history: [number, number][] = [];
11	winner: Winner | null = null;
12	shotActive = false;

GolfMatch [L4-153]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfMatch.constructor [L13-15]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly count = 5)

Direct writes: this.balls

Calls / delegates: newBall

GolfMatch.course [L16-18]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get course()

Return at L17: courses[this.hole]

```
get course() { return courses[this.hole]; }
```

GolfMatch.ball [L19-21]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get ball()

Return at L20: this.balls[this.turn - 1]

```
get ball() { return this.balls[this.turn - 1]; }
```

GolfMatch.takeShot [L22-28]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: takeShot(angle: number, power: number)

Direct writes: this.shotActive

Calls / delegates: shoot

Decision branches: L23: this.winner || this.shotActive || !shoot(this.ball, angle, power)

GolfMatch.finishShot [L29-58]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: finishShot()

Direct writes: this.shotActive, this.strokes[i], this.ball.holed, this.totals[0], this.totals[1], this.winner, this.balls, this.strokes, this.turn

Calls / delegates: this.balls.every, this.history.push, newBall

Decision branches: L30: !this.shotActive; L33: !this.ball.holed && this.strokes[i] >= 12; L37: this.balls.every((b) => b.holed); L41: this.hole + 1 >= this.count; L56: !this.balls[next - 1].holed

GolfMatch.save [L59-70]

Create/persist the component checkpoint or record described by the returned expression.

Signature: save()

Return at L60: { version: 1, count: this.count, hole: this.hole, turn: this.turn, balls: this.balls, strokes: this.strokes, totals: this.totals, history: this.history, }

```
save() { return { version: 1, count: this.count, hole: this.hole, turn: this.turn, balls: this.balls, strokes: this.strokes, totals: this.totals, history: this.history, }; }
```

GolfMatch.restore [L71-152]

Validate/replay saved state and fall back to a fresh legal instance when rejected.

Signature: static restore(data: unknown, count: number)

Calls / delegates: GolfMatch, [1, 2].includes, [d.strokes, d.totals].every, a.every, d.balls.every, d.history.every, d.totals.some, d.history.reduce, d.strokes.some, d.balls.some, course.obstacles.some, structuredClone

Return at L73: fresh

Return at L87: fresh

Return at L96: fresh

Return at L111: fresh

Return at L121: fresh

Return at L127: fresh

Return at L142: fresh

Return at L151: fresh

Decision branches: L73: !data || typeof data !== "object"; L75: d.version !== 1 || d.count !== count || !Number.isInteger(d.hole) || d.hole < 0 || d.hole >= count || ![1, 2].includes(d.turn) || !Array.isArray(d.balls) || d.balls.length !== 2 || !Array.isArray(d.history) || d.history. ... [continued in source reference]; L88: ![d.strokes, d.totals].every((a) => Array.isArray(a) && a.length === 2 && a.every((n) => Number.isInteger(n) && n >= 0 && n <= 65),); L97: !d.balls.every((b) => b && Number.isFinite(b.x + b.y) && b.x >= 7 && b.x <= 333 && b.y >= 7 && b.y <= 393 && typeof b.holed === "boolean" && b.vx === 0 && b.vy === 0,); L112: !d.history.every((a) => Array.isArray(a) && a.length === 2 && a.every((n) => Number.isInteger(n) && n >= 1 && n <= 13),) || d.totals.some((n, i) => n !== d.history.reduce((sum, h) => sum + h[i], 0)); L122: d.balls.every((b) => b.holed) || d.balls[d.turn - 1].holed || d.strokes.some((n) => n > 13); L129: d.balls.some((b) => !b.holed && course.obstacles.some((o) => b.x > o.x - 6.9 && b.x < o.x + o.w + 6.9 && b.y > o.y - 6.9 && b.y < o.y + o.h + 6.9,),)

Execution order (direct statements; nested callbacks stay within their owner):

L72-72 FirstStatement: const fresh = new GolfMatch(count);

L73-73 IfStatement: if (!data || typeof data !== "object") return fresh;

L74-74 FirstStatement: const d = data as ReturnType<GolfMatch["save"]>;

L75-87 IfStatement: if (d.version !== 1 ||

L88-96 IfStatement: if (![d.strokes, d.totals].every(

L97-111 IfStatement: if (!d.balls.every(

L112-121 IfStatement: if (!d.history.every(

L122-127 IfStatement: if (d.balls.every((b) => b.holed) ||

L128-128 FirstStatement: const course = courses[d.hole];

L129-142 IfStatement: if (d.balls.some(

L143-150 ExpressionStatement: Object.assign(fresh, { hole: d.hole,

L151-151 ReturnStatement: return fresh;

src/games/mini-golf/GolfPhysics.ts

RETAINED LEGACY / inactive factory | 151 lines | SHA-256 e421c252a515e095...

Retained legacy travel-substepped golf simulation, friction, capture and copied-ball trajectory forecast.

Imports: ./courses

Module constants and data

L9-9 BALL_RADIUS = 7

L10-10 FRICTION = 65

L11-11 MAX_POWER = 600

GolfBall [L2-8]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface GolfBall { x: number; y: number; vx: number; vy: number; holed: boolean; }
```

newBall [L12-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function newBall(course: Course): GolfBall

Return at L13: { ...course.start, vx: 0, vy: 0, holed: false }

```
export function newBall(course: Course): GolfBall { return { ...course.start, vx: 0, vy: 0, holed: false }; }
```

moving [L15-17]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function moving(ball: GolfBall)

Return at L16: Math.hypot(ball.vx, ball.vy) > 0

shoot [L18-30]

Validate and launch a normal shot through the authoritative physics API.

Signature: export function shoot(ball: GolfBall, angle: number, power: number)

Direct writes: ball.vx, ball.vy

Calls / delegates: moving

Decision branches: L19: ball.holed || moving(ball) || !Number.isFinite(angle + power) || power < 20

golfStep [L31-139]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function golfStep(b: GolfBall, c: Course, dt: number, impact: () => void = () => {},)

Direct writes: b.x, b.y, b.vx, b.vy, b.holed

Calls / delegates: moving, impact, bounce, [{ d: b.x - o.x, x: o.x - BALL_RADIUS - 0.01, y: b.y, nx: -1, ny: 0, }, { d: o.x + o.w - b.x, x: o.x + o.w + BALL_RADIUS + 0.01, y: b.y, nx: 1, ny: 0, }, { d: b.y - o.y, x: b.x, y: o.y - BALL_RADIUS - 0.01, nx: 0, ny: -1, }, { d: o.y + o.h - b.y, x: b.x, y: o.y + o.h + BALL_RADIUS + 0.01, nx: 0, ny: 1, },].sort

Decision branches: L37: b.holed || !moving(b); L49: dot < 0; L56: b.x < BALL_RADIUS; L60: b.x > W - BALL_RADIUS; L64: b.y < BALL_RADIUS; L68: b.y > H - BALL_RADIUS; L78: d >= BALL_RADIUS; L79: d > 0.00001; L124: Math.hypot(b.x - c.hole.x, b.y - c.hole.y) < 11 && speed < 150; L132: next < 8

Execution order (direct statements; nested callbacks stay within their owner):

L37-37 IfStatement: if (b.holed || !moving(b)) return;

L39-43 FirstStatement: const count = Math.max(1,

L44-138 ForStatement: for (let k = 0; k < count; k++) { b.x += b.vx * sub;

golfStep.bounce [L47-54]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: bounce = (nx: number, ny: number) =>

Direct writes: b.vx, b.vy

Calls / delegates: impact

Decision branches: L49: dot < 0

simulate [L140-150]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function simulate(course: Course, ball: GolfBall, angle: number, power: number,)

Calls / delegates: shoot, moving, golfStep

Return at L149: b

src/games/mini-golf/MiniGolfGame.ts

RETAINED LEGACY / inactive factory | 258 lines | SHA-256 3c5a904574c7a6bb...

Golf lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ../../shared/physics/FixedStep, ../../shared/utilities/WorkerTask, ./GolfMatch, ./GolfPhysics, ./GolfAI, ./courses, ./GolfPhysics

State, resources and initial values

Line	Field / declaration
13	private ctx!: GameContext;
14	private match!: GolfMatch;
15	private g!: Phaser.GameObjects.Graphics;
16	private heading!: Phaser.GameObjects.Text;
17	private caption!: Phaser.GameObjects.Text;
18	private note!: Phaser.GameObjects.Text;
19	private clock = new FixedStep();
20	private aim: { x: number; y: number } | null = null;
21	private paused = false;
22	private age = 0;
23	private pending = false;
24	private generation = 0;
25	private disposed = false;
26	private reported = false;
27	private ai?: WorkerTask< { course: Course; ball: GolfBall; difficulty: GameContext["options"]["difficulty"]; }, GolfShot >;

MiniGolfGame [L12-257]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

MiniGolfGame.initialize [L35-90]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.match, this.g, this.heading, this.caption, this.note, this.ai, this.aim

Calls / delegates: GolfMatch.restore, ctx.scene.add.graphics, label, ctx.setHint, WorkerTask, Worker, URL, ctx.input.drag, this.takeShot, this.save

Decision branches: L49: ctx.options.mode === "ai"; L59: this.paused || this.age < 0.4 || this.match.shotActive || this.match.winner || (this.ai && this.match.turn === 2); L68: Math.hypot(x - 40 - b.x, y - 200 - b.y) > 42

Execution order (direct statements; nested callbacks stay within their owner):

L36-36 ExpressionStatement: this.ctx = ctx;

L37-42 FirstStatement: const count = ctx.options.variant === "one"

L43-43 ExpressionStatement: this.match = GolfMatch.restore(ctx.restoredState, count);

L44-44 ExpressionStatement: this.g = ctx.scene.add.graphics();

L45-45 ExpressionStatement: this.heading = label(ctx.scene, 210, 144, "", 21);

L46-46 ExpressionStatement: this.caption = label(ctx.scene, 210, 173, "", 12);

L47-47 ExpressionStatement: this.note = label(ctx.scene, 210, 627, "", 12);

L48-48 ExpressionStatement: ctx.setHint("DRAG BACK FROM YOUR BALL · RELEASE TO PUTT");

L49-56 IfStatement: if (ctx.options.mode === "ai") this.ai = new WorkerTask(

L57-88 ExpressionStatement: ctx.input.drag((x, y) => {

L89-89 ExpressionStatement: this.save();

MiniGolfGame.takeShot [L91-96]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private takeShot(angle: number, power: number)

Direct writes: this.age

Calls / delegates: this.match.takeShot, this.ctx.audio.play

Decision branches: L92: this.match.takeShot(angle, power)

MiniGolfGame.save [L97-99]

Create/persist the component checkpoint or record described by the returned expression.

Signature: private save()

Calls / delegates: this.ctx.saveState, this.match.save

MiniGolfGame.update [L100-225]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.age, this.pending, y, this.reported

Calls / delegates: this.clock.advance, golfStep, this.ctx.audio.play, moving, m.finishShot, this.save, this.ai .run({ course: m.course, ball: { ...m.ball }, difficulty: this.ctx.options.difficulty, }) .then, this.ai .run, this.takeShot, g.clear, rounded, g.lineStyle, g.lineBetween, c.obstacles.forEach, circle, g.lineStyle(2, theme.ink).lineBetween, g.fillStyle(theme.white).fillTriangle, g.fillStyle, g.lineStyle(2, color, 0.7).strokeCircle, g.fillStyle(theme.white, 0.8).fillCircle, g.lineStyle(3, theme.white, 0.9).lineBetween, g.fillStyle(i <= power * 5 ? theme.orange : theme.white, 0.9,).fillRoundedRect, this.heading .setText(`HOLE ${m.hole + 1} / ${m.count} · PAR ${c.par}`) .setDepth, this.heading .setText, this.caption.setText(c.name.toUpperCase()).setDepth, this.caption.setText, c.name.toUpperCase, this.note .setText(this.aim ? "" : m.shotActive ? "NICE AND EASY…" : this.pending ? "BLUE IS PLANNING…" : `${m.turn === 1 ? "ORANGE" : "BLUE"} · STROKE ${m.strokes[m.turn - 1] + 1}`,) .setDepth, this.note .setText, this.ctx.setScore, this.ctx.finish

Decision branches: L101: this.paused; L105: m.shotActive; L107: !moving(m.ball); L108: m.ball.holed; L111: !m.winner; L115: this.ai && m.turn === 2 && !m.shotActive && !m.winner && !this.pending && this.age > 0.7; L132: this.disposed || generation !== this.generation; L134: !this.paused; L174: !b.holed; L175: !m.shotActive; L182: this.aim; L188: d > 0; L221: m.winner && this.age > 0.7 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L101-101 IfStatement: if (this.paused) return;

L102-102 ExpressionStatement: this.age += dt;

L103-103 FirstStatement: const m = this.match;

L104-114 ExpressionStatement: this.clock.advance(dt, (step) => { if (m.shotActive) {

L115-145 IfStatement: if (this.ai &&

L146-147 FirstStatement: const g = this.g, c = m.course;

L148-148 ExpressionStatement: g.clear();

L149-149 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, theme.green, 4);

L150-150 ExpressionStatement: rounded(g, 40, 200, 340, 400, 12, 0x93c57f, 4);

L151-151 ExpressionStatement: g.lineStyle(1, 0xffffff, 0.1);

L152-152 ForStatement: for (let y = 220; y < 600; y += 24) g.lineBetween(44, y, 376, y);

L153-156 ExpressionStatement: c.obstacles.forEach((o) => { rounded(g, 40 + o.x, 200 + o.y + 4, o.w, o.h, 3, 0x68976a, 0);

L157-157 ExpressionStatement: circle(g, 40 + c.hole.x, 200 + c.hole.y, 11, theme.ink, 0);

L158-163 ExpressionStatement: g.lineStyle(2, theme.ink).lineBetween(40 + c.hole.x,

L164-171 ExpressionStatement: g.fillStyle(theme.white).fillTriangle(40 + c.hole.x,

L172-173 FirstStatement: const b = m.ball, color = m.turn === 1 ? theme.orange : theme.blue;

L174-181 IfStatement: if (!b.holed) { if (!m.shotActive) {

L182-201 IfStatement: if (this.aim) { const dx = b.x - this.aim.x,

L202-204 ExpressionStatement: this.heading .setText(`HOLE ${m.hole + 1} / ${m.count} · PAR ${c.par}`)

L205-205 ExpressionStatement: this.caption.setText(c.name.toUpperCase()).setDepth(1);

L206-216 ExpressionStatement: this.note .setText(

L217-220 ExpressionStatement: this.ctx.setScore(m.totals[0] + (m.winner ? 0 : m.strokes[0]),

L221-224 IfStatement: if (m.winner && this.age > 0.7 && !this.reported) { this.reported = true;

MiniGolfGame.pause [L226-233]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused, this.aim, this.pending

Calls / delegates: this.clock.reset, this.ai?.cancel

MiniGolfGame.resume [L234-238]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused, this.age

Calls / delegates: this.clock.reset

MiniGolfGame.dispose [L239-247]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.disposed

Calls / delegates: this.ai?.cancel, this.g.destroy, this.heading.destroy, this.caption.destroy, this.note.destroy

MiniGolfGame.snapshot [L248-256]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.match.save

Return at L249: { ...this.match.save(), winner: this.match.winner, shotActive: this.match.shotActive, pending: this.pending, aim: this.aim, }

src/games/mini-golf/ai.worker.ts

RETAINED LEGACY / inactive factory | 4 lines | SHA-256 d099e50b92d40358...

Worker message entry for Golf. Restores/copies request state, runs the bounded local search and posts its result; cancellation is owned by WorkerTask/controller.

Imports: ./GolfAI

Top-level block L2-3

```
self.onmessage = (e) =>
  self.postMessage(planGolfShot(e.data.course, e.data.ball, e.data.difficulty));
```

src/games/mini-golf/config.ts

ACTIVE SOURCE / tool or application | 20 lines | SHA-256 87945a141d3e519b...

Eager lightweight metadata for Golf: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-19

```
export default {
  id: "mini-golf",
  name: "Ultimate Golf",
  description: "Four worlds. One little ball.",
  instructions:
    "Choose a course and challenge. Pull back anywhere on the course and release to putt. Avoid hazards and reach
the flag before running out of strokes. Focus enlarges the view.",
  color: "#b8d998",
  accent: "#93c57f",
  preview,
  order: 5,
  tag: "A LITTLE ADVENTURE",
  category: "duel",
  tournament: false,
  presentation: "custom",
  responsive: true,
  supportedModes: ["single-player", "local-two-player"],
} satisfies GameMetadata;
```

src/games/mini-golf/courses.ts

RETAINED LEGACY / inactive factory | 69 lines | SHA-256 e5b94e5dc2d515b4...

Retained legacy five small golf course geometries; not the active Ultimate course set.

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
  1 | export interface Rect {
  2 |  x: number;
  3 |  y: number;
  4 |  w: number;
  5 |  h: number;
  6 | }
  7 | export interface Course {
  8 |  id: string;
```

```
  9 |  name: string;
10 |  par: number;
11 |  start: { x: number; y: number };
12 |  hole: { x: number; y: number };
13 |  obstacles: Rect[];
14 | }
15 | export const COURSE_WIDTH = 340,
16 |  COURSE_HEIGHT = 400;
17 | export const courses: Course[] = [
18 |  {
19 |    id: "first-putt",
20 |    name: "The first putt",
21 |    par: 2,
22 |    start: { x: 80, y: 340 },
23 |    hole: { x: 260, y: 65 },
24 |    obstacles: [{ x: 115, y: 175, w: 110, h: 28 }],
25 |  },
26 |  {
27 |    id: "dogleg",
28 |    name: "Around the bend",
29 |    par: 3,
30 |    start: { x: 65, y: 345 },
31 |    hole: { x: 265, y: 65 },
32 |    obstacles: [{ x: 130, y: 130, w: 28, h: 270 }],
33 |  },
34 |  {
35 |    id: "zigzag",
36 |    name: "A little zigzag",
37 |    par: 4,
38 |    start: { x: 55, y: 345 },
39 |    hole: { x: 270, y: 45 },
40 |    obstacles: [
41 |      { x: 0, y: 235, w: 220, h: 24 },
42 |      { x: 120, y: 125, w: 220, h: 24 },
43 |    ],
44 |  },
45 |  {
46 |    id: "islands",
47 |    name: "Island hopping",
48 |    par: 3,
49 |    start: { x: 170, y: 350 },
50 |    hole: { x: 170, y: 45 },
51 |    obstacles: [
52 |      { x: 55, y: 200, w: 80, h: 55 },
53 |      { x: 205, y: 200, w: 80, h: 55 },
54 |      { x: 130, y: 105, w: 80, h: 35 },
55 |    ],
56 |  },
57 |  {
58 |    id: "last-lap",
59 |    name: "The last laugh",
60 |    par: 4,
61 |    start: { x: 50, y: 345 },
62 |    hole: { x: 280, y: 65 },
63 |    obstacles: [
64 |      { x: 95, y: 85, w: 28, h: 245 },
65 |      { x: 215, y: 165, w: 28, h: 235 },
66 |    ],
67 |  },
68 | ];
```

Module constants and data

L15-15 COURSE_WIDTH = 340

L16-16 COURSE_HEIGHT = 400

L17-68 courses = [{ id: "first-putt", name: "The first putt", par: 2, start: { x: 80, y: 340 }, hole: { x: 260, y: 65 }, obstacles: [{ x: 115, y: 175, w: 110, h: 28 }], }, { id: "dogleg", name: "Around the bend", par: 3, start: { x: 65, y: 345 }, hole: { x: 265, y: 65 }, obstacles: [{ x: 130, y: 130, w: 28, h: 270 }], }, { id: "zigzag", name: "A little zigzag", par: 4, start: { x: 55, y: 345 }, hole: { x: 270, y: 45 }, obstacles: [{ x: 0, y: 235, w: 220, h: 24 }, { x: 120, y: 125, w: 220, h: 24 },], }, { id: "islands", name: "Island hopping", par: 3, start: { x: 170, y: 350 }, hole: { x: 170, y: 45 }, obstacles: [{ x: 55, y: 200, w: 80, h: 55 }, { x: 205, y: 200, w: 80, h: 55 }, { x: 130, y: 105, w: 80, h: 35 },], }, { id: "last-lap", name: "The last laugh", par: 4, start: { x: 50, y: 345 }, hole: { x: 280, y: 65 }, obstacles: [{ x: 95, y: 85, w: 28, h: 245 }, { x: 215, y: 165, w: 28, h: 235 },], },]

Rect [L1-6]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Rect { x: number; y: number; w: number; h: number; }
```

Course [L7-14]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Course { id: string; name: string; par: number; start: { x: number; y: number }; hole: { x: number; y: number }; obstacles: Rect[]; }
```

src/games/mini-golf/index.ts

ACTIVE SOURCE / tool or application | 5 lines | SHA-256 041a13ec5bc7a97d...

Active golf entry: lazily imports ultimate/UltimateGolfGame. Parent-root legacy golf files are not imported here.

Imports: ../../shared/contracts/GameModule, ./ultimate/UltimateGolfGame

Top-level block L3-3

```
export default (options?: MatchOptions) => new UltimateGolfGame(options);
```

src/games/mini-golf/ultimate

src/games/mini-golf/ultimate/GolfCollision.ts

ACTIVE SOURCE / tool or application | 107 lines | SHA-256 7d174556903bf453...

Defines active golf radius/speed cap, overlap tests and separating relative-velocity contact responses.

Imports: ./courses

Module constants and data

L6-6 RADIUS = 7

L7-7 MAX_SPEED = 650

Ball [L2-5]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Ball extends Point { vx: number; vy: number; }
```

clamp [L8-9]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clamp = (v: number, a: number, b: number) =>

inside [L10-11]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: inside = (p: Point, r: Rect) =>

```
inside = (p: Point, r: Rect) => p.x >= r.x && p.x <= r.x + r.w && p.y >= r.y && p.y <= r.y + r.h
```

distance [L12-13]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: distance = (a: Point, b: Point) =>

cap [L14-20]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function cap(b: Ball)

Direct writes: b.vx, b.vy

Decision branches: L16: s > MAX_SPEED

overlaps [L21-28]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function overlaps(b: Point, r: Rect, radius = RADIUS)

Calls / delegates: clamp

Return at L22: (Math.hypot(b.x - clamp(b.x, r.x, r.x + r.w), b.y - clamp(b.y, r.y, r.y + r.h),) < radius)

reflect [L29-38]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function reflect(b: Ball, nx: number, ny: number, vx = 0, vy = 0, e = 1)

Direct writes: b.vx, b.vy

Calls / delegates: cap

Decision branches: L33: dot >= 0

hitRect [L39-65]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function hitRect(b: Ball, r: Rect, vx = 0, vy = 0)

Direct writes: nx, ny, cx, cy, b.x, b.y, b.vy

Calls / delegates: clamp, sides.indexOf, reflect, cap

Return at L64: hit

Decision branches: L45: d >= RADIUS; L46: d < 1e-9; L60: hit && vx && Math.abs(ny) < 1e-6

hitCircle [L66-74]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function hitCircle(b: Ball, c: Point, r: number, e = 1, vx = 0, vy = 0)

Direct writes: b.x, b.y

Calls / delegates: distance, reflect

Return at L73: reflect(b, nx, ny, vx, vy, e)

Decision branches: L68: d >= r + RADIUS

hitRotated [L75-106]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function hitRotated(b: Ball, c: Point, w: number, h: number, angle: number, spin = 0,)

Direct writes: b.x, b.y

Calls / delegates: hitRect, reflect

Return at L105: reflect(b, wx, wy, -ay * spin, ax * spin)

Decision branches: L95: local.x === before.x && local.y === before.y

src/games/mini-golf/ultimate/GolfDynamics.ts

ACTIVE SOURCE / tool or application | 291 lines | SHA-256 2beacbd3fa49f315...

Advances difficulty-scaled movers, lasers, tides, piston, doors/planks, UFO and safe recovery geometry.

Imports: ./courses, ./GolfRules, ./GolfRules, ./GolfCollision

State, resources and initial values

Line	Field / declaration
6	time = 0;
7	gate = 0;
8	plates = [false, false, false];
9	doors = [0, 0];
10	doorVelocity = [0, 0];
11	planks = Array(10).fill(0) as number[];
12	sink = 0;
13	portalCooldown = 0;
14	ufo = { active: false, done: false, warning: false, event: -1, next: 11.5, started: 0, cooldown: 0, x: -120, y: 990, direction: 1, hover: 0, carry: 0, section: 0, used: false, };

GolfDynamics [L5-290]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfDynamics.constructor [L30-35]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly course: Course, readonly difficulty: GolfDifficulty, private random: () => number = Math.random, readonly seed?: number,)

```
constructor( readonly course: Course, readonly difficulty: GolfDifficulty, private random: () => number = Math.random, readonly seed?: number, ) {}
```

GolfDynamics.eventRandom [L36-45]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private eventRandom(salt: number)

Direct writes: n

Calls / delegates: this.random

Return at L37: this.random()

Return at L44: ((n ^ (n >>> 15)) >>> 0) / 4294967296

Decision branches: L37: this.seed === undefined

GolfDynamics.mover [L46-62]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: mover(i: number)

Direct writes: offset

Return at L55: { ...m, x: m.from.x + (m.to.x - m.from.x) * t, y: m.from.y + (m.to.y - m.from.y) * t, vx: (((m.to.x - m.from.x) * dir) / m.seconds) * speed, vy: (((m.to.y - m.from.y) * dir) / m.seconds) * speed, }

Decision branches: L49: this.course.id === "shoreline"

GolfDynamics.laser [L63-65]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: laser(i: number)

Return at L64: (this.time + this.course.lasers[i].phase) % 3 >= 1.6

```
laser(i: number) { return (this.time + this.course.lasers[i].phase) % 3 >= 1.6; }
```

GolfDynamics.bridgeSafe [L66-68]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: bridgeSafe()

Return at L67: this.time % 8 < 3.5 || this.time % 8 >= 7.5

```
bridgeSafe() { return this.time % 8 < 3.5 || this.time % 8 >= 7.5; }
```

GolfDynamics.piston [L69-91]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: piston()

Direct writes: width, vx

Return at L86: { ...p, w: p.w + width, vx: vx * difficulties[this.difficulty].speed, }

Decision branches: L71: !p; L77: phase >= 1.2 && phase < 1.2 + extend; L80: phase >= 1.2 + extend && phase < 1.5 + extend; L82: phase >= 1.5 + extend

GolfDynamics.door [L92-97]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: door(i: number): Rect

Return at L93: { ...this.course.doors![i], y: this.course.doors![i].y + this.doors[i] * 100, }

```
door(i: number): Rect { return { ...this.course.doors![i], y: this.course.doors![i].y + this.doors[i] * 100, }; }
```

GolfDynamics.safe [L98-112]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: safe(p: Point)

Calls / delegates: inside, c.planks?.some, row.some, c.ground.some, c.bridges?.some, this.bridgeSafe, c.surfaces.some

Return at L101: !!c.planks?.some((row, b) => row.some((r, i) => inside(p, r) && this.planks[b * 5 + i] < [0.5, 0.2][b],),)

Return at L106: p.x >= 0 && p.x <= 1920 && p.y >= 70 && p.y <= 1080

Return at L107: (c.ground.some((r) => inside(p, r)) || !!c.bridges?.some((r) => this.bridgeSafe() && inside(p, r)) || c.surfaces.some((r) => r.kind === "ford" && inside(p, r)))

Decision branches: L100: c.river && inside(p, c.river); L106: !c.ground

GolfDynamics.update [L113-130]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, b: Ball, lastShot: Point = this.course.start)

Direct writes: this.time, this.portalCooldown, this.gate, this.planks[i], this.doors[i], this.doorVelocity[i]

Calls / delegates: this.planks.forEach, this.course.doors?.forEach, overlaps, this.updateUfo

Decision branches: L119: v > 0; L125: open; L126: !overlaps(b, r, 9); L129: this.course.id === "horizon"

GolfDynamics.updateUfo [L131-205]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private updateUfo(dt: number, b: Ball, lastShot: Point)

Direct writes: u.warning, u.cooldown, u.next, u.started, u.used, u.hover, u.done, u.active, u.direction, u.x, u.y, u.carry, b.x, b.y, b.vx, b.vy, section, target, u.section

Calls / delegates: this.eventRandom, this.dropSafe, distance, this.course.ground!.forEach, inside

Decision branches: L138: (!u.done || u.event >= 0) && !u.active && this.time >= next; L153: !u.active; L155: u.carry > 0; L160: u.carry <= 0; L162: section < 0; L163: section > 10; L164: section === 9; L172: this.dropSafe(p); L177: !target; L183: !u.used && u.cooldown === 0 && b.vx === 0 && b.vy === 0 && distance(b, u) < 80; L191: u.hover >= config.hover; L196: inside(b, r); L200: u.carry <= 0 && (u.x > 2040 || u.x < -120 || this.time - u.started >= config.duration)

Execution order (direct statements; nested callbacks stay within their owner):

L132-132 FirstStatement: const u = this.ufo;

L133-133 FirstStatement: const config = this.course.ufo!;

L134-134 FirstStatement: const next = u.event < 0 ? config.first : u.next;

L135-136 ExpressionStatement: u.warning = !u.active && this.time >= next - config.warning && this.time < next;

L137-137 ExpressionStatement: u.cooldown = Math.max(0, u.cooldown - dt);

L138-152 IfStatement: if ((!u.done || u.event >= 0) && !u.active && this.time >= next) { u.event++;

L153-153 IfStatement: if (!u.active) return;

L154-154 ExpressionStatement: u.x += u.direction * config.speed * dt;

L155-182 IfStatement: if (u.carry > 0) { u.carry -= dt;

L183-199 IfStatement: if (!u.used &&

L200-204 IfStatement: if (u.carry <= 0 &&

GolfDynamics.dropSafe [L206-221]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private dropSafe(p: Point)

Calls / delegates: this.safe, distance, c.fields.some, c.lasers.some, overlaps, c.circles.some, c.movers.some, this.mover, c.fans.some, c.portals?.some, c.surfaces.some, inside

Return at L208: (this.safe(p) && distance(p, c.hole) > 40 && !c.fields.some((f) => f.core && distance(p, f) < f.core + 30) && !c.lasers.some((r) => overlaps(p, r, 30)) && !c.circles.some((v) => distance(p, v) < v.r + 30) && !c.movers.some((_, i) => distance(p, this.mover(i)) < (this.mover(i).r ?? 0) + 30,) && !c.fans.some((f) => distance(p, f) < f.length / 2 + ... [continued in source reference]

GolfDynamics.recovery [L222-289]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: recovery(last: Point): Point

Direct writes: nx, ny, p.x, p.y

Calls / delegates: c.movers.forEach, clamp, distance, inside, c.fans.forEach, c.lasers.some, overlaps, this.safe

Return at L280: { ...c.start }

Return at L283: { x: 590, y: 210 }

Return at L284: { x: 590, y: 960 }

Return at L286: { ...c.start }

Return at L288: p

Decision branches: L235: m.r && distance(p, q) < m.r + 9; L238: (p.x - q.x) * nx + (p.y - q.y) * ny < 0; L244: m.w && inside(p, { x: Math.min(m.from.x, m.to.x) - 9, y: m.from.y - 9, w: Math.abs(dx) + m.w + 18, h: (m.h ?? 0) + 18, }); L261: distance(p, f) < sweep; L262: c.id === "horizon"; L266: c.piston && inside(p, { ...c.piston, w: c.piston.w + c.piston.extension + 9, x: c.piston.x - 9, y: c.piston.y - 9, h: c.piston.h + 18, }); L280: c.lasers.some((r) => overlaps(p, r)); L281: !this.safe(p); L282: c.river && inside(p, c.river); L283: p.y >= 165 && p.y <= 255; L284: p.y >= 915 && p.y <= 1005

Execution order (direct statements; nested callbacks stay within their owner):

L223-224 FirstStatement: const c = this.course, p = { ...last };

L225-258 ExpressionStatement: c.movers.forEach((m) => { const dx = m.to.x - m.from.x,

L259-265 ExpressionStatement: c.fans.forEach((f) => { const sweep = f.length / 2 + 9;

L266-279 IfStatement: if (c.piston &&

L280-280 IfStatement: if (c.lasers.some((r) => overlaps(p, r))) return { ...c.start };

L281-287 IfStatement: if (!this.safe(p)) { if (c.river && inside(p, c.river)) {

L288-288 ReturnStatement: return p;

src/games/mini-golf/ultimate/GolfInput.ts

ACTIVE SOURCE / tool or application | 99 lines | SHA-256 98d88dcc43590d46...

Maps pullback to shot velocity and defines the exact uniform responsive project/unproject camera transform.

Imports: ./courses, ./GolfCollision

State, resources and initial values

Line	Field / declaration
15	bounds: Rect = { x: 16, y: 64, w: 388, h: 624 };
16	zoom = 1;
17	targetZoom = 1;
18	center: Point = { x: 960, y: 575 };

shotVector [L3-12]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function shotVector(start: Point, end: Point)

Direct writes: x, y

Return at L11: { x, y, power: Math.min(1, speed / MAX_SPEED) }

Decision branches: L7: speed > MAX_SPEED

GolfView [L14-98]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfView.portrait [L19-21]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get portrait()

Return at L20: this.bounds.h >= this.bounds.w

```
get portrait() { return this.bounds.h >= this.bounds.w; }
```

GolfView.baseScale [L22-26]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get baseScale()

Return at L23: this.portrait ? Math.min(this.bounds.w / 1010, this.bounds.h / 1920) : Math.min(this.bounds.w / 1920, this.bounds.h / 1010)

GolfView.scale [L27-29]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get scale()

Return at L28: this.baseScale * this.zoom

```
get scale() { return this.baseScale * this.zoom; }
```

GolfView.origin [L30-35]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get origin()

Return at L31: { x: this.bounds.x + this.bounds.w / 2, y: this.bounds.y + this.bounds.h / 2, }

```
get origin() { return { x: this.bounds.x + this.bounds.w / 2, y: this.bounds.y + this.bounds.h / 2, }; }
```

GolfView.angle [L36-38]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get angle()

Return at L37: this.portrait ? Math.PI / 2 : 0

```
get angle() { return this.portrait ? Math.PI / 2 : 0; }
```

GolfView.project [L39-51]

Map source-world coordinates to the responsive screen with one uniform camera transform.

Signature: project(p: Point)

Return at L42: this.portrait ? { x: o.x - (p.y - this.center.y) * s, y: o.y + (p.x - this.center.x) * s, } : { x: o.x + (p.x - this.center.x) * s, y: o.y + (p.y - this.center.y) * s, }

```
project(p: Point) { const s = this.scale, o = this.origin; return this.portrait ? { x: o.x - (p.y - this.center.y) * s, y: o.y + (p.x - this.center.x) * s, } : { x: o.x + (p.x - this.center.x) * s, y: o.y + (p.y - this.center.y) * s, }; }
```

GolfView.unproject [L52-64]

Apply the exact inverse camera transform for legal world-space input.

Signature: unproject(p: Point)

Return at L55: this.portrait ? { x: this.center.x + (p.y - o.y) / s, y: this.center.y - (p.x - o.x) / s, } : { x: this.center.x + (p.x - o.x) / s, y: this.center.y + (p.y - o.y) / s, }

```
unproject(p: Point) { const s = this.scale, o = this.origin; return this.portrait ? { x: this.center.x + (p.y - o.y) / s, y: this.center.y - (p.x - o.x) / s, } : { x: this.center.x + (p.x - o.x) / s, y: this.center.y + (p.y - o.y) / s, }; }
```

GolfView.contains [L65-68]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: contains(p: Point)

Return at L67: p.x >= r.x && p.x <= r.x + r.w && p.y >= r.y && p.y <= r.y + r.h

```
contains(p: Point) { const r = this.bounds; return p.x >= r.x && p.x <= r.x + r.w && p.y >= r.y && p.y <= r.y + r.h; }
```

GolfView.configure [L69-71]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: configure(bounds: Rect)

Direct writes: this.bounds

GolfView.focus [L72-74]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: focus(_p: Point)

Direct writes: this.targetZoom

Calls / delegates: this.focusZoom

GolfView.focusZoom [L75-77]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private focusZoom()

Return at L76: Math.max(2.2, Math.min(4, 0.78 / this.baseScale))

GolfView.focused [L78-80]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get focused()

Return at L79: this.targetZoom > 1

```
get focused() { return this.targetZoom > 1; }
```

GolfView.transitioning [L81-83]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get transitioning()

Return at L82: Math.abs(this.zoom - this.targetZoom) > 0.015

GolfView.follow [L84-97]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: follow(p: Point, blend: number)

Direct writes: this.zoom, this.center.x, this.center.y

Decision branches: L86: Math.abs(this.zoom - this.targetZoom) < 0.001

src/games/mini-golf/ultimate/GolfPhysics.ts

ACTIVE SOURCE / tool or application | 369 lines | SHA-256 5f9b8974bf139ff2...

Authoritative active 240Hz ball simulation: shot validation, surfaces/forces, hazards/recovery, collision, capture, terminal states, snapshots and previews.

Imports: ./courses, ./courses/foundry, ./GolfDynamics, ./GolfCollision, ./GolfRules

State, resources and initial values

Line	Field / declaration
22	readonly ball: Ball;
23	readonly dynamics: GolfDynamics;
24	lastShot: Point;
25	strokes = 0;
26	status: "playing" | "won" | "lost" = "playing";
27	message = "";
28	messageTime = 0;
29	impacts = 0;
30	recoveries = 0;
31	private accumulator = 0;
32	readonly seed?: number;
33	private motionWindow = { time: 0, x: 0, y: 0, range: 0, impacts: 0 };

GolfPhysics [L21-368]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfPhysics.constructor [L34-48]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly course: Course, readonly difficulty: GolfDifficulty = 1, random: (() => number) | number = Math.random,)

Direct writes: this.ball, this.lastShot, this.seed, this.dynamics

Calls / delegates: GolfDynamics

GolfPhysics.limit [L49-51]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get limit()

Calls / delegates: strokeLimit

Return at L50: strokeLimit(this.course, this.difficulty)

GolfPhysics.score [L52-56]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get score()

Calls / delegates: scoreFor

Return at L53: this.status === "won" ? scoreFor(this.course, this.strokes, this.difficulty) : 0

GolfPhysics.stopped [L57-61]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get stopped()

Return at L58: (this.ball.vx === 0 && this.ball.vy === 0 && this.dynamics.ufo.carry <= 0)

```
get stopped() { return ( this.ball.vx === 0 && this.ball.vy === 0 && this.dynamics.ufo.carry <= 0 ); }
```

GolfPhysics.shootable [L62-66]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get shootable()

Return at L63: (this.status === "playing" && this.stopped && this.strokes < this.limit)

```
get shootable() { return ( this.status === "playing" && this.stopped && this.strokes < this.limit ); }
```

GolfPhysics.shoot [L67-82]

Validate and launch a normal shot through the authoritative physics API.

Signature: shoot(vx: number, vy: number)

Direct writes: this.lastShot, this.ball.vx, this.ball.vy, this.motionWindow.time

Calls / delegates: cap

Decision branches: L68: !this.shootable || !Number.isFinite(vx) || !Number.isFinite(vy) || Math.hypot(vx, vy) < 43.75

GolfPhysics.advance [L83-89]

Consume elapsed time using this helper's fixed-step/clock policy.

Signature: advance(seconds: number)

Direct writes: this.accumulator

Calls / delegates: this.tick

GolfPhysics.resetAccumulator [L90-92]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: resetAccumulator()

Direct writes: this.accumulator

GolfPhysics.recover [L93-101]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private recover(message: string)

Direct writes: this.dynamics.sink, this.dynamics.plates[1], this.dynamics.plates[2], this.message, this.messageTime

Calls / delegates: this.dynamics.recovery

GolfPhysics.tick [L102-295]

Advance obstacle dynamics, surface/field forces, ball integration, hazards, contacts, safe recovery and win/loss in the stated order.

Signature: tick(dt: number)

Direct writes: this.messageTime, this.message, friction, quicksand, b.vx, b.vy, pushed, b.x, b.y, d.planks[bank * 5 + i], d.gate, d.plates[i], d.portalCooldown, d.sink, impact, window.x, window.y, window.impacts, window.range, window.time, this.status

Calls / delegates: d.update, inside, distance, c.ground?.some, c.bridges.forEach, cap, c.planks?.forEach, row.forEach, c.plates?.forEach, d.safe, this.recover, c.lava?.some, c.lasers.some, d.laser, overlaps, hitCircle, d.mover, hitRect, hitRotated, d.piston, c.doors?.forEach, d.door

Decision branches: L103: this.status !== "playing"; L109: d.ufo.carry > 0; L118: inside(b, s); L119: s.friction !== undefined; L120: s.kind === "quicksand"; L121: s.push; L128: c.id === "foundry" && distance(b, oil) < oil.r; L131: dist < f.r && dist > (f.core ?? 0) + 8 && (!f.movingOnly || !this.stopped); L139: !f.movingOnly; L142: c.bridges && d.time % 8 >= 3 && d.time % 8 < 4 && !c.ground?.some((r) => inside(b, r)) && Math.hypot(b.vx, b.vy) < 150; L150: inside(b, r); L160: speed; L164: !pushed && next < 6; L167: inside(b, r) && d.planks[bank * 5 + i] === 0; L172: distance(b, p) < 26; L173: i === 0; L177: c.portals && d.portalCooldown === 0; L179: distance(b, portal.from) < 26; L190: !d.safe(b); L198: c.lava?.some((v) => distance(b, v) < v.r - 5); L202: c.lasers.some((r, i) => d.laser(i) && overlaps(b, r)); L208: quicksand && this.stopped; L210: d.sink >= 3; L233: c.diamond; L243: piston; L248: f.core; L251: contact && c.id === "foundry"; L256: impact; L258: window.time === 0; L269: window.time >= 8; L270: !this.stopped && window.range < 55 && this.impacts - window.impacts > 24; L281: !Number.isFinite(b.x + b.y + b.vx + b.vy) || b.x < -100 || b.x > 2020 || b.y < -100 || b.y > 1180; L291: Math.abs(b.x - c.hole.x) < 8.25 && Math.abs(b.y - c.hole.y) < 8.25; L294: this.strokes >= this.limit && this.stopped

Execution order (direct statements; nested callbacks stay within their owner):

L103-103 IfStatement: if (this.status !== "playing") return;

L104-106 FirstStatement: const b = this.ball, c = this.course,

L107-107 ExpressionStatement: d.update(dt, b, this.lastShot);

L108-108 ExpressionStatement: this.messageTime = Math.max(0, this.messageTime - dt);

L109-113 IfStatement: if (d.ufo.carry > 0) { this.message = "A lift from the locals";

L114-116 FirstStatement: let friction = 110, pushed = false,

L117-127 ForOfStatement: for (const s of c.surfaces) { if (inside(b, s)) {

L128-128 IfStatement: if (c.id === "foundry" && distance(b, oil) < oil.r) friction = 20;

L129-141 ForOfStatement: for (const f of c.fields) { const dist = distance(b, f);

L142-154 IfStatement: if (c.bridges &&

L155-155 ExpressionStatement: cap(b);

L156-156 ExpressionStatement: b.x += b.vx * dt;

L157-157 ExpressionStatement: b.y += b.vy * dt;

L158-159 FirstStatement: const speed = Math.hypot(b.vx, b.vy), next = Math.max(0, speed - friction * dt);

L160-163 IfStatement: if (speed) { b.vx *= next / speed;

L164-164 IfStatement: if (!pushed && next < 6) b.vx = b.vy = 0;

L165-170 ExpressionStatement: c.planks?.forEach((row, bank) => row.forEach((r, i) => {

L171-176 ExpressionStatement: c.plates?.forEach((p, i) => { if (distance(b, p) < 26) {

L177-189 IfStatement: if (c.portals && d.portalCooldown === 0) { for (const portal of c.portals)

L190-197 IfStatement: if (!d.safe(b)) { this.recover(

L198-201 IfStatement: if (c.lava?.some((v) => distance(b, v) < v.r - 5)) { this.recover("Too hot! Try another line");

L202-207 IfStatement: if (c.lasers.some((r, i) => d.laser(i) && overlaps(b, r))) { this.recover(

L208-214 IfStatement: if (quicksand && this.stopped) { d.sink += dt;

L215-215 FirstStatement: let impact = false;

L216-217 ForOfStatement: for (const v of c.circles) impact = hitCircle(b, v, v.r, v.bounce ?? 1) || impact;

L218-224 ForStatement: for (let i = 0; i < c.movers.length; i++) { const m = d.mover(i);

L225-232 ForOfStatement: for (const f of c.fans) { const angle = (d.time * f.spin * Math.PI) / 180,

L233-241 IfStatement: if (c.diamond) impact =

L242-242 FirstStatement: const piston = d.piston();

L243-243 IfStatement: if (piston) impact = hitRect(b, piston, piston.vx) || impact;

L244-246 ExpressionStatement: c.doors?.forEach((_, i) => { impact = hitRect(b, d.door(i), 0, d.doorVelocity[i]) || impact;

L247-252 ForOfStatement: for (const f of c.fields) if (f.core) {

L254-255 ForStatement: for (let pass = 0; pass < 2; pass++) for (const r of c.walls) impact = hitRect(b, r) || impact;

L256-256 IfStatement: if (impact) this.impacts++;

L257-257 FirstStatement: const window = this.motionWindow;

L258-263 IfStatement: if (window.time === 0) { window.x = b.x;

L264-264 ExpressionStatement: window.time += dt;

L265-268 ExpressionStatement: window.range = Math.max(window.range,

L269-280 IfStatement: if (window.time >= 8) { if (

L281-290 IfStatement: if (!Number.isFinite(b.x + b.y + b.vx + b.vy) ||

L291-294 IfStatement: if (Math.abs(b.x - c.hole.x) < 8.25 && Math.abs(b.y - c.hole.y) < 8.25) { Object.assign(b, c.hole, { vx: 0, vy: 0 });

GolfPhysics.exportState [L296-318]

Copy the ball/dynamics snapshot for isolated search simulations.

Signature: exportState()

Return at L298: { course: this.course.id, difficulty: this.difficulty, seed: this.seed, ball: { ...this.ball }, lastShot: { ...this.lastShot }, strokes: this.strokes, status: this.status, dynamics: { time: d.time, gate: d.gate, plates: [...d.plates], doors: [...d.doors], doorVelocity: [...d.doorVelocity], planks: [...d.planks], sink: d.sink, portalCooldown: d.port ... [continued in source reference]

```
exportState() { const d = this.dynamics; return { course: this.course.id, difficulty: this.difficulty, seed: this.seed, ball: { ...this.ball }, lastShot: { ...this.lastShot }, strokes: this.strokes, status: this.status, dynamics: { time: d.time, gate: d.gate, plates: [...d.plates], doors: [...d.doors], doorVelocity: [...d.doorVelocity], planks: [...d.planks], sink: d.sink, portalCooldown: d.portalCooldown, ufo: { ...d.ufo }, }, }; }
```

GolfPhysics.restoreSimulation [L319-327]

Install a trusted simulation snapshot in a copied forecast world and reset catch-up/window state.

Signature: restoreSimulation(state: ReturnType<GolfPhysics["exportState"]>)

Direct writes: this.motionWindow.time, this.lastShot, this.strokes, this.status

Calls / delegates: this.resetAccumulator, structuredClone

GolfPhysics.preview [L328-351]

Simulate an isolated bounded ghost path; never mutate the live ball.

Signature: preview(velocity: Point): Point[]

Direct writes: ghost.lastShot, ghost.dynamics.time, ghost.dynamics.portalCooldown, ghost.dynamics.ufo.done

Calls / delegates: GolfPhysics, ghost.shoot, ghost.tick, points.push

Return at L330: []

Return at L337: []

Return at L350: points

Decision branches: L330: !seconds || !this.shootable; L337: !ghost.shoot(velocity.x, velocity.y); L341: ghost.impacts || ghost.recoveries || ghost.dynamics.portalCooldown > this.dynamics.portalCooldown || ghost.status !== "playing"; L348: i % 4 === 0

GolfPhysics.snapshot [L352-367]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Return at L353: { course: this.course.id, difficulty: this.difficulty, ball: { ...this.ball }, strokes: this.strokes, limit: this.limit, status: this.status, time: this.dynamics.time, plates: [...this.dynamics.plates], gate: this.dynamics.gate, doors: [...this.dynamics.doors], recoveries: this.recoveries, shootable: this.shootable, }

```
snapshot() { return { course: this.course.id, difficulty: this.difficulty, ball: { ...this.ball }, strokes:
this.strokes, limit: this.limit, status: this.status, time: this.dynamics.time, plates: [...this.dynamics.plates],
gate: this.dynamics.gate, doors: [...this.dynamics.doors], recoveries: this.recoveries, shootable: this.shootable,
}; }
```

src/games/mini-golf/ultimate/GolfProgress.ts

ACTIVE SOURCE / tool or application | 39 lines | SHA-256 db2755b6aa8d332d...

Validates and stores four casual course bests separately from competition.

State, resources and initial values

Line	Field / declaration
3	best = [0, 0, 0, 0];

Module constants and data

L1-1 KEY = "pocket-rivals.ultimate-golf.v1"

GolfProgress [L2-38]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfProgress.constructor [L4-23]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor()

Direct writes: this.best

Calls / delegates: localStorage.getItem, data.best.map

Decision branches: L7: data?.version === 1 && Array.isArray(data.best) && data.best.length === 4

GolfProgress.record [L24-34]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: record(course: number, score: number)

Direct writes: this.best[course]

Calls / delegates: localStorage.setItem

GolfProgress.total [L35-37]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get total()

Calls / delegates: this.best.reduce

Return at L36: this.best.reduce((a, b) => a + b, 0)

src/games/mini-golf/ultimate/GolfRenderer.ts

ACTIVE SOURCE / tool or application | 291 lines | SHA-256 57c5b3f0818f6773...

Draws active course data, dynamics, ball/halo, rival and aim preview with retained masked Graphics.

Imports: phaser, ../../../core/ThemeManager, ./GolfInput, ./GolfPhysics, ./courses, ./courses/foundry, ./GolfRules

State, resources and initial values

Line	Field / declaration
9	private lastAim: Point | null = null;
10	private previewPoints: Point[] = [];
11	private g: Phaser.GameObjects.Graphics;
12	private frame: Phaser.GameObjects.Graphics;
13	private maskShape: Phaser.GameObjects.Graphics;
14	private mask: Phaser.Display.Masks.GeometryMask;

GolfRenderer [L8-290]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfRenderer.constructor [L15-26]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(scene: Phaser.Scene, readonly view: GolfView,)

Direct writes: this.frame, this.g, this.maskShape, this.mask

Calls / delegates: scene.add.graphics, scene.make.graphics, this.maskShape.fillStyle(0xffffff).fillRect, this.maskShape.fillStyle, this.maskShape.createGeometryMask, this.g.setMask

GolfRenderer.draw [L27-282]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: draw(world: GolfPhysics, aim: Point | null, rival?: Point, playerColor = theme.orange,)

Direct writes: this.previewPoints, this.lastAim

Calls / delegates: this.view.project, this.maskShape .clear() .fillStyle(0xffffff) .fillRect, this.maskShape .clear() .fillStyle, this.maskShape .clear, this.frame .clear() .lineStyle(3, theme.ink) .strokeRoundedRect, this.frame .clear() .lineStyle, this.frame .clear, g.clear() .setPosition(origin.x, origin.y) .setRotation(this.view.angle) .setScale, g.clear() .setPosition(origin.x, origin.y) .setRotation, g.clear() .setPosition, g.clear, g.fillStyle(color).fillRect, g.fillStyle, g.lineStyle(line, theme.ink).strokeRect, g.lineStyle, g.fillStyle(color).fillCircle, g.lineStyle(line, theme.ink).strokeCircle, rect, c.bridges?.forEach, d.bridgeSafe, c.planks?.forEach, row.forEach, g.lineStyle(3, theme.ink).lineBetween, c.surfaces.forEach, g.lineStyle(4, theme.ink, 0.5).lineBetween, g.fillStyle(theme.ink, 0.5).fillCircle, g.fillStyle(theme.ink, 0.15).fillCircle, circle, c.fields.forEach, g.lineStyle(3, theme.blue, 0.45) .strokeCircle(f.x, f.y, f.r) .strokeCircle, g.lineStyle(3, theme.blue, 0.45) .strokeCircle, c.lava?.forEach, c.circles.forEach, c.walls.forEach, g.save, g.translateCanvas, g.rotateCanvas, g.restore, d.mover, c.fans.forEach, d.piston, c.lasers.forEach, d.laser, c.plates?.forEach, c.doors?.forEach, d.door, c.portals?.forEach, g.lineStyle(5, color).strokeCircle, g.lineStyle(4, color).lineBetween, g.fillStyle(theme.blue, 0.3).fillCircle, g.fillStyle(theme.blue).fillEllipse, g.lineStyle(4, theme.ink).strokeEllipse, g.lineStyle(4, theme.ink).lineBetween, g.fillStyle(theme.orange).fillTriangle, g.lineStyle(3, theme.ink, 0.6).lineBetween, world.preview, this.previewPoints.forEach, g .fillStyle(theme.white, 1 - i / (this.previewPoints.length + 4)) .fillCircle, g .fillStyle, g.fillStyle(theme.white, 1 - i / 15).fillCircle, g.lineStyle(3, theme.blue, 0.85).strokeCircle, g.lineStyle(3, playerColor, 0.8).strokeCircle

Decision branches: L59: line; L63: line; L74: c.ground; L77: c.river; L83: t < limit; L85: t > 0; L106: r.push; L128: c.id === "foundry"; L133: f.core; L146: c.diamond; L164: m.r; L182: piston; L219: u.active; L220: u.carry > 0 || u.hover > 0; L241: aim && world.shootable; L251: c.id === "horizon" && preview && speed; L252: this.lastAim !== aim; L261: preview && speed; L271: rival

Execution order (direct statements; nested callbacks stay within their owner):

L33-36 FirstStatement: const g = this.g, c = world.course,

L37-38 FirstStatement: const bounds = this.view.bounds, origin = this.view.project({ x: 0, y: 0 });

L39-42 ExpressionStatement: this.maskShape .clear()

L43-52 ExpressionStatement: this.frame .clear()

L53-56 ExpressionStatement: g.clear() .setPosition(origin.x, origin.y)

L57-60 FirstStatement: const rect = (r: Rect, color: number, line = 3) => { g.fillStyle(color).fillRect(r.x, r.y, r.w, r.h);

L61-64 FirstStatement: const circle = (p: Point, r: number, color: number, line = 3) => { g.fillStyle(color).fillCircle(p.x, p.y, r);

L65-73 ExpressionStatement: rect({ x: 0, y: 0, w: 1920, h: 1080 },

L74-76 IfStatement: if (c.ground) for (const r of c.ground)

L77-77 IfStatement: if (c.river) rect(c.river, theme.blue, 0);

L78-78 ExpressionStatement: c.bridges?.forEach((r) => rect(r, d.bridgeSafe() ? 0xf7edcc : 0x6eafcb, 2));

L79-94 ExpressionStatement: c.planks?.forEach((row, b) => row.forEach((r, i) => {

L95-103 FirstStatement: const colors = { slick: 0x9eaaa2,

L104-127 ExpressionStatement: c.surfaces.forEach((r) => { rect(r, colors[r.kind], 0);

L128-128 IfStatement: if (c.id === "foundry") circle(oil, oil.r, 0x7c8d88, 2);

L129-136 ExpressionStatement: c.fields.forEach((f) => { g.lineStyle(3, theme.blue, 0.45)

L137-140 ExpressionStatement: c.lava?.forEach((v) => { circle(v, v.r, theme.orange, 4);

L141-144 ExpressionStatement: c.circles.forEach((v) => { circle(v, v.r, v.bounce ? theme.orange : theme.blue, 4);

L145-145 ExpressionStatement: c.walls.forEach((r) => rect(r, c.id === "temple" ? 0x849a78 : 0x788b85, 4));

L146-161 IfStatement: if (c.diamond) { g.save();

L162-171 ForStatement: for (let i = 0; i < c.movers.length; i++) { const m = d.mover(i);

L172-180 ExpressionStatement: c.fans.forEach((f) => { g.save();

L181-181 FirstStatement: const piston = d.piston();

L182-182 IfStatement: if (piston) rect(piston, theme.orange, 4);

L183-192 ExpressionStatement: c.lasers.forEach((r, i) => { rect(r, d.laser(i) ? 0xe56859 : 0xdcb3a6, 0);

L193-201 ExpressionStatement: c.plates?.forEach((p, i) => { circle(

L202-204 ExpressionStatement: c.doors?.forEach((_, i) => rect(d.door(i), d.doors[i] > 0.95 ? theme.blue : theme.orange, 5),

L205-217 ExpressionStatement: c.portals?.forEach((p, i) => { const color = [theme.orange, theme.blue, 0xc398d4][i];

L218-218 FirstStatement: const u = d.ufo;

L219-225 IfStatement: if (u.active) { if (u.carry > 0 || u.hover > 0)

L226-226 ExpressionStatement: circle(c.hole, 11, theme.ink, 0);

L227-232 ExpressionStatement: g.lineStyle(4, theme.ink).lineBetween(c.hole.x,

L233-240 ExpressionStatement: g.fillStyle(theme.orange).fillTriangle(c.hole.x + 42,

L241-270 IfStatement: if (aim && world.shootable) { const b = world.ball,

L271-274 IfStatement: if (rival) { g.lineStyle(3, theme.blue, 0.85).strokeCircle(rival.x, rival.y, 19);

L276-280 ExpressionStatement: g.lineStyle(3, playerColor, 0.8).strokeCircle(world.ball.x,

L281-281 ExpressionStatement: circle(world.ball, 7, theme.white, 2);

GolfRenderer.draw.rect [L57-60]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: rect = (r: Rect, color: number, line = 3) =>

Calls / delegates: g.fillStyle(color).fillRect, g.fillStyle, g.lineStyle(line, theme.ink).strokeRect, g.lineStyle

Decision branches: L59: line

GolfRenderer.draw.circle [L61-64]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: circle = (p: Point, r: number, color: number, line = 3) =>

Calls / delegates: g.fillStyle(color).fillCircle, g.fillStyle, g.lineStyle(line, theme.ink).strokeCircle, g.lineStyle

Decision branches: L63: line

GolfRenderer.dispose [L283-289]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.g.clearMask, this.mask.destroy, this.maskShape.destroy, this.g.destroy, this.frame.destroy

src/games/mini-golf/ultimate/GolfRules.ts

ACTIVE SOURCE / tool or application | 13 lines | SHA-256 75ba49bfbb493a14...

Defines challenge speed/stroke/preview/scoring records and the authoritative limit/score formula.

Imports: ./courses

Module constants and data

L3-7 difficulties = [{ name: "Easy", factor: 1.5, speed: 0.7, preview: 0.6, rate: 10, bonus: 100 }, { name: "Normal", factor: 1.1, speed: 1, preview: 0.3, rate: 25, bonus: 250 }, { name: "Hard", factor: 0.75, speed: 1.3, preview: 0, rate: 50, bonus: 500 },] as const

GolfDifficulty [L2-2]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type GolfDifficulty = 0 | 1 | 2;
```

strokeLimit [L8-9]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: strokeLimit = (course: Course, d: GolfDifficulty) =>

scoreFor [L10-12]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: scoreFor = (course: Course, strokes: number, d: GolfDifficulty) =>

Calls / delegates: strokeLimit

src/games/mini-golf/ultimate/UltimateGolfGame.ts

ACTIVE SOURCE / tool or application | 486 lines | SHA-256 83c91cbe3105dbd2...

Owns active golf mode selection, course/help screens, gestures, responsive camera/UI, updates, results and cleanup.

Imports: ../../../shared/ai/MatchRandom, ../../../shared/contracts/GameModule, ../../../shared/contracts/AIController, ../../../core/ThemeManager, ./courses, ./GolfRenderer, ./GolfInput, ./GolfRules, ./components/GolfUI, ./components/CourseArt, ./components/ModeArt, ./modes/CasualMode, ./modes/LocalVersusMode, ./modes/AIChallengeMode, ./modes/CompetitiveStats, ./modes/CompetitionRules

State, resources and initial values

Line	Field / declaration
23	private ctx!: GameContext;
24	private ui!: GolfUI;
25	private casual = new CasualMode();
26	private local?: LocalVersusMode;
27	private race?: AIChallengeMode;
28	private mode: Mode = "local";
29	private difficulty: Difficulty = "medium";
43	private plan = [0];
44	private renderer?: GolfRenderer;
45	private renderedWorld?: object;
46	private view = new GolfView();
47	private phase = "modes";
48	private paused = false;
49	private aim: ReturnType<typeof shotVector> | null = null;
50	private anchor: { x: number; y: number } | null = null;
51	private impacts = 0;
52	private recoveries = 0;

Mode [L21-21]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
type Mode = "casual" | "local" | "ai";
```

UltimateGolfGame [L22-485]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

UltimateGolfGame.constructor [L30-32]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private initialOptions?: MatchOptions)

Calls / delegates: this.applyOptions

Decision branches: L31: initialOptions

UltimateGolfGame.applyOptions [L33-42]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private applyOptions(options: MatchOptions)

Direct writes: this.mode, this.difficulty

Decision branches: L34: options.variant === "casual"; L36: options.mode === "ai"

UltimateGolfGame.world [L53-59]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: private get world()

Return at L54: this.mode === "casual" ? this.casual.world : this.mode === "local" ? this.local?.world : this.race?.worlds[0]

```
private get world() { return this.mode === "casual" ? this.casual.world : this.mode === "local" ? this.local?.world : this.race?.worlds[0]; }
```

UltimateGolfGame.course [L60-66]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: private get course()

Return at L61: this.mode === "casual" ? this.casual.course : this.mode === "local" ? this.plan[this.local!.hole] : this.plan[this.race!.holes[0]]

```
private get course() { return this.mode === "casual" ? this.casual.course : this.mode === "local" ? this.plan[this.local!.hole] : this.plan[this.race!.holes[0]]; }
```

UltimateGolfGame.initialize [L67-104]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.ui, this.anchor, this.aim

Calls / delegates: this.applyOptions, GolfUI, this.action, ctx.input.press, ctx.input.drag, this.view.contains, this.view.unproject, shotVector, this.world.shoot, ctx.audio.play, this.cancel, this.modes

Decision branches: L69: ctx.options; L71: !this.paused; L74: this.phase === "playing"; L79: this.phase !== "playing" || !this.world?.shootable || this.view.transitioning || !this.view.contains({ x, y }); L91: this.anchor; L95: this.phase === "playing" && this.anchor && this.world; L97: this.world.shoot(v.x, v.y)

Execution order (direct statements; nested callbacks stay within their owner):

L68-68 ExpressionStatement: this.ctx = ctx;

L69-69 IfStatement: if (ctx.options) this.applyOptions(ctx.options);

L70-72 ExpressionStatement: this.ui = new GolfUI((action) => { if (!this.paused) this.action(action);

L73-76 ExpressionStatement: ctx.input.press(["KeyF", "KeyR"], (code) => { if (this.phase === "playing")

L77-102 ExpressionStatement: ctx.input.drag((x, y) => {

L103-103 ExpressionStatement: this.modes();

UltimateGolfGame.action [L105-219]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private action(action: string)

Direct writes: this.mode, this.casual.difficulty, this.difficulty, this.phase, panel.querySelector("h2")!.textContent

Calls / delegates: this.ctx.audio.play, this.ctx.navigation?.pause, this.ctx.navigation?.exit, this.modes, this.select, action.startsWith, action.split, Number, this.startCompetitionOrCasual, courses.map, this.local?.begin, this.race?.begin, this.showPlay, this.ctx.finishArcade, this.ui.closeHelp, this.cancel, this.race?.pause, this.ui.help, this.view.focus, this.ui.focus, this.ui.field.querySelector, panel.querySelector, panel.querySelectorAll("p")[1]?.remove, panel.querySelectorAll, panel.insertAdjacentHTML, this.local?.forfeit, this.race?.forfeit, this.afterCompetition

Decision branches: L107: action === "pause"; L111: action === "exit"; L115: action === "back"; L116: this.phase === "modes"; L117: this.phase === "select"; L121: action === "courses"; L125: action.startsWith("mode:"); L130: action.startsWith("difficulty:"); L135: action.startsWith("ai:"); L140: action.startsWith("course:"); L144: action === "all"; L148: action === "begin"; L149: this.mode === "local"; L154: action === "rematch"; L158: action === "next"; L164: action === "finish"; L171: action === "close-help"; L176: action === "help" && this.phase === "playing"; L183: action === "focus" && this.phase === "playing" && this.world; L189: action === "retry" && this.phase === "playing"; L190: this.mode === "casual"; L210: action === "retry" && this.phase === "result"; L214: action === "forfeit"

Execution order (direct statements; nested callbacks stay within their owner):

L106-106 ExpressionStatement: this.ctx.audio.play("tap");

L107-110 IfStatement: if (action === "pause") { this.ctx.navigation?.pause();

L111-114 IfStatement: if (action === "exit") { this.ctx.navigation?.exit();

L115-120 IfStatement: if (action === "back") { if (this.phase === "modes") this.ctx.navigation?.exit();

L121-124 IfStatement: if (action === "courses") { this.select();

L125-129 IfStatement: if (action.startsWith("mode:")) { this.mode = action.split(":")[1] as Mode;

L130-134 IfStatement: if (action.startsWith("difficulty:")) { this.casual.difficulty = Number(action.split(":")[1]) as GolfDifficulty;

L135-139 IfStatement: if (action.startsWith("ai:")) { this.difficulty = action.split(":")[1] as Difficulty;

L140-143 IfStatement: if (action.startsWith("course:")) { this.startCompetitionOrCasual([Number(action.split(":")[1])]);

L144-147 IfStatement: if (action === "all") { this.startCompetitionOrCasual(courses.map((_, i) => i));

L148-153 IfStatement: if (action === "begin") { if (this.mode === "local") this.local?.begin();

L154-157 IfStatement: if (action === "rematch") { this.startCompetitionOrCasual(this.plan);

L158-163 IfStatement: if (action === "next") { this.startCompetitionOrCasual([

L164-170 IfStatement: if (action === "finish") { this.ctx.finishArcade(

L171-175 IfStatement: if (action === "close-help") { this.ui.closeHelp();

L176-182 IfStatement: if (action === "help" && this.phase === "playing") { this.cancel();

L183-188 IfStatement: if (action === "focus" && this.phase === "playing" && this.world) { this.cancel();

L189-209 IfStatement: if (action === "retry" && this.phase === "playing") { if (this.mode === "casual")

L210-213 IfStatement: if (action === "retry" && this.phase === "result") { this.startCompetitionOrCasual([this.casual.course]);

L214-218 IfStatement: if (action === "forfeit") { this.local?.forfeit();

UltimateGolfGame.cancel [L220-224]

Invalidate/cancel a gesture or asynchronous computation and clear pending state.

Signature: private cancel()

Direct writes: this.anchor, this.aim

Calls / delegates: this.ui?.meter

UltimateGolfGame.release [L225-233]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private release()

Direct writes: this.race, this.local, this.renderer, this.renderedWorld

Calls / delegates: this.cancel, this.race?.dispose, this.renderer?.dispose

UltimateGolfGame.modes [L234-262]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private modes()

Direct writes: this.phase

Calls / delegates: this.release, this.ui.title, this.ui.screen, ([["local", "2 Player Challenge", "Challenge a friend to finish first.", "#f9d7b5",], ["ai", "1 Player vs AI", "Race against a smart opponent.", "#c8e7eb"], ["casual", "1 Player Casual", "Complete courses, earn points, beat your best.", "#d9e9bc",],] as const) .map(([mode, title, desc, color]) => `<button class="golf-mode" data-action="mode:${mode}" style="background:${color}">${modeArt(mode)}<span><strong>${title}</strong><small>${desc}</small></span></button>`,) .join, ([["local", "2 Player Challenge", "Challenge a friend to finish first.", "#f9d7b5",], ["ai", "1 Player vs AI", "Race against a smart opponent.", "#c8e7eb"], ["casual", "1 Player Casual", "Complete courses, earn points, beat your best.", "#d9e9bc",],] as const) .map, modeArt

UltimateGolfGame.select [L263-277]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private select()

Direct writes: this.phase

Calls / delegates: this.release, this.ui.title, this.ui.screen, courses.map((c, i) => `<button class="golf-course" style="background:#${c.color.toString(16)}" data-action="course:${i}">${courseArt(c)}<strong>${c.name}</strong><small>${this.mode === "casual" && this.casual.progress.best[i] ? this.casual.progress.best[i] + " points" : c.subtitle}</small></button>`).join, courses.map, c.color.toString, courseArt, difficulties.map((d, i) => `<button data-action="difficulty:${i}" aria-pressed="${i === this.casual.difficulty}">${d.name}</button>`).join, difficulties.map, (["medium", "hard", "impossible"] as const).map((d) => `<button data-action="ai:${d}" aria-pressed="${d === this.difficulty}">${d[0].toUpperCase() + d.slice(1)}</button>`).join, (["medium", "hard", "impossible"] as const).map, d[0].toUpperCase, d.slice

UltimateGolfGame.startCompetitionOrCasual [L278-296]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private startCompetitionOrCasual(plan: number[])

Direct writes: this.plan, this.local, this.race

Calls / delegates: this.release, freshSeed, this.casual.start, this.showPlay, LocalVersusMode, AIChallengeMode, this.ready

Decision branches: L282: this.mode === "casual"; L286: this.mode === "local"

UltimateGolfGame.ready [L297-305]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private ready()

Direct writes: this.phase

Calls / delegates: this.cancel, this.ui.title, this.ui.screen, strokeLimit

UltimateGolfGame.showPlay [L306-315]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private showPlay()

Direct writes: this.phase, this.view

Calls / delegates: this.cancel, GolfView, this.ui.playing, this.view.configure, this.ui.bounds, this.view.follow, this.attachWorld, this.update

UltimateGolfGame.attachWorld [L316-321]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private attachWorld()

Direct writes: this.renderer, this.renderedWorld, this.impacts, this.recoveries

Calls / delegates: this.renderer?.dispose, GolfRenderer

UltimateGolfGame.casualResult [L322-332]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private casualResult()

Direct writes: this.phase

Calls / delegates: this.cancel, this.casual.record, this.ctx.audio.play, this.ui.title, this.ui.screen

UltimateGolfGame.afterCompetition [L333-355]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private afterCompetition()

Direct writes: this.phase

Calls / delegates: this.ready, this.cancel, recordCompetition, this.ctx.audio.play, this.ui.title, this.ui.screen, competition.totals.map((r, i) => `<tr><th>${names[i]}</th><td>${r.strokes + (this.mode === "ai" && !r.completed && competition.attempts[i].length === 0 ? this.race!.worlds[i].strokes : 0)}</td><td>${r.completed ? timeText(r.time) : "DNF"}</td></tr>`).join, competition.totals.map, timeText, competition.totals.some

Decision branches: L335: competition.phase === "playing"; L336: competition.phase !== "complete"

UltimateGolfGame.update [L356-430]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.impacts, this.recoveries

Calls / delegates: this.casual.update, this.local!.update, this.race!.update, this.attachWorld, this.cancel, this.view.follow, this.renderer?.draw, this.ctx.audio.play, this.ui.title, timeText, this.ui.message, this.ui.meter, this.ui.rival, this.casualResult, this.afterCompetition

Decision branches: L357: this.paused || this.phase !== "playing" || !this.world; L358: this.mode === "casual"; L359: this.mode === "local"; L362: this.renderedWorld !== w; L367: !w.shootable; L368: !this.anchor; L381: w.impacts !== this.impacts; L385: w.recoveries !== this.recoveries; L413: this.mode === "ai"; L416: aiDone; L427: this.mode === "casual"; L428: w.status !== "playing"

Execution order (direct statements; nested callbacks stay within their owner):

L357-357 IfStatement: if (this.paused || this.phase !== "playing" || !this.world) return;

L358-360 IfStatement: if (this.mode === "casual") this.casual.update(dt); else if (this.mode === "local") this.local!.update(dt);

L361-361 FirstStatement: const w = this.world;

L362-366 IfStatement: if (this.renderedWorld !== w) { this.attachWorld();

L367-367 IfStatement: if (!w.shootable) this.cancel();

L368-368 IfStatement: if (!this.anchor) this.view.follow(w.ball, 1 - Math.exp(-dt * 12));

L369-372 FirstStatement: const playerColor = this.mode === "local" && this.local?.player === 1

L373-380 ExpressionStatement: this.renderer?.draw(w,

L381-384 IfStatement: if (w.impacts !== this.impacts) { this.ctx.audio.play("hit");

L385-388 IfStatement: if (w.recoveries !== this.recoveries) { this.cancel();

L389-394 FirstStatement: const elapsed = this.mode === "local"

L395-396 FirstStatement: const prefix = this.mode === "local" ? `Player ${this.local!.player + 1} · ` : "";

L397-400 ExpressionStatement: this.ui.title(courses[this.course].name,

L401-402 FirstStatement: const playerDone = this.mode === "ai" && this.race!.attempts[0].length === this.plan.length;

L403-411 ExpressionStatement: this.ui.message(playerDone

L412-412 ExpressionStatement: this.ui.meter(this.aim?.power ?? null);

L413-426 IfStatement: if (this.mode === "ai") { const r = this.race!;

L427-429 IfStatement: if (this.mode === "casual") { if (w.status !== "playing") this.casualResult();

UltimateGolfGame.resize [L431-436]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: resize()

Calls / delegates: this.cancel, this.view.configure, this.ui.bounds, this.view.follow

Decision branches: L432: !this.ui; L435: this.world

UltimateGolfGame.pause [L437-443]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.cancel, this.race?.pause, this.world?.resetAccumulator, this.ui.pause

UltimateGolfGame.resume [L444-449]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

Calls / delegates: this.world?.resetAccumulator, this.ui.pause, this.resize

UltimateGolfGame.dispose [L450-453]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.release, this.ui.dispose

UltimateGolfGame.snapshot [L454-484]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.race.worlds[1].snapshot, this.race.ai.snapshot, this.world?.snapshot

Return at L455: { phase: this.phase, mode: this.mode, view: { zoom: this.view.zoom, targetZoom: this.view.targetZoom, scale: this.view.scale, center: { ...this.view.center }, bounds: this.view.bounds, }, aiming: !!this.anchor, progress: [...this.casual.progress.best], competition: this.local ? { player: this.local.player, phase: this.local.phase, elapsed: this.loc ... [continued in source reference]

src/games/mini-golf/ultimate/ai

src/games/mini-golf/ultimate/ai/GolfAI.ts

ACTIVE SOURCE / tool or application | 100 lines | SHA-256 abde5fc06b0b7979...

Active worker-planning scheduler with seeded identity, delay/generation/cancel guards and ordinary world.shoot calls.

Imports: ../../../../shared/ai/MatchRandom, ../../../../shared/utilities/WorkerTask, ../../../../shared/contracts/AIController, ../GolfPhysics, ./ShotPlanner

State, resources and initial values

Line	Field / declaration
7	readonly personality: MatchRandom;
8	private task: WorkerTask<PlanInput, ShotPlan>;
9	private busy = false;
10	private generation = 0;
11	private plan: ShotPlan | null = null;
12	private deadline = 0;
13	private time = 0;
14	private waypoint = 0;
15	private origin = { x: 0, y: 0 };
16	accelerated = false;

GolfAI [L6-99]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfAI.constructor [L17-30]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly difficulty: Difficulty, report: (ms: number) => void, seed?: number,)

Direct writes: this.personality, this.task

Calls / delegates: MatchRandom, WorkerTask, Worker, URL

GolfAI.update [L31-76]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, world: GolfPhysics)

Direct writes: this.time, this.plan, this.waypoint, this.busy, this.origin, this.deadline

Calls / delegates: world.shoot, this.personality.sample, this.task .run({ state: world.exportState(), difficulty: this.difficulty, waypoint: this.waypoint, delay, seed: this.personality.seed, replay: replayMode(), }) .then, this.task .run, world.exportState, replayMode

Decision branches: L33: !world.shootable; L37: this.plan && this.time >= this.deadline; L40: Math.hypot(world.ball.x - p.origin.x, world.ball.y - p.origin.y) < 12; L48: this.busy || this.plan || this.time < this.deadline; L69: token !== this.generation; L71: plan

Execution order (direct statements; nested callbacks stay within their owner):

L32-32 ExpressionStatement: this.time += dt;

L33-36 IfStatement: if (!world.shootable) { this.plan = null;

L37-47 IfStatement: if (this.plan && this.time >= this.deadline) { const p = this.plan;

L48-48 IfStatement: if (this.busy || this.plan || this.time < this.deadline) return;

L49-51 FirstStatement: const baseDelay = { medium: 2.2, hard: 1.6, impossible: 1.2 }[this.difficulty] *

L52-54 FirstStatement: const delay = this.accelerated ? 0.25

L55-55 FirstStatement: const token = this.generation;

L56-56 ExpressionStatement: this.busy = true;

L57-57 ExpressionStatement: this.origin = { ...world.ball };

L58-58 ExpressionStatement: this.deadline = this.time + delay;

L59-75 ExpressionStatement: void this.task .run({

GolfAI.cancel [L77-83]

Invalidate/cancel a gesture or asynchronous computation and clear pending state.

Signature: cancel()

Direct writes: this.busy, this.plan, this.deadline

Calls / delegates: this.task.cancel

GolfAI.reset [L84-87]

Reset this component's transient state for a new round/attempt.

Signature: reset()

Direct writes: this.waypoint

Calls / delegates: this.cancel

GolfAI.dispose [L88-90]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.cancel

GolfAI.snapshot [L91-98]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L92: { ...this.personality.snapshot(), thinking: this.busy, waypoint: this.waypoint, pending: !!this.plan, }

src/games/mini-golf/ultimate/ai/ShotPlanner.ts

ACTIVE SOURCE / tool or application | 206 lines | SHA-256 82649711213fa7fc...

Simulates bounded shot/wait candidates against a real copied 240Hz world, refines comparable landings and returns a legal plan.

Imports: ../../../../shared/ai/MatchRandom, ../GolfPhysics, ../courses, ./routes, ../../../../shared/contracts/AIController

PlanInput [L6-13]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface PlanInput { state: ReturnType<GolfPhysics["exportState"]>; difficulty: Difficulty; waypoint: number; delay: number; seed?: number; replay?: boolean; }
```

ShotPlan [L14-22]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface ShotPlan { vx: number; vy: number; wait: number; waypoint: number; origin: { x: number; y: number }; milliseconds: number; candidates: number; }
```

planShot [L24-205]

Forecast delay, evaluate bounded legal shot/wait simulations, refine the best landing and return a comparable seeded plan.

Signature: export function planShot(input: PlanInput): ShotPlan

Direct writes: trial.recoveries, trial.impacts, cost, best, shot, alternatives, alternatives.length

Calls / delegates: MatchRandom, performance.now, courses.findIndex, getRoute, GolfPhysics, base.restoreSimulation, advanceReached, base.tick, base.exportState, trial.restoreSimulation, trial.tick, trial.shoot, style.rank, alternatives.filter, alternatives.push, alternatives.sort, evaluate, alternatives .filter((v) => v.cost <= best + tolerance) .sort, alternatives .filter

Return at L198: { ...shot, origin, waypoint, milliseconds: performance.now() - start, candidates, }

Decision branches: L43: Math.hypot(base.ball.x - (t[2] ?? t[0]), base.ball.y - (t[3] ?? t[1])) > 25; L84: !trial.shoot(vx, vy); L87: trial.stopped || trial.status !== "playing"; L99: ci === 3; L100: Math.hypot(tx - 1810, ty - 200) < 60 && !trial.dynamics.plates[1]; L102: Math.hypot(tx - 1810, ty - 950) < 60 && !trial.dynamics.plates[2]; L105: cost < best; L109: cost <= best + tolerance; L148: best < 1 || (input.replay ? candidates >= candidateLimit * 0.7 : performance.now() - start > budget * 0.7); L163: input.replay ? candidates >= candidateLimit * 0.9 : performance.now() - start > budget * 0.9; L177: input.replay ? candidates >= candidateLimit : performance.now() - start >= budget; L194: comparable.length

Execution order (direct statements; nested callbacks stay within their owner):

L25-25 FirstStatement: const style = new MatchRandom(input.seed ?? input.state.seed);

L26-28 FirstStatement: const candidateLimit = { medium: 180, hard: 420, impossible: 800 }[input.difficulty

L29-31 FirstStatement: const start = performance.now(), ci = courses.findIndex((c) => c.id === input.state.course),

L32-32 FirstStatement: const budget = { medium: 65, hard: 140, impossible: 240 }[input.difficulty];

L33-37 FirstStatement: const base = new GolfPhysics(courses[ci],

L38-38 ExpressionStatement: base.restoreSimulation(input.state);

L39-39 FirstStatement: let waypoint = Math.min(input.waypoint, route.length - 1);

L40-50 FirstStatement: const advanceReached = () => { while (waypoint < route.length - 1) {

L53-53 ExpressionStatement: advanceReached();

L54-54 ForStatement: for (let i = 0; i < input.delay * 240; i++) base.tick(1 / 240);

L55-55 ExpressionStatement: advanceReached();

L56-58 FirstStatement: const [tx, ty, gx = tx, gy = ty] = route[waypoint], angle = Math.atan2(ty - base.ball.y, tx - base.ball.x),

L59-60 FirstStatement: const origin = { x: base.ball.x, y: base.ball.y }, snapshot = base.exportState();

L61-63 FirstStatement: let best = Infinity, candidates = 0,

L64-68 FirstStatement: const trial = new GolfPhysics(courses[ci],

L69-69 FirstStatement: const tolerance = { medium: 8, hard: 3, impossible: 1.5 }[input.difficulty];

L70-76 FirstStatement: let alternatives: { vx: number;

L77-129 FirstStatement: const evaluate = (a: number, p: number, wait: number) => { trial.restoreSimulation(snapshot);

L130-130 FirstStatement: const estimate = Math.max(45, Math.min(650, Math.sqrt(distance * 220)));

L131-135 FirstStatement: const powers = [Math.max(45, Math.min(650, estimate + (style.tempo - 1) * 4)),

L136-155 LabeledStatement: outer: for (const wait of [0, 0.6, 1.2, 2, 3, 4, 5, 6]) for (const da of [

L157-159 FirstStatement: const center = { ...shot }, pa = Math.atan2(center.vy, center.vx),

L160-169 LabeledStatement: refine: for (const da of [0, -0.018, 0.018, -0.009, 0.009]) for (const dp of [0, -12, 12, -6, 6, -3, 3]) {

L172-174 FirstStatement: const fine = { ...shot }, fineAngle = Math.atan2(fine.vy, fine.vx),

L175-188 LabeledStatement: polish: for (const da of [0, -0.002, 0.002]) for (const dp of [-1, 1, -0.5, 0.5]) {

L191-193 FirstStatement: const comparable = alternatives .filter((v) => v.cost <= best + tolerance)

L194-197 IfStatement: if (comparable.length) { const v = comparable[0];

L198-204 ReturnStatement: return { ...shot,

planShot.advanceReached [L40-50]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: advanceReached = () =>

Decision branches: L43: Math.hypot(base.ball.x - (t[2] ?? t[0]), base.ball.y - (t[3] ?? t[1])) > 25

planShot.evaluate [L77-129]

Score a board/candidate state for the surrounding AI search.

Signature: evaluate = (a: number, p: number, wait: number) =>

Direct writes: trial.recoveries, trial.impacts, cost, best, shot, alternatives, alternatives.length

Calls / delegates: trial.restoreSimulation, trial.tick, trial.shoot, style.rank, alternatives.filter, alternatives.push, alternatives.sort

Decision branches: L84: !trial.shoot(vx, vy); L87: trial.stopped || trial.status !== "playing"; L99: ci === 3; L100: Math.hypot(tx - 1810, ty - 200) < 60 && !trial.dynamics.plates[1]; L102: Math.hypot(tx - 1810, ty - 950) < 60 && !trial.dynamics.plates[2]; L105: cost < best; L109: cost <= best + tolerance

Execution order (direct statements; nested callbacks stay within their owner):

L78-78 ExpressionStatement: trial.restoreSimulation(snapshot);

L79-79 ExpressionStatement: trial.recoveries = 0;

L80-80 ExpressionStatement: trial.impacts = 0;

L81-81 ForStatement: for (let i = 0; i < wait * 240; i++) trial.tick(1 / 240);

L82-83 FirstStatement: const vx = Math.cos(a) * p, vy = Math.sin(a) * p;

L84-84 IfStatement: if (!trial.shoot(vx, vy)) return;

L85-88 ForStatement: for (let i = 0; i < 240 * 10; i++) { trial.tick(1 / 240);

L89-89 ExpressionStatement: candidates++;

L90-97 FirstStatement: let cost = trial.status === "won"

L99-104 IfStatement: if (ci === 3) { if (Math.hypot(tx - 1810, ty - 200) < 60 && !trial.dynamics.plates[1])

L105-108 IfStatement: if (cost < best) { best = cost;

L109-128 IfStatement: if (cost <= best + tolerance) { const rank = style.rank(

src/games/mini-golf/ultimate/ai/planner.worker.ts

ACTIVE SOURCE / tool or application | 4 lines | SHA-256 24100033f088e5ba...

Worker message entry for Golf. Restores/copies request state, runs the bounded local search and posts its result; cancellation is owned by WorkerTask/controller.

Imports: ./ShotPlanner

Top-level block L2-3

```
self.onmessage = (event: MessageEvent<PlanInput>) =>
  self.postMessage(planShot(event.data));
```

src/games/mini-golf/ultimate/ai/routes.ts

ACTIVE SOURCE / tool or application | 241 lines | SHA-256 145290f7ba24d0e6...

Provides course navigation waypoints and seeded comparable route selection, not scripted shot velocities.

Imports: ../../../../shared/contracts/AIController, ../../../../shared/ai/MatchRandom

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
/** Navigation waypoints only; every shot is planned against the live physics state. */
import type { Difficulty } from "../../../../shared/contracts/AIController";
import { mixSeed } from "../../../../shared/ai/MatchRandom";

const lane = (1920 - 196) / 6,
  c = (i: number) => 28 + i * (lane + 28) + lane / 2;

/**
 * Three distinct strategic routes per course:
 * - Route 0: Balanced / Central route (standard fair line)
 * - Route 1: Aggressive / Inside cut route (faster turns, direct angles)
 * - Route 2: Conservative / Flank route (wider arcs, safe clearance around movers/hazards)
 */
export const courseRoutes: number[][][][] = [
  // Course 0: Foundry
  [
    // Route 0: Central slalom
    [
      [c(0), 220],
      [c(1) + 110, 220],
      [c(1) + 110, 670],
      [c(1) - 100, 670],
      [c(1) - 100, 950],
      [680, 950],
      [680, 220],
      [1235, 220],
      [1235, 950],
      [c(4), 950],
      [c(4), 220],
      [c(5) - 120, 220],
      [c(5) - 120, 942],
      [c(5), 942],
    ],
    // Route 1: Left-flank & inside cuts
    [
      [c(0) - 30, 220],
      [c(1) + 85, 220],
      [c(1) + 85, 650],
      [c(1) - 85, 650],
      [c(1) - 85, 950],
      [730, 950],
      [730, 220],
      [1170, 220],
      [1170, 950],
      [c(4) + 60, 950],
      [c(4) + 60, 220],
      [c(5) - 100, 220],
      [c(5) - 100, 942],
      [c(5), 942],
    ],
    // Route 2: Wide-clearance outer path
    [
      [c(0) + 35, 220],
      [c(1) + 120, 220],
      [c(1) + 120, 680],
      [c(1) - 110, 680],
      [c(1) - 110, 950],
      [650, 950],
      [650, 220],
      [1245, 220],
      [1245, 950],
      [c(4) - 50, 950],
      [c(4) - 50, 220],
      [c(5) - 125, 220],
      [c(5) - 125, 942],
      [c(5), 942],
    ],
  ],
  // Course 1: Shoreline
  [
```

```
71 |    // Route 0: Standard South loop
72 |    [
73 |      [220, 990],
74 |      [220, 220],
75 |      [520, 220],
76 |      [520, 970],
77 |      [910, 970],
78 |      [910, 220],
79 |      [910, 150],
80 |      [1340, 150],
81 |      [1340, 930],
82 |      [1760, 930],
83 |      [1760, 220],
84 |      [1560, 200],
85 |    ],
86 |    // Route 1: Corner-cutting inside line
87 |    [
88 |      [220, 990],
89 |      [220, 220],
90 |      [520, 220],
91 |      [520, 970],
92 |      [910, 970],
93 |      [910, 480],
94 |      [910, 150],
95 |      [1340, 150],
96 |      [1340, 930],
97 |      [1760, 930],
98 |      [1760, 380],
99 |      [1560, 200],
100 |    ],
101 |    // Route 2: Safe paced circuit
102 |    [
103 |      [180, 990],
104 |      [180, 550],
105 |      [180, 220],
106 |      [520, 220],
107 |      [520, 650],
108 |      [520, 970],
109 |      [910, 970],
110 |      [910, 600],
111 |      [910, 220],
112 |      [910, 150],
113 |      [1340, 150],
114 |      [1340, 600],
115 |      [1340, 930],
116 |      [1760, 930],
117 |      [1760, 220],
118 |      [1560, 200],
119 |    ],
120 |  ],
121 |  // Course 2: Event Horizon
122 |  [
123 |    // Route 0: Portal 0 & 1 standard
124 |    [
125 |      [1100, 990],
126 |      [1810, 990],
127 |      [1840, 700, 1265, 340],
128 |      [290, 340],
129 |      [290, 590],
130 |      [700, 600, 1205, 600],
131 |      [1400, 575],
132 |    ],
133 |    // Route 1: Portal 0 early entry & inside center cut
134 |    [
135 |      [800, 990],
136 |      [1810, 990],
137 |      [1840, 700, 1265, 340],
138 |      [450, 340],
139 |      [290, 340],
140 |      [290, 590],
141 |      [700, 600, 1205, 600],
142 |      [1400, 575],
143 |    ],
```

```
144 |    // Route 2: Full runway momentum
145 |    [
146 |      [600, 990],
147 |      [1350, 990],
148 |      [1810, 990],
149 |      [1840, 700, 1265, 340],
150 |      [290, 340],
151 |      [290, 590],
152 |      [700, 600, 1205, 600],
153 |      [1400, 575],
154 |    ],
155 |  ],
156 |  // Course 3: Lost Temple  -  resting points clear bumpers, the mover sweep and quicksand.
157 |  [
158 |    // Route 0: South Bridge, Plate 1 then Plate 2
159 |    [
160 |      [520, 980],
161 |      [790, 960],
162 |      [1100, 900],
163 |      [1080, 570],
164 |      [1000, 570],
165 |      [1330, 570],
166 |      [1320, 290],
167 |      [1420, 220],
168 |      [1810, 200],
169 |      [1810, 950],
170 |      [1770, 660],
171 |      [1770, 570],
172 |      [1560, 575],
173 |    ],
174 |    // Route 1: South Bridge, Plate 2 then Plate 1 (Reversed outer plates)
175 |    [
176 |      [520, 980],
177 |      [790, 960],
178 |      [1100, 900],
179 |      [1080, 570],
180 |      [1000, 570],
181 |      [1330, 570],
182 |      [1330, 850],
183 |      [1810, 950],
184 |      [1810, 200],
185 |      [1770, 480],
186 |      [1770, 570],
187 |      [1560, 575],
188 |    ],
189 |    // Route 2: South Bridge with centered approach
190 |    [
191 |      [480, 980],
192 |      [790, 960],
193 |      [1080, 900],
194 |      [1080, 570],
195 |      [1000, 570],
196 |      [1330, 570],
197 |      [1320, 290],
198 |      [1420, 220],
199 |      [1810, 200],
200 |      [1810, 950],
201 |      [1770, 660],
202 |      [1770, 570],
203 |      [1560, 575],
204 |    ],
205 |  ],
206 | ];
207 |
208 | /** Default routes for backward compatibility. */
209 | export const routes: number[][][] = courseRoutes.map((variants) => variants[0]);
210 |
211 | /**
212 |  * Returns a deterministic route for the course given the difficulty, seed, and player style.
213 |  * Provides distinct paths across matches and difficulty levels.
214 |  */
215 | export function getRoute(
216 |  courseIndex: number,
```

```
217 |  difficulty: Difficulty,
218 |  seed = 0,
219 |  role = 0,
220 | ): number[][] {
221 |  const variants = courseRoutes[courseIndex];
222 |  if (!variants || variants.length === 0) return routes[courseIndex] ?? [];
223 |  // Mix high and low bits before selection: correlated seed/role sequences
224 |  // otherwise cancel in the low bits used by the four-entry Medium table.
225 |  const hash = mixSeed(seed, role + courseIndex * 7);
226 |  let variantIndex = 0;
227 |  if (difficulty === "medium") {
228 |    // Medium varies between safe lines (0 and 2) and alternate line (1)
229 |    const options = [0, 2, 0, 1];
230 |    variantIndex = options[hash % options.length];
231 |  } else if (difficulty === "hard") {
232 |    // Hard evenly uses all three variations
233 |    variantIndex = hash % variants.length;
234 |  } else {
235 |    // Impossible chooses among all variants, prioritizing direct lines
236 |    const options = [1, 0, 2];
237 |    variantIndex = options[hash % options.length];
238 |  }
239 |  return variants[variantIndex % variants.length];
240 | }
```

Module constants and data

L5-5 lane = (1920 - 196) / 6

L14-206 courseRoutes = Structured literal/expression (3769 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

L209-209 routes = courseRoutes.map((variants) => variants[0])

c [L6-6]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: c = (i: number) =>

```
c = (i: number) => 28 + i * (lane + 28) + lane / 2
```

getRoute [L215-240]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function getRoute(courseIndex: number, difficulty: Difficulty, seed = 0, role = 0,): number[][]

Direct writes: variantIndex

Calls / delegates: mixSeed

Return at L222: routes[courseIndex] ?? []

Return at L239: variants[variantIndex % variants.length]

Decision branches: L222: !variants || variants.length === 0; L227: difficulty === "medium"; L231: difficulty === "hard"

src/games/mini-golf/ultimate/components

src/games/mini-golf/ultimate/components/CourseArt.ts

ACTIVE SOURCE / tool or application | 80 lines | SHA-256 5a253c4b982a5b5a...

Renders a static course overview SVG from the actual course data without loading physics/AI.

Imports: ../courses

courseArt [L3-79]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function courseArt(c: Course)

Calls / delegates: n.toString(16).padStart, n.toString, c.ground.map((r) => rect(r, hex(c.color))).join, c.ground.map, rect, hex, c.surfaces .map((s) => rect(s, ({ slick: "#a6dce8", mud: "#ae8468", quicksand: "#d9b47e", sand: "#e9cf96", belt: "#a3ada4", wind: "#c3d4ed", ford: "#a8d2bc", } as const)[s.kind],),) .join, c.surfaces .map, [...(c.bridges ?? []), ...(c.planks ?? []).flat()] .map((r) => rect(r, "#e1be85", "#80694d")) .join, [...(c.bridges ?? []), ...(c.planks ?? []).flat()] .map, (c.planks ?? []).flat, c.fields .map((f) => `<circle cx="${f.x}" cy="${f.y}" r="${f.r}" fill="#9d8bc0" opacity=".25" stroke="#74658d" stroke-width="6"/>`,) .join, c.fields .map, c.circles .map((v) => `<circle cx="${v.x}" cy="${v.y}" r="${v.r}" fill="#edbd75" stroke="#242b32" stroke-width="7"/>`,) .join, c.circles .map, c.movers .map((m) => m.r ? `<circle cx="${(m.from.x + m.to.x) / 2}" cy="${(m.from.y + m.to.y) / 2}" r="${m.r}" fill="#a1a99e" stroke="#242b32" stroke-width="7"/>` : rect({ x: (m.from.x + m.to.x) / 2, y: (m.from.y + m.to.y) / 2, w: m.w ?? 10, h: m.h ?? 10, }, "#899a97", "#242b32",),) .join, c.movers .map,

c.walls.map((r) => rect(r, "#82977b", "#242b32")).join, c.walls.map, c.fans .map((f) => `<g transform="translate(${f.x} ${f.y}) rotate(35)"><path d="M${-f.length / 2} 0h${f.length}M0 ${-f.length / 2}v${f.length}" stroke="#242b32" stroke-width="22"/><circle r="${f.hub}" fill="#f49b58"/></g>`,) .join, c.fans .map, (c.portals ?? []) .map((p) => `<circle cx="${p.from.x}" cy="${p.from.y}" r="25" fill="#9b7fc4" stroke="#242b32" stroke-width="8"/><circle cx="${p.to.x}" cy="${p.to.y}" r="25" fill="#58addb" stroke="#242b32" stroke-width="8"/>`,) .join, (c.portals ?? []) .map, (c.plates ?? []) .map((p) => `<circle cx="${p.x}" cy="${p.y}" r="23" fill="#f4b558" stroke="#242b32" stroke-width="6"/>`,) .join, (c.plates ?? []) .map, c.lasers.map((r) => rect(r, "#df8b7e")).join, c.lasers.map, (c.doors ?? []).map((r) => rect(r, "#d3b571", "#242b32")).join, (c.doors ?? []).map

Return at L78: `<svg class="golf-course-preview" viewBox="0 0 1010 1920" aria-hidden="true"><g transform="translate(1080 0) rotate(90)">${rect({ x: 0, y: 70, w: 1920, h: 1010 }, c.ground ? "#465164" : hex(c.color))}${ground}${c.river ? rect(c.river, "#69afd0") : ""}${surfaces}${bridges}${fields}${portals}${walls}${circles}${movers}${fans}${c.lasers.map((r) => rec ... [continued in source reference]

Execution order (direct statements; nested callbacks stay within their owner):

L4-4 FirstStatement: const hex = (n: number) => `#${n.toString(16).padStart(6, "0")}`;

L5-6 FirstStatement: const rect = (r: Rect, fill: string, stroke = "", rx = 0) => `<rect x="${r.x}" y="${r.y}" width="${r.w}" height="${r.h}" rx="${rx}" fill="${fill}"${stroke ? ` stroke="${stroke}" stroke-width="8"` : ""

L7-9 FirstStatement: const ground = c.ground ? c.ground.map((r) => rect(r, hex(c.color))).join("")

L10-27 FirstStatement: const surfaces = c.surfaces .map((s) =>

L28-30 FirstStatement: const bridges = [...(c.bridges ?? []), ...(c.planks ?? []).flat()] .map((r) => rect(r, "#e1be85", "#80694d"))

L31-36 FirstStatement: const fields = c.fields .map(

L37-42 FirstStatement: const circles = c.circles .map(

L43-58 FirstStatement: const movers = c.movers .map((m) =>

L59-59 FirstStatement: const walls = c.walls.map((r) => rect(r, "#82977b", "#242b32")).join("");

L60-65 FirstStatement: const fans = c.fans .map(

L66-71 FirstStatement: const portals = (c.portals ?? []) .map(

L72-77 FirstStatement: const plates = (c.plates ?? []) .map(

L78-78 ReturnStatement: return `<svg class="golf-course-preview" viewBox="0 0 1010 1920" aria-hidden="true"><g transform="translate(1080 0) rotate(90)">${rect({ x: 0, y: 70, w: 1920, h: 1010 }, c.ground ? "#465164" : hex(c.c

courseArt.hex [L4-4]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: hex = (n: number) =>

```
hex = (n: number) => `#${n.toString(16).padStart(6, "0")}`
```

courseArt.rect [L5-6]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: rect = (r: Rect, fill: string, stroke = "", rx = 0) =>

```
rect = (r: Rect, fill: string, stroke = "", rx = 0) => `<rect x="${r.x}" y="${r.y}" width="${r.w}" height="${r.h}" rx="${rx}" fill="${fill}"${stroke ? ` stroke="${stroke}" stroke-width="8"` : ""}/>`
```

src/games/mini-golf/ultimate/components/GolfUI.ts

ACTIVE SOURCE / tool or application | 96 lines | SHA-256 6e356bb697ec9560...

Owns responsive DOM controls, screen/HUD/help/rival/power states and available course bounds.

Imports: ./golf.css, ../courses

State, resources and initial values

Line	Field / declaration
4	readonly root = document.createElement("div");
5	readonly field: HTMLDivElement;
6	private heading: HTMLElement;
7	private stat: HTMLElement;
8	private footer: HTMLElement;
9	private toast: HTMLElement;
10	private power: HTMLElement;
11	private opponent: HTMLElement;

GolfUI [L3-95]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

GolfUI.constructor [L12-33]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(action: (name: string) => void)

Direct writes: this.root.className, this.root.innerHTML, this.field, this.heading, this.stat, this.footer, this.toast, this.toast.className, this.power, this.power.className, this.power.innerHTML, this.opponent, this.opponent.className

Calls / delegates: this.root.querySelector, document.createElement, this.root.addEventListener, (e.target as HTMLElement).closest, action, document.querySelector("#game-stage")!.append, document.querySelector

Decision branches: L30: button && !button.disabled

GolfUI.title [L34-38]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: title(title: string, stat = "")

Direct writes: this.heading.textContent, this.stat.textContent, this.stat.hidden

Decision branches: L35: this.heading.textContent !== title; L36: this.stat.textContent !== stat

GolfUI.screen [L39-42]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: screen(html: string)

Direct writes: this.field.innerHTML, this.footer.innerHTML

GolfUI.playing [L43-47]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: playing(competitive = false)

Direct writes: this.toast.hidden, this.power.hidden, this.opponent.hidden, this.footer.innerHTML

Calls / delegates: this.field.replaceChildren

GolfUI.focus [L48-54]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: focus(active: boolean)

Direct writes: b.textContent

Calls / delegates: this.footer.querySelector, b.setAttribute

Decision branches: L50: b

GolfUI.message [L55-58]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: message(text: string)

Direct writes: this.toast.textContent, this.toast.hidden

Decision branches: L56: this.toast.textContent !== text

GolfUI.meter [L59-64]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: meter(power: number | null)

Direct writes: this.power.hidden, (this.power.firstElementChild as HTMLElement).style.width

Decision branches: L61: power !== null

GolfUI.rival [L65-68]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: rival(text: string)

Direct writes: this.opponent.textContent, this.opponent.hidden

Decision branches: L66: this.opponent.textContent !== text

GolfUI.help [L69-74]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: help(text: string, closeLabel = "Back to play")

Direct writes: panel.className, panel.innerHTML

Calls / delegates: document.createElement, this.field.append

GolfUI.closeHelp [L75-77]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: closeHelp()

Calls / delegates: this.field.querySelector(".help")?.remove, this.field.querySelector

GolfUI.bounds [L78-87]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: bounds(): Rect

Calls / delegates: this.field.getBoundingClientRect, this.root.parentElement!.getBoundingClientRect

Return at L81: { x: r.x - parent.x + 2, y: r.y - parent.y + 2, w: Math.max(1, r.width - 4), h: Math.max(1, r.height - 4), }

GolfUI.pause [L88-91]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause(paused: boolean)

Direct writes: this.root.inert

Calls / delegates: this.root.setAttribute

GolfUI.dispose [L92-94]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.root.remove

src/games/mini-golf/ultimate/components/ModeArt.ts

ACTIVE SOURCE / tool or application | 6 lines | SHA-256 bb17defbbdf97cb9...

Creates small original SVG illustrations for golf mode choices.

modeArt [L1-5]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function modeArt(mode: "ai" | "casual" | "local")

Calls / delegates: ball

Return at L4: `<svg viewBox="0 0 110 90" aria-hidden="true"><rect x="4" y="6" width="102" height="78" rx="22" fill="#d9e9bc" stroke="#242b32" stroke-width="4"/><path d="M22 60 Q55 73 77 29" fill="none" stroke="#fffdf5" stroke-width="5" stroke-dasharray="5 5"/><ellipse cx="81" cy="34" rx="10" ry="5" fill="#242b32"/><path d="M81 34 V11 L99 17 81 23" fill="#f2a360" ... [continued in source reference]

modeArt.ball [L2-3]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: ball = (x: number, y: number, color: string) =>

```
ball = (x: number, y: number, color: string) => `<circle cx="${x}" cy="${y}" r="15" fill="${color}" stroke="#242b32" stroke-width="4"/><circle cx="${x - 4}" cy="${y - 4}" r="3" fill="#fffdf5"/>`
```

src/games/mini-golf/ultimate/components/golf.css

ACTIVE SOURCE / tool or application | 310 lines | SHA-256 06af53d66fb3807b...

Presentation selectors and responsive layout rules; detailed selector/property blocks appear below.

Lines	Selector or rule	Declarations
1-12	.golf-ui	position: absolute; inset: 0; display: flex; flex-direction: column; pointer-events: none; color: var(--ink); font-family: "Nunito Sans", sans-serif; gap: 12px; padding: 8px 14px 12px; box-sizing: border-box;
12-15	.golf-ui *	box-sizing: border-box;

Lines	Selector or rule	Declarations
15-27	.golf-ui button	pointer-events: auto; min-height: 48px; border: 2px solid var(--ink); border-radius: 15px; background: #fffdf5; font-family: "Fredoka", sans-serif; font-weight: 600; font-size: 16px; padding: 8px 14px; touch-action: manipulation;
27-31	.golf-ui button:disabled	opacity: 0.5; cursor: default;
31-35	.golf-ui button:focus-visible	outline: 3px solid #58addb; outline-offset: 2px;
35-42	.golf-header	display: flex; align-items: center; gap: 10px; min-height: 48px; flex-shrink: 0;
42-47	.golf-heading	flex: 1; min-width: 0; line-height: 1.15;
47-54	.golf-heading strong	font: 600 clamp(17px, 3vw, 22px) "Fredoka"; display: block; overflow: hidden; text-overflow: ellipsis; white-space: nowrap;
54-60	.golf-heading small	display: block; font-size: 13px; margin-top: 3px; font-weight: 700;
60-65	.golf-icon-button	width: 48px; padding: 6px !important; flex-shrink: 0;
65-70	.golf-field	flex: 1; min-height: 0; position: relative;
70-77	.golf-footer	display: flex; justify-content: center; gap: 10px; flex-shrink: 0; padding-top: 2px;
77-81	.golf-footer button	flex: 1; max-width: 220px;
81-95	.golf-toast	position: absolute; bottom: 12px; left: 50%; transform: translateX(-50%); background: #fffdf5ee; border: 2px solid var(--ink); border-radius: 12px; padding: 8px 12px; font-size: 13px; text-align: center; max-width: 90%; pointer-events: none;
95-106	.golf-power	position: absolute; bottom: 8px; left: 25%; width: 50%; height: 8px; border: 1px solid var(--ink); background: #fffdf5; border-radius: 8px; overflow: hidden;
106-111	.golf-power i	display: block; height: 100%; background: var(--orange);
111-125	.golf-panel	position: absolute; inset: 0; overflow: auto; overscroll-behavior: contain; pointer-events: auto; background: var(--paper); padding: clamp(14px, 3vw, 28px); border-radius: 18px; display: flex; flex-direction: column; justify-content: safe center; gap: 16px;
125-130	.golf-panel h2	font: 600 clamp(22px, 4vw, 32px) "Fredoka"; margin: 0; text-align: center;
130-136	.golf-panel p	margin: 0; font-size: 15px; text-align: center; line-height: 1.5;
136-143	.golf-modes	display: grid; gap: 14px; width: 100%; max-width: 960px; margin: 0 auto;
143-152	.golf-mode	display: flex; align-items: center; text-align: left; gap: 16px; padding: 18px !important; min-height: 110px !important; box-shadow: 0 4px 0 #242b32;
152-156	.golf-mode svg	width: 76px; flex-shrink: 0;
156-160	.golf-mode strong	font-size: 21px; display: block;

Lines	Selector or rule	Declarations
160-166	.golf-mode small	font: 700 13px "Nunito Sans"; display: block; margin-top: 5px; line-height: 1.4;
166-172	.golf-choice	display: flex; gap: 8px; justify-content: center; flex-wrap: wrap;
172-175	.golf-choice button[aria-pressed="true"]	background: var(--orange);
175-183	.golf-courses	display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 12px; width: 100%; max-width: 760px; margin: 0 auto;
183-188	.golf-course	min-height: 86px !important; text-align: left; font-size: 19px !important;
188-193	.golf-course small	display: block; font: 700 12px "Nunito Sans"; margin-top: 6px;
193-196	.golf-primary	background: var(--orange) !important;
196-202	.golf-actions	display: flex; gap: 10px; justify-content: center; flex-wrap: wrap;
202-205	.golf-actions button	min-width: 140px;
205-213	.golf-result-table	border-collapse: separate; border-spacing: 0; max-width: 560px; width: 100%; margin: auto; font-size: 16px;
213-219	.golf-result-table th, .golf-result-table td	padding: 12px 8px; border-bottom: 1px solid #242b3244; text-align: center;
219-222	.golf-ui[aria-hidden="true"]	visibility: hidden;
222-228	.golf-ui .golf-panel.help	inset: 8% max(0px, calc((100% - 520px) / 2)); border: 3px solid var(--ink); box-shadow: 0 8px 0 #242b3222; z-index: 3;
228-240	.golf-opponent	position: absolute; top: 8px; right: 8px; padding: 7px 10px; background: #c8e7ebee; border: 2px solid var(--ink); border-radius: 12px; font-size: 12px; font-weight: 700; pointer-events: none;
241-244	.golf-modes	grid-template-columns: repeat(3, 1fr);
244-249	.golf-mode	flex-direction: column; text-align: center; justify-content: center;
249-252	.golf-mode svg	width: 100px;
252-255	.golf-panel	gap: 20px;
255-260	.golf-header	max-width: 1200px; width: 100%; margin: auto;
262-266	.golf-ui	padding: 5px 12px 8px; gap: 8px;

Lines	Selector or rule	Declarations
266-270	.golf-panel	padding: 10px; gap: 10px;
270-275	.golf-mode	min-height: 130px !important; padding: 10px !important; gap: 6px;
275-278	.golf-mode svg	width: 60px;
278-281	.golf-mode strong	font-size: 18px;
281-284	.golf-mode small	font-size: 12px;
284-287	.golf-course	min-height: 58px !important;
287-290	.golf-courses	grid-template-columns: repeat(4, minmax(0, 1fr));
290-293	.golf-heading small	font-size: 12px;
295-298	.golf-mode	transition: transform 0.12s ease;
298-301	.golf-mode:hover	transform: translateY(-2px);
302-304	.golf-course	display: flex; flex-direction: column; align-items: center; gap: 7px;
304-305	.golf-course strong	font-weight: 600; text-align: center; overflow-wrap: anywhere; max-width: 100%;
305-306	.golf-course small	margin-top: 0; text-align: center;
306-307	.golf-course-preview	display: block; width: 100%; height: 155px; border-radius: 9px;
309-309	.golf-course-preview	height: 85px;

src/games/mini-golf/ultimate/courses

src/games/mini-golf/ultimate/courses/foundry.ts

ACTIVE SOURCE / tool or application | 67 lines | SHA-256 1777260f21723ab8...

Active Ultimate Golf foundry course data in original 1920x1080 units. Geometry, hazards, dynamics parameters, tee/cup and base stroke limit are defined here.

Imports: ./types

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
1 | import { p, r, outer, type Course } from "./types";
2 | const lane = (1920 - 196) / 6,
3 |   x = (i: number) => 28 + i * (lane + 28),
```

```
  4 |  c = (i: number) => x(i) + lane / 2;
  5 | export const foundry: Course = {
  6 |  id: "foundry",
  7 |  name: "Foundry",
  8 |  subtitle: "Belts, beams & molten trouble",
  9 |  tip: "Follow the six winding lanes. Belts push, oil slides, and the red gates pulse.",
 10 |  baseLimit: 18,
 11 |  color: 0xf1c879,
 12 |  start: p(c(0), 982),
 13 |  hole: p(c(5), 942),
 14 |  walls: [
 15 |    ...outer(),
 16 |    ...Array.from({ length: 5 }, (_, i) =>
 17 |      r(x(i) + lane, i % 2 === 0 ? 328 : 98, 28, 724),
 18 |    ),
 19 |    r(c(0) - 68, 819, 46, 46),
 20 |    r(x(1), 598, 150, 18),
 21 |    r(x(1) + lane - 150, 738, 150, 18),
 22 |  ],
 23 |  circles: [
 24 |    { ...p(c(0) + 55, 692), r: 26 },
 25 |    { ...p(c(5) - 75, 862), r: 26 },
 26 |    { ...p(c(5) + 75, 862), r: 26 },
 27 |  ],
 28 |  movers: [
 29 |    {
 30 |      from: p(x(2) + 42, 722),
 31 |      to: p(x(2) + lane - 152, 722),
 32 |      seconds: (lane - 194) / 110,
 33 |      w: 110,
 34 |      h: 18,
 35 |    },
 36 |    {
 37 |      from: p(x(5) + 42, 398),
 38 |      to: p(x(5) + lane - 152, 398),
 39 |      seconds: (lane - 194) / 110,
 40 |      w: 110,
 41 |      h: 18,
 42 |    },
 43 |    {
 44 |      from: p(x(5) + lane - 152, 538),
 45 |      to: p(x(5) + 42, 538),
 46 |      seconds: (lane - 194) / 110,
 47 |      w: 110,
 48 |      h: 18,
 49 |    },
 50 |  ],
 51 |  surfaces: [
 52 |    { ...r(x(0), 428, lane, 64), kind: "belt", push: p(0, 750) },
 53 |    { ...r(x(3), 348, lane, 64), kind: "belt", push: p(0, -750) },
 54 |  ],
 55 |  fields: [{ ...p(c(3), 818), r: 170, strength: 450, core: 22 }],
 56 |  fans: [{ ...p(c(2), 478), length: 190, hub: 16, spin: 120 }],
 57 |  lasers: [{ ...r(x(4), 354, lane, 8), phase: 0 }],
 58 |  lava: [
 59 |    { ...p(c(4) - 55, 722), r: 38 },
 60 |    { ...p(c(4) + 55, 498), r: 38 },
 61 |    { ...p(c(5), 752), r: 38 },
 62 |  ],
 63 |  diamond: { ...p(c(1), 428), r: 34 },
 64 |  piston: { ...r(x(3), 533, 30, 70), extension: lane - 72 },
 65 | };
 66 | export const oil = { ...p(c(1), 882), r: 70 };
```

Module constants and data

L2-2 lane = (1920 - 196) / 6

L5-65 foundry = Structured literal/expression (1619 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

L66-66 oil = { ...p(c(1), 882), r: 70 }

x [L3-3]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: x = (i: number) =>

```
x = (i: number) => 28 + i * (lane + 28)
```

c [L4-4]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: c = (i: number) =>

```
c = (i: number) => x(i) + lane / 2
```

src/games/mini-golf/ultimate/courses/horizon.ts

ACTIVE SOURCE / tool or application | 88 lines | SHA-256 e639ae1b806d9e90...

Active Ultimate Golf horizon course data in original 1920x1080 units. Geometry, hazards, dynamics parameters, tee/cup and base stroke limit are defined here.

Imports: ./types

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
  1 | import { p, r, type Course } from "./types";
  2 | export const horizon: Course = {
  3 |  id: "horizon",
  4 |  ufo: {
  5 |    first: 11.5,
  6 |    interval: 28,
  7 |    warning: 1.5,
  8 |    speed: 170,
  9 |    duration: 13,
 10 |   hover: 1,
 11 |    carry: 0.8,
 12 |    cooldown: 8,
 13 |  },
 14 |  name: "Event Horizon",
 15 |  subtitle: "A little putt. A giant leap.",
 16 |  tip: "Stay on the platforms. Paired portals are shortcuts; the UFO can relocate a resting ball.",
 17 |  baseLimit: 20,
 18 |  color: 0xc7b5e4,
 19 |  start: p(140, 990),
 20 |  hole: p(1400, 575),
 21 |  walls: [],
 22 |  ground: [
 23 |    r(60, 940, 1800, 100),
 24 |    r(1760, 110, 100, 930),
 25 |    r(60, 110, 1800, 100),
 26 |    r(60, 110, 100, 750),
 27 |    r(60, 760, 1620, 100),
 28 |    r(1580, 290, 100, 570),
 29 |    r(240, 290, 1440, 100),
 30 |    r(240, 290, 100, 390),
 31 |    r(240, 470, 560, 210),
 32 |    r(800, 470, 320, 60),
 33 |    r(1120, 470, 380, 210),
 34 |  ],
 35 |  surfaces: [
 36 |    { ...r(500, 940, 350, 100), kind: "slick", friction: 25 },
 37 |    { ...r(500, 290, 350, 100), kind: "slick", friction: 25 },
 38 |    { ...r(60, 400, 100, 180), kind: "wind", push: p(-280, 0) },
 39 |  ],
 40 |  circles: [
 41 |    [110, 700],
 42 |    [1630, 420],
 43 |    [1330, 520],
 44 |    [1330, 640],
 45 |  ].map((v) => ({ ...p(v[0], v[1]), r: 20, bounce: 1.3 })),
 46 |  movers: [
 47 |    { from: p(700, 50), to: p(700, 250), seconds: 2.4, r: 30 },
 48 |    { from: p(1250, 250), to: p(1250, 50), seconds: 2, r: 26 },
 49 |    { from: p(1000, 250), to: p(1000, 430), seconds: 1.8, r: 22 },
```

```
50 |     { from: p(420, 430), to: p(420, 720), seconds: 3, r: 30 },
51 |   ],
52 |   fields: [
53 |     {
54 |       ...p(600, 720),
55 |       r: 200,
56 |       strength: 700,
57 |       quadratic: true,
58 |       movingOnly: true,
59 |       core: 28,
60 |     },
61 |     {
62 |       ...p(1450, 250),
63 |       r: 200,
64 |       strength: 700,
65 |       quadratic: true,
66 |       movingOnly: true,
67 |       core: 28,
68 |     },
69 |     {
70 |       ...p(960, 640),
71 |       r: 240,
72 |       strength: 500,
73 |       quadratic: true,
74 |       movingOnly: true,
75 |     },
76 |   ],
77 |   fans: [{ ...p(1200, 990), length: 150, hub: 16, spin: 90 }],
78 |   lasers: [
79 |     { ...r(1760, 450, 100, 8), phase: 0 },
80 |     { ...r(1580, 600, 100, 8), phase: 1.5 },
81 |   ],
82 |   portals: [
83 |     { from: p(1840, 700), to: p(1300, 340), angle: 180 },
84 |     { from: p(700, 600), to: p(1170, 600), angle: 0 },
85 |     { from: p(1300, 810), to: p(960, 310), angle: 90 },
86 |   ],
87 | };
```

Module constants and data

L2-87 horizon = Structured literal/expression (2127 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

src/games/mini-golf/ultimate/courses/index.ts

ACTIVE SOURCE / tool or application | 7 lines | SHA-256 319f19d086afd540...

Exports the ordered active course data and shared course geometry types.

Imports: ./foundry, ./shoreline, ./horizon, ./temple

Module constants and data

L5-5 courses = [foundry, shoreline, horizon, temple]

Top-level block L6-6

```
export type { Course, Point, Rect } from "./types";
```

src/games/mini-golf/ultimate/courses/shoreline.ts

ACTIVE SOURCE / tool or application | 67 lines | SHA-256 9501ae9ef9866e31...

Active Ultimate Golf shoreline course data in original 1920x1080 units. Geometry, hazards, dynamics parameters, tee/cup and base stroke limit are defined here.

Imports: ./types

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
1 | import { p, r, type Course } from "./types";
2 | export const shoreline: Course = {
3 |   id: "shoreline",
```

```
  4 |  name: "Shoreline",
  5 |  subtitle: "Read the tide. Ride the current.",
  6 |  tip: "Pale bridges flood with the tide. Sand slows you; wet sand slides. Water returns your shot.",
  7 |  baseLimit: 25,
  8 |  color: 0xa9dce4,
  9 |  start: p(130, 990),
 10 |  hole: p(1560, 200),
 11 |  walls: [],
 12 |  ground: [
 13 |    r(60, 760, 340, 270),
 14 |    r(140, 330, 170, 440),
 15 |    r(60, 110, 520, 230),
 16 |    r(450, 640, 200, 390),
 17 |    r(640, 920, 330, 110),
 18 |    r(860, 330, 110, 600),
 19 |    r(800, 110, 600, 230),
 20 |    r(1310, 330, 70, 380),
 21 |    r(1100, 700, 760, 330),
 22 |    r(1480, 110, 380, 280),
 23 |  ],
 24 |  surfaces: [
 25 |    ...[
 26 |      [140, 470, 170, 90],
 27 |      [1230, 820, 220, 150],
 28 |      [1590, 230, 130, 110],
 29 |      [860, 360, 110, 140],
 30 |    ].map((v) => ({
 31 |      ...r(v[0], v[1], v[2], v[3]),
 32 |      kind: "sand" as const,
 33 |      friction: 400,
 34 |    })),
 35 |    { ...r(360, 118, 210, 95), kind: "slick", friction: 20 },
 36 |    { ...r(1600, 940, 250, 80), kind: "slick", friction: 20 },
 37 |    { ...r(860, 520, 110, 220), kind: "ford", friction: 250, push: p(260, 0) },
 38 |  ],
 39 |  bridges: [r(470, 330, 100, 320), r(1700, 380, 110, 330)],
 40 |  circles: [
 41 |    ...[
 42 |      [300, 930],
 43 |      [220, 200],
 44 |      [1180, 225],
 45 |      [1790, 300],
 46 |    ].map((v) => ({ ...p(v[0], v[1]), r: 14 })),
 47 |    ...[
 48 |      [330, 800],
 49 |      [1000, 225],
 50 |      [1700, 160],
 51 |      [1545, 330],
 52 |    ].map((v) => ({ ...p(v[0], v[1]), r: 24, bounce: 1.3 })),
 53 |  ],
 54 |  movers: [
 55 |    { from: p(140, 625), to: p(266, 625), seconds: 126 / 110, w: 44, h: 30 },
 56 |    { from: p(1010, 125), to: p(1356, 125), seconds: 346 / 130, w: 44, h: 30 },
 57 |    { from: p(1250, 290), to: p(880, 290), seconds: 370 / 130, w: 44, h: 30 },
 58 |    { from: p(1450, 745), to: p(1816, 745), seconds: 366 / 150, w: 44, h: 30 },
 59 |  ],
 60 |  fields: [
 61 |    { ...p(730, 820), r: 125, strength: 380 },
 62 |    { ...p(1450, 560), r: 130, strength: 380 },
 63 |  ],
 64 |  fans: [],
 65 |  lasers: [],
 66 | };
```

Module constants and data

L2-66 shoreline = Structured literal/expression (1862 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

src/games/mini-golf/ultimate/courses/temple.ts

ACTIVE SOURCE / tool or application | 58 lines | SHA-256 4cf6cded63d5fc59...

Active Ultimate Golf temple course data in original 1920x1080 units. Geometry, hazards, dynamics parameters, tee/cup and base stroke limit are defined here.

Imports: ./types

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
  1 | import { p, r, outer, type Course } from "./types";
  2 | export const temple: Course = {
  3 |  id: "temple",
  4 |  name: "Lost Temple",
  5 |  subtitle: "Ancient paths. Clever little traps.",
  6 |  tip: "Cross a bridge before it crumbles. The first plate opens a gate for 6s. Touch both outer plates to
unlock the cup.",
  7 |  baseLimit: 18,
  8 |  color: 0xb8d998,
  9 |  start: p(120, 980),
 10 |  hole: p(1560, 575),
 11 |  walls: [
 12 |    ...outer(),
 13 |    r(1152, 98, 28, 422),
 14 |    r(1152, 620, 28, 432),
 15 |    r(1380, 300, 348, 28),
 16 |    r(1380, 822, 348, 28),
 17 |    r(1380, 300, 28, 550),
 18 |    r(1700, 300, 28, 220),
 19 |    r(1700, 620, 28, 230),
 20 |    r(190, 700, 430, 40),
 21 |    r(28, 400, 430, 40),
 22 |  ],
 23 |  river: r(620, 98, 140, 954),
 24 |  planks: [165, 915].map((y) =>
 25 |    Array.from({ length: 5 }, (_, i) => r(620 + i * 28, y, 28, 90)),
 26 |  ),
 27 |  surfaces: [
 28 |    { ...r(250, 850, 180, 140), kind: "mud", friction: 400 },
 29 |    { ...r(770, 260, 130, 180), kind: "slick", friction: 25 },
 30 |    { ...r(1740, 330, 140, 170), kind: "slick", friction: 25 },
 31 |    { ...r(850, 720, 200, 150), kind: "quicksand", friction: 400 },
 32 |    { ...r(1200, 120, 160, 160), kind: "quicksand", friction: 400 },
 33 |  ],
 34 |  movers: [
 35 |    { from: p(100, 570), to: p(560, 570), seconds: 3, r: 30 },
 36 |    { from: p(1070, 420), to: p(830, 420), seconds: 2.2, r: 28 },
 37 |    { from: p(1280, 380), to: p(1280, 760), seconds: 2.6, r: 30 },
 38 |  ],
 39 |  circles: [
 40 |    [250, 220],
 41 |    [430, 300],
 42 |    [450, 820],
 43 |    [1060, 960],
 44 |    [1480, 470],
 45 |    [1620, 690],
 46 |  ].map((v) => ({ ...p(v[0], v[1]), r: 22, bounce: 1.3 })),
 47 |  fields: [],
 48 |  fans: [{ ...p(1810, 570), length: 80, hub: 12, spin: 100 }],
 49 |  lasers: [
 50 |    r(1180, 330, 200, 8),
 51 |    r(1180, 820, 200, 8),
 52 |    r(1540, 98, 8, 202),
 53 |    r(1540, 850, 8, 202),
 54 |  ].map((v, i) => ({ ...v, phase: i * 0.75 })),
 55 |  plates: [p(1000, 570), p(1810, 200), p(1810, 950)],
 56 |  doors: [r(1152, 520, 28, 100), r(1700, 520, 28, 100)],
 57 | };
```

Module constants and data

L2-57 temple = Structured literal/expression (1787 characters). See the complete source reference; do not treat generated SVG path coordinates as gameplay variables.

src/games/mini-golf/ultimate/courses/types.ts

ACTIVE SOURCE / tool or application | 93 lines | SHA-256 50b4c4c975e005c0...

Typed course geometry/hazard/dynamics schemas plus p/r/outer coordinate constructors.

Point [L1-4]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Point { x: number; y: number; }
```

Rect [L5-8]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Rect extends Point { w: number; h: number; }
```

Circle [L9-12]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Circle extends Point { r: number; bounce?: number; }
```

Mover [L13-20]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Mover { from: Point; to: Point; seconds: number; r?: number; w?: number; h?: number; }
```

Surface [L21-25]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Surface extends Rect { friction?: number; push?: Point; kind: "slick" | "sand" | "belt" | "ford" | "mud" | "quicksand" | "wind"; }
```

Field [L26-31]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Field extends Circle { strength: number; quadratic?: boolean; movingOnly?: boolean; core?: number; }
```

Fan [L32-36]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Fan extends Point { length: number; hub: number; spin: number; }
```

Portal [L37-41]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Portal { from: Point; to: Point; angle: number; }
```

UFOConfig [L42-51]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface UFOConfig { first: number; interval: number; warning: number; speed: number; duration: number; hover: number; carry: number; cooldown: number; }
```

Course [L52-79]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Course { ufo?: UFOConfig; id: string; name: string; subtitle: string; tip: string; baseLimit: number; color: number; start: Point; hole: Point; walls: Rect[]; ground?: Rect[]; circles: Circle[]; movers: Mover[]; surfaces: Surface[]; fields: Field[]; fans: Fan[]; lasers: (Rect & { phase: number })[]; lava?: Circle[]; diamond?: Circle; portals?: Portal[]; bridges?: Rect[]; river?: Rect; planks?: Rect[][]; plates?: Point[]; doors?: Rect[]; piston?: Rect & { extension: number }; }
```

p [L80-80]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: p = (x: number, y: number): Point =>

```
p = (x: number, y: number): Point => ({ x, y })
```

r [L81-86]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: r = (x: number, y: number, w: number, h: number): Rect =>

```
r = (x: number, y: number, w: number, h: number): Rect => ({ x, y, w, h, })
```

outer [L87-92]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: outer = () =>

Calls / delegates: r

src/games/mini-golf/ultimate/modes

src/games/mini-golf/ultimate/modes/AIChallengeMode.ts

ACTIVE SOURCE / tool or application | 117 lines | SHA-256 c92fec6300a48592...

Advances independent human/AI worlds concurrently and determines completion/failure/race outcomes.

Imports: ../GolfPhysics, ../courses, ../ai/GolfAI, ../../../../shared/contracts/AIController, ./CompetitionRules

State, resources and initial values

Line	Field / declaration
14	readonly attempts: [Attempt[], Attempt[]] = [[], []];
15	readonly holes = [0, 0];
16	readonly elapsed = [0, 0];
17	readonly worlds: [GolfPhysics, GolfPhysics];
18	readonly ai: GolfAI;
19	phase: "ready" | "playing" | "complete" = "ready";
20	private failed: [boolean, boolean] = [false, false];

AIChallengeMode [L13-116]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

AIChallengeMode.constructor [L21-29]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly plan: number[], readonly seed: number, difficulty: Difficulty, report: (ms: number) => void,)

Direct writes: this.worlds, this.ai

Calls / delegates: this.create, GolfAI

AIChallengeMode.create [L30-36]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private create(hole: number)

Calls / delegates: GolfPhysics, seedFor

Return at L31: new GolfPhysics(courses[this.plan[hole]], 0, seedFor(this.seed, this.plan[hole]),)

AIChallengeMode.begin [L37-39]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: begin()

Direct writes: this.phase

Decision branches: L38: this.phase === "ready"

AIChallengeMode.update [L40-84]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.elapsed[p], this.worlds[p], this.failed[p], this.ai.accelerated, this.phase

Calls / delegates: this.ai.update, w.advance, this.attempts[p].push, this.create, this.ai.reset, this.ai.cancel

Decision branches: L41: this.phase !== "playing"; L43: this.attempts[p].length === this.plan.length || this.failed[p]; L47: p === 1; L49: w.status !== "playing" || this.elapsed[p] >= COURSE_TIMEOUT; L61: completed && this.holes[p] < this.plan.length - 1; L65: p === 1; L75: done0 && !done1; L78: done0 && done1

Execution order (direct statements; nested callbacks stay within their owner):

L41-41 IfStatement: if (this.phase !== "playing") return;

L42-70 ForOfStatement: for (const p of [0, 1] as const) { if (this.attempts[p].length === this.plan.length || this.failed[p])

L71-72 FirstStatement: const done0 = this.attempts[0].length === this.plan.length || this.failed[0];

L73-74 FirstStatement: const done1 = this.attempts[1].length === this.plan.length || this.failed[1];

L75-77 IfStatement: if (done0 && !done1) { this.ai.accelerated = true;

L78-83 IfStatement: if (done0 && done1) { // Both immutable attempt records now exist; time tolerance belongs to

AIChallengeMode.forfeit [L85-96]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: forfeit()

Direct writes: this.failed[0], this.phase

Calls / delegates: this.attempts[0].push, this.ai.cancel

Decision branches: L86: this.phase !== "playing"

AIChallengeMode.totals [L97-102]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get totals()

Calls / delegates: this.attempts.map, total

Return at L98: this.attempts.map((a) => total(a, this.plan.length)) as [ReturnType<typeof total>, ReturnType<typeof total>,]

AIChallengeMode.winner [L103-108]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get winner()

Calls / delegates: winner

Return at L104: "draw" as const

Return at L105: 2 as const

Return at L106: 1 as const

Return at L107: winner(...this.totals)

Decision branches: L104: this.failed[0] && this.failed[1]; L105: this.failed[0]; L106: this.failed[1]

AIChallengeMode.pause [L109-112]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Calls / delegates: this.ai.cancel, this.worlds.forEach, w.resetAccumulator

AIChallengeMode.dispose [L113-115]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.ai.dispose

src/games/mini-golf/ultimate/modes/CasualMode.ts

ACTIVE SOURCE / tool or application | 23 lines | SHA-256 968bd652d58e3519...

Starts ordinary course challenges and banks casual course records.

Imports: ../courses, ../GolfPhysics, ../GolfProgress, ../GolfRules

State, resources and initial values

Line	Field / declaration
6	readonly progress = new GolfProgress();
7	world?: GolfPhysics;
8	course = 0;
9	difficulty: GolfDifficulty = 1;

CasualMode [L5-22]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

CasualMode.start [L10-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: start(index: number)

Direct writes: this.course, this.world

Calls / delegates: GolfPhysics

Return at L13: this.world

CasualMode.update [L15-17]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Calls / delegates: this.world?.advance

CasualMode.record [L18-21]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: record()

Calls / delegates: this.progress.record

Decision branches: L19: this.world?.status === "won"

src/games/mini-golf/ultimate/modes/CompetitionRules.ts

ACTIVE SOURCE / tool or application | 43 lines | SHA-256 bed91803681d8c79...

Defines timeout/tolerance, attempt totals, completion/time/stroke comparison and deterministic competition seeds.

Module constants and data

L12-12 FINISH_TOLERANCE = 0.1

L13-13 COURSE_TIMEOUT = 180

Attempt [L1-6]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Attempt { time: number; strokes: number; completed: boolean; reason?: "timeout" | "strokes" | "forfeit" | "opponent"; }
```

Totals [L7-11]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Totals { time: number; strokes: number; completed: boolean; }
```

total [L14-20]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function total(attempts: Attempt[], count: number): Totals

Calls / delegates: attempts.reduce, attempts.every

Return at L15: { time: attempts.reduce((s, a) => s + a.time, 0), strokes: attempts.reduce((s, a) => s + a.strokes, 0), completed: attempts.length === count && attempts.every((a) => a.completed), }

winner [L21-28]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function winner(one: Totals, two: Totals): 1 | 2 | "draw"

Return at L22: one.completed ? 1 : 2

Return at L23: "draw"

Return at L24: one.strokes < two.strokes ? 1 : 2

Return at L26: one.time < two.time ? 1 : 2

Return at L27: "draw"

Decision branches: L22: one.completed !== two.completed; L23: !one.completed; L24: one.strokes !== two.strokes; L25: Math.abs(one.time - two.time) > FINISH_TOLERANCE

seedFor [L29-33]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function seedFor(seed: number, index: number)

Direct writes: n

Return at L32: (n ^ (n >>> 15)) >>> 0

seeded [L34-40]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function seeded(seed: number)

Direct writes: state

Return at L36: () => { state = (Math.imul(state, 1664525) + 1013904223) >>> 0; return state / 4294967296; }

Return at L38: state / 4294967296

timeText [L41-42]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: timeText = (seconds: number) =>

Calls / delegates: (seconds % 60).toFixed(1).padStart, (seconds % 60).toFixed

src/games/mini-golf/ultimate/modes/CompetitiveStats.ts

ACTIVE SOURCE / tool or application | 26 lines | SHA-256 0a5caf0fda1d9ec1...

Validates and persists separate golf competition win/draw counters.

Module constants and data

L1-1 KEY = "pocket-rivals.golf-versus.v1"

recordCompetition [L2-25]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function recordCompetition(mode: "local" | "ai", result: 1 | 2 | "draw",)

Direct writes: data[key]

Calls / delegates: localStorage.getItem, localStorage.setItem

Decision branches: L17: Number.isSafeInteger(n) && n >= 0

src/games/mini-golf/ultimate/modes/LocalVersusMode.ts

ACTIVE SOURCE / tool or application | 82 lines | SHA-256 bea8949930bd2316...

Owns seeded hot-seat handoff, timed attempts, course/tour progression and aggregate outcome.

Imports: ../courses, ../GolfPhysics, ./CompetitionRules

State, resources and initial values

Line	Field / declaration
12	readonly attempts: [Attempt[], Attempt[]] = [[], []];
13	player: 0 | 1 = 0;
14	hole = 0;
15	elapsed = 0;
16	world: GolfPhysics;
17	phase: "ready" | "playing" | "between" | "complete" = "ready";

LocalVersusMode [L11-81]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

LocalVersusMode.constructor [L18-23]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly plan: number[], readonly seed: number,)

Direct writes: this.world

Calls / delegates: this.create

LocalVersusMode.create [L24-30]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private create()

Calls / delegates: GolfPhysics, seedFor

Return at L25: new GolfPhysics(courses[this.plan[this.hole]], 0, seedFor(this.seed, this.plan[this.hole]),)

LocalVersusMode.begin [L31-37]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: begin()

Direct writes: this.world, this.elapsed, this.phase

Calls / delegates: this.create

Decision branches: L32: this.phase === "ready" || this.phase === "between"

LocalVersusMode.update [L38-44]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.elapsed

Calls / delegates: this.world.advance, this.finish

Decision branches: L39: this.phase !== "playing"; L42: this.world.status !== "playing" || this.elapsed >= COURSE_TIMEOUT

LocalVersusMode.finish [L45-68]

Resolve/report the owned terminal attempt/result using current guards.

Signature: private finish(reason?: Attempt["reason"])

Direct writes: this.phase, this.player, this.hole

Calls / delegates: this.attempts[this.player].push

Decision branches: L46: this.phase !== "playing"; L60: won && !reason && this.hole < this.plan.length - 1; L63: this.player === 0

LocalVersusMode.forfeit [L69-71]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: forfeit()

Calls / delegates: this.finish

LocalVersusMode.totals [L72-77]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get totals()

Calls / delegates: this.attempts.map, total

Return at L73: this.attempts.map((a) => total(a, this.plan.length)) as [ReturnType<typeof total>, ReturnType<typeof total>,]

LocalVersusMode.winner [L78-80]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get winner()

Calls / delegates: winner

Return at L79: winner(...this.totals)

src/games/perfect-timer

src/games/perfect-timer/PerfectTimerAI.ts

ACTIVE SOURCE / tool or application | 34 lines | SHA-256 86b325a3c1ad80d7...

Schedules an ordinary stop deadline in foreground elapsed milliseconds with calibrated clock error.

Imports: ../../shared/contracts/AIController, ../../shared/ai/MatchRandom

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private scheduledStopMs: number | null = null;

PerfectTimerAI [L4-33]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

PerfectTimerAI.constructor [L7-12]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(public readonly difficulty: Difficulty = "medium", seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

PerfectTimerAI.scheduleRound [L13-23]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: scheduleRound(targetCentiseconds: number, startPerfMs: number)

Direct writes: this.scheduledStopMs

Calls / delegates: r.sample

PerfectTimerAI.shouldStop [L24-26]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: shouldStop(elapsedMs: number)

Return at L25: this.scheduledStopMs !== null && elapsedMs >= this.scheduledStopMs

```
shouldStop(elapsedMs: number) { return this.scheduledStopMs !== null && elapsedMs >= this.scheduledStopMs; }
```

PerfectTimerAI.reset [L27-29]

Reset this component's transient state for a new round/attempt.

Signature: reset()

Direct writes: this.scheduledStopMs

PerfectTimerAI.snapshot [L30-32]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L31: this.personality.snapshot()

src/games/perfect-timer/PerfectTimerGame.ts

ACTIVE SOURCE / tool or application | 486 lines | SHA-256 95bfba8d147d5acd...

Perfect Timer lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/ai/MatchRandom, ./TimerClock, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ./PerfectTimerRules, ./PerfectTimerAI, ./TimerRenderer

State, resources and initial values

Line	Field / declaration
16	private ctx!: GameContext;
17	private random = new MatchRandom();
18	private match = new PerfectTimerMatch(this.random.next);
19	private ai?: PerfectTimerAI;
20	private renderer!: TimerRenderer;
22	private g!: Phaser.GameObjects.Graphics;
23	private uiTexts: Phaser.GameObjects.Text[] = [];
25	private bannerTitle!: Phaser.GameObjects.Text;
26	private targetText!: Phaser.GameObjects.Text;
27	private countdownText!: Phaser.GameObjects.Text;
28	private statusText!: Phaser.GameObjects.Text;
30	private p1BtnText!: Phaser.GameObjects.Text;
31	private p2BtnText!: Phaser.GameObjects.Text;
32	private revealText!: Phaser.GameObjects.Text;
34	private ready = [false, false];
35	private lastTick = -1;
36	private state: "WAITING_READY" | "COUNTDOWN" | "TIMING" | "REVEAL" = "WAITING_READY";
39	private countdownStep = 3;
40	private countdownTimer = 0;
42	private clock = new TimerClock();
43	private animTime = 0;
44	private revealAge = 0;
45	private paused = false;
46	private reported = false;
47	private roundResult?: TimerRoundResult;
50	private readonly p1Btn = { x: 45, y: 550, w: 330, h: 72 };
51	private readonly p2Btn = { x: 45, y: 140, w: 330, h: 72 };

PerfectTimerGame [L15-485]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

PerfectTimerGame.initialize [L53-122]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.renderer, this.g, this.ai, this.bannerTitle, this.targetText, this.countdownText, this.statusText, this.p2BtnText, this.p1BtnText, this.revealText

Calls / delegates: TimerRenderer, ctx.scene.add.graphics, PerfectTimerAI, label, this.bannerTitle.setFontStyle("700").setDepth, this.bannerTitle.setFontStyle, this.uiTexts.push, formatSeconds, this.targetText.setFontStyle("900").setDepth, this.targetText.setFontStyle, this.countdownText.setFontStyle("900").setDepth(3).setAlpha, this.countdownText.setFontStyle("900").setDepth, this.countdownText.setFontStyle, this.statusText.setFontStyle("800").setDepth, this.statusText.setFontStyle, this.p2BtnText.setFontStyle("800").setDepth, this.p2BtnText.setFontStyle, this.p2BtnText.setRotation, this.p1BtnText.setFontStyle("800").setDepth, this.p1BtnText.setFontStyle, this.revealText.setFontStyle("700").setDepth(4).setAlpha, this.revealText.setFontStyle("700").setDepth, this.revealText.setFontStyle, this.setupInputs, this.startRound, this.ctx.setScore

Decision branches: L58: ctx.options.mode === "ai"; L106: ctx.options.mode === "local"

Execution order (direct statements; nested callbacks stay within their owner):

L54-54 ExpressionStatement: this.ctx = ctx;

L55-55 ExpressionStatement: this.renderer = new TimerRenderer(ctx.scene);

L56-56 ExpressionStatement: this.g = ctx.scene.add.graphics();

L58-60 IfStatement: if (ctx.options.mode === "ai") { this.ai = new PerfectTimerAI(ctx.options.difficulty, this.random.seed);

L63-70 ExpressionStatement: this.bannerTitle = label(ctx.scene,

L71-71 ExpressionStatement: this.bannerTitle.setFontStyle("700").setDepth(2);

L72-72 ExpressionStatement: this.uiTexts.push(this.bannerTitle);

L75-82 ExpressionStatement: this.targetText = label(ctx.scene,

L83-83 ExpressionStatement: this.targetText.setFontStyle("900").setDepth(2);

L84-84 ExpressionStatement: this.uiTexts.push(this.targetText);

L87-87 ExpressionStatement: this.countdownText = label(ctx.scene, 210, 390, "", 48, "#e67e22");

L88-88 ExpressionStatement: this.countdownText.setFontStyle("900").setDepth(3).setAlpha(0);

L89-89 ExpressionStatement: this.uiTexts.push(this.countdownText);

L92-92 ExpressionStatement: this.statusText = label(ctx.scene, 210, 510, "Tap ready", 14, "#7f8c8d");

L93-93 ExpressionStatement: this.statusText.setFontStyle("800").setDepth(2);

L94-94 ExpressionStatement: this.uiTexts.push(this.statusText);

L97-104 ExpressionStatement: this.p2BtnText = label(ctx.scene,

L105-105 ExpressionStatement: this.p2BtnText.setFontStyle("800").setDepth(3);

L106-106 IfStatement: if (ctx.options.mode === "local") this.p2BtnText.setRotation(Math.PI);

L107-107 ExpressionStatement: this.uiTexts.push(this.p2BtnText);

L110-110 ExpressionStatement: this.p1BtnText = label(ctx.scene, 210, 586, "Ready", 18, "#ffffff");

L111-111 ExpressionStatement: this.p1BtnText.setFontStyle("800").setDepth(3);

L112-112 ExpressionStatement: this.uiTexts.push(this.p1BtnText);

L115-115 ExpressionStatement: this.revealText = label(ctx.scene, 210, 350, "", 15, "#242b32");

L116-116 ExpressionStatement: this.revealText.setFontStyle("700").setDepth(4).setAlpha(0);

L117-117 ExpressionStatement: this.uiTexts.push(this.revealText);

L119-119 ExpressionStatement: this.setupInputs();

L120-120 ExpressionStatement: this.startRound();

L121-121 ExpressionStatement: this.ctx.setScore(0, 0);

PerfectTimerGame.setupInputs [L124-181]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private setupInputs()

Calls / delegates: this.ctx.input.tap, this.advanceNext, this.handlePlayerAction, this.ctx.input.press

Decision branches: L127: this.paused || this.reported; L129: this.state === "REVEAL"; L130: this.revealAge > 0.8; L137: x >= this.p1Btn.x && x <= this.p1Btn.x + this.p1Btn.w && y >= this.p1Btn.y && y <= this.p1Btn.y + this.p1Btn.h; L148: x >= this.p2Btn.x && x <= this.p2Btn.x + this.p2Btn.w && y >= this.p2Btn.y && y <= this.p2Btn.y + this.p2Btn.h; L154: !this.ai; L163: this.paused || this.reported; L164: this.state === "REVEAL" && this.revealAge > 0.8; L172: this.paused || this.reported; L173: this.state === "REVEAL" && this.revealAge > 0.8; L177: !this.ai

Execution order (direct statements; nested callbacks stay within their owner):

L126-159 ExpressionStatement: this.ctx.input.tap((x, y) => { if (this.paused || this.reported) return;

L162-169 ExpressionStatement: this.ctx.input.press(["Space", "KeyA"], () => { if (this.paused || this.reported) return;

L171-180 ExpressionStatement: this.ctx.input.press(["Enter", "KeyL"], () => { if (this.paused || this.reported) return;

PerfectTimerGame.handlePlayerAction [L183-207]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private handlePlayerAction(player: 0 | 1)

Direct writes: this.ready[player]

Calls / delegates: this.ctx.audio.play, this.updateButtons, this.startCountdown, this.clock.elapsed, this.match.recordStop, this.triggerReveal

Decision branches: L184: this.state === "WAITING_READY"; L185: !this.ready[player]; L191: this.ready[0] && this.ready[1]; L195: this.state === "TIMING"; L196: this.match.stops[player] === null; L202: this.match.bothStopped

PerfectTimerGame.startRound [L209-228]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private startRound()

Direct writes: this.state, this.ready, this.roundResult, this.revealAge

Calls / delegates: this.revealText.setAlpha, this.countdownText.setAlpha, this.targetText.setAlpha, this.statusText.setAlpha, this.bannerTitle.setText, this.ctx.setHint, this.targetText.setText, formatSeconds, this.statusText .setText(this.ai ? "Ready" : "Tap ready") .setColor, this.statusText .setText, this.updateButtons

PerfectTimerGame.startCountdown [L230-242]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private startCountdown()

Direct writes: this.state, this.countdownStep, this.countdownTimer

Calls / delegates: this.statusText.setText("").setColor("#e67e22").setFontSize(14).setAlpha, this.statusText.setText("").setColor("#e67e22").setFontSize, this.statusText.setText("").setColor, this.statusText.setText, this.countdownText .setText("3") .setColor("#e67e22") .setFontSize(48) .setAlpha, this.countdownText .setText("3") .setColor("#e67e22") .setFontSize, this.countdownText .setText("3") .setColor, this.countdownText .setText, this.ctx.audio.play, this.updateButtons

PerfectTimerGame.startTimingPhase [L244-256]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private startTimingPhase()

Direct writes: this.state, this.lastTick

Calls / delegates: this.clock.start, this.countdownText.setAlpha, this.statusText.setText("").setColor("#e74c3c").setFontSize(14).setAlpha, this.statusText.setText("").setColor("#e74c3c").setFontSize, this.statusText.setText("").setColor, this.statusText.setText, this.ctx.audio.play, this.ai.scheduleRound, this.updateButtons

Decision branches: L252: this.ai

PerfectTimerGame.updateButtons [L258-283]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private updateButtons()

Calls / delegates: this.p1BtnText.setText, this.p2BtnText.setText

Decision branches: L259: this.state === "WAITING_READY"; L261: this.ai; L266: this.state === "COUNTDOWN"; L269: this.state === "TIMING"; L274: this.ai; L279: this.state === "REVEAL"

PerfectTimerGame.triggerReveal [L285-328]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private triggerReveal()

Direct writes: this.state, this.revealAge, this.roundResult, winnerStr

Calls / delegates: this.clock.pause, this.match.resolveRound, this.ctx.setScore, formatSeconds, this.ctx.audio.play, this.targetText.setAlpha, this.statusText.setAlpha, this.countdownText.setAlpha, this.revealText .setText(`TARGET: ${tarStr}\n\n` + `ORANGE: ${s0Str} (ERR ${err0Str})\n` + `${this.ai ? "AI" : "BLUE"}: ${s1Str} (ERR ${err1Str})\n\n` + `★ ${winnerStr} ★\n\n` + `TAP TO CONTINUE`,) .setAlpha, this.revealText .setText, this.updateButtons

Decision branches: L286: this.state !== "TIMING" || this.reported; L301: res.winner === 1; L304: res.winner === 2

Execution order (direct statements; nested callbacks stay within their owner):

L286-286 IfStatement: if (this.state !== "TIMING" || this.reported) return;

L287-287 ExpressionStatement: this.clock.pause();

L288-288 ExpressionStatement: this.state = "REVEAL";

L289-289 ExpressionStatement: this.revealAge = 0;

L290-290 ExpressionStatement: this.roundResult = this.match.resolveRound();

L291-291 ExpressionStatement: this.ctx.setScore(this.match.scores[0], this.match.scores[1]);

L293-293 FirstStatement: const res = this.roundResult;

L294-294 FirstStatement: const s0Str = formatSeconds(res.stops[0]);

L295-295 FirstStatement: const s1Str = formatSeconds(res.stops[1]);

L296-296 FirstStatement: const err0Str = formatSeconds(res.errors[0]);

L297-297 FirstStatement: const err1Str = formatSeconds(res.errors[1]);

L298-298 FirstStatement: const tarStr = formatSeconds(res.target);

L300-300 FirstStatement: let winnerStr: string;

L301-310 IfStatement: if (res.winner === 1) { winnerStr = "ORANGE WINS ROUND!";

L313-313 ExpressionStatement: this.targetText.setAlpha(0);

L314-314 ExpressionStatement: this.statusText.setAlpha(0);

L315-315 ExpressionStatement: this.countdownText.setAlpha(0);

L317-325 ExpressionStatement: this.revealText .setText(

L327-327 ExpressionStatement: this.updateButtons();

PerfectTimerGame.advanceNext [L330-344]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private advanceNext()

Direct writes: this.reported

Calls / delegates: this.ctx.finish, this.match.nextRound, this.startRound

Decision branches: L331: this.match.isMatchOver; L332: !this.reported

PerfectTimerGame.update [L346-400]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.animTime, this.countdownTimer, this.lastTick, this.revealAge

Calls / delegates: this.countdownText.setText("GO!").setColor, this.countdownText.setText, this.startTimingPhase, this.countdownText .setText(`${this.countdownStep}`) .setColor, this.countdownText .setText, this.ctx.audio.play, this.clock.elapsed, this.ai.shouldStop, this.match.recordStop, this.updateButtons, this.triggerReveal, this.advanceNext, this.render

Decision branches: L347: this.paused || this.reported; L350: this.state === "COUNTDOWN"; L352: this.countdownTimer <= 0; L354: this.countdownStep === 0; L365: this.state === "TIMING"; L368: tick !== this.lastTick && !this.match.bothStopped; L374: this.ai && this.match.stops[1] === null && this.ai.shouldStop(now); L379: this.match.bothStopped; L386: this.state === "TIMING" && now > maxMs; L387: this.match.stops[0] === null; L388: this.match.stops[1] === null; L392: this.state === "REVEAL"; L394: this.revealAge > 3.2

Execution order (direct statements; nested callbacks stay within their owner):

L347-347 IfStatement: if (this.paused || this.reported) return;

L348-348 ExpressionStatement: this.animTime += dt;

L350-397 IfStatement: if (this.state === "COUNTDOWN") { this.countdownTimer -= dt;

L399-399 ExpressionStatement: this.render();

PerfectTimerGame.render [L402-455]

Build or refresh visual/DOM output from current state.

Signature: private render()

Calls / delegates: g.clear, rounded, this.renderer.drawStopwatch

Decision branches: L452: this.state === "REVEAL"

Execution order (direct statements; nested callbacks stay within their owner):

L403-403 FirstStatement: const g = this.g;

L404-404 ExpressionStatement: g.clear();

L407-407 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xfcf8f2, 4);

L410-417 ExpressionStatement: this.renderer.drawStopwatch(g,

L420-423 FirstStatement: const p2Color = this.state === "TIMING" && this.match.stops[1] !== null

L424-433 ExpressionStatement: rounded(g,

L436-439 FirstStatement: const p1Color = this.state === "TIMING" && this.match.stops[0] !== null

L440-449 ExpressionStatement: rounded(g,

L452-454 IfStatement: if (this.state === "REVEAL") { rounded(g, 40, 235, 340, 230, 18, 0xffffff, 3);

PerfectTimerGame.pause [L457-460]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.clock.pause

Decision branches: L458: this.state === "TIMING"

PerfectTimerGame.resume [L462-465]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

Calls / delegates: this.clock.resume

Decision branches: L463: this.paused && this.state === "TIMING"

PerfectTimerGame.dispose [L467-473]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.reported

Calls / delegates: this.clock.pause, this.ai?.reset, this.g.destroy, t.destroy

PerfectTimerGame.snapshot [L475-484]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot

Return at L476: { ai: this.ai?.snapshot(), match: this.match, state: this.state, ready: [...this.ready], scores: [...this.match.scores], round: this.match.round, }

src/games/perfect-timer/PerfectTimerRules.ts

ACTIVE SOURCE / tool or application | 132 lines | SHA-256 fb7b45176c2287c5...

Owns centisecond targets/stops, error comparison, idempotent scoring and five-round/sudden-death progression.

State, resources and initial values

Line	Field / declaration
25	round = 1;
26	scores: [number, number] = [0, 0];
27	history: TimerRoundResult[] = [];
28	target = 1000;
29	stops: [number | null, number | null] = [null, null];
31	private resolved?: TimerRoundResult;
32	private usedTargets = new Set<number>();

Module constants and data

L1-1 TOTAL_ROUNDS = 5

L4-4 TARGET_PRESETS = [500, 743, 1000, 1267, 1525]

formatSeconds [L6-9]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function formatSeconds(centiseconds: number): string

Calls / delegates: (clamped / 100).toFixed

Return at L8: (clamped / 100).toFixed(2) + "s"

formatSecondsRaw [L11-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function formatSecondsRaw(centiseconds: number): string

Calls / delegates: (clamped / 100).toFixed

Return at L13: (clamped / 100).toFixed(2)

TimerRoundResult [L16-22]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface TimerRoundResult { round: number; target: number; // centiseconds (e.g. 1000 = 10.00s) stops: [number, number]; // recorded centiseconds for each player errors: [number, number]; // |stop - target| in centiseconds winner: 1 | 2 | "draw"; }
```

PerfectTimerMatch [L24-131]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

PerfectTimerMatch.constructor [L34-36]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private rng: () => number = Math.random)

Calls / delegates: this.startNewRound

PerfectTimerMatch.startNewRound [L38-54]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: startNewRound()

Direct writes: this.resolved, this.stops, candidate, this.target

Calls / delegates: TARGET_PRESETS.filter, this.usedTargets.has, this.rng, this.usedTargets.add

PerfectTimerMatch.recordStop [L56-64]

Convert validated foreground milliseconds to one integer-centisecond stop per player.

Signature: recordStop(player: 0 | 1, elapsedMs: number): number

Direct writes: this.stops[player]

Calls / delegates: RangeError

Return at L59: this.stops[player]!

Return at L63: centiseconds

Decision branches: L57: !Number.isFinite(elapsedMs) || elapsedMs < 0; L59: this.stops[player] !== null

PerfectTimerMatch.bothStopped [L66-68]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get bothStopped(): boolean

Return at L67: this.stops[0] !== null && this.stops[1] !== null

```
get bothStopped(): boolean { return this.stops[0] !== null && this.stops[1] !== null; }
```

PerfectTimerMatch.resolveRound [L70-101]

Compare two stop errors and cache one score result so repeated resolution cannot score again.

Signature: resolveRound(): TimerRoundResult

Direct writes: winner, this.resolved

Calls / delegates: Error, this.history.push

Return at L71: this.resolved

Return at L100: result

Decision branches: L71: this.resolved; L72: !this.bothStopped; L81: err0 < err1; L84: err1 < err0

PerfectTimerMatch.nextRound [L103-108]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: nextRound(): boolean

Calls / delegates: this.startNewRound

Decision branches: L104: !this.resolved || this.isMatchOver

PerfectTimerMatch.isSuddenDeath [L110-112]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get isSuddenDeath(): boolean

Return at L111: this.round > TOTAL_ROUNDS && this.scores[0] === this.scores[1]

```
get isSuddenDeath(): boolean { return this.round > TOTAL_ROUNDS && this.scores[0] === this.scores[1]; }
```

PerfectTimerMatch.isMatchOver [L114-123]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get isMatchOver(): boolean

Return at L117: (this.history.length === TOTAL_ROUNDS && this.scores[0] !== this.scores[1])

Return at L122: this.scores[0] !== this.scores[1]

Decision branches: L115: this.round < TOTAL_ROUNDS; L116: this.round === TOTAL_ROUNDS

PerfectTimerMatch.matchWinner [L125-130]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get matchWinner(): 1 | 2 | "draw" | null

Return at L128: 2

Return at L129: "draw"

Decision branches: L126: !this.isMatchOver; L127: this.scores[0] > this.scores[1]; L128: this.scores[1] > this.scores[0]

src/games/perfect-timer/TimerClock.ts

ACTIVE SOURCE / tool or application | 25 lines | SHA-256 598a3cf21c644a4c...

Banks/resumes a monotonic foreground timer so pause time is excluded.

State, resources and initial values

Line	Field / declaration
3	private started = 0;
4	private carried = 0;
5	private running = false;

TimerClock [L2-24]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TimerClock.start [L6-10]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: start(now = performance.now())

Direct writes: this.started, this.carried, this.running

TimerClock.elapsed [L11-13]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: elapsed(now = performance.now())

Return at L12: this.carried + (this.running ? Math.max(0, now - this.started) : 0)

TimerClock.pause [L14-17]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause(now = performance.now())

Direct writes: this.carried, this.running

Calls / delegates: this.elapsed

TimerClock.resume [L18-23]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume(now = performance.now())

Direct writes: this.started, this.running

Decision branches: L19: !this.running

src/games/perfect-timer/TimerRenderer.ts

ACTIVE SOURCE / tool or application | 75 lines | SHA-256 3e164bc20a72ed8b...

Draws the stopwatch body/face/needle and timing feedback.

Imports: phaser, ../../core/ThemeManager

TimerRenderer [L4-74]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TimerRenderer.constructor [L5-5]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private scene: Phaser.Scene)

```
constructor(private scene: Phaser.Scene) {}
```

TimerRenderer.drawStopwatch [L7-73]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: drawStopwatch(g: Phaser.GameObjects.Graphics, cx: number, cy: number, r: number, isActiveTiming: boolean, pulseTime: number,)

Calls / delegates: g.fillStyle, g.fillRoundedRect, g.lineStyle, g.strokeRoundedRect, g.strokeCircle, g.fillCircle, g.lineBetween

Decision branches: L54: isActiveTiming

Execution order (direct statements; nested callbacks stay within their owner):

L16-16 ExpressionStatement: g.fillStyle(0xd5dbdb);

L17-17 ExpressionStatement: g.fillRoundedRect(cx - 14, cy - r - 22, 28, 16, 5);

L18-18 ExpressionStatement: g.lineStyle(3, theme.ink);

L19-19 ExpressionStatement: g.strokeRoundedRect(cx - 14, cy - r - 22, 28, 16, 5);

L22-22 ExpressionStatement: g.lineStyle(3.5, theme.ink);

L23-23 ExpressionStatement: g.strokeCircle(cx, cy - r - 26, 11);

L26-26 ExpressionStatement: g.fillStyle(0xffffff);

L27-27 ExpressionStatement: g.fillCircle(cx, cy, r);

L28-28 ExpressionStatement: g.lineStyle(5, theme.ink);

L29-29 ExpressionStatement: g.strokeCircle(cx, cy, r);

L32-32 ExpressionStatement: g.fillStyle(0xfafafa);

L33-33 ExpressionStatement: g.fillCircle(cx, cy, r - 12);

L34-34 ExpressionStatement: g.lineStyle(2, 0xd0d3d4);

L35-35 ExpressionStatement: g.strokeCircle(cx, cy, r - 12);

L38-51 ForStatement: for (let i = 0; i < 12; i++) { const angle = (i * Math.PI) / 6;

L54-72 IfStatement: if (isActiveTiming) { const pulse = (Math.sin(pulseTime * 5) + 1) * 0.5; // 0..1

src/games/perfect-timer/config.ts

ACTIVE SOURCE / tool or application | 18 lines | SHA-256 82d2057835266ceb...

Eager lightweight metadata for Perfect Timer: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L4-17

```
export default {
  id: "perfect-timer",
  name: "Perfect Timer",
  description: "Stop the hidden stopwatch on the target with 0.01s precision.",
  instructions:
    "Remember the target. After GO, stop the hidden stopwatch at that duration. Closest, measured to 0.01s, wins the
round. Five rounds. Orange: Space/A. Blue: Enter/L.",
  color: "#fdebd0",
  accent: "#f5b041",
  preview,
  order: 15,
  tag: "TIME SENSE",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/perfect-timer/index.ts

ACTIVE SOURCE / tool or application | 4 lines | SHA-256 a920c72196fc2cbb...

Side-effect-free lazy factory for Perfect Timer; implementation initializes only after host selection.

Imports: ./PerfectTimerGame

Top-level block L3-3

```
export default () => new PerfectTimerGame();
```

src/games/pong

src/games/pong/PongAI.ts

ACTIVE SOURCE / tool or application | 155 lines | SHA-256 83951c38836f5ae6...

Predicts reflected ball intercepts and legal offensive/defensive paddle targets.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./PongPhysics, ../../core/ThemeManager, ../../shared/physics/MotionController, ../../shared/physics/FixedStep

State, resources and initial values

Line	Field / declaration
16	readonly personality: MatchRandom;
17	private shotOffset = 0;
25	private reaction = 0;
26	private target = 210;
27	private control = 210;
28	private motion = new MotionController(210, 0);
29	private aimError = 0;
30	private lastHits = -1;

reflectedX [L10-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function reflectedX(x: number, left: number, right: number)

Return at L13: left + (phase <= width ? phase : 2 * width - phase)

```
export function reflectedX(x: number, left: number, right: number) { const width = right - left, phase = (((x - left) % (2 * width)) + 2 * width) % (2 * width); return left + (phase <= width ? phase : 2 * width - phase); }
```

PongAI [L15-154]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

PongAI.snapshot [L18-24]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L19: { ...this.personality.snapshot(), target: this.target, shotOffset: this.shotOffset, }

PongAI.constructor [L31-36]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: SeedInput,)

Direct writes: this.personality

Calls / delegates: MatchRandom

PongAI.reset [L37-42]

Reset this component's transient state for a new round/attempt.

Signature: reset()

Direct writes: this.reaction, this.target, this.control, this.lastHits

Calls / delegates: this.motion.reset

PongAI.update [L43-153]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, w: PongWorld)

Direct writes: this.target, this.lastHits, this.reaction, this.aimError, this.shotOffset, this.control

Calls / delegates: this.personality.sample, reflectedX, clamp, this.motion.move

Return at L44: w.paddles[1].x

Return at L152: this.control

Decision branches: L44: w.done; L68: w.cooldown > 0; L73: this.reaction <= 0 && w.cooldown <= 0; L82: b.vy < 0; L102: w.hits !== this.lastHits; L121: b.vy < 0; L131: reachable

Execution order (direct statements; nested callbacks stay within their owner):

L44-44 IfStatement: if (w.done) return w.paddles[1].x;

L45-67 FirstStatement: const settings = { medium: {

L68-71 IfStatement: if (w.cooldown > 0) { this.target = 210;

L72-72 ExpressionStatement: this.reaction -= dt;

L73-139 IfStatement: if (this.reaction <= 0 && w.cooldown <= 0) { this.reaction = this.personality.sample(

L140-151 ExpressionStatement: this.control = clamp(this.motion.move(

L152-152 ReturnStatement: return this.control;

src/games/pong/PongGame.ts

ACTIVE SOURCE / tool or application | 221 lines | SHA-256 2ccdcd82cdb94c78...

Pong lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../shared/ai/MatchRandom, ./PongState, ../../shared/rules/MatchClock, ./PongAI, phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/physics/FixedStep, ../../shared/components/drawing, ../../shared/effects/Impacts, ./PongPhysics

State, resources and initial values

Line	Field / declaration
17	private ctx!: GameContext;
18	private world!: PongWorld;
19	private clock = new FixedStep();
20	private matchClock = new MatchClock();
21	private graphics!: Phaser.GameObjects.Graphics;
22	private bg!: Phaser.GameObjects.Graphics;
23	private status!: Phaser.GameObjects.Text;
24	private effects = new Impacts();
25	private sinceSave = 0;
26	private paused = false;
27	private ai?: PongAI;
28	private keys = new Set<string>();
29	private trail = Array.from({ length: 8 }, () => ({ x: 210, y: 381 }));
30	private trailAt = 0;

PingPong [L16-220]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

PingPong.initialize [L31-62]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.ai, this.keys, this.bg, x, this.graphics, this.status, this.world

Calls / delegates: PongAI, restoredAISeed, ctx.input.dragHalves, ctx.input.keys, ctx.setHint, ctx.scene.add.graphics, rounded, g.fillStyle(0xffffff, 0.08).fillRect, g.fillStyle, g.lineStyle(2, 0xffffff, 0.65).strokeRect, g.lineStyle, g.lineStyle(3, theme.white, 0.75).lineBetween, circle, g.lineStyle(2, theme.white, 0.6).strokeCircle, label, PongWorld, this.effects.burst, ctx.audio.play, this.goal, restorePong, ctx.setScore, this.save

Decision branches: L33: ctx.options.mode === "ai"

PingPong.goal [L63-75]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private goal(p: Player)

Direct writes: v.x, v.y, this.world.done

Calls / delegates: this.ctx.setScore, this.ctx.audio.play, this.trail.forEach, this.matchClock.score, this.ctx.finish

Decision branches: L71: winner !== null

PingPong.update [L76-164]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: w.done, this.sinceSave, t.x, t.y, rx

Calls / delegates: this.matchClock.advance, this.ctx.setClock, this.ctx.setHint, this.ctx.finish, this.clock.advance, w.step, this.paddleTarget, this.ai.update, this.save, clamp, w.paddles.map, g.clear, g.fillStyle(theme.white, 0.08 + i * 0.045).fillCircle, g.fillStyle, w.paddles.forEach, g.fillStyle(theme.ink, 0.15).fillRoundedRect, rounded, g.lineStyle(2, theme.white, 0.7).lineBetween, g.lineStyle, circle, this.effects.draw, this.status .setText(w.cooldown > 0.5 ? "READY?" : w.cooldown > 0 ? "GO!" : "") .setAlpha, this.status .setText

Return at L116: p.x

Return at L126: clamp(rx, FIELD.left + 44, FIELD.right - 44)

Decision branches: L77: this.paused; L81: this.matchClock.overtime; L83: winner !== null; L95: this.sinceSave >= 1; L110: w.cooldown <= 0 && !w.done; L116: w.done; L124: p.vx > 0 && rx > target; L125: p.vx < 0 && rx < target; L130: w.cooldown <= 0

Execution order (direct statements; nested callbacks stay within their owner):

L77-77 IfStatement: if (this.paused) return;

L78-78 FirstStatement: const w = this.world;

L79-79 FirstStatement: const winner = this.matchClock.advance(dt, w.scores);

L80-80 ExpressionStatement: this.ctx.setClock(this.matchClock.text, this.matchClock.remaining <= 10);

L81-82 IfStatement: if (this.matchClock.overtime) this.ctx.setHint("SUDDEN DEATH · NEXT POINT WINS");

L83-87 IfStatement: if (winner !== null) { w.done = true;

L88-93 ExpressionStatement: this.clock.advance(dt, (step) => { w.step(step, [

L94-94 ExpressionStatement: this.sinceSave += dt;

L95-98 IfStatement: if (this.sinceSave >= 1) { this.sinceSave = 0;

L99-99 FirstStatement: const rem = w.cooldown <= 0 && !w.done ? this.clock.remainder : 0;

L100-104 FirstStatement: const renderBallX = clamp(w.ball.x + w.ball.vx * rem,

L105-109 FirstStatement: const renderBallY = clamp(w.ball.y + w.ball.vy * rem,

L110-114 IfStatement: if (w.cooldown <= 0 && !w.done) { const t = this.trail[this.trailAt++ % 8];

L115-127 FirstStatement: const renderPaddles = w.paddles.map((p, i) => { if (w.done) return p.x;

L128-128 FirstStatement: const g = this.graphics;

L129-129 ExpressionStatement: g.clear();

L130-138 IfStatement: if (w.cooldown <= 0) for (let i = 0; i < 8; i++) {

L139-158 ExpressionStatement: w.paddles.forEach((p, i) => { const px = renderPaddles[i];

L159-159 ExpressionStatement: circle(g, renderBallX, renderBallY, 9, theme.white, 3);

L160-160 ExpressionStatement: this.effects.draw(g, dt);

L161-163 ExpressionStatement: this.status .setText(w.cooldown > 0.5 ? "READY?" : w.cooldown > 0 ? "GO!" : "")

PingPong.paddleTarget [L165-185]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private paddleTarget(player: 0 | 1, step: number)

Calls / delegates: this.keys.has, Number

Return at L167: pointer.x

Return at L181: (this.world.paddles[player].x + (Number(right) - Number(left)) * speed * step)

Decision branches: L167: pointer.down

PingPong.save [L186-192]

Create/persist the component checkpoint or record described by the returned expression.

Signature: private save()

Calls / delegates: this.ctx.saveState, savePong, this.ai?.snapshot

Decision branches: L187: !this.world.done

PingPong.pause [L193-197]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.save, this.clock.reset

PingPong.resume [L198-201]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

Calls / delegates: this.clock.reset

PingPong.dispose [L202-206]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.bg.destroy, this.graphics.destroy, this.status.destroy

PingPong.snapshot [L207-219]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot, this.world.paddles.map

Return at L208: { ai: this.ai?.snapshot(), ball: { ...this.world.ball }, paddles: this.world.paddles.map((p) => ({ ...p })), timer: { remaining: this.matchClock.remaining, overtime: this.matchClock.overtime, }, scores: [...this.world.scores], cooldown: this.world.cooldown, }

src/games/pong/PongPhysics.ts

ACTIVE SOURCE / tool or application | 84 lines | SHA-256 2504a440a8604704...

Integrates ball/paddle movement, swept paddle hits, angle/spin response, walls and goals.

Imports: ../../core/ThemeManager, ../../shared/physics/FixedStep, ../../shared/contracts/GameModule

State, resources and initial values

Line	Field / declaration
5	ball = { x: 210, y: 381, vx: 110, vy: 340, r: 9 };
6	paddles = [{ x: 210, y: 608, vx: 0 }, { x: 210, y: 154, vx: 0 },];
10	scores: [number, number] = [0, 0];
11	cooldown = 1.35;
12	done = false;
13	hits = 0;

PongWorld [L4-83]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

PongWorld.constructor [L14-17]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private impact: (x: number, y: number) => void = () => {}, private scored: (p: Player) => void = () => {},)

```
constructor( private impact: (x: number, y: number) => void = () => {}, private scored: (p: Player) => void = () => {}, ) {}
```

PongWorld.step [L18-71]

Move legally bounded paddles, integrate the ball, resolve swept paddle-plane/side contacts and detect goals.

Signature: step(dt: number, targets: [number, number])

Direct writes: p.x, p.vx, this.cooldown, b.x, b.y, b.vx, b.vy

Calls / delegates: this.paddles.forEach, clamp, this.impact, this.goal

Decision branches: L28: this.done; L29: this.cooldown > 0; L37: b.x - b.r < FIELD.left; L42: b.x + b.r > FIELD.right; L50: b.vy * direction > 0 && (oldY - contact) * direction <= 0 && (b.y - contact) * direction >= 0 && Math.abs(b.x - p.x) <= 44 + b.r; L69: b.y < FIELD.top - b.r * 2; L70: b.y > FIELD.bottom + b.r * 2

Execution order (direct statements; nested callbacks stay within their owner):

L19-27 ExpressionStatement: this.paddles.forEach((p, i) => { const old = p.x;

L28-28 IfStatement: if (this.done) return;

L29-32 IfStatement: if (this.cooldown > 0) { this.cooldown -= dt;

L33-34 FirstStatement: const b = this.ball, oldY = b.y;

L35-35 ExpressionStatement: b.x += b.vx * dt;

L36-36 ExpressionStatement: b.y += b.vy * dt;

L37-41 IfStatement: if (b.x - b.r < FIELD.left) { b.x = FIELD.left + b.r;

L42-46 IfStatement: if (b.x + b.r > FIELD.right) { b.x = FIELD.right - b.r;

L47-68 ExpressionStatement: this.paddles.forEach((p, i) => { const direction = i === 0 ? 1 : -1,

L69-70 IfStatement: if (b.y < FIELD.top - b.r * 2) this.goal(1); else if (b.y > FIELD.bottom + b.r * 2) this.goal(2);

PongWorld.goal [L72-82]

Award one point, reset serve/cooldown and check the local point target while the controller retains its match clock.

Signature: private goal(player: Player)

Direct writes: this.done, this.cooldown, this.hits, this.ball.x, this.ball.y, this.ball.vx, this.ball.vy

Calls / delegates: this.scored

src/games/pong/PongState.ts

ACTIVE SOURCE / tool or application | 74 lines | SHA-256 354805cd08762343...

Serializes and validates a Pong rally, score, clock and AI identity checkpoint.

Imports: ./PongPhysics, ../../shared/rules/MatchClock

savePong [L3-13]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function savePong(w: PongWorld, clock: MatchClock)

Calls / delegates: w.paddles.map

Return at L4: { version: 1, ball: { ...w.ball }, paddles: w.paddles.map((p) => ({ ...p })), scores: [...w.scores], cooldown: w.cooldown, hits: w.hits, clock: { remaining: clock.remaining, overtime: clock.overtime }, }

restorePong [L14-73]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function restorePong(w: PongWorld, clock: MatchClock, value: unknown)

Direct writes: w.scores, w.cooldown, w.hits

Calls / delegates: d.scores.every, d.paddles.every, w.paddles.forEach

Decision branches: L16: !d || d.version !== 1 || !d.ball || !d.clock || !Array.isArray(d.paddles) || d.paddles.length !== 2 || !Array.isArray(d.scores) || d.scores.length !== 2; L27: !d.scores.every((n) => Number.isInteger(n) && n >= 0 && n < 5) || !Number.isFinite(d.cooldown) || d.cooldown < -0.02 || d.cooldown > 1.35 || !Number.isSafeInteger(d.hits) || d.hits < 0; L36: typeof d.clock.overtime !== "boolean" || !Number.isFinite(d.clock.remaining) || d.clock.remaining <= 0 || d.clock.remaining > (d.clock.overtime ? 60 : 120); L44: !Number.isFinite(b.x + b.y + b.vx + b.vy) || b.x < 33 || b.x > 387 || b.y < 93 || b.y > 669 || Math.hypot(b.vx, b.vy) > 1000; L53: !d.paddles.every((p) => p && Number.isFinite(p.x + p.vx) && p.x >= 68 && p.x <= 352 && Math.abs(p.vx) <= 1800.01,)

Execution order (direct statements; nested callbacks stay within their owner):

L15-15 FirstStatement: const d = value as ReturnType<typeof savePong>;

L16-26 IfStatement: if (!d ||

L27-35 IfStatement: if (!d.scores.every((n) => Number.isInteger(n) && n >= 0 && n < 5) ||

L36-42 IfStatement: if (typeof d.clock.overtime !== "boolean" ||

L43-43 FirstStatement: const b = d.ball;

L44-52 IfStatement: if (!Number.isFinite(b.x + b.y + b.vx + b.vy) ||

L53-63 IfStatement: if (!d.paddles.every(

L64-64 ExpressionStatement: Object.assign(w.ball, b, { r: 9 });

L65-67 ExpressionStatement: w.paddles.forEach((p, i) => Object.assign(p, { x: d.paddles[i].x, vx: d.paddles[i].vx }),

L68-68 ExpressionStatement: w.scores = [...d.scores] as [number, number];

L69-69 ExpressionStatement: w.cooldown = d.cooldown;

L70-70 ExpressionStatement: w.hits = d.hits;

L71-71 ExpressionStatement: Object.assign(clock, d.clock);

L72-72 ReturnStatement: return true;

src/games/pong/config.ts

ACTIVE SOURCE / tool or application | 16 lines | SHA-256 bdd93d155b344443...

Eager lightweight metadata for Pong: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-15

```
export default {
  id: "ping-pong",
  name: "Pong",
  description: "A little spin. A lot of rivalry.",
  instructions:
    "Drag anywhere on your half to move your paddle, or use A/D for orange and Left/Right arrows for blue. In solo mode, either key pair moves your paddle. Orange plays from the bottom, blue from the top. First to 5 or the lead after 2 minutes wins. A nonzero tie gets 1 minute of sudden death.",
  color: "#b8d998",
  accent: "#e5f1da",
  tag: "QUICK REFLEXES",
  order: 1,
  preview,
  supportedModes: ["local-two-player", "single-player"],
} satisfies GameMetadata;
```

src/games/pong/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 f2a4471c4d567afc...

Side-effect-free lazy factory for Pong; implementation initializes only after host selection.

Imports: ./PongGame

Top-level block L2-2

```
export default () => new PingPong();
```

src/games/spike-attacks

src/games/spike-attacks/AI.ts

ACTIVE SOURCE / tool or application | 159 lines | SHA-256 e4213eb29dcf2702...

Game-local Spike Attacks observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./ArenaPhysics, ./Rules

State, resources and initial values

Line	Field / declaration
82	readonly personality: MatchRandom;
83	private wave = -1;
84	private delay = 0;
85	private route: Point[] = [];
86	private lastPlanMs = 0;

Module constants and data

L77-80 candidates = Array.from({ length: 15 * 15 }, (_, i) => ({ x: 20 + (i % 15) * 20, y: 20 + Math.floor(i / 15) * 20, }))

segmentClear [L11-31]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function segmentClear(a: Point, b: Point, blocks: readonly Block[])

Direct writes: lo, hi

Calls / delegates: blocks.every

Return at L13: blocks.every((box) => { let lo = 0, hi = 1; for (const axis of ["x", "y"] as const) { const v = b[axis] - a[axis], min = box[axis] - 9, max = box[axis] + (axis === "x" ? box.w : box.h) + 9; if (Math.abs(v) < 1e-9) { if (a[axis] <= min || a[axis] >= max) return true; } else { const t1 = (min - a[axis]) / v, t2 = (max - a[axis]) / v; lo = Math.max(lo ... [continued in source reference]

Return at L29: lo > hi || hi <= 1e-7

Decision branches: L20: Math.abs(v) < 1e-9; L21: a[axis] <= min || a[axis] >= max

coverRoute [L32-76]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function coverRoute(start: Point, goal: Point, blocks: readonly Block[] = BLOCKS,): Point[]

Direct writes: dist[0], u, dist[v], prev[v], i

Calls / delegates: clearPoint, blocks .flatMap((b) => [{ x: b.x - 11, y: b.y - 11 }, { x: b.x + b.w + 11, y: b.y - 11 }, { x: b.x - 11, y: b.y + b.h + 11 }, { x: b.x + b.w + 11, y: b.y + b.h + 11 },]) .filter, blocks .flatMap, nodes.map, Set, seen.has, seen.add, segmentClear, route.unshift

Return at L37: []

Return at L72: []

Return at L75: route

Decision branches: L37: !clearPoint(goal, 9.5, blocks); L57: !seen.has(i) && (u < 0 || dist[i] < dist[u]); L58: u < 0 || !Number.isFinite(dist[u]); L59: u === 1; L62: !seen.has(v) && v !== u && segmentClear(nodes[u], nodes[v], blocks); L66: cost < dist[v]; L72: !Number.isFinite(dist[1])

Execution order (direct statements; nested callbacks stay within their owner):

L37-37 IfStatement: if (!clearPoint(goal, 9.5, blocks)) return [];

L38-49 FirstStatement: const nodes = [start,

L50-52 FirstStatement: const dist = nodes.map(() => Infinity), prev = nodes.map(() => -1),

L53-53 ExpressionStatement: dist[0] = 0;

L54-71 ForStatement: for (let k = 0; k < nodes.length; k++) { let u = -1;

L72-72 IfStatement: if (!Number.isFinite(dist[1])) return [];

L73-73 FirstStatement: const route: Point[] = [];

L74-74 ForStatement: for (let i = 1; i > 0; i = prev[i]) route.unshift(nodes[i]);

L75-75 ReturnStatement: return route;

SpikeAI [L81-158]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SpikeAI.constructor [L87-92]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

SpikeAI.update [L93-154]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, m: SpikeMatch, player: 0 | 1 = 1)

Direct writes: this.wave, this.route, cost, last, best, this.delay, this.lastPlanMs

Calls / delegates: performance.now, m.map.blocks.map, (this.difficulty === "medium" ? basic : candidates) .filter((c) => safeFrom(c, m.direction, 0.5, m.map.blocks)) .map((goal) => ({ goal, d: Math.hypot(goal.x - p.x, goal.y - p.y) })) .sort((a, b) => a.d - b.d) .slice, (this.difficulty === "medium" ? basic : candidates) .filter((c) => safeFrom(c, m.direction, 0.5, m.map.blocks)) .map((goal) => ({ goal, d: Math.hypot(goal.x - p.x, goal.y - p.y) })) .sort, (this.difficulty === "medium" ? basic : candidates) .filter((c) => safeFrom(c, m.direction, 0.5, m.map.blocks)) .map, (this.difficulty === "medium" ? basic : candidates) .filter, safeFrom, coverRoute, r.rank, r.sample, this.route.shift, m.target

Decision branches: L94: m.winner || m.between || !m.players[player].alive; L97: this.wave !== m.wave; L121: !route.length; L133: cost < best; L146: this.delay > 0; L153: t

Execution order (direct statements; nested callbacks stay within their owner):

L94-94 IfStatement: if (m.winner || m.between || !m.players[player].alive) return;

L95-96 FirstStatement: const p = m.players[player], r = this.personality;

L97-144 IfStatement: if (this.wave !== m.wave) { const started = performance.now();

L145-145 ExpressionStatement: this.delay -= dt;

L146-146 IfStatement: if (this.delay > 0) return;

L147-151 WhileStatement: while (this.route.length > 1 &&

L152-152 FirstStatement: const t = this.route[0];

L153-153 IfStatement: if (t) m.target(player, t.x, t.y);

SpikeAI.computationMs [L155-157]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get computationMs()

Return at L156: this.lastPlanMs

```
get computationMs() { return this.lastPlanMs; }
```

src/games/spike-attacks/ArenaPhysics.ts

ACTIVE SOURCE / tool or application | 171 lines | SHA-256 b77167891523fe44...

Maintained implementation/configuration file. Use the source-grounded block map and complete companion listing for its exact control flow.

Imports: ./Maps

Module constants and data

L2-2 SIZE = 320

L3-3 RADIUS = 9

L4-4 SPEED = 160

L5-5 SHAFT = 72

L9-9 BLOCKS = MAPS[0].blocks

Point [L6-6]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Point = { x: number; y: number };
```

Block [L7-7]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Block = Point & { w: number; h: number };
```

Side [L8-8]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Side = 0 | 1 | 2 | 3;
```

laneCenter [L10-10]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: laneCenter = (lane: number) =>

```
laneCenter = (lane: number) => 40 + lane * 80
```

clearPoint [L11-29]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function clearPoint(p: Point, margin = RADIUS + 0.5, blocks: readonly Block[] = BLOCKS,)

Calls / delegates: blocks.every

Return at L16: (p.x >= margin && p.x <= SIZE - margin && p.y >= margin && p.y <= SIZE - margin && blocks.every((b) => p.x < b.x - margin || p.x > b.x + b.w + margin || p.y < b.y - margin || p.y > b.y + b.h + margin,))

attackDepth [L31-53]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function attackDepth(side: Side, lane: number, blocks: readonly Block[] = BLOCKS,)

Direct writes: depth

Calls / delegates: laneCenter

Return at L52: depth

Decision branches: L41: c + SHAFT / 2 <= low || c - SHAFT / 2 >= high

localPoint [L54-62]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function localPoint(p: Point, side: Side): Point

Return at L55: side === 0 ? p : side === 1 ? { x: SIZE - p.x, y: p.y } : side === 2 ? { x: p.y, y: p.x } : { x: SIZE - p.y, y: p.x }

```
export function localPoint(p: Point, side: Side): Point { return side === 0 ? p : side === 1 ? { x: SIZE - p.x, y: p.y } : side === 2 ? { x: p.y, y: p.x } : { x: SIZE - p.y, y: p.x }; }
```

segmentDistance [L63-71]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function segmentDistance(p: Point, a: Point, b: Point)

Return at L70: Math.hypot(p.x - a.x - t * dx, p.y - a.y - t * dy)

pistonHits [L73-109]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function pistonHits(p: Point, side: Side, lane: number, length: number, radius = RADIUS,)

Calls / delegates: localPoint, laneCenter, segmentDistance

Return at L102: (Math.min(segmentDistance(q, a, b), segmentDistance(q, b, d), segmentDistance(q, d, a),) <= radius)

Decision branches: L80: length <= 0; L84: q.x + radius < 0 || q.x - radius > length || Math.abs(q.y - c) > SHAFT / 2 + radius; L92: base > 0 && Math.hypot(q.x - nearX, q.y - nearY) <= radius; L96: q.x >= base && q.x <= length && Math.abs(q.y - c) <= ((SHAFT / 2) * (length - q.x)) / 14

Execution order (direct statements; nested callbacks stay within their owner):

L80-80 IfStatement: if (length <= 0) return false;

L81-83 FirstStatement: const q = localPoint(p, side), c = laneCenter(lane),

L84-89 IfStatement: if (q.x + radius < 0 ||

L90-91 FirstStatement: const nearX = Math.max(0, Math.min(base, q.x)), nearY = Math.max(c - SHAFT / 2, Math.min(c + SHAFT / 2, q.y));

L92-92 IfStatement: if (base > 0 && Math.hypot(q.x - nearX, q.y - nearY) <= radius) return true;

L93-95 FirstStatement: const a = { x: base, y: c - SHAFT / 2 }, b = { x: length, y: c },

L96-101 IfStatement: if (q.x >= base &&

L102-108 ReturnStatement: return (Math.min(

safeFrom [L110-123]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function safeFrom(p: Point, side: Side, margin = 0.5, blocks: readonly Block[] = BLOCKS,)

Calls / delegates: clearPoint, [0, 1, 2, 3].every, pistonHits, attackDepth

Return at L116: (clearPoint(p, RADIUS + 0.5, blocks) && [0, 1, 2, 3].every((l) => !pistonHits(p, side, l, attackDepth(side, l, blocks), RADIUS + margin),))

movePlayer [L125-170]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function movePlayer(p: Point, dx: number, dy: number, dt: number, blocks: readonly Block[] = BLOCKS,)

Direct writes: p.x, p.y

Decision branches: L133: !len; L143: p.y > b.y - RADIUS && p.y < b.y + b.h + RADIUS && p.x > b.x - RADIUS && p.x < b.x + b.w + RADIUS; L157: p.x > b.x - RADIUS && p.x < b.x + b.w + RADIUS && p.y > b.y - RADIUS && p.y < b.y + b.h + RADIUS

Execution order (direct statements; nested callbacks stay within their owner):

L132-132 FirstStatement: const len = Math.hypot(dx, dy);

L133-133 IfStatement: if (!len) return;

L134-135 FirstStatement: const distance = Math.min(SPEED * dt, len), parts = Math.max(1, Math.ceil(distance / 3));

L136-137 FirstStatement: const sx = ((dx / len) * distance) / parts, sy = ((dy / len) * distance) / parts;

L138-169 ForStatement: for (let i = 0; i < parts; i++) { const oldX = p.x,

src/games/spike-attacks/MapPreview.ts

ACTIVE SOURCE / tool or application | 17 lines | SHA-256 69d5ea0a8a3492f0...

Creates a lightweight static SVG map preview for eager metadata.

Imports: ./Maps

mapPreview [L3-16]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function mapPreview(map?: SpikeMap)

Direct writes: tiles

Calls / delegates: n.toString(16).padStart, n.toString, hex, blocks.map((b) => `<rect x="${b.x + 20}" y="${b.y + 23}" width="${b.w}" height="${b.h}" rx="8" fill="#66546f"/><rect x="${b.x + 20}" y="${b.y + 20}" width="${b.w}" height="${b.h}" rx="8" fill="#93809c" stroke="#242b32" stroke-width="7"/>`).join, blocks.map

Return at L15: `<svg class="map-preview" width="80" height="80" viewBox="0 0 360 360" aria-hidden="true"><rect x="11" y="11" width="338" height="338" rx="15" fill="#242b32"/>${tiles}${blocks.map((b) => `<rect x="${b.x + 20}" y="${b.y + 23}" width="${b.w}" height="${b.h}" rx="8" fill="#66546f"/><rect x="${b.x + 20}" y="${b.y + 20}" width="${b.w}" height="${b.h}" r ... [continued in source reference]

mapPreview.hex [L10-10]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: hex = (n: number) =>

```
hex = (n: number) => `#${n.toString(16).padStart(6, "0")}`
```

src/games/spike-attacks/Maps.ts

ACTIVE SOURCE / tool or application | 56 lines | SHA-256 e9de3d3d9bcf9277...

Lightweight map definitions and preview data; canonical selected geometry is shared by rules, AI and rendering.

Imports: ./ArenaPhysics

Complete geometry / route parameter listing

These exact coordinates and parameters define the data used by the module. Edit the data owner and verify dependent collision, rendering and AI routes together.

```
 1 | import type { Block, Point } from "./ArenaPhysics";
 2 | export interface SpikeMap {
 3 |  id: string;
 4 |  name: string;
 5 |  description: string;
 6 |  blocks: readonly Block[];
 7 |  spawns: readonly [Point, Point];
 8 |  floor: readonly [number, number];
 9 | }
10 | const spawns = [
11 |  { x: 108, y: 208 },
12 |  { x: 212, y: 208 },
13 | ] as const;
14 | export const MAPS: readonly SpikeMap[] = [
15 |  {
16 |    id: "courtyard",
17 |    name: "Courtyard",
18 |    description: "Three staggered shelters.",
19 |    floor: [0xf0c4a2, 0xe7af98],
20 |    spawns,
21 |    blocks: [
22 |      { x: 0, y: 0, w: 72, h: 72 },
23 |      { x: 160, y: 80, w: 72, h: 72 },
24 |      { x: 240, y: 240, w: 80, h: 80 },
25 |    ],
26 |  },
27 |  {
28 |    id: "split-steps",
29 |    name: "Split Steps",
30 |    description: "A reversed diagonal escape.",
31 |    floor: [0xd6e3c7, 0xbdd3b7],
32 |    spawns,
33 |    blocks: [
34 |      { x: 0, y: 240, w: 72, h: 80 },
35 |      { x: 80, y: 80, w: 72, h: 72 },
36 |      { x: 240, y: 0, w: 80, h: 72 },
37 |    ],
38 |  },
39 |  {
40 |    id: "crosswind",
41 |    name: "Crosswind",
42 |    description: "Cover shifts across the arena.",
```

```
43 |    floor: [0xd7d7ea, 0xbfc7df],
44 |    spawns,
45 |    blocks: [
46 |      { x: 0, y: 80, w: 72, h: 72 },
47 |      { x: 160, y: 0, w: 72, h: 72 },
48 |      { x: 240, y: 240, w: 80, h: 80 },
49 |    ],
50 |  },
51 | ];
52 | export function selectMap(id: string | undefined, seed: number): SpikeMap {
53 |  if (id === "random") return MAPS[(seed >>> 0) % MAPS.length];
54 |  return MAPS.find((map) => map.id === id) ?? MAPS[0];
55 | }
```

Module constants and data

L10-13 spawns = [{ x: 108, y: 208 }, { x: 212, y: 208 },] as const

L14-51 MAPS = [{ id: "courtyard", name: "Courtyard", description: "Three staggered shelters.", floor: [0xf0c4a2, 0xe7af98], spawns, blocks: [{ x: 0, y: 0, w: 72, h: 72 }, { x: 160, y: 80, w: 72, h: 72 }, { x: 240, y: 240, w: 80, h: 80 },], }, { id: "split-steps", name: "Split Steps", description: "A reversed diagonal escape.", floor: [0xd6e3c7, 0xbdd3b7], spawns, blocks: [{ x: 0, y: 240, w: 72, h: 80 }, { x: 80, y: 80, w: 72, h: 72 }, { x: 240, y: 0, w: 80, h: 72 },], }, { id: "crosswind", name: "Crosswind", description: "Cover shifts across the arena.", floor: [0xd7d7ea, 0xbfc7df], spawns, blocks: [{ x: 0, y: 80, w: 72, h: 72 }, { x: 160, y: 0, w: 72, h: 72 }, { x: 240, y: 240, w: 80, h: 80 },], },]

SpikeMap [L2-9]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface SpikeMap { id: string; name: string; description: string; blocks: readonly Block[]; spawns: readonly [Point, Point]; floor: readonly [number, number]; }
```

selectMap [L52-55]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function selectMap(id: string | undefined, seed: number): SpikeMap

Calls / delegates: MAPS.find

Return at L53: MAPS[(seed >>> 0) % MAPS.length]

Return at L54: MAPS.find((map) => map.id === id) ?? MAPS[0]

Decision branches: L53: id === "random"

src/games/spike-attacks/Rules.ts

ACTIVE SOURCE / tool or application | 170 lines | SHA-256 f5583cc2f384c3a5...

Owns seeded piston phases, legal targets, survival rounds and independent foreground deadline.

Imports: ./Maps, ../../shared/ai/MatchRandom, ./ArenaPhysics

State, resources and initial values

Line	Field / declaration
13	readonly map: SpikeMap;
14	players: [Survivor, Survivor];
15	scores: [number, number] = [0, 0];
16	winner: 1 | 2 | "draw" | null = null;
17	wave = 1;
18	round = 1;
19	direction: Side = 0;
20	phase: Phase = "warning";
21	phaseAge = 0;
22	warning = 1.85;
23	between = 0;
24	elapsed = 0;
25	total = 0;
26	readonly lengths = [0, 0, 0, 0];
27	private rng: MatchRandom;

Phase [L4-4]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Phase = "warning" | "extend" | "hold" | "retract" | "recover";
```

Survivor [L5-11]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Survivor { x: number; y: number; tx: number; ty: number; alive: boolean; }
```

SpikeMatch [L12-169]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SpikeMatch.constructor [L28-36]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly seed: number, mapId?: string,)

Direct writes: this.map, this.players, this.rng, this.direction

Calls / delegates: selectMap, this.player, MatchRandom, this.rng.next

SpikeMatch.player [L37-44]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private player(i: number): Survivor

Return at L38: { ...this.map.spawns[i], tx: this.map.spawns[i].x, ty: this.map.spawns[i].y, alive: true, }

```
private player(i: number): Survivor { return { ...this.map.spawns[i], tx: this.map.spawns[i].x, ty:
this.map.spawns[i].y, alive: true, }; }
```

SpikeMatch.warningDuration [L45-47]

Compute the visible warning interval from the current wave.

Signature: warningDuration()

Return at L46: Math.max(1.05, 1.85 - (this.wave - 1) * 0.045)

SpikeMatch.extensionSpeed [L48-50]

Compute the bounded piston extension speed from the current wave.

Signature: extensionSpeed()

Return at L49: Math.min(940, 620 + (this.wave - 1) * 18)

SpikeMatch.remaining [L51-53]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get remaining()

Return at L52: Math.max(0, 120 - this.elapsed)

SpikeMatch.advanceClock [L54-64]

Advance the separate uncapped foreground match deadline and determine expiry result.

Signature: advanceClock(dt: number)

Direct writes: this.elapsed, this.winner

Decision branches: L55: this.winner; L57: this.elapsed >= 120

SpikeMatch.target [L65-75]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: target(i: 0 | 1, x: number, y: number)

Direct writes: this.players[i].tx, this.players[i].ty

Decision branches: L66: this.winner || this.between || !this.players[i].alive || !Number.isFinite(x + y)

SpikeMatch.stop [L76-79]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: stop(i: 0 | 1)

Direct writes: this.players[i].tx, this.players[i].ty

SpikeMatch.nextWave [L80-89]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private nextWave()

Direct writes: this.direction, this.phase, this.phaseAge, this.warning

Calls / delegates: this.rng.next, this.warningDuration, this.lengths.fill

SpikeMatch.step [L90-98]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number)

Direct writes: left

Calls / delegates: this.tick

SpikeMatch.tick [L99-168]

Advance warning/extension/hold/retraction/recovery, legal player motion, swept piston danger and survival points.

Signature: private tick(dt: number)

Direct writes: this.total, this.between, this.players, this.phaseAge, this.warning, this.phase, this.lengths[i], p.alive, this.winner

Calls / delegates: this.player, this.nextWave, this.lengths.slice, this.warningDuration, attackDepth, this.extensionSpeed, this.lengths.fill, movePlayer, pistonHits, this.players.some, this.scores.some

Decision branches: L100: this.winner; L102: this.between; L104: this.between < 1e-8; L114: this.phase === "warning"; L116: this.warning < 1e-8; L121: this.phase === "extend"; L127: this.phaseAge >= 320 / this.extensionSpeed(); L131: this.phase === "hold"; L132: this.phaseAge >= 0.32 + Math.min(0.18, this.wave * 0.008); L136: this.phase === "retract"; L142: t === 1; L147: this.phaseAge >= 0.18; L151: pistonHits(p, this.direction, lane, Math.max(old[lane], this.lengths[lane]),); L161: this.players.some((p) => !p.alive); L162: this.players[0].alive !== this.players[1].alive; L165: this.scores.some((s) => s >= 5)

Execution order (direct statements; nested callbacks stay within their owner):

L100-100 IfStatement: if (this.winner) return;

L101-101 ExpressionStatement: this.total += dt;

L102-111 IfStatement: if (this.between) { this.between = Math.max(0, this.between - dt);

L112-112 FirstStatement: const old = this.lengths.slice();

L113-113 ExpressionStatement: this.phaseAge += dt;

L114-147 IfStatement: if (this.phase === "warning") { this.warning = Math.max(0, this.warningDuration() - this.phaseAge);

L148-160 ForOfStatement: for (const p of this.players) { movePlayer(p, p.tx - p.x, p.ty - p.y, dt, this.map.blocks);

L161-167 IfStatement: if (this.players.some((p) => !p.alive)) { if (this.players[0].alive !== this.players[1].alive)

src/games/spike-attacks/SpikeArt.ts

ACTIVE SOURCE / tool or application | 177 lines | SHA-256 f8f4434855590400...

Spike Attacks original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: phaser, ../../core/ThemeManager, ../../shared/components/drawing, ./ArenaPhysics, ./Rules, ./SpikeView

Module constants and data

L7-7 BOARD = { x: 62, y: 226, scale: 296 / SIZE }

L8-11 JOYS = [{ x: 210, y: 595 }, { x: 210, y: 158 },]

Stick [L12-12]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Stick = { x: number; y: number; active: boolean };
```

edge [L13-24]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function edge(g: Phaser.GameObjects.Graphics, side: number)

Calls / delegates: g.translateCanvas, g.scaleCanvas, g.rotateCanvas

Decision branches: L14: side === 1; L17: side === 2; L20: side === 3

drawArena [L25-176]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function drawArena(g: Phaser.GameObjects.Graphics, m: SpikeMatch, v: ReturnType<SpikeView["sample"]>, sticks: Stick[], solo = false,)

Calls / delegates: rounded, g.fillStyle(0xffffff, 0.12).fillCircle, g.fillStyle, g.save, g.translateCanvas, g.scaleCanvas, g.fillStyle(m.map.floor[(x + y) % 2]).fillRect, g.lineStyle(1.4, 0xffe4bd, 0.4).strokeRect, g.lineStyle, edge, laneCenter, g.fillStyle(0x303d45).fillRect, g.fillStyle(warning ? 0xf4b55f : active && m.phase !== "recover" ? 0xee8d73 : 0x91aaa7, warning ? 0.65 + 0.35 * Math.sin(m.phaseAge * 10) ** 2 : 1,).fillRect, g.fillStyle(0x242b32, 0.15).fillRect, g.fillStyle(0xd6a270).fillRect, g.fillStyle(0x93633e, 0.35).fillRect, g.lineStyle(2, theme.ink).lineBetween, g.fillStyle(0xeaf1eb).fillPoints, g.lineStyle(2.5, theme.ink).strokePoints, g.lineStyle(1.5, 0xb7c8cc).lineBetween, g.restore, g.lineStyle(2, 0xc2b1c5, 0.7).lineBetween, g.fillStyle(theme.ink, 0.15).fillEllipse, g.rotateCanvas, circle, g.fillStyle(theme.ink).fillCircle(-2, -4, 1.4).fillCircle, g.fillStyle(theme.ink).fillCircle, g.lineStyle(2.5, color, 0.8).strokeCircle, g.lineStyle(2, color, 0.4).strokeCircle, g.fillStyle(theme.ink, 0.28).fillCircle, g.fillStyle(theme.white, 0.3).fillEllipse

Decision branches: L143: !p.alive; L153: i === 1 && solo

Execution order (direct statements; nested callbacks stay within their owner):

L32-32 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xd2dfe3, 4);

L33-34 ForStatement: for (let i = 0; i < 7; i++) g.fillStyle(0xffffff, 0.12).fillCircle(45 + i * 53, 378 + (i % 2) * 45, 25);

L35-35 ExpressionStatement: g.save();

L36-36 ExpressionStatement: g.translateCanvas(BOARD.x, BOARD.y);

L37-37 ExpressionStatement: g.scaleCanvas(BOARD.scale, BOARD.scale);

L38-38 ExpressionStatement: rounded(g, -9, -9, 338, 338, 10, 0x39434b, 3);

L39-48 ForStatement: for (let y = 0; y < 4; y++) for (let x = 0; x < 4; x++) {

L50-118 ForStatement: for (let side = 0; side < 4; side++) { g.save();

L119-128 ForOfStatement: for (const b of m.map.blocks) { rounded(g, b.x + 2, b.y + 5, b.w, b.h, 7, 0x664f6b, 2);

L129-150 ForOfStatement: for (const i of [0, 1] as const) { const p = v.players[i],

L151-151 ExpressionStatement: g.restore();

L152-175 ForOfStatement: for (const i of [0, 1] as const) { if (i === 1 && solo) continue;

src/games/spike-attacks/SpikeControls.ts

ACTIVE SOURCE / tool or application | 98 lines | SHA-256 392344e8345da3bb...

Maps keyboard/gamepad direction, dead zones, stable slots and neutral-after-pause behavior.

State, resources and initial values

Line	Field / declaration
44	private slots = new Map<number, 0 | 1>();
45	private neutralRequired = new Set<number>();

Module constants and data

L2-11 KEY_CODES = ["KeyW", "KeyA", "KeyS", "KeyD", "ArrowUp", "ArrowLeft", "ArrowDown", "ArrowRight",]

Direction [L1-1]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Direction = { x: number; y: number };
```

ZERO [L12-12]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: ZERO = (): Direction =>

```
ZERO = (): Direction => ({ x: 0, y: 0 })
```

keyboardDirection [L13-28]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function keyboardDirection(keys: ReadonlySet<string>, player: 0 | 1, solo: boolean,): Direction

Calls / delegates: Number, keys.has

Return at L20: { x: Number((wasd && keys.has("KeyD")) || (arrows && keys.has("ArrowRight"))) - Number((wasd && keys.has("KeyA")) || (arrows && keys.has("ArrowLeft"))), y: Number((wasd && keys.has("KeyS")) || (arrows && keys.has("ArrowDown"))) - Number((wasd && keys.has("KeyW")) || (arrows && keys.has("ArrowUp"))), }

Pad [L29-29]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
type Pad = Pick<Gamepad, "index" | "connected" | "axes" | "buttons">;
```

direction [L30-41]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function direction(pad: Pad): Direction

Calls / delegates: Number, pressed, ZERO

Return at L34: { x: dx, y: dy }

Return at L38: ZERO()

Return at L40: { x: x * scale, y: y * scale }

Decision branches: L34: dx || dy; L38: length < 0.2

direction.pressed [L31-31]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: pressed = (i: number) =>

```
pressed = (i: number) => pad.buttons[i]?.pressed === true
```

SpikeGamepads [L43-97]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SpikeGamepads.read [L46-77]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: read(pads: readonly (Pad | null)[], solo: boolean): [Direction, Direction]

Direct writes: slot, output[slot]

Calls / delegates: ZERO, pads .filter((p): p is Pad => !!p?.connected) .sort, pads .filter, this.slots.keys, connected.some, this.slots.delete, this.neutralRequired.delete, this.slots.get, this.slots.values, occupied.includes, this.slots.set, direction, this.neutralRequired.has

Return at L76: output

Decision branches: L52: !connected.some((p) => p.index === index); L58: slot === undefined; L65: slot === undefined; L68: solo && slot === 1; L70: this.neutralRequired.has(pad.index); L71: !d.x && !d.y

SpikeGamepads.poll [L78-89]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: poll(solo: boolean): [Direction, Direction]

Calls / delegates: this.read, navigator.getGamepads, ZERO

Return at L80: this.read(typeof navigator !== "undefined" && navigator.getGamepads ? [...navigator.getGamepads()] : [], solo,)

Return at L87: [ZERO(), ZERO()]

SpikeGamepads.pause [L90-92]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Calls / delegates: this.slots.keys, this.neutralRequired.add

SpikeGamepads.dispose [L93-96]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.slots.clear, this.neutralRequired.clear

src/games/spike-attacks/SpikeGame.ts

ACTIVE SOURCE / tool or application | 225 lines | SHA-256 d5f0f080b1a97ee4...

Spike Attacks lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../shared/physics/FixedStep, ../../shared/ai/MatchRandom, ./Rules, ./AI, ./SpikeView, ./SpikeControls, ./SpikeArt

State, resources and initial values

Line	Field / declaration
11	private ctx!: GameContext;
12	private m = new SpikeMatch(freshSeed());
13	private ai?: SpikeAI;
14	private g!: Phaser.GameObjects.Graphics;
15	private clip!: Phaser.GameObjects.Graphics;
16	private mask!: Phaser.Display.Masks.GeometryMask;
17	private fixed = new FixedStep();
18	private view = new SpikeView();
19	private pose?: ReturnType<SpikeView["sample"]>;
20	private keys = new Set<string>();
21	private pads = new SpikeGamepads();
22	private movement = [{ x: 0, y: 0, source: "none" }, { x: 0, y: 0, source: "none" },];
26	private sticks: Stick[] = [{ x: 0, y: 0, active: false }, { x: 0, y: 0, active: false },];
30	private paused = false;
31	private done = false;
32	private scores = "";
33	private finishAge = 0;

SpikeGame [L10-224]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SpikeGame.initialize [L34-76]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.clip, this.mask, this.ai, this.m, this.sticks[i].active, this.keys, this.pose

Calls / delegates: ctx.scene.add.graphics, ctx.scene.make.graphics, this.clip.fillStyle(0xffffff).fillRoundedRect, this.clip.fillStyle, this.clip.createGeometryMask, this.g.setMask, SpikeAI, SpikeMatch, freshSeed, ctx.input.drag, this.stick, this.release, ctx.input.keys, ctx.setHint, ctx.setScore, ctx.setClock, this.view.reset, this.view.sample, this.draw, ctx.audio.play

Decision branches: L41: ctx.options.mode === "ai"; L49: i === 1 && this.ai; L53: this.paused || this.done || Math.hypot(x - center.x, y - center.y) > 58

Execution order (direct statements; nested callbacks stay within their owner):

L35-35 ExpressionStatement: this.ctx = ctx;

L36-36 ExpressionStatement: this.g = ctx.scene.add.graphics();

L37-37 ExpressionStatement: this.clip = ctx.scene.make.graphics({ x: 0, y: 0 });

L38-38 ExpressionStatement: this.clip.fillStyle(0xffffff).fillRoundedRect(22, 110, 376, 542, 22);

L39-39 ExpressionStatement: this.mask = this.clip.createGeometryMask();

L40-40 ExpressionStatement: this.g.setMask(this.mask);

L41-43 IfStatement: if (ctx.options.mode === "ai") { this.ai = new SpikeAI(ctx.options.difficulty);

L44-47 ExpressionStatement: this.m = new SpikeMatch(this.ai?.personality.seed ?? freshSeed(),

L48-67 ForOfStatement: for (const i of [0, 1] as const) { if (i === 1 && this.ai) continue;

L68-68 ExpressionStatement: this.keys = ctx.input.keys(KEY_CODES);

L69-69 ExpressionStatement: ctx.setHint("");

L70-70 ExpressionStatement: ctx.setScore(0, 0);

L71-71 ExpressionStatement: ctx.setClock("2:00", false);

L72-72 ExpressionStatement: this.view.reset(this.m);

L73-73 ExpressionStatement: this.pose = this.view.sample(this.m, 1, 0);

L74-74 ExpressionStatement: this.draw();

L75-75 ExpressionStatement: ctx.audio.play("start");

SpikeGame.stick [L77-84]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private stick(i: 0 | 1, x: number, y: number)

Direct writes: this.sticks[i].x, this.sticks[i].y

SpikeGame.release [L85-88]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private release(i: 0 | 1)

Direct writes: this.sticks[i]

Calls / delegates: this.m.stop

SpikeGame.update [L89-165]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.movement[i], this.scores, this.pose, this.finishAge, this.done

Calls / delegates: this.m.advanceClock, this.pads.poll, keyboardDirection, this.fixed.advance, this.view.capture, this.m.target, this.m.stop, this.ai?.update, this.m.players.map, this.m.step, this.ctx.audio.play, this.view.reset, this.m.scores.join, this.ctx.setScore, this.ctx.setClock, this.view.sample, this.ctx.reportAI, this.draw, this.ctx.finish

Decision branches: L90: this.paused || this.done; L94: i === 1 && this.ai; L112: i === 1 && this.ai; L114: x || y; L127: this.m.phase !== phase; L128: this.m.phase === "extend"; L129: this.m.phase === "hold"; L130: this.m.phase === "retract"; L131: this.m.phase === "warning"; L133: round !== this.m.round; L138: alive[i] && !this.m.players[i].alive; L141: this.m.scores.join() !== this.scores; L142: this.scores; L158: this.m.winner; L160: this.finishAge > 0.7

Execution order (direct statements; nested callbacks stay within their owner):

L90-90 IfStatement: if (this.paused || this.done) return;

L91-91 ExpressionStatement: this.m.advanceClock(dt);

L92-92 FirstStatement: const pads = this.pads.poll(!!this.ai);

L93-108 ForOfStatement: for (const i of [0, 1] as const) { if (i === 1 && this.ai) continue;

L109-140 ExpressionStatement: this.fixed.advance(dt, (h) => { this.view.capture(this.m);

L141-145 IfStatement: if (this.m.scores.join() !== this.scores) { if (this.scores) this.ctx.audio.play("goal", 0.65);

L146-146 FirstStatement: const seconds = Math.ceil(this.m.remaining);

L147-150 ExpressionStatement: this.ctx.setClock(`${Math.floor(seconds / 60)}:${String(seconds % 60).padStart(2, "0")}`,

L151-155 ExpressionStatement: this.pose = this.view.sample(this.m,

L156-156 ExpressionStatement: this.ctx.reportAI(this.ai?.computationMs ?? 0);

L157-157 ExpressionStatement: this.draw();

L158-164 IfStatement: if (this.m.winner) { this.finishAge += dt;

SpikeGame.draw [L166-179]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Calls / delegates: this.g.clear, drawArena, this.sticks.map

SpikeGame.pause [L180-191]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused, d.x, d.y, d.source

Calls / delegates: this.fixed.reset, this.pads.pause, this.movement.forEach, this.release

SpikeGame.resume [L192-194]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

SpikeGame.dispose [L195-201]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.done

Calls / delegates: this.pads.dispose, this.g.destroy, this.mask.destroy, this.clip.destroy

SpikeGame.snapshot [L202-223]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.personality.snapshot

Return at L203: { sharedArena: true, map: this.m.map.id, visibleControls: this.ai ? [0] : [0, 1], players: this.m.players, scores: this.m.scores, warning: this.m.warning, direction: this.m.direction, wave: this.m.wave, round: this.m.round, phase: this.m.phase, lengths: this.m.lengths, between: this.m.between, winner: this.m.winner, remaining: this.m.remaining, sti ... [continued in source reference]

src/games/spike-attacks/SpikeView.ts

ACTIVE SOURCE / tool or application | 48 lines | SHA-256 7533dba51d343280...

Interpolates player positions, facing and piston extension for drawing only.

Imports: ./Rules

State, resources and initial values

Line	Field / declaration
4	private previous = [{ x: 108, y: 208 }, { x: 212, y: 208 },];
8	private lengths = [0, 0, 0, 0];
9	private angles = [0, 0];

SpikeView [L3-47]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SpikeView.capture [L10-14]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: capture(m: SpikeMatch)

Calls / delegates: this.lengths.splice

SpikeView.reset [L15-18]

Reset this component's transient state for a new round/attempt.

Signature: reset(m: SpikeMatch)

Direct writes: this.angles

Calls / delegates: this.capture

SpikeView.sample [L19-46]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: sample(m: SpikeMatch, alpha: number, dt: number)

Direct writes: this.angles[i]

Calls / delegates: m.lengths.map, m.players.map

Return at L21: { lengths: m.lengths.map((n, i) => this.lengths[i] + (n - this.lengths[i]) * t,), players: m.players.map((p, i) => { const dx = p.tx - p.x, dy = p.ty - p.y, moving = Math.hypot(dx, dy) > 1; if (moving) { const goal = Math.atan2(dy, dx) + Math.PI / 2; const delta = Math.atan2(Math.sin(goal - this.angles[i]), Math.cos(goal - this.angles[i]),); th ... [continued in source reference]

Return at L37: { ...p, x: this.previous[i].x + (p.x - this.previous[i].x) * t, y: this.previous[i].y + (p.y - this.previous[i].y) * t, angle: this.angles[i], moving, }

Decision branches: L29: moving

src/games/spike-attacks/config.ts

ACTIVE SOURCE / tool or application | 33 lines | SHA-256 b0265cd77b617be4...

Eager lightweight metadata for Spike Attacks: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./MapPreview, ./Maps, ./assets/preview.svg?raw

Top-level block L5-32

```
export default {
  id: "spike-attacks",
  name: "Spike Attacks",
  description: "Read the warning. Find your cover.",
  instructions:
    "Drag your joystick: Orange below, Blue above. Hide behind blocks to avoid the spikes. The flashing edge warns
before a strike; spikes return to their sockets. Attacks get faster. Last survivor scores; simultaneous hits replay.
First to 5 or the lead at 2:00; ties draw. Solo: WASD or arrows. Local: Orange WASD, Blue arrows. Gamepads: left
stick or D-pad; first pad Orange, second Blue.",
  color: "#e5dcc5",
  accent: "#a3aaa2",
  preview,
  order: 20,
  tag: "TAKE COVER",
  variants: [
    {
      id: "random",
      name: "Random map",
      description: "A fresh arena each match.",
      preview: mapPreview(),
    },
    ...MAPS.map((m) => ({
      id: m.id,
      name: m.name,
      description: m.description,
      preview: mapPreview(m),
    })),
  ],
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/spike-attacks/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 6c34a44f65908650...

Side-effect-free lazy factory for Spike Attacks; implementation initializes only after host selection.

Imports: ./SpikeGame

Top-level block L2-2

```
export default () => new SpikeGame();
```

src/games/tic-tac-toe/classic

src/games/tic-tac-toe/classic/ClassicAI.ts

ACTIVE SOURCE / tool or application | 131 lines | SHA-256 0f419daa34041578...

Performs Classic minimax/alpha-beta and difficulty/personality-aware legal move selection.

Imports: ../../../shared/ai/MatchRandom, ../../../shared/contracts/AIController, ../../../shared/contracts/GameResult, ../shared/lines, ./ClassicRules

State, resources and initial values

Line	Field / declaration
90	readonly personality: MatchRandom;
91	readonly delay: number;

Module constants and data

L6-6 ORDER = [4, 0, 2, 6, 8, 1, 3, 5, 7]

classicMove [L8-86]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function classicMove(board: TicTacToeBoard, difficulty: Difficulty, random = Math.random, preference: (move: number) => number = () => 1,): number

Direct writes: score, cells[i], value, alpha, beta, best, choices

Calls / delegates: ORDER.filter, random, line.every, cells.every, line.filter, search, choices.push, choices.map, weights.reduce, choices.at

Return at L17: -1

Return at L19: legal[Math.floor(random() * legal.length)]

Return at L37: p === player ? 100 + depth : -100 - depth

Return at L48: score

Return at L65: value

Return at L83: choices[i]

Return at L85: choices.at(-1)!

Decision branches: L17: board.winner || !legal.length; L18: difficulty === "medium" && random() < 0.32; L36: p && line.every((i) => cells[i] === p); L39: cells.every(Boolean); L40: depth === 0; L45: !other; L46: !own; L52: cells[i]; L56: turn === player; L63: beta <= alpha; L73: score > best; L76: score === best; L83: value <= 0

Execution order (direct statements; nested callbacks stay within their owner):

L14-15 FirstStatement: const cells = [...board.cells], player = board.turn;

L16-16 FirstStatement: const legal = ORDER.filter((i) => !cells[i]);

L17-17 IfStatement: if (board.winner || !legal.length) return -1;

L18-19 IfStatement: if (difficulty === "medium" && random() < 0.32) return legal[Math.floor(random() * legal.length)];

L20-27 FirstStatement: const limit = difficulty === "impossible"

L28-66 FunctionDeclaration: function search(turn: Player,

L67-67 FirstStatement: let best = -Infinity;

L68-68 FirstStatement: let choices: number[] = [];

L69-77 ForOfStatement: for (const i of legal) { cells[i] = player;

L78-79 FirstStatement: const weights = choices.map(preference), total = weights.reduce((a, b) => a + b, 0);

L80-80 FirstStatement: let value = random() * total;

L81-84 ForStatement: for (let i = 0; i < choices.length; i++) { value -= weights[i];

L85-85 ReturnStatement: return choices.at(-1)!;

classicMove.search [L28-66]

Explore a bounded game tree with the local turn/evaluation policy.

Signature: function search(turn: Player, depth: number, alpha: number, beta: number,): number

Direct writes: score, cells[i], value, alpha, beta

Calls / delegates: line.every, cells.every, line.filter, search

Return at L37: p === player ? 100 + depth : -100 - depth

Return at L48: score

Return at L65: value

Decision branches: L36: p && line.every((i) => cells[i] === p); L39: cells.every(Boolean); L40: depth === 0; L45: !other; L46: !own; L52: cells[i]; L56: turn === player; L63: beta <= alpha

Execution order (direct statements; nested callbacks stay within their owner):

L34-38 ForOfStatement: for (const line of WIN_LINES) { const p = cells[line[0]];

L39-39 IfStatement: if (cells.every(Boolean)) return 0;

L40-49 IfStatement: if (depth === 0) { let score = 0;

L50-50 FirstStatement: let value = turn === player ? -Infinity : Infinity;

L51-64 ForOfStatement: for (const i of ORDER) { if (cells[i]) continue;

L65-65 ReturnStatement: return value;

ClassicAI [L89-130]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ClassicAI.constructor [L92-102]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly difficulty: Difficulty, seed?: SeedInput,)

Direct writes: this.personality, this.delay

Calls / delegates: MatchRandom, this.personality.sample

ClassicAI.choose [L103-123]

Choose a legal AI action from observations and difficulty/search policy.

Signature: choose(board: TicTacToeBoard)

Calls / delegates: classicMove, MatchRandom, this.personality.turnSeed, board.cells.filter, [0, 2, 6, 8].includes, [1, 3, 5, 7].includes

Return at L105: classicMove(board, this.difficulty, new MatchRandom(this.personality.turnSeed(board.cells.filter(Boolean).length),).next, (move) => 1 + (role === 0 && move === 4 ? 3 : role === 1 && [0, 2, 6, 8].includes(move) ? 3 : role === 2 && [1, 3, 5, 7].includes(move) ? 3 : role === 3 && move % 3 === (this.personality.side < 0 ? 0 : 2) ? 3 : 0),)

ClassicAI.snapshot [L124-129]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L125: { ...this.personality.snapshot(), strategy: ["center", "corners", "edges", "flank"][this.personality.role], }

src/games/tic-tac-toe/classic/ClassicGame.ts

ACTIVE SOURCE / tool or application | 161 lines | SHA-256 1352e4af2040ffec...

Tic-Tac-Toe lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../../shared/components/TurnView, ../../../shared/ai/MatchRandom, ./ClassicAI, phaser, ../../../shared/contracts/GameModule, ../../../core/ThemeManager, ../../../shared/components/drawing, ./ClassicRules

State, resources and initial values

Line	Field / declaration
13	private ctx!: GameContext;
14	private ai?: ClassicAI;
15	private board = new TicTacToeBoard();
16	private graphics!: Phaser.GameObjects.Graphics;
17	private bg!: Phaser.GameObjects.Graphics;
18	private turn!: Phaser.GameObjects.Text;
19	private ages = Array(9).fill(0);
20	private endAge = 0;
21	private reported = false;
22	private paused = false;
23	private elapsed = 0;
24	private thinkFor = 0;

TicTacToe [L12-160]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TicTacToe.initialize [L25-53]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.board, this.ai, this.ages, this.bg, this.turn, this.graphics

Calls / delegates: TicTacToeBoard.restore, ClassicAI, restoredAISeed, this.board.cells.map, ctx.setScore, ctx.setHint, ctx.scene.add.graphics, label, ctx.input.tap, this.place

Decision branches: L28: ctx.options.mode === "ai"; L40: this.paused || (ctx.options.mode === "ai" && this.board.turn === 2) || this.elapsed < 0.2 || x < 48 || x >= 372 || y < 219 || y >= 543

TicTacToe.place [L54-61]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private place(cell: number)

Direct writes: this.ages[cell], this.thinkFor

Calls / delegates: this.board.play, this.ctx.audio.play, this.ctx.saveState, this.board.save, this.ai?.snapshot, this.refreshTurn

Decision branches: L55: !this.board.play(cell)

TicTacToe.refreshTurn [L62-83]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private refreshTurn()

Calls / delegates: this.bg.clear, rounded, turnBackground, this.bg .lineStyle(4, theme.ink) .lineBetween(156, 234, 156, 528) .lineBetween(264, 234, 264, 528) .lineBetween(63, 327, 357, 327) .lineBetween, this.bg .lineStyle(4, theme.ink) .lineBetween(156, 234, 156, 528) .lineBetween(264, 234, 264, 528) .lineBetween, this.bg .lineStyle(4, theme.ink) .lineBetween(156, 234, 156, 528) .lineBetween, this.bg .lineStyle(4, theme.ink) .lineBetween, this.bg .lineStyle, this.turn.setText

TicTacToe.update [L84-140]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.elapsed, this.thinkFor, this.ages[i], this.endAge, this.reported

Calls / delegates: this.refreshTurn, this.place, this.ai!.choose, g.clear, this.board.cells.forEach, symbol, g.lineStyle(8, theme.ink, 0.65).lineBetween, g.lineStyle, this.ctx.setScore, this.ctx.finish

Decision branches: L85: this.paused; L88: !this.board.winner && this.ctx.options.mode === "ai" && this.board.turn === 2; L94: this.thinkFor > Math.max(0.75, this.ai!.delay); L100: !p; L113: this.board.winner; L115: this.board.line; L128: this.endAge > 0.7 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L85-85 IfStatement: if (this.paused) return;

L86-86 ExpressionStatement: this.elapsed += dt;

L87-87 ExpressionStatement: this.refreshTurn();

L88-96 IfStatement: if (!this.board.winner &&

L97-97 FirstStatement: const g = this.graphics;

L98-98 ExpressionStatement: g.clear();

L99-112 ExpressionStatement: this.board.cells.forEach((p, i) => { if (!p) return;

L113-139 IfStatement: if (this.board.winner) { this.endAge += dt;

TicTacToe.pause [L141-143]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

TicTacToe.resume [L144-146]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

TicTacToe.dispose [L147-151]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.bg.destroy, this.graphics.destroy, this.turn.destroy

TicTacToe.snapshot [L152-159]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot

Return at L153: { ai: this.ai?.snapshot(), cells: [...this.board.cells], turn: this.board.turn, winner: this.board.winner, }

src/games/tic-tac-toe/classic/ClassicRules.ts

ACTIVE SOURCE / tool or application | 44 lines | SHA-256 9dc3f0969f20493a...

Owns Classic Tic-Tac-Toe legal moves, winning lines, draws and history validation.

Imports: ../../../shared/contracts/GameModule, ../shared/lines

State, resources and initial values

Line	Field / declaration
5	cells: (Player | null)[] = Array(9).fill(null);
6	turn: Player = 1;
7	winner: Winner | null = null;
8	line: readonly number[] | null = null;
9	history: number[] = [];

TicTacToeBoard [L4-43]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TicTacToeBoard.save [L10-12]

Create/persist the component checkpoint or record described by the returned expression.

Signature: save()

Return at L11: { version: 1, moves: [...this.history] }

```
save() { return { version: 1, moves: [...this.history] }; }
```

TicTacToeBoard.restore [L13-21]

Validate/replay saved state and fall back to a fresh legal instance when rejected.

Signature: static restore(data: unknown)

Calls / delegates: TicTacToeBoard, board.play

Return at L17: board

Return at L19: new TicTacToeBoard()

Return at L20: board

Decision branches: L16: d?.version !== 1 || !Array.isArray(d.moves) || d.moves.length > 9; L19: !board.play(move)

TicTacToeBoard.play [L22-42]

Apply a requested legal move or start a selected application module, depending on the owner.

Signature: play(index: number)

Direct writes: this.cells[index], this.winner, this.line, this.turn

Calls / delegates: this.history.push, line.every, this.cells.every

Decision branches: L23: !Number.isInteger(index) || index < 0 || index > 8 || this.cells[index] || this.winner; L34: line.every((i) => this.cells[i] === this.turn); L39: this.cells.every(Boolean)

Top-level block L3-3

```
export { WIN_LINES };
```

src/games/tic-tac-toe

src/games/tic-tac-toe/config.ts

ACTIVE SOURCE / tool or application | 29 lines | SHA-256 092be9346c29f70e...

Eager lightweight metadata for Tic-Tac-Toe: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-28

```
export default {
  id: "tic-tac-toe",
  name: "Tic-Tac-Toe",
  description: "The classic battle of wits.",
  instructions:
    "Take turns tapping an empty square. Orange is x and goes first. Blue is ○. Make a row of three to win.",
  color: "#f1b49f",
  accent: "#fae2d8",
  tag: "THINK AHEAD",
  order: 3,
  preview,
  variants: [
    {
      id: "classic",
      name: "Classic  -  3x3",
      description: "The familiar little rivalry.",
    },
    {
      id: "ultimate",
      name: "Ultimate  -  9x9",
      description: "Nine boards. One big strategy.",
      hasTutorial: true,
    },
  ],
  supportedModes: ["local-two-player", "single-player"],
} satisfies GameMetadata;
```

src/games/tic-tac-toe/index.ts

ACTIVE SOURCE / tool or application | 13 lines | SHA-256 90ccbab49b6969ae...

Lazily chooses the Classic or Ultimate factory and optional tutorial from the selected variant.

Imports: ../../shared/contracts/GameModule

tutorial [L9-12]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export async function tutorial(variant: string, ui: TutorialUI)

Calls / delegates: (await import("./ultimate/UltimateTutorial")).ultimateTutorial, import

Decision branches: L10: variant === "ultimate"

Top-level block L5-8

```
export default async (options: MatchOptions) =>
  options.variant === "ultimate"
    ? new (await import("./ultimate/UltimateGame")).UltimateGame()
    : new (await import("./classic/ClassicGame")).TicTacToe();
```

src/games/tic-tac-toe/shared

src/games/tic-tac-toe/shared/lines.ts

ACTIVE SOURCE / tool or application | 11 lines | SHA-256 218a4fa8bbf95dcd...

Defines the eight winning triples shared only within the Tic-Tac-Toe module.

Module constants and data

L1-10 WIN_LINES = [[0, 1, 2], [3, 4, 5], [6, 7, 8], [0, 3, 6], [1, 4, 7], [2, 5, 8], [0, 4, 8], [2, 4, 6],] as const

src/games/tic-tac-toe/ultimate

src/games/tic-tac-toe/ultimate/UltimateAI.ts

ACTIVE SOURCE / tool or application | 51 lines | SHA-256 99556e3d3f2ed304...

Wraps a cancellable Ultimate search worker with seed and difficulty inputs.

Imports: ../../../shared/ai/MatchRandom, ../../../shared/utilities/WorkerTask, ../../../shared/contracts/AIController, ./UltimateRules

State, resources and initial values

Line	Field / declaration
6	readonly personality: MatchRandom;
10	private task: WorkerTask< { id: number; state: unknown; difficulty: Difficulty; seed: number; replay: boolean; }, { move: number } >;

UltimateAI [L5-50]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

UltimateAI.snapshot [L7-9]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L8: this.personality.snapshot()

UltimateAI.constructor [L20-33]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, report: (ms: number) => void = () => {}, seed?: number,)

Direct writes: this.personality, this.task

Calls / delegates: MatchRandom, WorkerTask, Worker, URL

UltimateAI.choose [L34-43]

Choose a legal AI action from observations and difficulty/search policy.

Signature: async choose(board: UltimateBoard)

Calls / delegates: this.task.run, board.save, replayMode, board.legalMoves

Return at L42: answer?.move ?? board.legalMoves()[0] ?? -1

UltimateAI.cancel [L44-46]

Invalidate/cancel a gesture or asynchronous computation and clear pending state.

Signature: cancel()

Calls / delegates: this.task.cancel

UltimateAI.dispose [L47-49]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.task.cancel

src/games/tic-tac-toe/ultimate/UltimateGame.ts

ACTIVE SOURCE / tool or application | 315 lines | SHA-256 0c3c24cad14246bf...

Tic-Tac-Toe lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: ../../../shared/components/TurnView, ../../../shared/ai/MatchRandom, phaser, ../../../shared/contracts/GameModule, ../../../core/ThemeManager, ../../../shared/components/drawing, ./UltimateRules, ./UltimateAI

State, resources and initial values

Line	Field / declaration
13	private ctx!: GameContext;
14	private board = new UltimateBoard();
15	private g!: Phaser.GameObjects.Graphics;
16	private bg!: Phaser.GameObjects.Graphics;
17	private turn!: Phaser.GameObjects.Text;
18	private route!: Phaser.GameObjects.Text;
19	private focusLabel!: Phaser.GameObjects.Text;
20	private zone!: Phaser.GameObjects.Zone;
21	private ai?: UltimateAI;
22	private ages = Array(81).fill(0);
23	private paused = false;
24	private pending = false;
25	private disposed = false;
26	private generation = 0;
27	private thinking = 0;
28	private elapsed = 0;
29	private endAge = 0;
30	private reported = false;
31	private focused = false;
32	private focusAge = 1;
33	private claimedAges = Array(9).fill(1);

UltimateGame [L12-314]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

UltimateGame.initialize [L34-89]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.board, this.ages, this.ai, this.bg, this.turn, this.route, this.g, this.focusLabel, this.zone, this.focused, this.focusAge

Calls / delegates: UltimateBoard.restore, this.board.cells.map, UltimateAI, restoredAISeed, ctx.setScore, ctx.setHint, ctx.scene.add.graphics, rounded, label, ctx.scene.add .zone(210, 610, 220, 44) .setInteractive({ useHandCursor: true }) .on, ctx.scene.add .zone(210, 610, 220, 44) .setInteractive, ctx.scene.add .zone, ctx.audio.play, ctx.input.tap, this.place

Decision branches: L38: ctx.options.mode === "ai"; L56: this.board.forced !== null; L63: this.paused || this.elapsed < 0.2 || this.board.winner || (this.ai && this.board.turn === 2) || y < 214 || y >= 568 || x < 33 || x >= 387; L74: this.focused && this.board.forced !== null; L84: bx > 2 || by > 2 || lx >= 114 || ly >= 114

Execution order (direct statements; nested callbacks stay within their owner):

L35-35 ExpressionStatement: this.ctx = ctx;

L36-36 ExpressionStatement: this.board = UltimateBoard.restore(ctx.restoredState);

L37-37 ExpressionStatement: this.ages = this.board.cells.map((c) => (c ? 1 : 0));

L38-43 IfStatement: if (ctx.options.mode === "ai") this.ai = new UltimateAI(

L44-44 ExpressionStatement: ctx.setScore(0, 0);

L45-45 ExpressionStatement: ctx.setHint("");

L46-46 ExpressionStatement: this.bg = ctx.scene.add.graphics();

L47-47 ExpressionStatement: rounded(this.bg, 24, 112, 372, 538, 20, theme.peach, 4);

L48-48 ExpressionStatement: this.turn = label(ctx.scene, 210, 161, "", 12);

L49-49 ExpressionStatement: this.route = label(ctx.scene, 210, 188, "", 11);

L50-50 ExpressionStatement: this.g = ctx.scene.add.graphics();

L51-51 ExpressionStatement: this.focusLabel = label(ctx.scene, 210, 610, "", 13);

L52-61 ExpressionStatement: this.zone = ctx.scene.add .zone(210, 610, 220, 44)

L62-88 ExpressionStatement: ctx.input.tap((x, y) => { if (

UltimateGame.place [L90-101]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private place(move: number)

Direct writes: this.claimedAges[i], this.ages[move], this.thinking, this.focused

Calls / delegates: this.board.play, this.board.boards.forEach, this.ctx.audio.play, this.ctx.saveState, this.board.save, this.ai?.snapshot

Decision branches: L92: !this.board.play(move); L94: v && !before[i]; L100: this.board.forced === null

UltimateGame.update [L102-195]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.elapsed, this.focusAge, this.claimedAges, this.thinking, this.pending, this.ages, this.endAge, this.reported

Calls / delegates: this.claimedAges.map, this.ai.choose(this.board).then, this.ai.choose, this.place, this.bg.clear, rounded, turnBackground, this.turn.setText, this.route.setText, this.focusLabel.setText, this.g.setAlpha, this.ages.map, this.g.clear, this.drawBoard, this.g .lineStyle(7, theme.ink, 0.8) .lineBetween, this.g .lineStyle, this.ctx.finish

Decision branches: L103: this.paused || this.disposed; L107: this.ai && this.board.turn === 2 && !this.board.winner && !this.pending; L114: this.thinking > 0.75; L118: this.disposed || generation !== this.generation; L120: !this.paused; L160: focus !== null; L170: this.board.winner; L172: this.board.line; L186: this.endAge > 0.85 && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L103-103 IfStatement: if (this.paused || this.disposed) return;

L104-104 ExpressionStatement: this.elapsed += dt;

L105-105 ExpressionStatement: this.focusAge += dt;

L106-106 ExpressionStatement: this.claimedAges = this.claimedAges.map((a) => a + dt);

L107-123 IfStatement: if (this.ai &&

L124-124 ExpressionStatement: this.bg.clear();

L125-125 ExpressionStatement: rounded(this.bg, 24, 112, 372, 538, 20, turnBackground(this.board.turn), 4);

L126-136 ExpressionStatement: this.turn.setText(this.board.winner

L137-143 ExpressionStatement: this.route.setText(this.board.winner

L144-152 ExpressionStatement: this.focusLabel.setText(this.board.winner

L153-153 ExpressionStatement: this.g.setAlpha(0.65 + 0.35 * Math.min(1, this.focusAge / 0.16));

L154-154 ExpressionStatement: this.ages = this.ages.map((age, i) => (this.board.cells[i] ? age + dt : 0));

L155-155 ExpressionStatement: this.g.clear();

L156-159 FirstStatement: const focus = this.focused && this.board.forced !== null && !this.board.winner

L160-169 IfStatement: if (focus !== null) this.drawBoard(focus, 33, 214, 354, true); else

L170-194 IfStatement: if (this.board.winner) { this.endAge += dt;

UltimateGame.drawBoard [L196-279]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private drawBoard(b: number, x: number, y: number, size: number, active: boolean,)

Calls / delegates: rounded, g.lineStyle(2, this.board.turn === 1 ? theme.orange : theme.blue,).strokeRoundedRect, g.lineStyle, g.lineBetween, g.lineStyle(5, theme.muted) .lineBetween(x + size * 0.3, y + size * 0.44, x + size * 0.7, y + size * 0.44,) .lineBetween, g.lineStyle(5, theme.muted) .lineBetween, symbol

Decision branches: L224: active && !status; L239: status; L240: status === "draw"; L267: !mark

Execution order (direct statements; nested callbacks stay within their owner):

L203-205 FirstStatement: const g = this.g, status = this.board.boards[b],

L206-223 ExpressionStatement: rounded(g,

L224-229 IfStatement: if (active && !status) { g.lineStyle(

L230-234 ExpressionStatement: g.lineStyle(size > 114 ? 3 : 1.7,

L235-238 ForStatement: for (let k = 1; k < 3; k++) { g.lineBetween(x + k * cell, y + 7, x + k * cell, y + size - 7);

L239-264 IfStatement: if (status) { if (status === "draw") {

L265-278 ForStatement: for (let c = 0; c < 9; c++) { const mark = this.board.cells[b * 9 + c];

UltimateGame.pause [L280-285]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused, this.pending

Calls / delegates: this.ai?.cancel

UltimateGame.resume [L286-289]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused, this.thinking

UltimateGame.dispose [L290-300]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.disposed

Calls / delegates: this.ai?.dispose, this.g.destroy, this.bg.destroy, this.turn.destroy, this.route.destroy, this.focusLabel.destroy, this.zone.destroy

UltimateGame.snapshot [L301-313]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot

Return at L302: { ai: this.ai?.snapshot(), cells: [...this.board.cells], boards: [...this.board.boards], turn: this.board.turn, forced: this.board.forced, winner: this.board.winner, pending: this.pending, focused: this.focused, moves: this.board.history.length, }

src/games/tic-tac-toe/ultimate/UltimateRules.ts

ACTIVE SOURCE / tool or application | 89 lines | SHA-256 67e9fdf0c6abe678...

Owns 81-cell Ultimate routing, local/macro results, legal moves, cloning and replayed saves.

Imports: ../../../shared/contracts/GameResult, ../shared/lines

State, resources and initial values

Line	Field / declaration
9	cells: (Player | null)[] = Array(81).fill(null);
10	boards: (Winner | null)[] = Array(9).fill(null);
11	turn: Player = 1;
12	forced: number | null = null;
13	winner: Winner | null = null;
14	line: readonly number[] | null = null;
15	history: number[] = [];

UltimateSave [L3-6]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface UltimateSave { version: 1; moves: number[]; }
```

UltimateBoard [L8-88]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

UltimateBoard.legal [L16-26]

Enumerate/check legal actions under the current board and terminal state.

Signature: legal(move: number)

Return at L17: (Number.isInteger(move) && move >= 0 && move < 81 && !this.winner && !this.cells[move] && !this.boards[Math.floor(move / 9)] && (this.forced === null || Math.floor(move / 9) === this.forced))

UltimateBoard.legalMoves [L27-32]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: legalMoves()

Calls / delegates: this.legal, out.push

Return at L29: out

Return at L31: out

Decision branches: L29: this.winner; L30: this.legal(i)

UltimateBoard.play [L33-56]

Apply a requested legal move or start a selected application module, depending on the owner.

Signature: play(move: number)

Direct writes: this.cells[move], this.boards[b], this.winner, this.line, this.forced, this.turn

Calls / delegates: this.legal, this.history.push, WIN_LINES.some, line.every, this.cells.slice(b * 9, b * 9 + 9).every, this.cells.slice, this.boards.every

Decision branches: L34: !this.legal(move); L40: WIN_LINES.some((line) => line.every((i) => this.cells[b * 9 + i] === p)); L44: this.cells.slice(b * 9, b * 9 + 9).every(Boolean); L47: line.every((i) => this.boards[i] === p); L52: !this.winner && this.boards.every(Boolean)

UltimateBoard.clone [L57-67]

Create independent state for simulation/search without mutating the live instance.

Signature: clone()

Direct writes: b.cells, b.boards, b.turn, b.forced, b.winner, b.line, b.history

Calls / delegates: UltimateBoard

Return at L66: b

UltimateBoard.save [L68-70]

Create/persist the component checkpoint or record described by the returned expression.

Signature: save(): UltimateSave

Return at L69: { version: 1, moves: [...this.history] }

```
save(): UltimateSave { return { version: 1, moves: [...this.history] }; }
```

UltimateBoard.restore [L71-78]

Validate/replay saved state and fall back to a fresh legal instance when rejected.

Signature: static restore(data: unknown)

Calls / delegates: UltimateBoard, b.play

Return at L75: b

Return at L76: new UltimateBoard()

Return at L77: b

Decision branches: L74: d?.version !== 1 || !Array.isArray(d.moves) || d.moves.length > 81; L76: !b.play(move)

UltimateBoard.key [L79-87]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: key()

Calls / delegates: this.cells.map((v) => v ?? 0).join, this.cells.map

Return at L80: (this.cells.map((v) => v ?? 0).join("") + ":" + this.forced + ":" + this.turn)

src/games/tic-tac-toe/ultimate/UltimateSearch.ts

ACTIVE SOURCE / tool or application | 189 lines | SHA-256 a2f678c388ec070f...

Evaluates local/macro threats and runs bounded iterative deepening, move ordering and transposition search.

Imports: ../../../shared/ai/MatchRandom, ./UltimateRules, ../shared/lines, ../../../shared/contracts/AIController, ../../../shared/contracts/GameResult

Entry [L6-10]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
interface Entry { depth: number; value: number; flag: "exact" | "lower" | "upper"; }
```

SearchResult [L11-15]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface SearchResult { move: number; depth: number; nodes: number; }
```

evaluate [L16-61]

Score a board/candidate state for the surrounding AI search.

Signature: export function evaluate(board: UltimateBoard, player: Player, style?: MatchRandom,)

Direct writes: score

Calls / delegates: line.filter, line.some

Return at L22: board.winner === "draw" ? 0 : board.winner === player ? 1000000 : -1000000

Return at L60: score

Decision branches: L21: board.winner; L33: !closed; L34: !other; L35: !own; L39: board.boards[b]; L40: board.boards[b] === player; L41: board.boards[b] === enemy; L49: !other; L50: !own; L52: board.cells[b * 9 + 4] === player; L53: board.cells[b * 9 + 4] === enemy; L55: style; L58: p

Execution order (direct statements; nested callbacks stay within their owner):

L21-26 IfStatement: if (board.winner) return board.winner === "draw"

L27-27 FirstStatement: const enemy = player === 1 ? 2 : 1;

L28-28 FirstStatement: let score = 0;

L29-37 ForOfStatement: for (const line of WIN_LINES) { const own = line.filter((i) => board.boards[i] === player).length,

L38-54 ForStatement: for (let b = 0; b < 9; b++) { if (board.boards[b]) {

L55-59 IfStatement: if (style) for (let b = 0; b < 9; b++) {

L60-60 ReturnStatement: return score;

ultimateMove [L63-188]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function ultimateMove(board: UltimateBoard, difficulty: Difficulty, overrides: { budgetMs?: number; random?: () => number; seed?: number; replay?: boolean; } = {},): SearchResult

Direct writes: best, alpha, beta, value, roundBest, completeDepth

Calls / delegates: MatchRandom, board.legalMoves, performance.now, Map, Symbol, b .legalMoves() .map((move) => { const next = b.clone(); next.play(move); return { move, next, value: evaluate(next, player, style) }; }) .sort, b .legalMoves() .map, b .legalMoves, b.clone, next.play, evaluate, style.rank, ordered, random, roots.slice, b.key, table.get, search, table.set, roots.sort

Return at L81: { move: -1, depth: 0, nodes: 0 }

Return at L92: b .legalMoves() .map((move) => { const next = b.clone(); next.play(move); return { move, next, value: evaluate(next, player, style) }; }) .sort((a, z) => (b.turn === player ? z.value - a.value : a.value - z.value) || style.rank(a.move) - style.rank(z.move),)

Return at L97: { move, next, value: evaluate(next, player, style) }

Return at L109: { move: candidates[Math.floor(random() * candidates.length)].move, depth: 0, nodes: 0, }

Return at L127: evaluate(b, player, style)

Return at L133: cached.value

Return at L136: cached.value

Return at L161: value

Return at L187: { move: best, depth: completeDepth, nodes }

Decision branches: L81: !legal.length; L107: difficulty === "medium" && random() < 0.16; L122: nodes > settings.nodes || ((nodes & 31) === 0 && performance.now() >= deadline); L127: b.winner || depth === 0; L132: cached && cached.depth >= depth; L133: cached.flag === "exact"; L134: cached.flag === "lower"; L136: alpha >= beta; L141: b.turn === player; L148: alpha >= beta; L150: table.size < 16000; L168: performance.now() >= deadline; L170: score > value || (score === value && style.rank(node.move) < style.rank(roundBest)); L181: value >= 1000000; L183: e !== TIMEOUT

Execution order (direct statements; nested callbacks stay within their owner):

L73-77 FirstStatement: const settings = { medium: { ms: 25, depth: 2, nodes: 5000 },

L78-80 FirstStatement: const style = new MatchRandom(overrides.seed ?? overrides.random), random = style.next,

L81-81 IfStatement: if (!legal.length) return { move: -1, depth: 0, nodes: 0 };

L82-86 FirstStatement: const player = board.turn, deadline = overrides.replay

L87-89 FirstStatement: let nodes = 0, completeDepth = 0,

L90-90 FirstStatement: const TIMEOUT = Symbol();

L91-104 FunctionDeclaration: function ordered(b: UltimateBoard) { return b

L105-105 FirstStatement: const roots = ordered(board);

L106-106 ExpressionStatement: best = roots[0].move;

L107-114 IfStatement: if (difficulty === "medium" && random() < 0.16) { const candidates = roots.slice(0, Math.min(5, roots.length));

L115-162 FunctionDeclaration: function search(b: UltimateBoard,

L163-186 ForStatement: for (let depth = 1; depth <= settings.depth; depth++) { let roundBest = best,

L187-187 ReturnStatement: return { move: best, depth: completeDepth, nodes };

ultimateMove.ordered [L91-104]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function ordered(b: UltimateBoard)

Calls / delegates: b .legalMoves() .map((move) => { const next = b.clone(); next.play(move); return { move, next, value: evaluate(next, player, style) }; }) .sort, b .legalMoves() .map, b .legalMoves, b.clone, next.play, evaluate, style.rank

Return at L92: b .legalMoves() .map((move) => { const next = b.clone(); next.play(move); return { move, next, value: evaluate(next, player, style) }; }) .sort((a, z) => (b.turn === player ? z.value - a.value : a.value - z.value) || style.rank(a.move) - style.rank(z.move),)

Return at L97: { move, next, value: evaluate(next, player, style) }

ultimateMove.search [L115-162]

Explore a bounded game tree with the local turn/evaluation policy.

Signature: function search(b: UltimateBoard, depth: number, alpha: number, beta: number,): number

Direct writes: alpha, beta, value

Calls / delegates: performance.now, evaluate, b.key, table.get, ordered, search, table.set

Return at L127: evaluate(b, player, style)

Return at L133: cached.value

Return at L136: cached.value

Return at L161: value

Decision branches: L122: nodes > settings.nodes || ((nodes & 31) === 0 && performance.now() >= deadline); L127: b.winner || depth === 0; L132: cached && cached.depth >= depth; L133: cached.flag === "exact"; L134: cached.flag === "lower"; L136: alpha >= beta; L141: b.turn === player; L148: alpha >= beta; L150: table.size < 16000

Execution order (direct statements; nested callbacks stay within their owner):

L121-121 ExpressionStatement: nodes++;

L122-126 IfStatement: if (nodes > settings.nodes ||

L127-127 IfStatement: if (b.winner || depth === 0) return evaluate(b, player, style);

L128-131 FirstStatement: const key = b.key(), cached = table.get(key),

L132-137 IfStatement: if (cached && cached.depth >= depth) { if (cached.flag === "exact") return cached.value;

L138-138 FirstStatement: let value = b.turn === player ? -Infinity : Infinity;

L139-149 ForOfStatement: for (const node of ordered(b)) { const candidate = search(node.next, depth - 1, alpha, beta);

L150-160 IfStatement: if (table.size < 16000) table.set(key, {

L161-161 ReturnStatement: return value;

src/games/tic-tac-toe/ultimate/UltimateTutorial.ts

ACTIVE SOURCE / tool or application | 51 lines | SHA-256 1fa0e4250adea8ad...

Builds and advances the module-owned interactive Ultimate rules tutorial.

Imports: ../../../shared/contracts/GameModule

ultimateTutorial [L2-50]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function ultimateTutorial(ui: TutorialUI)

Direct writes: closed, step, actions[`cell${i}`], chosen

Calls / delegates: diagram, ui.done, render, ui.show

Decision branches: L25: step === 2

Execution order (direct statements; nested callbacks stay within their owner):

L3-5 FirstStatement: let step = 0, chosen = 4,

L6-7 FirstStatement: const diagram = () => `<svg viewBox="0 0 180 180" aria-label="Nine boards, highlighted destination">${Array.from({ length: 9 }, (_, i) => `<rect x="${(i % 3) * 60 + 3}" y="${Math.floor(i / 3) * 60 + 3

L8-48 FunctionDeclaration: function render() { const titles = [

L49-49 ExpressionStatement: render();

ultimateTutorial.diagram [L6-7]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: diagram = () =>

ultimateTutorial.render [L8-48]

Build or refresh visual/DOM output from current state.

Signature: function render()

Direct writes: closed, step, actions[`cell${i}`], chosen

Calls / delegates: diagram, ui.done, render, ui.show

Decision branches: L25: step === 2

Execution order (direct statements; nested callbacks stay within their owner):

L9-13 FirstStatement: const titles = ["Nine little boards. One big game.",

L14-18 FirstStatement: const details = ["Win a small board with three in a row. Win the match by claiming three small boards in a row. Orange plays first, anywhere.",

L19-22 FirstStatement: const demo = step === 1

L23-38 FirstStatement: const actions: Record<string, () => void> = { next: () => {

L39-43 ForStatement: for (let i = 0; i < 9; i++) actions[`cell${i}`] = () => {

L44-47 ExpressionStatement: ui.show(` ULTIMATE · HOW TO PLAY · ${step + 1}/3</div><h2 id="dialog-title">${titles[step]}</h2>${demo}<p>${details[step]}</p><button class="primary-button" data-action="next">${s

src/games/tic-tac-toe/ultimate/ai.worker.ts

ACTIVE SOURCE / tool or application | 13 lines | SHA-256 6ca0dc6405a177f1...

Worker message entry for Tic-Tac-Toe. Restores/copies request state, runs the bounded local search and posts its result; cancellation is owned by WorkerTask/controller.

Imports: ./UltimateRules, ./UltimateSearch

Top-level block L3-12

```
self.onmessage = ({ data }) => {
  const board = UltimateBoard.restore(data.state);
  const start = performance.now(),
    result = ultimateMove(board, data.difficulty, data);
  self.postMessage({
    id: data.id,
    ...result,
    milliseconds: performance.now() - start,
  });
};
```

src/games/water-guess

src/games/water-guess/Geometry.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 3a0d42798024063b...

Defines the common 310-unit visible tube height used by renderer and AI.

Module constants and data

L2-2 TUBE_HEIGHT = 310

src/games/water-guess/WaterGuessAI.ts

ACTIVE SOURCE / tool or application | 32 lines | SHA-256 c0ce972259d97342...

Estimates a one-logical-pixel tube level with difficulty-dependent bounded perceptual error.

Imports: ../../shared/contracts/AIController, ../../shared/ai/MatchRandom, ./Geometry

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;

WaterGuessAI [L4-31]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

WaterGuessAI.constructor [L6-11]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(public readonly difficulty: Difficulty = "medium", seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

WaterGuessAI.estimate [L12-27]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: estimate(actualHundredths: number)

Calls / delegates: r.sample

Return at L26: Math.max(0, Math.min(10000, Math.round(measured + noise)))

WaterGuessAI.snapshot [L28-30]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.personality.snapshot

Return at L29: this.personality.snapshot()

src/games/water-guess/WaterGuessGame.ts

ACTIVE SOURCE / tool or application | 557 lines | SHA-256 5a3fbabe47c5b3eb...

Water Guess lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/components/TurnView, ../../shared/ai/MatchRandom, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ./WaterGuessRules, ./WaterGuessAI, ./WaterRenderer

State, resources and initial values

Line	Field / declaration
26	private ctx!: GameContext;
27	private view!: TurnView;
28	private aiWait = 0;
29	private random = new MatchRandom();
30	private match = new WaterGuessMatch(this.random.next);
31	private ai?: WaterGuessAI;
32	private renderer!: WaterRenderer;
34	private g!: Phaser.GameObjects.Graphics;
35	private uiTexts: Phaser.GameObjects.Text[] = [];
36	private buttonTexts: Phaser.GameObjects.Text[] = [];
38	private bannerTitle!: Phaser.GameObjects.Text;
39	private promptText!: Phaser.GameObjects.Text;
40	private inputBufferText!: Phaser.GameObjects.Text;
41	private p1StatusText!: Phaser.GameObjects.Text;
42	private p2StatusText!: Phaser.GameObjects.Text;
43	private revealText!: Phaser.GameObjects.Text;
45	private buttons: KeyButton[] = [];
46	private inputBuffer = "";
47	private activeGuesser: 0 | 1 = 0;
49	private state: "FILLING" | "GUESSING" | "REVEAL" = "FILLING";
50	private fillAnimProgress = 0;
51	private animTime = 0;
52	private revealAge = 0;
53	private paused = false;
54	private reported = false;
55	private roundResult?: WaterRoundResult;

KeyButton [L16-23]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
interface KeyButton { label: string; x: number; y: number; w: number; h: number; action: () => void; }
```

WaterGuessGame [L25-556]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

WaterGuessGame.initialize [L57-133]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.renderer, this.g, this.ai, this.bannerTitle, this.promptText, this.inputBufferText, this.p1StatusText, this.p2StatusText, this.revealText, this.view

Calls / delegates: WaterRenderer, ctx.scene.add.graphics, WaterGuessAI, label, this.bannerTitle.setFontStyle("700").setDepth, this.bannerTitle.setFontStyle, this.uiTexts.push, this.promptText.setFontStyle("800").setDepth, this.promptText.setFontStyle, this.inputBufferText.setFontStyle("900").setDepth, this.inputBufferText.setFontStyle, this.p1StatusText.setFontStyle("700").setDepth, this.p1StatusText.setFontStyle, this.p2StatusText.setFontStyle("700").setDepth, this.p2StatusText.setFontStyle, this.revealText.setFontStyle("700").setDepth(3).setAlpha, this.revealText.setFontStyle("700").setDepth, this.revealText.setFontStyle, this.setupKeypad, TurnView, this.promptText.setVisible, this.p1StatusText.setVisible, this.p2StatusText.setVisible, this.bannerTitle.setFontSize, this.setupInputs, this.startRound, this.ctx.setScore

Decision branches: L62: ctx.options.mode === "ai"

Execution order (direct statements; nested callbacks stay within their owner):

L58-58 ExpressionStatement: this.ctx = ctx;

L59-59 ExpressionStatement: this.renderer = new WaterRenderer(ctx.scene);

L60-60 ExpressionStatement: this.g = ctx.scene.add.graphics();

L62-64 IfStatement: if (ctx.options.mode === "ai") { this.ai = new WaterGuessAI(ctx.options.difficulty, this.random.seed);

L67-74 ExpressionStatement: this.bannerTitle = label(ctx.scene,

L75-75 ExpressionStatement: this.bannerTitle.setFontStyle("700").setDepth(2);

L76-76 ExpressionStatement: this.uiTexts.push(this.bannerTitle);

L79-86 ExpressionStatement: this.promptText = label(ctx.scene,

L87-87 ExpressionStatement: this.promptText.setFontStyle("800").setDepth(2);

L88-88 ExpressionStatement: this.uiTexts.push(this.promptText);

L91-91 ExpressionStatement: this.inputBufferText = label(ctx.scene, 210, 502, " - %", 16, "#242b32");

L92-92 ExpressionStatement: this.inputBufferText.setFontStyle("900").setDepth(2);

L93-93 ExpressionStatement: this.uiTexts.push(this.inputBufferText);

L96-103 ExpressionStatement: this.p1StatusText = label(ctx.scene,

L104-104 ExpressionStatement: this.p1StatusText.setFontStyle("700").setDepth(2);

L105-105 ExpressionStatement: this.uiTexts.push(this.p1StatusText);

L107-114 ExpressionStatement: this.p2StatusText = label(ctx.scene,

L115-115 ExpressionStatement: this.p2StatusText.setFontStyle("700").setDepth(2);

L116-116 ExpressionStatement: this.uiTexts.push(this.p2StatusText);

L119-119 ExpressionStatement: this.revealText = label(ctx.scene, 210, 552, "", 13, "#242b32");

L120-120 ExpressionStatement: this.revealText.setFontStyle("700").setDepth(3).setAlpha(0);

L121-121 ExpressionStatement: this.uiTexts.push(this.revealText);

L123-123 ExpressionStatement: this.setupKeypad();

L124-124 ExpressionStatement: this.view = new TurnView(ctx.scene, [this.g, ...this.uiTexts]);

L125-125 ExpressionStatement: this.promptText.setVisible(false);

L126-126 ExpressionStatement: this.p1StatusText.setVisible(false);

L127-127 ExpressionStatement: this.p2StatusText.setVisible(false);

L128-128 ExpressionStatement: this.bannerTitle.setFontSize(11);

L129-129 ExpressionStatement: this.setupInputs();

L131-131 ExpressionStatement: this.startRound();

L132-132 ExpressionStatement: this.ctx.setScore(0, 0);

WaterGuessGame.setupKeypad [L135-181]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private setupKeypad()

Direct writes: this.buttons, this.buttonTexts

Calls / delegates: this.buttons.push, addKey, this.appendDigit, this.appendDecimal, this.backspace, this.submitCurrentGuess, label, txt.setFontStyle("800").setDepth, txt.setFontStyle, this.buttonTexts.push, this.uiTexts.push

Execution order (direct statements; nested callbacks stay within their owner):

L136-136 ExpressionStatement: this.buttons = [];

L137-146 FirstStatement: const addKey = (lbl: string,

L149-149 FirstStatement: const r1 = ["1", "2", "3", "4", "5"];

L150-152 ForStatement: for (let i = 0; i < 5; i++) { addKey(r1[i], 28 + i * 72, 524, 64, 34, () => this.appendDigit(r1[i]));

L155-155 FirstStatement: const r2 = ["6", "7", "8", "9", "0"];

L156-158 ForStatement: for (let i = 0; i < 5; i++) { addKey(r2[i], 28 + i * 72, 564, 64, 34, () => this.appendDigit(r2[i]));

L161-161 ExpressionStatement: addKey(".", 28, 604, 64, 34, () => this.appendDecimal());

L162-162 ExpressionStatement: addKey("⌫ DEL", 100, 604, 76, 34, () => this.backspace());

L163-163 ExpressionStatement: addKey("CONFIRM", 184, 604, 204, 34, () => this.submitCurrentGuess());

L166-166 ExpressionStatement: this.buttonTexts = [];

L167-180 ForOfStatement: for (const b of this.buttons) { const isConfirm = b.label === "CONFIRM";

WaterGuessGame.setupKeypad.addKey [L137-146]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: addKey = (lbl: string, x: number, y: number, w: number, h: number, action: () => void,) =>

Calls / delegates: this.buttons.push

WaterGuessGame.setupInputs [L183-260]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private setupInputs()

Direct writes: { x, y }

Calls / delegates: this.ctx.input.tap, this.view.point, this.advanceNext, this.ctx.audio.play, b.action, this.ctx.input.press, /^(Digit|Numpad)[0-9]$/.test, code.replace("Digit", "").replace, code.replace, /^[0-9]$/.test, this.appendDigit, this.appendDecimal, this.backspace, this.submitCurrentGuess

Decision branches: L186: !this.view.ready; L188: this.paused || this.reported; L190: this.state === "REVEAL"; L191: this.revealAge > 0.8; L197: this.state !== "GUESSING" || !this.view.ready; L201: x >= b.x && x <= b.x + b.w && y >= b.y && y <= b.y + b.h; L235: this.paused || this.reported; L236: this.state === "REVEAL"; L237: this.revealAge > 0.8; L240: this.state !== "GUESSING" || !this.view.ready; L242: /^(Digit|Numpad)[0-9]$/.test(code); L244: /^[0-9]$/.test(char); L248: code === "Period" || code === "NumpadDecimal"; L251: code === "Backspace"; L254: code === "Enter" || code === "Space"

Execution order (direct statements; nested callbacks stay within their owner):

L185-207 ExpressionStatement: this.ctx.input.tap((x, y) => { if (!this.view.ready) return;

L210-231 FirstStatement: const digitCodes = ["Digit1",

L232-259 ExpressionStatement: this.ctx.input.press([...digitCodes, "Period", "NumpadDecimal", "Backspace", "Enter", "Space"],

WaterGuessGame.appendDigit [L262-270]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private appendDigit(digit: string)

Direct writes: this.inputBuffer

Calls / delegates: this.inputBuffer.indexOf, this.updateBufferDisplay

Decision branches: L263: this.inputBuffer.length >= 6; L266: dotIdx !== -1 && this.inputBuffer.length - dotIdx > 2

WaterGuessGame.appendDecimal [L272-280]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private appendDecimal()

Direct writes: this.inputBuffer

Calls / delegates: this.inputBuffer.includes, this.updateBufferDisplay

Decision branches: L273: this.inputBuffer.includes("."); L274: this.inputBuffer.length === 0

WaterGuessGame.backspace [L282-287]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private backspace()

Direct writes: this.inputBuffer

Calls / delegates: this.inputBuffer.slice, this.updateBufferDisplay

Decision branches: L283: this.inputBuffer.length > 0

WaterGuessGame.updateBufferDisplay [L289-301]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private updateBufferDisplay()

Calls / delegates: this.promptText .setText(`${pName}: ENTER GUESS (0.01% PRECISION)`) .setColor, this.promptText .setText, this.inputBufferText.setText

Decision branches: L296: this.inputBuffer.length === 0

WaterGuessGame.startRound [L303-334]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private startRound()

Direct writes: this.state, this.fillAnimProgress, this.inputBuffer, this.roundResult, this.revealAge, this.activeGuesser, this.aiWait

Calls / delegates: this.revealText.setAlpha, this.bannerTitle.setText, this.ctx.setHint, this.random.sample, this.p2StatusText.setText("AI: LOCKED ✓").setColor, this.p2StatusText.setText, this.p2StatusText.setText("BLUE: WAITING").setColor, this.view.set, this.p1StatusText.setText("ORANGE: WAITING").setColor, this.p1StatusText.setText, this.promptText.setAlpha, this.inputBufferText.setAlpha, this.updateBufferDisplay, this.buttonTexts.forEach, t.setAlpha, this.ctx.audio.play

Decision branches: L319: this.ai

WaterGuessGame.submitCurrentGuess [L336-364]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private submitCurrentGuess()

Direct writes: this.inputBuffer, this.activeGuesser

Calls / delegates: parsePercentInput, this.match.submit, this.ctx.audio.play, this.p1StatusText.setText("ORANGE: LOCKED ✓").setColor, this.p1StatusText.setText, this.p2StatusText.setText("BLUE: LOCKED ✓").setColor, this.p2StatusText.setText, this.triggerReveal, this.view.set, this.updateBufferDisplay

Decision branches: L337: this.inputBuffer.length === 0 || !this.view.ready; L341: !ok; L345: this.activeGuesser === 0; L354: this.match.bothSubmitted; L358: !this.ai

WaterGuessGame.triggerReveal [L366-400]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private triggerReveal()

Direct writes: this.state, this.revealAge, this.roundResult

Calls / delegates: this.view.set, this.buttonTexts.forEach, t.setAlpha, this.match.resolveRound, this.ctx.setScore, formatPercent, this.ctx.audio.play, this.revealText .setText(`${res.winner === "draw" ? "A TIE!" : "NICE GUESS!"}\n` + `Actual ${actStr}\n` + `${g0Str} · ${g1Str}\n` + `Continue -> `,) .setAlpha, this.revealText .setText, this.promptText.setAlpha, this.inputBufferText.setAlpha

Decision branches: L367: this.state !== "GUESSING" || this.reported; L381: res.winner === 1; L383: res.winner === 2

Execution order (direct statements; nested callbacks stay within their owner):

L367-367 IfStatement: if (this.state !== "GUESSING" || this.reported) return;

L368-368 ExpressionStatement: this.state = "REVEAL";

L369-369 ExpressionStatement: this.revealAge = 0;

L370-370 ExpressionStatement: this.view.set(false);

L371-371 ExpressionStatement: this.buttonTexts.forEach((t) => t.setAlpha(0));

L372-372 ExpressionStatement: this.roundResult = this.match.resolveRound();

L373-373 ExpressionStatement: this.ctx.setScore(this.match.scores[0], this.match.scores[1]);

L375-375 FirstStatement: const res = this.roundResult;

L376-376 FirstStatement: const g0Str = formatPercent(res.guesses[0]);

L377-377 FirstStatement: const g1Str = formatPercent(res.guesses[1]);

L379-379 FirstStatement: const actStr = formatPercent(res.actual);

L381-387 IfStatement: if (res.winner === 1) { this.ctx.audio.play("capture");

L389-396 ExpressionStatement: this.revealText .setText(

L398-398 ExpressionStatement: this.promptText.setAlpha(0);

L399-399 ExpressionStatement: this.inputBufferText.setAlpha(0);

WaterGuessGame.advanceNext [L402-416]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private advanceNext()

Direct writes: this.reported

Calls / delegates: this.ctx.finish, this.match.nextRound, this.startRound

Decision branches: L403: this.match.isMatchOver; L404: !this.reported

WaterGuessGame.update [L418-449]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.animTime, this.aiWait, this.fillAnimProgress, this.state, this.revealAge

Calls / delegates: this.view.update, this.match.submit, this.ai.estimate, this.triggerReveal, this.updateBufferDisplay, this.advanceNext, this.render

Decision branches: L419: this.paused || this.reported; L422: this.state === "GUESSING" && this.ai && this.match.guesses[1] === null; L428: this.aiWait === 0; L430: this.match.bothSubmitted; L434: this.state === "FILLING"; L436: this.fillAnimProgress >= 1; L440: this.state === "REVEAL"; L443: this.revealAge > 3.2

WaterGuessGame.render [L451-527]

Build or refresh visual/DOM output from current state.

Signature: private render()

Direct writes: btnColor

Calls / delegates: g.clear, rounded, turnBackground, this.renderer.drawTube

Decision branches: L498: this.state === "GUESSING" || this.state === "FILLING"; L501: b.label === "CONFIRM"; L503: b.label === "⌫ DEL"; L511: this.state === "REVEAL"

Execution order (direct statements; nested callbacks stay within their owner):

L452-452 FirstStatement: const g = this.g;

L453-453 ExpressionStatement: g.clear();

L456-479 ExpressionStatement: rounded(g,

L482-485 FirstStatement: const waterLevel = this.state === "FILLING"

L487-495 ExpressionStatement: this.renderer.drawTube(g,

L498-508 IfStatement: if (this.state === "GUESSING" || this.state === "FILLING") { for (const b of this.buttons) {

L511-526 IfStatement: if (this.state === "REVEAL") { const winner = this.roundResult?.winner;

WaterGuessGame.pause [L529-531]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

WaterGuessGame.resume [L533-535]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

WaterGuessGame.dispose [L537-540]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.reported

Calls / delegates: this.view.destroy

WaterGuessGame.snapshot [L542-555]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.snapshot

Return at L543: { ai: this.ai?.snapshot(), match: this.match, state: this.state, activeGuesser: this.activeGuesser, rotation: this.view.rotation, turning: !this.view.ready, aiWait: this.aiWait, inputBuffer: this.inputBuffer, scores: [...this.match.scores], round: this.match.round, }

src/games/water-guess/WaterGuessRules.ts

ACTIVE SOURCE / tool or application | 136 lines | SHA-256 ac28b5ae00d168bb...

Owns integer percentage targets, one guess each, absolute-error scoring, idempotent resolution and sudden death.

State, resources and initial values

Line	Field / declaration
31	round = 1;
32	scores: [number, number] = [0, 0];
33	history: WaterRoundResult[] = [];
34	actual = 0;
35	guesses: [number | null, number | null] = [null, null];
36	private resolved?: WaterRoundResult;
37	private usedActuals = new Set<number>();

Module constants and data

L1-1 TOTAL_ROUNDS = 5

L2-2 MIN_PERCENT = 1000

L3-3 MAX_PERCENT = 9000

formatPercent [L5-8]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function formatPercent(hundredths: number): string

Calls / delegates: (clamped / 100).toFixed

Return at L7: (clamped / 100).toFixed(2) + "%"

formatValue [L10-13]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function formatValue(hundredths: number): string

Calls / delegates: (clamped / 100).toFixed

Return at L12: (clamped / 100).toFixed(2)

parsePercentInput [L15-20]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function parsePercentInput(input: string): number

Calls / delegates: input.trim().replace, input.trim, isNaN

Return at L19: Math.max(0, Math.min(10000, Math.round(num * 100)))

Decision branches: L18: isNaN(num)

WaterRoundResult [L22-28]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface WaterRoundResult { round: number; actual: number; // 0..10000 guesses: [number, number]; // 0..10000 errors: [number, number]; // absolute error in hundredths winner: 1 | 2 | "draw"; }
```

WaterGuessMatch [L30-135]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

WaterGuessMatch.constructor [L39-41]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private rng: () => number = Math.random)

Calls / delegates: this.startNewRound

WaterGuessMatch.startNewRound [L43-57]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: startNewRound()

Direct writes: this.resolved, this.guesses, candidate, this.actual

Calls / delegates: this.rng, this.usedActuals.has, this.usedActuals.add

WaterGuessMatch.submit [L59-65]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: submit(player: 0 | 1, guessHundredths: number): boolean

Direct writes: this.guesses[player]

Decision branches: L60: !Number.isFinite(guessHundredths); L61: this.guesses[player] !== null

WaterGuessMatch.bothSubmitted [L67-69]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get bothSubmitted(): boolean

Return at L68: this.guesses[0] !== null && this.guesses[1] !== null

```
get bothSubmitted(): boolean { return this.guesses[0] !== null && this.guesses[1] !== null; }
```

WaterGuessMatch.resolveRound [L71-103]

Compare two submitted integer errors and cache one score result so repeated resolution cannot score again.

Signature: resolveRound(): WaterRoundResult

Direct writes: winner, this.resolved

Calls / delegates: Error, this.history.push

Return at L72: this.resolved

Return at L102: result

Decision branches: L72: this.resolved; L73: !this.bothSubmitted; L82: err0 < err1; L85: err1 < err0

Execution order (direct statements; nested callbacks stay within their owner):

L72-72 IfStatement: if (this.resolved) return this.resolved;

L73-75 IfStatement: if (!this.bothSubmitted) { throw new Error("Cannot resolve round: both players must submit guesses");

L76-76 FirstStatement: const g0 = this.guesses[0]!;

L77-77 FirstStatement: const g1 = this.guesses[1]!;

L78-78 FirstStatement: const err0 = Math.abs(g0 - this.actual);

L79-79 FirstStatement: const err1 = Math.abs(g1 - this.actual);

L81-81 FirstStatement: let winner: 1 | 2 | "draw";

L82-91 IfStatement: if (err0 < err1) { winner = 1;

L93-99 FirstStatement: const result: WaterRoundResult = { round: this.round,

L100-100 ExpressionStatement: this.resolved = result;

L101-101 ExpressionStatement: this.history.push(result);

L102-102 ReturnStatement: return result;

WaterGuessMatch.nextRound [L105-110]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: nextRound(): boolean

Calls / delegates: this.startNewRound

Decision branches: L106: !this.resolved || this.isMatchOver

WaterGuessMatch.isSuddenDeath [L112-114]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get isSuddenDeath(): boolean

Return at L113: this.round > TOTAL_ROUNDS && this.scores[0] === this.scores[1]

```
get isSuddenDeath(): boolean { return this.round > TOTAL_ROUNDS && this.scores[0] === this.scores[1]; }
```

WaterGuessMatch.isMatchOver [L116-127]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get isMatchOver(): boolean

Return at L120: (this.history.length === TOTAL_ROUNDS && this.scores[0] !== this.scores[1])

Return at L126: this.scores[0] !== this.scores[1]

Decision branches: L117: this.round < TOTAL_ROUNDS; L118: this.round === TOTAL_ROUNDS

WaterGuessMatch.matchWinner [L129-134]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get matchWinner(): 1 | 2 | "draw" | null

Return at L132: 2

Return at L133: "draw"

Decision branches: L130: !this.isMatchOver; L131: this.scores[0] > this.scores[1]; L132: this.scores[1] > this.scores[0]

src/games/water-guess/WaterRenderer.ts

ACTIVE SOURCE / tool or application | 163 lines | SHA-256 3d1a08ae62844c69...

Draws the tube/liquid and reveal presentation using shared tube geometry.

Imports: phaser, ./Geometry, ../../core/ThemeManager, ./WaterGuessRules

Module constants and data

L6-13 TUBE = { cx: 210, yTop: 155, // 100.00% level yBottom: 465, // 0.00% level width: 82, innerHalfW: 36, height: TUBE_HEIGHT, // usable height: 465 - 155 = 310 }

WaterRenderer [L15-162]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

WaterRenderer.constructor [L16-16]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private scene: Phaser.Scene)

```
constructor(private scene: Phaser.Scene) {}
```

WaterRenderer.drawTube [L18-161]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: drawTube(g: Phaser.GameObjects.Graphics, currentWaterLevelHundredths: number, // 0..10000 animTime: number, revealed: boolean, actualHundredths: number, guess0Hundredths: number | null, guess1Hundredths: number | null,)

Direct writes: p

Calls / delegates: g.fillStyle, g.fillRoundedRect, g.lineStyle, g.lineBetween, g.beginPath, g.moveTo, g.lineTo, g.closePath, g.fillPath, g.fillRect, g.strokePath, g.strokeRoundedRect

Decision branches: L53: Math.abs(p - 0.25) < 0.01 || Math.abs(p - 0.5) < 0.01 || Math.abs(p - 0.75) < 0.01; L69: waterHeight > 0; L106: i === 0; L136: revealed; L148: guess0Hundredths !== null; L155: guess1Hundredths !== null

Execution order (direct statements; nested callbacks stay within their owner):

L27-27 FirstStatement: const { cx, yTop, yBottom, innerHalfW, height } = TUBE;

L30-30 ExpressionStatement: g.fillStyle(0xeaf4fc, 0.85);

L31-37 ExpressionStatement: g.fillRoundedRect(cx - innerHalfW,

L40-40 ExpressionStatement: g.lineStyle(2, 0x95a5a6, 0.7);

L41-45 FirstStatement: const ticks = [{ pct: 0.25, label: "25%" },

L46-49 ForOfStatement: for (const t of ticks) { const y = yBottom - t.pct * height;

L51-51 ExpressionStatement: g.lineStyle(1, 0xbdc3c7, 0.4);

L52-61 ForStatement: for (let p = 0.1; p < 1.0; p += 0.1) { if (

L64-64 FirstStatement: const frac = Math.max(0, Math.min(1, currentWaterLevelHundredths / 10000));

L65-65 FirstStatement: const waterHeight = frac * height;

L66-66 FirstStatement: const waterBaseY = yBottom;

L67-67 FirstStatement: const waterSurfaceY = waterBaseY - waterHeight;

L69-110 IfStatement: if (waterHeight > 0) { // Main water fill

L113-113 ExpressionStatement: g.fillStyle(0xffffff, 0.35);

L114-114 ExpressionStatement: g.fillRoundedRect(cx - innerHalfW + 6, yTop, 7, height - 10, 3);

L117-117 ExpressionStatement: g.lineStyle(4, theme.ink);

L118-124 ExpressionStatement: g.strokeRoundedRect(cx - innerHalfW,

L127-127 ExpressionStatement: g.lineStyle(5, theme.ink);

L128-133 ExpressionStatement: g.lineBetween(cx - innerHalfW - 6,

L136-160 IfStatement: if (revealed) { // Actual green marker

src/games/water-guess/config.ts

ACTIVE SOURCE / tool or application | 18 lines | SHA-256 a6b002ba00914e8f...

Eager lightweight metadata for Water Guess: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L4-17

```
export default {
  id: "water-guess",
  name: "Water Guess",
  description: "Estimate the water percentage with 0.01% precision.",
  instructions:
    "Observe the vertical water tube and enter your percentage guess with two decimal places (0.01% precision). The player whose guess is closest to the actual water level wins the round. Five rounds per match.",
  color: "#d9eaf7",
  accent: "#8bc2ea",
  preview,
  order: 14,
  tag: "PRECISION EYE",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/games/water-guess/index.ts

ACTIVE SOURCE / tool or application | 4 lines | SHA-256 b836f1b347743bf5...

Side-effect-free lazy factory for Water Guess; implementation initializes only after host selection.

Imports: ./WaterGuessGame

Top-level block L3-3

```
export default () => new WaterGuessGame();
```

src/arcade/snake

src/arcade/snake/SnakeGame.ts

ACTIVE SOURCE / tool or application | 174 lines | SHA-256 d88893c93f7de8e0...

Snake lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ./SnakeRules

State, resources and initial values

Line	Field / declaration
7	private ctx!: GameContext;
8	private world = new SnakeWorld();
9	private g!: Phaser.GameObjects.Graphics;
10	private labels: Phaser.GameObjects.Text[] = [];
11	private keys = new Set<string>();
12	private pressed = new Set<string>();
13	private paused = false;
14	private reported = false;
15	private age = 0;
16	private start: { x: number; y: number } | null = null;

SnakeGame [L6-173]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SnakeGame.initialize [L17-76]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.keys, dir, this.start

Calls / delegates: ctx.scene.add.graphics, ctx.setHint, ctx.input.keys, this.labels.push, label, ctx.input.tap, this.world.steer, ctx.input.drag

Decision branches: L39: this.paused; L41: Math.abs(x - 210) < 24 && Math.abs(y - 574) < 23; L42: Math.abs(y - 620) < 24; L43: Math.abs(x - 152) < 24; L44: Math.abs(x - 210) < 24; L45: Math.abs(x - 268) < 24; L47: dir; L51: y < 130 || y > 550; L57: !this.start; L61: Math.hypot(dx, dy) > 15

Execution order (direct statements; nested callbacks stay within their owner):

L18-18 ExpressionStatement: this.ctx = ctx;

L19-19 ExpressionStatement: this.g = ctx.scene.add.graphics();

L20-20 ExpressionStatement: ctx.setHint("ARROWS OR WASD · SWIPE OR USE THE PAD");

L21-30 ExpressionStatement: this.keys = ctx.input.keys(["ArrowUp",

L31-37 ForOfStatement: for (const [x, y, text] of [[210, 574, "↑"],

L38-48 ExpressionStatement: ctx.input.tap((x, y) => { if (this.paused) return;

L49-75 ExpressionStatement: ctx.input.drag((x, y) => {

SnakeGame.update [L77-149]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.age, this.pressed, this.reported

Calls / delegates: this.pressed.has, this.world.steer, Set, this.world.update, this.ctx.audio.play, g.clear, rounded, g.fillStyle, g.fillCircle, g.fillStyle(theme.ink).fillCircle, circle, g.lineStyle(2, theme.ink).lineBetween, g.lineStyle, this.ctx.setArcadeHUD, this.ctx.finishArcade

Decision branches: L78: this.paused; L91: !this.pressed.has(key); L94: this.age > 0.6; L95: this.world.score > before; L111: i === 0; L121: w.food; L141: w.dead && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L78-78 IfStatement: if (this.paused) return;

L79-79 ExpressionStatement: this.age += dt;

L80-89 FirstStatement: const map: Record<string, Direction> = { ArrowUp: "up",

L90-91 ForOfStatement: for (const key of this.keys) if (!this.pressed.has(key)) this.world.steer(map[key]);

L92-92 ExpressionStatement: this.pressed = new Set(this.keys);

L93-93 FirstStatement: const before = this.world.score;

L94-94 IfStatement: if (this.age > 0.6) this.world.update(dt);

L95-95 IfStatement: if (this.world.score > before) this.ctx.audio.play("goal");

L96-97 FirstStatement: const w = this.world, g = this.g;

L98-98 ExpressionStatement: g.clear();

L99-99 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, theme.green, 4);

L100-100 ExpressionStatement: rounded(g, 38, 138, 344, 404, 8, 0xdde9c4, 3);

L101-101 ExpressionStatement: g.fillStyle(theme.ink, 0.08);

L102-103 ForStatement: for (let y = 0; y < 20; y++) for (let x = 0; x < 17; x++) g.fillCircle(50 + x * 20, 150 + y * 20, 1);

L104-104 FirstStatement: const t = Math.min(1, w.accumulator / w.interval);

L105-120 ForStatement: for (let i = w.body.length - 1; i >= 0; i--) { const c = w.body[i],

L121-129 IfStatement: if (w.food) { circle(g, 50 + w.food.x * 20, 150 + w.food.y * 20, 6, theme.orange, 2);

L130-136 ForOfStatement: for (const [x, y] of [[210, 574],

L137-140 ExpressionStatement: this.ctx.setArcadeHUD(w.score,

L141-148 IfStatement: if (w.dead && !this.reported) { this.reported = true;

SnakeGame.pause [L150-155]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused, this.start

Calls / delegates: this.world.clearInput, this.pressed.clear

SnakeGame.resume [L156-159]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused, this.age

SnakeGame.dispose [L160-163]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.g.destroy, this.labels.forEach, l.destroy

SnakeGame.snapshot [L164-172]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.world.body.map

Return at L165: { body: this.world.body.map((c) => ({ ...c })), food: this.world.food, score: this.world.score, dead: this.world.dead, direction: this.world.direction, }

src/arcade/snake/SnakeRules.ts

ACTIVE SOURCE / tool or application | 96 lines | SHA-256 e3d0866960c15e47...

Owns grid movement, queued legal direction changes, tail-vacancy collision, food and speed progression.

State, resources and initial values

Line	Field / declaration
13	readonly width = 17;
14	readonly height = 20;
15	body: Cell[] = [{ x: 8, y: 11 }, { x: 7, y: 11 }, { x: 6, y: 11 },];
20	previous: Cell[] = this.body.map((c) => ({ ...c }));
21	direction: Direction = "right";
22	food: Cell | null = null;
23	score = 0;
24	dead = false;
25	won = false;
26	private queue: Direction[] = [];
27	accumulator = 0;

Module constants and data

L6-11 vectors = { up: { x: 0, y: -1 }, right: { x: 1, y: 0 }, down: { x: 0, y: 1 }, left: { x: -1, y: 0 }, }

Direction [L1-1]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type Direction = "up" | "right" | "down" | "left";
```

Cell [L2-5]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Cell { x: number; y: number; }
```

SnakeWorld [L12-95]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SnakeWorld.constructor [L28-30]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private random = Math.random)

Calls / delegates: this.spawnFood

SnakeWorld.interval [L31-33]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get interval()

Return at L32: Math.max(0.065, 0.18 - this.score * 0.0015)

SnakeWorld.steer [L34-46]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: steer(dir: Direction)

Calls / delegates: this.queue.at, this.queue.push

Decision branches: L38: this.queue.length >= 2 || last === dir || (a.x + b.x === 0 && a.y + b.y === 0)

SnakeWorld.clearInput [L47-49]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clearInput()

Direct writes: this.queue

SnakeWorld.update [L50-57]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.accumulator

Calls / delegates: this.tick

Decision branches: L51: this.dead

SnakeWorld.tick [L58-81]

Execute one authoritative discrete or fixed simulation step.

Signature: tick()

Direct writes: this.direction, this.dead, this.previous, this.score

Calls / delegates: this.queue.shift, this.body.slice, solid.some, this.body.map, this.body.unshift, this.spawnFood, this.body.pop

Decision branches: L59: this.dead; L65: head.x < 0 || head.x >= this.width || head.y < 0 || head.y >= this.height || solid.some((c) => c.x === head.x && c.y === head.y); L77: eating

SnakeWorld.spawnFood [L82-94]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: spawnFood()

Direct writes: this.food, this.dead, this.won

Calls / delegates: this.body.some, free.push, this.random

Decision branches: L86: !this.body.some((c) => c.x === x && c.y === y); L87: !free.length

src/arcade/snake/config.ts

ACTIVE SOURCE / tool or application | 18 lines | SHA-256 34e56340b8c300d5...

Eager lightweight metadata for Snake: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-17

```
export default {
  id: "snake",
  name: "Classic Snake",
  description: "One more bite. One more turn.",
  instructions:
    "Use the direction pad, swipe the garden, or press arrows/WASD. Eat fruit, grow longer, and avoid the walls and
your tail.",
  color: "#b8d998",
  accent: "#9ac480",
  preview,
  order: 10,
  tag: "CHASE YOUR PERSONAL BEST",
  category: "arcade",
  tournament: false,
  supportedModes: ["single-player"],
} satisfies GameMetadata;
```

src/arcade/snake/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 b80a44363bccdcda...

Side-effect-free lazy factory for Snake; implementation initializes only after host selection.

Imports: ./SnakeGame

Top-level block L2-2

```
export default () => new SnakeGame();
```

src/arcade/space-shooter

src/arcade/space-shooter/Powerups.ts

ACTIVE SOURCE / tool or application | 36 lines | SHA-256 e21e9c6d129c740d...

Defines pickup kinds/names/colors/durations, deterministic wave rewards and restricted extra-life eligibility.

Module constants and data

L3-13 POWERUPS = { spread: { name: "CANNON UPGRADE", color: 0xf5ad69, duration: 0 }, rapid: { name: "RAPID FIRE", color: 0xf2ce72, duration: 14 }, laser: { name: "PIERCING LASER", color: 0x76c8db, duration: 16 }, drone: { name: "WINGMATES", color: 0xb2a0d1, duration: 18 }, shield: { name: "SHIELD READY", color: 0x81bdc1, duration: 0 }, life: { name: "+1 LIFE", color: 0xe4a9b4, duration: 0 }, }

PowerupKind [L1-2]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export type PowerupKind = "spread" | "rapid" | "laser" | "drone" | "shield" | "life";
```

Pickup [L14-21]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Pickup { id: number; x: number; y: number; kind: PowerupKind; active: boolean; age: number; }
```

waveReward [L22-26]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function waveReward(wave: number): PowerupKind

Return at L23: (["spread", "rapid", "drone", "laser", "shield"] as const)[(wave - 1) % 5]

```
export function waveReward(wave: number): PowerupKind { return (["spread", "rapid", "drone", "laser", "shield"] as
const)[ (wave - 1) % 5 ]; }
```

canDropLife [L28-35]

Permit rare extra lives only on six-wave milestones, after90s since the last drop and below four lives.

Signature: export function canDropLife(wave: number, elapsed: number, lastDrop: number, lives: number,)

Return at L34: wave % 6 === 0 && elapsed - lastDrop >= 90 && lives < 4

```
export function canDropLife( wave: number, elapsed: number, lastDrop: number, lives: number, ) { return wave % 6 ===
0 && elapsed - lastDrop >= 90 && lives < 4; }
```

src/arcade/space-shooter/ShooterArt.ts

ACTIVE SOURCE / tool or application | 276 lines | SHA-256 cc27eda7ce0fcbb5...

Star Patrol original procedural artwork and drawing helpers. Geometry here presents state; legal rules/collision stay in their authoritative model.

Imports: phaser, ../../core/ThemeManager, ../../shared/components/drawing, ./Powerups, ./ShooterWorld

Module constants and data

L7-9 ENEMY_COLORS = [0xe8a16f, 0x7bb9cf, 0x8e9d94, 0xa58bc0, 0xedc676, 0xa596be,]

G [L6-6]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
type G = Phaser.GameObjects.Graphics;
```

powerIcon [L10-60]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function powerIcon(g: G, kind: PowerupKind, x: number, y: number, size = 1,)

Calls / delegates: g.save, g.translateCanvas, g.scaleCanvas, g.lineStyle, g.lineBetween(0, 7, 0, -7) .lineBetween(-3, 6, -8, -4) .lineBetween, g.lineBetween(0, 7, 0, -7) .lineBetween, g.lineBetween, g.fillStyle(theme.ink).fillPoints, g.fillStyle, g.lineBetween(-3, -8, -3, 8) .lineBetween(3, -8, 3, 8) .lineBetween, g.lineBetween(-3, -8, -3, 8) .lineBetween, g.strokeTriangle(-5, -7, -10, 5, 0, 5).strokeTriangle, g.strokeTriangle, g.strokePoints, g.fillStyle(theme.ink) .fillCircle(-4, -3, 5) .fillCircle(4, -3, 5) .fillTriangle, g.fillStyle(theme.ink) .fillCircle(-4, -3, 5) .fillCircle, g.fillStyle(theme.ink) .fillCircle, g.restore

Decision branches: L21: kind === "spread"; L25: kind === "rapid"; L37: kind === "laser"; L41: kind === "drone"; L43: kind === "shield"

Execution order (direct statements; nested callbacks stay within their owner):

L17-17 ExpressionStatement: g.save();

L18-18 ExpressionStatement: g.translateCanvas(x, y);

L19-19 ExpressionStatement: g.scaleCanvas(size, size);

L20-20 ExpressionStatement: g.lineStyle(2, theme.ink);

L21-58 IfStatement: if (kind === "spread") g.lineBetween(0, 7, 0, -7)

L59-59 ExpressionStatement: g.restore();

drawShooter [L61-275]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function drawShooter(g: G, w: ShooterWorld, age: number, alpha: number, firing: boolean,)

Calls / delegates: g.clear, rounded, g.fillGradientStyle(0xe3dfec, 0xe3dfec, 0xc7c4dc, 0xc7c4dc).fillRoundedRect, g.fillGradientStyle, g.fillStyle(theme.ink, i % 3 === 0 ? 0.18 : 0.1).fillCircle, g.fillStyle, g.fillStyle(POWERUPS.rapid.color).fillRoundedRect, g.fillStyle(POWERUPS.laser.color).fillRoundedRect, powerIcon, g.lineStyle(2, POWERUPS.drone.color).lineBetween, g.lineStyle, circle, g.lineStyle(1, theme.ink, 0.25).strokeCircle, g.save, g.translateCanvas, g.lineStyle(2, 0xc7745b, 0.5 + t * 0.5).strokeCircle, g.fillStyle(0xffe1b0).fillCircle, g.lineStyle(4, theme.ink) .lineBetween(-17, 12, -17, 21) .lineBetween(0, 12, 0, 21) .lineBetween, g.lineStyle(4, theme.ink) .lineBetween(-17, 12, -17, 21) .lineBetween, g.lineStyle(4, theme.ink) .lineBetween, g.lineStyle(2, 0xe9dcfa).lineBetween, g.fillStyle(color).fillRoundedRect, g.lineStyle(3, theme.ink).strokeRoundedRect, g.fillStyle(color) .fillTriangle(-12, -3, -22, 9, -10, 7) .fillTriangle, g.fillStyle(color) .fillTriangle, g.lineStyle(3, 0xd5e1cf) .lineBetween(-9, -6, 9, -6) .lineBetween, g.lineStyle(3, 0xd5e1cf) .lineBetween, g.fillStyle(theme.ink).fillRoundedRect, g.fillStyle(0xfff9e9).fillCircle(-4, 0, 1.4).fillCircle, g.fillStyle(0xfff9e9).fillCircle, g.lineStyle(5, theme.ink).lineBetween, g.lineStyle(2.5, theme.ink) .lineBetween(-9, 10, -13, 14) .lineBetween, g.lineStyle(2.5, theme.ink) .lineBetween, g.lineStyle(3, theme.ink, 0.22).lineBetween, g.lineStyle(3, e.kind === 5 ? 0xfff4ce : 0x56687c).lineBetween, g.restore, g.lineStyle(10, 0x8bd2e2, 0.18).lineBetween, g.lineStyle(p.enemy ? 4 : p.laser ? 4 : 3.5, p.enemy ? 0xb96559 : p.laser ? 0x3b9fbd : 0xfffbeb,).lineBetween, g.fillStyle(POWERUPS[p.kind].color, 0.2).fillCircle, g.lineStyle(2, color, 1 - t).strokeCircle, g.fillStyle(color, 1 - t).fillCircle, g.fillStyle(theme.blue).fillTriangle, g.lineStyle(2, theme.ink).strokeTriangle, g.lineStyle(2.5, POWERUPS.shield.color, 0.7).strokeCircle, g.fillStyle(theme.orange, w.invulnerable > 0 ? 0.55 + 0.45 * Math.sin(age * 18) ** 2 : 1,).fillTriangle, g.lineStyle(3, theme.ink).strokeTriangle, g.lineStyle(3, theme.ink) .lineBetween(x - 10, y + 1, x - 10, y - 11) .lineBetween, g.lineStyle(3, theme.ink) .lineBetween, g.fillStyle(0xfff8d6).fillTriangle, g.lineStyle(4, theme.ink).strokeRoundedRect

Decision branches: L89: w.rapid > 0; L97: w.laser > 0; L105: w.drone > 0; L114: w.shield; L119: e.alive; L124: (e.charge ?? 0) > 0; L129: e.kind === 5; L136: e.kind === 4; L141: e.kind === 1; L146: e.kind === 2; L152: e.kind === 3; L158: (e.maxHp ?? e.hp) > 1; L176: p.active; L179: p.laser; L191: p.active; L214: b.kind === "kill"; L226: w.drone > 0; L246: w.shield; L261: w.weaponLevel > 0; L265: firing && !w.transitioning

Execution order (direct statements; nested callbacks stay within their owner):

L68-68 ExpressionStatement: g.clear();

L69-69 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xd8d4e6, 4);

L70-76 ExpressionStatement: g.fillGradientStyle(0xe3dfec, 0xe3dfec, 0xc7c4dc, 0xc7c4dc).fillRoundedRect(28,

L77-85 ForStatement: for (let i = 0; i < 35; i++) { const x = 38 + ((i * 137) % 345),

L87-87 ExpressionStatement: rounded(g, 37, 125, 59, 25, 8, theme.ink, 0);

L88-88 ExpressionStatement: rounded(g, 103, 125, 170, 25, 8, 0xfff9ed, 1.5);

L89-96 IfStatement: if (w.rapid > 0) g.fillStyle(POWERUPS.rapid.color).fillRoundedRect(

L97-104 IfStatement: if (w.laser > 0) g.fillStyle(POWERUPS.laser.color).fillRoundedRect(

L105-113 IfStatement: if (w.drone > 0) { powerIcon(g, "drone", 300, 138, 0.75);

L114-117 IfStatement: if (w.shield) { circle(g, 344, 138, 12, POWERUPS.shield.color, 1.5);

L118-174 ForOfStatement: for (const e of w.enemies) if (e.alive) {

L175-189 ForOfStatement: for (const p of w.projectiles) if (p.active) {
L190-205 ForOfStatement: for (const p of w.pickups) if (p.active) {
L206-223 ForOfStatement: for (const b of w.bursts) { const t = b.age / 0.45,
L224-225 FirstStatement: const x = w.player.x, y = w.player.y;
L226-245 IfStatement: if (w.drone > 0) for (const side of [-1, 1]) {
L246-247 IfStatement: if (w.shield) g.lineStyle(2.5, POWERUPS.shield.color, 0.7).strokeCircle(x, y, 27);
L248-251 ExpressionStatement: g.fillStyle(theme.orange,
L252-259 ExpressionStatement: g.lineStyle(3, theme.ink).strokeTriangle(x,
L260-260 ExpressionStatement: rounded(g, x - 5, y - 4, 10, 14, 4, theme.blue, 2);
L261-264 IfStatement: if (w.weaponLevel > 0) g.lineStyle(3, theme.ink)
L265-273 IfStatement: if (firing && !w.transitioning) g.fillStyle(0xfff8d6).fillTriangle(
L274-274 ExpressionStatement: g.lineStyle(4, theme.ink).strokeRoundedRect(24, 112, 372, 538, 20);

src/arcade/space-shooter/ShooterWorld.ts

ACTIVE SOURCE / tool or application | 426 lines | SHA-256 11e9affc2bccf9f0...

Owns Star Patrol movement, swept projectile contacts, wave lifecycle, enemies, pickups, upgrades, damage and bounded pools.

Imports: ./Powerups, ./Waves

State, resources and initial values

Line	Field / declaration
63	player = { x: 210, y: 599 };
64	enemies: Enemy[] = [];
65	projectiles: Projectile[] = Array.from({ length: 80 }, () => ({ x: 0, y: 0, previousX: 0, previousY: 0, vx: 0, vy: 0, enemy: false, active: false, damage: 1, pierce: 1, laser: false, hits: [], }));
79	pickups: Pickup[] = Array.from({ length: 8 }, (_, id) => ({ id, x: 0, y: 0, kind: "spread", active: false, age: 0, }));
87	bursts: Burst[] = [];
88	score = 0;
89	lives = 3;
90	wave = 1;
91	dead = false;
92	invulnerable = 0;
93	weaponLevel = 0;
94	rapid = 0;
95	laser = 0;
96	drone = 0;
97	shield = false;
98	pickupSerial = 0;
99	shotSerial = 0;
100	kills = 0;
101	elapsed = 0;
102	announcement = { text: "", remaining: 0, kind: "spread" as PowerupKind };
103	private direction = 1;
104	private shotClock = 0;
105	private enemyClock = 1.1;
106	private nextWave = 0;
107	private lastLife = -Infinity;
108	private dropSerial = 0;
109	private shotCount = 0;

Enemy [L8-19]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Enemy { x: number; y: number; hp: number; kind: number; alive: boolean; maxHp?: number; phase?: number; charge?: number; drop?: PowerupKind; hit?: number; }
```

Projectile [L20-33]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Projectile { x: number; y: number; previousX: number; previousY: number; vx: number; vy: number; enemy: boolean; active: boolean; damage: number; pierce: number; laser: boolean; hits: Enemy[]; }
```

Burst [L34-39]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Burst { x: number; y: number; age: number; kind: "hit" | "kill" | "pickup"; }
```

contact [L41-61]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: function contact(p: Projectile, x: number, y: number, rx: number, ry: number)

Direct writes: low, high

Return at L51: Infinity

Return at L58: Infinity

Return at L60: low

Decision branches: L50: Math.abs(delta) < 1e-8; L51: Math.abs(start - center) > radius; L58: low > high

ShooterWorld [L62-425]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ShooterWorld.constructor [L110-112]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private random = Math.random)

Calls / delegates: this.spawnWave

ShooterWorld.spawnWave [L113-122]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: spawnWave()

Direct writes: this.enemies, this.direction, this.enemyClock, p.active, p.hits.length, this.invulnerable

Calls / delegates: formation, this.projectiles.forEach

ShooterWorld.speed [L123-127]

Calculate the current legal movement speed from stage/age/score.

Signature: get speed()

Calls / delegates: waveSpeed

Return at L126: waveSpeed(this.wave, killed)

Decision branches: L125: !e.alive

ShooterWorld.weaponName [L128-134]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get weaponName()

Return at L129: this.laser > 0 ? "PIERCING LASER" : ["BLASTER", "TWIN CANNON", "TRIPLE CANNON", "FIVE CANNON"][this.weaponLevel]

```
get weaponName() { return this.laser > 0 ? "PIERCING LASER" : ["BLASTER", "TWIN CANNON", "TRIPLE CANNON", "FIVE CANNON"][ this.weaponLevel ]; }
```

ShooterWorld.fire [L135-160]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private fire(x: number, y: number, enemy: boolean, vx = 0, vy?: number)

Direct writes: p, p.hits.length

Decision branches: L141: !this.projectiles[i].active; L145: !p

ShooterWorld.shoot [L161-179]

Validate and launch a normal shot through the authoritative physics API.

Signature: private shoot()

Calls / delegates: this.fire

Decision branches: L165: this.weaponLevel === 1; L166: this.weaponLevel >= 2; L170: this.weaponLevel >= 3; L174: this.drone > 0 && this.shotCount++ % 2 === 0

ShooterWorld.attack [L180-196]

Fire a role-dependent ordinary enemy pattern, including aimed/paired/boss projectiles.

Signature: private attack(e: Enemy)

Calls / delegates: this.fire

Decision branches: L182: e.kind === 3 || e.kind === 5; L190: e.kind === 5; L193: e.kind === 1 && this.wave >= 2

ShooterWorld.drop [L197-230]

Choose enemy/wave/random upgrade drops, apply restricted extra-life eligibility and reuse a bounded pickup slot.

Signature: private drop(e: Enemy)

Direct writes: kind, this.lastLife

Calls / delegates: canDropLife, this.random, this.pickups.find, this.pickups .filter((p) => p.kind !== "life") .sort, this.pickups .filter

Decision branches: L199: e.kind === 4 && canDropLife(this.wave, this.elapsed, this.lastLife, this.lives); L204: !kind && this.kills % 8 === 0; L214: !kind; L215: kind === "life"; L221: p

Execution order (direct statements; nested callbacks stay within their owner):

L198-198 FirstStatement: let kind = e.drop;

L199-203 IfStatement: if (e.kind === 4 &&

L204-213 IfStatement: if (!kind && this.kills % 8 === 0) { const choices: PowerupKind[] = [

L214-214 IfStatement: if (!kind) return;

L215-215 IfStatement: if (kind === "life") this.lastLife = this.elapsed;

L216-220 FirstStatement: const p = this.pickups.find((p) => !p.active) ??

L221-229 IfStatement: if (p) Object.assign(p, {

ShooterWorld.collect [L231-250]

Apply a collected weapon/timed power/shield/life upgrade, update its bounded duration and visible announcement.

Signature: private collect(p: Pickup)

Direct writes: p.active, this.weaponLevel, this.shield, this[kind], this.announcement

Calls / delegates: this.effect

Decision branches: L235: kind === "spread"; L236: kind === "shield"; L237: kind === "life"

ShooterWorld.effect [L251-254]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private effect(x: number, y: number, kind: Burst["kind"])

Calls / delegates: this.bursts.shift, this.bursts.push

Decision branches: L252: this.bursts.length >= 24

ShooterWorld.hurt [L255-271]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private hurt()

Direct writes: this.shield, this.invulnerable, this.weaponLevel, this.dead, p.active

Calls / delegates: this.effect, this.projectiles.forEach

Decision branches: L256: this.invulnerable > 0; L257: this.shield; L267: this.lives <= 0

ShooterWorld.step [L272-421]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number, targetX: number, firing: boolean)

Direct writes: this.elapsed, this.invulnerable, this[name], this.announcement.remaining, b.age, this.bursts, this.player.x, p.age, p.y, p.x, p.active, this.shotClock, this.nextWave, this.direction, e.y, e.x, e.hit, e.charge, this.enemyClock, p.previousX, p.previousY, nearest, target, target.hp, target.hit, target.alive, this.score

Calls / delegates: this.bursts.filter, this.collect, this.shoot, this.spawnWave, this.enemies.filter, living.some, this.attack, living.filter, this.random, attackInterval, contact, this.hurt, p.hits.includes, p.hits.push, this.effect, this.drop, this.enemies.some, this.projectiles.forEach

Decision branches: L273: this.dead; L290: p.active; L293: p.y > 450 && Math.abs(p.x - this.player.x) < 85; L295: Math.abs(p.x - this.player.x) < 24 && Math.abs(p.y - this.player.y) < 25; L300: p.y > 640 || p.age > 9; L303: firing && this.shotClock <= 0 && !this.transitioning; L307: this.nextWave > 0; L309: this.nextWave <= 0; L317: living.some((e) => e.x + this.direction * speed * dt < 43 || e.x + this.direction * speed * dt > 377,); L339: (e.charge ?? 0) > 0; L341: e.charge === 0; L345: this.enemyClock <= 0 && living.length; L356: e && !e.charge; L360: !p.active; L365: p.enemy; L366: this.invulnerable <= 0 &&

contact(p, this.player.x, this.player.y, 17, 14) !== Infinity; L378: e.alive && !p.hits.includes(e); L386: t < nearest; L391: !target; L394: p.pierce <= 0; L398: target.hp <= 0; L407: p.x < 26 || p.x > 394 || p.y < 116 || p.y > 646; L408: this.dead; L410: living.some((e) => e.alive && e.y > 563); L412: !this.dead; L416: !this.enemies.some((e) => e.alive)

Execution order (direct statements; nested callbacks stay within their owner):

L273-273 IfStatement: if (this.dead) return;

L274-274 ExpressionStatement: this.elapsed += dt;

L275-275 ExpressionStatement: this.invulnerable = Math.max(0, this.invulnerable - dt);

L276-277 ForOfStatement: for (const name of ["rapid", "laser", "drone"] as const) this[name] = Math.max(0, this[name] - dt);

L278-278 ExpressionStatement: this.announcement.remaining = Math.max(0, this.announcement.remaining - dt);

L279-279 ForOfStatement: for (const b of this.bursts) b.age += dt;

L280-280 ExpressionStatement: this.bursts = this.bursts.filter((b) => b.age < 0.45);

L281-288 ExpressionStatement: this.player.x = Math.max(44,

L289-301 ForOfStatement: for (const p of this.pickups) if (p.active) {

L302-302 ExpressionStatement: this.shotClock -= dt;

L303-306 IfStatement: if (firing && this.shotClock <= 0 && !this.transitioning) { this.shoot();

L307-314 IfStatement: if (this.nextWave > 0) { this.nextWave -= dt;

L315-316 FirstStatement: const living = this.enemies.filter((e) => e.alive), speed = this.speed;

L317-326 IfStatement: if (living.some(

L327-343 ForOfStatement: for (const e of living) { const sway =

L344-344 ExpressionStatement: this.enemyClock -= dt;

L345-358 IfStatement: if (this.enemyClock <= 0 && living.length) { const front = living.filter(

L359-409 ForOfStatement: for (const p of this.projectiles) { if (!p.active) continue;

L410-420 IfStatement: if (living.some((e) => e.alive && e.y > 563)) { this.hurt();

ShooterWorld.transitioning [L422-424]

Derived read-only view of existing state; the returned expression below defines its value.

Signature: get transitioning()

Return at L423: this.nextWave > 0

```
get transitioning() { return this.nextWave > 0; }
```

src/arcade/space-shooter/SpaceShooterGame.ts

ACTIVE SOURCE / tool or application | 178 lines | SHA-256 511a937df6114264...

Star Patrol lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../shared/components/drawing, ../../shared/physics/FixedStep, ../../shared/ai/MatchRandom, ./ShooterWorld, ./ShooterArt

State, resources and initial values

Line	Field / declaration
9	private ctx!: GameContext;
10	private world = new ShooterWorld(new MatchRandom(freshSeed()).next);
11	private g!: Phaser.GameObjects.Graphics;
12	private clip!: Phaser.GameObjects.Graphics;
13	private mask!: Phaser.Display.Masks.GeometryMask;
14	private banner!: Phaser.GameObjects.Text;
15	private level!: Phaser.GameObjects.Text;
16	private weapon!: Phaser.GameObjects.Text;
17	private toast!: Phaser.GameObjects.Text;
18	private keys = new Set<string>();
19	private target = 210;
20	private dragging = false;
21	private paused = false;
22	private reported = false;
23	private age = 0;
24	private soundAge = 0;
25	private clock = new FixedStep();

SpaceShooterGame [L8-177]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

SpaceShooterGame.initialize [L26-64]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.clip, this.mask, this.banner, this.level, this.weapon, this.toast, this.keys, this.target, this.dragging

Calls / delegates: ctx.scene.add.graphics, ctx.scene.make.graphics, this.clip.fillStyle(0xffffff).fillRoundedRect, this.clip.fillStyle, this.clip.createGeometryMask, this.g.setMask, label, ctx.input.keys, ctx.setHint, ctx.input.drag, this.draw

Decision branches: L47: this.paused || x < 24 || x > 396 || y < 400 || y > 650

Execution order (direct statements; nested callbacks stay within their owner):

L27-27 ExpressionStatement: this.ctx = ctx;

L28-28 ExpressionStatement: this.g = ctx.scene.add.graphics();

L29-29 ExpressionStatement: this.clip = ctx.scene.make.graphics({ x: 0, y: 0 });

L30-30 ExpressionStatement: this.clip.fillStyle(0xffffff).fillRoundedRect(22, 110, 376, 542, 22);

L31-31 ExpressionStatement: this.mask = this.clip.createGeometryMask();

L32-32 ExpressionStatement: this.g.setMask(this.mask);

L33-33 ExpressionStatement: this.banner = label(ctx.scene, 210, 375, "", 22);

L34-34 ExpressionStatement: this.level = label(ctx.scene, 66, 138, "LV 1", 12, "#fff9ed");

L35-35 ExpressionStatement: this.weapon = label(ctx.scene, 188, 138, "BLASTER", 11);

L36-36 ExpressionStatement: this.toast = label(ctx.scene, 210, 548, "", 14);

L37-43 ExpressionStatement: this.keys = ctx.input.keys(["ArrowLeft",

L44-44 ExpressionStatement: ctx.setHint("");

L45-62 ExpressionStatement: ctx.input.drag((x, y) => {

L63-63 ExpressionStatement: this.draw();

SpaceShooterGame.update [L65-106]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.age, this.target, this.soundAge, this.reported

Calls / delegates: this.clock.advance, this.keys.has, w.step, this.ctx.audio.play, this.draw, this.ctx.setArcadeHUD, "♥ ".repeat, this.ctx.finishArcade

Decision branches: L66: this.paused || this.reported; L77: this.age < 0.7; L78: this.keys.has("ArrowLeft") || this.keys.has("KeyA"); L80: this.keys.has("ArrowRight") || this.keys.has("KeyD"); L82: !this.dragging; L85: w.score > before.score; L86: w.lives < before.lives; L87: w.pickupSerial > before.pickup; L92: w.wave > before.wave; L93: w.shotSerial > before.shot && this.age - this.soundAge > 0.18; L99: w.dead && !this.reported

Execution order (direct statements; nested callbacks stay within their owner):

L66-66 IfStatement: if (this.paused || this.reported) return;

L67-67 ExpressionStatement: this.age += Math.min(dt, 0.1);

L68-75 FirstStatement: const w = this.world, before = {

L76-84 ExpressionStatement: this.clock.advance(dt, (h) => { if (this.age < 0.7) return;

L85-85 IfStatement: if (w.score > before.score) this.ctx.audio.play("hit", 0.35);

L86-86 IfStatement: if (w.lives < before.lives) this.ctx.audio.play("thud", 0.6);

L87-91 IfStatement: if (w.pickupSerial > before.pickup) this.ctx.audio.play(

L92-92 IfStatement: if (w.wave > before.wave) this.ctx.audio.play("start", 0.5);

L93-96 IfStatement: if (w.shotSerial > before.shot && this.age - this.soundAge > 0.18) { this.ctx.audio.play("shoot", 0.12);

L97-97 ExpressionStatement: this.draw();

L98-98 ExpressionStatement: this.ctx.setArcadeHUD(w.score, "♥ ".repeat(w.lives));

L99-105 IfStatement: if (w.dead && !this.reported) { this.reported = true;

SpaceShooterGame.draw [L107-130]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Calls / delegates: drawShooter, this.keys.has, this.level.setText, this.weapon.setText, this.banner.setText, this.toast .setText(t > 0 ? w.announcement.text : "") .setAlpha(Math.min(1, t * 3)) .setY, this.toast .setText(t > 0 ? w.announcement.text : "") .setAlpha, this.toast .setText

SpaceShooterGame.pause [L131-135]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused, this.dragging

Calls / delegates: this.clock.reset

SpaceShooterGame.resume [L136-139]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

Calls / delegates: this.clock.reset

SpaceShooterGame.dispose [L140-148]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.reported

Calls / delegates: this.g.destroy, this.mask.destroy, this.clip.destroy, [this.banner, this.level, this.weapon, this.toast].forEach, t.destroy

SpaceShooterGame.snapshot [L149-176]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: w.enemies.filter((e) => e.alive).map, w.enemies.filter, w.projectiles .filter((p) => p.active) .map, w.projectiles .filter, w.pickups.filter((p) => p.active).map, w.pickups.filter

Return at L151: { player: { ...w.player }, score: w.score, lives: w.lives, wave: w.wave, dead: w.dead, enemies: w.enemies.filter((e) => e.alive).map((e) => ({ ...e })), projectiles: w.projectiles .filter((p) => p.active) .map(({ hits, ...p }) => ({ ...p })), pickups: w.pickups.filter((p) => p.active).map((p) => ({ ...p })), gear: { level: w.weaponLevel, rapid: w.r ... [continued in source reference]

src/arcade/space-shooter/Waves.ts

ACTIVE SOURCE / tool or application | 52 lines | SHA-256 f3100bebb5c59177...

Defines wave formation/enemy roles, hit points, movement-speed curve and telegraphed attack interval.

Imports: ./ShooterWorld, ./Powerups

waveSpeed [L3-4]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: waveSpeed = (wave: number, killed = 0) =>

attackInterval [L5-6]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: attackInterval = (wave: number) =>

formation [L7-51]

Create wave-specific rows/columns, enemy roles/hitpoints and the every-fourth-wave boss.

Signature: export function formation(wave: number): Enemy[]

Calls / delegates: enemies.push, waveReward

Return at L50: enemies

Decision branches: L39: wave % 4 === 0

Execution order (direct statements; nested callbacks stay within their owner):

L8-10 FirstStatement: const enemies: Enemy[] = [], rows = Math.min(5, 2 + Math.floor(wave / 2)),

L11-38 ForStatement: for (let r = 0; r < rows; r++) for (let c = 0; c < columns; c++) {

L39-49 IfStatement: if (wave % 4 === 0) enemies.push({

L50-50 ReturnStatement: return enemies;

src/arcade/space-shooter/config.ts

ACTIVE SOURCE / tool or application | 18 lines | SHA-256 d0d3000a4fe826c2...

Eager lightweight metadata for Star Patrol: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-17

```
export default {
  id: "space-shooter",
  name: "Star Patrol",
  description: "Small ship. Endless courage.",
  instructions:
    "Drag to steer/fire; arrows/A/D + Space. Catch carrier loot for cannons, lasers, wingmates and shields. Lives
are rare. Ace every 4 levels.",
  color: "#d8d4e6",
  accent: "#aaa3c6",
  preview,
  order: 11,
  tag: "A LITTLE SPACE ADVENTURE",
  category: "arcade",
  tournament: false,
  supportedModes: ["single-player"],
} satisfies GameMetadata;
```

src/arcade/space-shooter/index.ts

ACTIVE SOURCE / tool or application | 3 lines | SHA-256 21e609b2663b9e16...

Side-effect-free lazy factory for Star Patrol; implementation initializes only after host selection.

Imports: ./SpaceShooterGame

Top-level block L2-2

```
export default () => new SpaceShooterGame();
```

src/future-games/archery-duel

src/future-games/archery-duel/AI.ts

DEFERRED / unregistered | 51 lines | SHA-256 3ae43ecab1dda2de...

Game-local Archery Duel observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private attempt = -1;
7	private x = 0;
8	private y = 0;
9	private readyDelay = 0;

ArcheryAI [L4-50]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ArcheryAI.constructor [L10-15]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

ArcheryAI.update [L16-49]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(m: ArcheryMatch)

Direct writes: this.attempt, this.x, this.y, this.readyDelay

Calls / delegates: r.next, r.sample

Return at L43: p.age >= this.readyDelay

Return at L44: p.stage === "horizontal" ? Math.abs(p.x - this.x) < 1.8 : p.stage === "vertical" ? Math.abs(p.y - this.y) < 1.8 : false

Decision branches: L17: m.winner; L20: this.attempt !== p.attempt; L43: p.stage === "ready"

Execution order (direct statements; nested callbacks stay within their owner):

L17-17 IfStatement: if (m.winner) return false;

L18-19 FirstStatement: const p = m.players[1], r = this.personality;

L20-42 IfStatement: if (this.attempt !== p.attempt) { this.attempt = p.attempt;

L43-43 IfStatement: if (p.stage === "ready") return p.age >= this.readyDelay;

L44-48 ReturnStatement: return p.stage === "horizontal" ? Math.abs(p.x - this.x) < 1.8

src/future-games/archery-duel/ArcheryGame.ts

DEFERRED / unregistered | 139 lines | SHA-256 622ffd439e84cafc...

Archery Duel lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ../../shared/physics/FixedStep, ../../shared/ai/MatchRandom, ./Rules, ./AI

State, resources and initial values

Line	Field / declaration
10	private ctx!: GameContext;
11	private m = new ArcheryMatch(freshSeed());
12	private ai?: ArcheryAI;
13	private g!: Phaser.GameObjects.Graphics;
14	private texts: Phaser.GameObjects.Text[] = [];
15	private fixed = new FixedStep();
16	private paused = false;
17	private done = false;
18	private scores = "";

ArcheryGame [L9-138]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ArcheryGame.initialize [L19-38]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.texts, this.ai, this.m

Calls / delegates: ctx.scene.add.graphics, label, ArcheryAI, ArcheryMatch, ctx.input.tap, this.tap, ctx.input.press, ctx.setHint, this.draw

Decision branches: L26: ctx.options.mode === "ai"; L31: x >= 24 && x <= 396 && y >= 112 && y <= 650

ArcheryGame.tap [L39-45]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private tap(p: 0 | 1, ai = false)

Calls / delegates: this.m.tap, this.ctx.audio.play

Decision branches: L40: this.paused || this.done || (p === 1 && this.ai && !ai); L41: this.m.tap(p)

ArcheryGame.update [L46-63]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.scores, this.done

Calls / delegates: this.fixed.advance, this.ai?.update, this.tap, this.m.step, this.m.players.map, s.join, this.ctx.audio.play, this.ctx.setScore, this.draw, this.ctx.finish

Decision branches: L47: this.paused || this.done; L49: this.ai?.update(this.m); L53: s.join() !== this.scores; L54: this.scores; L59: this.m.winner

ArcheryGame.draw [L64-117]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Calls / delegates: g.clear, rounded, g.fillStyle(color).fillCircle, g.fillStyle, g.lineStyle(3, theme.ink).strokeCircle, g.lineStyle, g.lineStyle(2, theme.white).strokeCircle, g.fillStyle(theme.ink).fillCircle, g.lineStyle(1, theme.ink, 0.3).lineBetween, g.lineStyle(4, theme.ink).lineBetween, g.lineStyle(3, color, 1 - p.age / 0.7).strokeCircle, this.texts[i].setText

Decision branches: L83: p.stage !== "ready"; L89: p.stage === "flight"; L95: p.stage === "impact"

Execution order (direct statements; nested callbacks stay within their owner):

L65-65 FirstStatement: const g = this.g;

L66-66 ExpressionStatement: g.clear();

L67-67 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xdbe8cf, 4);

L68-116 ForOfStatement: for (const i of [0, 1] as const) { const p = this.m.players[i],

ArcheryGame.pause [L118-121]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.fixed.reset

ArcheryGame.resume [L122-124]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

ArcheryGame.dispose [L125-129]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.done

Calls / delegates: this.g.destroy, this.texts.forEach, t.destroy

ArcheryGame.snapshot [L130-137]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.personality.snapshot

Return at L131: { players: this.m.players, winner: this.m.winner, time: this.m.time, ai: this.ai?.personality.snapshot(), }

src/future-games/archery-duel/Rules.ts

DEFERRED / unregistered | 106 lines | SHA-256 262f1ba3536744bd...

Owns seeded targets, three-tap crosshair locking, delayed exact-radius hit scoring and deadline.

Imports: ../../shared/ai/MatchRandom

State, resources and initial values

Line	Field / declaration
20	readonly players: [Archer, Archer];
21	winner: 1 | 2 | "draw" | null = null;
22	time = 0;

Target [L2-7]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Target { x: number; y: number; r: number; points: number; }
```

Archer [L8-18]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Archer { score: number; attempt: number; stage: "ready" | "horizontal" | "vertical" | "flight" | "impact"; x: number; y: number; direction: number; age: number; targets: Target[]; lastPoints: number; }
```

ArcheryMatch [L19-105]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ArcheryMatch.constructor [L23-36]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly seed: number)

Direct writes: this.players

Calls / delegates: this.targets, make

ArcheryMatch.constructor.make [L24-34]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: make = (): Archer =>

Calls / delegates: this.targets

ArcheryMatch.targets [L37-45]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private targets(n: number)

Calls / delegates: MatchRandom, mixSeed, r.next

Return at L41: [{ x: left, y: 40 + r.next() * 90, r: 28, points: 1 }, { x: right, y: 30 + r.next() * 110, r: 14, points: 2 },]

ArcheryMatch.tap [L46-62]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: tap(i: 0 | 1)

Direct writes: p.stage, p.x, p.y, p.direction, p.age

Decision branches: L48: this.winner || p.age < 0.12; L49: p.stage === "ready"; L54: p.stage === "horizontal"; L58: p.stage === "vertical"

ArcheryMatch.step [L63-104]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number)

Direct writes: this.time, p.age, p[key], p.direction, p.lastPoints, p.score, p.stage, p.targets, this.winner

Calls / delegates: p.targets.find, this.targets

Decision branches: L64: this.winner; L68: p.stage === "horizontal" || p.stage === "vertical"; L72: p[key] > limit; L76: p[key] < 0; L80: p.stage === "flight" && p.age >= 0.38; L87: p.stage === "impact" && p.age >= 0.7; L96: a || b; L97: this.time >= 180 && !this.winner

Execution order (direct statements; nested callbacks stay within their owner):

L64-64 IfStatement: if (this.winner) return;

L65-65 ExpressionStatement: this.time += dt;

L66-93 ForOfStatement: for (const p of this.players) { p.age += dt;

L94-95 FirstStatement: const a = this.players[0].score >= 10, b = this.players[1].score >= 10;

L96-96 IfStatement: if (a || b) this.winner = a && b ? "draw" : a ? 1 : 2;

L97-103 IfStatement: if (this.time >= 180 && !this.winner) this.winner =

src/future-games/archery-duel/config.ts

DEFERRED / unregistered | 17 lines | SHA-256 44a3beb7ed1f4d0c...

Eager lightweight metadata for Archery Duel: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "archery-duel",
  name: "Archery Duel",
  description: "Lock your aim. Let it fly.",
  instructions:
    "Three taps: start horizontal aiming, lock X, then lock Y and fire. Large target: 1 point. Small: 2. First to 10
wins. At three minutes, higher score wins; ties draw. Orange: Space/A. Blue: Enter/L.",
  color: "#dbe8cf",
  accent: "#a6b998",
  preview,
  order: 17,
  tag: "AIM TRUE",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/future-games/archery-duel/index.ts

DEFERRED / unregistered | 3 lines | SHA-256 67b7abbaa436590d...

Side-effect-free lazy factory for Archery Duel; implementation initializes only after host selection.

Imports: ./ArcheryGame

Top-level block L2-2

```
export default () => new ArcheryGame();
```

src/future-games/car-duel

src/future-games/car-duel/AI.ts

DEFERRED / unregistered | 59 lines | SHA-256 7623eb802dbca5c0...

Game-local Car Duel observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private wait = 0;
7	private control = 0;
8	private retreat = 0;

CarAI [L4-58]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

CarAI.constructor [L9-14]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

CarAI.update [L15-57]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, m: CarMatch)

Direct writes: this.control, this.wait, this.retreat

Calls / delegates: r.sample

Return at L22: this.control

Return at L40: this.control

Return at L56: this.control

Decision branches: L16: m.winner || m.between; L22: this.wait > 0; L38: Math.abs(predicted) > 0.42; L45: cautious && separation < 95 && closing < -90 && this.retreat <= 0; L50: separation < 70 && Math.abs(c.omega) > 0.8 && this.difficulty !== "medium"

Execution order (direct statements; nested callbacks stay within their owner):

L16-20 IfStatement: if (m.winner || m.between) { this.control = 0;

L21-21 ExpressionStatement: this.wait -= dt;

L22-22 IfStatement: if (this.wait > 0) return this.control;

L23-26 FirstStatement: const r = this.personality, c = m.cars[1],

L27-35 ExpressionStatement: this.wait = r.sample(this.difficulty === "medium"

L36-37 FirstStatement: const predicted = c.angle + c.omega * (this.difficulty === "medium" ? 0.08 : 0.22);

L38-41 IfStatement: if (Math.abs(predicted) > 0.42) { this.control = Math.sign(predicted);

L42-44 FirstStatement: const closing = (other.vx - c.vx) * Math.sign(distance), separation = Math.abs(distance),

L45-46 IfStatement: if (cautious && separation < 95 && closing < -90 && this.retreat <= 0) this.retreat = 0.35 + r.tempo * 0.08;

L47-47 ExpressionStatement: this.retreat -= this.wait;

L48-49 ExpressionStatement: this.control = this.retreat > 0 ? -Math.sign(distance) : Math.sign(distance);

L50-55 IfStatement: if (separation < 70 &&

L56-56 ReturnStatement: return this.control;

src/future-games/car-duel/CarGame.ts

DEFERRED / unregistered | 165 lines | SHA-256 f83c8c4b6bea0d6d...

Car Duel lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ../../shared/physics/FixedStep, ./Rules, ./AI, ./CarInput, ./Physics

State, resources and initial values

Line	Field / declaration
11	private ctx!: GameContext;
12	private m = new CarMatch();
13	private ai?: CarAI;
14	private controls!: CarInput;
15	private keys = new Set<string>();
16	private g!: Phaser.GameObjects.Graphics;
17	private texts: Phaser.GameObjects.Text[] = [];
18	private fixed = new FixedStep();
19	private paused = false;
20	private done = false;
21	private score = "";

CarGame [L10-164]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

CarGame.initialize [L22-39]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.ai, this.controls, this.keys, this.texts

Calls / delegates: ctx.scene.add.graphics, CarAI, CarInput, ctx.input.keys, label, this.texts.push, ctx.setHint, this.draw

Decision branches: L25: ctx.options.mode === "ai"

CarGame.update [L40-75]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.score, this.done

Calls / delegates: this.fixed.advance, this.controls.value, Number, this.keys.has, this.ai.update, this.m.step, this.m.scores.join, this.ctx.audio.play, this.ctx.setScore, this.draw, this.ctx.finish

Decision branches: L41: this.paused || this.done; L65: this.m.scores.join() !== this.score; L66: this.score; L71: this.m.winner

Execution order (direct statements; nested callbacks stay within their owner):

L41-41 IfStatement: if (this.paused || this.done) return;

L42-64 ExpressionStatement: this.fixed.advance(dt, (h) => { const one = Math.max(

L65-69 IfStatement: if (this.m.scores.join() !== this.score) { if (this.score) this.ctx.audio.play("metal");

L70-70 ExpressionStatement: this.draw();

L71-74 IfStatement: if (this.m.winner) { this.done = true;

CarGame.draw [L76-137]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Direct writes: x

Calls / delegates: g.clear, rounded, g.fillStyle(0xaab9a0).beginPath().moveTo, g.fillStyle(0xaab9a0).beginPath, g.fillStyle, g.lineTo, terrain, g.lineTo(380, 490).closePath().fillPath, g.lineTo(380, 490).closePath, g.lineStyle(3, theme.ink).beginPath, g.lineStyle, g.moveTo, g.strokePath, [[-26, -12], [26, -12], [26, 12], [-26, 12],].map, point, g.fillStyle(c.alive ? color : theme.muted) .beginPath() .moveTo, g.fillStyle(c.alive ? color : theme.muted) .beginPath, corners.slice(1).forEach, corners.slice, g.closePath().fillPath().lineStyle(3, theme.ink).strokePath, g.closePath().fillPath().lineStyle, g.closePath().fillPath, g.closePath, g.fillStyle(theme.ink).fillCircle, g.fillStyle(theme.white).fillCircle, head, g.fillStyle(c.alive ? color : theme.muted).fillCircle, g.lineStyle(2, theme.ink).strokeCircle, this.texts.forEach, t.setDepth, this.texts[0].setText

Decision branches: L86: x === 0

Execution order (direct statements; nested callbacks stay within their owner):

L77-77 FirstStatement: const g = this.g;

L78-78 ExpressionStatement: g.clear();

L79-79 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xcddfdb, 4);

L80-80 ExpressionStatement: rounded(g, 40, 185, 340, 315, 12, theme.white, 2);

L81-81 ExpressionStatement: g.fillStyle(0xaab9a0).beginPath().moveTo(40, 490);

L82-82 ForStatement: for (let x = 0; x <= 340; x += 4) g.lineTo(40 + x, 200 + terrain(x));

L83-83 ExpressionStatement: g.lineTo(380, 490).closePath().fillPath();

L84-84 ExpressionStatement: g.lineStyle(3, theme.ink).beginPath();

L85-88 ForStatement: for (let x = 0; x <= 340; x += 4) { if (x === 0) g.moveTo(40, 200 + terrain(x));

L89-89 ExpressionStatement: g.strokePath();

L90-121 ForOfStatement: for (const i of [0, 1] as const) { const c = this.m.cars[i],

L122-132 ForOfStatement: for (const b of BUTTONS) rounded(

L133-133 ExpressionStatement: this.texts.forEach((t) => t.setDepth(2));

L134-136 ExpressionStatement: this.texts[0].setText(this.m.between ? "DRIVERS, GET READY…" : `Round ${this.m.round}`,

CarGame.pause [L138-142]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.fixed.reset, this.controls.pause

CarGame.resume [L143-146]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

Calls / delegates: this.controls.resume

CarGame.dispose [L147-152]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.done

Calls / delegates: this.controls.dispose, this.g.destroy, this.texts.forEach, t.destroy

CarGame.snapshot [L153-163]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.personality.snapshot

Return at L154: { cars: this.m.cars, scores: this.m.scores, round: this.m.round, between: this.m.between, time: this.m.time, winner: this.m.winner, ai: this.ai?.personality.snapshot(), }

src/future-games/car-duel/CarInput.ts

DEFERRED / unregistered | 74 lines | SHA-256 a33050c5ca0fcfb6...

Owns separate per-pointer car-button holds and explicit pause/disposal subscriptions.

Imports: phaser

State, resources and initial values

Line	Field / declaration
9	private held = new Map<number, { p: 0 | 1; d: number }>();
10	private enabled = true;

Module constants and data

L2-7 BUTTONS = [{ x: 40, y: 550, w: 74, h: 70, p: 0, d: -1 }, { x: 122, y: 550, w: 74, h: 70, p: 0, d: 1 }, { x: 224, y: 550, w: 74, h: 70, p: 1, d: -1 }, { x: 306, y: 550, w: 74, h: 70, p: 1, d: 1 },] as const

CarInput [L8-73]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

CarInput.constructor [L11-21]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private scene: Phaser.Scene, private ai: boolean,)

Calls / delegates: scene.input.on, scene.game.canvas.addEventListener

CarInput.button [L22-27]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private button(p: Phaser.Input.Pointer)

Calls / delegates: this.scene.cameras.main.getWorldPoint, BUTTONS.find

Return at L24: BUTTONS.find((b) => v.x >= b.x && v.x <= b.x + b.w && v.y >= b.y && v.y <= b.y + b.h,)

CarInput.down [L28-32]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private down = (p: Phaser.Input.Pointer) =>

Calls / delegates: this.button, this.held.set

Decision branches: L30: this.enabled && b && !(b.p === 1 && this.ai)

CarInput.move [L33-38]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private move = (p: Phaser.Input.Pointer) =>

Direct writes: owner.d

Calls / delegates: this.held.get, this.button

Decision branches: L35: !owner

CarInput.up [L39-41]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private up = (p: Phaser.Input.Pointer) =>

Calls / delegates: this.held.delete

CarInput.clear [L42-44]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: clear = () =>

Calls / delegates: this.held.clear

CarInput.value [L45-55]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: value(p: 0 | 1)

Calls / delegates: [...this.held.values()] .filter((v) => v.p === p) .reduce, [...this.held.values()] .filter, this.held.values

Return at L46: Math.max(-1, Math.min(1, [...this.held.values()] .filter((v) => v.p === p) .reduce((s, v) => s + v.d, 0),),)

CarInput.pause [L56-59]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.enabled

Calls / delegates: this.clear

CarInput.resume [L60-63]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.enabled

Calls / delegates: this.clear

CarInput.dispose [L64-72]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.pause, this.scene.input.off, this.scene.game.canvas.removeEventListener

src/future-games/car-duel/Physics.ts

DEFERRED / unregistered | 135 lines | SHA-256 06553e19ba8ddd4c...

Custom 120Hz car-body approximation: terrain normals, shared head/chassis shapes, acceleration/rotation caps, contacts and impulses.

Module constants and data

L12-12 INERTIA = 560

Car [L1-11]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Car { x: number; y: number; vx: number; vy: number; angle: number; omega: number; facing: 1 | -1; alive: boolean; grounded: boolean; }
```

terrain [L13-16]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function terrain(x: number)

Return at L15: 260 - (Math.abs(d) < 1 ? 25 * (1 - d * d) ** 2 : 0)

point [L17-21]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function point(c: Car, x: number, y: number)

Return at L20: { x: c.x + x * cs - y * sn, y: c.y + x * sn + y * cs }

head [L22-24]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function head(c: Car)

Calls / delegates: point

Return at L23: point(c, -c.facing * 5, -22)

colliders [L25-32]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function colliders(c: Car)

Calls / delegates: [[-14, 0, 12], [14, 0, 12], [-21, 12, 10], [21, 12, 10],].map, point

Return at L26: [[-14, 0, 12], [14, 0, 12], [-21, 12, 10], [21, 12, 10],].map(([x, y, r]) => ({ ...point(c, x, y), r }))

impulse [L33-44]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: function impulse(c: Car, x: number, y: number, nx: number, ny: number, j: number,)

Direct writes: c.vx, c.vy, c.omega

integrate [L45-92]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function integrate(c: Car, input: number, dt: number)

Direct writes: input, c.vx, c.vy, c.omega, c.x, c.y, c.angle, c.grounded, c.alive

Calls / delegates: colliders, terrain, impulse, head

Decision branches: L46: !c.alive; L65: depth > 0; L76: vn < 0; L81: q.x - q.r < 0; L83: c.vx < 0; L85: q.x + q.r > 340; L87: c.vx > 0; L91: h.y + 7 >= terrain(h.x) || h.x - 7 < 0 || h.x + 7 > 340

Execution order (direct statements; nested callbacks stay within their owner):

L46-46 IfStatement: if (!c.alive) return;

L47-47 ExpressionStatement: input = Math.max(-1, Math.min(1, input));

L48-48 ExpressionStatement: c.vx += input * (c.grounded ? 145 : 35) * dt;

L49-49 ExpressionStatement: c.vy += 520 * dt;

L50-50 ExpressionStatement: c.omega -= input * (c.grounded ? 1.1 : 3.2) * dt;

L51-51 ExpressionStatement: c.vx *= Math.exp(-0.65 * dt);

L52-52 ExpressionStatement: c.omega *= Math.exp(-0.65 * dt);

L53-53 ExpressionStatement: c.vx = Math.max(-180, Math.min(180, c.vx));

L54-54 ExpressionStatement: c.vy = Math.max(-260, Math.min(300, c.vy));

L55-55 ExpressionStatement: c.omega = Math.max(-5, Math.min(5, c.omega));

L56-56 ExpressionStatement: c.x += c.vx * dt;

L57-57 ExpressionStatement: c.y += c.vy * dt;

L58-58 ExpressionStatement: c.angle += c.omega * dt;

L59-59 ExpressionStatement: c.angle = Math.atan2(Math.sin(c.angle), Math.cos(c.angle));

L60-60 ExpressionStatement: c.grounded = false;

L61-89 ForStatement: for (let pass = 0; pass < 3; pass++) for (const q of colliders(c)) {

L90-90 FirstStatement: const h = head(c);

L91-91 IfStatement: if (h.y + 7 >= terrain(h.x) || h.x - 7 < 0 || h.x + 7 > 340) c.alive = false;

collide [L93-134]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function collide(a: Car, b: Car)

Direct writes: a.alive, b.alive, a.x, a.y, b.x, b.y

Calls / delegates: head, colliders, bc.some, ac.some, impulse

Decision branches: L98: bc.some((q) => Math.hypot(q.x - ah.x, q.y - ah.y) < q.r + 7); L100: ac.some((q) => Math.hypot(q.x - bh.x, q.y - bh.y) < q.r + 7); L102: Math.hypot(ah.x - bh.x, ah.y - bh.y) < 14; L112: over <= 0; L126: relative < 0

Execution order (direct statements; nested callbacks stay within their owner):

L94-97 FirstStatement: const ah = head(a), bh = head(b),

L98-99 IfStatement: if (bc.some((q) => Math.hypot(q.x - ah.x, q.y - ah.y) < q.r + 7)) a.alive = false;

L100-101 IfStatement: if (ac.some((q) => Math.hypot(q.x - bh.x, q.y - bh.y) < q.r + 7)) b.alive = false;

L102-105 IfStatement: if (Math.hypot(ah.x - bh.x, ah.y - bh.y) < 14) { a.alive = false;

L106-133 ForOfStatement: for (const p of ac) for (const q of bc) {

src/future-games/car-duel/Rules.ts

DEFERRED / unregistered | 59 lines | SHA-256 010afff9bef31c53...

Owns car spawn/round scoring, simultaneous-death/stall replays and match deadline.

Imports: ./Physics

State, resources and initial values

Line	Field / declaration
3	cars: [Car, Car] = this.spawn();
4	scores: [number, number] = [0, 0];
5	winner: 1 | 2 | "draw" | null = null;
6	between = 1;
7	round = 1;
8	time = 0;
9	roundTime = 0;
10	private first = true;

CarMatch [L2-58]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

CarMatch.spawn [L11-23]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: private spawn(): [Car, Car]

Calls / delegates: [0, 1].map

Return at L12: [0, 1].map((i) => ({ x: i ? 265 : 75, y: 224, vx: 0, vy: 0, angle: 0, omega: 0, facing: i ? -1 : 1, alive: true, grounded: false, })) as [Car, Car]

CarMatch.step [L24-57]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number, controls: [number, number])

Direct writes: this.time, this.winner, this.between, this.cars, this.roundTime, this.first

Calls / delegates: this.spawn, integrate, collide, this.cars.some, this.scores.some

Decision branches: L25: this.winner; L27: this.time >= 240; L36: this.between; L38: !this.between; L41: !this.first; L50: this.cars.some((c) => !c.alive) || this.roundTime >= 40; L51: this.cars[0].alive !== this.cars[1].alive; L54: this.scores.some((s) => s >= 5)

Execution order (direct statements; nested callbacks stay within their owner):

L25-25 IfStatement: if (this.winner) return;

L26-26 ExpressionStatement: this.time += dt;

L27-35 IfStatement: if (this.time >= 240) { this.winner =

L36-45 IfStatement: if (this.between) { this.between = Math.max(0, this.between - dt);

L46-46 ExpressionStatement: this.roundTime += dt;

L47-47 ExpressionStatement: integrate(this.cars[0], controls[0], dt);

L48-48 ExpressionStatement: integrate(this.cars[1], controls[1], dt);

L49-49 ExpressionStatement: collide(...this.cars);

L50-56 IfStatement: if (this.cars.some((c) => !c.alive) || this.roundTime >= 40) { if (this.cars[0].alive !== this.cars[1].alive)

src/future-games/car-duel/config.ts

DEFERRED / unregistered | 17 lines | SHA-256 a01b15559e241d85...

Eager lightweight metadata for Car Duel: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "car-duel",
  name: "Car Duel",
  description: "Bump, balance, and protect your driver.",
  instructions:
    "Hold left/right to drive and rotate in the air. Hit the other driver while protecting your head. Head contact
with terrain, walls or the rival car loses the round. Simultaneous hits replay. First to 5; four-minute cap uses
scores. Orange controls left: A/D. Blue right: arrows.",
  color: "#cddfdb",
  accent: "#aab9a0",
  preview,
  order: 22,
  tag: "HEADS UP",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/future-games/car-duel/index.ts

DEFERRED / unregistered | 3 lines | SHA-256 98418e818b8ed51d...

Side-effect-free lazy factory for Car Duel; implementation initializes only after host selection.

Imports: ./CarGame

Top-level block L2-2

```
export default () => new CarGame();
```

src/future-games/chicken-jump

src/future-games/chicken-jump/AI.ts

DEFERRED / unregistered | 43 lines | SHA-256 3e4cba5353782f49...

Game-local Chicken Jump observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private newAttempt = true;
7	private offset = 0;
8	private delay = 0;

ChickenAI [L4-42]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ChickenAI.constructor [L9-14]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

ChickenAI.update [L15-41]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, m: ChickenMatch)

Direct writes: this.newAttempt, this.offset, this.delay

Calls / delegates: this.personality.sample, m.speed

Return at L40: Math.abs(p.block - target) <= m.speed(p) * dt + 2

Decision branches: L17: m.winner; L18: p.phase !== "sliding"; L22: this.newAttempt; L38: this.delay > 0

src/future-games/chicken-jump/ChickenGame.ts

DEFERRED / unregistered | 143 lines | SHA-256 be5289b6e0d8aa90...

Chicken Jump lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ../../shared/physics/FixedStep, ../../shared/ai/MatchRandom, ./Rules, ./AI

State, resources and initial values

Line	Field / declaration
10	private ctx!: GameContext;
11	private m = new ChickenMatch(freshSeed());
12	private ai?: ChickenAI;
13	private g!: Phaser.GameObjects.Graphics;
14	private text!: Phaser.GameObjects.Text;
15	private fixed = new FixedStep();
16	private paused = false;
17	private done = false;
18	private score = "";

ChickenGame [L9-142]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ChickenGame.initialize [L19-35]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.text, this.ai, this.m

Calls / delegates: ctx.scene.add.graphics, label, ChickenAI, ChickenMatch, ctx.input.tap, this.stop, ctx.input.press, ctx.setHint, this.draw

Decision branches: L23: ctx.options.mode === "ai"; L28: x >= 24 && x <= 396 && y >= 112 && y <= 650

ChickenGame.stop [L36-39]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private stop(p: 0 | 1, ai = false)

Calls / delegates: this.m.stop, this.ctx.audio.play

Decision branches: L37: this.paused || this.done || (p === 1 && this.ai && !ai); L38: this.m.stop(p)

ChickenGame.update [L40-57]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.score, this.done

Calls / delegates: this.fixed.advance, this.ai?.update, this.stop, this.m.step, this.m.players.map, scores.join, this.ctx.audio.play, this.ctx.setScore, this.draw, this.ctx.finish

Decision branches: L41: this.paused || this.done; L43: this.ai?.update(h, this.m); L47: scores.join() !== this.score; L48: this.score; L53: this.m.winner

ChickenGame.draw [L58-121]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Calls / delegates: g.clear, rounded, g.lineStyle(3, theme.ink).lineBetween, g.lineStyle, g.fillStyle(color).fillTriangle, g.fillStyle, g.fillStyle(theme.ink).fillCircle, g.fillStyle(color) .fillCircle(x - 5, y - 24, 5) .fillCircle, g.fillStyle(color) .fillCircle, g.lineStyle(2, theme.ink, 0.3).lineBetween, this.text.setText

Decision branches: L80: j === 1; L116: p.phase === "sliding"

Execution order (direct statements; nested callbacks stay within their owner):

L59-59 FirstStatement: const g = this.g;

L60-60 ExpressionStatement: g.clear();

L61-61 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, 0xf6e6b8, 4);

L62-119 ForOfStatement: for (const i of [0, 1] as const) { const p = this.m.players[i],

L120-120 ExpressionStatement: this.text.setText("Tap to lock");

ChickenGame.pause [L122-125]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.fixed.reset

ChickenGame.resume [L126-128]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

ChickenGame.dispose [L129-133]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.done

Calls / delegates: this.g.destroy, this.text.destroy

ChickenGame.snapshot [L134-141]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.personality.snapshot

Return at L135: { players: this.m.players, time: this.m.time, winner: this.m.winner, ai: this.ai?.personality.snapshot(), }

src/future-games/chicken-jump/Rules.ts

DEFERRED / unregistered | 108 lines | SHA-256 6ce62a01aa4117aa...

Owns sliding blocks, legal launch, descending geometric landings, retries and stack race.

Imports: ../../shared/ai/MatchRandom

State, resources and initial values

Line	Field / declaration
15	readonly players: [Chicken, Chicken];
16	winner: 1 | 2 | "draw" | null = null;
17	time = 0;

Chicken [L2-13]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Chicken { score: number; x: number; block: number; direction: number; width: number; phase: "sliding" | "jumping" | "miss"; y: number; vy: number; age: number; stack: { x: number; width: number }[]; }
```

ChickenMatch [L14-107]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

ChickenMatch.constructor [L18-34]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(seed: number)

Direct writes: this.players

Calls / delegates: MatchRandom, r.next, make

ChickenMatch.constructor.make [L21-32]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: make = (): Chicken =>

```
make = (): Chicken => ({ score: 0, x: 100, block: start, direction: r.side, width: 65, phase: "sliding", y: 0, vy: 0, age: 0, stack: [], })
```

ChickenMatch.speed [L35-37]

Calculate the current legal movement speed from stage/age/score.

Signature: speed(p: Chicken)

Return at L36: 92 + Math.min(85, p.score * 8)

ChickenMatch.stop [L38-46]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: stop(i: 0 | 1)

Direct writes: p.phase, p.vy, p.y, p.age

Decision branches: L40: this.winner || p.phase !== "sliding"

ChickenMatch.step [L47-106]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number)

Direct writes: this.time, p.age, p.block, p.direction, p.vy, p.y, p.x, p.width, p.phase, this.winner

Calls / delegates: this.speed, p.stack.push, p.stack.shift

Decision branches: L48: this.winner; L52: p.phase === "sliding"; L55: p.block < edge; L59: p.block > 200 - edge; L63: p.phase === "jumping"; L67: p.vy > 0 && old <= -26 && p.y >= -26 && Math.abs(p.x - p.block) <= p.width / 2 + 6; L79: p.stack.length > 6; L85: p.y > 60; L89: p.age > 0.65; L98: a || b; L99: this.time >= 180 && !this.winner

Execution order (direct statements; nested callbacks stay within their owner):

L48-48 IfStatement: if (this.winner) return;

L49-49 ExpressionStatement: this.time += dt;

L50-95 ForOfStatement: for (const p of this.players) { p.age += dt;

L96-97 FirstStatement: const a = this.players[0].score >= 10, b = this.players[1].score >= 10;

L98-98 IfStatement: if (a || b) this.winner = a && b ? "draw" : a ? 1 : 2;

L99-105 IfStatement: if (this.time >= 180 && !this.winner) this.winner =

src/future-games/chicken-jump/config.ts

DEFERRED / unregistered | 17 lines | SHA-256 9d8fe955bc17e033...

Eager lightweight metadata for Chicken Jump: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "chicken-jump",
  name: "Chicken Jump",
  description: "One tap. One block. A taller tower.",
  instructions:
    "Tap when the sliding block is under your chicken. Land to stack a point; a miss retries that height. First to 10 wins. Three-minute cap: highest stack wins, equal stacks draw. Orange: Space/A. Blue: Enter/L.",
  color: "#f6e6b8",
  accent: "#c6ad85",
  preview,
  order: 16,
  tag: "STACK & HOP",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/future-games/chicken-jump/index.ts

DEFERRED / unregistered | 3 lines | SHA-256 06d8f97994a7fbb8...

Side-effect-free lazy factory for Chicken Jump; implementation initializes only after host selection.

Imports: ./ChickenGame

Top-level block L2-2

```
export default () => new ChickenGame();
```

src/future-games/flappy-jump

src/future-games/flappy-jump/AI.ts

DEFERRED / unregistered | 47 lines | SHA-256 a30ebcecfe932114...

Game-local Flappy Jump observations, difficulty settings, seeded decision state and ordinary legal actions; it does not award scores directly.

Imports: ../../shared/ai/MatchRandom, ../../shared/contracts/AIController, ./Rules

State, resources and initial values

Line	Field / declaration
5	readonly personality: MatchRandom;
6	private attempt = 0;
7	private waiting = false;
8	private offset = 0;
9	private delay = 0;

FlappyAI [L4-46]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FlappyAI.constructor [L10-15]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(private difficulty: Difficulty, seed?: number,)

Direct writes: this.personality

Calls / delegates: MatchRandom

FlappyAI.update [L16-45]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number, m: FlappyMatch)

Direct writes: this.waiting, this.offset, this.delay

Calls / delegates: r.sample, projectedLanding

Return at L41: (this.delay <= 0 && Math.abs(projectedLanding(p) - p.hoop.x - this.offset) < 1.5)

Decision branches: L18: m.winner; L19: p.ball || p.jumping; L23: !this.waiting

src/future-games/flappy-jump/FlappyGame.ts

DEFERRED / unregistered | 124 lines | SHA-256 cc8655471a9cf1a6...

Flappy Jump lifecycle controller: creates scene-owned display/input, coordinates local rules/physics/AI, renders state, reports one terminal result and disposes resources.

Imports: phaser, ../../shared/contracts/GameModule, ../../core/ThemeManager, ../../shared/components/drawing, ../../shared/physics/FixedStep, ../../shared/ai/MatchRandom, ./Rules, ./AI

State, resources and initial values

Line	Field / declaration
10	private ctx!: GameContext;
11	private m = new FlappyMatch(freshSeed());
12	private ai?: FlappyAI;
13	private g!: Phaser.GameObjects.Graphics;
14	private text!: Phaser.GameObjects.Text;
15	private fixed = new FixedStep();
16	private paused = false;
17	private done = false;
18	private score = "";

FlappyGame [L9-123]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FlappyGame.initialize [L19-35]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.ctx, this.g, this.text, this.ai, this.m

Calls / delegates: ctx.scene.add.graphics, label, FlappyAI, FlappyMatch, ctx.input.tap, this.jump, ctx.input.press, ctx.setHint, this.draw

Decision branches: L23: ctx.options.mode === "ai"; L28: x >= 24 && x <= 396 && y >= 112 && y <= 650

FlappyGame.jump [L36-39]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private jump(p: 0 | 1, ai = false)

Calls / delegates: this.m.jump, this.ctx.audio.play

Decision branches: L37: this.paused || this.done || (p === 1 && this.ai && !ai); L38: this.m.jump(p)

FlappyGame.update [L40-57]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(dt: number)

Direct writes: this.score, this.done

Calls / delegates: this.fixed.advance, this.ai?.update, this.jump, this.m.step, this.m.players.map, scores.join, this.ctx.audio.play, this.ctx.setScore, this.draw, this.ctx.finish

Decision branches: L41: this.paused || this.done; L43: this.ai?.update(h, this.m); L47: scores.join() !== this.score; L48: this.score; L53: this.m.winner

FlappyGame.draw [L58-102]

Draw the current state and presentation effects; legal state is owned by rules/physics.

Signature: private draw()

Calls / delegates: g.clear, rounded, g.lineStyle(3, theme.ink).lineBetween, g.lineStyle, g.lineStyle(4, theme.ink).lineBetween, g.lineStyle(2, 0x9aaca8) .lineBetween(hx - 21, hy, hx - 11, hy + 28) .lineBetween(hx + 21, hy, hx + 11, hy + 28) .lineBetween, g.lineStyle(2, 0x9aaca8) .lineBetween(hx - 21, hy, hx - 11, hy + 28) .lineBetween, g.lineStyle(2, 0x9aaca8) .lineBetween, g.fillStyle(color) .fillCircle(hx - 21, hy, 3) .fillCircle, g.fillStyle(color) .fillCircle, g.fillStyle, projectedLanding, g.lineStyle(6, ready ? 0x72a96e : 0xd6d3c8).lineBetween, g.fillStyle(theme.white).fillCircle, g.lineStyle(3, theme.ink).strokeCircle, g.fillStyle(color).fillCircle, g.lineStyle(2, theme.ink).strokeCircle, g.lineBetween

Decision branches: L95: p.ball

Execution order (direct statements; nested callbacks stay within their owner):

L59-59 FirstStatement: const g = this.g;

L60-60 ExpressionStatement: g.clear();

L61-61 ExpressionStatement: rounded(g, 24, 112, 372, 538, 20, theme.peach, 4);

L62-101 ForOfStatement: for (const i of [0, 1] as const) { const p = this.m.players[i],

FlappyGame.pause [L103-106]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

Calls / delegates: this.fixed.reset

FlappyGame.resume [L107-109]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

FlappyGame.dispose [L110-114]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Direct writes: this.done

Calls / delegates: this.g.destroy, this.text.destroy

FlappyGame.snapshot [L115-122]

Return diagnostic copies/observations for development and tests.

Signature: snapshot()

Calls / delegates: this.ai?.personality.snapshot

Return at L116: { players: this.m.players, time: this.m.time, winner: this.m.winner, ai: this.ai?.personality.snapshot(), }

src/future-games/flappy-jump/Rules.ts

DEFERRED / unregistered | 158 lines | SHA-256 013148249c720ef8...

Owns running/jump/release basketball trajectories, swept hoop scoring and match termination.

Imports: ../../shared/ai/MatchRandom

State, resources and initial values

Line	Field / declaration
36	players: [Jumper, Jumper];
37	winner: 1 | 2 | "draw" | null = null;
38	time = 0;

BasketBall [L2-9]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface BasketBall { x: number; y: number; vx: number; vy: number; age: number; scored: boolean; }
```

Jumper [L10-20]

Compile-time data contract/type; it describes shape and allowed values, not a runtime update.

```
export interface Jumper { x: number; y: number; vy: number; direction: number; score: number; jumping: boolean;
released: boolean; hoop: { x: number; y: number }; ball?: BasketBall; }
```

walk [L21-28]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: export function walk(x: number, direction: number, seconds: number)

Direct writes: p

Return at L27: p

Decision branches: L24: p < 45

projectedLanding [L29-34]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function projectedLanding(p: Jumper)

Calls / delegates: walk

Return at L33: releaseX + 225 * t

FlappyMatch [L35-157]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

FlappyMatch.constructor [L39-51]

Establish this component's fields and injected dependencies; active module resources normally begin in initialize.

Signature: constructor(readonly seed: number)

Direct writes: this.players

Calls / delegates: this.hoop, make

FlappyMatch.constructor.make [L40-49]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: make = (): Jumper =>

Calls / delegates: this.hoop

FlappyMatch.hoop [L52-55]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: private hoop(n: number)

Calls / delegates: MatchRandom, mixSeed, r.next

Return at L54: { x: 250 + r.next() * 38, y: 68 + r.next() * 15 }

FlappyMatch.jump [L56-63]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: jump(i: 0 | 1)

Direct writes: p.jumping, p.released, p.vy

Decision branches: L58: this.winner || p.jumping || p.ball

FlappyMatch.step [L64-156]

Advance the local simulation by dt, enforce contacts/transitions and update legal state.

Signature: step(dt: number)

Direct writes: this.time, p.x, p.direction, p.vy, p.y, p.released, p.ball, p.jumping, b.age, b.vy, b.x, b.y, b.vx, b.scored, p.hoop, this.winner

Calls / delegates: this.hoop

Decision branches: L65: this.winner; L69: p.x > 195; L73: p.x < 45; L77: p.jumping; L81: prev < 0 && p.vy >= 0 && !p.released && !p.ball; L92: p.y >= 155; L99: !b; L111: d < 10; L117: vn < 0; L124: b.y >= p.hoop.y - 36 && b.y <= p.hoop.y + 7 && b.x + 7 >= board && oldX + 7 < board; L133: !b.scored && oldY < p.hoop.y && b.y >= p.hoop.y && b.vy > 0; L136: crossX > p.hoop.x - 14 && crossX < p.hoop.x + 14; L141: b.y > 190 || b.x > 355 || b.x < -15 || b.age > 3; L143: b.scored; L148: a || b; L149: this.time >= 240 && !this.winner

Execution order (direct statements; nested callbacks stay within their owner):

L65-65 IfStatement: if (this.winner) return;

L66-66 ExpressionStatement: this.time += dt;

L67-145 ForOfStatement: for (const p of this.players) { p.x += p.direction * 70 * dt;

L146-147 FirstStatement: const a = this.players[0].score >= 10, b = this.players[1].score >= 10;

L148-148 IfStatement: if (a || b) this.winner = a && b ? "draw" : a ? 1 : 2;

L149-155 IfStatement: if (this.time >= 240 && !this.winner) this.winner =

src/future-games/flappy-jump/config.ts

DEFERRED / unregistered | 17 lines | SHA-256 c004cd46538e7436...

Eager lightweight metadata for Flappy Jump: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/contracts/GameModule, ./assets/preview.svg?raw

Top-level block L3-16

```
export default {
  id: "flappy-jump",
  name: "Flappy Jump",
  description: "Time the jump. Sink the shot.",
  instructions:
    "Your player runs back and forth. Tap to jump: the ball releases at the apex. The floor mark glows when your
timing lines up a basket. Only downward passes through the hoop score. First to 10, four-minute cap uses baskets.
Orange: Space/A. Blue: Enter/L.",
  color: "#f3bdab",
  accent: "#d69d89",
  preview,
  order: 21,
  tag: "JUMP SHOT",
  supportedModes: ["local-two-player", "single-player"],
  tournament: true,
} satisfies GameMetadata;
```

src/future-games/flappy-jump/index.ts

DEFERRED / unregistered | 3 lines | SHA-256 ba849306a232a4f1...

Side-effect-free lazy factory for Flappy Jump; implementation initializes only after host selection.

Imports: ./FlappyGame

Top-level block L2-2

```
export default () => new FlappyGame();
```

examples/tap-duel

examples/tap-duel/config.ts

EXAMPLE / not shipped | 16 lines | SHA-256 ea12c5c57d426f67...

Eager lightweight metadata for Tap Duel example: stable ID, card/help text, colors, preview, supported modes, order, variants and tournament eligibility. It must not import the game implementation.

Imports: ../../shared/interfaces/MiniGame, ./assets/preview.svg?raw

Top-level block L3-15

```
export default {
  id: "tap-duel",
  name: "Tap Duel",
  description: "First to ten taps wins.",
  instructions:
    "Orange taps the bottom half. Blue taps the top half. First to ten wins.",
  color: "#d5c2e9",
  accent: "#eadff6",
  preview,
  order: 4,
  tag: "QUICK FINGERS",
  supportedModes: ["local-two-player"],
} satisfies GameMetadata;
```

examples/tap-duel/index.ts

EXAMPLE / not shipped | 53 lines | SHA-256 39a8fc9995d71788...

Complete small TapDuel class/factory demonstrating independent state, host inputs, scoring and disposal.

Imports: phaser, ../../shared/interfaces/MiniGame, ../../core/ThemeManager, ../../shared/components/drawing

State, resources and initial values

Line	Field / declaration
6	private score: [number, number] = [0, 0];
7	private paused = false;
8	private finished = false;
9	private graphics!: Phaser.GameObjects.Graphics;
10	private labels: Phaser.GameObjects.Text[] = [];

TapDuel [L5-51]

Component state and methods are grouped under this class. Its fields and individual blocks are indexed below.

TapDuel.initialize [L11-39]

Create owned state, graphics/DOM and input bindings from the host GameContext; this is the start of an active module instance.

Signature: initialize(ctx: GameContext)

Direct writes: this.graphics, this.labels, this.finished

Calls / delegates: ctx.setHint, ctx.setScore, ctx.scene.add.graphics, rounded, label, label(ctx.scene, 210, 250, "TAP!", 34).setRotation, ctx.input.tap, ctx.audio.play, ctx.finish

Decision branches: L22: this.paused || this.finished || x < FIELD.left || x > FIELD.right || y < FIELD.top || y > FIELD.bottom; L34: this.score[player - 1] === 10

TapDuel.update [L40-40]

Advance foreground state for this component, coordinate delegated helpers and/or refresh presentation according to the source steps below.

Signature: update(_dt: number)

```
update(_dt: number) {}
```

TapDuel.pause [L41-43]

Freeze component activity and clear/cancel transient input or pending work as implemented below.

Signature: pause()

Direct writes: this.paused

TapDuel.resume [L44-46]

Re-enable the component with a fresh/clean timing or input state.

Signature: resume()

Direct writes: this.paused

TapDuel.dispose [L47-50]

Release owned resources, listeners, DOM/graphics or worker work.

Signature: dispose()

Calls / delegates: this.graphics.destroy, this.labels.forEach, text.destroy

Top-level block L52-52

```
export default () => new TapDuel();
```

scripts

scripts/bundle-report.mjs

ACTIVE SOURCE / tool or application | 53 lines | SHA-256 45045c8e76659c9e...

Reads build manifest/chunk graphs, verifies lazy registration and application-shell payload budgets, and writes docs/bundle-report.json.

Imports: node:fs/promises, node:zlib

Module constants and data

L3-3 manifest = JSON.parse(await readFile("dist/.vite/manifest.json", "utf8"))

L4-4 initial = new Set()

L12-12 rows = []

L23-25 gameChunks = Object.keys(manifest).filter((k) => /src\/(games|arcade)\/.+\/index\.ts$/.test(k),)

L28-28 registered = []

L38-38 app = rows.find((r) => r.module === "index.html")

L43-47 report = { initial: rows, lazyGames: gameChunks, totalInitialGzip: rows.reduce((s, r) => s + r.gzip, 0), }

visit [L5-9]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: function visit(key)

Calls / delegates: initial.has, initial.add, visit

Decision branches: L6: initial.has(key)

Top-level block L10-11

```
for (const [key, chunk] of Object.entries(manifest))
  if (chunk.isEntry) visit(key);
```

Top-level block L13-22

```
for (const key of initial) {
  const chunk = manifest[key],
    code = await readFile("dist/" + chunk.file);
  rows.push({
    module: key,
    file: chunk.file,
    bytes: code.length,
    gzip: gzipSync(code).length,
  });
}
```

Top-level block L26-27

```
if (gameChunks.some((k) => initial.has(k)))
  throw Error("A game implementation leaked into the initial payload");
```

Top-level block L29-35

```
for (const root of ["games", "arcade"])
  for (const folder of await readdir("src/" + root, { withFileTypes: true }))
    if (folder.isDirectory()) {
      const files = await readdir("src/" + root + "/" + folder.name);
      if (files.includes("config.ts") && files.includes("index.ts"))
        registered.push("src/" + root + "/" + folder.name + "/index.ts");
    }
```

Top-level block L36-37

```
if (registered.some((k) => !gameChunks.includes(k)))
  throw Error("A registered game is missing its lazy chunk");
```

Top-level block L39-42

```
if (!app || app.gzip > 24000)
  throw Error(
    "Application shell exceeded its 24KB gzip budget; inspect imports",
  );
```

Top-level block L48-51

```
await writeFile(
  "docs/bundle-report.json",
  JSON.stringify(report, null, 2) + "\n",
);
```

Top-level block L52-52

```
console.log(JSON.stringify(report, null, 2));
```

scripts/check-module.mjs

ACTIVE SOURCE / tool or application | 61 lines | SHA-256 316820584c20c7a8...

Copies the Tap Duel example into a disposable project copy and verifies folder-only discovery/build without changing the real collection.

Imports: node:fs/promises, node:os, node:path, vite, node:child_process

Module constants and data

L14-14 root = process.cwd()

L15-17 temp = await realpath(await mkdtemp(path.join(tmpdir(), "pocket-module-check-")),)

Top-level block L18-60

```
try {
  for (const p of ["src", "public", "index.html", "tsconfig.json"])
    await cp(path.join(root, p), path.join(temp, p), { recursive: true });
  await cp(
    path.join(root, "examples/tap-duel"),
    path.join(temp, "src/games/tap-duel"),
    { recursive: true },
  );
  await symlink(
    path.join(root, "node_modules"),
    path.join(temp, "node_modules"),
    "dir",
  );
  const check = spawnSync(
    process.execPath,
    [
      path.join(root, "node_modules/typescript/bin/tsc"),
      "--noEmit",
      "--project",
      path.join(temp, "tsconfig.json"),
    ],
    { encoding: "utf8" },
  );
  if (check.status !== 0) throw Error(check.stdout + check.stderr);
  await build({
    root: temp,
    configFile: false,
    logLevel: "error",
    build: { target: "es2022" },
  });
  const assets = await readdir(path.join(temp, "dist/assets"));
  let code = "";
  for (const file of assets.filter((x) => x.endsWith(".js")))
    code += await readFile(path.join(temp, "dist/assets", file), "utf8");
  for (const needle of ["Tap Duel", "tap-duel", "TAP YOUR HALF"])
    if (!code.includes(needle))
      throw Error(`Missing discovered module: ${needle}`);
  console.log(
    "PASS: copied only a game folder; metadata and lazy implementation compiled automatically. Existing source
unchanged.",
  );
} finally {
  await rm(temp, { recursive: true, force: true });
}
```

scripts/measure-expansion.mjs

ACTIVE SOURCE / tool or application | 78 lines | SHA-256 47580e550ed6eb65...

Browser/CDP performance sampling utility; requires a running server and records snapshot measurements in docs JSON reports.

Imports: @playwright/test, node:fs/promises

Module constants and data

L5-5 url = process.env.POCKET_QA_URL ?? "http://127.0.0.1:5173"

L6-6 browser = await chromium.launch({ channel: "chrome", headless: true })

L7-7 results = []

Top-level block L8-77

try { const page = await browser.newPage({ viewport: { width: 412, height: 915 }, isMobile: true, hasTouch: true, deviceScaleFactor: 1, }); const errors = []; page.on("pageerror", (e) => errors.push(e.message)); await page.goto(`${url}/?diagnostics&aiSeed=77`); for (const name of ["Gravity Runner", "Happy Birds", "Spike Attacks",]) { await page .getByRole("button", { name: `Play ${name}`, exact: true }) .click(); await page.getByRole("radio", { name: /1 Player/ }).click(); await page.getByRole("radio", { name: "Impossible", exact: true }).click(); await page.getByRole("button", { name: /Let's play/ }).click(); await page.waitForFunction(() => window.__POCKET__?.scene.ready); await page.wai ... [continued in source reference]

Delegates: browser.newPage, page.on, errors.push, page.goto, page .getByRole("button", { name: `Play ${name}`, exact: true }) .click, page .getByRole, page.getByRole("radio", { name: /1 Player/ }).click, page.getByRole, page.getByRole("radio", { name: "Impossible", exact: true }).click, page.getByRole("button", { name: /Let's play/ }).click, page.waitForFunction, page.waitForTimeout, samples.push, page.evaluate, page.keyboard.press, page

.getByRole("button", { name: "Back to all games", exact: true }) .click, Error, results.push, errors.join, new Date().toISOString, Date, writeFile, JSON.stringify, console.log, results.map, samples.map, Math.round, samples.at, browser.close

scripts/measure-performance.mjs

ACTIVE SOURCE / tool or application | 66 lines | SHA-256 ab2f90fb48714183...

Browser/CDP performance sampling utility; requires a running server and records snapshot measurements in docs JSON reports.

Imports: playwright, node:fs/promises

Module constants and data

L3-3 browser = await chromium.launch({ channel: "chrome", headless: true })

L4-9 page = await browser.newPage({ viewport: { width: 412, height: 915 }, isMobile: true, hasTouch: true, deviceScaleFactor: 1, })

L18-18 results = []

tap [L19-25]

Small local helper. The exact implementation below defines its transformation/return behavior.

Signature: async function tap(x, y)

Calls / delegates: page.locator("canvas").boundingBox, page.locator, page.touchscreen.tap

Top-level block L10-10

```
await page.goto("http://127.0.0.1:5173/?diagnostics");
```

Top-level block L11-13

```
await page
  .getByRole("button", { name: "1 Player Arcade", exact: true })
  .click();
```

Top-level block L14-16

```
await page
  .getByRole("button", { name: "Play Ultimate Golf", exact: true })
  .click();
```

Top-level block L17-17

```
await page.waitForTimeout(500);
```

Top-level block L26-42

```
for (let i = 0; i < 4; i++) {
  await tap(210, 204 + i * 94);
  await page.waitForTimeout(1500);
  const samples = [];
  for (let n = 0; n < 6; n++) {
    await page.waitForTimeout(500);
    samples.push(
      await page.evaluate(() => ({
        diagnostics: window.__POCKET__.diagnostics,
        children: window.__POCKET__.scene.children,
        heap: performance.memory?.usedJSHeapSize,
      })),
    );
  }
  results.push({ course: i + 1, samples });
  await tap(81, 659);
}
```

Top-level block L43-55

```
await writeFile(
  "docs/performance-report.json",
  JSON.stringify(
    {
      environment:
        "Headless desktop Chrome, mobile viewport 412x915, DPR 1, development build; not an Android hardware
benchmark",
      measuredAt: new Date().toISOString(),
      results,
    },
    null,
    2,
  ) + "\n",
);
```

Top-level block L56-64

```
console.log(
  results.map((r) => ({
    course: r.course,
    fps: r.samples.map((s) => s.diagnostics.fps),
    frameMs: r.samples.at(-1).diagnostics.frameMs,
    init: r.samples.at(-1).diagnostics.initializations,
    children: r.samples.at(-1).children,
  })),
);
```

Top-level block L65-65

```
await browser.close();
```

scripts/offline-plugin.ts

ACTIVE SOURCE / tool or application | 52 lines | SHA-256 c012a84e50db4ab8...

Vite build plugin emitting sw.js with content version, embedded shell/full file lists, shell installation, cache-all messaging and cache-first fetch.

Imports: vite, node:crypto

offlinePlugin [L4-51]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: export function offlinePlugin(): Plugin

Calls / delegates: Set, initial.has, initial.add, chunk.imports.forEach, css?.forEach, visit, /\/(nunito-sans|fredoka)-.*\.woff2$/.test, createHash("sha256") .update(JSON.stringify(Object.keys(bundle))) .digest("hex") .slice, createHash("sha256") .update(JSON.stringify(Object.keys(bundle))) .digest, createHash("sha256") .update, createHash, this.emitFile

Return at L5: { name: "pocket-offline", apply: "build", generateBundle(_options, bundle) { const files = ["./", "./index.html", "./icon.svg", ...Object.keys(bundle).map((f) => "./" + f),], initial = new Set<string>(["./", "./index.html", "./icon.svg"]); const visit = (name: string) => { if (initial.has("./" + name)) return; initial.add("./" + name); const chun ... [continued in source reference]

Decision branches: L17: initial.has("./" + name); L20: chunk?.type === "chunk"; L27: item.type === "chunk" && item.isEntry; L28: /\/(nunito-sans|fredoka)-.*\.woff2$/.test(item.fileName)

Execution order (direct statements; nested callbacks stay within their owner):

L5-50 ReturnStatement: return { name: "pocket-offline",

offlinePlugin.generateBundle [L8-49]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: generateBundle(_options, bundle)

Calls / delegates: Set, initial.has, initial.add, chunk.imports.forEach, css?.forEach, visit, /\/(nunito-sans|fredoka)-.*\.woff2$/.test, createHash("sha256") .update(JSON.stringify(Object.keys(bundle))) .digest("hex") .slice, createHash("sha256") .update(JSON.stringify(Object.keys(bundle))) .digest, createHash("sha256") .update, createHash, this.emitFile

Decision branches: L17: initial.has("./" + name); L20: chunk?.type === "chunk"; L27: item.type === "chunk" && item.isEntry; L28: /\/(nunito-sans|fredoka)-.*\.woff2$/.test(item.fileName)

Execution order (direct statements; nested callbacks stay within their owner):

L9-15 FirstStatement: const files = ["./",

L16-25 FirstStatement: const visit = (name: string) => { if (initial.has("./" + name)) return;

L26-30 ForOfStatement: for (const item of Object.values(bundle)) { if (item.type === "chunk" && item.isEntry) visit(item.fileName);

L31-34 FirstStatement: const version = createHash("sha256") .update(JSON.stringify(Object.keys(bundle)))

L35-48 ExpressionStatement: this.emitFile({ type: "asset",

offlinePlugin.generateBundle.visit [L16-25]

Local implementation helper. The direct calls, writes, return expressions and branch conditions below identify its work without hiding delegated behavior.

Signature: visit = (name: string) =>

Calls / delegates: initial.has, initial.add, chunk.imports.forEach, css?.forEach

Decision branches: L17: initial.has("./" + name); L20: chunk?.type === "chunk"

scripts/verify-math-bank.py

ACTIVE SOURCE / tool or application | 57 lines | SHA-256 334510d339caa813...

SymPy generator/verifier that intentionally overwrites the 100-question university.json bank and math-validation.json report. It is not a read-only checker.

Lines 1-3: """Generate 100 compact questions and independently evaluate each stated operation with SymPy. Run in a development venv containing SymPy. No Python runtime ships with the game. """

Lines 7-7: x=s.symbols('x',real=True)

Lines 7-7: n=s.symbols('n',integer=True,positive=True)

Lines 8-8: rows=[]

Lines 8-8: evidence=[]

Lines 9-18: def add(family,k,topic,question,expected,computed,explanation,seconds=12,difficulty='medium',options=None): expected=s.sympify(expected); computed=s.simplify(computed) assert s.simplify(expected-computed)==0,(family,k,expected,computed) opts=options or [expected,expected+1,expected-1] opts=[s.sympify(v) for v in opts] assert all(s.simplify(opts[i]-opts[j])!=0 for i in range(3) for j in range(i)) correct=len(rows)%3;opts[0],opts[correct]=opts[correct],opts[0] uid=f'{family}-{k}' rows.append(dict(... [continued in source reference]

Lines 19-49: for k in range(2,6): add('log-chain',k,'Derivatives',rf'\left.\frac{{d}}{{dx}}\ln(1+{k}x)\right|_{{x=0}}',k,s.diff(s.log(1+k*x),x).subs(x,0),rf'\frac{{{k}}}{{1+{k}x}}\bigg|_{{0}}={k}')
add('power-at-one',k,'Derivatives',rf'\left.\frac{{d}}{{dx}}x^{{{k}}}\right|_{{x=1}}',k,s.diff(x**k,x).subs(x,1),rf'{k}x^{{{k-1}}}\big|_1={k}',8,'easy') add('exp-curvature',k,'Derivatives',rf'\left.\frac{{d^2}}{{dx^2}}e^{{{k}x}}\right|_{{x=0}}',k*k,s.diff(s.exp(k*x),x,2).subs(x,0),rf'{k}^2e^{{{k}x}}\big|_0={k*k}') ... [continued in source reference]

Lines 50-50: assert len(rows)==100 and len({r['id'] for r in rows})==100 and len({r['questionLatex'] for r in rows})==100

Lines 51-51: root=Path(__file__).resolve().parents[1]

Lines 52-52: out=root/'src/games/math-quiz/questions/university.json'

Lines 53-53: out.write_text(json.dumps(rows,indent=2)+'\n')

Lines 54-54: report=dict(count=len(rows),engine='SymPy '+s.__version__,method='Symbolic derivatives, limits, integrals, matrix determinants, exact identities and combinatorial formulas; all distractors checked pairwise.',independentHumanReview=False,mentalTime='Editorial estimates; not measured with students.',sha256=hashlib.sha256(out.read_bytes()).hexdigest(),results=evidence)

Lines 55-55: (root/'docs/math-validation.json').write_text(json.dumps(report,indent=2)+'\n')

Lines 56-56: print('Verified 100 answer keys, 300 pairwise option distinctions, and 100 unique IDs/statements.')

android/app/src/androidTest/java/com/getcapacitor/myapp

android/app/src/androidTest/java/com/getcapacitor/myapp/ExampleInstrumentedTest.java

ACTIVE SOURCE / tool or application | 27 lines | SHA-256 ff50b4c110a74343...

Native Android wrapper or template test. Complete short native source is reproduced below.

```
package com.getcapacitor.myapp;

import static org.junit.Assert.*;

import android.content.Context;
import androidx.test.ext.junit.runners.AndroidJUnit4;
import androidx.test.platform.app.InstrumentationRegistry;
import org.junit.Test;
import org.junit.runner.RunWith;

/**
* Instrumented test, which will execute on an Android device.
*
* @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
*/
```

```
@RunWith(AndroidJUnit4.class)
public class ExampleInstrumentedTest {

    @Test
    public void useAppContext() throws Exception {
        // Context of the app under test.
        Context appContext = InstrumentationRegistry.getInstrumentation().getTargetContext();

        assertEquals("com.getcapacitor.app", appContext.getPackageName());
    }
}
```

android/app/src/main/java/com/pocketrivals/games

android/app/src/main/java/com/pocketrivals/games/MainActivity.java

ACTIVE SOURCE / tool or application | 6 lines | SHA-256 a6ed806695cfd01e...

Native Android wrapper or template test. Complete short native source is reproduced below.

```
package com.pocketrivals.games;

import com.getcapacitor.BridgeActivity;

public class MainActivity extends BridgeActivity {}
```

android/app/src/test/java/com/getcapacitor/myapp

android/app/src/test/java/com/getcapacitor/myapp/ExampleUnitTest.java

ACTIVE SOURCE / tool or application | 19 lines | SHA-256 d4045ae8fac1f080...

Native Android wrapper or template test. Complete short native source is reproduced below.

```
package com.getcapacitor.myapp;

import static org.junit.Assert.*;

import org.junit.Test;

/**
* Example local unit test, which will execute on the development machine (host).
*
* @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
*/
public class ExampleUnitTest {

    @Test
    public void addition_isCorrect() throws Exception {
        assertEquals(4, 2 + 2);
    }
}
```

▪

capacitor.config.ts

ACTIVE SOURCE / tool or application | 19 lines | SHA-256 24009bc8cc7830d6...

Build/test/native tool configuration. Complete top-level settings are reproduced below.

Imports: @capacitor/cli

Module constants and data

L2-17 config = { appId: "com.pocketrivals.games", appName: "Pocket Rivals", webDir: "dist", backgroundColor: "#f8f5eb", android: { backgroundColor: "#f8f5eb", allowMixedContent: false }, plugins: { SplashScreen: { launchAutoHide: true, launchShowDuration: 500, backgroundColor: "#f8f5eb", showSpinner: false, androidScaleType: "CENTER_INSIDE", }, }, }

Top-level block L18-18

```
export default config;
```

playwright.config.ts

ACTIVE SOURCE / tool or application | 27 lines | SHA-256 05835d431a6e89a8...

Build/test/native tool configuration. Complete top-level settings are reproduced below.

Imports: @playwright/test

Top-level block L2-26

```
export default defineConfig({
  testDir: "tests/browser",
  workers: 1,
  timeout: 30000,
  use: {
    baseURL: "http://127.0.0.1:5173",
    ...devices["Pixel 7"],
    browserName: "chromium",
    launchOptions: { channel: "chrome" },
    screenshot: "only-on-failure",
  },
  webServer: [
    {
      command: "npm run dev",
      url: "http://127.0.0.1:5173",
      reuseExistingServer: true,
    },
    {
      command: "npm run preview -- --port 4173",
      url: "http://127.0.0.1:4173",
      reuseExistingServer: true,
    },
  ],
  reporter: "list",
});
```

vite.config.ts

ACTIVE SOURCE / tool or application | 13 lines | SHA-256 850439b516f1137c...

Build/test/native tool configuration. Complete top-level settings are reproduced below.

Imports: vite, ./scripts/offline-plugin

Top-level block L3-12

```
export default defineConfig({
  plugins: [offlinePlugin()],
  base: "./",
  build: {
    target: "es2022",
    manifest: true,
    rollupOptions: { output: { manualChunks: { phaser: ["phaser"] } } },
  },
  server: { port: 5173, strictPort: true },
});
```

vitest.config.ts

ACTIVE SOURCE / tool or application | 5 lines | SHA-256 f33818d0f5c5d26d...

Build/test/native tool configuration. Complete top-level settings are reproduced below.

Imports: vitest/config

Top-level block L2-4

```
export default defineConfig({
  test: { include: ["tests/unit/**/*.test.ts"], testTimeout: 20000 },
});
```

14. Tests and fixtures: every file and declared check

Test titles are source declarations, not a statement that the test passed in this edition. See the validation ledger for executed checks. Parameterized templates expand to multiple runtime cases; their literal titles are preserved.

tests/browser

tests/browser/ai-refinement.spec.ts

109 source lines. SHA-256 7f27bcb2fc282a1b...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L10-79: seeded personalities, all AI lifecycles, and worker cleanup survive repeated navigation

L80-108: fixed seed repeats a strategy move and saved match retains its personality

Helper/type state L2-2: state = (p: Page) =>

Helper/type tap L3-9: async function tap(p: Page, x: number, y: number)

Helper/type constructor L21-24: constructor(url: string | URL, options?: WorkerOptions)

Helper/type terminate L25-28: terminate()

tests/browser/air-hockey-keyboard.spec.ts

82 source lines. SHA-256 91d637b19a9b3fa6...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L16-59: Air Hockey keyboard moves both players in 2P mode with WASD and Arrows

L61-81: Air Hockey solo accepts both WASD and Arrow keys for the human mallet

Helper/type mallets L3-4: mallets = (page: Page): Promise<{ x: number; y: number }[]> =>

Helper/type start L6-14: async function start(page: Page, solo = false)

tests/browser/antigravity-review.spec.ts

99 source lines. SHA-256 720d7d1cd5df679f...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L14-43: Perfect Timer excludes pauses, locks stops, advances once and cleans up

L44-67: Water Guess accepts decimal keypad and completes a five-round local match

L68-98: Water Guess rotates the real keypad and delays AI guesses

Helper/type state L2-3: state = (p: Page) =>

Helper/type launch L4-13: async function launch(p: Page, name: string, ai = false)

Helper/type tap L78-79: tap = async (x: number, y: number) =>

tests/browser/app.spec.ts

187 source lines. SHA-256 6d476a53f1b20e8d...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L31-53: four illustrated rivalry games, mobile scrolling and persistent sound controls

L54-90: full tic-tac-toe win, rematch, draw, persistence, and reset

L143-160: repeated switching releases every input listener and game object

L161-186: resize keeps the entire portrait field visible; backgrounding requires resume

Helper/type snapshot L2-3: snapshot = (page: Page) =>

Helper/type start L4-9: async function start(page: Page, name: string)

Helper/type tapGame L10-17: async function tapGame(page: Page, x: number, y: number)

Helper/type menu L18-24: async function menu(page: Page)

Helper/type point L98-104: point = (x: number, y: number, id: number) =>

tests/browser/birds-stars.spec.ts

63 source lines. SHA-256 ebd8a5eff16b6e00...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L15-62: star race shows real collectibles; a hit animates gray to color through the middle with no countdown

Helper/type game L2-3: game = (page: Page) =>

Helper/type launch L4-14: async function launch(page: Page)

tests/browser/clean-ui.spec.ts

86 source lines. SHA-256 b019a2b68d1afd00...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L10-39: Hammer local field faces each player while host navigation remains fixed

L40-85: new games enter tournaments automatically and a real new-game round crowns its winner

Helper/type state L2-2: state = (p: Page) =>

Helper/type tap L3-9: async function tap(p: Page, x: number, y: number)

tests/browser/expansion-phase2.spec.ts

68 source lines. SHA-256 a65c9d6ea0da77fd...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L12-30: Gravity Runner flips via touch, scores survival and pauses cleanly

L31-47: Happy Birds flaps through touch input and finishes a real star race

L48-67: Spike Attacks supports movement, real cover and simultaneous scene cleanup

Helper/type snap L2-2: snap = (p: Page) =>

Helper/type open L3-11: async function open(p: Page, name: string)

tests/browser/expansion.spec.ts

277 source lines. SHA-256 dc367f97da592ba2...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L20-54: Dots and Boxes completes a full local match, rematches and runs worker AI

L55-70: effects volume persists and no music control is presented

L72-111: Hammer Hit alternates rotated turns, delays AI, and reports three-strike averages

L113-150: Light Fingers cooldowns, legal steals, pause and a complete match

L152-181: Fruit Duel completes ten legal fruit slices and AI responds to real fruit

L183-247: Math Quiz renders formulas, validates all mobile widths and completes a real ten-answer match

L249-276: Lumberjack real controls, seeded trees, three rounds and a result

Helper/type state L2-2: state = (p: Page) =>

Helper/type tap L3-9: async function tap(p: Page, x: number, y: number)

Helper/type open L10-19: async function open(p: Page, name: string, ai = false)

tests/browser/fruit-sword-path.spec.ts

37 source lines. SHA-256 b695951b3561a296...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L2-36: sword traverses, recovers to original side, locks input and freezes on pause

Helper/type s L5-5: s = () =>

tests/browser/fruit-timer-effects.spec.ts

99 source lines. SHA-256 5890d2b96ea5c9b1...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L26-80: fruit decoys cause recoverable cooldown, real fruit splits, sound plays and pause freezes

L81-98: Perfect Timer schedules clock ticks and stop click, pause stops ticking

Helper/type state L2-3: state = (p: Page) =>

Helper/type open L4-10: async function open(p: Page, name: string)

Helper/type audioSpy L11-25: async function audioSpy(p: Page)

tests/browser/golf-temple-ai.spec.ts

51 source lines. SHA-256 8a02690222be7518...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

Helper/type opponent L19-22: opponent = () =>

L2-50: for (const difficulty of ["Medium", "Hard", "Impossible"]) test(`Lost Temple ${difficulty} AI completes the actual worker-driven course`, async ({ page, }) => { test.setTimeout(180000); const errors: string[] = []; page.on("pageerror", (e) => errors.push(e.message)); await page.goto("/?aiSeed=123"); await page .getByRole("button", { name: "Play Ultimate Golf", exact: true }) .click(); await page.getByRole("button", { name: /1 Player vs AI/ }).click(); await page.getByRole("button", { name: diffi ... [continued in source reference]

tests/browser/gravity-landscapes.spec.ts

147 source lines. SHA-256 6a441d810bb8e5d3...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L15-47: Happy Birds Space and arrows flap separate birds; pause blocks input

L82-107: Gravity map previews are distinct and fit a small phone

L108-146: both runners can land on the middle rail while both outside walls are missing

Helper/type state L2-3: state = (page: Page) =>

Helper/type launch L4-14: async function launch(page: Page, name: string, map?: string, ai?: string)

tests/browser/gravity-shared.spec.ts

84 source lines. SHA-256 416a3c6e35c38d67...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L2-55: shared gravity corridor accepts simultaneous touch flips, pauses and cleans up

L57-83: AI and a legal following player traverse the shared obstacle course

Helper/type state L14-15: state = () =>

Helper/type state L68-69: state = () =>

tests/browser/light-lumber-polish.spec.ts

43 source lines. SHA-256 10097ddd973e8abe...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L4-42: steal animation and compact Lumberjack fields remain playable

Helper/type state L2-3: state = (p: any) =>

Helper/type open L9-17: async function open(name: string)

tests/browser/map-previews.spec.ts

105 source lines. SHA-256 5f2c21bfe292cd8f...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L2-45: Spike map picker shows distinct previews without horizontal overflow on a small phone

L46-75: Golf shows four actual portrait course overviews and still launches Temple

L77-104: Happy Birds retains the shared arena with a clean rounded presentation

tests/browser/math-quiz-visual.spec.ts

62 source lines. SHA-256 b3d710fc1a3c0c19...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L3-61: Math Quiz visual capture across viewports and formulas

tests/browser/offline.spec.ts

107 source lines. SHA-256 965d68dce12cfb59...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L2-106: packaged build reloads and lazy-loads all seven games and AI workers without a network

Helper/type tap L92-96: tap = async (x: number, y: number) =>

tests/browser/phase2.spec.ts

229 source lines. SHA-256 3b08fb20546e6992...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L37-57: game setup preserves branding, variant selection and an interactive routing tutorial

L84-105: Classic AI takes only its own turn and survives rematch setup

L106-132: Ultimate worker plays legal routed moves and restores a saved match

L133-161: Ultimate local play rejects illegal boards, provides focus view, and completes a full match

L162-199: two-round tournament persists progress, scores a draw and crowns a champion

L200-228: AI tournament includes every eligible game and resumes an interrupted Ultimate round

Helper/type state L2-2: state = (p: Page) =>

Helper/type tap L3-9: async function tap(p: Page, x: number, y: number)

Helper/type launch L10-21: async function launch(p: Page, name: string, ai = false, variant?: string)

Helper/type ultimateTap L22-30: async function ultimateTap(p: Page, index: number)

tests/browser/phase3.spec.ts

258 source lines. SHA-256 be5581629b0ac9f2...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L52-81: Connect Four completes a gravity win, rematches, saves and runs worker AI

L82-125: Arcade has independent high scores, Snake controls, pause and game over

L126-167: Star Patrol touch firing, keyboard controls, responsive layout and pooled projectiles

L168-209: new games are lazy, Arcade is excluded from tournaments, and navigation releases scenes

L210-232: match countdown uses elapsed time and freezes while backgrounded

L234-257: Pong and Hockey restore foreground clocks after closing the page

Helper/type state L2-2: state = (p: Page) =>

Helper/type tap L3-9: async function tap(p: Page, x: number, y: number)

Helper/type launch L10-21: async function launch(p: Page, name: string, ai = false, variant?: string)

Helper/type leave L22-29: async function leave(p: Page)

Helper/type drag L30-51: async function drag(p: Page, x: number, y: number, toX: number, toY: number)

Helper/type drag.point L33-37: point = (x: number, y: number) =>

tests/browser/pong-keyboard.spec.ts

58 source lines. SHA-256 2900d6e053c30680...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L16-45: Pong keyboard moves both players, clamps at edges and clears on pause

L47-57: Pong solo accepts arrows and A/D for the human paddle

Helper/type paddles L3-4: paddles = (page: Page): Promise<{ x: number }[]> =>

Helper/type start L6-14: async function start(page: Page, solo = false)

tests/browser/precision-games.spec.ts

111 source lines. SHA-256 97c6b826c2350bb6...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L16-48: Water Guess supports 0.01% precision keypad input and completes a match

L50-85: Perfect Timer supports centisecond precision and completes a match

L87-110: Hammer Hit displays non-overlapping UI layout, dome marquee, and score popups

Helper/type state L3-3: state = (p: Page) =>

Helper/type open L5-14: async function open(p: Page, name: string, ai = false)

tests/browser/premium-motion.spec.ts

88 source lines. SHA-256 2d9f2d66eb292f73...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L10-26: deferred games stay off the menu; active games still launch

L27-44: Happy Birds has no timed ending and freezes recovery while paused

L45-87: Happy Birds shares a single arena, accepts both touch flaps and disposes on exit

Helper/type snap L2-2: snap = (p: Page) =>

Helper/type open L3-9: async function open(p: Page, name: string)

tests/browser/spike-shared.spec.ts

213 source lines. SHA-256 f212207b2bcd502d...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L15-67: shared arena has independent joysticks, smooth legal motion and cancellation on pause

L68-88: attacks really extend, AI takes cover, and the shared match finishes without fabricated points

L89-125: deadline excludes pause time; desktop mouse joystick and repeated exits stay clean

L127-151: solo arrow keys control Orange and both local keyboard schemes stay active

L152-184: browser gamepad input moves legally and held sticks cannot creep after pause

L186-212: three maps and Random launch, while solo hides the AI control

Helper/type state L2-2: state = (p: Page) =>

Helper/type launch L3-14: async function launch(p: Page, ai = false)

Helper/type point L26-30: point = (id: number, x: number, y: number) =>

tests/browser/star-patrol.spec.ts

124 source lines. SHA-256 df13417121617ef6...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L3-47: production Star Patrol launches with mobile and desktop controls

L49-123: Star Patrol earns real loot, advances levels and freezes gear on pause

Helper/type snapshot L63-63: snapshot = () =>

Helper/type point L68-72: point = (x: number) =>

tests/browser/ultimate-golf.spec.ts

266 source lines. SHA-256 58dfaf12ca1414b0...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: ../fixtures/golf-solutions, @playwright/test

L35-89: golf responsive touch, focus, pause and four courses

L90-123: golf mouse aiming, footer spacing and landscape cleanup

L124-183: golf local handoff, real AI shots and clean rematches

L184-265: a complete Temple run uses real touch shots, banks its score and persists its record

Helper/type state L3-3: state = (p: Page) =>

Helper/type center L4-7: async function center(p: Page)

Helper/type open L8-16: async function open(p: Page)

Helper/type gesture L17-34: async function gesture(p: Page, cancel = false)

Helper/type shot L197-222: async function shot(vx: number, vy: number)

tests/fixtures

tests/fixtures/golf-ai-audit.test.ts

34 source lines. SHA-256 b5b9aca75ed69230...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/mini-golf/ultimate/courses, ../../src/games/mini-golf/ultimate/GolfPhysics, ../../src/games/mini-golf/ultimate/ai/ShotPlanner

L5-33: for (let c = 0; c < 4; c++) it(`AI route audit ${c}`, () => { const w = new GolfPhysics(courses[c], 0, 123); let waypoint = 0; for (let turn = 0; turn < w.limit; turn++) { const p = planShot({ state: w.exportState(), waypoint, difficulty: "impossible", delay: 0.65, }); waypoint = p.waypoint; for (let i = 0; i < (0.65 + p.wait) * 240; i++) w.tick(1 / 240); if (!w.shootable) { for (let i = 0; i < 240 * 10; i++) { w.tick(1 / 240); if (w.shootable || w.status !== "playing") break; } } w.shoot(p.vx, ... [continued in source reference]

tests/fixtures/golf-audit.config.ts

5 source lines. SHA-256 607d320de1e02682...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest/config

L2-4: export default defineConfig({ test: { include: ["tests/fixtures/golf-ai-audit.test.ts"] }, });

tests/fixtures/golf-route-search.ts

138 source lines. SHA-256 968a2a4bfbf5dde9...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/mini-golf/ultimate/courses, ../../src/games/mini-golf/ultimate/GolfPhysics

Helper/type c L5-5: c = (i: number) =>

Helper/type clone L61-76: function clone(w: GolfPhysics)

L4-4 lane: (1920 - 196) / 6

L6-60 routes: [[[c(0), 220], [c(1) + 110, 220], [c(1) + 110, 670], [c(1) - 100, 670], [c(1) - 100, 950], [680, 950], [680, 220], [1235, 220], [1235, 950], [c(4), 950], [c(4), 220], [c(5) - 120, 220], [c(5) - 120, 942], [c(5), 942],], [[220, 990], [220, 220], [520, 220], [520, 970], [910, 970], [910, 220], [910, 150], [1340, 150], [1340, 930], [1760, 930], [1760, 220], [1560, 200],], [[1100, 990], [1810, 990], [1840, 700, 1265, 340], [290, 340], [290, 590], [700, 600, 1205, 600], [1400, 575],], [[520, ... [continued in source reference]

L77-137: for (let course = 0; course < 4; course++) it(`full course route ${course + 1}`, () => { let w = new GolfPhysics(courses[course], 0, () => 0.9); const shots: number[][] = []; for (const [tx, ty, gx = tx, gy = ty] of routes[course]) { for (let attempt = 0; attempt < 3; attempt++) { if (w.status === "won") break; const distance = Math.hypot(w.ball.x - tx, w.ball.y - ty); if (distance < 22) break; const angle = Math.atan2(ty - w.ball.y, tx - w.ball.x); let best: GolfPhysics | undefined, bestCost = ... [continued in source reference]

tests/fixtures/golf-solutions.ts

66 source lines. SHA-256 8dec92f0b0d17005...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

L2-65 golfSolutions: [[[0, 3.122849337825751e-14, -510], [0, 299.87970221664955, -8.494951350867446], [0, 11.16493409955141, 314.8020715410759], [0, -339.912382744990 74, 59.03026390439822], [0, 63.14509843361572, 566.491567937078], [0, 239.99849308558947, 0.8504802444897002], [0, 58.3925488586842, -309.5404823892768], [0, 359.97571982845443, -4.181044604651763], [0, 49.020819070272076, 582.9425008503666], [0, 271.64106563615695, 300.3932946323088], [0, -58.78747923929417, -46.57287069410212], [0, 209.9974041048571 ... [continued in source reference]

tests/future

tests/future/archery-duel.spec.ts

31 source lines. SHA-256 88234efdf04f918b...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L12-30: Archery Duel uses three input stages and AI scores real impacts

Helper/type snap L2-2: snap = (p: Page) =>

Helper/type open L3-11: async function open(p: Page, name: string)

tests/future/car-duel.spec.ts

48 source lines. SHA-256 29969c587c5c7ff7...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L2-47: Car Duel holds independent touch controls, cancels on pause, and removes own listeners

Helper/type state L13-13: state = () =>

Helper/type point L17-21: point = (id: number, x: number) =>

tests/future/chicken-jump.spec.ts

38 source lines. SHA-256 34afbb75e19267cc...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L12-37: Chicken Jump accepts independent controls, animates legal landings, pauses and exits

Helper/type snap L2-2: snap = (p: Page) =>

Helper/type open L3-11: async function open(p: Page, name: string)

tests/future/flappy-jump.spec.ts

28 source lines. SHA-256 17e36c30a7bc5929...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: @playwright/test

L12-27: Flappy Jump releases a physical ball and AI sinks a basket

Helper/type snap L2-2: snap = (p: Page) =>

Helper/type open L3-11: async function open(p: Page, name: string)

tests/unit

tests/unit/ai-refinement.test.ts

525 source lines. SHA-256 29c381b21ac0e8e2...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, node:fs, ../../src/shared/ai/MatchRandom, ../../src/games/tic-tac-toe/classic/ClassicAI, ../../src/games/tic-tac-toe/classic/ClassicRules, ../../src/games/light-fingers/AI, ../../src/games/light-fingers/Rules, ../../src/games/fruit-duel/AI, ../../src/games/fruit-duel/Rules, ../../src/games/hammer-hit/AI, ../../src/games/hammer-hit/Rules, ../../src/games/lumberjack/AI, ../../src/games/lumberjack/Rules, ../../src/games/pong/PongAI, ../../src/games/pong/PongPhysics, ../../src/games/air-hockey/AirHockeyAI, ../../src/games/air-hockey/AirHockeyPhysics, ../../src/games/connect-four/ConnectFourRules, ../../src/games/connect-four/ConnectFourAI, ../../src/games/dots-and-boxes/Rules, ../../src/games/dots-and-boxes/AI, ../../src/games/tic-tac-toe/ultimate/UltimateRules, ../../src/games/tic-tac-toe/ultimate/UltimateSearch, ../../src/games/mini-golf/ultimate/GolfPhysics, ../../src/games/mini-golf/ultimate/courses, ../../src/games/mini-golf/ultimate/ai/ShotPlanner

L44-55: 24 reproducible match identities and isolated random streams

L58-90: 100 Classic matches have legal varied openings and optimal Impossible outcomes

L91-240: 100 reflex and survival matches vary timing without illegal actions

L241-305: 100 physics scenarios preserve speed/bounds and stable seeded trajectories

L307-333: Classic Impossible never loses against any legal human line across every personality

L334-372: seeded board search replays, restores inputs, and varies equal-score openings

L373-401: 100 varied midgame strategy scenarios return legal actions within node budgets

L402-434: Golf seeded candidate simulations replay legal shots, without post-simulation errors

L436-451: completed physics matches never advance AI decisions or move its controls

L452-474: reaction and precision distributions separate the three difficulties

L476-505: 100 complete Dots matches preserve extra turns and score only owned boxes

L507-524: Golf chooses varied comparably good simulated shots rather than post-simulation noise

Helper/type walk L313-329: walk = (b: TicTacToeBoard, side: 1 | 2) =>

tests/unit/antigravity-review.test.ts

78 source lines. SHA-256 ba49833584bf6d2e...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/perfect-timer/TimerClock, ../../src/games/perfect-timer/PerfectTimerRules, ../../src/games/water-guess/WaterGuessRules, ../../src/games/perfect-timer/PerfectTimerAI, ../../src/games/water-guess/WaterGuessAI

L5-47: formal review regressions

L6-21: excludes repeated pause/background intervals from high-resolution timing

L22-34: resolves each timer round exactly once and rejects nonfinite time

L35-46: resolves each water round exactly once and rejects invalid guesses

L50-77: 100 seeds replay diverse targets and increasingly accurate AI without forced Impossible errors

tests/unit/archery-duel.test.ts

37 source lines. SHA-256 c7aa385d38a679a2...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/future-games/archery-duel/Rules, ../../src/future-games/archery-duel/AI

L4-20: debounces stages and scores only real circle hits once

L21-36: 100 Impossible matches finish through legal timed taps with seed-varying targets

tests/unit/audio.test.ts

68 source lines. SHA-256 7044aa1d5d45562e...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/core/AudioManager, ../../src/core/StorageManager

L5-14: retains old scores and validates volume on restore

L15-67: caps procedural voices and clears them on background/disposal

Helper/type param L16-20: param = () =>

Helper/type FakeAudio L22-50: class FakeAudio

Helper/type FakeAudio.createGain L26-28: createGain()

Helper/type FakeAudio.createOscillator L29-40: createOscillator()

Helper/type FakeAudio.resume L41-43: resume()

Helper/type FakeAudio.suspend L44-46: suspend()

Helper/type FakeAudio.close L47-49: close()

tests/unit/car-duel.test.ts

51 source lines. SHA-256 2e27bf1014bef67d...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/future-games/car-duel/Rules, ../../src/future-games/car-duel/AI, ../../src/future-games/car-duel/Physics

L9-18: idle cars settle stably and retain legal wheel contact

L19-35: head contact loses the round and simultaneous head impacts replay

L36-50: 100 seeded duel simulations remain finite with bounded controls and resolve matches

tests/unit/chicken-jump.test.ts

43 source lines. SHA-256 96c272e9d289aed9...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/future-games/chicken-jump/Rules, ../../src/future-games/chicken-jump/AI

L4-15: geometric landings stack ten; misses cannot score; duplicate stops rejected

L16-31: 100 seeded Impossible opponents legally complete and freeze after the result

L32-42: Medium retries vary their alignment rather than repeating an impossible offset forever

tests/unit/dots-and-boxes.test.ts

111 source lines. SHA-256 aa772a6cbe2f7a8a...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/dots-and-boxes/Rules, ../../src/games/dots-and-boxes/AI

L9-110: Dots and Boxes

L10-16: maps every edge to a touch target and rejects blank space

L17-27: rejects illegal and duplicate edges, claims two adjacent boxes and retains the turn

L28-37: finishes exactly once with all 16 boxes owned and restores legal histories

L38-48: all difficulties capture an available box without modifying the input

L49-55: Impossible finds an exact terminal endgame and returns a legal move

L56-73: scores agree with owned box geometry through 100 reproducible full games

L74-109: supports 4x4, 5x5, and 6x6 board sizes with exact geometry and AI compatibility

tests/unit/engine.test.ts

271 source lines. SHA-256 c67f1aa6ca318c0b...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/pong/PongPhysics, ../../src/games/air-hockey/AirHockeyPhysics, ../../src/games/tic-tac-toe/classic/ClassicRules, ../../src/shared/physics/FixedStep, ../../src/core/StorageManager, ../../src/core/ScoreManager

L12-63: Pong

L13-20: sweeps a fast ball against a paddle

L21-28: steers off-center contacts and transfers paddle motion

L29-45: scores once, serves automatically and finishes at five

L46-62: gives the same 3s simulation at %i FPS

L64-171: Air hockey

L65-76: releases a puck pinned between a mallet and a rail

L78-88: reflects a high-speed puck from a stationary mallet

L89-99: transfers mallet velocity and caps resulting puck speed

L100-111: requires the entire puck to enter the goal

L112-122: bounces off closed goal-line sections

L123-134: constrains each mallet to its own half

L135-148: is deterministic at %i rendering FPS

L149-170: survives 60k steps of fast adversarial input without nonfinite state

L172-216: Tic-tac-toe

L173-180: detects a win on %s

L181-187: rejects occupied cells without consuming a turn

L188-192: detects a draw

L193-215: checks every possible reachable game tree

L217-270: Persistence and lifecycle

L218-238: stores wins once per round, includes draws, and survives reload

L239-251: handles corrupted or blocked localStorage

L252-260: sanitizes invalid values

L261-269: bounds catch-up after interruptions and discards paused time

Helper/type simulate L49-55: simulate = (hz: number) =>

Helper/type sim L136-142: sim = (hz: number) =>

Helper/type walk L195-212: walk = (w: TicTacToeBoard) =>

tests/unit/expansion-framerate.test.ts

67 source lines. SHA-256 a3ecfd12511f01ce...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/shared/physics/FixedStep, ../../src/future-games/chicken-jump/Rules, ../../src/future-games/chicken-jump/AI, ../../src/future-games/archery-duel/Rules, ../../src/future-games/archery-duel/AI, ../../src/games/gravity-runner/Rules, ../../src/games/gravity-runner/AI, ../../src/games/happy-birds/Rules, ../../src/games/happy-birds/AI, ../../src/games/spike-attacks/Rules, ../../src/games/spike-attacks/AI, ../../src/future-games/flappy-jump/Rules, ../../src/future-games/flappy-jump/AI, ../../src/future-games/car-duel/Rules, ../../src/future-games/car-duel/AI

L17-66: all seven seeded simulations produce identical outcomes at 30, 60 and 120 render FPS

tests/unit/flappy-jump.test.ts

58 source lines. SHA-256 bbc3dcacb0727e65...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/future-games/flappy-jump/Rules, ../../src/future-games/flappy-jump/AI

L4-29: only legal descending hoop crossings score once

L30-45: rim collision reflects only approaching motion and duplicate jumps fail

L46-57: 100 Impossible seeds complete ten baskets through simulated legal shots

tests/unit/fruit-duel.test.ts

139 source lines. SHA-256 2d38cda5bec2649a...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/fruit-duel/Rules, ../../src/games/fruit-duel/AI

L4-18: empty swings and all three decoys cause cooldown, never points

L19-39: all seven real fruits score once, judgment survives object changes, ties replay

L40-58: real fruit is guaranteed within three reveals and a legal ten-slice match ends

L59-83: 100 seeds per difficulty keep AI actions legal and Impossible avoids decoys

L85-115: sword return is the input cooldown, returns to its original side, and mirrors fairly

L117-138: each mirrored sweep reaches the fruit and the final sword finishes returning

Helper/type random L63-66: random = () =>

tests/unit/golf-refinement.test.ts

171 source lines. SHA-256 99b59a213f6c95bc...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/mini-golf/ultimate/courses, ../../src/games/mini-golf/ultimate/GolfPhysics, ../../src/games/mini-golf/ultimate/GolfInput, ../../src/games/mini-golf/ultimate/modes/LocalVersusMode, ../../src/games/mini-golf/ultimate/modes/CompetitionRules, ../../src/games/mini-golf/ultimate/ai/ShotPlanner, ../fixtures/golf-solutions

L15-170: Golf refinement

L16-51: uses a uniform invertible camera across phone and landscape sizes

L52-68: every portal exit has ball clearance on its destination platform

L69-83: repeats UFO visits and provides a warning before a visit

L84-99: seeded obstacle sequences and simulation restoration are reproducible

L100-120: hot-seat pauses menus, resets identical conditions, and scores only one attempt

L121-132: compares completion, strokes, elapsed time and simultaneous finishes explicitly

L133-152: all four real completion routes remain valid at different fixed frame rates

L153-169: each AI difficulty produces bounded legal shots from real course state

tests/unit/golf-routes.test.ts

94 source lines. SHA-256 92ceceaf2a6ec5c6...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/mini-golf/ultimate/courses, ../../src/games/mini-golf/ultimate/GolfPhysics, ../fixtures/golf-solutions, ../../src/games/mini-golf/ultimate/ai/routes, ../../src/games/mini-golf/ultimate/modes/AIChallengeMode

L27-39: AI selects multiple different routes across all 3 difficulties and seeds

L41-76: AIChallengeMode allows human to finish when AI finishes first and awards win on strokes

L78-92: AIChallengeMode breaks ties on time only when strokes are equal

tests/unit/golf-temple-ai.test.ts

102 source lines. SHA-256 18903e64de8de614...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/mini-golf/ultimate/GolfPhysics, ../../src/games/mini-golf/ultimate/courses/temple, ../../src/games/mini-golf/ultimate/ai/ShotPlanner, ../../src/games/mini-golf/ultimate/ai/routes, ../../src/shared/ai/MatchRandom, ../../src/games/mini-golf/ultimate/modes/CompetitionRules, ../../src/games/mini-golf/ultimate/GolfCollision

L12-28: Temple AI resting waypoints clear permanent colliders, quicksand and the moving-boulder sweep

L86-101: reached Temple waypoints are recorded before forecasting movement during reaction time

tests/unit/gravity-runner.test.ts

198 source lines. SHA-256 2c5f78de4b70d413...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/gravity-runner/Rules, ../../src/games/gravity-runner/AI, ../../src/games/gravity-runner/Maps, ../../src/games/gravity-runner/RunnerPhysics

L17-34: simultaneous falls replay; legal survival points finish at five

L35-47: 100 seeded opponents at every difficulty beat idle players using legal flips

L48-73: 100 seeds per difficulty traverse two minutes of increasingly fast shared hazards

L74-86: four maps select reproducibly, random seeds cover all themes, and explicit maps work

L87-105: double wall gaps always have a continuous rail and ample entry/exit clearance

L106-128: middle platform lands on both faces without jitter; front penetration remains lethal

L129-145: spikes, ledges and beams stay lethal at their visible bounds

L146-168: no time limit or forced replay; speed rises smoothly and geometry stays bounded

L169-188: reaction/search budgets differ by difficulty while every accepted action remains legal

L190-197: a returning wall cannot teleport a fallen runner back into the corridor

Helper/type runner L10-16: runner = (y = 160, gravity: -1 | 1 = 1): Runner =>

tests/unit/gravity-variation.test.ts

134 source lines. SHA-256 447431a4bbf329cc...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/gravity-runner/CourseStream, ../../src/games/gravity-runner/Rules, ../../src/games/gravity-runner/AI, ../../src/games/gravity-runner/ObstacleGeometry, ../../src/games/gravity-runner/RunnerPhysics

L18-52: each map generates only its own signature traps and nonperiodic seeded combinations

L53-65: speed has a calm start, smooth staged increases and a safe endless ceiling

L66-86: later sections vary spacing and raise hazard density without shortening legal transit below 245

L87-116: jets pulse, phase gates really open, and visual polygons match legal collisions

L117-133: 100 Impossible routes per map establish legal two-minute paths through all trap families

tests/unit/hammer-hit.test.ts

75 source lines. SHA-256 15b4bbe55c9f0ced...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/hammer-hit/Rules, ../../src/games/hammer-hit/AI

L4-22: Hammer compares arithmetic means using exact integer totals

L23-37: Hammer allows exactly three legal strikes and rejects duplicate taps

L38-51: Hammer AI completes every difficulty at different frame rates using real timing

L53-74: Impossible uses approximately 70/25/5 bands through legal needle timing

tests/unit/hammer-presentation.test.ts

18 source lines. SHA-256 2be26297c57eb280...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/hammer-hit/StrikePhysics, ../../src/shared/physics/FixedStep

L4-11: puck has gravity, only a perfect strike contacts the bell, and all flights settle

L12-17: ring impulse decays and fixed presentation agrees at 30/60/120 FPS

tests/unit/happy-birds.test.ts

222 source lines. SHA-256 6cbb0f9346bf814f...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/happy-birds/Rules, ../../src/games/happy-birds/AI, ../../src/games/happy-birds/ArenaPhysics, ../../src/games/happy-birds/Recovery, ../../src/shared/ai/MatchRandom, ../../src/games/happy-birds/Stars

L33-50: flaps obey launch and flap cooldown; safe walls bounce without awarding stars

L51-77: triangle collisions start an independent three-second recovery without lost stars

L78-94: the fifteenth real pickup ends the race; contacts cannot award duplicate points

L95-108: one shared star is awarded to its nearest collector; exact same-step finishes can draw

L109-119: recovering birds never collect stars and do not interrupt their opponent

L120-139: stars spawn reproducibly away from spikes, and collectible collision is swept

L140-154: recovery moves smoothly to the middle and restores all body colors without a countdown

L155-167: all difficulties finish genuine 15-star races across 100 seeds using legal flaps

L168-176: a race does not expire before either participant collects fifteen

L177-189: wall warning changes wait until every active bird clears that wall

L191-221: one shared star ranges across the arena without fixed slots or nearby repeat positions

Helper/type collect L17-32: function collect(m: BirdsMatch, player: 0 | 1)

tests/unit/light-fingers.test.ts

76 source lines. SHA-256 7f476019c96a6676...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/light-fingers/Rules, ../../src/games/light-fingers/AI

L4-14: Light Fingers applies cooldown without changing scores for early taps and rejects multi-tap exploits

L15-26: Light Fingers cannot accept a claim until the shared signal is presented

L27-53: simultaneous grabs cooldown both players and legal steals end at five

L54-75: Light AI never anticipates the signal and reacts with a legal minimum delay

tests/unit/lumberjack.test.ts

6 source lines. SHA-256 d34954edbcc51f8c...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/lumberjack/Rules, ../../src/games/lumberjack/AI

L2-2: Lumberjack seeded streams always leave at least one legal chopping side

L3-3: Lumberjack counts one valid chop, rejects repeats and detects branches

L4-4: Lumberjack runs exactly three base rounds and adds sudden death on a tie

L5-5: Lumberjack AI uses legal chop timing, finishes a three-round match and respects pause-by-update

tests/unit/math-quiz.test.ts

198 source lines. SHA-256 1586885682a47007...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, katex, node:crypto, node:fs, ../../src/games/math-quiz/questions/Bank, ../../src/games/math-quiz/questions/ProceduralUniversity, ../../src/games/math-quiz/questions/Basic, ../../src/games/math-quiz/questions/Question, ../../src/games/math-quiz/questions/History, ../../src/games/math-quiz/Rules, ../../src/games/math-quiz/AI, ../../docs/math-validation.json

L14-30: 100 university questions have verified distinct keys, complete metadata and valid strict LaTeX

L32-63: procedural university generator produces diverse, valid, strict LaTeX questions across all required topics

L65-82: total combined university question pool has over 250 distinct instances and deck interleaves topics

L84-105: repetition tracker correctly bounds history and fallback preserves gameplay when exhausted

L107-157: 10000 basic questions span all 7 categories with exact mathematical answers and 3 distinct options

L159-181: quiz resolves incorrect responses, simultaneous answers and a complete match once

L183-197: quiz AI follows the validated key only after its calibrated delay

tests/unit/perfect-timer.test.ts

170 source lines. SHA-256 da1d8b42cbd60ae3...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/perfect-timer/PerfectTimerRules, ../../src/games/perfect-timer/PerfectTimerAI

L10-113: Perfect Timer Rules & Precision

L11-18: formats centiseconds to 0.01s resolution accurately

L20-29: converts millisecond elapsed time to integer centiseconds with consistent rounding

L31-38: prevents repeated stops from overwriting initial stop time

L40-56: calculates centisecond absolute error and determines winner accurately

L58-71: declares round tied when absolute errors in centiseconds are identical

L73-112: progresses across 5 rounds and handles sudden death if tied

L115-169: Perfect Timer AI

L116-168: schedules stops reflecting difficulty error distributions

tests/unit/phase2.test.ts

342 source lines. SHA-256 21b1baefff25d68f...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/tic-tac-toe/classic/ClassicRules, ../../src/games/tic-tac-toe/classic/ClassicAI, ../../src/games/tic-tac-toe/ultimate/UltimateRules, ../../src/games/tic-tac-toe/ultimate/UltimateSearch, ../../src/games/air-hockey/AirHockeyPhysics, ../../src/games/air-hockey/rink, ../../src/games/pong/PongPhysics, ../../src/games/pong/PongAI, ../../src/games/air-hockey/AirHockeyAI, ../../src/core/TournamentManager, ../../src/core/StorageManager, ../../src/shared/contracts/GameModule, ../../src/games/tic-tac-toe/shared/lines

L19-101: hockey contact regression

L20-36: resolves corner %s, %s without a wall trap

L37-45: never nudges a playable stationary puck

L46-56: repairs nonfinite unplayable state only after the watchdog interval

L57-84: handles prolonged repeated high speed contacts and initial complete overlap

L85-100: ends a match exactly once after five goals

L102-128: physics AI plays through ordinary constrained targets

L103-106: reflects predicted trajectories over multiple rails

L107-127: %s never exceeds human movement limits

L129-162: Classic AI

L130-153: Impossible as player %i cannot lose against any legal strategy

L154-161: restores only valid move histories

L163-264: Ultimate exact rules

L164-171: allows all 81 opening cells and routes by local cell

L172-182: claims a mini-board on line %s

L183-190: closes a full drawn mini-board

L191-200: grants free choice when sent to a won or drawn board

L201-214: wins the large board on line %s

L215-222: draws globally when no unresolved boards remain

L223-238: restores routing, closed boards and turn by replaying legal history

L239-253: %s AI obeys routing through a complete game

L254-263: finds an immediate global win

L265-341: Tournament state machine

L279-288: supports arbitrary registered-game contracts and shuffled configurable rounds

L289-301: scores wins and half-point draws once, then finishes with consistent totals

L302-317: persists the round token and checkpoint across closure

L318-332: invalidates stale callbacks on restart and gives rematches fresh identities

L333-340: rejects corrupted state and recomputes stored points

Helper/type visit L134-149: function visit(board: TicTacToeBoard)

Helper/type memory L270-278: function memory()

tests/unit/phase3.test.ts

467 source lines. SHA-256 13fc2061ad4075ad...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/shared/rules/MatchClock, ../../src/games/air-hockey/PossessionClock, ../../src/shared/physics/MotionController, ../../src/games/pong/PongAI, ../../src/games/pong/PongPhysics, ../../src/games/air-hockey/AirHockeyAI, ../../src/games/air-hockey/AirHockeyPhysics, ../../src/games/connect-four/ConnectFourRules, ../../src/games/connect-four/ConnectFourAI, ../../src/games/mini-golf/courses, ../../src/games/mini-golf/GolfPhysics, ../../src/games/mini-golf/GolfMatch, ../../src/games/mini-golf/GolfAI, ../../src/arcade/snake/SnakeRules, ../../src/arcade/space-shooter/ShooterWorld, ../../src/core/StorageManager, ../../src/core/ArcadeScoreManager, ../../src/shared/contracts/AIController

L30-79: Match clocks

L54-61: sudden death wins immediately; 5 wins without waiting; results are idempotent

L62-69: large stalls account for both periods; future 0-0 overtime draws

L70-78: clock uses elapsed seconds and is independent of frame frequency

L80-112: Possession

L81-95: counts one half with hysteresis, resets crossing, awards once

L96-111: center starts neutral; penalty reaches five and resets both mallets

L113-164: Stable legal AI motion

L114-135: controller caps acceleration and converges without a limit cycle

L165-235: Connect Four rules and bounded search

L166-174: has all 69 winning windows and rejects full/illegal columns

L188-214: recognizes every geometrical winning direction and a full draw

L227-234: Impossible blocks a loss and restoration rejects stale illegal history

L236-329: Mini Golf physics, turns and planning

L245-252: captures a correctly powered putt, counts a stroke once and never shoots a moving ball

L253-268: walls and obstacles survive high-speed impacts without tunneling

L269-296: both rivals play independent balls; fewer strokes wins; ties are explicit

L297-303: restores a valid checkpoint and rejects malformed positions

L315-328: Impossible can complete all five designed courses with real physics

L330-423: Solo Arcade

L331-341: Snake food grows score/body and reversal is rejected

L342-366: Snake treats a vacating tail legally, detects walls/body, and has fixed timing

L367-393: Shooter uses swept projectiles, scores kills, advances waves and bounds its pool

L394-409: Shooter respects damage immunity and ends at zero lives

L410-422: high scores persist independently of rival scores and never decrease

L425-466: Physical match checkpoints

L426-443: restores Pong clock and rally, rejecting invalid saves

L444-458: restores Hockey possession independently of the match clock

L459-465: keeps records in memory when localStorage is blocked

tests/unit/premium-motion.test.ts

64 source lines. SHA-256 092062de1fbb70c1...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/gravity-runner/Rules, ../../src/games/gravity-runner/GravityView, ../../src/games/happy-birds/Rules, ../../src/games/happy-birds/BirdView, ../../src/core/GameRegistry

L7-17: deferred games are absent from active discovery and tournaments

L18-44: gravity presentation interpolates without changing physics or snapping flips

L45-63: bird wall turns blend continuously and respawn clears old trails

tests/unit/spike-attacks.test.ts

210 source lines. SHA-256 c6024f87b79c5f52...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/spike-attacks/Rules, ../../src/games/spike-attacks/AI, ../../src/games/spike-attacks/ArenaPhysics, ../../src/games/spike-attacks/Maps, ../../src/games/spike-attacks/SpikeView

L16-25: pistons stop at the first real block and cannot strike through cover

L26-39: movement is speed limited, slides against blocks and never tunnels

L40-67: real shared attacks score once, replay simultaneous deaths and return to their socket

L68-87: warning and strike speed ramp up, with at least one reachable safe option per edge

L88-109: 100 seeded matches across all maps keep both AI players inside legal geometry

L110-138: all three maps have reachable cover on every edge and no permanent safe camping spot

L139-157: an exposed AI dies exactly like a human, even when its reaction cannot save it

L158-178: foreground deadline, completion guards and interpolation preserve legal state

L180-209: difficulty has distinct seeded reaction timing and terminal AI never acts

tests/unit/spike-controls.test.ts

68 source lines. SHA-256 f4a8611a9c409ce7...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/spike-attacks/SpikeControls

L18-34: solo arrows and WASD both control Orange; local keyboards remain independent

L35-51: gamepads handle dead zones, D-pad and two stable player assignments

L52-67: solo never assigns a controller to the AI and pause requires neutral before moving

Helper/type pad L6-17: function pad(index: number, x = 0, y = 0, dpad = -1)

tests/unit/star-patrol.test.ts

239 source lines. SHA-256 e7cf881e23fceaf9...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/arcade/space-shooter/ShooterWorld, ../../src/arcade/space-shooter/Waves, ../../src/arcade/space-shooter/Powerups, ../../src/shared/ai/MatchRandom

L42-67: levels introduce armor, aimed gunners and aces, without overlapping their spawn silhouettes

L68-79: defeating marked ships really drops gear, and every eighth ordinary kill gives additional loot

L80-96: life drops are rare milestone rewards with a ninety-second cooldown and a four-life cap

L97-124: caught upgrades add legal multi-shot cannons, faster firing and temporary wingmates

L125-146: piercing lasers hit nearer ships first and cannot damage the same armor repeatedly

L147-159: enemy attacks are warned before release and gunners aim within finite legal velocities

L160-188: shield absorbs one hit; damage preserves immunity and removes only one cannon tier

L189-204: loot survives level transitions and missed capsules expire rather than accumulate

L205-238: one hundred seeded powered scenarios retain bounded pools and stop every action after death

Helper/type guard L16-16: guard = (): Enemy =>

Helper/type pickup L17-26: function pickup(w: ShooterWorld, kind: PowerupKind)

Helper/type kill L27-41: function kill(w: ShooterWorld, e: Enemy)

Helper/type hit L165-175: hit = () =>

tests/unit/turn-view.test.ts

29 source lines. SHA-256 caaf2eab99a87874...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/shared/components/TurnView

L3-28: turn view rotates module content, inverts input, blocks transition, and destroys owned objects

tests/unit/ultimate-golf.test.ts

238 source lines. SHA-256 d37aa4022a7ee780...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: ../../src/games/mini-golf/ultimate/GolfProgress, vitest, ../../src/games/mini-golf/ultimate/courses, ../../src/games/mini-golf/ultimate/GolfPhysics, ../../src/games/mini-golf/ultimate/GolfCollision, ../../src/games/mini-golf/ultimate/GolfInput, ../../src/games/mini-golf/ultimate/GolfRules

L21-237: Ultimate Golf native port

L22-35: retains all four original layouts, starts, cups and limits

L36-45: has identical simulation at 30, 60 and 120 FPS

L46-56: rejects duplicate shots and illegal powers and settles with linear friction

L57-71: uses separating normals, including embedded centres and rotated colliders

L72-79: never tunnels through a thin wall at maximum speed

L80-91: captures every cup and allows a win on the final stroke

L92-99: ends after the last missed stroke stops

L100-109: returns hazards to the prior shot without charging another stroke

L110-123: tides flood and drain, ford flows, and contacts do not change the tide clock

L124-132: portals route with a cooldown and preserve speed

L133-144: UFO lift drops onto a legal platform and releases controls

L145-166: temple bridge cracks, regenerates, plates open doors and recovery clears outer plates

L167-174: quicksand recovery waits three stationary seconds

L175-190: rotation/inverse transforms and power are resolution-independent in 9:16 and focus views

L191-200: bounds the space preview and never mutates live state

L201-221: persists per-course bests and rejects malformed records

L222-236: survives prolonged moving-obstacle simulation at every difficulty

Helper/type run L18-20: run = (w: GolfPhysics, seconds: number, fps = 60) =>

tests/unit/water-guess.test.ts

180 source lines. SHA-256 7844c4c3c1492980...

Test/fixture implementation. Test titles and source imports identify its behavioral contract; fixtures are not shipped gameplay.

Dependencies / tested owners: vitest, ../../src/games/water-guess/WaterGuessRules, ../../src/games/water-guess/WaterGuessAI, ../../src/games/water-guess/WaterRenderer

L13-134: Water Guess Rules & Precision

L14-27: formats and parses decimal hundredths accurately

L29-37: generates actual water levels with 0.01% granularity in 10.00%..90.00%

L39-56: calculates absolute error and determines round winners correctly

L58-71: declares round tied when absolute errors are exactly identical

L73-133: handles 5-round match victory and sudden-death extension

L136-153: Water Guess Tube Geometry & Linearity

L137-152: maintains strict uniform linear cross-section and usable height

L155-179: Water Guess AI

L156-178: produces appropriate error ranges across difficulties with seeded determinism

Helper/type calcY L145-146: calcY = (hundredths: number) =>

15. Every folder, resource, data file and document

The inventory below excludes installed dependencies, Git internals, build/cache directories, this handbook workspace and machine-local SDK configuration. Copied Android public assets are generated and listed as one regeneration group. Source/reference files introduced by this handbook are listed in its reproduction note.

▪

.gitignore

Rules excluding generated/local files from Git.

.prettierrc.json

JSON object with keys: overrides.

```
{ "overrides": [{ "files": "*.svg", "options": { "parser": "html" } }] }
```

ADDING_GAMES.md

Existing project document. See [the module guide](docs/ADDING_GAMES.md) and the complete [Tap Duel example](examples/tap-duel/index.ts). Adding a valid folder and rebuilding is sufficient; no application registry or menu edits are required.

Sections: Adding games

AGENTS.md

Maintenance constraints for this folder; read before editing.

Sections: Pocket Rivals - universal rules

README.md

Existing project document. An original offline game collection for one phone: 15 local/AI rivalry games, two solo arcade games, Ultimate Golf's own three modes, and persistent tournaments. TypeScript · Phaser 3 · Vite · Capacitor 8. The existing cream/pastel visual identity, local typography, bold outlines and illustrated cards are preserved.

Sections: Pocket Rivals, Run, Play, Development, Android

index.html

Application document with menu, game-stage and overlay roots plus module startup.

```
<!doctype html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport"
content="width=device-width,initial-scale=1,maximum-scale=1,user-scalable=no,viewport-fit=cover"><meta
name="theme-color" content="#f8f5eb"><meta name="description" content="Three little games. One big rivalry. Play
Pocket Rivals together, on one screen, completely offline."><link rel="icon" type="image/svg+xml"
href="/icon.svg"><title>Pocket Rivals · Two players, one screen</title></head><body><div id="app"><div
id="game-stage"></div><main id="menu"></main><div id="overlays"></div></div><script type="module"
src="/src/main.ts"></script></body></html>
```

package-lock.json

Resolved dependency graph for reproducible npm ci; generated by package management.

package.json

JSON object with keys: name, version, private, type, scripts, dependencies, devDependencies, engines.

```
{
  "name": "pocket-rivals",
  "version": "0.3.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host 127.0.0.1",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview --host 127.0.0.1",
    "test": "vitest run",
    "test:browser": "playwright test",
    "android:sync": "npm run build && cap sync android",
    "android:open": "cap open android",
    "android:debug": "npm run android:sync && cd android && ./gradlew assembleDebug",
    "android:release": "npm run android:sync && cd android && ./gradlew bundleRelease",
```

```
    "test:modules": "node scripts/check-module.mjs",
    "format": "prettier --write \"src/**/*.{ts,css,svg}\" \"tests/**/*.ts\" \"scripts/*.{ts,mjs}\"
\"examples/**/*.{ts,svg}\" \"*.{ts,json,md}\" \"docs/*.md\"",
    "test:bundle": "node scripts/bundle-report.mjs"
  },
  "dependencies": {
    "@capacitor/android": "^8.0.0",
    "@capacitor/app": "^8.0.0",
    "@capacitor/core": "^8.0.0",
    "@capacitor/splash-screen": "^8.0.0",
    "@fontsource/fredoka": "^5.2.0",
    "@fontsource/nunito-sans": "^5.2.0",
    "katex": "^0.19.0",
    "phaser": "^3.90.0"
  },
  "devDependencies": {
    "@capacitor/cli": "^8.0.0",
    "@playwright/test": "^1.55.0",
    "@types/katex": "^0.16.8",
    "@types/node": "^24.0.0",
    "prettier": "^3.9.10",
    "typescript": "^5.9.0",
    "vite": "^7.1.0",
    "vitest": "^3.2.0"
  },
  "engines": {
    "node": ">=22.21.1"
  }
}
```

tsconfig.json

JSON object with keys: compilerOptions, include.

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
    "noEmit": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "types": ["vite/client"]
  },
  "include": ["src", "tests", "*.ts"]
}
```

vercel.json

JSON object with keys: $schema, framework, buildCommand, outputDirectory, headers.

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "headers": [
    {
      "source": "/sw.js",
      "headers": [{ "key": "Cache-Control", "value": "no-cache" }]
    }
  ]
}
```

android

android/.gitignore

Rules excluding generated/local files from Git.

android/build.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.

buildscript {

    repositories {
        google()
        mavenCentral()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:8.13.0'

        // NOTE: Do not place your application dependencies here; they belong
        // in the individual module build.gradle files
    }
}

apply from: "variables.gradle"

allprojects {
    repositories {
        google()
        mavenCentral()
    }
}

task clean(type: Delete) {
    delete rootProject.buildDir
}
```

android/capacitor.settings.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN
include ':capacitor-android'
project(':capacitor-android').projectDir = new File('../node_modules/@capacitor/android/capacitor')

include ':capacitor-app'
project(':capacitor-app').projectDir = new File('../node_modules/@capacitor/app/android')

include ':capacitor-splash-screen'
project(':capacitor-splash-screen').projectDir = new File('../node_modules/@capacitor/splash-screen/android')
```

android/gradle.properties

Gradle or tool properties; wrapper properties pin the Gradle download/distribution.

android/gradlew

Gradle wrapper launch script for Unix/Windows; selects the pinned wrapper runtime.

android/gradlew.bat

Gradle wrapper launch script for Unix/Windows; selects the pinned wrapper runtime.

android/settings.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
include ':app'
include ':capacitor-cordova-android-plugins'
project(':capacitor-cordova-android-plugins').projectDir = new File('./capacitor-cordova-android-plugins/')

apply from: 'capacitor.settings.gradle'
```

android/variables.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
ext {
    minSdkVersion = 24
```

```
    compileSdkVersion = 36
    targetSdkVersion = 36
    androidxActivityVersion = '1.11.0'
    androidxAppCompatVersion = '1.7.1'
    androidxCoordinatorLayoutVersion = '1.3.0'
    androidxCoreVersion = '1.17.0'
    androidxFragmentVersion = '1.8.9'
    coreSplashScreenVersion = '1.2.0'
    androidxWebkitVersion = '1.14.0'
    junitVersion = '4.13.2'
    androidxJunitVersion = '1.3.0'
    androidxEspressoCoreVersion = '3.7.0'
    cordovaAndroidVersion = '14.0.1'
}
```

android/app

android/app/.gitignore

Rules excluding generated/local files from Git.

android/app/build.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
apply plugin: 'com.android.application'

// Upload-key credentials are provided only through environment variables.
def releaseStore = System.getenv('POCKET_KEYSTORE')
def releaseStorePassword = System.getenv('POCKET_STORE_PASSWORD')
def releaseAlias = System.getenv('POCKET_KEY_ALIAS')
def releaseKeyPassword = System.getenv('POCKET_KEY_PASSWORD')
def hasSigning = [releaseStore, releaseStorePassword, releaseAlias, releaseKeyPassword].every { it }
def releaseRequested = gradle.startParameter.taskNames.any { it.toLowerCase().contains('release') }
if (releaseRequested && !hasSigning) {
    throw new GradleException('Release signing requires POCKET_KEYSTORE, POCKET_STORE_PASSWORD, POCKET_KEY_ALIAS and
POCKET_KEY_PASSWORD. See docs/ANDROID_RELEASE.md.')
}


android {
    namespace = "com.pocketrivals.games"
    compileSdk = rootProject.ext.compileSdkVersion
    defaultConfig {
        applicationId "com.pocketrivals.games"
        minSdkVersion rootProject.ext.minSdkVersion
        targetSdkVersion rootProject.ext.targetSdkVersion
        versionCode (project.findProperty("appVersionCode") ?: "1").toInteger()
        versionName (project.findProperty("appVersionName") ?: "0.3.0")
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
        aaptOptions {
            // Files and dirs to omit from the packaged assets dir, modified to accommodate modern web apps.
            // Default: https://android.googlesource.com/platform/frameworks/base/+/282e181b58cf72b6ca770dc7ca5f91f
135444502/tools/aapt/AaptAssets.cpp#61
            ignoreAssetsPattern = '!.svn:!.git:!.ds_store:!*.scc:.*:!CVS:!thumbs.db:!picasa.ini:!*~'
        }
    }
    signingConfigs {
        release {
            if (hasSigning) {
                storeFile file(releaseStore)
                storePassword releaseStorePassword
                keyAlias releaseAlias
                keyPassword releaseKeyPassword
            }
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
```

```
        }
    }
}

repositories {
    flatDir{
        dirs '../capacitor-cordova-android-plugins/src/main/libs', 'libs'
    }
}

dependencies {
    implementation fileTree(include: ['*.jar'], dir: 'libs')
    implementation "androidx.appcompat:appcompat:$androidxAppCompatVersion"
    implementation "androidx.coordinatorlayout:coordinatorlayout:$androidxCoordinatorLayoutVersion"
    implementation "androidx.core:core-splashscreen:$coreSplashScreenVersion"
    implementation project(':capacitor-android')
    testImplementation "junit:junit:$junitVersion"
    androidTestImplementation "androidx.test.ext:junit:$androidxJunitVersion"
    androidTestImplementation "androidx.test.espresso:espresso-core:$androidxEspressoCoreVersion"
    implementation project(':capacitor-cordova-android-plugins')
}

apply from: 'capacitor.build.gradle'
```

android/app/capacitor.build.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN

android {
  compileOptions {
      sourceCompatibility JavaVersion.VERSION_21
      targetCompatibility JavaVersion.VERSION_21
  }
}

apply from: "../capacitor-cordova-android-plugins/cordova.variables.gradle"
dependencies {
    implementation project(':capacitor-app')
    implementation project(':capacitor-splash-screen')

}


if (hasProperty('postBuildExtras')) {
  postBuildExtras()
}
```

android/app/proguard-rules.pro

Project configuration or resource; complete textual content is in the companion source reference where applicable.

android/app/src/main

android/app/src/main/AndroidManifest.xml

Native Android resource or manifest. Defines app/activity/orientation/theme and launcher intent.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="false" android:usesCleartextTraffic="false"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <activity
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|locale|smallestScreenSize|screenL
```

```
ayout|uiMode|navigation|density"
            android:name=".MainActivity"
            android:label="@string/title_activity_main"
            android:theme="@style/AppTheme.NoActionBarLaunch"
            android:launchMode="singleTask"
            android:screenOrientation="portrait"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

    <!-- Permissions -->


</manifest>
```

android/app/src/main/assets

android/app/src/main/assets/capacitor.config.json

JSON object with keys: appId, appName, webDir, backgroundColor, android, plugins.

android/app/src/main/assets/capacitor.plugins.json

JSON array of 2 entries.

android/app/src/main/res/drawable

android/app/src/main/res/drawable/ic_launcher_background.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable/pocket_mark.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-land-hdpi

android/app/src/main/res/drawable-land-hdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-land-mdpi

android/app/src/main/res/drawable-land-mdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-land-xhdpi

android/app/src/main/res/drawable-land-xhdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-land-xxhdpi

android/app/src/main/res/drawable-land-xxhdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-land-xxxhdpi

android/app/src/main/res/drawable-land-xxxhdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-port-hdpi

android/app/src/main/res/drawable-port-hdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-port-mdpi

android/app/src/main/res/drawable-port-mdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-port-xhdpi

android/app/src/main/res/drawable-port-xhdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-port-xxhdpi

android/app/src/main/res/drawable-port-xxhdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-port-xxxhdpi

android/app/src/main/res/drawable-port-xxxhdpi/splash.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/drawable-v24

android/app/src/main/res/drawable-v24/ic_launcher_foreground.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/layout

android/app/src/main/res/layout/activity_main.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/mipmap-anydpi-v24

android/app/src/main/res/mipmap-anydpi-v24/ic_launcher.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/mipmap-anydpi-v24/ic_launcher_round.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/mipmap-anydpi-v26

android/app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/mipmap-hdpi

android/app/src/main/res/mipmap-hdpi/ic_launcher.png

Historical screenshot or native raster resource; 72x72 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-hdpi/ic_launcher_foreground.png

Historical screenshot or native raster resource; 162x162 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-hdpi/ic_launcher_round.png

Historical screenshot or native raster resource; 72x72 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-mdpi

android/app/src/main/res/mipmap-mdpi/ic_launcher.png

Historical screenshot or native raster resource; 48x48 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-mdpi/ic_launcher_foreground.png

Historical screenshot or native raster resource; 108x108 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-mdpi/ic_launcher_round.png

Historical screenshot or native raster resource; 48x48 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xhdpi

android/app/src/main/res/mipmap-xhdpi/ic_launcher.png

Historical screenshot or native raster resource; 96x96 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xhdpi/ic_launcher_foreground.png

Historical screenshot or native raster resource; 216x216 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xhdpi/ic_launcher_round.png

Historical screenshot or native raster resource; 96x96 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xxhdpi

android/app/src/main/res/mipmap-xxhdpi/ic_launcher.png

Historical screenshot or native raster resource; 144x144 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xxhdpi/ic_launcher_foreground.png

Historical screenshot or native raster resource; 324x324 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xxhdpi/ic_launcher_round.png

Historical screenshot or native raster resource; 144x144 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xxxhdpi

android/app/src/main/res/mipmap-xxxhdpi/ic_launcher.png

Historical screenshot or native raster resource; 192x192 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xxxhdpi/ic_launcher_foreground.png

Historical screenshot or native raster resource; 432x432 pixels. Density-specific launcher icon resource.

android/app/src/main/res/mipmap-xxxhdpi/ic_launcher_round.png

Historical screenshot or native raster resource; 192x192 pixels. Density-specific launcher icon resource.

android/app/src/main/res/values

android/app/src/main/res/values/colors.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/values/ic_launcher_background.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/values/strings.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/values/styles.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/xml

android/app/src/main/res/xml/config.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/app/src/main/res/xml/file_paths.xml

Native Android resource or manifest. Resource references connect native branding, layout, provider paths or themes.

android/capacitor-cordova-android-plugins

android/capacitor-cordova-android-plugins/build.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
ext {
    androidxAppCompatVersion = project.hasProperty('androidxAppCompatVersion') ?
rootProject.ext.androidxAppCompatVersion : '1.7.1'
    cordovaAndroidVersion = project.hasProperty('cordovaAndroidVersion') ? rootProject.ext.cordovaAndroidVersion :
'14.0.1'
}

buildscript {
    repositories {
        google()
        mavenCentral()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:8.13.0'
    }
}

apply plugin: 'com.android.library'

android {
    namespace = "capacitor.cordova.android.plugins"
    compileSdk = project.hasProperty('compileSdkVersion') ? rootProject.ext.compileSdkVersion : 36
    defaultConfig {
        minSdkVersion project.hasProperty('minSdkVersion') ? rootProject.ext.minSdkVersion : 24
        targetSdkVersion project.hasProperty('targetSdkVersion') ? rootProject.ext.targetSdkVersion : 36
        versionCode 1
        versionName "1.0"
    }
    lintOptions {
        abortOnError = false
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_21
        targetCompatibility JavaVersion.VERSION_21
    }
}

repositories {
    google()
```

```
    mavenCentral()
    flatDir{
        dirs 'src/main/libs', 'libs'
    }
}

dependencies {
    implementation fileTree(dir: 'src/main/libs', include: ['*.jar'])
    implementation "androidx.appcompat:appcompat:$androidxAppCompatVersion"
    implementation "org.apache.cordova:framework:$cordovaAndroidVersion"
    // SUB-PROJECT DEPENDENCIES START

    // SUB-PROJECT DEPENDENCIES END
}

// PLUGIN GRADLE EXTENSIONS START
apply from: "cordova.variables.gradle"
// PLUGIN GRADLE EXTENSIONS END

for (def func : cdvPluginPostBuildExtras) {
    func()
}
```

android/capacitor-cordova-android-plugins/cordova.variables.gradle

Gradle project/plugin/dependency/signing configuration. Capacitor-named files are generated connection glue; regenerate through cap sync when appropriate.

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN
ext {
  cdvMinSdkVersion = project.hasProperty('minSdkVersion') ? rootProject.ext.minSdkVersion : 24
  // Plugin gradle extensions can append to this to have code run at the end.
  cdvPluginPostBuildExtras = []
  cordovaConfig = [:]
}
```

android/capacitor-cordova-android-plugins/src/main

android/capacitor-cordova-android-plugins/src/main/AndroidManifest.xml

Native Android resource or manifest. Defines app/activity/orientation/theme and launcher intent.

```
<?xml version='1.0' encoding='utf-8'?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:amazon="http://schemas.amazon.com/apk/res/android">
<application  >

</application>

</manifest>
```

android/capacitor-cordova-android-plugins/src/main/java

android/capacitor-cordova-android-plugins/src/main/java/.gitkeep

Empty marker retaining a native resource/source directory.

android/capacitor-cordova-android-plugins/src/main/res

android/capacitor-cordova-android-plugins/src/main/res/.gitkeep

Empty marker retaining a native resource/source directory.

android/gradle/wrapper

android/gradle/wrapper/gradle-wrapper.jar

Binary pinned Gradle wrapper launcher; no gameplay logic to edit.

android/gradle/wrapper/gradle-wrapper.properties

Gradle or tool properties; wrapper properties pin the Gradle download/distribution.

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.14.3-all.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

docs

docs/ADDING_GAMES.md

Existing project document. 1. Create `src/games/<folder>/` for a duel, or `src/arcade/<folder>/` for a solo arcade game. 2. Add `config.ts`, `index.ts`, your implementation, and a small original SVG preview. 3. Implement `MiniGame` from `src/shared/contracts/GameModule.ts`. 4. Run tests and rebuild. Metadata discovery, the menu, lazy loader and ... [continued in source reference]

Sections: Add a game without editing core, Metadata, Working example, Isolation and performance checklist, Commands, Additional reference modules

docs/AI_REFINEMENT.md

Existing project document. Scope: Pong, Air Hockey, Classic and Ultimate Tic-Tac-Toe, Connect Four, Dots and Boxes, Hammer Hit, Light Fingers, Fruit Duel, Lumberjack, and the active Ultimate Golf port. Math Quiz is owned by concurrent development and was not edited. Water Guess and Perfect Timer appeared during concurrent development and already ... [continued in source reference]

Sections: AI refinement, Match identity and replay, Changes and causes, Performance and boundaries, Verification, Recorded results (initial AI pass), Follow-up integration

docs/ANDROID_RELEASE.md

Existing project document. Status as of **2026-10-13**: Capacitor Android project generated and configured. Java/Android SDK are unavailable on this workstation; native compilation, emulator execution and device testing remain outstanding. No publication has been attempted.

Sections: Android build and Google Play preparation, Toolchain, Debug APK, Identity and versions, Signing without repository secrets, Target SDK and native behavior, Publisher release checklist

docs/ANTIGRAVITY_REVIEW.md

Existing project document. Scope: preserve Antigravity's artwork and mechanics; inspect rules, AI, input, scoring, lifecycle and integration. Math Quiz source, question banks, rendering and AI were excluded from edits. Its tests ran read-only.

Sections: Formal review - Water Guess, Perfect Timer, Hammer Hit, Confirmed findings and repairs, Verification and limits, Additional Golf integration findings

docs/ARCHITECTURE.md

Existing project document. Pocket Rivals has one persistent Phaser instance and one reusable host scene. The DOM owns scrolling menus and accessible modal controls; Phaser owns gameplay and its HUD. Default logical coordinates are 420x760, fitted to the viewport and rendered at up to 2x device resolution. Modules may declare responsive custom pr ... [continued in source reference]

Sections: Architecture, Lifecycle and navigation, Module boundaries, Persistence and tournaments, AI and performance, Web and Android, Expanded rivalry collection

docs/EXPANSION_COMPLETION.md

Existing project document. The remaining seven games from specifications/13-games.md are implemented as separate lazy modules. Together with the existing six, all thirteen requested entries are present. The reference PDF was not supplied: written rules govern implementation, while numerical tuning and timeout defaults are original and documented ... [continued in source reference]

Sections: Seven-game expansion and final integration, Integration, Cleaner interface, Verification status

docs/GRAVITY_BIRDS_REFINEMENT.md

Existing project document. Details below record an earlier milestone. See GRAVITY_TRAP_VARIATION.md for later Gravity changes and HAPPY_BIRDS_STARS.md for the subsequent star race and recovery rules.

Sections: Gravity Runner / Happy Birds refinement, Verification

docs/GRAVITY_RUNNER_REFINEMENT.md

Existing project document. The supplied YouTube short was viewed in the in-app browser. Screenshots at approximately 21% and 55% showed two runners on a shared vertical corridor, grassy stepped walls, gaps and central bars. The new original artwork follows this composition using Pocket Rivals colors and typography.

Sections: Gravity Runner refinement - 16 October 2026, Motion and deadline follow-up

docs/GRAVITY_TRAP_VARIATION.md

Existing project document. The fixed five-pattern loop is replaced by a seeded weighted stream. Each map has its own trap pool and motif weights. The previous two motifs are excluded from selection; side, count, width, depth, spacing and dynamic phase vary. Only an early rail lesson is guaranteed. New sections continue indefinitely; old geometry ... [continued in source reference]

Sections: Gravity Runner: map-specific endless courses

docs/HAPPY_BIRDS_STARS.md

Existing project document. First to 15 actual star pickups wins. One shared gold star replaces the old survival point system; nearest contact earns the pickup, while exactly simultaneous equal contacts share it. Equal same-step finishes draw. There is no two-minute deadline. Collisions retain every collected star and never award an opponent a po ... [continued in source reference]

Sections: Happy Birds: star race and animated recovery

docs/PRIVACY.md

Existing project document. Draft for publication, 2026-10-13. Add your publisher identity, contact email, and effective release date before hosting this as the public privacy policy.

Sections: Pocket Rivals privacy and data handling

docs/PROGRESS.md

Existing project document. The original AI refinement pass is complete. See AI_REFINEMENT.md for profiles, replay, audit data and limits. Math Quiz remains owned by Antigravity; no Math Quiz source was edited in this pass.

Sections: Development status - 15 October 2026

docs/QA.md

Existing project document. All commands ran locally. Nothing was deployed or pushed. Chrome automation uses a mobile viewport/touch emulation; it is not physical Android testing.

Sections: Verification record - 2026-10-13, Automated checks, Gameplay evidence, Measured performance, Remaining verification / limits

docs/SPIKE_ATTACKS_REFINEMENT.md

Existing project document. The supplied [reference short](https://www.youtube.com/shorts/p1eZhSl-U_g) was viewed in the in-app browser, including frames around 18%, 36%, 69% and 83%. It shows two players in one tiled square arena, three solid cover blocks, four wide piston spikes per edge, warning lamps and extending/retracting attacks. The prev ... [continued in source reference]

Sections: Spike Attacks - 16 October 2026, Changes, Verification, Local preview and controls follow-up, AI collision and maps follow-up

docs/STAR_PATROL.md

Existing project document. Marked gold supply ships carry a visible reward icon and drop a real falling capsule when defeated. Every eighth ordinary kill also adds randomized gear. Loot persists between levels, expires if missed and gently attracts toward a nearby ship in the collection area. Score is earned through combat; no automatic gear is ... [continued in source reference]

Sections: Star Patrol combat update, Verification

docs/TEMPLE_AI_FIX.md

Existing project document. The replay audit reproduced repeated shots around the bumper at (1060,960). One route aimed at the bumper center, which cannot satisfy its 25-unit waypoint radius around a radius-22 collider plus the radius-seven ball. Other variants aimed just behind it and approached from the obstructed side. New approach points clea ... [continued in source reference]

Sections: Lost Temple AI correction - 16 October 2026

docs/THIRD_PARTY_NOTICES.md

Existing project document. All dependencies are resolved in `package-lock.json`. Preserve their full license notices when distributing source or binaries.

Sections: Third-party notices

docs/UI_REFINEMENT.md

Existing project document. Validation: 238 existing unit tests passed; all four Hammer tests including the new distribution check passed after the timing refinement. Five targeted browser scenarios passed (Water rounds/keypad, Hammer rotation/alternating turns, Light Fingers false starts/pause/results). Final TypeScript/Vite build and bundle che ... [continued in source reference]

Sections: Interface refinement - 15 October 2026

docs/ULTIMATE_GOLF.md

Existing project document. Source studied: [abdullah-b41/L1T1-Game-Project](https://github.com/abdullah-b41/L1T1-Game-Project), commit `06dcdc65b06e9767e9089a032f1e465fe8ee931b`, local `~/Documents/Game-References/Ultimate-Golf/the_ultimate_golf/the_ultimate_golf.c` (6,015 lines). The source credits Abdullah Al Nafi and Syed Abdul Fahim. The ref ... [continued in source reference]

Sections: Ultimate Golf port, Courses and mechanics, Original rules retained, Reliability adaptations and scope, Evidence, Golf modes

docs/ai-audit.json

JSON object with keys: description, records.

docs/air-hockey.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/arcade.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/archery-duel-review.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/bundle-report.json

JSON object with keys: initial, lazyGames, totalInitialGzip.

docs/chicken-jump-review.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/connect-four.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/dots-and-boxes.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/expansion-performance.json

JSON object with keys: environment, measuredAt, results.

docs/flappy-jump-review.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/fruit-decoy.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/fruit-sliced.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/fruit-sword-rest.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/fruit-sword-return.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-1-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-1.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-2-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-2.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-3-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-3.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-4-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-4.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-Event-Horizon-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-Foundry-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-Lost-Temple-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-Shoreline-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-browser-completion.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-course-previews-mobile.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/golf-course-previews.png

Historical screenshot or native raster resource; 2688x2625 pixels. Generated review evidence; runtime art is in source.

docs/golf-course-select.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf-layout-1280.png

Historical screenshot or native raster resource; 3360x2363 pixels. Generated review evidence; runtime art is in source.

docs/golf-layout-320.png

Historical screenshot or native raster resource; 840x1491 pixels. Generated review evidence; runtime art is in source.

docs/golf-layout-844.png

Historical screenshot or native raster resource; 2216x1024 pixels. Generated review evidence; runtime art is in source.

docs/golf-modes.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/golf2-focus-mobile.png

Historical screenshot or native raster resource; 390x844 pixels. Generated review evidence; runtime art is in source.

docs/gravity-blue.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/gravity-green.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/gravity-ice.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/gravity-map-previews.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/gravity-middle-rail.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/gravity-purple.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/gravity-runner-review.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/gravity-shared-mobile.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/gravity-shared-obstacles.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/gravity-shared-run.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/gravity-shared-start.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/hammer-clean-blue.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/hammer-clean-orange.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/hammer-fixed.png

Historical screenshot or native raster resource; 420x820 pixels. Generated review evidence; runtime art is in source.

docs/hammer-hit-strike.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/hammer-hit.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/hammer-ring.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/happy-birds-keyboard.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/happy-birds-mobile.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/happy-birds-refined.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/happy-birds-review.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/happy-birds-shared.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/happy-birds-simple-preview.png

Historical screenshot or native raster resource; 819x475 pixels. Generated review evidence; runtime art is in source.

docs/happy-stars-flying.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/happy-stars-gray.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/happy-stars-return.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/light-hands.png

Historical screenshot or native raster resource; 420x820 pixels. Generated review evidence; runtime art is in source.

docs/light-reach.png

Historical screenshot or native raster resource; 420x820 pixels. Generated review evidence; runtime art is in source.

docs/light-stolen.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/lumber-chop.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/lumber-polish.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/math-quiz-mobile.png

Historical screenshot or native raster resource; 840x1491 pixels. Generated review evidence; runtime art is in source.

docs/math-validation.json

JSON object with keys: count, engine, method, independentHumanReview, mentalTime, sha256, results.

docs/menu-mobile.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/mini-golf.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/perfect-timer.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/performance-report.json

JSON object with keys: environment, measuredAt, results.

docs/ping-pong.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/results.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/setup-ai.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/snake-paused.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/snake.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-attacks-polished-preview.png

Historical screenshot or native raster resource; 819x475 pixels. Generated review evidence; runtime art is in source.

docs/spike-attacks-review.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-keyboard-controller.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-map-courtyard.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-map-crosswind.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-map-previews.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/spike-map-random.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-map-split-steps.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-shared-mobile.png

Historical screenshot or native raster resource; 945x1680 pixels. Generated review evidence; runtime art is in source.

docs/spike-shared-start.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/spike-shared-strike.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/star-patrol-desktop.png

Historical screenshot or native raster resource; 3360x2100 pixels. Generated review evidence; runtime art is in source.

docs/star-patrol-upgraded.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/star-patrol.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/temple-ai-hard.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/temple-ai-impossible.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/temple-ai-medium.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/tic-tac-toe.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/tournament-champion.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/ultimate-focus.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/ultimate-golf.png

Historical screenshot or native raster resource; 412x915 pixels. Generated review evidence; runtime art is in source.

docs/ultimate-tutorial.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/ultimate.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/water-clean-blue.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/water-clean-orange.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/water-guess.png

Historical screenshot or native raster resource; 1082x2202 pixels. Generated review evidence; runtime art is in source.

docs/water-result-color.png

Historical screenshot or native raster resource; 420x820 pixels. Generated review evidence; runtime art is in source.

docs/winner-color.png

Historical screenshot or native raster resource; 420x820 pixels. Generated review evidence; runtime art is in source.

docs/licenses

docs/licenses/capacitor-android-LICENSE

Third-party license text copied for distribution notices; preserve attribution and terms.

docs/licenses/capacitor-app-LICENSE

Third-party license text copied for distribution notices; preserve attribution and terms.

docs/licenses/capacitor-core-LICENSE

Third-party license text copied for distribution notices; preserve attribution and terms.

docs/licenses/capacitor-splash-screen-LICENSE

Third-party license text copied for distribution notices; preserve attribution and terms.

docs/licenses/fredoka-LICENSE

Third-party license text copied for distribution notices; preserve attribution and terms.

docs/licenses/nunito-sans-LICENSE

Third-party license text copied for distribution notices; preserve attribution and terms.

docs/licenses/phaser-LICENSE.md

Existing project document. The MIT License (MIT)

Sections:

docs/specifications

docs/specifications/13-games.md

Existing project document. You are working on **Pocket Rivals**, an existing minimalist multiplayer mini-game collection developed using:

Sections: POCKET RIVALS - GRAND CODEX IMPLEMENTATION PROMPT, 13 New Mini-Games, Advanced Mathematical Challenges, AI Opponents, Sound Effects, Tournaments & Cross-Platform Optimization, 1. PROJECT OVERVIEW, Fundamental Development Requirement, Priority Order, 2. PRE-IMPLEMENTATION ANALYSIS, 3. GAMES TO IMPLEMENT, 4. DETAILED GAME SPECIFICATIONS, GAME 01 - CAR DUEL, Gameplay Concept, Rules, Controls, Physics Requirements, AI, Audio, GAME 02 - ARCHERY DUEL, Gameplay Concept, Rules, Scoring, Important Details

examples/tap-duel/assets

examples/tap-duel/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

public

public/icon.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/arcade/snake

src/arcade/snake/AGENTS.md

Maintenance constraints for this folder; read before editing. SnakeRules.ts owns the grid, fixed movement interval, two-direction input queue, safe tail movement and food spawning. SnakeGame.ts only renders/interpolates and maps pad/swipe/keyboard input. Keep movement deterministic under different render rates. Pause clears queued directions. Report finishArcade; never duel point ... [continued in source reference]

Sections: Solo arcade module

src/arcade/snake/assets

src/arcade/snake/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/arcade/space-shooter

src/arcade/space-shooter/AGENTS.md

Maintenance constraints for this folder; read before editing. ShooterWorld.ts owns fixed-step combat, swept segment collisions, damage immunity and bounded pools (80 projectiles, 8 pickups, 24 effects). Waves.ts defines formations, enemy types and difficulty progression. Powerups.ts owns gear metadata and rare-life eligibility. ShooterArt.ts draws original vectors; SpaceShooterGa ... [continued in source reference]

Sections: Solo arcade module

src/arcade/space-shooter/assets

src/arcade/space-shooter/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/future-games

src/future-games/README.md

Existing project document. Flappy Jump (`flappy-jump`), Car Duel (`car-duel`), Archery Duel (`archery-duel`) and Chicken Jump (`chicken-jump`) are paused projects, preserved intact with their assets, metadata, AI, rules and module instructions.

Sections: Future games

src/future-games/archery-duel

src/future-games/archery-duel/AGENTS.md

Maintenance constraints for this folder; read before editing.

Sections: Archery Duel

src/future-games/archery-duel/assets

src/future-games/archery-duel/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/future-games/car-duel

src/future-games/car-duel/AGENTS.md

Maintenance constraints for this folder; read before editing.

Sections: Car Duel

src/future-games/car-duel/assets

src/future-games/car-duel/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/future-games/chicken-jump

src/future-games/chicken-jump/AGENTS.md

Maintenance constraints for this folder; read before editing.

Sections: Chicken Jump

src/future-games/chicken-jump/assets

src/future-games/chicken-jump/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/future-games/flappy-jump

src/future-games/flappy-jump/AGENTS.md

Maintenance constraints for this folder; read before editing.

Sections: Flappy Jump

src/future-games/flappy-jump/assets

src/future-games/flappy-jump/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/air-hockey

src/games/air-hockey/AGENTS.md

Maintenance constraints for this folder; read before editing. AirHockeyPhysics.ts integrates at 120 Hz with three collision substeps. rink.ts owns rounded rails, goals and reachability. Resolve only approaching normals; project penetration consistently, preserve tangent motion, and interpolate mallets over all substeps. The watchdog repairs only unreachable/invalid states after 0 ... [continued in source reference]

Sections: Air Hockey module

src/games/air-hockey/assets

src/games/air-hockey/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/connect-four

src/games/connect-four/AGENTS.md

Maintenance constraints for this folder; read before editing. ConnectFourRules.ts owns gravity, legal columns, 69 winning windows, undo and replay validation. ConnectFourAI.ts mutates/undoes only a worker-local board and must restore it even on timeout. Search uses iterative deepening, bounded alpha-beta and transposition entries. ConnectFourGame blocks input during the drop anim ... [continued in source reference]

Sections: Connect Four module

src/games/connect-four/assets

src/games/connect-four/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/dots-and-boxes

src/games/dots-and-boxes/AGENTS.md

Maintenance constraints for this folder; read before editing. Rules.ts owns 40 edges and 16 boxes. Horizontal edges 0-19, vertical 20-39. Only completed boxes score; capture retains turn, including double captures. Saves replay legal history. AI is cancellable worker alpha-beta with extra-turn awareness, capture/chain ordering and bounded endgame search; no perfect-play claim. Ga ... [continued in source reference]

Sections: Dots and Boxes

src/games/dots-and-boxes/assets

src/games/dots-and-boxes/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/fruit-duel

src/games/fruit-duel/AGENTS.md

Maintenance constraints for this folder; read before editing. Rules owns seven real-fruit IDs (0-6), three decoys (7 gift, 8 medal, 9 flower), reveal timing, cooldowns and first-to-ten. Real fruit is guaranteed within three reveals. Any real fruit scores; there is no target-memory task. Empty/decoy swings cause 1.35s recovery and never give points to the rival. Successful swings ... [continued in source reference]

Sections: Fruit Duel

src/games/fruit-duel/assets

src/games/fruit-duel/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/gravity-runner

src/games/gravity-runner/AGENTS.md

Maintenance constraints for this folder; read before editing. One shared portrait corridor; cross-track physics y=0..170, progress scrolls vertically. Both runners share distance, hazards and legal flips. Orange Space/A/lower touch half; Blue Up/Enter/L/upper half. First five survival points; simultaneous deaths replay. Runs have no time limit or forced reset: they end only on co ... [continued in source reference]

Sections: Gravity Runner

src/games/gravity-runner/assets

src/games/gravity-runner/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/hammer-hit

src/games/hammer-hit/AGENTS.md

Maintenance constraints for this folder; read before editing. Rules.ts owns exactly three integer strike scores per player. Compare integer totals; divide by three only for display/result averages. Never use best strike or rounded averages to choose a winner. The controller runs alternating full-field turns; the rule engine retains separate sine gauges. Reset the active gauge aft ... [continued in source reference]

Sections: Hammer Hit

src/games/hammer-hit/assets

src/games/hammer-hit/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/happy-birds

src/games/happy-birds/AGENTS.md

Maintenance constraints for this folder; read before editing. One shared 320x460 arena; auto-flying birds bounce on safe walls and flap through ordinary input. First to 15 collected stars wins, with a draw for equal same-step finishes. No survival points, score penalties, round resets or match deadline. One 1.1-second opening beat precedes the continuous race.

Sections: Happy Birds

src/games/happy-birds/assets

src/games/happy-birds/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/light-fingers

src/games/light-fingers/AGENTS.md

Maintenance constraints for this folder; read before editing. Rules owns waiting/signal/active/between/done and first-to-five. Signal becomes claimable only after the render update presents it. FirstResponse arbitrates the first tap per player; same-frame ties leave the gem available and lock both players briefly. Early/missed grabs impose a 1.25s cooldown with no score changes; ... [continued in source reference]

Sections: Light Fingers

src/games/light-fingers/assets

src/games/light-fingers/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/lumberjack

src/games/lumberjack/AGENTS.md

Maintenance constraints for this folder; read before editing. Rules owns identical seeded segment streams, each player's chop index, side, idle budget and three-round totals. Adjacent segments never carry branches on opposite sides, ensuring a legal choice exists. Check both current/next branches before a chop. Defaults: 20-second round cap, 85ms input cooldown, idle allowance fa ... [continued in source reference]

Sections: Lumberjack

src/games/lumberjack/assets

src/games/lumberjack/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/math-quiz

src/games/math-quiz/AGENTS.md

Maintenance constraints for this folder; read before editing. Banked university questions: `questions/university.json` (100 verified static questions) supplemented by procedural question families in `questions/ProceduralUniversity.ts` across 8 topics (Limits, Derivatives, Integrals, Trigonometry, Algebra, Matrices, Complex numbers, Probability) providing >430+ distinct valid inst ... [continued in source reference]

Sections: Math Quiz

src/games/math-quiz/assets

src/games/math-quiz/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/math-quiz/questions

src/games/math-quiz/questions/university.json

JSON array of 100 entries. Static 100-question bank: edit schema/correct answers carefully; verify-math-bank.py overwrites it.

Topic	Static questions
Algebra	16
Complex numbers	12
Derivatives	20
Integrals	16
Limits	16

Topic	Static questions
Matrices	12
Probability	8

Example data shape (one current entry):

```
{
  "id": "log-chain-2",
  "topic": "Derivatives",
  "difficulty": "medium",
  "questionLatex": "\\left.\\frac{d}{dx}\\ln(1+2x)\\right|_{x=0}",
  "optionsLatex": [
    "2",
    "3",
    "1"
  ],
  "correctIndex": 0,
  "explanationLatex": "\\frac{2}{1+2x}\\bigg|_{0}=2",
  "estimatedSeconds": 12
}
```

src/games/mini-golf

src/games/mini-golf/AGENTS.md

Maintenance constraints for this folder; read before editing. `index.ts` now loads `ultimate/UltimateGolfGame.ts`. Read `ultimate/AGENTS.md` and `docs/ULTIMATE_GOLF.md` for the active port. Registration ID remains `mini-golf`; display name is Ultimate Golf. It is an Arcade entry with its own casual, AI race and hot-seat modes, excluded from global tournaments. Do not add a second ... [continued in source reference]

Sections: Golf entry

src/games/mini-golf/assets

src/games/mini-golf/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/mini-golf/ultimate

src/games/mini-golf/ultimate/AGENTS.md

Maintenance constraints for this folder; read before editing. AI identities use the lazy `shared/ai/MatchRandom.ts` utility; keep tactical parameters inside this module and preserve personality across rounds. See `docs/AI_REFINEMENT.md` for seeded replay and audit commands. Preserve legal actions and bounded worker/motion behavior when adding variety.

Sections: Ultimate Golf

src/games/perfect-timer

src/games/perfect-timer/AGENTS.md

Maintenance constraints for this folder; read before editing. Preserve Antigravity's original layout/renderer. Rules own integer precision, submissions, idempotent resolution and five-round/sudden-death progression; reject nonfinite input. AI is seeded per match and stays inside this folder. Controllers own scene objects; host input subscriptions are automatically removed. Pause/ ... [continued in source reference]

Sections: Perfect Timer

src/games/perfect-timer/assets

src/games/perfect-timer/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/pong

src/games/pong/AGENTS.md

Maintenance constraints for this folder; read before editing. PongPhysics.ts owns swept paddle collisions; PongAI.ts owns prediction; PongState.ts validates saved rallies. Game registration ID is ping-pong: do not rename it. PongGame.ts connects MatchClock to physics and HUD. Never reset the match clock on a goal.

Sections: Pong module

src/games/pong/assets

src/games/pong/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/spike-attacks

src/games/spike-attacks/AGENTS.md

Maintenance constraints for this folder; read before editing. One shared 320x320 arena with three selectable layouts (Courtyard, Split Steps, Crosswind) plus seeded Random. Maps.ts owns blocks, spawns and floor colors; config derives variants from it. Each layout has three fixed rectangular cover blocks and four retracting piston lanes on every edge. ArenaPhysics is the source of ... [continued in source reference]

Sections: Spike Attacks

src/games/spike-attacks/assets

src/games/spike-attacks/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/tic-tac-toe

src/games/tic-tac-toe/AGENTS.md

Maintenance constraints for this folder; read before editing. The entry factory lazily imports classic/ClassicGame.ts or ultimate/UltimateGame.ts. Each variant owns its rules and AI. shared/lines.ts contains only module-local winning lines. UltimateRules is authoritative for routing/closed boards and restores legal move histories. UltimateGame maps board*9+cell into normal/focus ... [continued in source reference]

Sections: Tic-Tac-Toe module

src/games/tic-tac-toe/assets

src/games/tic-tac-toe/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

src/games/water-guess

src/games/water-guess/AGENTS.md

Maintenance constraints for this folder; read before editing. Preserve Antigravity's original layout/renderer. Rules own integer precision, submissions, idempotent resolution and five-round/sudden-death progression; reject nonfinite input. AI is seeded per match and stays inside this folder. Controllers own scene objects; host input subscriptions are automatically removed. Pause/ ... [continued in source reference]

Sections: Water Guess

src/games/water-guess/assets

src/games/water-guess/assets/preview.svg

Original static vector preview/icon. Menu art is independent from live physics/rendering.

Generated and local directories

Directory/group	Purpose and regeneration
dist/	Vite output: hashed JS/CSS, worker chunks, fonts, .vite/manifest.json, icon, HTML and sw.js. Run npm run build.
android/app/src/main/assets/public/	Capacitor copy of a web build, including manifest/chunks/fonts/sw and Cordova compatibility stubs. Run npm run android:sync; this copy can lag src.
node_modules/	Installed dependency internals, font/KaTeX/Phaser assets, caches and package binaries. Recreate with npm ci.
android/.gradle, android/**/build	Gradle caches and compiled APK/AAB/intermediates; regenerate with the appropriate Gradle task.
test-results/, playwright-report/	Generated browser artifacts and reports; regenerated by Playwright.
tmp/pdfs/handbook/	Render/QC previews and execution logs for this edition.
output/pdf/	Final PDF, editable combined handbook, complete textual source reference and coverage ledger.
../.git and ../tmp	Parent repository metadata and unrelated exam-grading scans; not game source.

16. Snapshot, coverage and validation ledger

Coverage item	Count / interpretation
Maintained code files in this edition	305 (source, tools, configurations, tests; generated native copies excluded)
Files under src/	225 parsed code/CSS files
Named TypeScript blocks under src/	1137 classes/types/functions/methods/getters (anonymous callbacks belong to their containing block)
Inventory entries	599 including generated Android public-copy entries, excluding dependency/build/cache internals
Code file atlas + test entries	305; coverage.json checks every parsed maintained file is represented
Source hashes	Full SHA-256 per code file in code-index.json and coverage.json
Snapshot time	16 October 2026 at 18:14:01 GMT+6

Source is editable and other work may proceed concurrently. The PDF and companion source reference describe the exact extracted sources identified by their hashes. A later successful build or an older QA note does not validate different source bytes.

Application source changes for this request: None. Only documentation/generation artifacts were authored. Other concurrent project work was observed; source fingerprints define this edition.

npm run build: Passed at the initial review checkpoint (strict TypeScript and production Vite build). Vite reported the large shared Phaser chunk warning. The source reference may include later concurrent work; see any final snapshot check below.

npm run test:bundle: Passed at the initial checkpoint; 18 registered entry paths were verified and totalInitialGzip was 360135 bytes. The report is a build snapshot.

npm test: Initial run: 30 files passed, one file failed; 287 tests passed and one Car Duel 100-seed stress case exceeded the configured 20000ms timeout. No failed assertion was reported for that case.

Car Duel isolated recheck: npx vitest run tests/unit/car-duel.test.ts passed all 3 tests; the stress case completed in 12.534 seconds. No application/test configuration was changed for the recheck.

Browser and native checks: Not rerun for this documentation request. Existing project QA records are historical; no physical Android/iOS validation or APK/AAB release is claimed.

PDF verification: The delivered edition is rendered and inspected separately; coverage.json checks that every indexed maintained code file has an atlas/test entry.

Final production build: npm run build passed again after the Star Patrol source additions (strict TypeScript plus Vite; 3.26s Vite build). The shared large-chunk warning remains.

Final bundle check: npm run test:bundle passed again against the rebuilt dist. The report describes that build, not a physical device.

Final unit run: npx vitest run --maxWorkers=2: 31 files passed and one failed; 296 tests passed and the same Car Duel 100-seed stress test exceeded 20000ms again. The new Star Patrol unit suite passed. The earlier isolated Car Duel run passed; the full-suite timeout remains unresolved and is not represented as a green suite.

Existing docs/QA.md and earlier JSON measurements are historical records. Their older module counts, course camera descriptions and feature claims may differ from the current source. The active module path, current source expressions and this edition's validation ledger take precedence for understanding this snapshot.

Rebuilding the documentation

From the project root, run node docs/handbook/extract.mjs to refresh the source index, review/update docs/handbook/guide.md and validation.json, then run the bundled Python interpreter with docs/handbook/build.py. The PDF skill's operation marker is a session authoring step and is not part of application code. Re-render and inspect the PDF after content changes.

Files introduced for this document: guide.md (authored explanations), extract.mjs (read-only source index extraction), code-index.json (captured source, symbol/data/effect maps), build.py (document and companion generator), validation.json (honest executed-check ledger), and optional verify.py (artifact coverage/layout checks). These live under docs/handbook; final deliverables live under output/pdf.